The Parliament of Machines

The Desk Volume

An Audio Course in Distributed Systems — formal statements, proofs, protocols, traces, and problems

Narrated by Jeeves · Written for and with P. Milovanov, Esq.

Edition v1.12.0

How to Use This Volume

Each chapter of this volume, save the last, is the desk companion to one episode — and each is written to stand on its own. A chapter walks its episode's ground in the same order, one narrated pass — the same story, told for the page, with less chat and more proof — and the formal apparatus is set where the story reaches it: the model box before the first claim, the definitions and theorems at their story positions, the proofs immediately after their statements, the protocol boxes and trace exhibits at the dramas they record. The course's spoken names — the Post, the doppelgänger, the crowded club, the aunts — are standing vocabulary here, introduced beside their formal twins. The Whiteboard and the debt register close each narrated body, and then come the problems in three tiers. The closing chapter is apparatus — the registers map, the index of protocol boxes, and the pronunciation register — assembled once the season was complete. Tier I is calisthenics; Tier II is the real thing, including each chapter's *sabotage* problem, in which a plausibly broken protocol is supplied and your task is to exhibit the fatal execution; Tier III is starred and optional. Hints are segregated and spoiler-safe. Solutions are complete for Tier I, complete for the Tier II problems that finish arguments the chapters sketch, and withheld (hints only) for Tier III. Nothing here appears unmotivated by the audio, and nothing in the audio pretends to a proof that is not here.

Contents

Chapter 1

The Trouble with Other Computers

Companion to Episode One. The episode tells the story — the umbrella shop, the four silences, the two generals on their hills; this chapter keeps the record: the three forfeits and the adversary's treaty, the Post's charter formalized as fair-loss links with the layered construction upward, the season's models fixed on three axes, the naive counter's three disasters drawn as trace exhibits, and the Two Generals theorem written out as a clean induction. The proof technique it christens — the doppelgänger argument — will fell every major result to come. Statements, proofs, protocol boxes, problems.

1.1 Parts One and Two — The Scattering, and the End of Honest Dying

Why distribute — three honest reasons and a candid fourth:

- **scale:** the workload exceeds any one machine;
- **geography:** light is slow — Toronto–Sydney, a sixth of a second round trip before any machine works;
- **fault tolerance:** one machine is one fate — one power supply, one kernel, one fire;
- **the organization already did:** the architecture ships the org chart.

The three forfeits. What is given up is the paradise the rest of the field buys back at retail prices — three mercies so ordinary they are invisible, forfeited all at once, categorically: there is no such thing as slightly distributed.

- **shared memory:** one authoritative state, a fact of the matter about the value of x;
- **shared fate:** failure total and honest, the whole stage aware of the death;
- **one clock:** a single timeline on which every event has its place.

Partial failure and the four silences. The defining pathology, from the second forfeit: a machine stops mid-operation — holding locks, halfway through an update — and *nobody is notified*. Worse, the observer cannot diagnose: a request has gone unanswered, and four worlds fit the evidence — *the four silences*:

- the server is dead;
- the server is slow (in the bath, to be embarrassed later);

- the request was lost;
- the reply was lost *after the work was done*.

All four present the identical face; the retry is the cure in the third world and a catastrophe in the fourth (Problem 1.5). A timeout is therefore a *verdict*, not a parameter — death declared without a body, sometimes wrongly, and wrongly at a price; surviving the wrongly condemned (leases, fencing, epochs) is Chapter 5's machinery.

The adversary. Everything harmful in the course is laid at one door: the *adversary* — Murphy's law furnished with a will and a timetable, arranging the worst permitted event at the worst moment, having read the protocol first. Two disciplines follow: correctness means correctness against his *every* move, not his likely one; and he is *leashed* — permitted only what the model allows, the model being ours to write and his to obey. His coats: the Post; Chapter 3's scheduler, freezing a healthy process for any finite age; Chapter 5's Byzantine colleague.

Crashed and slow, then, are not hard to distinguish but *indistinguishable*; the season's sharpest instrument receives its formal home at once.

Definition 1.1 (processes, messages, executions)**.** A *process* is a deterministic state machine with a local state, which takes steps: sending messages, delivering (receiving) messages, and local computation. Processes communicate only by messages over *links*. An *execution* is a sequence of steps of all processes together with the adversary's delivery decisions; a process's *view* in an execution is the sequence of its own inputs and delivered messages. Two executions are *indistinguishable to process p* if *p*'s view is identical in both; a deterministic *p* behaves identically in indistinguishable executions. This one-sentence observation is the doppelgänger argument, and it is the sharpest tool in the drawer.

1.2 Part Three — The Post

The adversary's first and favourite disguise: the network, engaged for the season as *the Post* — not a wire but a postal service, queues and couriers subject to weather nobody can see. Letters are messages *m*; the charter is the informal reading of the fair-loss link (Definition 1.4 below). Five clauses — the Post may:

- **delay** any letter arbitrarily (clause one);
- **lose** any letter silently, no return-to-sender (clause two);
- **duplicate** any letter (clause three) — and the clause is self-inflicted besides: a retrying sender atop a merely lossy network manufactures duplication and reordering endogenously, so the charter describes the system you build, not only the wire you bought (Problem 1.6);
- **reorder** letters (clause four);
- but never **forge** (clause five, formally the no-creation property FL3/PL3): every letter that arrives was genuinely written by its named sender — the single mercy, whose revocation is Chapter 5's business.

Nor is TCP a counterexample: a treaty negotiated clause by clause — sequence numbers against reordering, retransmission against loss, deduplication against the retransmission's own duplicates — holding within one connection, while it lives; every connection break re-asks the field's oldest question: *did my last letter arrive?*

The fallacies of distributed computing. (Deutsch and colleagues, Sun Microsystems, mid-nineties; the last added by Gosling.) Eight false beliefs of essentially every newcomer, each expensive:

1. *The network is reliable* — the founding fallacy; this chapter is its refutation.
2. *Latency is zero* — the floor is the speed of light; every synchronous network call is a latency bet.
3. *Bandwidth is infinite* — finite, shared, scarcest during the incident.
4. *The network is secure* — other courses' matter; Chapter 5 borders it.
5. *Topology does not change* — routes reconverge, preferably mid-request.
6. *There is one administrator* — dozens, across companies.
7. *Transport cost is zero* — serialization is compute, bytes are money.
8. *The network is homogeneous* — every vintage ever deployed, simultaneously, forever.

The documentary record. Bailis and Kingsbury's "The Network is Reliable" (2014), the evidence behind fallacy the first: published, attributable partitions at every scale — redundancy correlated in the wiring closet, maintenance that partitioned what it maintained, sharks with a taste for undersea fibre. The finding that matters: *partitions happen* — the network splits into provinces, each internally healthy, unable to distinguish "the other province is dead" from "the road between us is out." Not pessimism; actuarial science. Both texts are assigned in the reading.

1.3 Part Four — The Rules of Engagement

The course's single most important methodological habit: *every claim is conditional on a model, and a claim quoted without its model is not a fact but a noise* — change the assumptions and the theorem falls silent. A model is a point on three axes — timing, process failure, links — fixed now, formally, for the season.

Definition 1.2 (timing models)**.** A *timing model* fixes what a protocol may assume about the passage of time — the relative speed of processes and the delay of messages. Three are standard, in decreasing order of generosity to the protocol:

- **Synchronous:** known finite bounds hold *always*: every correct process takes a step within a known time, and every sent message is delivered within a known delay. Equivalently, computation proceeds in numbered rounds (the round model of the model box below). Here a timeout is a verdict that never errs.
- **Asynchronous:** *no* bound on process speed or message delay is assumed; the adversary controls both, constrained only by the link's own liveness (fair loss delivers eventually, but "eventually" may be arbitrarily late). A slow process and a dead one are then indistinguishable (Def. 1.1) — the model in which the sharpest impossibilities bite.
- **Partially synchronous:** (Dwork, Lynch, and Stockmeyer, 1988) the bounds exist but are unknown, or hold only after some unknown *global stabilisation time* (GST): before GST the system behaves asynchronously, after it, synchronously. This is the honest model of a real network — usually timely, occasionally not — and the one in which consensus becomes purchasable (Chapters 4–5).

A result is only as strong as the timing model it is proved in: an impossibility in the synchronous model is the most damning, for it survives every relaxation; a possibility in the synchronous model is the most suspect, for it may not survive asynchrony. *Course default: asynchronous.*

Definition 1.3 (process-failure models)**.** A *failure model* fixes how a faulty process may deviate from its protocol. A process that never deviates in a given execution is *correct*; the rest are *faulty*. Four are standard, in increasing order of malice:

- **Crash-stop:** a process follows its protocol exactly until, at some moment, it halts permanently and takes no further step. Nothing false is ever emitted; the only sin is silence.
- **Crash-recovery:** a crashed process may restart, losing its volatile state but retaining whatever it committed to stable storage (disk) beforehand. Amnesia rather than death — the survivor returns with holes in its memory (Paxos's persistence clause, Chapter 4; Problem 1.8).
- **Omission:** a process otherwise follows its protocol but may drop messages it should have sent or received — a fault of its own ports rather than of the link, and the least-used of the four, blurring as it does into link loss.
- **Byzantine:** (Lamport, Shostak, and Pease, 1982) a faulty process may deviate *arbitrarily*: send anything, to anyone, at any time, including well-crafted lies and collusion. The catch-all for bugs, corruption, compromise, and malice — and the model whose fault bounds are dearest to meet (Chapter 5).

Each model subsumes the milder ones: a protocol tolerating Byzantine faults tolerates crashes, but not the reverse. *Course default: crash-stop.*

The third axis is the Post's charter, formalized — and then *built upon*: reliability is never discovered, only manufactured, and always out of unreliable parts.

Definition 1.4 (fair-loss links)**.** A *fair-loss link* from p to q offers primitives $\mathsf{send}(m)$ and $\mathsf{deliver}(m)$ with:

FL1 *(fair loss)* if p sends m to q infinitely often and q is correct, then q delivers m infinitely often;

FL2 *(finite duplication)* if p sends m to q finitely often, then q delivers m at most finitely often;

FL3 *(no creation)* if q delivers m with sender p, then m was previously sent to q by p.

Fair loss is the formal name for *negligent but not criminal*: any single letter may vanish, but persistent resending defeats the Post eventually, and the Post invents nothing.

Definition 1.5 (stubborn links)**.** A *stubborn link* offers the same primitives with: **SL1** *(stubborn delivery)* if a correct p sends m once to a correct q, then q delivers m infinitely often; **SL2** *(no creation)* as FL3.

Definition 1.6 (perfect links)**.** A *perfect (reliable) link* offers the same primitives with: **PL1** *(reliable delivery)* if a correct p sends m to a correct q, then q eventually delivers m; **PL2** *(no duplication)* no message is delivered more than once; **PL3** *(no creation)* as FL3.

The ladder is climbed — stubbornness purchased by unbounded retransmission, perfection by deduplication atop it — in the coda after this chapter's main event (Lemma 1.2 and Protocol Box 2), and the top rung is yours: Problem 1.7.

The chapter's models, fixed before any formal claim — a habit this volume enforces without exception:

Model of this chapter

For the impossibility (Theorem 1.1). Two processes A (Alexandra) and B (Basil). *Deliberately generous timing*: the synchronous round model — computation proceeds in numbered rounds; in each round every process sends messages as a deterministic function of its state, then receives whatever the adversary delivers from that round, then updates state. Processes do not fail. The adversary may suppress (lose) any subset of messages. A holds an input $v \in \{\mathsf{attack}, \mathsf{abstain}\}$. The point of

the generosity: the theorem is about *loss alone*; if coordination fails here, it fails a fortiori in the asynchronous model, where the adversary also controls delay.

Course default hereafter (unless a chapter declares otherwise): asynchronous timing, crash-stop processes, fair-loss links (Def. 1.4).

1.4 Part Five — The Counter That Could Not Count

The season's protocols are performed by a standing repertory company: replicas *Bingo*, *Tuppy*, and *Gussie* — processes p_1, p_2, p_3 — Barmy and Oofy on the bench, and *Bertie*, an external client c, presently in the market for an umbrella. The casting notes (Gussie slow, Tuppy literal, Bingo usually in the chair) are permanent equipment; the whimsy is load-bearing. The written counterpart is the trace diagram, its convention fixed now for the season; every counterexample drama gets its picture in this notation.

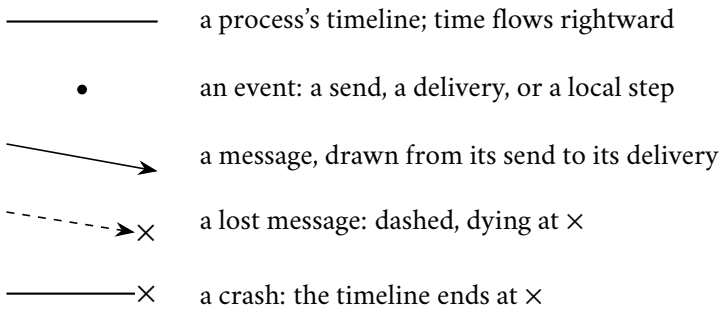


Exhibit 1.1 (conventions). A duplicated delivery is drawn as two message arrows leaving a single send event.

The scene: the three gentlemen keep the inventory of an ambitious online purveyor of umbrellas — a counter of orders placed today, one copy each, because Part One decided the shop must survive the loss of a machine. The protocol, by a designer who has not yet taken this course:

Protocol — Box 1. The naive replicated counter (the inaugural bug)

At each replica p_i, for a counter replicated on replicas $p_1, \ldots, p_n$:

```
1: state: c ← 0

2: upon client request ⟨INC⟩:
3:     c ← c + 1
4:     for each replica q ≠ p_i: send(⟨INC⟩, q)

5: upon deliver(⟨INC⟩):
6:     c ← c + 1
```

No identifiers, no acknowledgments, no retransmission, no ordering. Every one of those absences is a separate disaster: Exhibits 1.2 and 1.3 stage two of them, Problem 1.4 drills a third, and the third performance (the racing delivery notes) showed that for non-commuting state the protocol shape itself is unsalvageable without an order — the subject of Chapters 2 and 4.

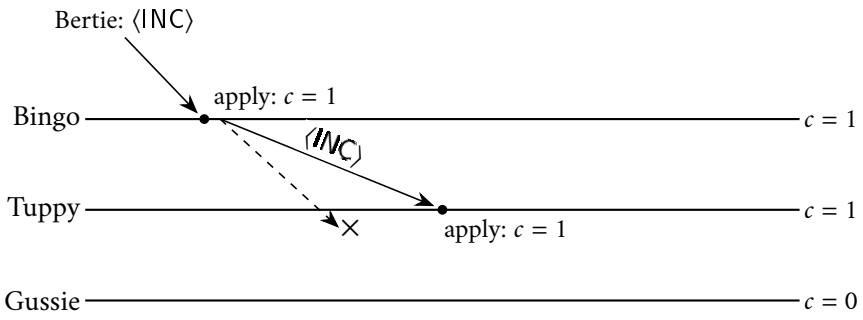


Exhibit 1.2 (the lost letter; performance the first). Bertie's order lands at Bingo; the propagation to Tuppy arrives; the propagation to Gussie dies in the Post. Divergence, and *silent* divergence: every machine is locally content, and the ensemble is wrong.

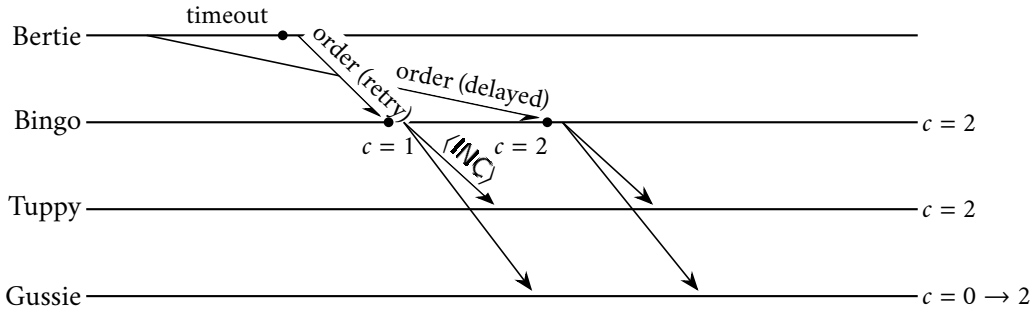


Exhibit 1.3 (the double charge; performance the second). The original order was not lost but *delayed*; the prudent retry and the ambling original both arrive, and the system converges — unanimously — on a world in which Bertie bought two umbrellas. Nobody misbehaved. Deduplication by identifier is the cure, and Problem 1.8 is the cautionary tale about administering it frugally.

Performance the third: the racing delivery notes. No exhibit, only a change of state; the skeleton:

- **the writes:** Bertie posts two delivery notes — "leave with the porter," then "ring the top bell";
- **clause four exercised:** they reach Gussie in the other order; his copy settles on the porter, his colleagues' on the bell — every letter delivered exactly once, and the copies disagree;
- **the diagnosis:** for *register* state, order is the whole game — and, per Chapter 2, who wrote last does not currently even mean anything;
- **the contrast:** the counter forgave the shuffling — addition commutes. File under *commutativity forgives disorder* (the order-insensitivity of commuting operations); it returns in Chapters 7 and 12 wearing formal dress.

The audit. Divergence by loss; double charge by duplication-via-retry; divergence by reordering — and *not one machine failed*: all of it weather, exercised on a protocol too innocent to have read the charter. The deepest question from the wreckage: Bingo says one, Gussie says nought — which is *right?* No master copy exists; the copies are all there is. An authority must be appointed — surviving the appointee's death, the Post's meddling with the election, and two gentlemen each believing himself appointed: *consensus*, Chapters 3–5 — first the impossibility in the default model, then the lawyerly, partially-synchronous ways it is done every second of every day.

1.5 Part Six — Two Generals on Two Hills

The season's cornerstone (Akkoyunlu–Ekanadham–Huber 1975; christened and connected to transaction commit by Gray 1978). Two hills, the enemy in the valley — too strong for either division alone, too weak for both; General Alexandra holds the plan — *attack at dawn* — and the only instrument of coordination is a messenger who may be captured, silently, on any crossing. The intuition: each acknowledgment *relocates* the doubt rather than discharging it — a hot coal passed across the valley, held at dawn by whoever sent the last rider. The problem, formally:

Definition 1.7 (the Coordinated Attack problem)**.** Two processes; A holds an input $v \in \{\mathsf{attack}, \mathsf{abstain}\}$; each process must output exactly one decision in $\{\mathsf{attack}, \mathsf{abstain}\}$. A protocol *solves Coordinated Attack* if in *every* execution permitted by the model:

- *(Termination)* both processes decide;
- *(Agreement)* both processes decide the same value;
- *(Validity)* if $v = \mathsf{abstain}$, both decide abstain; and if $v = \mathsf{attack}$ and no message is lost, both decide attack.

Validity is the usefulness clause: without it, the protocol "always abstain" would satisfy the rest by universal cowardice.

Theorem 1.1 (Two Generals; coordinated attack is unachievable)**.** *In the model of this chapter's model box — synchronous rounds, reliable processes, links that may lose any subset of messages — no deterministic protocol solves Coordinated Attack.*

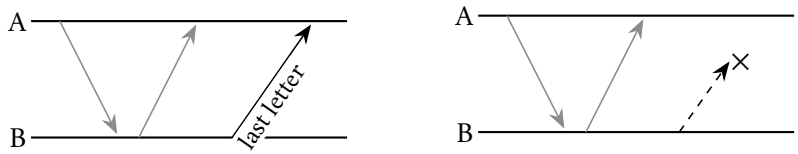
Proof. Suppose, for contradiction, that a deterministic protocol P solves Coordinated Attack in the stated model. Since processes are deterministic, an execution of P is completely determined by the input v and by the adversary's choice of which messages to suppress. Let α be the execution with $v = \mathsf{attack}$ in which the adversary loses nothing. By Termination and Validity, in α both processes decide attack, and they do so after finitely many rounds; let $m_1, m_2, \ldots, m_k$ be the messages delivered in α, listed in order of the round of their delivery, ties within a round broken by any fixed rule, so that m_k is a latest delivery.

For $0 \le i \le k$, let α_i denote the execution with $v = \mathsf{attack}$ in which the adversary delivers exactly $m_1, \ldots, m_{k-i}$ and suppresses everything else — including any message a process might newly send in consequence of the suppression. Thus $\alpha_0 = \alpha$. *Claim: in every α_i, both processes decide attack.* Induction on i; the base case is α. For the step, let $m := m_{k-i}$ be the last message delivered in α_i, sent by s to r in round t; α_{i+1} additionally suppresses m. Compare the two executions from s's chair. In the round model, what s sends in each round is a function of s's state at that round's start, which is a function of the deliveries s has received in earlier rounds. Deliveries *to* s are identical in α_i and α_{i+1}: in rounds up to t they are the same messages among $m_1, \ldots, m_{k-i-1}$, and after round t neither execution delivers anything at all. So s's view is identical in both executions — the two are doppelgängers to s — and deterministic s therefore decides in α_{i+1} exactly what it decided in α_i: by the induction hypothesis, attack. And since P is assumed correct *in every execution*, Agreement holds in α_{i+1} in particular, so r decides attack as well. The claim follows, and taking $i = k$: in α_k, the execution with input attack in which *no message whatever is delivered*, both processes decide attack.

Now let β be the execution with $v = \mathsf{abstain}$ in which the adversary likewise suppresses every message. Process B holds no input, and in both α_k and β it delivers nothing; its view is identical in the two — one final doppelgänger — so B decides in β what it decides in α_k, namely attack. But Validity requires that in β both processes decide abstain. This is the contradiction, and no such protocol P exists. □

Remark 1.8 (the same knife, two grips)*.* The induction has a companion telling, as a minimal counterexample — the *thriftiest counsel of war*: take a correct protocol of minimal best-case message count, examine the last

letter of its best-case execution, and observe it is either decoration (strike it — contradicting thrift) or pivotal (capture it — the sender, unable to distinguish the worlds, attacks alone). That telling and the induction above are the same argument holding the knife by different ends: each peeling step of the induction is precisely the "decoration or pivotal" dilemma, resolved. Exhibit 1.4 draws the two worlds of a single peeling step. Note also the bookkeeping of the proof: Termination bought finiteness, Agreement propagated the decision along the chain, and Validity anchored both ends. Each hypothesis is spent exactly where it is needed — a habit of audit worth acquiring, since it tells you which weakening of the problem might escape the theorem, and Problems 1.9 and 1.12 explore exactly those escapes.



World I: delivered **World II**: captured

Exhibit 1.4 (the doppelgänger pair). One peeling step of Theorem 1.1: the grey arrows are the protocol's earlier traffic, identical in both worlds; the sender of the last letter, *B*, has an identical timeline in World I and World II and therefore behaves identically in both. If *A*'s decision differs with the letter's fate, one general attacks alone; if it does not, the letter was decoration. Either way, the protocol dies.

The proof analyzed no strategies — there are infinitely many. It did one thing, twice: found two executions some participant cannot tell apart, in which correctness demands different behaviour. That move is the field's master key, *the doppelgänger argument* (Def. 1.1) — met already, unnamed, in crashed-versus-slow; it returns for FLP and CAP (Chapter 3), the Byzantine bound (Chapter 5), and the blocking of two-phase commit (Chapter 8).

Remark 1.9 (what is and is not forbidden)*.* The theorem forbids *certainty*, not usefulness. It does not forbid protocols that agree with overwhelming probability (Problem 1.9); it does not apply in gentler models, e.g. with a bound on total losses (Problem 1.12); and it says nothing against protocols that replace "both act at the deadline" with "both eventually act" — a change of specification, not a loophole. What it forbids exactly: guaranteed common action, over unreliable delivery, by a deadline. Two-phase commit will inherit this in Chapter 8, paying in the currency of blocking.

Remark 1.10 (common knowledge)*.* The deep reading of Theorem 1.1 is epistemic: attacking safely requires not just that both generals know the plan, but that each knows the other knows, and so on without end — *common knowledge* in the sense of Halpern and Moses (1990), who proved that no finite exchange over unreliable channels can create it. Problem 1.11 walks the ambitious reader through the connection; the full treatment is beyond this chapter, and I say so plainly rather than gesture otherwise.

1.6 Coda — The Ladder of Links, Built

The theorem forbids guaranteed *coordination*; it says nothing against guaranteed *delivery*: under the very charter that kills the generals' certainty, reliable point-to-point delivery is manufactured out of unreliable parts. The first rung is brute persistence as an algorithm.

Lemma 1.2 (stubbornness is purchasable)**.** *Stubborn links (Def. 1.5) can be implemented over fair-loss links (Def. 1.4) by unbounded retransmission (Protocol Box 2).*

Proof. Consider Protocol Box 2: on sb-send(m, q) the sender adds (m, q) to a set *sent* and thereafter reissues fl-send(m, q) for every element of *sent* forever, at any positive frequency. **SL1**: if the sender is correct, (m, q) remains in *sent* forever, so m is fl-sent to q infinitely often; if q is correct, FL1 yields that q fl-delivers m infinitely often, and each fl-delivery triggers an sb-delivery. **SL2**: an sb-delivery occurs only on an fl-delivery, which by FL3 was preceded by an fl-send of m by its named sender, which occurs only for messages actually sb-sent. □

Protocol — Box 2. Stubborn point-to-point link, over fair-loss

```
1: state: sent ← ∅

2: upon sb-send(m, q):
3:     sent ← sent ∪ {(m, q)};   fl-send(m, q)

4: every Δ time units:
5:     for each (m, q) ∈ sent: fl-send(m, q)

6: upon fl-deliver(m, p):
7:     sb-deliver(m, p)
```

Correct by Lemma 1.2; checked against the treatment in Cachin–Guerraoui–Rodrigues. Brute persistence as an algorithm. The *sent* set never shrinks and the repetition never stops; both costs are the price of the guarantee, and both are what Problem 1.7 teaches you to negotiate down.

The step from stubborn to *perfect* — suppressing the infinite repetition without losing the delivery — is deliberately left to you: Problem 1.7 completes the layered construction, full solution given.

1.7 Part Seven — Impossible in Principle, Routine in Practice

The season's axioms, binding hereafter:

- **Fact the first:** machines fail — independently, partially, without notice.
- **Fact the second:** messages are delayed, lost, duplicated, and reordered — and the sender cannot tell which.
- **Fact the third:** there is no shared clock — nor, per Chapter 2, a shared *now* for one to report.

Every impossibility in the volume is a consequence of the three; every system sir operates is a negotiated settlement with them.

The reconciliation triage. The theory proves things impossible, and the industry does those very things before lunch, daily — first instance: coordinated action over unreliable messages, impossible by Theorem 1.1, performed by every online purchase. The reconciliation always lives in one of three places:

- **the model differs:** the impossibility was proved for a harsher world than the system inhabits;
- **the guarantee differs:** the theorem forbids certainty; the system delivers overwhelming probability, and marketing rounded up;
- **the price was paid where you were not looking:** availability during partitions, latency on every write, a lease, a human on call — the invoice filed where visitors do not go.

Model, guarantee, price: finding which is *the* senior-engineer skill; the season sharpens it into a formal triage.

1.8 The Whiteboard

For the design review.

1. Fix the model before the architecture: say aloud what the network may do, which failures you insure against, what timing you rent.
2. The catechism, for every arrow in the diagram: what happens when this message is *lost*? *duplicated*? *delayed* — and arrives anyway, after you acted on its absence? "That cannot happen" is a fallacy with a diagram.
3. Silence is not evidence. A timeout is a purchased guess; ask what happens when the condemned walks back in.
4. Never say "reliable" without saying *against what*.

The register. Debts issued: (1) the Post will learn to forge — payable Chapter 5, with a famous theorem attached. (2) Crashed versus slow is never resolved, only managed — instalments in Chapters 3, 5, and 11. (3) There is no shared clock — payable immediately: Chapter 2.

1.9 Problems

Tier I — Calisthenics

Problem 1.1 (fallacy audit)**.** For each vignette, name every fallacy of distributed computing it embodies, and state the first incident you would expect. *(a)* A mobile note-taking application saves each edit locally and syncs it "instantly" to the server; concurrent edits are resolved by "last write wins, by timestamp." *(b)* Two services in one datacentre call each other with thirty-second timeouts and unlimited retries on timeout; requests carry no identifiers; the runbook states that network issues are transient and retry storms are impossible. *(c)* A two-datacentre deployment replicates over a "reliable dedicated fibre link"; a cron job promotes the secondary when three consecutive pings to the primary fail.

Problem 1.2 (model classification)**.** Classify each system by timing model (Def. 1.2), process-failure model (Def. 1.3), and link assumptions, as in the model box. *(a)* An avionics control bus: message delivery guaranteed within a known bound; components fail only by halting. *(b)* A planet-scale web service: virtual machines pause and migrate without notice; processes restart with their disks intact; traffic crosses the public internet. *(c)* A single-rack cluster with an engineered lossless fabric whose interface cards nonetheless drop frames silently under overload. *(d)* A settlement network operated jointly by mutually distrustful banks over leased lines with occasional long outages.

Problem 1.3 (link-property check)**.** For each channel, state which of FL1–FL3 and PL1–PL3 hold, with a one-line justification or counterexample. *(a)* Delivers every message exactly once, but arbitrarily late. *(b)* Loses each transmission independently with probability one half; never duplicates; never corrupts. *(c)* Delivers the first transmission of each distinct message and silently discards all retransmissions of it.

Problem 1.4 (trace reading)**.** Replicas Bingo, Tuppy, Gussie run Protocol Box 1; all counters start at nought. Events, in real-time order: (1) a client increment lands at Bingo; (2) Bingo's propagation to Tuppy is delivered; (3) Bingo's propagation to Gussie is delivered *twice* (the Post duplicated it); (4) a client increment lands

at Gussie; (5) Gussie's propagation to Bingo is delivered; (6) Gussie's propagation to Tuppy is lost. Give the final counter at each replica, draw the trace in the Exhibit 1.1 convention, and mark the earliest moment at which the ensemble is already doomed to disagree forever.

Problem 1.5 (the four silences)**.** A client's request to a server has produced no reply within the timeout. *(a)* Enumerate the four worlds consistent with this evidence. *(b)* For each world, state what later observation, if any, would reveal it after the fact. *(c)* Exhibit two of the worlds as doppelgängers: describe the client's view and argue no protocol lets the client distinguish them *at the moment the timeout fires*. *(d)* In which worlds is an immediate retry harmless, and in which is it a double execution? What single property of the *operation* makes the distinction irrelevant?

Problem 1.6 (the charter is self-inflicted)**.** Assume a Post that only *loses and delays* — it never duplicates and never reorders distinct sends. Senders, however, retransmit on timeout. Construct executions showing that the application layer above nonetheless observes *(a)* duplicate deliveries of one logical message and *(b)* deliveries out of the order in which the logical messages were first sent. Conclude, in one sentence, why clauses three and four of the charter must be assumed even on networks that formally lack them.

Tier II — The Real Thing

Problem 1.7 (perfect links, built by hand)**.** Construct perfect links (Def. 1.6) over fair-loss links (Def. 1.4). Give the protocol for sender and receiver, using sequence numbers, a delivered-set, and acknowledgments; then *(a)* prove PL1, PL2, PL3; *(b)* identify precisely what the acknowledgments buy — correctness, or economy — by describing the protocol's behaviour with them deleted; *(c)* identify every piece of state that grows without bound, and say what assumption would be needed to bound it. *(Full solution provided.)*

Problem 1.8 (sabotage: the frugal clerk)**.** A designer, having read Problem 1.7, adds deduplication to the replicated counter. Client operations carry per-client sequence numbers, and the client retransmits an operation until acknowledged. Each replica keeps: the counter c, written synchronously to disk on every apply; and a table $high[s]$ — the highest sequence number applied per client s — kept in memory, "because it is rebuilt by traffic anyway." On $\langle \mathsf{INC}, s, n \rangle$: if $n > high[s]$, apply $c \leftarrow c + 1$ to disk and set $high[s] \leftarrow n$; in all cases acknowledge. Replicas may crash and recover (crash-recovery model); recovery reloads c from disk and starts $high$ empty. *(a)* Exhibit an execution in which one logical increment is applied twice. *(b)* State the invariant the design intends between $high$ and the disk state, and where it breaks. *(c)* Give the minimal repair and argue its correctness, including why the two persistent writes must be atomic with respect to each other. *(Certified fatal per the standing rules of this course: the fatal execution is written out in the solutions.)*

Problem 1.9 (riders and risk)**.** Alexandra intends the attack and may send messengers, each captured independently with probability p; captures are silent. *(a)* Protocol: she dispatches n riders with the order and attacks at dawn unconditionally; Basil attacks if and only if at least one rider arrives. Show the probability of a solitary attack is exactly p^n, and compute the least n for which it falls below one in a million when $p = \frac{1}{2}$. *(b)* Modify: Alexandra attacks only if she receives at least one acknowledging rider from Basil. Enumerate the disagreement modes now possible and show the anxiety has moved, not died. *(c)* Prove that *every* protocol in this setting either never attacks or has disagreement probability strictly greater than zero. *(Hint provided.)*

Problem 1.10 (idempotency and its garbage)**.** Client operations carry globally unique identifiers; each replica keeps the set D of identifiers already applied and ignores repeats. *(a)* Prove this gives the counter effectively-once application per identifier, under fair-loss links and arbitrary client retries, replicas not crashing. *(b)* D grows forever. Propose a garbage-collection rule, then show that for *any* rule that ever

discards an identifier the Post might still redeliver, the double-apply returns — and conclude that bounded deduplication memory is possible only by purchasing a bound on message lifetime, i.e. by strengthening the model. *(This is the deduplication-window lemma in embryo; it returns, industrialized, in Chapter 10.) (Hint provided.)*

Tier III — For the Ambitious (starred)

Problem 1.11 (★ common knowledge)**.** Following Halpern and Moses: for a fact φ, write $E\varphi$ for "both generals know φ," and define common knowledge $C\varphi$ as the infinite conjunction $\varphi \wedge E\varphi \wedge EE\varphi \wedge \cdots$. *(a)* Argue that any protocol solving Coordinated Attack yields, at the moment of attack, C("the attack is on"). *(b)* Show each additional level of the conjunction requires at least one further successfully delivered message, and conclude no finite execution over lossy links attains $C\varphi$ for any φ not common knowledge at the outset. *(c)* Reconcile with practice: what weakening of C do real systems settle for? *(Hints only.)*

Problem 1.12 (★ the budgeted Post)**.** Same round model as Theorem 1.1, but the adversary's total budget is at most f suppressed messages across the whole execution, and f is known. *(a)* Give a protocol solving Coordinated Attack with $f+1$ messages, and prove it. *(b)* Prove no protocol using at most f messages in its worst case can succeed. *(c)* Now let the budget be f *per round*, rounds unbounded. Show the impossibility of Theorem 1.1 returns. Moral: it is not the quantity of loss that kills, but its inexhaustibility. *(Hints only.)*

1.10 Hints

Hint (for Problem 1.9)**.** (a) Disagreement is exactly "all n captured." (b) Trace, for each pattern of captures, who stands alone at dawn. (c) Every execution in the peeling chain of Theorem 1.1's proof, and the final β, occurs with probability at least $p^{(\text{messages sent})} > 0$ once you fix the protocol; a deterministic protocol correct with probability one would have to behave correctly in each of them, and the proof of Theorem 1.1 shows it cannot.

Hint (for Problem 1.10)**.** (a) Set membership is exactly PL2's argument. (b) Fix the rule; the adversary reads it. Find the identifier it discards earliest, and have the Post redeliver just after. The contrapositive is the lemma: no double-apply forever $\Rightarrow$ some identifier is remembered as long as the Post can redeliver it $\Rightarrow$ memory is bounded only if redelivery lifetime is.

Hint (for Problem 1.11)**.** (a) If at the moment of attack some level E^k failed, exhibit the doppelgänger in which a general attacks alone. (b) Induction on k, with Theorem 1.1's peeling as the engine: the last delivered message is the only possible carrier of the topmost E. (c) Consider "eventually both know," and timeouts as purchased belief.

Hint (for Problem 1.12)**.** (a) Pigeonhole: $f+1$ riders, at most f captures. (b) The adversary can afford to suppress everything; then replay the α_k-versus-β doppelgänger. (c) In the peeling argument, check the adversary's budget: each α_i asks it to suppress only messages it can afford round by round.

1.11 Solutions

Tier I in full (tersely); Tier II in full where the problem completes an argument begun above (1.7, 1.8); hints only for Tier III, by standing policy.

Solution (1.1). *(a)* Fallacies 1 and 2 (reliable, zero-latency); the timestamp clause also leans on a shared clock that does not exist — Chapter 2's subject; first incident: silently lost edits under concurrent writes. *(b)* Fallacies 1 and 3; no identifiers means retries are duplicates (Exhibit 1.3); "storms impossible" ignores that timeouts correlate under load — the metastability of Chapter 11; first incident: a double-applied operation during a latency spike. *(c)* Fallacies 1 and 5, plus the timeout-as-verdict error: three lost pings do not distinguish a dead primary from a partitioned one; first incident: two primaries — the split brain of Chapter 5.

Solution (1.2). *(a)* Synchronous; crash-stop; reliable bounded-delay links. *(b)* Asynchronous; crash-recovery (disks survive); fair-loss. *(c)* Effectively synchronous timing with *omission* link faults — "lossless" fabrics lose at the edges. *(d)* Partially synchronous; Byzantine (mutual distrust); fair-loss with long partitions.

Solution (1.3). *(a)* PL1–PL3 all hold (perfection does not promise punctuality); hence FL1–FL3 hold as well. *(b)* FL1 holds (infinitely many independent trials with success probability one half deliver infinitely often, with probability one — note the "almost surely," a nicety the definitions absorb); FL2, FL3 hold; PL1 fails (any single send may be lost). *(c)* FL3, FL2 hold; *FL1 fails*: send m infinitely often and it is delivered exactly once. Moral: this channel is not a fair-loss link at all — it is a deduplicating layer masquerading as one, and layers must be named for what they guarantee, not where they sit.

Solution (1.4). Bingo: 1 (own) + 1 (Gussie's) = 2. Tuppy: 1 (Bingo's) + 0 (Gussie's lost) = 1. Gussie: 2 (Bingo's, duplicated) + 1 (own) = 3. The ensemble is doomed no later than event (3): with the duplicate applied and no deduplication or retransmission anywhere in Protocol Box 1, no future execution of the protocol reconciles the copies.

Solution (1.5). *(a)* Server dead; server slow; request lost; reply lost after execution. *(b)* Dead: an operator finds the body. Slow: the reply arrives late. Request lost: server-side audit shows no arrival. Reply lost: the side effect exists without a reply — discoverable only by inspecting effects. *(c)* Worlds two and four at timeout: the client's view is "sent, then silence" in both; any client behaviour is a function of that view, hence identical — doppelgängers by Definition 1.1. *(d)* Harmless in world three; double-executing in world four; *idempotence* of the operation dissolves the distinction, which is why Problem 1.10 and Chapter 10 make it a design axiom rather than a hope.

Solution (1.6). *(a)* Send m; the Post delays it past the timeout; retransmit; both copies arrive: two deliveries of one logical message. *(b)* Send m_1; it is delayed. Timeout; while the retransmission of m_1 is pending, send m_2, which arrives promptly; then m_1 (either copy) arrives: delivery order m_2, m_1 inverts send order. Conclusion: any layer that retries above a merely lossy network *manufactures* duplication and reordering; the charter describes the system you build, not only the wire you bought.

Solution (1.7). Layer it: stubbornness first, then deduplication, then economy.

Protocol — Box 3. Perfect link over fair-loss (solution to Problem 1.7)

Sender p, per destination q:

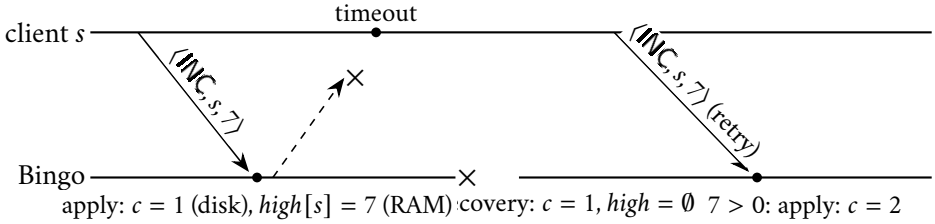
1: **state:** $n \leftarrow 0$; $pend \leftarrow \emptyset$
2: **upon** pl-send(m, q): $n \leftarrow n{+}1$; $pend \leftarrow pend \cup \{(m, n)\}$; fl-send$(\langle \text{DATA}, m, n\rangle, q)$
3: **every** Δ: **for each** $(m, k) \in pend$: fl-send$(\langle \text{DATA}, m, k\rangle, q)$
4: **upon** fl-deliver$(\langle \text{ACK}, k\rangle, q)$: remove the pair with index k from $pend$

Receiver q, per source p:

1: **state:** $seen[p] \leftarrow \emptyset$
2: **upon** fl-deliver$(\langle \text{DATA}, m, k\rangle, p)$:
3: fl-send$(\langle \text{ACK}, k\rangle, p)$
4: **if** $k \notin seen[p]$: $seen[p] \leftarrow seen[p] \cup \{k\}$; pl-deliver$(m, p)$

(a) **PL1.** Suppose correct p pl-sends m to correct q, assigned index k. Either an $\langle \text{ACK}, k\rangle$ is eventually fl-delivered at p, which by FL3 was sent by q, which q does only upon fl-delivering $\langle \text{DATA}, m, k\rangle$ — so q delivered the data; on first such delivery, $k \notin seen$ and q pl-delivers m. Or no ack is ever fl-delivered: then (m, k) stays in $pend$ forever, the data is fl-sent infinitely often, FL1 gives infinitely many fl-deliveries at q, and the first triggers the pl-delivery. **PL2.** q pl-delivers index k from p only when $k \notin seen[p]$, and inserts k in the same step; so at most once per index, and each pl-send uses a fresh index. (Two pl-sends of equal payload m are distinct logical messages with distinct indices — the abstraction deliberately distinguishes them.) **PL3.** A pl-delivery requires an fl-delivery of $\langle \text{DATA}, m, k\rangle$, which by FL3 was fl-sent by p, which the protocol does only for pl-sent messages. *(b)* Delete the acks: PL1–PL3 still hold by the second branch of the PL1 argument — the acknowledgments buy *economy only* (retransmission ceases; *pend* shrinks), never the guarantee. Reliability came from stubbornness plus deduplication. *(c)* $seen[p]$ grows forever (mitigable to a single counter if indices are delivered in order — but ordering is not among PL1–PL3; that is a stronger link, and Chapter 10's opening subject); *pend* is bounded only if acks eventually arrive, i.e. only between correct parties. Bounding *seen* outright is exactly Problem 1.10(b): a model purchase.

Solution (1.8)**.** *(a)* The fatal execution, in the convention of Exhibit 1.1:



The acknowledgment dies in the Post; Bingo crashes after the synchronous apply; recovery restores $c = 1$ but an empty table; the retry's sequence number seven exceeds the reborn $high[s] = 0$ and is applied again. One umbrella, two charges. *(b)* Intended invariant: $n \leq high[s]$ *if and only if operation* (s, n)*'s effect is in the durable state.* The crash breaks the only-if direction: the effect is durable, the record of it was not. The two facts lived at different durabilities, and the crash chose between them. *(c)* Repair: make the record as durable as the effect, *atomically* — one write (or one write-ahead log record) containing both the increment and (s, n); recovery rebuilds *high* from the log. Atomicity is not pedantry: persist the record before the effect and a crash between them loses the increment (the retry, now filtered, never reapplies it — a permanently

undercounted umbrella); effect before record is part (a). The pair must survive or perish together — which is, in miniature, the transaction, and Chapter 8's opening bar.

Solution (1.9, 1.10, 1.11, 1.12)**.** By standing policy: hints only (the Hints section above). For 1.9(a), the arithmetic checkpoint: $n = 20$, since $(\frac{1}{2})^{20} = \frac{1}{1{,}048{,}576} < 10^{-6}$ and $(\frac{1}{2})^{19} > 10^{-6}$.

1.12 The Reading

The fallacies of distributed computing (Deutsch and colleagues, Sun Microsystems, mid-1990s). Five minutes; a list with scars. Read it as an audit against your last three incidents, and score honestly. Items one, five, and six are this chapter's material; item two was priced, with Toronto and Sydney, in Part One. It has not aged, because physics has not.

Bailis and Kingsbury, "The Network is Reliable," ACM Queue, 2014. The documentary record behind the Post's charter: a catalogue of published, attributable network partitions across cloud providers, enterprises, and measurement studies. Skim the incident inventory; savour the running observation that redundancy so often fails *correlatedly* — redundant on the diagram, coupled in the failure domain. Read it as an actuary reads flood data; it is why the model box is not pessimism.

Standing companion: Kleppmann, *Designing Data-Intensive Applications*, chapters 5–9. Assigned once, for the whole season, for parallel reading at leisure. Chapter 8 ("The Trouble with Distributed Systems") is this chapter's terrain in prose and will make an excellent second telling; chapters 5 and 9 will meet us again in Chapters 6 and 7 of this course.

Chapter 2

Time, Clocks, and the Ordering of Events

Companion to Episode Two. The episode tells the story — the patent clerk in Bern, the leap second that sat the internet down, the auditor's telephone; this chapter keeps the record: the sins of physical clocks priced, the two planks that survive distribution, happens-before with concurrency as its honest residue, the Lamport clock and its clock condition proven, the vector clock and its two-way characterization proven entire, cuts and their consistency, and the Chandy–Lamport photograph of a system that never holds still, made legitimate by a shuffling argument. Statements, proofs, protocol boxes, problems.

2.1 Part One — The Sins of Physical Clocks

The chapter pays the third debt of Chapter 1's register — *there is no shared clock* — and pays it in full. The lineage is declared at the door: Lamport's "Time, Clocks, and the Ordering of Events in a Distributed System" (1978) took its central idea directly from special relativity, where the patent clerk in Bern had shown in 1905 that *simultaneity is not a fact about the world* — no master clock in the sky, no universal now — with the message playing the light signal's role: the fastest thing by which one place can influence another. The engineer's instinct that this is a synchronization problem is wrong twice over: the watches disagree (the engineering layer, priced first), and even perfect watches would not answer the question asked of them (the finding, priced last).

The quartz layer. Inside nearly every machine, a sliver of quartz whose vibration is the machine's heartbeat, off the stamped frequency by some tens of parts per million — twenty parts per million is a second and three quarters *per day*, the better part of a minute a month. Nor is the drift constant: it breathes with temperature, hence with load — the clocks run measurably differently during the incident than before it.

The discipline, and its lab coat. The network time protocol — NTP — disciplines the fleet through strata of servers descending from atomic clocks and satellite receivers. Its central assumption is a Chapter 1 fallacy in a lab coat: the client measures a round trip to a time server and assumes *symmetric paths*, splitting the difference to estimate its offset. On real networks, congested one way and routed differently in each direction, the assumption fails silently — a bias no amount of polling repairs, since every poll assumes the same. Good conditions: within a millisecond or a few on a local network, tens of milliseconds across the wide area. Bad ones: nothing at all, while continuing to report success.

The discrete indignities. The catalogue; the episode stages each in full.

- **slewing versus stepping:** a daemon correcting a wrong clock either *slews* — runs it slightly fast or slow until it converges, so time never jumps — or *steps* it in one motion, so time lurches and occasionally moves *backwards*; every piece of software that computes a duration by subtracting wall-clock readings has an opinion about backwards time, usually an exception at three in the morning.
- **the paused guest:** a virtual machine wakes with its clock frozen in the past, then experiences the correction as a seizure.
- **the leap second:** the Earth does not keep atomic time, so the custodians of UTC periodically decree a minute of sixty-one seconds. On 30 June 2012 the scheduled leap second tripped a defect in the Linux kernel's timing machinery; healthy machines across the internet sat down at midnight and spun, an airline's check-in system most memorably among them.
- **the leap smear:** the remedy published openly by Google — dilute the extra second homeopathically across the whole day, every clock imperceptibly and coordinatedly slow, none ever *suddenly* wrong; civilized mendacity. The fine print is the chapter's thesis in miniature: smearing is a local decision, so fleets smearing on different schedules disagree *by design* for the duration. Among machines, even the corrections to time are relative.

Two clocks, two jobs. Every machine offers two clocks: the *time-of-day* clock — disciplined by NTP, naming the date, subject to slewing, stepping, smearing, and decree — and the *monotonic* clock, a stopwatch counting since boot, promising nothing about what o'clock it is and everything about never running backwards. Durations, timeouts, and intervals are the monotonic clock's business; the time-of-day clock names moments for humans and does nothing else — a respectable fraction of the industry's mystery timeouts trace to a duration computed across a clock step. Mark the asymmetry: the monotonic clock is trustworthy precisely because it has renounced knowing what time it is. The season is full of that trade.

The moral, in two layers.

- **engineering:** cross-machine clock disagreement (milliseconds on a good day, nothing in particular on a bad one) is *larger than the events it would need to order* — a same-building message takes a fraction of a millisecond. Sorting cross-machine events by wall-clock timestamp is, at fine grain, noise wearing the costume of measurement; the merged log showing a reply before its request was committing perjury with digits.
- **the finding:** suppose the engineering solved and every clock agreed to the nanosecond. The timeline would *still* not answer the question actually cared about — never "which happened first" but "*could this have influenced that*." Two events a nanosecond apart on opposite sides of a datacentre are ordered by the perfect clocks, and the order is causally meaningless: no influence could have crossed between them. The perfect timeline orders things whose order cannot matter and is silent about the only relation with consequences. Physics offered the patent clerk the same bargain, and he declined it. So shall we.

2.2 Part Two — Dismantling the Question

The question under dissection — event *a* on Bingo, event *b* on Gussie: which happened first? — presumes a master timeline in which every event has its row and the order of rows is a fact. Physics gave that picture up in 1905: events causally sealed off from each other — too far apart, too close in time, for light to cross — are ordered differently by observers in different states of motion, with no fact of the matter to arbitrate.

The trade stages the same quarrel daily: two services' logs, clocks skewed a few milliseconds, yield two internally coherent histories of the same events, differing only in whose clock was believed. If no message passed between the two events, "which is correct?" has exactly the standing of the train passenger's quarrel with the platform: none. Order is real only where influence is possible; where influence is impossible, order is a bookkeeping convention.

The dictionary. In a distributed system the fastest — indeed the only — carrier of influence between machines is the message; the wall clock carries none. Lamport drew the correspondence himself, and it is load-bearing, not decoration — a physicist and a distributed-systems theorist can, with this small dictionary, read one another's impossibility results:

- **the light signal** becomes the message — the fastest carrier of influence; in our world, the only one;
- **the light cone** becomes the event's causal future — everything reachable from it by roads of letters;
- **spacelike separation** becomes concurrency — merely elsewhere;
- **the observer,** with his private simultaneity, becomes a log collation, with its private ordering of other machines' events;
- **the invariant** — the order of events connected by a signal, agreed by all observers — becomes the order of events connected by messages, agreed by all collations.

The two planks. The complete inventory of bedrock; every other ordering claim across machines is built from these two or is decoration:

- **program order:** within a single process, order is real — one machine executing one sequence of steps, its diary trustworthy about its own day;
- **the postal plank:** a message is received after it is sent — whatever the Post does to a letter, it does not deliver into the past.

It seems a poverty; it is a foundation, and the rest of the chapter is the architecture — beginning with the substrate, restated formally, and the models under which every claim below is made.

Definition 2.1 (events and executions, recalled and sharpened)**.** Processes are as in Definition 1.1: deterministic machines whose steps are *events* of three kinds — local steps, sends, and receives. An *execution* α is a (finite or infinite) sequence of events such that (i) the subsequence at each process is consistent with that process's machine; (ii) every $\textsc{receive}(m)$ is preceded in α by its matching $\textsc{send}(m)$; and (iii) the channel discipline of the model box holds. Events of the reference trace (Exhibit 2.1) are named $b_1, b_2, \ldots$ per process in program order.

Model of this chapter

For orders and clocks (Defs. 2.2–2.8, Thms. 2.1–2.2). Asynchronous message passing among deterministic processes $p_1, \ldots, p_n$ (Def. 1.1); no process failures in this chapter; links promise only *no creation* — every delivered message was previously sent. Loss, duplication, and reordering are irrelevant to the definitions: happens-before is defined over the deliveries that occur. No wall clocks appear anywhere.

For the photograph (Def. 2.10, Thm. 2.4). A strictly better Post, purchased and declared: channels are *reliable* and *first-in-first-out* — every message sent on a channel is delivered, exactly once, in the order sent — and the postal map is strongly connected, so markers can reach everyone. Buildable

over fair-loss links by Chapter 1's constructions (retransmit, deduplicate) plus per-channel sequence numbers; the FIFO clause is spent at exactly one step of Theorem 2.4 and nowhere else (Rem. 2.11).

Failures: deliberately absent. Detecting them is Chapter 3's business; photographing across them is Chapter 10's recovery business. In this chapter the company is immortal and merely unpunctual.

2.3 Part Three — Before, After, and Elsewhere

Now the order the two planks support. Lamport called it *happens-before* — read "a happens before b," written $a \to b$ — and it earns its three clauses stated precisely, because it is the load-bearing definition of the season and possibly of the field.

Definition 2.2 (happens-before; Lamport 1978)**.** The relation $\to$ on the events of an execution is the smallest relation such that

HB1 if a and b occur at the same process and a earlier, then $a \to b$;

HB2 if $a = \textsc{send}(m)$ and $b = \textsc{receive}(m)$, then $a \to b$;

HB3 if $a \to b$ and $b \to c$, then $a \to c$.

Equivalently: $a \to b$ iff there is a *causal path* $a = e_0, e_1, \ldots, e_k = b$ with $k \geq 1$, each consecutive pair related by HB1 or HB2. The relation is a strict partial order: every HB1/HB2 step moves strictly later in the execution sequence, so no cycles arise.

Nothing is before anything else unless these rules force it. The episode walks the chain with the company; the skeleton: Bingo's entry precedes his send (HB1); the send precedes Tuppy's receipt (HB2); HB1 again through Tuppy's desk, HB2 into Gussie's receipt, and HB3 chains the procession — Bingo's morning entry happens before Gussie's receipt, through two letters and three desks. Influence had a road, and the relation records the roads. Gussie's earlier cup of tea is related by no clause to any of it. The desk's drill-ground, fixed now:

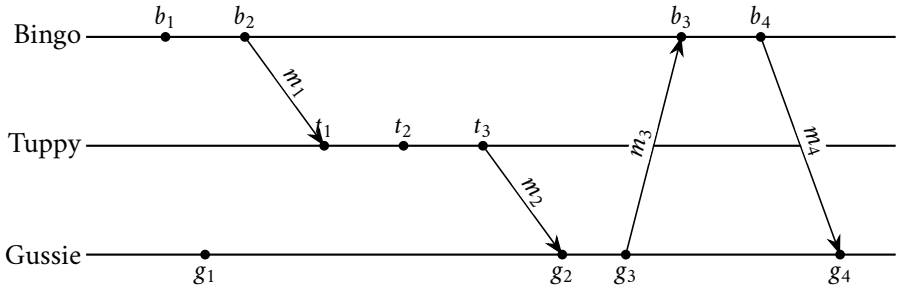


Exhibit 2.1 (the reference trace). Three processes, eleven events, four messages: $m_1 : b_2 \to t_1$; $m_2 : t_3 \to g_2$; $m_3 : g_3 \to b_3$; $m_4 : b_4 \to g_4$. Events b_1, g_1, t_2 are local steps (g_1 is, canonically, the tea). All of this chapter's calisthenics compute on this trace.

The season's phrase for the tea's standing, to be used exactly and often: *concurrent* — not before, not after, merely elsewhere.

Definition 2.3 (concurrency)**.** Distinct events a, b are *concurrent*, written $a \parallel b$, iff neither $a \to b$ nor $b \to a$. The relation is symmetric and *not* transitive: on Exhibit 2.1, $b_2 \parallel g_1$ and $g_1 \parallel t_2$, yet $b_2 \to t_2$.

The same trace, seen truly. Exhibit 2.1 draws the events along a horizontal axis that reads, irresistibly and wrongly, as time — the very timeline Part One abolished. Strip that axis and redraw the trace as what happens-before actually is, a directed acyclic graph, and the partial order shows its true shape.

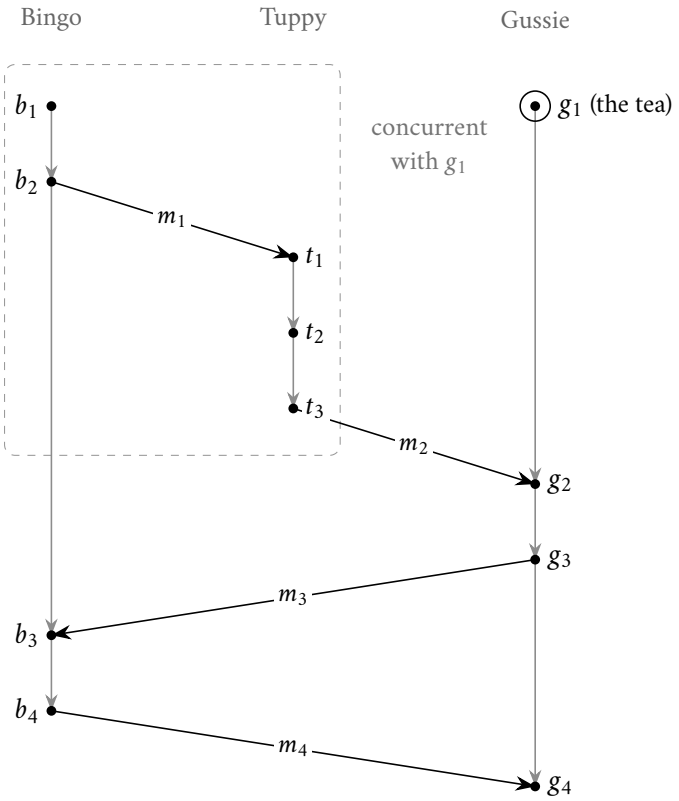


Exhibit 2.2 (the reference trace as a causal graph). The eleven events of Exhibit 2.1, redrawn as the directed acyclic graph of happens-before. The horizontal axis is gone: columns mark only *which desk*, and height is *causal depth*, so a downward road is the sole meaning of "before." Two edge-classes, and only two — the transitive reduction — corresponding to the two planks of Part Two: grey edges are program order (HB1, same desk); black arrows are messages (HB2, a letter), $m_1, \ldots, m_4$ as before. Then $a \to b$ iff a directed road runs from a down to b, and $a \parallel b$ iff none runs either way. The tea g_1 is the picture's lesson: no road reaches it and it reaches nothing until g_2, so it is concurrent with the whole upstream block $\{b_1, b_2, t_1, t_2, t_3\}$ (dashed) — and because that block holds both b_2 and t_2, which are themselves ordered ($b_2 \to t_1 \to t_2$), one sees at a glance why "$\parallel$" does not chain: $b_2 \parallel g_1$ and $g_1 \parallel t_2$, yet $b_2 \to t_2$. Not a timeline but a lattice of roads.

What concurrency means. Not "at the same time" — abolished; that is the whole point — and nothing about wall clocks at all: no road of influence connects the two events, in either direction. Happens-before is *potential* causality, generously drawn — it can overstate influence, never understate it; its negation is iron-clad — no protocol, no forensic reconstruction can make concurrent events cause and effect. "Concurrent" is not a confession of ignorance about the true order; it is the report that *there is no true order to be ignorant of*.

Ancestor or cousin. The genealogical image is kept: ordered events are ancestor and descendant; concurrent events are cousins — neither descended from the other. Formally: comparable or incomparable

under $\rightarrow$. From the trace: t_3 and g_1 are concurrent, whatever any wall clock would report. The season asks the question a hundred times, and the answer moves money; Problem 2.1 is the drill.

Totalizing the partial order. Happens-before is a *partial* order — a lattice of roads, not a timeline — and engineers itch to force every pair into line. Exactly two honest ways, both priced by the season:

- **manufacture** an order — arbitrary but consistent, at the cost of meaning: the next two Parts build the machinery and post the warning signs (Remark 2.7);
- **rent** one from a distinguished process through whom all events pass — at the cost of a throne, and thrones must be defended: Chapter 4.

What one may not do is read a total order off the wall clocks and call it causality — Part One's perjury, now with a theorem behind the indictment.

2.4 Part Four — The Lamport Clock

Having demolished the wall clock's claim to order events, Lamport's paper does something with the confidence of a conjuror: it builds a clock anyway — a *logical* clock, made of nothing but a counter, that captures exactly as much of "before" as actually exists, and not one drop more. The target first, then the instrument.

Definition 2.4 (logical clocks; the clock condition)**.** A *logical clock* is an assignment C of values from a totally ordered set to events, computed by each process from local information only. C satisfies the *clock condition* if $a \rightarrow b \implies C(a) < C(b)$.

The mechanism is three rules that fit in a waistcoat pocket: tick before every event — the "tick" is the increment preceding every event, here and in Box 5 — carry the stamp on every letter; and on receipt, first raise the counter to the larger of its own value and the carried one, then tick.

Protocol — Box 4. The Lamport clock (Lamport 1978)

At each process:

1: **state:** $c \leftarrow 0$

2: **upon** any event e (local step, send, or receive):
3: **if** $e = \mathsf{deliver}(m)$: $c \leftarrow \max\big(c,\ \mathrm{stamp}(m)\big)$
4: $c \leftarrow c + 1$; $C(e) \leftarrow c$
5: **if** $e = \mathsf{send}(m, q)$: attach $\mathrm{stamp}(m) \leftarrow c$

Checked against Lamport (1978), implementation rules IR1–IR2, with unit increments. The tick precedes every event; a receive is stamped downstream of the merge. Theorem 2.1 is its warranty and Remark 2.6 its warning label.

Definition 2.5 (the Lamport clock)**.** The algorithm of Box 4; $C(e)$ denotes the counter value it assigns to event e.

The reference run, in small numbers (the full computation is Solution 2.2):

- **Bingo:** morning entry $C = 1$; send $C = 2$, the letter carrying the number two.
- **Tuppy, idle at nought, receives:** raise to the larger of nought and two, tick — the receipt is stamped 3; his entry, 4; his send to Gussie, 5.

- **Gussie, whose solitary tea sits at** 1**, receives:** the larger of one and five is five, tick, 6. The merge rule is the whole trick: the letter did not merely carry news; it carried *time*, and dragged the recipient's clock forward to meet it.

The instrument's warranty is the clock condition, and its proof fits in two sentences: causality is a road, the counter climbs at every paving stone, and a finite chain of strict increases ends strictly higher than it began. Written out:

Theorem 2.1 (the clock condition holds; Lamport 1978)**.** *The Lamport clock of Box 4 satisfies the clock condition: if $a \to b$ then $C(a) < C(b)$.*

Proof of Theorem 2.1. Let $a \to b$, witnessed by a causal path $a = e_0, e_1, \ldots, e_k = b$, $k \geq 1$. Consider one step. If e_i and e_{i+1} occur at the same process with e_i earlier (HB1), then between the two the counter never decreases and is ticked at least once — before e_{i+1} itself if not sooner — so $C(e_{i+1}) \geq C(e_i) + 1$. If $e_i = \textsc{send}(m)$ and $e_{i+1} = \textsc{receive}(m)$ (HB2), the recipient first raises its counter to at least $\text{stamp}(m) = C(e_i)$ and then ticks for the receive, so again $C(e_{i+1}) \geq C(e_i) + 1$. A finite chain of strict increases ends strictly higher than it began: $C(b) \geq C(a) + k > C(a)$. □

Now the trap, which catches working engineers by the dozen: the clock condition is a one-way street, and the run above already contains the counterexample — the tea stamped 1, Bingo's send stamped 2, the two events concurrent. The numbers impose an order the world does not contain.

Remark 2.6 (the converse fails; what a comparison licenses)*.* Exhibit 2.3: $C(g_1) = 1 < 2 = C(b_2)$, yet $g_1 \parallel b_2$. The sole inference licensed by $C(a) < C(b)$ is $\neg(b \to a)$ — "b did not happen before a." Sorting by Lamport time produces a linear order *consistent with* causality (Rem. 2.7); reading that order *as* causality is the wall-clock perjury of Part One, recommitted with better tooling.

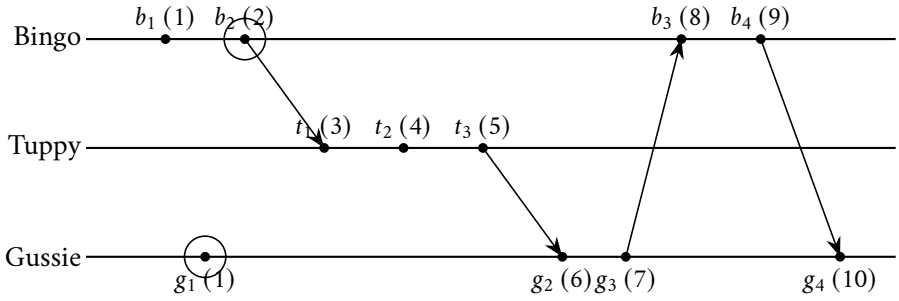


Exhibit 2.3 (Lamport stamps; the inversion). Each event carries its Box-4 stamp in parentheses. The circled pair is the warning label of Remark 2.6 made visible: $C(g_1) = 1 < 2 = C(b_2)$, yet $g_1 \parallel b_2$ — the numbers impose an order the world does not contain. The full computation is Solution 2.2.

An engineer who sorts events by Lamport time and reads the result as a causal history has manufactured exactly the fiction Part Two demolished — it has been done in postmortem documents by people who cite the paper. What, then, is the manufactured order *for*? Break the remaining ties — concurrent events with equal counters — by any fixed convention; alphabetical order of the gentlemen's names will do.

Remark 2.7 (the manufactured total order)*.* Fix any total order on process names and define $a \Rightarrow b$ iff $C(a) < C(b)$, or $C(a) = C(b)$ and a's process precedes b's by name. Then $\Rightarrow$ is a strict total order on events extending $\to$: two events at one process never share a stamp (the tick), cross-process ties break by name, and Theorem 2.1 ensures $\to$-related pairs already agree. Lamport's mutual-exclusion protocol runs on $\Rightarrow$; so does the replicated state machine (Chapter 5).

An order manufactured, not discovered — arbitrary where the world was silent, consistent with every road that does exist, and computable by everyone from local information; the warning label stays glued on.

Mutual exclusion, and the acorn. Lamport's paper puts ⇒ straight to work: mutual exclusion with no coordinator. Every request is broadcast, stamped, and queued; every process sorts its copy of the queue by the manufactured order; a claimant helps himself when his request heads his own copy and he has heard something later-stamped from everyone else — proof that no earlier-stamped request loiters in the Post. Agreement by determinism applied to a shared order, not by negotiation. And the paper's throwaway generalization — *replicas of a deterministic machine applying the same commands in the same total order remain perfect copies forever* — is the replicated state machine, tossed off in 1978 and industrialized for thirty years since; Chapter 5 collects the inheritance.

Lamport clocks in livery. Every *term* in Raft, every *ballot* in Paxos, every *epoch*, *generation*, and *fencing token* from Chapter 4 onward is a Lamport clock under an assumed name: a counter that only rises, carried on messages, merged by taking the maximum, letting a recipient say "this instruction is stale — I have seen a higher number" without consulting any wall clock. When a freshly elected leader raises the epoch and the deposed leader's letters begin to bounce, the merge rule is drawing a salary. The paper is dated 1978; the payroll is current.

The modest summary: a cheap, consistent, manufactured total order, plus a one-way guarantee about the real partial one. What one counter cannot do is tell "ordered" from "merely elsewhere." The repair is delightfully literal: carry more counters.

2.5 Part Five — The Vector Clock

The diagnosis dictates the repair. A single counter compresses an event's entire causal ancestry into one number; compression loses information; two very different pasts can compress to comparable numbers, and the numbers then imply an order the pasts do not have. To *characterize* causality — "before, after, or elsewhere" from the stamps alone — the stamp must summarize the past without losing it: one counter *per colleague*. This is the vector clock, introduced independently by Colin Fidge in Australia and Friedemann Mattern in Germany (Fidge 1988; Mattern 1989) — the field discovering the same instrument twice, presumably concurrently. Each process keeps a full roster — an entry for every member — read in the season's fixed template, "Bingo's clock reads: Bingo three, Tuppy one, Gussie nought," written $V = (3, 1, 0)$, coordinates fixed as (Bingo, Tuppy, Gussie) = (p_1, p_2, p_3); an event's vector timestamp is a précis of its entire ancestry. The rules mirror Box 4's: tick your own entry on any event; carry the whole roster on every letter; on receipt, merge entrywise, then tick your own.

Protocol — Box 5. The vector clock (Fidge 1988; Mattern 1989)

At process p_i, for a company of n:

1: **state:** $V \leftarrow (0, \ldots, 0) \in \mathbb{N}^n$

2: **upon** any event e:
3: **if** $e = \mathsf{deliver}(m)$: **for each** j: $V[j] \leftarrow \max\big(V[j],\ \mathrm{stamp}(m)[j]\big)$
4: $V[i] \leftarrow V[i] + 1$; $\quad V(e) \leftarrow V$
5: **if** $e = \mathsf{send}(m, q)$: attach $\mathrm{stamp}(m) \leftarrow V$

Checked against Fidge (1988) and Mattern (1989). Conventions in the literature differ on where the receive's own tick falls; ours ticks *after* the merge and stamps the receive with the result, exactly as the ledger lemma of Theorem 2.2 requires.

One roster is *below* another when every entry is less than or equal and at least one entry is strictly less; formally:

Definition 2.8 (the vector clock; the vector order)**.** The algorithm of Box 5 over n processes; $V(e) \in \mathbb{N}^n$ denotes the roster it assigns to event e. For rosters U, W: $U \le W$ iff $U[i] \le W[i]$ for every i; $U < W$ iff $U \le W$ and $U \ne W$; $U \parallel W$ (*incomparable*) iff neither $U \le W$ nor $W \le U$.

The reference run, now in rosters:

- **Bingo:** morning entry $(1,0,0)$; send $(2,0,0)$, the letter carrying that whole roster.
- **Tuppy:** receipt — merge, then tick — $(2,1,0)$; his entry, $(2,2,0)$; his send to Gussie, $(2,3,0)$.
- **Gussie, whose solitary tea gave him** $(0,0,1)$**:** receipt — merge and tick — $(2,3,2)$: the ancestry, itemized — downstream of two of Bingo's events, three of Tuppy's, and one prior event of his own.
- **The tea against Bingo's send:** $(0,0,1)$ versus $(2,0,0)$ — each ahead of the other somewhere; neither below the other; incomparable.

The theorem that makes this the chapter's second instrument is stated with the respect an if-and-only-if deserves — both directions, this time: the stamps no longer merely respect causality; they *encode* it, completely.

Theorem 2.2 (the vector characterization; Fidge 1988; Mattern 1989)**.** *For distinct events a, b of an execution:*

$$a \to b \iff V(a) < V(b),$$

and consequently $a \parallel b \iff V(a) \parallel V(b)$.

The strategy first, then the argument entire. The forward direction is the Lamport argument run entrywise — roads lift rosters. The converse is where the extra information earns its keep, and its engine is the *ledger lemma*: the p-entry of an event's roster counts exactly the p-events in its causal past — a large number in your ledger is testimony, and testimony travels only by post.

Proof of Theorem 2.2. For an event e and a process p, let

$$K_p(e) \;=\; \{\, p\text{-events } f \;:\; f \to e \text{ or } f = e \,\}.$$

Each $K_p(e)$ is a *prefix* of p's timeline: if f' precedes f at p and $f \in K_p(e)$, then $f' \to f$ (HB1) and hence $f' \to e$ by transitivity (or $f' \to e$ directly when $f = e$), so $f' \in K_p(e)$.

Step 1 (the ledger lemma). For every event e and every process p:

$$V(e)[p] \;=\; |K_p(e)|.$$

"The p-entry of an event's roster counts exactly the p-events in its causal past, itself included when the event is p's own."

We induct along the execution order. Let e occur at process q, and let e^- be q's previous event (if any).

Case: e is a local step or a send. Then e receives nothing, so every causal path into e must make its final step by HB1 — through e^-. Hence $f \to e$ iff $f \to e^-$ or $f = e^-$. For $p \ne q$: $K_p(e) = K_p(e^-)$, and Box 5 leaves $V[p]$ unchanged. For $p = q$: $K_q(e) = K_q(e^-) \cup \{e\}$, one element larger, and Box 5 ticks $V[q]$ by one. (Base case, e the first event of q: $K_q(e) = \{e\}$ and the tick raises $V[q]$ from 0 to 1; $K_p(e) = \emptyset$ for $p \ne q$, matching the untouched zeros.)

Case: $e = \textsc{receive}(m)$, *with* $s = \textsc{send}(m)$. A causal path into e makes its final step either by HB1 (through e^-) or by HB2 (through s). Hence

$$K_p(e) \;=\; K_p(e^-) \,\cup\, K_p(s) \,\cup\, \big(\{e\} \text{ if } p = q\big).$$

Now the decisive observation: $K_p(e^-)$ and $K_p(s)$ are both prefixes of p's timeline, and *the union of two prefixes is the longer of them*, so $|K_p(e^-) \cup K_p(s)| = \max\big(|K_p(e^-)|, |K_p(s)|\big) = \max\big(V(e^-)[p], \text{stamp}(m)[p]\big)$ by the induction hypothesis — which is precisely the entrywise merge of Box 5; the subsequent tick accounts for $\{e\}$ when $p = q$. This proves the lemma, and it is "testimony travels only by post" made mechanical: entries change only at ticks and merges, and merges happen only at receipts.

Step 2 (the theorem). ($\Rightarrow$) Let $a \to b$, with a at process p and b at process q. For every process r, $K_r(a) \subseteq K_r(b)$: any f with $f \to a$ or $f = a$ satisfies $f \to b$ through $a \to b$. By the lemma, $V(a) \le V(b)$ entrywise. Strictness at coordinate q: $b \in K_q(b)$, but $b \notin K_q(a)$ — for $b \to a$ together with $a \to b$ would close a cycle in a strict order, and $b = a$ is excluded — so $V(a)[q] < V(b)[q]$. Hence $V(a) < V(b)$.

($\Leftarrow$) Let $V(a) < V(b)$, and let a be the k-th event of its process p, so that $V(a)[p] = k$ by the lemma ($K_p(a)$ is exactly p's first k events). Then $V(b)[p] \ge k$, so $K_p(b)$, a prefix of p's timeline of size at least k, contains p's k-th event — namely a. Thus $a \to b$ or $a = b$; the events are distinct; so $a \to b$.

Finally, $a \parallel b \iff V(a) \parallel V(b)$ is the conjunction of the two directions, negated. □

Concurrency thereby acquires its final and best characterization: two events are concurrent exactly when each is ignorant of some part of the other's past — mutual, certified, provable ignorance. That is what "merely elsewhere" cashes out to.

Detecting concurrency in anger. Chapter 1's third performance — Bertie's "leave with the porter" and its correction "ring the top bell" — replayed with rosters; the episode stages both runs. The skeleton:

- **true succession:** both notes enter at Bingo; the original forwards stamped $(4, 2, 1)$, the correction $(5, 2, 1)$. Ordered stamps — the correction demonstrably saw the original — so every replica, whatever the arrival order, applies the elder first or discards it: the stamp, not the arrival time, carries the truth.
- **rivals:** Bertie sends the porter note via Bingo — stamped $(4, 2, 1)$ — and the bell note directly to Gussie, who stamps it $(2, 2, 3)$. Incomparable; concurrent — written in mutual ignorance, and no protocol may silently pretend one superseded the other.

What to *do* with certified rivals — keep both and ask Bertie; merge under some lawful rule — belongs to Chapters 6–7; the shopping cart obliged to hold two kettles at once is this moment at industrial scale. The point here is sharper: the vector clock converts "the replicas mysteriously disagree" into "the replicas hold two provably concurrent writes" — from a haunting into a diagnosis. Diagnoses have treatments; hauntings merely have postmortems.

The costs, priced by the roster.

- **freight:** an entry per member on every letter; a fleet of thousands makes the stamps heavier than many payloads.
- **membership:** the roster presumes you know who the company *is* — itself a distributed agreement problem, met properly when agreement is.
- **no compression:** shrinking the roster while still answering "ancestor or cousin" exactly is a vain hope — Problem 2.11 walks to the theorem that the vector's width is the problem's true dimension,

not an implementation's clumsiness. The industry *rations* the truth instead: track fewer participants (versions per key rather than per node); prune the retired; or retreat to the one-number clock and its one-way honesty. The *dotted version vector* refinement is named here and left wrapped for Chapter 6, where the shopping cart must hold two kettles at once.

Side by side, the honest summary of the two instruments: the Lamport clock is one number — cheap, total after tiebreaking, manufactured, one-way. The vector clock is a roster — costly, partial, *truthful*, two-way. One gives an order to act on; the other gives the facts of the matter. Mature systems tend to carry both and confuse neither.

2.6 Part Six — A Glimpse of Disciplined Delivery

A corridor, glimpsed and deliberately not entered. The Post may show a process an effect before its cause — Gussie's answer, racing a luckier route, reaching Bingo before Tuppy's question — and nothing in Chapter 1's charter forbids it. With vector clocks the rudeness is *detectable*: the answer's roster confesses an ancestor Bingo has not yet seen, so Bingo sets the letter aside, unopened, until its prerequisites arrive. Done systematically, this is *causal delivery*: a Post that never shows anyone an effect before its cause — still free to shuffle *concurrent* letters, which have no order to violate, but forbidden to invert the roads. A genuine consistency contract at a particular altitude: stronger than the Post's anarchy, far cheaper than a total order. The catalogue of such contracts, prices attached, is Chapter 7 — where causal consistency joins a hierarchy, and its practical children, the *session guarantees*, turn out to be things your systems already half-promise under folk names like read-your-own-writes. The corridor exists; the vector clock unlocked it. Onward; the photograph awaits.

2.7 Part Seven — The Photograph

The set piece, and the problem before the miracle: the chapter's own paradox in a business suit. One operates a small bank and wants its *global state* — every balance, everywhere, at once — for reasons that are Tuesday: a backup; an audit (does the money add up); a detector (is the system deadlocked, is a computation finished, has an invariant quietly broken). Every such question asks "what is the state of the whole system, *now*?" — and the whole chapter has been spent executing "now." The question seems not merely hard but ill-posed; the backup must be taken anyway.

The naive audit. Bingo keeps Bertie's current account, £60; Tuppy his savings, £40 — one hundred pounds in the world. Bertie moves ten: Bingo debits (ledger £50) and posts the ten to Tuppy. The auditor telephones each gentleman once while the letter is in the Post:

- **ten pounds vanished:** Bingo after the debit (fifty), Tuppy before the arrival (forty) — £90; the in-flight letter counted nowhere.
- **ten pounds minted:** Tuppy just after the arrival (fifty), Bingo just before the debit (sixty) — £110. Mark the shape: an *arrival* included without its *departure* — an effect without its cause — a picture corresponding to no state the system was ever in, or could ever have been in.

Problem 2.6 enumerates all four interleavings and names each one's sin.

What a good photograph means. Draw a single frontier across the bank's space-time diagram, separating each timeline into a past and a future. The frontier, formally, is a *cut*; "no effect without cause" is

the cut's *consistency*. Arrows may cross the frontier forwards — a letter sent in the past, still undelivered: money in flight, honestly recorded as such — but no arrow crosses backwards.

Definition 2.9 (cuts; consistent cuts)**.** A *cut* K of an execution is a set of events whose restriction to each process is a prefix of that process's events; its *frontier* is the last K-event at each process. K is *consistent* iff every receive in K has its matching send in K: no effect without its cause. (Check: this is exactly closure of K under $\rightarrow$ — HB1 closure is the prefix property, HB2 closure is the stated clause, and HB3 adds nothing new; Problem 2.9.)

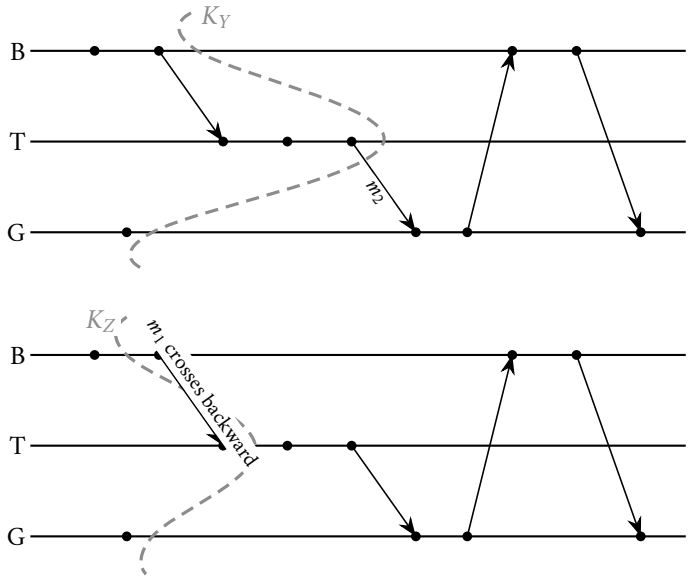


Exhibit 2.4 (two frontiers). Top: the cut K_Y (Bingo through b_2, Tuppy through t_3, Gussie through g_1) is consistent; m_2 crosses it *forward* — mail in flight, recorded as channel state, entirely lawful. Bottom: the cut K_Z (Bingo through b_1, Tuppy through t_1, Gussie through g_1) is inconsistent: $t_1 = \textsc{receive}(m_1)$ lies inside while $b_2 = \textsc{send}(m_1)$ lies outside — an effect without its cause. Problem 2.5 classifies these and two more.

The redefinition, pure Lamport in spirit: abandon "the state at an instant," which does not exist, and pursue "the state along a consistent cut," which does. The frontier need not be straight nor correspond to any wall-clock moment; it need only respect the roads. What the snapshot must deliver — and the currency of reachability between global states, $S \rightsquigarrow S'$ (some legal execution leads from S to S'), in which its strongest promise is denominated — is fixed now:

Definition 2.10 (global state of a cut; the snapshot problem)**.** For a consistent cut K: the *global state* $S(K)$ comprises, for each process, its state after executing exactly its K-prefix, and, for each channel $c = (p, q)$, the *cross-frontier mail* $L_c(K) = \{\, m \text{ sent on } c \text{ in } K : \textsc{receive}(m) \notin K \,\}$. The *snapshot problem*: during a live execution, the processes must, by exchanging ordinary messages, record process states and channel states equal to $S(K)$ for *some* consistent cut K of that execution. The *strong form* adds: $S(K)$ is reachable from the global state at which the protocol began, and the global state at which it completed is reachable from $S(K)$. Theorem 2.4 delivers the strong form.

The instrument. The gentlemen cannot see the diagram — they live inside it — and they cannot stop: a bank that pauses the world to take its picture has solved the problem by not having a system. The photograph must be taken live, from within, while the letters keep flying. Chandy and Lamport (1985) — the same

Lamport, seven years on — supply an algorithm whose entire mechanism is one special letter and two rules. The special letter is the *marker*, the control message of Box 6, drawn densely dotted in the exhibits from here to the season's end (a convention added this chapter); the declared purchase is this chapter's model box — channels reliable and first-in-first-out — and the exact instant where the FIFO clause earns its entire keep is marked below. The rules:

- **the recording rule:** the initiator records his own ledger, at once, and *immediately* — before conducting one further scrap of business — posts a marker on every outgoing channel;
- **the marker rule:** any gentleman receiving his *first* marker does the same at once — records his ledger, notes that marker's channel as empty, posts markers on all his outgoing channels — and for every other incoming channel opens a *docket* (formally, that channel's recorded state), collecting every ordinary letter arriving after his own recording but before that channel's marker, whereupon the docket closes.

The marker is a sweep — a line drawn down each channel: everything ahead of me on this route belongs to the past of the photograph; everything behind me, to its future.

Protocol — Box 6. The Chandy–Lamport snapshot (1985)

Control message ⟨marker⟩ (tagged with an audit identifier if audits may overlap). At each process:

1: **state:** *recorded* ← **false**; **for each** incoming channel c: docket(c) ← $\perp$ (unopened)

2: **procedure** RECORD():
3: record own application state; *recorded* ← **true**
4: **immediately** — before any further application send:
5: **for each** outgoing channel: send(⟨marker⟩)
6: **for each** incoming channel c with docket(c) = $\perp$: docket(c) ← ⟨ ⟩ (open)

7: **upon** initiating the audit: RECORD()

8: **upon** deliver(⟨marker⟩) on channel c:
9: **if** ¬*recorded*: docket(c) ← ⟨ ⟩, closed (photographs empty); RECORD()
10: **else**: close docket(c) (its contents are c's recorded state)

11: **upon** deliver(m) on channel c, m an application message:
12: process m normally; **if** docket(c) is open: append m to docket(c)

13: locally complete when *recorded* and all dockets closed;
14: the fragments (state, dockets) travel to the collector by ordinary mail.

Checked against Chandy and Lamport (1985), §3 (their Marker-Sending and Marker-Receiving Rules). The word **immediately** on line 4 *is* the algorithm — it is what welds the marker's channel position to the sender's recording instant, and it is the entire content of the FIFO purchase (Rem. 2.11). Problem 2.10 is what happens when an implementer economizes on it.

The audited run. Bertie's ten pounds in flight, so the money may be watched being counted exactly once (Exhibit 2.5):

- **the state of play:** Bingo has debited — ledger £50 — and the tenner is in the Post toward Tuppy (ledger £40);
- **Tuppy initiates:** records £40; marker to Bingo, immediately; docket opened on the incoming channel from Bingo;
- **the tenner arrives first:** Tuppy has already recorded, so it does not touch the recorded forty — he credits his *live* ledger, now fifty, and into the open docket it goes: ten pounds, noted as in flight;
- **the marker reaches Bingo:** his first — records £50; notes the channel from Tuppy as empty; posts a marker back;
- **first-in-first-out collects its salary:** that marker, posted after the tenner on the same route, cannot overtake it — the sweep arrives behind the money, never ahead; the docket closes on exactly one item;
- **the assembly:** forty recorded, fifty recorded, ten in the docket — one hundred pounds, conserved to the penny, while the money was moving, without a pause and without a wall clock.

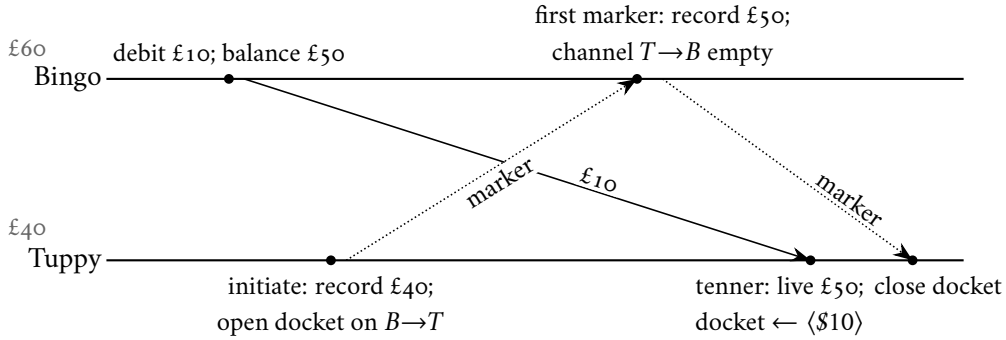


Exhibit 2.5 (the audited bank; Part Seven's walkthrough). Markers are drawn densely dotted — a season convention added this chapter. Tuppy initiates mid-transfer. On the channel Bingo → Tuppy, Bingo's marker is enqueued *behind* the tenner and, by FIFO, may not overtake it — the exact moment the model box's purchase is spent. The assembled photograph: $40 + $50 + $10 in flight = $100, conserved to the penny, without a pause and without a wall clock.

With the full company, the same sentence pronounced louder: markers go out on *every* outgoing channel; each gentleman, on his first marker, records, returns markers, and opens dockets on the remaining incoming channels, closing each as its own marker arrives. Every channel is swept exactly once; every gentleman records exactly once; the audit is complete when every docket has closed. The bill: one marker per channel — order n^2 per photograph for a fully connected company — trifling for three, worth budgeting for three thousand, and one reason the industrial costume (Chapter 10) sends its markers down a pipeline rather than around a clique.

Why it works — always, not merely here — needs one piece of equipment: the "innocent reshuffling" the coda leans on, stated as a lemma. Adjacent concurrent events may be transposed, and nothing any process could testify to changes — the relative order of concurrent events is a coordinate of the description, not of the system. Problem 2.7 has you build it by hand; the proof is given in full.

Lemma 2.3 (commutation of concurrent neighbours)**.** *Let $\alpha = \langle \dots, e, f, \dots \rangle$ be an execution in which e immediately precedes f and $e \parallel f$. Then $\alpha' = \langle \dots, f, e, \dots \rangle$, the same sequence with the pair transposed, is an execution with the same events, the same message pairing, the same per-process local histories, and the same*

happens-before relation; and the global state at every position outside the transposed pair — in particular before e and after the pair — is identical in α and α'.

Proof of Lemma 2.3. Write P and Q for the processes of e and f. First, $P \neq Q$: adjacent events of one process are HB1-ordered, contradicting $e \parallel f$. Second, f is not the receive of a message sent at e: that is HB2, again contradicting $e \parallel f$.

α' is an execution. (a) Per-process subsequences are unchanged — only the interleaving of P's and Q's steps moved — and a deterministic machine's behaviour depends only on its own prior events and the contents of messages it has received, all unchanged. (b) If $f = \textsc{receive}(m_f)$, the matching send precedes f in α and is not e; it therefore still precedes f in α'. (c) Channel discipline. All sends on a given channel are events of one process, and all receives of one channel occur at one process; since $P \neq Q$, the pair e, f cannot contain two sends, nor two receives, of the same channel, so the per-channel order of sends and the per-channel order of receives are both unchanged, and the FIFO pairing (the k-th receive matches the k-th send) is untouched.

Invariants. Events, message pairing, and per-process local histories are identical by construction; hence HB1 and HB2, and with them $\rightarrow$, are identical. For states: each process's state at each of its own events depends only on its local history, unchanged; and at any position of the sequence outside the transposed pair, the set of executed events is the same in α and α', with identical per-process prefixes and identical channel contents (same sends minus same receives, in the same per-channel order). In particular the global states immediately before e and immediately after the pair coincide. Between the two transposed positions the executions differ — which is the point. □

With the lemma in hand, the correctness of the photograph, entire: termination, consistency of the cut, exactness of the dockets, and "a photograph of a possible present" — the reachability guarantee of part (iii).

Theorem 2.4 (the photograph; Chandy–Lamport 1985)**.** *Run Box 6, initiated by any single process, within an execution α over this chapter's model. Then:*

(o) ***Termination.*** *Every process records exactly once; every channel carries exactly one marker; every docket closes.*

(i) ***Consistency.*** *The set K of* pre-recording *events — events at each process before that process records — is a consistent cut.*

(ii) ***Accounting.*** *The recorded process states and dockets are exactly the global state $S(K)$; in particular, for every channel c,* $\mathrm{docket}(c) = L_c(K)$*, the cross-frontier mail.*

(iii) ***A possible present.*** *There is an execution α', obtained from α by finitely many transpositions of Lemma 2.3 — hence with identical local histories at every process — in which $S(K)$ is the actual global state at the moment the last pre-recording event completes. Consequently $S_{\text{init}} \rightsquigarrow S(K) \rightsquigarrow S_{\text{compl}}$, where S_{init} and S_{compl} are the global states at the protocol's initiation and completion.*

Proof of Theorem 2.4. (o) Termination. The initiator sends one marker on every outgoing channel; every process, on its first marker, records and does likewise; the "first marker" guard ensures no process records twice. Channels are reliable and the postal map strongly connected, so by induction on distance from the initiator every process eventually receives a marker and records; consequently every process sends exactly one marker per outgoing channel, every channel carries exactly one marker, and every docket eventually closes.

Setup. For each process q let $\mathrm{rec}(q)$ be the point at which q records. An application event at q is *pre-recording* if it occurs before $\mathrm{rec}(q)$, *post-recording* otherwise; K is the set of pre-recording events. (Markers

and the recording acts are control events, superimposed on the application execution and not counted among its events.) By construction K restricts to a prefix at each process.

Key Lemma (where FIFO is spent). If $\text{SEND}(m)$ is post-recording, then $\text{RECEIVE}(m)$ is post-recording. *Proof.* Let m travel channel $c = (p, q)$. Since p sends m after recording, and Box 6 emits p's marker on c *immediately* upon recording — before any further application send — the marker entered c ahead of m; FIFO keeps it ahead; so the marker reaches q before m does, and its arrival forces q to record if it had not already. Hence q has recorded before m arrives; the lemma is proved.

(i) Consistency. K is a union of per-process prefixes. If $\text{RECEIVE}(m) \in K$ it is pre-recording; by the Key Lemma's contrapositive $\text{SEND}(m)$ is pre-recording, hence in K. By Definition 2.9, K is a consistent cut.

(ii) Accounting. Fix a channel $c = (p, q)$. The marker enters c at $\text{rec}(p)$; by FIFO, the messages arriving at q on c *ahead of the marker* are exactly those sent *before* $\text{rec}(p)$ — the pre-recording sends — and by reliability every one of them does arrive ahead of the marker, hence before the docket closes. The docket collects arrivals after $\text{rec}(q)$ and before c's marker; therefore

$$\text{docket}(c) \;=\; \{\, m \text{ on } c : \text{SEND}(m) \in K,\ \text{RECEIVE}(m) \notin K \,\} \;=\; L_c(K).$$

The channel on which a process's *first* marker arrived is recorded empty, correctly: on that channel every pre-recording send arrived ahead of the marker, i.e., before $\text{rec}(q)$ — such messages are received *in* K and are not cross-frontier mail. Finally, each process records its state at $\text{rec}(q)$, which is its state after executing exactly its K-prefix. The recorded data is therefore precisely $S(K)$.

(iii) A possible present. Events before the first recording are all pre-recording; events after the last recording are all post-recording; the mixture is confined to the window between. While some post-recording event e immediately precedes a pre-recording event f, transpose them. This is licensed: the two are at different processes (each process's pre-recording events all precede its post-recording ones); adjacency admits $e \to f$ only by HB1 (same process — excluded) or HB2 (e a send received at f — excluded, since by the Key Lemma the receive of a post-recording send is post-recording, while f is pre-recording); and $f \to e$ is impossible since causal paths run forward in α. Hence $e \parallel f$ and Lemma 2.3 applies: the result is an execution with the same events, local histories, $\to$, and boundary states. Each transposition strictly reduces the number of ordered pairs (post-recording before pre-recording), which is finite; so the process terminates in an execution α' in which every pre-recording event precedes every post-recording one.

In α', consider the position just after the last pre-recording event. Every process has executed exactly its K-prefix, so its state is its recorded state; every channel holds exactly the messages sent in K and not received in K — that is, $L_c(K)$, the dockets. The global state at that position is therefore $S(K)$, occupied by the legal execution α'. Since α' agrees with α before the window and, statewise, from the window's end onward (Lemma 2.3 preserves states outside each transposed pair; induct over the transpositions), α' passes through S_{init}, then $S(K)$, then S_{compl}:

$$S_{\text{init}} \rightsquigarrow S(K) \rightsquigarrow S_{\text{compl}}.$$

And because local histories in α' are identical to those in α, no process — and no client interrogating any process — could distinguish the world in which the photograph "really happened." □

Remark 2.11 (the FIFO purchase, itemized). The FIFO hypothesis is spent exactly once — in the Key Lemma of the proof (a post-recording send is never received pre-recording, because the sender's marker entered the channel first and cannot be overtaken) — and nowhere else. Problem 2.12 buys the same theorem in a different currency: a one-bit colour smeared onto every letter.

What a photograph is for. The classic answer — the one Chandy and Lamport built the paper around — is the detection of stable properties: conditions which, once true, stay true. Here the plurality of presents turns from liability into licence: the photograph depicts a state the system could genuinely have occupied, situated between the audit's beginning and its end — so a stable truth read off it holds now and ever after, and a stable falsehood had not yet occurred when the audit began.

Corollary 2.5 (stable properties; the licensed uses of a photograph)**.** *Call a predicate φ on global states* stable *if $\varphi(S)$ and $S \rightsquigarrow S'$ imply $\varphi(S')$. Then $\varphi(S(K))$ implies φ holds at completion and ever after; and $\neg\varphi(S(K))$ implies φ did not hold at initiation. Deadlock, termination of a computation, loss of a token, and permanent breakage of an invariant are stable. "The queue is exactly seven deep" is not, and about such properties the photograph testifies to nothing.*

Proof of Corollary 2.5. By Theorem 2.4(iii), $S_{\text{init}} \rightsquigarrow S(K) \rightsquigarrow S_{\text{compl}}$. If $\varphi(S(K))$, stability propagates φ along $S(K) \rightsquigarrow S_{\text{compl}}$ and every continuation beyond. If $\varphi(S_{\text{init}})$, stability gives $\varphi(S(K))$; contrapositively, $\neg\varphi(S(K))$ implies $\neg\varphi(S_{\text{init}})$. □

Practical notes. Recording is entirely local — each gentleman ends the protocol holding his own ledger copy and dockets — so the fragments are gathered by ordinary post, as leisurely as one pleases: recorded fragments do not age. Nor need there be one photographer: markers carry an audit identifier, dockets are kept per audit, and independent runs interleave without quarrel. Throughout, the letters keep flying; the pause that could not be afforded was a pause nobody needed.

The plurality of presents. The chapter's designated moment of wonder, standing on the mathematics above. The assembled state — Tuppy forty, Bingo fifty, ten in the Post — obtained at no actual moment of the true history: the photograph depicts an instant that never happened, and it is nonetheless a *legitimate* instant, by part (iii). Herd every pre-recording event before every post-recording one, one innocent transposition of Lemma 2.3 at a time — the cut's consistency is exactly what makes the herding possible — and the recorded state is the state between the herds, reachable from the initiation, the true completion reachable from it, no process or client able to testify against it. There is no master clock — that is the whole chapter — so the system's history was never one sequence of instants but a lattice of roads, and *every* consistent cut through that lattice is as much a "now" as any other. The algorithm did not fake a present; it selected one, legitimately, from the plurality the mathematics actually offers. The season permits itself one admiration per chapter, and this chapter's is spent here: a small piece of philosophy that compiles. One receipt before the close, so the ledger stays honest — this exact algorithm runs, under an assumed name and industrial tailoring, inside stream-processing machinery sir personally operates, its markers travelling the pipelines at this hour. The unmasking is scheduled, with full ceremony, for Chapter 10.

2.8 The Whiteboard

For the design review.

1. Never order cross-node events by wall clock: a cross-machine timestamp comparison is an opinion wearing digits, and at message granularity not a well-informed one. If a design sorts, joins, or adjudicates by cross-machine timestamps — last-write-wins by clock, log-merge forensics, expiry decided by somebody else's watch — demand the uncertainty bound in writing, or call it folklore with a schema (Chapter 8 buys the bound, and the price will clarify why nobody gets it for free).

2. Two clocks, two jobs, never crossed: durations, timeouts, and intervals belong to the monotonic clock; naming moments for humans belongs to the time-of-day clock. Any duration computed by subtracting wall-clock readings is a latent incident with a date of its own choosing.

3. If concurrent writes can happen, choose the regime on purpose: carry causality explicitly (vectors, version stamps) so that rivalry is detectable and adjudicable, or rent a total order from a leader so that rivalry is impossible by construction (Chapter 4). Either is defensible; drifting between the two, the industry's default posture, is precisely where the lost updates live.

4. A snapshot is not a pause: any design that requires stopping the world for a consistent view is paying for a theorem nobody proved; markers deliver consistency in motion. Corollary for the estate you already operate: interrogate the checkpoint and backup tooling about what its "consistent" actually means — a consistent cut, or a superstition with a cron schedule.

The register. Paid, with a small ceremony — the season's first debt to move from the left column to the right: debt #3, *no shared clock* (issued Ep. 1), and note the currency — not a synchronization protocol, which would have been paying a falsehood with a gadget, but a replacement theory of order, under which the clock's absence stopped being a defect and became a fact with structure. Issued, four fresh entries, each due later in the season: physical clocks return, armed and expensive — a company purchasing better physics and tendering the apology in hardware (Ch. 8); the photograph returns in industrial costume, its markers in the pipelines (Ch. 10); a total order may be rented from a leader — the throne, and the defence of the throne (Ch. 4); and causal delivery, glimpsed through a doorway, becomes a purchasable contract with a price tag attached (Ch. 7).

2.9 Problems

All computations refer to the reference trace, Exhibit 2.1, unless a problem states otherwise.

Tier I — Calisthenics

Problem 2.1 (ordering drill)**.** Classify each pair as $\rightarrow$, $\leftarrow$, or $\|$, exhibiting the causal path where one exists: (b_1, t_2), (g_1, b_2), (t_2, g_3), (b_3, g_3), (t_1, g_1), (b_4, g_2).

Problem 2.2 (Lamport stamps and their inversions)**.** Compute $C(e)$ for all eleven events by Box 4. Then list every pair involving g_1 whose stamps are ordered while the events are concurrent, and every pair involving g_1 whose stamps tie. State the one inference that $C(a) < C(b)$ does license.

Problem 2.3 (vector clocks)**.** Compute $V(e)$ for all eleven events by Box 5, coordinates (Bingo, Tuppy, Gussie). Verify Theorem 2.2 on the pairs (b_2, g_2), (g_1, t_2), and (t_2, b_4).

Problem 2.4 (the merge drill)**.** Let $u = (3, 1, 4)$, $v = (2, 5, 0)$, $w = (3, 5, 4)$. (a) Compute the entrywise suprema $\sup(u, v)$, $\sup(u, w)$, $\sup(v, w)$. (b) Which pairs are comparable under the vector order? (c) Suppose a, b, c are distinct events with $V(c) = \sup(V(a), V(b))$. Show that $a \rightarrow c$ and $b \rightarrow c$. (You will want: the map $e \mapsto V(e)$ is injective on events — prove it from the ledger lemma.)

Problem 2.5 (frontiers, classified)**.** Four cuts of the reference trace, given by per-process prefixes: $X = (\leq b_2,\ \leq t_1,\ \leq g_1)$; $Y = (\leq b_2,\ \leq t_3,\ \leq g_1)$; $Z = (\leq b_1,\ \leq t_1,\ \leq g_1)$; $W = (\leq b_4,\ \leq t_1,\ \leq g_3)$. Which are consistent? For each consistent cut, list every channel's cross-frontier mail; for each inconsistent one, name the offending letter.

Problem 2.6 (the auditor's telephone)**.** The bank of Part Seven: Bingo holds £60, Tuppy £40; one transfer of £10 is executed (debit d at Bingo, letter m, credit r at Tuppy). A naive auditor telephones each gentleman once and sums the two spoken balances. (a) Enumerate the four cases (call to Bingo before or after d) $\times$ (call to Tuppy before or after r) and give each audit's total. (b) Treating the frontier "just after each call" as a cut, say for each bad total whether the sin is an *inconsistent cut* or a *consistent cut with an ignored channel*. (c) Conclude which sin the markers of Box 6 cure and which the dockets cure.

Tier II — The Real Thing

Problem 2.7 (the swap lemma, built by hand)**.** Prove Lemma 2.3 directly from Definitions 2.1–2.3: (a) show the two events occur at different processes and are not a matched send–receive pair; (b) show the transposed sequence is an execution (programs, message availability, channel discipline); (c) show events, pairing, local histories, and $\rightarrow$ coincide; (d) show the global state at every position outside the pair coincides. This is the engine inside Theorem 2.4(iii); the coda's "innocent reshuffling" is exactly this lemma, iterated.

Problem 2.8 (serialization freedom)**.** (a) Show that *every* linear extension β of $\rightarrow$ on the events of an execution α is itself an execution. (b) Show β is reachable from α by finitely many transpositions of Lemma 2.3. (c) Conclude that for $e \parallel f$ there are executions, indistinguishable to every process from α, in which e precedes f and in which f precedes e — the relative order of concurrent events is a coordinate of the description, not of the system.

Problem 2.9 (the lattice of frontiers)**.** (a) Prove the parenthetical of Definition 2.9: a cut is consistent iff it is closed under $\rightarrow$ (in particular, HB3 adds no new obligation). (b) Show consistent cuts are closed under union and intersection. (c) Show $\downarrow e = \{f : f \rightarrow e\} \cup \{e\}$ is the least consistent cut containing e, and compute $\downarrow g_2$ on the reference trace. (d) Conclude from (b) that the consistent cuts form a *lattice* under $\subseteq$, with $\cup$ the least upper bound and $\cap$ the greatest lower bound — this volume's one structure carrying both, the states of Chapter 7 (Definition 7.13) carrying joins alone.

Problem 2.10 (sabotage: the thrifty auditor)**.** A well-meaning implementer economizes on Box 6: *"upon recording, a process first dispatches any application messages already queued in its outgoing tray, and only then emits its markers — the markers ride at the back of the tray, to save a round of envelopes."* All else is unchanged. (a) Exhibit an execution of the two-gentleman bank in which the audit reports £110 while the true total is £100 throughout. (b) Identify precisely which statement in the proof of Theorem 2.4 dies — and which, perhaps surprisingly, survives. (c) State the minimal repair, and point to the exact word of Box 6 that constitutes it.

Tier III — For the Ambitious (starred)

Problem 2.11 ($\star$ the incompressible roster)**.** Suppose some scheme assigns to every event of every n-process execution a stamp $W(e) \in \mathbb{R}^k$ such that, for all distinct a, b: $a \rightarrow b \iff W(a) < W(b)$ (entrywise $\leq$, strict somewhere). Show $k \geq n$. Guided route: (a) show such a W yields k linear extensions of $\rightarrow$ whose intersection is exactly $\rightarrow$ — so the *order dimension* of $\rightarrow$ is at most k; (b) consult Charron-Bost (1991) for an n-process computation whose causality order has dimension exactly n, and conclude. Moral: the vector's width is not an implementation's clumsiness but the problem's true dimension; the industry *rations* the roster (versions per key, retired members pruned) — it does not compress it.

Problem 2.12 ($\star$ photographs without first-in-first-out)**.** Channels are reliable but may reorder. Design a snapshot in the style of Lai and Yang (1987): every application message carries one bit — *white* if its sender had not yet recorded, *red* otherwise. (a) Specify the protocol: when does a process record, what do the

dockets collect, and how does a docket know it may close? (b) Prove the pre-recording cut is consistent. (c) Say exactly which two services FIFO was providing in Box 6, and what purchases replace them here.

2.10 Hints

Hint (Problem 2.9)**.** For (a): a causal path's final step into a cut-member is HB1 or HB2, and each of those is already closed by the two stated clauses; induct backwards along the path. For (c): $\downarrow e$ is a prefix at each process by transitivity, and receive-closed because HB2 $\subseteq \rightarrow$.

Hint (Problem 2.10)**.** Follow the second tenner. Arrange for the marker to reach Bingo while Bertie's next order sits in the outgoing tray.

Hint (Problem 2.11)**.** For (a): tie-break every coordinate by one fixed linear extension of $\rightarrow$ itself; check that ordered pairs agree in all k linearizations and concurrent pairs disagree between some two. For (b): the crown poset is the shape to look for.

Hint (Problem 2.12)**.** The colour answers the only question the marker ever answered: was this letter posted before or after its sender's photograph? What the colour cannot do is announce that the *last* white letter has arrived — FIFO's second, quieter service. Sell that separately: a per-channel count, sent once, at recording time.

2.11 Solutions

Solution (to Problem 2.1)**.** $b_1 \rightarrow t_2$ (path b_1, b_2, m_1, t_1, t_2). $g_1 \parallel b_2$ (no path either way). $t_2 \rightarrow g_3$ (path t_2, t_3, m_2, g_2, g_3). $b_3 \leftarrow g_3$, i.e. $g_3 \rightarrow b_3$ (message m_3). $t_1 \parallel g_1$. $b_4 \leftarrow g_2$, i.e. $g_2 \rightarrow b_4$ (path g_2, g_3, m_3, b_3, b_4).

Solution (to Problem 2.2)**.** C: b_1=1, b_2=2, t_1=3, t_2=4, t_3=5, g_1=1, g_2=6, g_3=7, b_3=8, b_4=9, g_4=10. Ordered-but-concurrent pairs involving g_1: (g_1, b_2), (g_1, t_1), (g_1, t_2), (g_1, t_3) — stamps $1 < 2, 3, 4, 5$, all four pairs concurrent. Tie: (b_1, g_1), both stamped 1, concurrent. The sole licensed inference from $C(a) < C(b)$: $\neg(b \rightarrow a)$.

Solution (to Problem 2.3)**.** V: b_1=(1, 0, 0), b_2=(2, 0, 0), t_1=(2, 1, 0), t_2=(2, 2, 0), t_3=(2, 3, 0), g_1=(0, 0, 1), g_2=(2, 3, 2), g_3=(2, 3, 3), b_3=(3, 3, 3), b_4=(4, 3, 3), g_4=(4, 3, 4). Checks: $V(b_2) = (2, 0, 0) < (2, 3, 2) = V(g_2)$ and indeed $b_2 \rightarrow g_2$ via m_1, t_3, m_2. $V(g_1) = (0, 0, 1)$ and $V(t_2) = (2, 2, 0)$ are incomparable; indeed $g_1 \parallel t_2$. $V(t_2) = (2, 2, 0) < (4, 3, 3) = V(b_4)$; indeed $t_2 \rightarrow b_4$ via $t_3, m_2, g_2, g_3, m_3, b_3$.

Solution (to Problem 2.4)**.** (a) All three suprema equal $(3, 5, 4)$. (b) $u < w$ and $v < w$; $u \parallel v$ (first coordinate favours u, second favours v). (c) Injectivity: if $V(e) = V(f)$ with e at p, the ledger lemma gives $V(f)[p] = V(e)[p] = |K_p(e)| \geq$ the index of e, so $e \in K_p(f)$: $e \rightarrow f$ or $e = f$; symmetrically $f \rightarrow e$ or $f = e$; a two-way $\rightarrow$ closes a cycle, so $e = f$. Now $V(a) \leq \sup(V(a), V(b)) = V(c)$ and $a \neq c$, so by injectivity $V(a) < V(c)$, whence $a \rightarrow c$ by Theorem 2.2; likewise $b \rightarrow c$.

Solution (to Problem 2.5)**.** X: consistent; every channel empty (m_1's send and receive both inside; nothing else sent). Y: consistent; cross-frontier mail $\{m_2\}$ on Tuppy $\rightarrow$ Gussie, all other channels empty. Z: inconsistent; offending letter m_1 (t_1 inside, b_2 outside). W: inconsistent; offending letter m_2 (g_2 inside via $\leq g_3$, while t_3 lies outside $\leq t_1$). Note m_3 is blameless in W: send g_3 and receive b_3 are both inside.

Solution (to Problem 2.6)**.** (a) Before d / before r: 60 + 40 = 100. Before d / after r: 60 + 50 = 110. After d / before r: 50 + 40 = 90. After d / after r: 50 + 50 = 100. (b) The 110 case is an *inconsistent cut*: r inside, d

outside — the credit without its debit; £10 minted. The 90 case is a *consistent* cut whose cross-frontier mail (m, in flight) the telephone audit simply ignores; £10 burned. (c) Markers enforce consistency — no minting; dockets record the crossing mail — no burning. The naive audit commits whichever sin the interleaving selects; Box 6 commits neither.

Solution (to Problem 2.7)**.** (a) Adjacent events of one process are HB1-ordered and a matched send–receive pair is HB2-ordered; either contradicts $e \parallel f$. (b) Per-process subsequences are unchanged, and a deterministic machine's next step depends only on its own history and the contents of delivered messages, both unchanged; if $f = \textsc{receive}(m_f)$, its send precedes f in α and is not e, hence still precedes f in α'; all sends of one channel issue from one process and all receives of one channel land at one process, so with $P \neq Q$ the pair contains at most one send and at most one receive of any given channel — per-channel send order and receive order are unchanged, and the FIFO pairing (the k-th receive matches the k-th send) is untouched. (c) Events, pairing, and per-process orders are unchanged, hence HB1, HB2, and their transitive closure $\rightarrow$ are unchanged; local histories likewise. (d) At any position outside the pair, the executed multiset of events and every per-process prefix coincide, so process states coincide; channel contents are the same sends minus the same receives in the same per-channel order.

Solution (to Problem 2.8)**.** (a) β extends $\rightarrow$, which contains per-process order (program order preserved) and every send-to-receive pair (availability preserved); per-channel receive order equals the receiving process's program order, preserved; the pairing is unchanged, so channel discipline holds; by determinism each process, seeing an identical local history, performs identical events. (b) Induct on the number of pairs ordered oppositely by α and β. If nonzero, some *adjacent* pair e, f of α is ordered oppositely in β; then $\neg(e \rightarrow f)$ (else β would violate $\rightarrow$) and $\neg(f \rightarrow e)$ (else α would); so $e \parallel f$, and Lemma 2.3 transposes them, reducing the count by one while preserving local histories. (c) Incomparable elements of a partial order admit linear extensions with either relative order; apply (a) and (b) to each. Since local histories are identical throughout, no process — and no client of any process — can testify to the relative order of concurrent events. It is a coordinate of the description, not of the system.

Solution (to Problem 2.9)**.** (Sketch; the hint carries the weight.) (a) Closure under HB1 is the prefix property; under HB2, the stated clause; a causal path into a member enters by one of these steps, so induction along the path gives full $\rightarrow$-closure — HB3 adds nothing. (b) Both clauses are preserved by $\cup$ and $\cap$. (c) $\downarrow e$ is a prefix at each process (transitivity) and receive-closed (HB2 $\subseteq\!\rightarrow$), hence consistent; any consistent $K \ni e$ is $\rightarrow$-closed by (a), hence contains $\downarrow e$. On the trace: $\downarrow g_2 = \{b_1, b_2, t_1, t_2, t_3, g_1, g_2\}$. (d) Immediate from (b): under $\subseteq$, the union of two consistent cuts is a consistent cut above both and contained in every such, and dually for the intersection.

Solution (to Problem 2.10)**.** (a) The minting run, Exhibit 2.6. Initially Bingo £60, Tuppy £40. (1) Bingo debits £10 (live £50) and posts the first tenner. (2) Tuppy initiates: records £40; sends his marker; opens the docket on Bingo $\rightarrow$ Tuppy. (3) The first tenner arrives: Tuppy's live balance £50; docket $\langle \$10 \rangle$. (4) Tuppy's marker reaches Bingo, who records £50 — but the thrifty rule first dispatches Bertie's queued order: Bingo debits £10 (live £40) and posts the *second* tenner, and only then his marker. By FIFO the second tenner travels *ahead* of the marker. (5) It lands at Tuppy in the still-open docket: $\langle \$10, \$10 \rangle$; live £60. (6) The marker arrives; the docket closes. The audit assembles $40 + 50 + 20 = \$110$. The live total was £100 at every instant. Ten pounds minted.

(b) What dies is the *accounting*, part (ii): Box 6's marker enters the channel at the instant of recording, which is what makes "ahead of the marker" synonymous with "sent pre-recording"; the thrifty variant enqueues the marker late, the synonymy breaks, and the docket over-collects — it admits the second tenner, a *post-recording* send, so $\text{docket}(c) \supsetneq L_c(K)$. The £10 that letter carries is still inside Bingo's recorded £50;

the photograph counts it twice. What survives, perhaps surprisingly, is the Key Lemma and with it consistency, part (i): the second tenner is sent post-recording *and* received post-recording (Tuppy recorded long before), so the cut K itself is unharmed. The sabotage kills the bookkeeping, not the frontier — which is precisely why every individual record looks locally honest while the ledger lies.

(c) Restore the word **immediately** (Box 6, line 4): between recording and marker emission, no application sends whatsoever. The word is the algorithm.

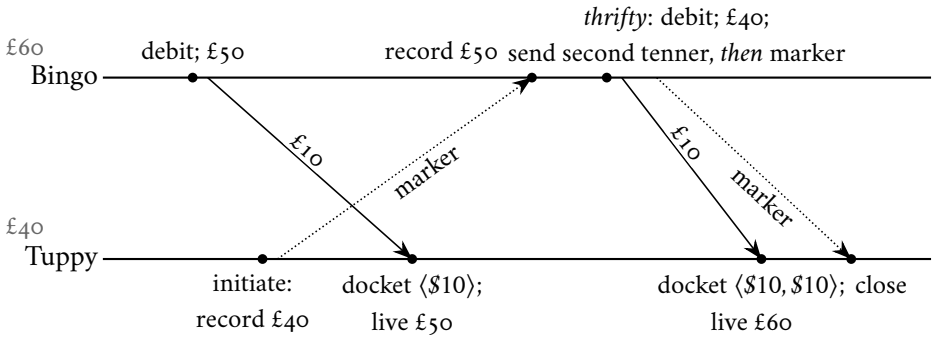


Exhibit 2.6 (the thrifty auditor's minting run; Solution 2.10). The second tenner — a post-recording send — rides ahead of the belated marker and lands in the open docket. Audit: $40 + 50 + 20 = \$110$ against a true £100. Fatality certified.

Solution (to Problem 2.11)**.** (Pointer.) (a) Fix one linear extension σ of $\rightarrow$ and, for each coordinate i, linearize events by $W(\cdot)[i]$ with ties broken by σ. If $a \rightarrow b$ then every coordinate has $W(a)[i] \leq W(b)[i]$ and σ agrees, so a precedes b in all k orders; if $a \parallel b$ then $W(a)$ and $W(b)$ disagree strictly in both directions, so some order puts a first and another b first. The intersection of the k linear orders is exactly $\rightarrow$: its order dimension is at most k. (b) Charron-Bost (1991) exhibits, for every n, an n-process computation whose causality order has dimension exactly n; hence $k \geq n$.

Solution (to Problem 2.12)**.** (Pointer; the hint is most of it.) Record before processing the first red letter: then any letter processed while white had a white sender — the pre-recording cut is receive-closed, hence consistent, and the Key Lemma has been repurchased with a bit instead of an ordering. Dockets collect white letters arriving after recording; each sender, upon recording, posts on every channel a count of the whites it sent there, and the docket closes when the count is met. FIFO's two services — separating pre from post, and announcing the last of the pre — are bought separately: a colour and an integer. Lai and Yang (1987).

2.12 The Reading

Lamport, "Time, Clocks, and the Ordering of Events in a Distributed System," Communications of the ACM, 1978. This chapter's text, and short. When the field established a prize for its most influential work, in the year 2000, this paper was the inaugural selection; the honour has since taken Dijkstra's name. Read it whole; the happens-before relation and the clock condition are its first half, the mutual-exclusion protocol its second — and when you reach the small paragraph about state machines, pause and smile: you are holding an acorn, and Chapter 5 is the oak.

Chandy and Lamport, "Distributed Snapshots: Determining Global States of Distributed Systems," ACM Transactions on Computer Systems, 1985. Brief, complete, and quietly perfect. As you read, mark the exact sentence where first-in-first-out is spent, and compare it with our Key Lemma; notice it is spent nowhere else. Their stable-property applications are Corollary 2.5 in the original tailoring.

The vector clock's two births: Fidge (1988); Mattern, "Virtual Time and Global States of Distributed Systems" (1989). Optional. The same instrument, discovered independently on opposite sides of the planet — presumably concurrently. Read either for the characterization theorem in its discoverer's own notation.

For the starred doors. Charron-Bost, "Concerning the size of logical clocks in distributed systems," Information Processing Letters, 1991 (Problem 2.11). Lai and Yang, "On distributed snapshots," Information Processing Letters, 1987 (Problem 2.12).

Standing companion: Kleppmann, chapter 8. His pages on unreliable clocks pair with Part One; his treatment of happens-before and concurrency detection, with Parts Three through Five. Assigned once for the season; here it earns its keep.

Chapter 3

What Mathematics Forbids

Companion to Episode Three. The volume's flagship, and the proof this volume was built to hold. The episode tells the story — the death certificate of 1985, the decade of moving corners, the tablebase adversary at its patient work; this chapter keeps the record: the consensus problem in three clauses; the scheduler's formal commission; the valence apparatus and the Fischer–Lynch–Paterson impossibility written out whole, to the last inequality; the fine print and the three gates — partial synchrony, randomization, failure detectors — through which every working system escapes; the CAP theorem in Gilbert and Lynch's dress, disinfected of folklore; and the season's standing triage — impossible, expensive, merely inadvisable. Crashed-versus-slow, filed in Chapter 1 as the primordial nuisance, is promoted here to murder weapon. Statements, proofs, gates, problems.

3.1 Part One — A Modest Ask

In April of 1985 the Journal of the ACM printed Fischer, Lynch, and Paterson's "Impossibility of Distributed Consensus with One Faulty Process" — a death certificate with the cause of death in the title; nine pages, the fatal argument three, circulated since 1983 and first presented at a symposium on database systems — which tells you who was bleeding. It ended a decade of agreement protocols whose corners — arrangements of delays and silences under which each protocol hung forever or inked two verdicts — were patched, moved, and never closed: the corner is not in the protocol; the corner is in the problem.

Why one bit is the centre. Chapter 1's umbrella counter needs an authority appointed by parts that fail, in a way that survives the appointee's death; Chapter 2's total order must be rented from a leader, and an election is an agreement about who won. One bit, agreed upon by machines that fail, is the atom of coordination: a sequence of settled bits is a log, and a log driven through identical machines is, per Lamport 1978, a replicated anything. The theorem says the atom cannot be split — and mark its peculiar cruelty from the start: no letter is lost, none forged, none duplicated; the theorem kills with delay alone, wielded with perfect judgment. Loss defeated two generals; mere unpunctuality defeats any number of them.

The problem, plainly. Some number of gentlemen — our three — must jointly make a single yes-or-no decision. Each arrives with a private ballot, nought or one, formed from local evidence; they correspond through the Post as much as they please; at some point each writes his decision in his ledger, in ink — once written, never revised. Success is three clauses:

- **Agreement:** no two gentlemen ink different decisions — not "rarely differ"; never, in any execution, under any weather.

- **Validity:** the decision is some gentleman's actual original opinion — in particular a unanimous ballot prevails. The clause bars universal cowardice: always inking nought satisfies Agreement magnificently and decides nothing.
- **Termination:** every gentleman who remains alive eventually writes. The meeting ends — the clause to watch, the way one watches the quietest man in the room.

And one condition over the whole exercise: the three guarantees must be delivered even if *one* gentleman fails — crash-stop, in Chapter 1's vocabulary: he halts silently, at any moment of the adversary's choosing, and never returns. One chair, possibly empty, and nobody told whose or when or whether.

The possibly-empty chair. It is the entire difficulty. Without it the protocol writes itself: every gentleman posts his ballot to every other, waits until all ballots are in hand, applies the same fixed rule — every letter arrives eventually, and all three clauses follow trivially. With it, the committee is caught between waiting for a colleague who may never speak and proceeding without a colleague who may merely be clearing his throat: wait for all ballots and Termination dies with the emptied chair; act on a partial count and two gentlemen may act on two *different* partial counts, with Agreement in the gravest peril. Crashed-versus-slow, promoted from operational nuisance to the exact hinge on which the possibility of agreement turns. A single bit; it is difficult to ask less — which is why the impossibility is a monument, not a curiosity.

The problem, formally. The ballots are the inputs x_i; the ink is a write-once register.

Definition 3.1 (the consensus problem)**.** Each process p_i holds an input $x_i \in \{0, 1\}$ and has a write-once output register y_i, initially blank. A process *decides* v by writing $y_i \leftarrow v$; written ink is never revised. A protocol *solves consensus tolerating one fault* if in every admissible run (Def. 3.4):

- *(Agreement)* no two processes decide different values;
- *(Validity)* the decided value is some process's input; in particular a unanimous input is the only permissible decision;
- *(Termination)* every process that does not crash eventually decides.

FLP prove their theorem under a hypothesis weaker than Validity — *non-triviality*: both 0-deciding and 1-deciding runs exist — which makes the impossibility stronger, since any protocol satisfying Validity satisfies non-triviality. We state Validity because the course does; every proof below uses it only through non-triviality except Lemma 3.2, where the difference is cosmetic (Problem 3.2).

Remark 3.2 (what Validity does not ask; coordination in the pure)*.* Validity is the whole of the constraint on *which* value may be inked, and it is a strikingly thin one: it asks that the decision be *somebody's* input, never that it be the right input, the prudent input, or an input any process finds tolerable. The vernacular picture of "reaching agreement" — a negotiation among parties who hold private utilities and ratify a proposal only where it falls favourably against them — adds exactly such a constraint, and in the adding becomes a different and strictly harder problem, one this course does not treat. The omission is deliberate and load-bearing. Strip the value of all merit and what remains to be arranged is *coordination in the pure* — the bare securing of one common answer against crash and asynchrony — and it is coordination in the pure that Theorem 3.4 proves unattainable by guarantee. Restore the preferences and every result below only hardens: a protocol that cannot promise agreement on an arbitrary bit certainly cannot promise it on the acceptable ones. The bit is stripped naked so that its impossibility may be laid to coordination and to nothing else.

3.2 Part Two — Opposing Counsel

Every impossibility proof in this course is a game; Chapter 1 called the opposing player the Post's malice and promised it a formal commission. Here it is: the *scheduler* — not weather but opposing counsel of formidable diligence, who has read the protocol entire before the game begins. A protocol is a published rulebook, and a guarantee must hold against every arrangement of delays — that is, against the cleverest one; assuming the adversary reads the playbook is not pessimism but the definition of a guarantee.

The powers, and the leash.

- **The arsenal:** timing, and timing alone. When letters sit undelivered in the Post's bag, it chooses which lands next; when gentlemen are ready to step, who moves and who stands idle, and for how long. No bound to respect, no clock to answer to.
- **The leash:** *fairness*, with two straps — every letter posted to a living gentleman is eventually delivered, and every living gentleman is eventually given his next turn, again and again, without end. Gussie may be benched indefinitely; he must always be let back in.
- **The dark privilege:** at most once, retire a gentleman permanently — the crash — and never file notice.
- **The reform:** no letter lost, none forged, none duplicated. The Post has been reformed, sir — punctuality itself, minus the punctuality.

The powers amount to something familiar: for any finite stretch the scheduler can make a healthy gentleman indistinguishable from a corpse — letters withheld, turns denied — then produce him alive, bookwork intact, at the moment maximally inconvenient. Crashed-versus-slow is its entire craft, promoted below from operational nuisance to the murder weapon of a mathematical argument — Chapter 1's debt, paid in full.

Victory conditions, asymmetric. Our side's play is fixed the moment the rulebook is published; every variation in the game's unfolding is the scheduler's. We must win every fair play, up to one crash, all three clauses holding; counsel needs one fair play in which some clause fails. Industry's "works" means no one has yet been paid to find the line of play; mathematics' "works" means counsel defeated in advance, for all lines, forever — the distance between testing and proof. And the fairness leash is what makes the theorem deep: an adversary allowed to suppress letters forever is merely Chapter 1's Post, settled by two generals. Fischer, Lynch, and Paterson conceded the network everything the engineer could dream of demanding — every letter delivered, every process scheduled, forever — and showed the concession does not help; when a theorem survives every concession, the trouble was never where you were looking.

The commission, notarized — both of the chapter's terrains fixed before any formal claim is made, as Chapter 1's discipline demands.

Model of this chapter

For the impossibility (Defs. 3.1–3.5, Theorem 3.4) — the FLP model, checked against the 1985 paper. Processes $p_1, \ldots, p_n$ ($n \geq 2$) are deterministic automata communicating by messages. The *message buffer* is a multiset of (addressee, message) pairs, with two operations: $\mathsf{send}(p, m)$ adds a pair; $\mathsf{receive}(p)$ removes and returns some message addressed to p, *or returns the special value* $\varnothing$ (null) — the choice is the adversary's. A *step* of p: one receive, then a state change and a finite set of sends, both determined by p's state and the value received. Timing: fully asynchronous — no

bounds on delay or relative speed; the adversary orders all steps and deliveries. **The reformed Post:** within admissible runs (Def. 3.4) no message is lost, none duplicated, none forged; channels need not preserve order. Failures: crash-stop, *at most one* process in any run. This chapter's demonstrations use $n = 3$ (the company); the theorem holds for all $n \geq 2$.

For one bad afternoon (Theorem 3.5) — the Gilbert–Lynch setting. An asynchronous network of at least two nodes implementing a single read/write register (Def. 3.9); clients issue reads and writes to any node. A *partition* splits the nodes into two or more non-empty components; every message between components is lost while it lasts. Nodes do not fail; the network does. Consistency means linearizability (Def. 3.9, by promissory note — the full treatment is Chapter 7's); availability means every request received by any node eventually receives a response.

The vocabulary of play, exactly as the 1985 paper keeps it: a position of the game — everyone's books, plus the bag — is a *configuration*; the bag is the *message buffer*; a move, a turn, is an *event* applied to a configuration.

Definition 3.3 (configurations, events, schedules)**.** A *configuration* C comprises the local state of every process together with the message buffer. An *event* $e = (p, m)$ — the delivery to p of message m addressed to p, or of $m = \varnothing$ — is *applicable* to C iff $m = \varnothing$ or m lies in C's buffer addressed to p; applying it yields the configuration $e(C)$ in which p has taken the corresponding step. A *schedule* σ is a finite or infinite sequence of events, applied left to right; a finite σ applicable to C yields $\sigma(C)$, and every configuration so obtained is *reachable* from C. Note the standing fact (used silently throughout): an event applicable to C remains applicable at every configuration reachable from C by a schedule not containing it — undelivered mail stays in the bag.

A full unfolding of the game is a run; a fair play — the leash honoured — is an *admissible* one, and the dark privilege appears as the single permitted fault.

Definition 3.4 (runs; admissibility; deciding runs)**.** A *run* from C is a (finite or infinite) schedule applicable to C. A process is *faulty* in an infinite run if it takes only finitely many steps, *correct* otherwise. A run is *admissible* if at most one process is faulty and every message addressed to a correct process is eventually delivered. A run is *deciding* if some process decides in it. Total correctness: every admissible run of the protocol is deciding — this is the form of Termination the adversary attacks.

3.3 Part Three — The Proof on the Fence

The set piece, in four movements: a vocabulary, an opening, a small geometric lemma, and the heart. The proof's single greatest idea is a way of seeing. Freeze any position of the game and ask about its futures: if only nought-endings lie ahead, the position is committed to nought — 0-*valent*; if only one-endings, 1-*valent*; if the scheduler still holds the power to steer the game to either verdict, *bivalent* — on the fence. Valence is a fact about the position, not about anybody's knowledge of it — a chess position can be lost beyond rescue while both players believe the game open; valence is the tablebase view, and counsel is *granted* the tablebase: unlimited analysis is fair play in an impossibility argument, where even the strongest legal adversary must be beaten.

Definition 3.5 (valence)**.** For a configuration C reachable in a given protocol, let $V(C) \subseteq \{0, 1\}$ be the set of values decided in configurations reachable from C. C is *bivalent* if $V(C) = \{0, 1\}$, v-*valent* (committed to v) if $V(C) = \{v\}$. In a totally correct protocol $V(C) \neq \emptyset$ (some admissible continuation decides), so every configuration is exactly one of the three.

The bolt that fastens the whole proof: ink implies commitment — so a position on the fence is a position where no pen has yet touched any page.

Remark 3.6 (ink implies commitment; the fence has clean books)*.* In a protocol satisfying Agreement, any configuration in which some process has decided v is v-valent: a reachable decision of $1 - v$ would put two inks in one run. Contrapositively, *a bivalent configuration contains no decision anywhere* — the adversary that preserves bivalence forever thereby prevents every decision forever. Indecision is not a failure to reach the goal; indecision is the goal.

The adversary's campaign has exactly two obligations: begin on the fence, and never leave it. One tool first — Chapter 2's swap lemma in consensus dress: two letters addressed to different desks may land in either order and the game does not notice. The transposition of merely-elsewhere events, a philosophical instrument there, is here a weapon — the *diamond*, after the shape of its picture (Exhibit 3.2 below).

Lemma 3.1 (commutation; disjoint schedules)**.** *Let σ_1, σ_2 be finite schedules applicable to C such that no process takes a step in both. Then σ_2 is applicable to $\sigma_1(C)$, σ_1 to $\sigma_2(C)$, and*

$$\sigma_2(\sigma_1(C)) = \sigma_1(\sigma_2(C)).$$

Proof of Lemma 3.1. Induct on $|\sigma_1| + |\sigma_2|$; the essential case is two single events $e_1 = (p_1, m_1)$, $e_2 = (p_2, m_2)$ with $p_1 \neq p_2$, both applicable to C. Applicability is preserved: e_1 neither consumes m_2 (messages are consumed only by their addressee, and m_2 is addressed to $p_2 \neq p_1$) nor changes p_2's state; so e_2 is applicable to $e_1(C)$, and symmetrically. In $e_2(e_1(C))$ and $e_1(e_2(C))$: process p_1 has taken the same step from the same local state with the same delivered value in both, likewise p_2, and no other process moved; local states agree. The buffer in both equals C's buffer minus $\{m_1, m_2\}$ plus the sends of the two steps — and each step's sends are determined by its process's prior state and delivered value, identical in both orders. The two configurations are equal. For the induction step, splice one event at a time across the other schedule using the essential case. □

The opening. Might the protocol commit from the first whistle, its verdict a foregone function of the ballots? Lemma 3.2 kills the hope — skeleton: the paper-doll chain of adjacent initial configurations, plus one silenced juror — and the doppelgänger performs its first murder of the chapter doing so.

Lemma 3.2 (initial bivalence)**.** *Every protocol solving consensus tolerating one fault has a bivalent initial configuration.*

Proof of Lemma 3.2. Suppose instead that every initial configuration is univalent. By Validity, the all-zeros initial configuration is 0-valent (every reachable decision is 0) and the all-ones is 1-valent. March from all-zeros to all-ones flipping one process's input at a time — a path of n+1 initial configurations, each adjacent pair differing at a single process. The path begins at a 0-valent configuration and ends at a 1-valent one, and every configuration on it is univalent by supposition; hence some adjacent pair C^-, C^+ has C^- 0-valent and C^+ 1-valent, the two differing only in the input of a single process q.

Consider any admissible deciding run from C^- in which q takes no steps — one exists, since a run in which q crashes at the outset and every message among the others is delivered is admissible, and total correctness makes it deciding. Let σ be a finite deciding prefix of its schedule; σ contains no event of q. Then σ is applicable to C^+ as well: the two initial configurations differ only in q's local input, no event of σ touches q, and by induction along σ every process other than q has identical state and identical deliveries in the two executions (the doppelgänger, mechanized). The deciding process decides the same value v in both. But $\sigma(C^-)$ is reachable from a 0-valent configuration, forcing $v = 0$, and $\sigma(C^+)$ from a 1-valent one, forcing $v = 1$. Contradiction; some initial configuration is bivalent. (Exhibit 3.1. Under bare non-triviality

the endpoints are supplied differently: if all initial configurations were univalent and all of one valence, no run could ever decide the other value; so both valences occur among them, and the cube being connected, an adjacent split pair exists. The remainder is unchanged.) □

Note the economy: the scheduling powers went unused — a row of paper dolls, one silenced juror, and the game must open with both verdicts alive. And note the lever, which rhymes throughout the chapter: the *tolerance* of a crash — the constitutional requirement that survivors carry on — let the adversary show them two worlds through the hole where Gussie used to be. (This chapter's dramas happen in state space, so its exhibits draw configurations as boxes and steps as arrows — the game-tree convention, beside the season's space-time convention of Exhibit 1.1.)

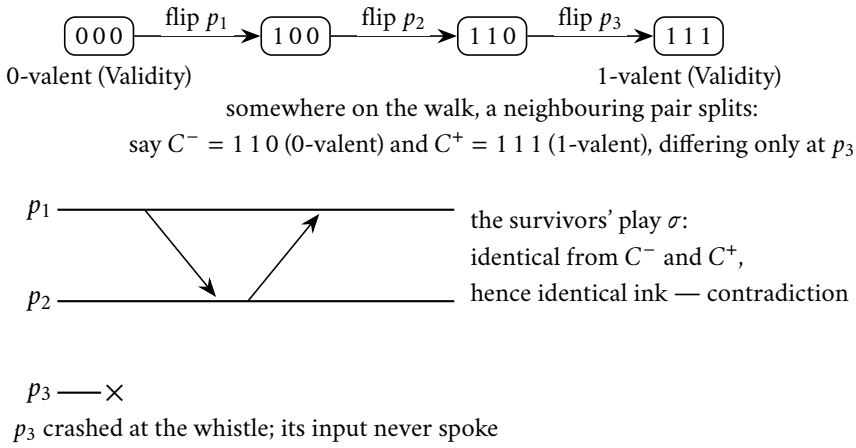


Exhibit 3.1 (initial bivalence; the paper-doll chain and the silenced juror). Top: the walk from all-noughts to all-ones, one ballot flip per step; if every corner were committed, some adjacent pair splits. Bottom: crash the differing juror before its first step; the two worlds are doppelgängers to the survivors, who must nonetheless decide — and decide identically, contradicting the split. Lemma 3.2.

The heart. The campaign's remaining vocabulary: the withheld letter — any letter the leash is currently owed — is the *pivot*, the event e; the single neighbouring step later found to separate a nought-committing landing from a one-committing one is the *switch*, the event e'. The claim that wins the war: from the fence, the scheduler can pay any owed letter — fairness honoured — and land still on the fence; the leash constrains *which* letters land, and the fence is defended by choosing *when*. Fairness, it turns out, is no protection at all. The proof stands below to the last inequality, with the neighbour construction most retellings compress to a sentence (Problem 3.6 rehearses it as a worked drill); the skeleton for review:

- **suppose** every position in which the pivot has just landed is committed;
- **both verdicts occur** among those landings (Step 1);
- **neighbours:** walking a schedule that avoids the pivot, some single step — the switch — separates a nought-committing landing from a one-committing one (Step 2);
- **switch at another desk:** the diamond commutes pivot and switch, making the one-committed landing reachable from the nought-committed one — absurd (Step 3, Exhibit 3.2);
- **switch at the same desk:** bench that process; the survivors, unable to tell benched from dead, must play to a verdict; both withheld letters commute across their play, delivering a nought-committed and

a one-committed future to the *verdict position* — a configuration with ink dry on its books (Step 4, Exhibit 3.3).

Lemma 3.3 (the pivot lemma; FLP's third)**.** *Let C be a bivalent configuration of a totally correct protocol and let $e = (p, m)$ be an event applicable to C. Let $\mathcal{C}$ be the set of configurations reachable from C by schedules not containing e, and let $\mathcal{D} = \{\, e(E) : E \in \mathcal{C} \,\}$ — the positions in which e has just landed. Then $\mathcal{D}$ contains a bivalent configuration.*

Proof of Lemma 3.3. Suppose instead that every configuration in $\mathcal{D}$ is univalent.

Step 1: $\mathcal{D}$ contains configurations of both valences. Since C is bivalent, for each $i \in \{0, 1\}$ some i-valent E_i is reachable from C. If E_i is reached by a schedule avoiding e, then $E_i \in \mathcal{C}$ and $F_i := e(E_i) \in \mathcal{D}$; F_i is reachable from the i-valent E_i, so $V(F_i) \subseteq \{i\}$, and being univalent by supposition, F_i is i-valent. Otherwise e occurs en route: write the schedule as $\sigma_1\, e\, \sigma_2$; then $F_i := e(\sigma_1(C)) \in \mathcal{D}$, and E_i is reachable from F_i, so $i \in V(F_i)$ and univalence makes F_i i-valent. Either way $\mathcal{D}$ has an i-valent member, for both i.

Step 2: neighbours. Call $E \in \mathcal{C}$ an *i-class* configuration if $e(E)$ is i-valent; by the supposition and Step 1, every member of $\mathcal{C}$ has a class and both classes are inhabited. C itself lies in some class; relabel values so that C is 0-class. Pick a 1-class $E^* \in \mathcal{C}$ and a schedule from C to E^* avoiding e; walking it, the class flips somewhere: there exist $C_0, C_1 \in \mathcal{C}$ and a single event $e' = (p', m')$ with

$$C_1 = e'(C_0), \qquad D_0 := e(C_0) \text{ is 0-valent}, \qquad D_1 := e(C_1) \text{ is 1-valent}.$$

(The walk begins 0-class and reaches 1-class; take the first flip. Problem 3.6 rehearses Steps 1–2 as a worked drill.)

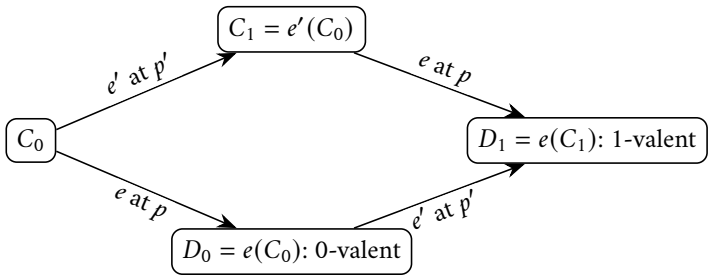
Step 3: the switch at another desk. Suppose $p' \neq p$. The single-event schedules e and e' involve distinct processes, so Lemma 3.1 gives $e'(D_0) = e'(e(C_0)) = e(e'(C_0)) = D_1$: the 1-valent D_1 is reachable from the 0-valent D_0, whence $1 \in V(D_0) = \{0\}$ — contradiction. (Exhibit 3.2.)

Step 4: the switch at the same desk. Suppose $p' = p$. Consider an admissible deciding run from C_0 in which p takes no steps (it exists: crash p at C_0, deliver everything else; total correctness decides it); let σ be a finite deciding prefix, so σ contains no event of p, and let $A = \sigma(C_0)$ — the verdict configuration; some process has decided in A, so by Remark 3.6, A is univalent. Both e and e' (and hence the two-event schedule $e'e$) involve only p, and σ involves only processes other than p; Lemma 3.1 therefore commutes them:

$$e(A) = e(\sigma(C_0)) = \sigma(e(C_0)) = \sigma(D_0), \qquad e(e'(A)) = \sigma(e(e'(C_0))) = \sigma(D_1).$$

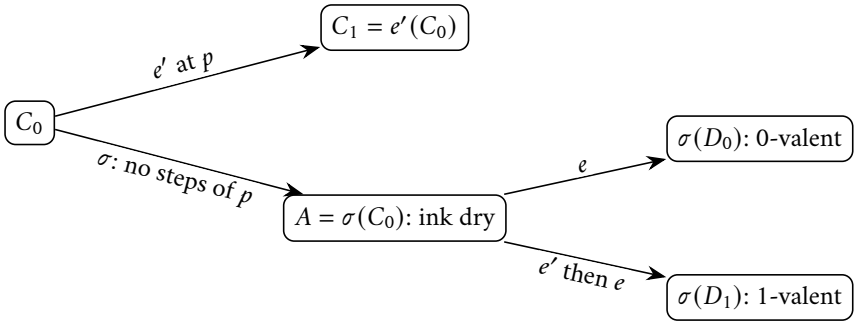
Now $\sigma(D_0)$ is reachable from the 0-valent D_0, so from A a 0-valent configuration is reachable (via e); and $\sigma(D_1)$ is reachable from the 1-valent D_1, so from A a 1-valent configuration is reachable (via e' then e). Hence $V(A) = \{0, 1\}$: the verdict configuration is bivalent — with ink dry on its books, contradicting Remark 3.6. (Exhibit 3.3.)

Both cases are absurd; the supposition falls, and $\mathcal{D}$ contains a bivalent configuration. □



$p' \neq p$: the square commutes (Lemma 3.1),
so the 1-valent D_1 is reachable from the 0-valent D_0 — absurd

Exhibit 3.2 (the diamond; the switch at another desk). Step 3 of Lemma 3.3: when the pivot e and the switch e' land at different desks, both routes around the square meet, and a committed position would have to re-open.



both futures alive at A: the verdict position is bivalent
with ink dry — contradicting Remark 3.6

Exhibit 3.3 (the switch at the same desk; the benched juror). Step 4 of Lemma 3.3: with p benched, the survivors play σ to a verdict at A — they cannot tell benched from crashed, and tolerance obliges them to finish. Both withheld letters commute across σ, delivering a 0-valent and a 1-valent future to a decided configuration.

Remark 3.7 (where each hypothesis is spent)*.* Audit the proof as Chapter 1 taught: *determinism* powers every doppelgänger and the diamond; *one-crash tolerance* manufactures the deciding run without p (Step 4) and without q (Lemma 3.2) — the fire escape the burglar uses; *asynchrony* lets the adversary withhold e and bench p for unboundedly long without breaching the leash — benched and crashed are doppelgängers to the survivors, which is Chapter 1's crashed-versus-slow, spent here as ammunition. Fairness is never breached: every withheld letter is eventually paid (Theorem 3.4).

The endgame: the rota. The war is won; the scheduler's full campaign assembles as the round-robin construction of Theorem 3.4's proof:

- **open** at a fenced position, guaranteed by Lemma 3.2;
- **serve** the gentlemen round and round forever; on each gentleman's turn, declare his oldest owed letter the pivot;
- **land it** by Lemma 3.3, the fence held;
- **audit the leash,** strap by strap: the run is fair — it is fairness itself — and nobody ever crashes; the dark privilege stays sheathed from first move to the — but there is no last move.

Theorem 3.4 (Fischer–Lynch–Paterson, 1985)**.** *In the asynchronous model with crash-stop failures, no deterministic protocol solves consensus tolerating one fault: every protocol satisfying Agreement and Validity in all admissible runs has an admissible run — indeed one with no crash and no message loss — in which no process ever decides.*

Proof of Theorem 3.4. Suppose a protocol solves consensus tolerating one fault. By Lemma 3.2 fix a bivalent initial configuration $C_{(0)}$. Maintain the processes in a queue $p_1, \ldots, p_n$ (any initial order). Given bivalent $C_{(k)}$, stage k+1 is: let p be the process at the head of the queue, and let m be the oldest message addressed

to p in $C_{(k)}$'s buffer — "oldest" by buffer entry order, ties broken arbitrarily — or $m = \varnothing$ if none. Set $e = (p, m)$: an applicable event. By Lemma 3.3 (with $C = C_{(k)}$) some bivalent $D \in \mathcal{D}$ exists: that is, there is a finite schedule σ_k avoiding e, followed by e itself, with $C_{(k+1)} := e(\sigma_k(C_{(k)}))$ bivalent. Apply it; move p to the tail of the queue.

The resulting infinite run is admissible with *zero* faults: every process heads the queue infinitely often and takes a step at each of its stages (and possibly within the σ_k's), so all processes are correct; and every message is eventually delivered — once posted, a message addressed to p has only finitely many predecessors in the buffer, each stage at p's head consumes its oldest pending message, and p heads the queue every n stages, so the message's seniority strictly improves until it is served. Yet every $C_{(k)}$ is bivalent — and no configuration anywhere in the run, the σ_k-interiors included, contains a decision: had some process decided at a configuration X of the run, X would be univalent (Remark 3.6), every configuration after X would satisfy $V \subseteq V(X)$, and the next $C_{(k)}$ — bivalent, and reachable from X — could not exist. So no process ever decides: Termination fails in an admissible run with no crash and no loss. No such protocol exists. □

What the adversary did. It lost nothing, forged nothing, duplicated nothing, crashed no one; its entire output, over an infinite game, was a sequence of perfectly legal decisions about *when*. Yet the crash did the killing — not a crash that happened; the crash that *might*: twice — the opening, the same-desk case — the survivors' obligation to finish without any colleague is precisely what the scheduler borrows, marching them down the corridor built for the emergency before producing the missing man alive. The tolerance is the vulnerability; the fire escape is how the burglar gets in. Everything that proceeds on silence can be led by silence. Crashed-versus-slow, Chapter 1's primordial nuisance, turns out to be a conservation law: the corner the tinkerers kept moving cannot close, because it is load-bearing.

3.4 Part Four — The Fine Print

An impossibility theorem misremembered is worse than none — Chapter 1's rule — and this theorem is misremembered for a living. What was proven: in the asynchronous model, no *deterministic* protocol can *guarantee* Agreement, Validity, and *Termination* for even a single bit, in every fair execution, while tolerating even one crash-stop failure. Each emphasized word is load-bearing, and each is a hinge on which Part Five's gates turn:

- **Termination is the only casualty:** the killing run never touches Agreement or Validity — it never lets anyone decide at all. The whiteboard phrasing: *safety is free; liveness is the hostage*. Protocols exist — sir operates several — that never, under any weather, ink two different verdicts; what none can add is the promise that the meeting always ends. The parliament that cannot be made to lie can still be made to filibuster; Chapter 4's protocols divide the estate exactly so — safety invoking no clocks, no timeouts, no luck; liveness beginning, in effect, "assuming the weather improves" — FLP, cited as fine print.
- **Deterministic:** the scheduler's campaign was a tablebase strategy — it steered toward the fence because, holding the rulebook, it could compute where the fence was. A coin in any gentleman's pocket clouds the tablebase — not yet a solution, but a doorway, with Ben-Or standing in it (Part Five).
- **One faulty process:** the title's modesty, and its menace — no lying, no forging, no Byzantine aunts; one possible crash-stop, the gentlest failure in the taxonomy, poisons the well. And mind the grammar of "possible": in the killing run nobody crashes; it is the *obligation* to survive a crash — the fire escape

the protocol is required to build — that the adversary turns. A protocol for immortal machines escapes trivially; one for the actual world walks in.

- **No loss:** the detail the folklore always drops. Two Generals was an impossibility of *loss*: kill a letter and knowledge cannot become common. FLP is an impossibility of *delay*: deliver every letter, and mere freedom about when still forbids guaranteed agreement. Two theorems about two liberties of the Post, bracketing the field's misery from both ends; "networks are unreliable" is half the lesson, for the reliable network with no latency bound is also a theorem-grade adversary.

What was not proven. Not that consensus is impossible in practice; not that the etcd cluster is broken. Real timing noise departs the killing schedule's razor edge almost immediately — one letter landing a moment early, and the parliament falls off the fence and decides — which is why the clusters decide millions of times a day. What is forbidden is a certain sentence ever being true: "this system is *guaranteed* to decide, whatever the timing." A vendor page containing it is selling a theorem violation; one is entitled — Chapter 1's triage — to ask which word is quietly redefined: the model, the guarantee, or the price. The scope, in its formal frame:

Remark 3.8 (scope; what is and is not forbidden). The casualty is Termination alone: the killing run of Theorem 3.4 never violates Agreement or Validity — it never lets anyone decide. Protocols exist whose safety is unconditional under arbitrary asynchrony (Chapter 4 exhibits one); what none can add is a termination *guarantee*. Each italicized hypothesis is load-bearing and is a gate: *asynchronous* (under partial synchrony consensus is solvable with safety always and termination after stabilization — Dwork–Lynch–Stockmeyer 1988; Chapter 4); *deterministic* (Ben-Or 1983 terminates with probability one — Box 7, Problem 3.10); and the *interface* (an eventually-accurate suspicion oracle restores solvability with a correct majority — Chandra–Toueg 1996; the weakest such oracle is the eventual leader, Chandra–Hadzilacos–Toueg 1996, stated here with respect and proved nowhere in this course). The theorem does not predict routine failure: the killing schedule is a single tablebase thread, and real timing noise departs from it almost surely; its production shadow is livelock — the election storm — not deadlock.

3.5 Part Five — The Loopholes

A good impossibility theorem is not a wall but a survey: it tells you precisely where the fence runs, and therefore precisely where the gates must be. The emphasized words of Part Four were the survey markers — *asynchronous*, *deterministic*, *guarantee* — three words, three gates, through which the entire practising industry files daily; the gate is stamped in the passport at each crossing.

Gate one: partial synchrony. Renegotiate *asynchronous*. Cynthia Dwork, Nancy Lynch — the same Lynch, fencing from both sides — and Larry Stockmeyer, "Consensus in the Presence of Partial Synchrony," Journal of the ACM, 1988; its two readings are known from Definition 1.2 (bounds exist but are unknown, or hold only after some unknown stabilization time). In partial synchrony consensus is solvable — safety holding *unconditionally* through the wild weather, termination guaranteed once the weather settles: conditional exactly where FLP says it must be, unconditional exactly where safety is free. Every timeout in every consensus deployment is this gate in miniature — a wager that the bound has probably been reached; every adaptive timeout, doubling patiently, gropes for the unknown bound from below. The industry does not defeat FLP; it rents a better model by the millisecond and keeps the receipts. Chapter 4 lives entirely inside this gate.

Gate two: randomization. Renegotiate *deterministic*. Michael Ben-Or, "Another Advantage of Free Choice," 1983 — the year FLP was first presented; this field's monuments are short. The advantage: a coin the scheduler cannot read. Rounds of votes; a decisive round is adopted and eventually inked; an indecisive one ends in a private coin flip and a fresh try. The scheduler still controls all timing — but the fence campaign required steering, steering requires forecasting reactions, and a coin has no forecast: the tablebase is blind one move ahead, forever. The price, stated with equal honesty: termination only *with probability one* — certainty of a new and weaker currency, no fixed deadline promised, and with naive private coins the expected wait grows exponential in the parliament's size, repaired to civilized figures by coins shared among the company. The industry uses the gate less than the theory admires it, gate one being cheaper; but it is the *deterministic* in FLP doing exact work, and the chapter's one box is deliberately from the far side: the theorems forbid; the box is the earliest famous escape.

Protocol — Box 7. Ben-Or's randomized consensus (1983), crash-fault form

n processes, at most t crashes, $2t < n$; each process p holds estimate $x \in \{0, 1\}$. All messages are broadcast to all (including the sender). Round $r = 1, 2, \ldots$ at each process:

1: **phase one:** broadcast $\langle \textsc{report}, r, x \rangle$
2: wait for $n - t$ messages $\langle \textsc{report}, r, * \rangle$
3: **if** more than $n/2$ of *all* n possible reports seen carry the same v:
4: broadcast $\langle \textsc{proposal}, r, v \rangle$
5: **else** broadcast $\langle \textsc{proposal}, r, ? \rangle$
6: **phase two:** wait for $n - t$ messages $\langle \textsc{proposal}, r, * \rangle$
7: **if** at least $t + 1$ of them carry the same value $v \neq ?$: decide v
8: **if** at least one of them carries some $v \neq ?$: $x \leftarrow v$
9: **else** $x \leftarrow$ fair coin flip
10: proceed to round $r + 1$ (deciders too: participation continues, which is what lets laggards catch up; practical variants stop after one further round)

Checked against Ben-Or (1983); the guard on line 3 is a majority of n, not of the $n - t$ received — this is what makes two different proposals in one round impossible (two majorities of n share a reporter: the crowded-club lemma, making its first appearance a chapter early). Safety is deterministic; only *waiting time* is random. Problem 3.10 walks safety, validity, and the almost-sure termination argument, and names honestly which adversary the elementary argument defeats.

Gate three: failure detectors. Renegotiate the *interface* — the subtlest gate and, for the trade, the most clarifying. Tushar Chandra and Sam Toueg, Journal of the ACM, 1996: leave the timing model alone and package the timing assumption behind an interface — a *suspicion service* at each gentleman's elbow, a list of colleagues currently suspected of being dead. Formally an unreliable failure detector, and the unreliability is the entire point: it may slander the living, acquit the dead for a while, change its mind hourly. The celebrated answer to how much eventual honesty suffices: enough that eventually, some one correct gentleman is never again suspected by any survivor — that, plus a majority of the parliament alive, and consensus is solvable. The companion result — Chandra with Vassos Hadzilacos and Toueg, same journal, same year — closes the question from below: the *weakest* sufficient oracle is exactly an eventual-leader service, one that eventually points every survivor at the same living gentleman (stated here with respect, proof declined — genuinely beyond this course). And the abstraction is not theoretical: a suspicion service is a timeout with its epistemology showing — every health check in the fleet, every phi-accrual dial, every lease expiry implements one; the theory's contribution is the specification, how little accuracy suffices and what the

protocol may lawfully do with slanderous evidence. Chapter 5 arms the detectors with leases and fences; Chapter 11 prices their false positives in retry storms.

The common commodity. All three gates purchase the same good by different contracts — partial synchrony with time, randomization with luck, detectors with delegated suspicion: a *stretch of settled asymmetry*, some interval in which one distinguished gentleman is, and is believed to be, in charge for long enough to shepherd a decision through. That is the commodity FLP proves cannot be manufactured from asynchronous determinism alone — and Chapter 4's Synod is precisely the art of acquiring it gracefully.

3.6 Part Six — One Bad Afternoon

From the profound impossibility to the famous one — the theorem every engineer can state and few state correctly. It concerns not eternity but one bad afternoon — formally, an execution containing a partition. The dates, exact: in July 2000, at the ACM's Symposium on Principles of Distributed Computing, Eric Brewer's invited keynote proposed, as engineering folk-wisdom in the honourable sense, that of consistency, availability, and tolerance of network partitions a system may have at most two; in 2002 Seth Gilbert and Nancy Lynch — the same Lynch a third time in this chapter; she fences on every piste in this field — published the short formalization and proof in SIGACT News, and the conjecture became the CAP theorem. Brewer's christening, Gilbert and Lynch's mathematics.

A theorem is its definitions, and every abuse of this one begins with a blurred letter:

- **C, consistency:** linearizability — the replicated service behaves, observably, as if there were a single copy, applying operations one at a time, respecting real time; every read returns the latest completed write, full stop (Def. 3.9; full legal rights in Chapter 7). Not the transactions department's "consistent," not "satisfies my invariants," not eventual anything — the strictest promise on the shelf, wearing the field's most abused word.
- **A, availability:** *total* — every request arriving at any non-failed node eventually receives a meaningful response; attend to the quantifiers: one lonely node on the wrong side of a partition, unable to answer a read, forfeits it. The marketing department's A is a fraction with nines in it; the two share a letter and nothing else.
- **P, partition tolerance:** the guarantee in question survives the Post severing itself — provinces isolated, every letter between them lost while the outage lasts. Mark the asymmetry that undoes most recitations: C and A are goods you promise; P is weather (Chapter 1's documentary record: the Bailis–Kingsbury catalogue, the correlated redundancy, the sharks); "choosing" partition intolerance is choosing to have no promise for an afternoon that will certainly come.

Definition 3.9 (the register service; linearizability; availability)**.** A *read/write register service* accepts `write`(v) and `read`() requests at any node, each eventually returning a response (an acknowledgment; a value). An execution of the service is *linearizable* (atomic) if there is a total order on the completed (and a subset of the pending) operations, consistent with real time — each operation ordered somewhere between its invocation and its response — under which every read returns the value of the latest preceding write. (This suffices for now; Chapter 7 gives the definition its full formal apparatus and its locality theorem.) The service is *available* if every request received by any node eventually receives a response.

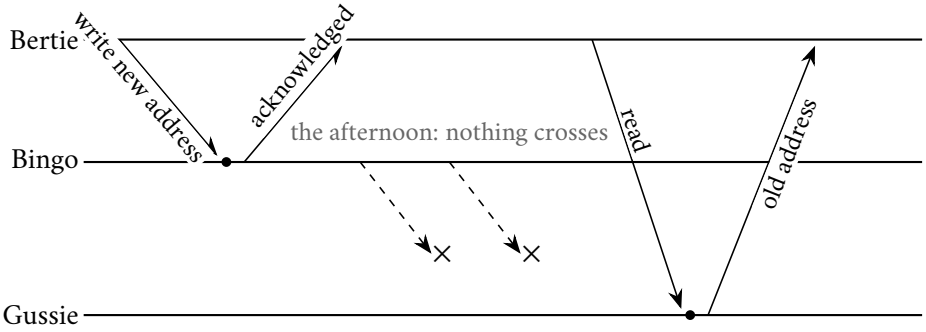
The theorem, with the company's own cast — Bertie's write completing on Bingo's side of the silence, his read answered on Gussie's. The skeleton is the doppelgänger's fourth appearance in this chapter: availability forces both a completed write on one side and an answered read on the other; no information crossed;

indistinguishability forces the stale answer. Choose your betrayal — the course's name for the consistency-or-availability dichotomy under partition.

Theorem 3.5 (Brewer's conjecture, proved; Gilbert–Lynch 2002)**.** *In the asynchronous network model, no implementation of a read/write register service guarantees both linearizability and availability in all executions, including those in which the network partitions.*

Proof of Theorem 3.5. Suppose an implementation guarantees both properties. Partition the nodes into non-empty components G_1 and G_2; from time t_0 every message between them is lost. Let the register's value at t_0 be v_0. A client submits $\mathsf{write}(v_1)$, $v_1 \neq v_0$, to a node in G_1 at t_0. By availability the write completes with an acknowledgment at some finite time $t_1 > t_0$; call this execution α_1. After t_1, a client submits $\mathsf{read}()$ to a node in G_2; by availability it returns a value at some finite $t_2 > t_1$.

Compare with the execution α_0 that is identical within G_2 — same initial states, same intra-G_2 traffic, same timing — but in which the write was never issued. Since no message crosses the partition in either execution, every node of G_2 has the same view in α_1 as in α_0 (induction on G_2's events: initial states equal, deliveries equal); the read therefore returns the same value in both. In α_0, linearizability forces the read to return v_0 (no other write exists). Hence in α_1 the read returns v_0 as well — but in α_1 the write of v_1 completed at $t_1 < t_2$, so every linearization must order it before the read, which must therefore return $v_1 \neq v_0$. Linearizability fails in α_1; the two guarantees cannot coexist. (Exhibit 3.4. Gilbert and Lynch run the same knife in a partially synchronous model, where it cuts equally; see the Reading.) □



to Gussie's province this world is the writeless world;
identical observations compel the identical (stale) answer

Exhibit 3.4 (one bad afternoon; Theorem 3.5). The write completes on Bingo's side (availability); the read answers on Gussie's side (availability); no letter crosses the severed Post, so the read returns the old value and linearizability dies — or Gussie stays silent and availability dies. Choose your betrayal.

The mathematics is honestly this small — it fit on a keynote slide in 2000 — which makes the subsequent decade of abuse the more instructive. The folklore, prosecuted exhibit by exhibit:

- **"Choose two of three":** menu fraud, twice over. P is not chosen; it arrives — a "CA system" is a system with no stated behaviour for an afternoon that is certainly coming; not a category but a confession, an unstated model in Chapter 1's sense. In fair weather the theorem is silent: nothing prevents a system from being consistent *and* available on every ordinary day, and the good ones are. The honest sentence is smaller and sharper: *when* a partition strikes, and only then, each affected operation must betray either consistency or availability.
- **The licence plates:** an industry of taxonomists stamping each database CP or AP, as though the theorem issued plates. The letters are extremes — C full linearizability, A every-node-answers —

and real systems are neither pole: configurable per operation, quorum-tunable, serving linearizable writes and stale-tolerant reads in the same breath. The label compresses a contract worth reading into a syllable worth nothing — Kleppmann's 2015 plea, assigned in the Reading. When a vendor page says "CP," the question is Chapter 1's: which failures, which guarantee, at which price — show me the afternoon in the documentation.

- **The binary that is a dial:** in 1999 Armando Fox and the same Brewer published "Harvest, Yield, and Scalable Tolerant Systems," the grown-up vocabulary for degradation: *yield*, the fraction of requests answered; *harvest*, the fraction of the relevant data each answer reflects. The partitioned search service answering every query from the reachable half of its index keeps its yield and halves its harvest; the banking service refusing minority-side writes spends yield to keep harvest whole. Between "consistent" and "available" runs a negotiated frontier (surveyed in Remark 3.10), and systems that degrade *well* chose their point on it deliberately, in peacetime, in writing (Problem 3.7 makes both instruments formal).
- **Partition-fixation:** the omission that costs the most money — the inference that a sturdy network makes strong consistency free. The corrective is *PACELC* (Abadi, IEEE Computer, 2012): if Partition, the trade is Availability versus Consistency — CAP, conceded; Else, in ordinary weather, which is most weather, the trade is Latency versus Consistency. One-copy behaviour on a perfectly healthy network still costs coordination — a quorum's round trip or a leader's confirmation before acknowledgment, on *every* write, in the currency users feel. That sentence explains the respectable half of the eventual-consistency industry: geo-replicated systems relax consistency not mainly because partitions terrify them but because the speed of light bills them daily — Chapter 1 priced Toronto to Sydney at a sixth of a second; Chapter 8 pays the bill in the other direction. CAP is the afternoon the network betrays you; PACELC is every day, including that one.

The formal middle ground:

Remark 3.10 (the partially synchronous variant; the middle ground). Gilbert and Lynch prove further that the impossibility survives partial synchrony — clocks and bounded delays do not rescue both properties through a partition — and that weakened contracts become achievable there (their *t-connected consistency*: stale reads permitted during partitions, reconciliation obligatory after). Between the poles runs a negotiated frontier: bounded-staleness answers, explicit partial results, locally accepted writes with declared reconciliation debts. Harvest and yield (Fox–Brewer 1999; Problem 3.7) are its surveying instruments, and PACELC (Abadi 2012) its peacetime ledger: if Partition, Availability versus Consistency; Else, Latency versus Consistency — the coordination toll is collected on every write on the best afternoon of the year.

The pattern, once more: the doppelgänger felled consensus in the grand theorem and the two-out-of-three dream in the small one, with the identical thrust — two worlds, one observer, no telling them apart. The master key, as advertised; it returns wearing a notary's collar to convict two-phase commit.

3.7 Part Seven — The Triage

Two theorems in this chapter, of very different weights, both routinely miscited — the correct moment to install the course's standing triage, promised in Chapter 1. When any claim of the form "that cannot be done" crosses the desk — in a design review, a vendor negotiation, a postmortem, an architecture council — it goes into one of three bins, and the sorting is the most consequential single habit this course will leave behind.

- **Bin one, impossible:** a theorem applies — this model, this guarantee, this exact ask — and the answer is no, permanently, for everyone — the vendor's deck included. Guaranteed-terminating deterministic consensus in pure asynchrony with a possible crash: Theorem 3.4. Linearizability with every-node availability through a partition: Theorem 3.5. Certainty of common action over a channel that may lose the last letter: Theorem 1.1. The correct response is never effort; it is renegotiation — change the model (buy synchrony, buy detectors), change the guarantee (probability one, eventual, bounded-stale), or change the ask. To *place* a claim here one must produce the theorem and match its model clause by clause — a theorem quoted without its model is a noise (Chapter 1); most things called impossible in industry are not in this bin.
- **Bin two, expensive:** possible, proven so, shipping in production — and carrying a price tag that arrives on schedule, forever. Consensus under partial synchrony: timeout tuning, election storms in bad weather, a liveness clause reading "when the network settles." Linearizability across a continent: a round trip — or seven milliseconds of purchased physics — per write, every write, peacetime included; PACELC's E-clause is this bin's ledger. Exactly-once *effect* (Chapter 10): idempotence engineering throughout. The correct response is neither refusal nor heroism; it is an invoice — name the recurring cost, decide who pays it, write both down. Most of the season from Chapter 4 onward walks this bin.
- **Bin three, merely inadvisable:** possible, affordable — and unwise, for this workload, this year. Consensus on the critical path of every read when a lease would do; strong consistency for a cache whose staleness nobody can observe; a cross-region transaction where a schema change would dissolve the need — a proverb in due course. The bin of judgment, with a diagnostic all its own: bins one and two are properties of mathematics and markets; bin three is a property of the *requirements*, the only bin whose contents change when the business does.

The senior failure mode is misfiling between two and three — paying bin-two invoices for guarantees the product stopped needing, or declaring bin-three inconveniences "impossible" to end a meeting. Which bin, on whose theorem, whose invoice, or whose judgment — the conversation that follows is shorter and considerably more honest than the one it replaces.

3.8 The Whiteboard

For the design review.

1. State the model before citing the impossibility — either impossibility, any impossibility. FLP names its terms: asynchronous, deterministic, one crash tolerated, guaranteed termination. CAP names its terms: a partition in progress, total availability, linearizability. Quote the theorem with its terms attached or not at all; a three-letter incantation without its model is heraldry, and design reviews are not the College of Arms.
2. Safety is free; liveness is the hostage. Interrogate any consensus-bearing design at exactly the joint the theorem names: what does it do *while it cannot decide*? What do clients see during the election storm, and what is the story when the storm outlives the deadline? A design with a crisp answer has read its FLP; a design that answers "that won't happen" has not, and the scheduler is patient.
3. CAP is a theorem about one bad afternoon; PACELC is about every day. Budget both, separately, in writing: the partition posture — which operations betray consistency and which betray availability when the Post severs, chosen in peacetime, tested on schedule — and the peacetime toll — what every write pays for the selected consistency level, in milliseconds, on the invoice the users can feel.

A review that discusses only the afternoon has priced the flood and forgotten the rent; between the poles, degrade deliberately, with the dials that have names: harvest and yield.

4. Triage every impossibility claim into its bin — impossible / expensive / merely inadvisable — and demand the bin's paperwork: a theorem with matching model for the first; a recurring invoice with a named payer for the second; a requirements argument, dated this fiscal year, for the third.

The register. Instalment paid: debt (2) of Chapter 1, crashed-versus-slow — managed, never cured — received its formal apparatus (the scheduler) and its promotion to murder weapon; the debt is not retired — it cannot be retired; that is rather the theorem — but it is henceforth managed by mathematics instead of anecdote, with instalments remaining in Chapter 5 (leases, fences) and Chapter 11 (pricing false suspicion). Debts issued: (1) partial synchrony, surveyed as gate one — cashed in Chapter 4, when the Synod moves in and furnishes it; (2) the suspicion services of gate three — machinery in Chapter 5, price in Chapter 11; (3) CAP's C, linearizability, defined by gesture and promissory note — read its full legal rights in Chapter 7.

3.9 Problems

Tier I — Calisthenics

Problem 3.1 (valence drill)**.** A toy protocol's reachable configurations form the finite game tree below: arrows are the adversary's available moves; leaves are deciding configurations labelled with their decision.

Node	Successors (or decision)
A	B, C
B	D, E
C	E, F
D	decides 0
E	G, H
F	decides 1
G	decides 0
H	decides 0

(a) Classify every node as 0-valent, 1-valent, or bivalent. (b) List every *critical* move: an arrow from a bivalent node to a univalent one. (c) At which nodes could a process have already decided, consistently with Remark 3.6? (d) If the adversary plays to avoid decisions forever, where does it get stuck, and why does no such trap exist for Theorem 3.4's adversary?

Problem 3.2 (the fence walk)**.** Three processes, binary inputs: eight initial configurations. A colleague claims a correct one-fault-tolerant protocol may commit at the whistle, with the valence of each initial configuration equal to the *majority* of the three inputs. (a) Exhibit the walk of Lemma 3.2 for this claim and point to a splitting pair. (b) Describe the crash execution that refutes the claim at that pair, naming who is crashed and what the survivors see. (c) Show the same refutation works for *any* committed-at-the-whistle claim, not just majority. (d) Where exactly did (b) use one-fault tolerance? (e) Rewrite the endpoints of the walk under bare non-triviality (Def. 3.1), without Validity.

Problem 3.3 (the leash, audited)**.** Three processes; classify each infinite run as admissible or not, and if not, name the violated clause (Def. 3.4): (a) the rota of Theorem 3.4's proof; (b) a run in which one message to a correct process is withheld forever while its addressee takes infinitely many null steps; (c) a run in which p_3

takes no steps after some point and p_1 crashes; (d) a run in which every message is delivered but p_2, alive, is given only finitely many steps; (e) a run in which p_3 crashes and every message addressed to p_1 and p_2 is eventually delivered.

Problem 3.4 (one afternoon, catalogued)**.** Five nodes; a partition isolates $\{D, E\}$ from $\{A, B, C\}$ for one hour. During the hour: a write of v_2 (over v_1) completes at A; a read at B returns v_2; a read at D returns v_1; a write at E is refused with an error. (a) Which responses, if any, violate linearizability? (b) Which behaviours forfeit availability in Gilbert–Lynch's total sense? (c) Classify the system's evident posture on each side of the partition. (d) Compute the hour's yield if the four operations above were the only requests, and the harvest of the read at D if the register's ideal answer reflects all completed writes.

Problem 3.5 (triage drill)**.** File each claim in its bin — impossible / expensive / merely inadvisable — citing the theorem (with model match), the recurring invoice, or the requirements argument: (a) "our asynchronous replicated store guarantees failover within one second, always"; (b) "clients in every region read their nearest replica, linearizably, throughout partitions"; (c) "all writes are strictly serializable across three continents"; (d) "every cache read goes through consensus for freshness"; (e) "our queue delivers each message exactly once, guaranteed"; (f) "our queue's consumers process each message exactly once, effectively."

Tier II — The Real Thing

Problem 3.6 (the compressed case, completed)**.** Prove Steps 1 and 2 of Lemma 3.3 in full, from Definitions 3.3–3.5: (a) $\mathcal{D}$ contains members of both valences (mind both sub-cases: e avoided, e en route); (b) the class walk yields neighbours $C_1 = e'(C_0)$ with $D_i = e(C_i)$ i-valent; justify the relabeling that lets the walk start in class 0. (c) State precisely where (a) uses the standing fact that e remains applicable throughout C. *(Full solution.)*

Problem 3.7 (harvest and yield, formalized)**.** Following Fox and Brewer: for an execution window, *yield* is the fraction of issued requests that receive responses; for a query whose ideal answer aggregates a data set S, the *harvest* of a response is the fraction of S it reflects. (a) Formalize both for a service sharded over n nodes, data uniform, a partition leaving k of n shards reachable from the serving side. (b) Show a service that always answers achieves yield 1 with harvest k/n on aggregate queries, and that any deterministic always-answering service admits a query whose harvest is at most k/n. (c) Show that enforcing a harvest floor $h > k/n$ costs yield: the requests that cannot be served at floor h during the partition are exactly the aggregate queries, and compute the resulting yield for a workload that is a fraction q aggregates. (d) Conclude with the design sentence: harvest and yield are the two currencies in which the partitioned afternoon may be paid, and the exchange rate is the workload mix.

Problem 3.8 (agreement among the dead)**.** *Uniform agreement* strengthens Agreement: no two processes — crashed later or not — ever decide differently. (a) Exhibit a protocol and an admissible execution in which Agreement (among the correct) holds but uniform agreement fails: a process decides, externalizes nothing, and crashes; the survivors decide otherwise. (b) Explain in one paragraph why the distinction has a body count in practice: what may a process do with its decision *before* crashing? (c) Show Theorem 3.4 applies a fortiori to uniform consensus. (d) In the *synchronous* crash model both variants are solvable; look up (or conjecture) whether they cost the same in rounds, and record the moral: uniformity is a real good with a real price. *(Hint provided; solution sketches (a)–(c).)*

Problem 3.9 (sabotage: the impatient tribunal)**.** A well-meaning colleague, having read Part Five, designs "consensus" for three processes with a rotating chairman and timeouts standing in for a failure detector. Round r has chairman $p_{(r \bmod 3)+1}$. Rules, at each process, current estimate x:

- *Chairman of round r:* broadcast $\langle \textsc{propose}, r, x \rangle$; on receiving $\langle \textsc{ack}, r \rangle$ from at least one other process (a majority of three with himself), decide x and broadcast $\langle \textsc{decide}, x \rangle$.
- *Any process, on $\langle \textsc{propose}, r, v \rangle$ in its current round r:* set $x \leftarrow v$; reply $\langle \textsc{ack}, r \rangle$ to the chairman.
- *Any process, on $\langle \textsc{decide}, v \rangle$:* decide v.
- *Timeouts:* a non-chairman waiting in round r for a proposal, or a process that has acknowledged but seen no decision, advances on timeout to round $r + 1$, keeping its estimate. The chairman likewise advances if his acknowledgment is slow.
- Stale-round messages (r less than the recipient's current round) are ignored.

Inputs: $x_1 = 1$, $x_2 = 0$, $x_3 = 0$. (a) Exhibit an admissible execution in which two correct processes ink different values. (b) Name the missing discipline in one sentence, and say which protocol of Chapter 4 supplies it. (c) Explain why no re-tuning of the timeouts repairs the protocol, citing the theorem that says so. *(Certified fatal per the standing rules: the split-brain execution is written out in the solutions, ink and all.)*

Tier III — For the Ambitious (starred)

Problem 3.10 (★ Ben-Or, guided)**.** Throughout, Box 7 runs with $2t < n$. (a) *(one proposal per round)* show that if $\langle \textsc{proposal}, r, v \rangle$ and $\langle \textsc{proposal}, r, w \rangle$ are both sent with $v, w \neq ?$, then $v = w$ — two majorities of n share a reporter. (b) *(decision cascades)* show that if some process decides v in round r, every process completing round r adopts v, and every process completing round r+1 decides v — pigeonhole t+1 against $n-t$. (c) *(safety, validity)* conclude Agreement; show unanimous inputs decide in round one. (d) *(termination)* prove: if at the start of a round all correct estimates agree, all correct processes decide within two rounds; then show that against an adversary that fixes its schedule *before* the coins are flipped, each round ends with agreeing estimates with probability at least 2^{-n}, so termination has probability one and expected round count at most $2^n \cdot 2$. (e) State honestly why (d)'s adversary is weaker than the scheduler of this chapter (it may not adapt to coin outcomes), and record the pointer that the theorem holds against the adaptive adversary as well, with a more careful argument (Aguilera and Toueg, 1998), and that shared coins repair the exponential expectation (the door stays ajar for the cryptographically inclined). *(Hints only.)*

Problem 3.11 (★ the impossibility propagates)**.** *Atomic broadcast*: all correct processes deliver the same messages in the same order; every message broadcast by a correct process is eventually delivered. (a) Reduce consensus to atomic broadcast in the asynchronous crash model: each process broadcasts its input and decides the first value delivered; verify Agreement, Validity, Termination. (b) Conclude from Theorem 3.4 that no deterministic atomic broadcast protocol tolerates one crash in the asynchronous model. (c) Extend to state-machine replication: any deterministic fault-tolerant replicated log in the asynchronous model must, somewhere, be spending one of the three gates' currencies — find the sentence in your favourite system's documentation where the purchase is disclosed, or the absence where it is not. *(Hints only.)*

3.10 Hints

Hint (Problem 3.6)**.** For (a), the two sub-cases differ in where e sits relative to the schedule reaching E_i; in the en-route case, cut the schedule at e and let the prefix define the member of $\mathcal{D}$. For (b), the walk is along a schedule inside C; the class function is total on C because every member of $\mathcal{D}$ is univalent by supposition.

Hint (Problem 3.8)**.** For (a): a chairman who decides his own value on receipt of one acknowledgment, then crashes before his decide-message survives, while a fallback path decides the other value, is enough — the

tribunal of Problem 3.9 can be arranged to do it. For (c): a uniform-consensus protocol is in particular a consensus protocol.

Hint (Problem 3.9)**.** Let the round-one chairman's proposal reach p_3 promptly but p_2 slowly; let acknowledgments and decide-messages dawdle. Two chairmen, two majorities of two, no common member — the crowded club needs majorities of the *same* parliament, and acknowledgments here carry no promise about the future. Fatal execution: Solution and Exhibit 3.5.

Hint (Problem 3.10)**.** (a) Each process reports once per round. (b) $(t+1)+(n-t) > n$. (d) Condition on the round's fixed message schedule; whatever subsets are waited for, either some process sees a proposal (then all coin flips landing on that proposal's value suffice) or none does (then all coins landing equal suffice); both events have probability at least 2^{-n}.

Hint (Problem 3.11)**.** (a) Total order makes "first delivered" identical everywhere; Validity holds because only inputs are broadcast; Termination because a correct process's broadcast is delivered. (b) Contrapositive.

3.11 Solutions

Tier I in full (tersely); Tier II in full for Problems 3.6 and 3.9 (the first works the pivot lemma's neighbour construction, the second is the sabotage certification), solution sketch for Problems 3.7 and 3.8; hints only for Tier III, by standing policy.

Solution (to Problem 3.1)**.** (a) D, G, H: 0-valent (deciding); F: 1-valent; E: 0-valent (G, H only); B: 0-valent (D, E); C: bivalent (E reaches 0, F reaches 1); A: bivalent (via B reaches 0, via C, F reaches 1). (b) Critical moves: $A \to B$ (0-committing), $C \to E$ (0-committing), $C \to F$ (1-committing). (c) Only at univalent nodes; and only with the decided value matching the valence — so possibly at D, E, F, G, H and B (value 0); never at A or C. (d) At C every move commits; the fence cannot be held. No such trap exists in Theorem 3.4 because Lemma 3.3 guarantees a bivalent successor through *every* owed delivery — infinite trees with the pivot property have no forced-commitment nodes on the adversary's chosen path.

Solution (to Problem 3.2)**.** (a) Walk $000 \to 100 \to 110 \to 111$: majority-valences $0, 0, 1, 1$; the split pair is $(100, 110)$, differing at p_2. (b) Crash p_2 at the whistle in both worlds. Survivors p_1, p_3 see input multiset $\{1, 0\}$ in both, receive identical deliveries, and by determinism ink identically — contradicting 0-valence of 100 and 1-valence of 110 simultaneously. (c) Any total valence-assignment on the cube agreeing with Validity at the two unanimous corners changes value along some edge of any corner-to-corner path; the argument of (b) applies at that edge verbatim — nothing about majority was used. (d) In the survivors' obligation to decide at all: without one-fault tolerance they may lawfully wait for p_2 forever, and the contradiction evaporates. (e) If all initial configurations were univalent with a single common valence v, no run decides $1 - v$ anywhere, contradicting non-triviality; so both valences occur among the corners, and any path between differently-valent corners supplies the split pair.

Solution (to Problem 3.3)**.** (a) Admissible; zero faulty (shown in the proof of Theorem 3.4). (b) Inadmissible: delivery clause violated (the addressee is correct — infinitely many steps — but the message never arrives). (c) Inadmissible: two faulty processes. (d) Inadmissible: p_2 is faulty by definition (finitely many steps), which is permitted — but then the run must deliver all messages to the *correct* processes; if it does, it is admissible with faulty set $\{p_2\}$ — so: admissible iff no second process is starved and all mail to p_1, p_3 arrives; as posed ("every message is delivered") it is admissible, with p_2 counted as the one fault. The lesson: "alive but starved forever" *is* the model's crash — benched-forever and crashed are the same run, which is exactly the doppelgänger the proofs spend. (e) Admissible: one faulty, mail to the correct delivered.

Solution (to Problem 3.4)**.** (a) The read at D violates linearizability: it follows (in real time) the completed write of v_2 yet returns v_1. The read at B is consistent; the refusal at E is not a linearizability violation (no response, no misordering). (b) The refusal at E forfeits total availability (E is non-failed and answered with an error, not a meaningful response); everything else answered. (c) Majority side: consistent and available (it can be both — it holds the quorum); minority side: D chose availability over consistency, E chose consistency (refusal) over availability — one system, two postures, which is precisely why CP/AP licence plates fail. (d) Yield $3/4$; harvest of D's read: it reflects the state as of the partition's onset — all writes except v_2, hence (with two completed writes in history, v_1 then v_2) $1/2$ of the relevant write-set; as a fraction of the ideal single-value answer, simply: stale by one write.

Solution (to Problem 3.5)**.** (a) Impossible as guaranteed: detection of the dead coordinator in bounded time in an asynchronous model contradicts Theorem 3.4's terrain (crashed-versus-slow); re-file as expensive under partial synchrony with the timeout named. (b) Impossible: Theorem 3.5 (linearizability, total availability, partition — all three claimed). (c) Expensive: a round trip or purchased clock uncertainty per commit (Chapter 8's invoice); possible and shipping. (d) Merely inadvisable (usually): a lease (Chapter 5) delivers the freshness at a fraction of the cost; the requirements argument decides. (e) Impossible: Two Generals (Theorem 1.1) — the last acknowledgment problem; "guaranteed exactly-once delivery" over a lossy channel is bin one. (f) Expensive: idempotence or deduplication engineering throughout (Chapters 1 and 10) — possible, priced, routine.

Solution (to Problem 3.6)**.** (a) Let $i \in \{0, 1\}$. C bivalent gives an i-valent E_i reachable from C by some schedule τ. *Case $e \notin \tau$:* then $E_i \in \mathcal{C}$ by definition, and since e is applicable to C and no event of τ delivers m (only e does), e remains applicable at E_i — this is where the standing fact of Definition 3.3 is spent, answering (c); so $F_i := e(E_i) \in \mathcal{D}$. F_i is reachable from E_i, so $V(F_i) \subseteq V(E_i) = \{i\}$; by the supposition F_i is univalent, hence i-valent. *Case $e \in \tau$:* write $\tau = \sigma_1 e \sigma_2$ with $e \notin \sigma_1$. Then $\sigma_1(C) \in \mathcal{C}$ and $F_i := e(\sigma_1(C)) \in \mathcal{D}$; moreover $E_i = \sigma_2(F_i)$ is reachable from F_i, so $i \in V(F_i)$, and univalence forces F_i i-valent. Both valences therefore occur in $\mathcal{D}$.

(b) Define, for $E \in \mathcal{C}$, $\mathrm{class}(E)$ = the valence of $e(E)$ — total by the supposition that all of $\mathcal{D}$ is univalent. By (a) both classes are inhabited: choose witnesses $W_0, W_1 \in \mathcal{C}$ with $\mathrm{class}(W_j) = j$. $C \in \mathcal{C}$ has some class; set $b = \mathrm{class}(C)$ and relabel the values $0 \leftrightarrow 1$ if necessary so that $b = 0$ — the relabeling is harmless because every definition in play is symmetric in the two values, and (a) survives relabeling since it produced witnesses of both classes. Take the schedule (avoiding e) from C to W_1; its configurations all lie in $\mathcal{C}$; the class starts at 0 and ends at 1, so consecutive configurations $C_0, C_1 = e'(C_0)$ exist with classes 0, 1: that is, $D_0 = e(C_0)$ is 0-valent and $D_1 = e(C_1)$ is 1-valent.

Solution (to Problem 3.7)**.** (Sketch.) (a) Yield = |responded|/|issued| over the window; harvest of a response to query Q with ideal input S_Q: $|S_Q \cap \text{reachable}|/|S_Q|$. (b) Always-answering gives yield 1; an aggregate over all shards has $|S_Q \cap \text{reachable}| = k/n \cdot |S_Q|$ under uniformity; for the "at most" clause, the adversary partitions away the shards the service cannot reconstruct, and determinism bars invention. (c) Aggregates cannot meet the floor, point reads can; yield $= 1 - q$ (aggregates refused), harvest = 1 on everything served. (d) As stated: the workload mix q is the exchange rate between the two currencies.

Solution (to Problem 3.8)**.** (Sketch.) (a) Run Problem 3.9's tribunal with the schedule of its solution, but crash p_1 immediately after his decide step and before his $\langle \textsc{decide}, 1\rangle$ letters leave (or lose nothing: let them be delivered after the others decided — the violation among *correct* processes then needs p_1 dead, so crash him; the surviving pair agree on 0). Agreement-among-correct holds; the corpse holds contrary ink. (b) The body count: a process may act on its decision before crashing — release a lock, ship an order, acknowledge a client; uniform agreement is the difference between "the dead disagreed harmlessly" and "the dead charged

the card." (c) Uniform agreement implies Agreement, so a uniform-consensus protocol solves consensus, and Theorem 3.4 forbids it a fortiori in the asynchronous model. (d) Pointer: in the synchronous crash model both are solvable; early-deciding bounds differ (uniformity costs a round in well-studied settings — Charron-Bost and Schiper's line of work); moral as stated.

Solution (to Problem 3.9)**.** (a) The split-brain execution, drawn as Exhibit 3.5. Inputs $x_1 = 1$, $x_2 = 0$, $x_3 = 0$; round one chairman p_1.

(1) p_1 broadcasts $\langle$PROPOSE, 1, 1$\rangle$. The Post delivers it to p_3 promptly; the copy to p_2 dawdles.

(2) p_3 (in round one) sets $x_3 \leftarrow 1$ and replies $\langle$ACK, 1$\rangle$; the acknowledgment travels slowly but surely.

(3) p_2, hearing no proposal, times out and advances to round two. The stale proposal later reaches p_2 and is ignored (stale rule). p_3, having acknowledged and seen no decision, times out in turn and advances to round two, *keeping* $x_3 = 1$? — no: attend to the rule; he keeps his *current* estimate, which the round-one proposal already overwrote to 1. To arrange the split we need p_3's acknowledgment to have committed him to nothing — and it has not: no rule binds an acknowledger's future. Continue.

(4) Round two, chairman p_2, estimate 0: broadcasts $\langle$PROPOSE, 2, 0$\rangle$. It reaches p_3 (now in round two): p_3 sets $x_3 \leftarrow 0$ and acknowledges. p_2 receives the acknowledgment: majority of two (with himself); p_2 **decides** 0 and broadcasts $\langle$DECIDE, 0$\rangle$; p_3, receiving it, decides 0 as well.

(5) Only now does the Post deliver p_3's *round-one* acknowledgment to p_1 — still lawfully in round one, his timeout generous. Majority of two (with himself): p_1 **decides** 1 and broadcasts $\langle$DECIDE, 1$\rangle$, which arrives at two gentlemen who have already inked 0.

Two correct processes, two inks. Every delay was finite; every message was delivered; the execution is admissible with zero crashes. Fatality certified.

(b) The missing discipline: an acknowledgment binds its maker to nothing about later rounds — the protocol lacks the promise-and-adopt machinery by which a new chairman must first learn what an old majority may already have committed. Chapter 4's phase one (Paxos's prepare/promise, with adopt-the-highest) supplies exactly this. (c) No timeout tuning helps: the executions above violate no timing bound the protocol could enforce, because in the asynchronous model the adversary may realize any finite pattern of delays; and a protocol of this rotating-suspicion shape that *always* terminated would contradict Theorem 3.4. Timeouts may make the split rare; only the missing discipline makes it impossible — safety must never rest on the clock.

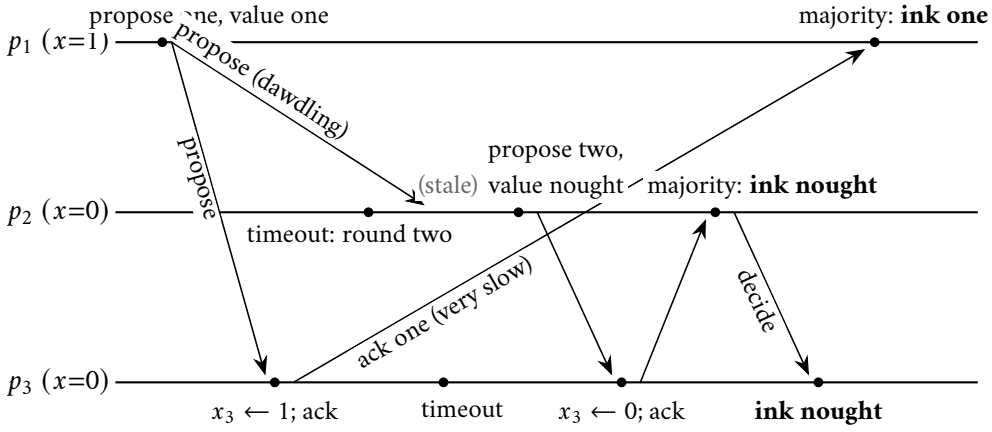


Exhibit 3.5 (the impatient tribunal's split brain; Solution 3.9). The round-one acknowledgment, slow but lawful, arrives after the parliament has moved on and decided nought elsewhere; the deposed chairman,

none the wiser, completes his old majority and inks one. Nobody crashed; every letter arrived. Fatality certified.

Solution (to Problems 3.10 and 3.11)**.** By standing policy: hints only (the Hints section above). For Problem 3.10(a), the arithmetic checkpoint: two sets of reporters each of size exceeding $n/2$ intersect; the shared reporter sent one report.

3.12 The Reading

Fischer, Lynch, and Paterson, "Impossibility of Distributed Consensus with One Faulty Process," Journal of the ACM, April 1985. This chapter's text, and the volume's namesake proof. Nine pages; the fatal argument is three. First presented in 1983, at a symposium on database systems — which tells you who was bleeding. Read the model section slowly (their message buffer is this chapter's bag, their Lemma 1 our diamond), then read Lemma 3 twice: it is the pivot lemma, its neighbour construction rehearsed here as Problem 3.6. Honoured with the field's influential-paper prize in 2001, its second year. When you finish, look up and recall that this fenced off a corner of the possible for every machine that will ever be built.

Gilbert and Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services," ACM SIGACT News, June 2002. Equally short. Read for the definitions above all: what *available* totally means, what *atomic* precisely is; then their second theorem — the partially synchronous variant the keynotes never mention — and the t-connected consistency of their final section, which is the negotiated frontier of Remark 3.10 in their own tailoring. As disinfectant afterwards: Kleppmann, "A Critique of the CAP Theorem" (2015) — the plea about the licence plates.

Abadi, "Consistency Tradeoffs in Modern Distributed Database System Design," IEEE Computer, February 2012. The PACELC column. An evening's read, one acronym, and a permanent upgrade to the design-review vocabulary: the peacetime latency toll, named and billed. Read alongside the Whiteboard's third item.

For the starred doors. Ben-Or, "Another Advantage of Free Choice: Completely Asynchronous Agreement Protocols," PODC 1983 (Problem 3.10) — a handful of pages; this field's monuments are short. Aguilera and Toueg, "Failure Detection and Randomization: A Hybrid Approach to Solve Consensus," SIAM Journal on Computing, 1998, for the adaptive-adversary analysis. Chandra and Toueg, "Unreliable Failure Detectors for Reliable Distributed Systems," and Chandra, Hadzilacos, and Toueg, "The Weakest Failure Detector for Solving Consensus," both Journal of the ACM, 1996 — the third gate, surveyed by its builders.

Standing companion: Kleppmann, chapter 9. His treatment of linearizability and the CAP critique pairs with Part Six; his pages on total order broadcast anticipate Problem 3.11 and Chapter 5. Assigned once for the season; in this chapter it earns its keep twice.

Chapter 4

The Part-Time Parliament

Companion to Episode Four. The episode tells the story — the island manuscript and its eight years in the drawer, three naive parliaments dead on stage, the counting chairman staged to his ruin; this chapter keeps the record: the single-decree problem sized, the crowded club proved in one line, the Synod derived through its scars and boxed against "Paxos Made Simple," the invariant's induction written out in full, the fine print — liveness sold separately, persistence load-bearing, two sabotages certified fatal — Raft boxed to Figure-2 fidelity with leader completeness proved after Ongaro's thesis, Viewstamped Replication restored to its rightful seniority, and Herlihy's universality theorem, by which this one small agreement is the atom of everything the season builds hereafter. Statements, proofs, traces, problems.

4.1 Part One — One Law at a Time

Fault-tolerant consensus: the working escape through the very gate Chapter 3 surveyed. The episode derives it as theatre — three naive parliaments dying instructive deaths — and the record keeps every scar with its rule attached: recited, Paxos is a liturgy of prepares and promises that seem arbitrary; derived, every rule is a scar with a story.

The provenance. In 1990 Leslie Lamport submitted to the ACM's *Transactions on Computer Systems* a paper describing the parliament of a vanished Greek island called Paxos, its correctness argument dressed in fictional archaeology. The reviewers found the result interesting and the presentation insufferable; the paper sat in a drawer for eight years while the industry reinvented fragments of it, usually wrongly, at considerable expense. Published at last in 1998, jokes intact.

The problem, sized; the cast. Half of Paxos's reputation for difficulty is three problems solved at once by people who thought they were solving one. This chapter treats the *single-decree* problem — one law, decided once — and nothing else: the log of many laws is Chapter 5's business, and elections enter only as liveness machinery, exactly where Theorem 3.4 says they must. The scene: one by-law (whether Thursday deliveries shall be free); proposers anywhere — Bingo moves one version, Tuppy another; ledger-keepers Bingo, Tuppy, and Gussie; Bertie merely wishing to learn the outcome. The Post delays and reorders without bound; the legislators crash and *recover* with whatever the written ledger preserved — whence *amnesia forbidden*: anything a gentleman must stand behind later, he writes in the ledger before answering (Problem 4.7 is the certified disaster).

Model of this chapter

For the Synod and for Raft (this whole chapter). Asynchronous message passing; the Post may delay and reorder without bound but neither forges nor (for the liveness discussions) permanently loses — retransmission is assumed beneath the protocol, per Chapter 1's constructions. Failures: *crash-recovery*: a process may stop at any point and later resume with exactly its *stable storage* (its ledger) intact and its volatile state gone. Every protocol variable marked persistent in Boxes 8 and 9 is written to stable storage *before* any message reporting it is sent. Quorums are majorities of a *fixed* membership (reconfiguration is debt #14, deferred to Chapter 5); only pairwise quorum intersection is used by any proof (Rem. 4.3).

Safety versus liveness, divided per Theorem 3.4. All safety claims below hold in the bare model above — any schedule, any finite number of crashes and recoveries, duelling proposers included. No termination claim is made or provable there; liveness discussions (Rem. 4.8) assume partial synchrony after stabilization plus a single distinguished proposer/leader, i.e. exactly the purchases Chapter 3's gates license.

The three offices. Three *roles*, not three species of machine: a *proposer* — whoever currently has ambitions for the by-law, Bingo and Tuppy both, which is precisely the trouble; an *acceptor* — keeper of the ledger and caster of votes, the parliament proper, the only office whose conduct decides safety; a *learner* — anyone wishing to discover what was chosen without necessarily having voted. One machine ordinarily holds all three commissions; keep the offices separate, for every rule below belongs to exactly one. The specification: a value is *chosen* when some quorum has all accepted it at one ballot; termination is deliberately not promised — the correct engineering response to Fischer–Lynch–Paterson: since no protocol can promise both under asynchrony, build one whose promises never include the impossible part.

Definition 4.1 (single-decree consensus; roles; chosen)**.** Fixed acceptors $a_1, \ldots, a_n$ (the parliament); any number of proposers; learners at leisure. Proposers hold input values (motions). A value v is *chosen* iff there exist a ballot b and a quorum Q of acceptors such that every member of Q has accepted the proposal (b, v). Required, in every execution of the model box: *(Agreement)* at most one value is chosen, ever; *(Validity)* any chosen value is some proposer's motion. Termination is deliberately not required (Theorem 3.4); liveness is discussed, not promised, in Remark 4.8.

Remark 4.2 ("chosen" is a silent, stable fact)*.* Read the definition for what it does *not* contain: no announcement, no instant of proclamation, no participant who must know. "Chosen" is a predicate on the acceptors' collective ledgers, true exactly when some quorum's acceptances of one (b, v) all exist, and it may hold while every gentleman in the house believes the matter open — the witness of Lemma 4.1 need not know what he carries. Three consequences, each load-bearing. *First*, being chosen and anyone *knowing* it are distinct events: knowledge is not delivered but *collated*, and a learner arriving cold must take the house's deposition — reading the parliament is itself a ballot — rather than await a herald. *Second*, there is no privileged instant at which the fact is born: asynchrony admits no global "now" (Chapter 2's ghost: there is no *the* current state, only states along cuts), so "the moment of choosing" is not well-defined across the house. *Third* — and this is what rescues the predicate from that relativity — "chosen" is *stable*: once true along any consistent cut it is true along every later one, which is the entire content of *irrevocability* (the Agreement clause above). A stable predicate is exactly the kind on which all consistent observers eventually agree, even without agreeing on *when*; the naive "a quorum *currently* agrees" of the coming false start is unstable, and so has no observer-independent truth at all — two learners, two audits, two verdicts. The protocol's whole

cleverness is spent engineering a fact whose *becoming-true* no one witnesses, and then forbidding that fact ever to be un-made.

Why majorities. The candidates for "quorum," priced:

- **Unanimity:** one gentleman in the bath and the house is struck dumb — the fault-tolerance ambition surrendered entire (we distribute *because* machines fail).
- **A fixed pair of appointees:** concentrated fragility — lose one and no quorum can ever form again.
- **Majorities:** the risk spread evenly and priced honestly — a house of three proceeds despite any one absentee, five despite any two, the house in general despite the silence of just under half its members.

The deeper truth: nothing in this chapter's proofs uses "more than half" as such — only the crowded-club property, every quorum overlapping every other, of which majorities are the cheapest symmetric purchase.

Lemma 4.1 (the crowded club)**.** *Any two majorities of the same finite set intersect: if* $|Q_1|, |Q_2| > n/2$ *then* $Q_1 \cap Q_2 \neq \emptyset$*.*

Proof of Lemma 4.1. If $Q_1 \cap Q_2 = \emptyset$ then $n \geq |Q_1 \cup Q_2| = |Q_1| + |Q_2| > n/2 + n/2 = n$. □

What it buys: a majority is a quorum with *memory* — whatever one majority has done, any later majority contains a *witness* to it, who need not know what he carries. Every protocol in this chapter is an exercise in arranging for the right witness to be in the right club at the right time.

Remark 4.3 (what the proofs actually use)*.* Every argument in this chapter uses quorums only through Lemma 4.1 — pairwise intersection — never through "more than half" as such. Any *quorum system* (a family of subsets, every two of which intersect) supports the Synod verbatim; majorities are the cheapest symmetric choice. Sharper still, the two phases can use *different* families, and only cross-phase intersection is required: that refinement is Howard, Malkhi, and Spiegelman's flexible Paxos, and it is starred as Problem 4.10.

4.2 Part Two — The Derivation, Act One: Three Ways to Die

Three naive parliaments, each death kept beside its exhibit, for the deaths are the reasons the rules exist; each is the past failing to bind the future.

Attempt the first: first come, first served. Every gentleman accepts the first motion to reach him, and never another; a motion supported by a majority is chosen. Safe by the crowded club — two motions cannot both collect majorities — and dead in one honest run (Exhibit 4.1):

- **The motions:** Bingo moves free Thursday delivery; Tuppy, concurrently — merely elsewhere, as Chapter 2 taught us to say — a sixpence surcharge; Gussie, that the shop stock tea.
- **The wall:** each gentleman accepts his own motion first; the votes fall one–one–one, and every seat is spent.
- **The verdict:** no crash, no lost mail — the parliament has voted itself into a wall, permanently, by the ordinary operation of its own rule.

The lesson: acceptors must be able to change their minds, or split votes end the republic.

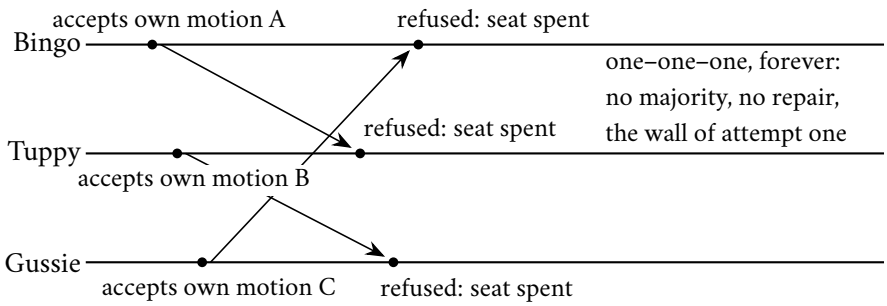


Exhibit 4.1 (the split-vote wall). Accept-once majority voting: three motions land first at their own desks; every later letter is refused; no majority ever forms and none ever can. Safe the way a sealed room is burglar-proof.

Attempt the second: latest word wins. Accept every motion, each superseding the last; let "chosen" mean a majority *currently* agrees. Bertie audits two ledgers of three for free delivery — a majority; Tuppy's dawdling surcharge letters then land, superseding; Bertie's aunt, an hour later and equally diligent, finds a majority for the surcharge. Two audits, two verdicts, each impeccable at its instant, and no future audit safe — there is no moment after which the mails are known to be empty. "Currently agrees" is a fiction in a world of letters — Chapter 2's ghost: no *the* current state, only states along cuts. Choice must be *irrevocable* — the entire content of the word. Wanted: mind-changing that is *ordered*, the old thought never displacing the new.

Attempt the second and a half: appoint a chairman. Let only Bingo propose — magnificent while Bingo stands healthy, and wrecked on Chapter 3's terrain: the chair goes silent, crashed-versus-slow is unknowable, and a successor appointed on timeout may coexist with a chairman merely slow — *two* gentlemen proposing with full authority, the rival-motions problem wearing a sash. Leadership does not solve agreement; it *presupposes* it. The house first needs machinery that stays safe *while the chairmanship is confused*; the thought returns in Part Four with the price tag attached.

Attempt the third: ballots. The object around which everything turns: a gentleman wishing to move a motion first takes a *ballot number* — totally ordered, uniquely owned — and the rule becomes: higher ballots supersede lower ones, never the reverse; a motion is chosen when a majority accepts it *at the same ballot number*. The wall of attempt one falls — a higher ballot gathers the house afresh — and a ballot number is a Lamport clock in parade uniform: a counter that only rises, carried on letters, existing to let a recipient pronounce the word *stale*. The payroll of Chapter 2 remains current.

Definition 4.4 (ballots)**.** $\mathcal{B}$ is an unbounded totally ordered set of *ballot numbers*, partitioned among proposers so no two proposers ever use the same ballot (round-robin residues suffice: with two proposers, odds and evens). Each ballot has at most one proposal (b, v), issued by b's owner.

Ballots die the most instructive death of the three (Exhibit 4.2):

- **Ballot one:** Bingo moves free delivery; the whole house accepts at one ballot — *chosen*, irrevocably, by our own definition; Bertie is informed.
- **Ballot two:** Tuppy — confirmation letters dawdling, colleagues to all appearances never having voted — grows lawfully impatient, takes ballot two, and moves his surcharge.

- **The overwrite:** every gentleman, obeying higher-supersedes-lower, accepts ballot two; the free delivery was chosen — and then *unchosen*; agreement is dead by overwrite, and nobody misbehaved.

Ballots, which order thoughts, do nothing to inform them.

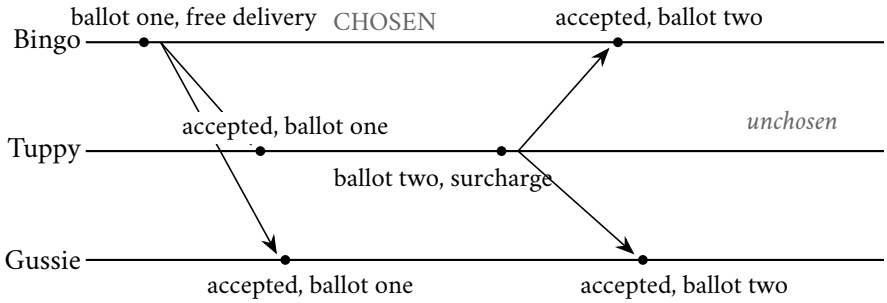


Exhibit 4.2 (the stale-ballot overwrite; ballots without promises). Free delivery is chosen at ballot one by the full house; Tuppy, ignorant, runs ballot two with a different motion, and "higher supersedes" obediently destroys the choice. A new ballot must not be free: phase one exists to make ballot two *learn*.

The lesson, in capitals: **a new ballot must not be free.** The right to propose must carry an obligation — to *learn*, first, what the house may already have done, and to be bound by it. The repair problem, precisely: any value chosen at any ballot must be the only value any higher ballot can possibly propose. Futures may ratify the past; they may never contradict it.

4.3 Part Three — The Derivation, Act Two: The Synod Stands

The repair comes in two rules; each earns its paragraph.

Repair the first: the promise. If a new ballot must learn the past, the past must hold still while being learned — Chapter 2's photograph requirement in new dress — so a ballot's life splits into two phases. Phase one, the *prepare*: the proposer of ballot b sends Box 8's phase-1a message $\langle \textsc{prepare}, b \rangle$, moving no motion yet. A gentleman who has not already dealt with a higher ballot replies with two things: the *promise* — *I shall entertain nothing below b* — persisted in his ledger before the reply leaves the desk ($promised \leftarrow b$, amnesia forbidden); and the *confession* — the highest-numbered proposal he has ever *accepted*, ballot and value both, the reply's pair (ab, av). Promises from a majority close phase one, and the promise freezes the ground under the proposer's feet: the majority that answered can no longer quietly accept a straggling lower-ballot motion.

Repair the second: adopt the highest. Phase one closed, what may the proposer move? Box 8's phase-2a value rule, the one separating everyone who has memorized Paxos from everyone who has understood it: among the confessions in hand, propose the value of the one bearing the *highest* ballot number; only if every confession in the majority is empty may the proposer move his own motion. The episode tries the alternatives on the debris run, five gentlemen for elbow room; the skeleton:

- **Ballot one:** Bingo moves the surcharge; exactly one accept letter lands, at Oofy. One acceptance of five: nothing chosen, the mere *fossil* of an ambition — *debris*: formally, a minority acceptance of an abandoned ballot.

- **Ballot two:** Tuppy, having prepared against a majority that confessed nothing (Oofy not among them), moves free delivery; his accepts land at Tuppy, Gussie, and Barmy — three of five at one ballot: *chosen*, whether or not anyone yet knows it.
- **Ballot three:** Bingo prepares against Bingo, Oofy, and Gussie; the confessions in his hand: nothing; ballot one, the surcharge; ballot two, free delivery.
- **The wrong rules on the bench:** "adopt any confessed value" lets Bingo lawfully pick the surcharge — a chosen law overwritten, dead on arrival; "adopt the plurality" resolves a one–one tie by whim — dead the same way, merely ashamed of it.

Both die because both treat confessions as opinions to be polled, when they are *strata to be read*: age is recorded in ballot numbers, not headcounts. Only the highest-numbered confession reads the strata true; Lemma 4.3 proves the rule not merely sufficient but *forced* (Problem 4.4 weighs popularity against strata).

Remark 4.5 (deference to the past is safety, not freshness)*.* The adoption rule is easily misread as a preference for recent proposals over stale ones, as though a value surviving from an abandoned ballot were spoiled goods to be discarded in favour of something fresher. It is the reverse. A proposer adopts the highest confession not because newer values are better — consensus ranks values not at all (Rem. 3.2); the sole constraint on a decision is that some process proposed it — but because a value a quorum accepted at an earlier ballot may already be *chosen*, and Agreement forbids any later ballot contradicting a choice already made. The deference therefore runs *backwards* in time: the proposer binds himself to the past precisely so as never to overturn a verdict he cannot see, and he does so even when — as in the clean run below, where the confessed value was never chosen — the past holds mere debris, for from inside the room a fossil and a law are indistinguishable, and only the cautious reading is safe against both. The ballot numbers are thus not a freshness ranking but a memory: strata to be read, not opinions to be polled, and Lemma 4.3 shows the highest-first reading to be the unique rule under which the future can only ratify. "Stale" is the wrong lens entirely — the rule does not shun old values, it *honours* the one old value that might already be law.

With it, the Synod of Paxos stands complete. The acceptor's entire estate is two persistent entries:

Definition 4.6 (acceptor state)**.** Each acceptor keeps, on stable storage: $promised \in \mathcal{B} \cup \{\bot\}$ — the highest ballot whose PREPARE it has answered; and $accepted \in (\mathcal{B} \times V) \cup \{\bot\}$ — the highest-ballot proposal it has accepted, with its value. Box 8 is the only writer of either.

The Synod end to end. Phase one: prepare; promise persisted; confession returned. Phase two: holding a majority's promises, the proposer sends accept with the value the confessions force upon him, his own only if the past is silent; each gentleman, unless promised meanwhile to a higher ballot, persists the acceptance ($accepted \leftarrow (b, v)$, before the phase-2b reply) and acknowledges. A majority accepted at one ballot: chosen, every future ballot a hostage of the past. Two phases, two rules of refusal, one rule of adoption, a ledger, and a crowded room; everything else ever written about Paxos is performance notes.

Protocol — Box 8. The Synod: single-decree Paxos (checked against "Paxos Made Simple")

Persistent acceptor state per Def. 4.6; all writes to it complete before the reply they enable is sent.

Acceptor:

```
1: upon ⟨PREPARE, b⟩ from proposer P:
2:     if promised = ⊥ or b > promised:
3:         promised ← b  (persist)
4:         reply ⟨PROMISE, b, accepted⟩  (the confession: (ab, av) or ⊥)
```

```
5:      else ignore  (or reply NACK promised, an economy)

6:  upon ⟨ACCEPT, b, v⟩:
7:      if promised = ⊥ or b ≥ promised:
8:          promised ← b;  accepted ← (b, v)  (persist both)
9:          reply ⟨ACCEPTED, b⟩  (to proposer and/or learners)
10:     else ignore
```

Proposer (owner of ballot b, motion m):

```
1:  send ⟨PREPARE, b⟩ to the acceptors
2:  collect ⟨PROMISE, b, ·⟩ replies
3:  when a quorum Q′ has replied:
4:      C ← the non-⊥ confessions in the replies
5:      if C ≠ ∅: v ← av of the highest-ab member of C
6:      else v ← m
7:      send ⟨ACCEPT, b, v⟩ to the acceptors  (phase two, once)
8:  a value is chosen when some quorum has replied ⟨ACCEPTED, b⟩ for one ballot b
```

Checked against Lamport, "Paxos Made Simple" (2001): phases 1a/1b/ 2a/2b with the same guards ($b >$ *promised* for prepare, $b \geq$ *promised* for accept — an acceptor may accept the very ballot it just promised), the same confession, the same value rule, and the paper's explicit persistence requirement. Learning schemes (§2.3 there) per Remark 4.10.

Remark 4.7 (the four messages, and the three gates). Box 8 in words, for the reader who wants the wiring without the pseudocode. *Four message types* cross the room, and no others: PREPARE(b), a proposer announcing ballot b without yet naming a value; PROMISE$(b, \cdot)$, an acceptor's reply carrying two distinct things — the promise proper (a vow about the future: *I will entertain nothing below b*) and, riding in the same envelope, the *confession* (a disclosure about the past: the acceptor's highest accepted (ab, av), or $\perp$); ACCEPT(b, v), the proposer moving the adopted value; and ACCEPTED(b), an acceptor's assent, posted to proposer and learners alike. The two names are not two words for one thing: a vow about the future is not a report of the past, and they vary independently — an acceptor may promise while confessing $\perp$. (The *confession* is our name for what Lamport's *Part-Time Parliament* calls the LastVote message; "promise" for the phase-1b reply is standard usage.) *Three gates* decide everything. The proposer's: he advances from phase one to phase two only while holding PROMISES from a whole quorum — short of a quorum he does not proceed, but abandons b and retries at a higher ballot. Each acceptor's two, differing by a hair that matters: he grants a PROMISE iff the offered ballot stands *strictly above* his last ($b >$ *promised*), and he ACCEPTS iff it stands *no lower* ($b \geq$ *promised*) — so an acceptor may accept the very ballot it has just promised, and neither gate ever opens downward. A value is chosen the instant a quorum has passed the third gate for one ballot; nothing announces it (Rem. 4.2).

The safety argument, written against the boxed lines. First the bookkeeping: one ballot, one proposal.

Lemma 4.2 (one value per ballot)**.** *For each ballot b, at most one proposal (b, v) is ever issued, and every acceptance at b is of that proposal.*

Proof of Lemma 4.2. Ballot b belongs to a single proposer (Def. 4.4), which issues at most one phase-2a message per ballot it owns (Box 8, proposer line 8: phase two is entered at most once per prepared ballot, with one value fixed there). Acceptors accept at b only the proposal (b, v) delivered to them (Box 8, acceptor lines 8–10), by no-forgery the one issued. □

Then the invariant — futures may only ratify — its skeleton in words: the phase-one quorum of the higher ballot meets the choosing quorum of the lower; the shared witness accepted *before* promising (his promise would otherwise have barred that acceptance), so his confession is numbered at least b; by induction, every confession numbered $\geq b$ carries the chosen value; adopt-the-highest does the rest. Step 1 — the promise-ordering subtlety — is the hinge; Problem 4.6 completes it as a standalone lemma.

Lemma 4.3 (the invariant: futures may only ratify)**.** *In any execution of Box 8: if the value v is chosen at ballot b, then every proposal issued at any ballot $b' > b$ has value v.*

Proof of Lemma 4.3. Fix the execution and let v be chosen at ballot b via the quorum Q: every $a \in Q$ accepted (b, v). We prove by strong induction on $b' > b$: *every proposal issued at b' has value v.*

So let (b', w) be issued, and assume the claim for all ballots in (b, b'). The proposer of b' entered phase two holding PROMISE replies for b' from a quorum Q' (Box 8, proposer lines 4–7). By Lemma 4.1 pick a witness $a \in Q \cap Q'$.

Step 1 (the ordering subtlety). At the moment a answered $\langle \text{PREPARE}, b' \rangle$, it had already accepted (b, v). For suppose not: then a's acceptance of (b, v) came after its promise. But answering the prepare set *promised* $\geq b'$ (Box 8, acceptor line 4), the variable is persistent and non-decreasing (its only assignment raises it, and recovery restores it), and the acceptance guard (acceptor line 8) requires $b \geq \textit{promised} \geq b' > b$ — absurd. So the acceptance preceded the promise. (This step is why the promise is persisted *before* the reply: a crash between reply and write re-opens exactly this door; Problem 4.7.)

Step 2 (the confession's floor). Therefore a's reply to b' confessed its then-highest accepted proposal (ab, av) with $ab \geq b$. Its ceiling: $ab < b'$, since a confessed acceptance is an acceptance, and a could not accept any ballot $\geq b'$ before promising b' (the acceptance guard again, applied at the earlier time). So some confession in the proposer's hands is numbered in $[b, b')$; in particular the *highest* confession (ab^*, av^*) across Q' satisfies $b \leq ab^* < b'$.

Step 3 (the strata read true). Every confession numbered in $[b, b')$ carries value v: a confession (ab, av) is an acceptance of the proposal issued at ab (Lemma 4.2); if $ab = b$ that proposal is (b, v); if $b < ab < b'$ the induction hypothesis gives its value as v. Hence $av^* = v$.

Step 4 (adoption). Box 8's value rule (proposer line 6) sets $w = av^*$ whenever any confession is non-empty — and Step 2 produced one. So $w = v$, completing the induction. □

The reference trace. Exhibit 4.3 — the trace the Tier-I calisthenics compute on; the steps:

- **The fossil:** the house of three; Gussie's ledger holds, from an abandoned early skirmish, accepted $(2, \text{S})$; nothing was ever chosen.
- **Phase one:** Bingo opens ballot five; Tuppy promises with nothing to confess; Gussie promises and confesses $(2, \text{S})$; Bingo's own ledger answers likewise.
- **Adoption:** the rule reads the room and Bingo's own ambitions go in the drawer — though *nothing was ever chosen*: the rule cannot see whether a confession was chosen, so it treats every confessed value as *possibly* chosen and defers to the highest — cheap deference, insurance against a catastrophe no one can rule out from inside the room.
- **Phase two:** accept, ballot five, S; Tuppy and Gussie persist it; a majority at one ballot — S is *chosen*, and Bertie may be told something that will never afterwards be untrue.

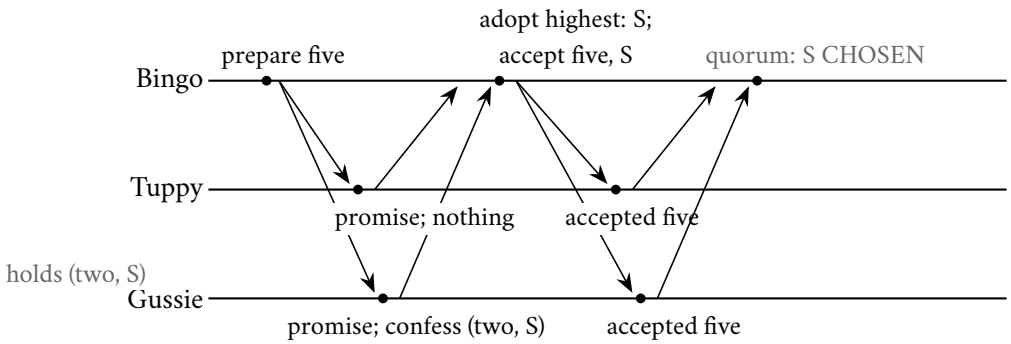


Exhibit 4.3 (a clean Synod run; the reference trace). Gussie's ledger holds a fossil — accepted, ballot two, motion S, never chosen. Bingo's ballot five must adopt it anyway: the rule cannot see whether a confession was chosen, so it defers to the highest. Cheap deference, certain safety. Tier-I calisthenics compute on this trace.

The shape of the finished machine: promises bind the past against revision from below; adoption binds the future against invention from above. Once chosen, always chosen — not because any gentleman remembers the choosing, but because the choice is held by the *structure*: lodged in the intersection of clubs, carried by witnesses who do not know they are witnesses.

4.4 Part Four — The Fine Print

Three clauses and two footnotes, each priced.

Clause one: unconditional safety. The argument above invoked no clocks, no timeouts, no probabilities, and no luck: pure bookkeeping about ballots and majorities, valid under any schedule, any pattern of crashes and recoveries the ledgers survive — delivered in the asynchronous model where Chapter 3's theorem lives. The theorem is not contradicted; what was quietly never promised is that any ballot ever *finishes*. The audit is two lines: two chosen values collide either within one ballot (impossible: one proposer, one proposal per ballot) or across ballots (impossible: the invariant).

Theorem 4.4 (Synod safety)**.** *In every execution of Box 8 under the model box — any schedule, any crashes and recoveries — at most one value is chosen, and any chosen value is some proposer's motion.*

Proof of Theorem 4.4. Agreement. Suppose v_1 is chosen at b_1 and v_2 at b_2, with $b_1 \le b_2$. If $b_1 = b_2$: both are acceptances at one ballot, so $v_1 = v_2$ by Lemma 4.2. If $b_1 < b_2$: being chosen at b_2 requires acceptances of the proposal issued at b_2 (Lemma 4.2), whose value is v_1 by Lemma 4.3; so $v_2 = v_1$.

Validity. By induction on ballots: the value proposed at any ballot is either the proposer's own motion or, by the adoption rule, the value of some confessed — hence earlier-proposed — proposal; the chain of adoptions descends strictly in ballot number and terminates at a motion. □

Clause two: liveness, purchased separately. The purchase is compulsory; the failure, staged beside Exhibit 4.4:

- **Prepare three:** Bingo prepares; the house promises.
- **Prepare four:** before Bingo's accepts land, Tuppy — hearing nothing — prepares; the house, obedient to its promises, is closed below four; Bingo's accepts are politely refused.

- **Prepare five, six, …:** Bingo, superseded, prepares five; Tuppy's accepts die against five; Tuppy prepares six — the rhythm is audible.

Two flawlessly polite gentlemen, each forever cancelling the other's phase two: the *duelling proposers*, formally the non-terminating prepare-interference execution — Chapter 3's forever-on-the-fence as an operational hazard with names attached. An epoch counter ratcheting upward at a steady, fruitless clip is this minuet on the monitoring, a 1985 theorem calling the tune.

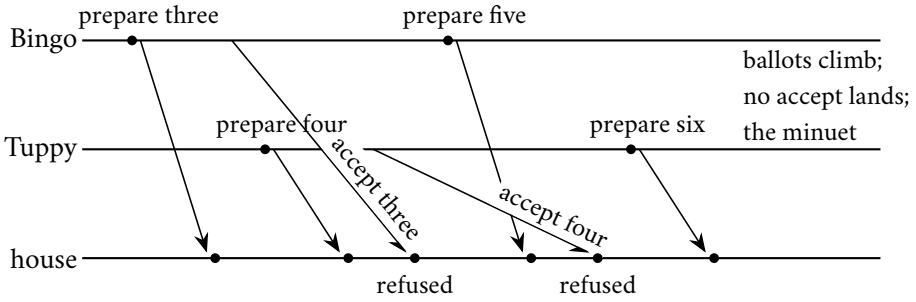


Exhibit 4.4 (the duelling proposers). Each prepare voids the rival's pending phase two; the house (drawn as one line for the quorum's behaviour) refuses each accept against its newer promise. An admissible non-terminating execution — Theorem 3.4 walking the production floor. The cure is a chair, not a rule: Remark 4.8.

The escape is the gate Chapter 3 licenses — a *distinguished proposer* — priced in the remark below; its close is the most quoted moral of the chapter: the leader is a liveness optimization; safety lives in the quorums.

Remark 4.8 (liveness, purchased separately). No execution of Box 8 violates safety, but Exhibit 4.4's duel — two proposers alternately preparing ever-higher ballots, each voiding the other's phase two — is an admissible non-terminating execution, as Theorem 3.4 requires some to be. The standard purchase: a *distinguished proposer* (elected by timeout, held by heartbeat — an implementation of the eventual-leader oracle Ω, Chapter 3's third gate, here cashing its weakest-detector cheque), plus partial synchrony so the distinction eventually sticks. Then phase one succeeds once and phase two repeats — the multi-decree amortization (multi-Paxos) taken up in Chapter 5. Two simultaneous believers in the chair cost throughput, never safety: stale ballots die against promises.

Clause three: the ledgers. Every load-bearing word above has been *written* — the promise before the reply, the acceptance before the acknowledgment — and this is the failure model's bill, not implementation fussiness. A gentleman who promises ballot four, naps, and wakes with an empty head will cheerfully accept ballot three — the very acceptance his promise forbade — and two such nappers manufacture two chosen values in an afternoon. "Because it is rebuilt by traffic anyway" is a sentence heard in production reviews; Problem 4.7 stages it, certified fatal. Amnesia is not a degraded mode of the protocol; it is a different protocol, and a wrong one.

Remark 4.9 (persistence is load-bearing). Theorem 4.4's proof reads the acceptor's *promised* and *accepted* across crashes; if either reverts on recovery, Lemma 4.3's ordering argument dies and two values can be chosen in one afternoon — Problem 4.7 manufactures exactly that, and Exhibit 4.8 certifies it. The write precedes the reply; there is no lighter correct discipline.

Footnote one: the learners. Chosen and *known to be chosen* are different events — in the debris run, free delivery was law while every gentleman believed the matter open. Choice lives in the ledgers collectively; knowledge requires collation.

Remark 4.10 (learning; a read is a ballot)*.* Choice and knowledge of choice are separate events. Acceptors report acceptances to learners (or to the chair, who announces); a learner holding a quorum's acceptances at one ballot holds a certificate. A cold-start learner must not trust a bare poll of ledgers — it may observe a mid-ballot mixture (the Chapter-2 auditors' sin in new dress); the clean method is to run a ballot: prepare, adopt the highest, propose it, and read what the house confirms. Mark precisely what that ballot certifies, for it is a common slip: a phase-one sweep returns only the *candidate* the adoption rule would force — the highest confession, which may be mere debris never chosen (the clean run's lone fossil) — so a confession proves *candidacy*, not choice. Only a quorum's ACCEPTED at one ballot certifies that a value is chosen; learning is therefore a *completed* ballot, not a solicitation of promises. The cost of reads through consensus, and what leases may lawfully shortcut, is Chapter 5's business.

Footnote two: the invoice. Two round trips of the Post per decision, each touching a majority, plus a ledger write on every gentleman at each step — extortionate for a parliament deciding by-laws all day, which is what a replicated database is. The remedy is the chairman now lawfully earned: a *stable* leader runs phase one *once* — one standing prepare, a majority's standing promises — and thereafter each decision costs a single accept round. That arrangement — the Synod with a standing chairman and a numbered docket of decisions — is multi-Paxos, what every production deployment actually runs, and the door at which Chapter 5 knocks.

4.5 Part Five — Raft, or Understandability as an Engineering Requirement

Twenty-odd years on, Paxos wins in production and simultaneously acquires a folklore: fiendish, never implemented correctly from the papers. Some of that was costume; some real — composing the small Synod into a practical replicated log leaves load-bearing decisions as exercises for the implementer, and implementers' exercises become outages.

The premise. In 2014 Diego Ongaro and John Ousterhout published, at the USENIX technical conference, a protocol called Raft, built on a premise the field found mildly scandalous: *understandability is an engineering requirement* — operators debug at three in the morning what they can hold in their heads. They measured it: a formal user study, students quizzed on both protocols, Raft's comprehension scores materially higher, the quizzes published. The method is the transferable part: decompose into nearly independent concerns — leader election, log replication, safety, membership — so each fits in a head separately; prefer one strong leader, because asymmetry is narratable; shrink the state space wherever a rule simplifies at modest cost. In one line: if operators cannot re-derive your invariants at three in the morning, your invariants are decorative.

The furniture is this chapter's machinery in working clothes; every rule in Box 9 is a comparison over exactly these parts. Time divides into *terms* — a term is a ballot wearing a calendar, the Lamport clock drawing yet another salary — and "up to date" is the log-ending order $\preceq$ fixed now.

Definition 4.11 (Raft vocabulary)**.** Time divides into *terms* $1, 2, \ldots$; each term holds at most one leader. At every moment each server occupies exactly one of three *offices*: *follower* (passive — answers letters, never initiates), *candidate* (a follower whose election timeout expired, soliciting votes), or *leader* (a candidate who

won; all client business flows through it). All begin as followers. Each server persists *currentTerm*, *votedFor* (at most one vote per term), and a *log* of entries; the entry at index i carries the term in which a leader created it. Volatile, rebuilt after a crash: *commitIndex*, the highest index the server knows committed. An entry is *committed* when it is durable on a quorum *and* the commit rule of Box 9 licenses the announcement (the distinction is Problem 4.8). For logs L, L' with final entries of terms t, t' and lengths ℓ, ℓ': $L \preceq L'$ (L' is *at least as up to date*) iff $t < t'$, or ($t = t'$ and $\ell \leq \ell'$). A voter grants its vote only to candidates whose log is at least as up to date as its own.

The term rule. One rule governs every letter before any handler reads its contents. Every Raft message, in both directions, carries its sender's *currentTerm*; the recipient compares first. Higher: adopt the term, clear the vote — a fresh term is a fresh slate — and become a follower, whatever the office held; a candidate abandons the candidacy, a leader the gavel. Lower: refuse the letter unread, the reply carrying the recipient's own term so the sender may stand down. This one comparison retires a stale chairman without ever establishing that he died: the deposed leader deposes himself the moment anybody addresses him in the language of a later reign. It stands first in both handlers of Box 9, and it is half of what keeps Theorem 4.7's witness armed.

The essentials, boxed against the paper's own summary figure — twice over: first the rules in summary, then, in Box 9b, the handlers stepped:

Protocol — Box 9. Raft essentials (checked against the 2014 paper's Figure 2)

Persistent per server, written before any reply leaves the desk: *currentTerm*, *votedFor*, *log*. Volatile, rebuilt after a crash: *commitIndex*; on a leader, per follower: *nextIndex* (the first slot to try) and *matchIndex* (the highest slot known replicated there).

1: **offices:** every server is exactly one of follower, candidate, leader; all begin as followers
2: **the term rule, on every message in either direction, before its contents are read:** a term above *currentTerm* is adopted at once — persist it, clear *votedFor*, become a follower, whatever the office held; a term below *currentTerm* is refused, the reply carrying *currentTerm* so the sender stands down
3: **election:** a follower hearing no AppendEntries for its randomized timeout increments *currentTerm*, votes for itself (persist), becomes a candidate, and sends every peer ⟨RequestVote, *currentTerm*, *lastLogIndex*, *lastLogTerm*⟩ — the ending of its own log
4: **upon** ⟨RequestVote, t, candidate log ending⟩, the term rule having run:
5: grant iff *votedFor* $\in \{\perp,$ candidate$\}$ **and** own log $\preceq$ candidate log (Def. 4.11); persist the vote before the reply leaves
6: **an election ends in one of three ways:** a quorum of grants (the candidate's own vote counted) makes it leader of the term; an AppendEntries bearing an equal or later term makes it a follower again; or its timeout expires once more — a split vote — and it stands afresh in the next term
7: **leader:** on a client motion, append to own log at the next index, stamped *currentTerm*; send each follower, from that follower's *nextIndex*: ⟨AppendEntries, *currentTerm*, *prevIndex*, *prevTerm*, *entries*, *commitIndex*⟩; empty *entries* at a steady cadence is the *heartbeat*, carrying the other fields all the same
8: **upon** ⟨AppendEntries, t, *prevIndex*, *prevTerm*, *entries*, *leaderCommit*⟩, the term rule having run:
9: **consistency check:** refuse unless own log holds an entry of term *prevTerm* at index *prevIndex*
10: **else:** delete any suffix conflicting with *entries*; append what is new (persist); set

commitIndex ← min(*leaderCommit*, index of last new entry); acknowledge

11: **repair:** on a refusal the leader decrements that follower's *nextIndex* and retries, walking backward until the ledgers meet, then shipping its own log forward; **leader append-only:** it never deletes or overwrites an entry of its own log

12: **commit rule:** the leader advances *commitIndex* to *i* only when (a) *matchIndex* ≥ *i* at a quorum, its own log counted, **and** (b) the entry at *i* carries *the leader's own current term*; earlier-term entries commit only transitively, beneath such an *i*

13: **apply:** every server applies committed entries to its state machine in index order as its *commitIndex* advances; the leader answers the client upon applying the motion

Checked against Ongaro and Ousterhout (2014), Figure 2 and §§5.1–5.4: the offices and the term rule, the vote guards, the consistency check, conflict-suffix deletion, the commit-index advance by *leaderCommit*, the up-to-date restriction, and the §5.4.2 commitment restriction — clause (b) of the commit rule — whose necessity is Problem 4.8.

The same constitution once more, at the grain of Box 8 — one move per line, every guard and write explicit, the persistence points marked. Box 9 is for review; Box 9b is for implementation; they are one protocol, and the walkthroughs that follow pace its two halves in prose.

Protocol — Box 9b. Raft, stepped: the handlers in full (checked against the same Figure 2)

State per Box 9's headnote, with the one ledger line Figure 2 carries and the summary elides: volatile *lastApplied* — the highest index yet applied to the state machine, initially zero, never above *commitIndex*. As throughout: every persistent write completes before the reply it enables leaves the desk.

Every server (the term rule, then the apply loop):

1: **upon** any request or reply bearing term *t*, before its contents are read:

2: **if** *t* > *currentTerm*: *currentTerm* ← *t*; *votedFor* ← ⊥ (persist both); become follower; then handle the message below

3: **if** *t* < *currentTerm*: refuse a request, the reply carrying *currentTerm* and *false*; discard a stale reply

4: **whenever** *commitIndex* > *lastApplied*: *lastApplied* ← *lastApplied* + 1; apply *log*[*lastApplied*] to the state machine (index order, no gaps)

Follower:

1: **upon** its randomized election timeout — re-armed by each AppendEntries from the sitting leader and by each vote it grants: become candidate and stand (next block)

Candidate (standing at a fresh term):

1: *currentTerm* ← *currentTerm* + 1; *votedFor* ← self (persist both); re-arm the election timer

2: send every peer ⟨RequestVote, *currentTerm*, *lastLogIndex*, *lastLogTerm*⟩ — the ending of its own log

3: **upon** replies ⟨*voteGranted* = true⟩ at *currentTerm* from a quorum, its own vote counted: become leader (its block below)

4: **upon** an AppendEntries at *t* = *currentTerm* (a higher *t* deposed it already, at the term rule): become follower and handle the letter

5: **upon** election timeout again — a split vote — stand afresh from line 1, in the next term

Any office, on RequestVote (the voter's guards):

1: **upon** ⟨RequestVote, $t, C, lastLogIndex, lastLogTerm$⟩ from candidate C, the term rule having run ($t = currentTerm$):
2: **if** $votedFor \in \{\bot, C\}$ **and** own log $\preceq$ the candidate's ending (Def. 4.11: term before length):
3: $votedFor \leftarrow C$ (persist, before the reply leaves); re-arm the election timer
4: reply ⟨$currentTerm, voteGranted$ = true⟩
5: **else** reply ⟨$currentTerm, voteGranted$ = false⟩ (the vote is not spent; the timer not re-armed)

Leader:

1: **on accession:** $nextIndex[f] \leftarrow lastLogIndex + 1$; $matchIndex[f] \leftarrow 0$, for every peer f (volatile); send the inaugural empty AppendEntries at once, and again each heartbeat interval, idle or not
2: **upon** a client motion m: append m at the next free index, stamped $currentTerm$ (persist); the answer waits for line 7
3: **whenever** $lastLogIndex \geq nextIndex[f]$ for a peer f, and on each heartbeat: send f ⟨AppendEntries, $currentTerm, prevIndex, prevTerm, entries, commitIndex$⟩ with $prevIndex = nextIndex[f] - 1$, $prevTerm = log[prevIndex].term$, $entries = log[nextIndex[f] \ldots]$ (empty on a bare heartbeat)
4: **upon** ⟨$success$ = true⟩ from f, entries through index i written: $matchIndex[f] \leftarrow i$; $nextIndex[f] \leftarrow i + 1$
5: **upon** ⟨$success$ = false⟩ from f — the consistency check refused: $nextIndex[f] \leftarrow nextIndex[f] - 1$; retry (the backward walk; the leader's own log is never rewritten — leader append-only)
6: **whenever** some $i > commitIndex$ has $matchIndex \geq i$ at a quorum, own log counted, **and** $log[i].term = currentTerm$: $commitIndex \leftarrow i$ (clause (b): the sitting term only; earlier-term entries commit beneath such an i)
7: the client is answered after its motion is applied (the apply loop above)

Any office, on AppendEntries (the consistency check and the write):

1: **upon** ⟨AppendEntries, $t, prevIndex, prevTerm, entries, leaderCommit$⟩, the term rule having run ($t = currentTerm$): re-arm the election timer (the letter is the sitting chair's)
2: **if** own log holds no entry of term $prevTerm$ at index $prevIndex$: reply ⟨$currentTerm, success$ = false⟩ (the consistency check; nothing written)
3: **else:**
4: delete the suffix conflicting with $entries$ — same index, different term — and append what is new (persist)
5: **if** $leaderCommit > commitIndex$: $commitIndex \leftarrow$ min($leaderCommit$, index of last new entry)
6: reply ⟨$currentTerm, success$ = true⟩

Checked, clause for clause, against the same Figure 2: both receiver implementations (conflict-suffix deletion and the min included), and the Rules for Servers for all four roles — the apply loop with its *lastApplied* ledger restored, the accession initializations, the inaugural heartbeat, the backward walk, and the §5.4.2 commitment restriction, clause (b) of the commit rule. Two glosses are the desk's own: the randomized draw of the timeout is the paper's §5.2, filed by Figure 2 under its margins; and the discard of stale replies is hygiene the figure leaves tacit.

The election, stepped. Each term opens with one; the moves:

- **Standing:** a follower hearing nothing from the chair for his *election timeout* — the purchased verdict it always is — increments *currentTerm*, votes for himself (persisted), becomes a candidate, and writes every peer a RequestVote carrying the new term and the *ending* of his own log.
- **The voter's three questions, in order:** the term rule first; then the vote — *votedFor* already spent this term, refuse; then the ending — grant only to a candidate log at least as up to date as the voter's own ($\preceq$ of Def. 4.11, term before length), the veto on which committed law will shortly hang. The vote is persisted *first*, then granted — amnesia forbidden: a gentleman who forgets his vote across a crash manufactures two leaders with his own hand.
- **Three endings:** a quorum of grants (his own vote counted) makes him leader of the term, announced by writing to everybody at once; a letter from a leader of his term or later returns him to the ranks; or nobody wins — a *split vote* — and the house stands afresh in the next term. Raft would sooner waste a term than risk two leaders in one.
- **Holding the house:** *heartbeats* — empty replication letters at a steady cadence — suppress elections (a follower who hears from the chair has no cause to stand) and carry the leader's term, quietly deposing any stale claimant in earshot.
- **Dissolving duels:** *randomized* election timeouts — each gentleman draws his patience from a range, so someone almost surely collects his quorum before a rival stirs. Chapter 3's pinch of randomness: the storm still possible, embarrassingly improbable per round.

And the crowded club yields the first of the paper's five warranty clauses in the time it takes to say it:

Lemma 4.5 (Raft election safety)**.** *At most one server becomes leader in any given term.*

Proof of Lemma 4.5. A leader of term t collected granted votes for t from a quorum. Each server persists *votedFor* for term t before granting (Box 9), and grants at most once per term. Two leaders of term t would give two quorums whose shared member (Lemma 4.1) granted twice. Contradiction. □

The remaining clauses — *leader append-only* (a leader never deletes or overwrites its own log), log matching, leader completeness, and, resting on the others, state-machine safety — belong to Raft's real subject: the *log*, the numbered ledger of laws that Chapter 5 makes the star.

The replication letter. An AppendEntries carries four items, one of which does the work of the protocol: the leader's term; the entries being sent, if any; the stamp — index and term — of the slot *immediately preceding* them, the subject of the *consistency check*; and the leader's *commitIndex*. The follower's rule: look up the named predecessor slot in his own log before writing anything. An entry there of that index and term: leader and follower agree at least that far back — accept, write, acknowledge. Anything else — slot empty, or a different term: refuse, without arguing and without proposing amendments.

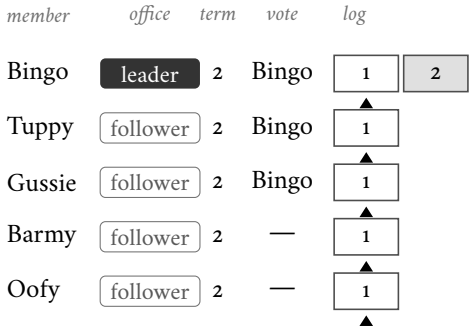
The repair, stepped. On a refusal the leader walks backward until the two logs meet, then ships his own version forward; followers never negotiate. The worked instance:

- **The probe:** the leader of term four, believing Tuppy matched through slot seven, writes: *here is slot eight; the slot before it, slot seven, bears term four.*
- **The refusal:** Tuppy finds term two at his slot seven — residue of an earlier reign — and refuses.

- **The walk:** the leader decrements Tuppy's *nextIndex* and retries: *slots seven and eight; before them, slot six, term four*. The check passes.
- **The overwrite:** Tuppy deletes the conflicting entry at slot seven and writes the leader's slots seven and eight in its place.

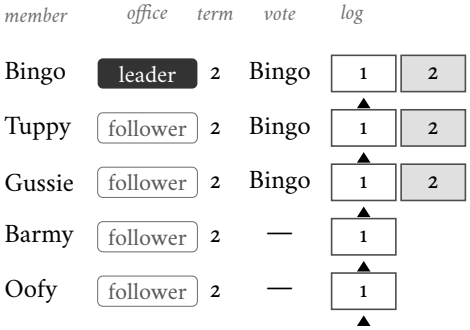
A slot replicated on a majority becomes *committed* — chosen, in this chapter's vocabulary — under the commit rule whose necessity Barmy shortly demonstrates. The announcement's plumbing: the leader, counting a quorum of acknowledgements (*matchIndex*) for an entry of his own term, advances *commitIndex*, applies the motion, and answers the client; followers learn from the *leaderCommit* field of the very next letter, an empty heartbeat sufficing — the leader knows one letter before the house does, a permanent asymmetry and not a defect. Exhibit 4.5 photographs the happy case, one panel per beat.

beat one: the leader appends and posts



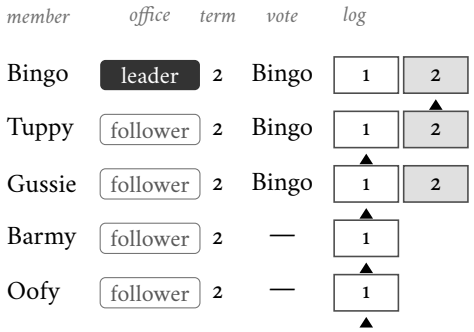
Bertie's motion reaches the chair alone. Bingo appends (2 : 2) and posts append-entries to all four: term 2; the entry; predecessor stamp slot 1, term 1; commit index 1.

beat two: the check passes at two desks



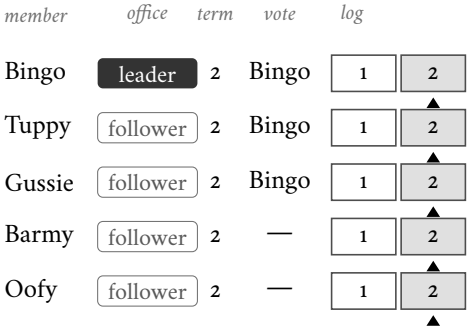
Tuppy and Gussie find term 1 at slot one: the check passes — write, acknowledge. The letters to Barmy and Oofy dawdle in the Post.

beat three: a quorum at the sitting term



Two acknowledgements plus his own desk: a quorum at the sitting term. Bingo advances his commit index, applies, and answers Bertie; the house does not know yet.

beat four: the commit index travels



The next heartbeat carries commit index 2; the dawdling letters land at last. Five identical logs, every triangle at slot two.

Exhibit 4.5 (the happy case, desk by desk). Filmstrip conventions, holding through Exhibit 4.7: each panel narrates, beneath its grid, the moves of one beat, and the grid photographs the whole house *after* those moves have completed — the state the beat leaves behind. Per member: the office (dashed: paused or crashed), the current term, the vote cast in that term (— while unspent), and the log, one cell per slot, the numeral being the term stamped in the entry. One shaded motion appears here: the light cells are the term-two motion, the only law in play in the happy case. A darker shade is reserved for the rival term-three motion, which enters with the figure eight of Exhibit 4.6 — Problem 4.8's staging reuses this opening scene. The filled triangle beneath a cell is that member's commit index; a hollow triangle, a commit index advanced by the sabotaged counting rule. Here every letter lands, and one round trip commits.

The discipline underneath is log matching, maintained by the consistency check at every single write — an induction that never starts wrong.

Lemma 4.6 (log matching; Ongaro–Ousterhout)**.** *If the logs of two servers contain entries with the same index and the same term, then the entries are identical and so are the two logs at every lower index.*

Proof of Lemma 4.6. Both claims by induction on the appending history. A leader creates at most one entry per (index, term) pair — it assigns consecutive indices within its term and never overwrites its own log (leader append-only, Box 9) — so equal (index, term) implies the same created entry. For the prefix claim: a follower installs the entry at index i with term t only through an AppendEntries message whose consistency check it passed — the message carries the (index, term) of the predecessor entry, and the follower refuses unless its own log matches at index $i-1$ (Box 9). Inductively the leader's and follower's logs agree through $i-1$ at installation time, and any later divergence at or below i would require an overwrite, which followers perform only by truncating a conflicting suffix and re-installing under the same check. □

The up-to-date veto. The check's elegance breeds a trap: followers' unreplicated tails can be overwritten, and an election can crown a gentleman whose ledger lacks another's recent scribbles. The voting restriction of Def. 4.11 closes it, and the safety argument is the Synod's wearing electoral dress: a committed slot lives on a majority; a winning candidate persuaded a majority; the clubs intersect; and the shared voter's vote was conditional — granted only to a ledger at least as up to date as his own, which a ledger missing his committed slot cannot be. The crowded club supplies the witness, the up-to-date rule arms him: where the Synod's proposer *reads* the past out of a majority and adopts it, Raft's electorate refuses any future that has not already done its reading. The full induction, structured after Ongaro's thesis:

Theorem 4.7 (leader completeness)**.** *If a log entry is committed (per Box 9's rule) in term t, then that entry is present in the log of the leader of every term $t' > t$.*

Proof of Theorem 4.7. Structured after Ongaro's thesis. Let the entry e (index i, term t) be committed in term t: the leader of term t counted e durable on a quorum Q *within term t* (Box 9's rule). Induct on $t' > t$, assuming the leaders of all terms in (t, t') — those that existed — held e.

The leader L' of t' won votes from a quorum Q'; pick a witness $a \in Q \cap Q'$ (Lemma 4.1). The witness stored e before voting: it appended e in term t, and between t and its vote in t', no entry it held at index i was ever displaced — displacement requires an AppendEntries from some leader of an intermediate term, every such leader held e at index i by the induction hypothesis, and log matching (Lemma 4.6) makes their transmissions at index i be e itself.

Now the veto arithmetic. When a voted for L', the up-to-date comparison (Def. 4.11) held: a's log $\preceq$ L''s log. Suppose toward contradiction L' lacks e. Case (i): L''s final term equals t. Then L''s final entry was created by the leader of term t (Lemma 4.6's first claim), i.e. L''s log is a subsequence of that leader's log truncated at some length; lacking e means L''s log is shorter than $i \le$ the length of a's log, whose final term is at least t — if a's final term is exactly t, the length comparison fails ($\ell_a \ge i > \ell_{L'}$), and if it exceeds t, the term comparison already fails; either way a's log $\not\preceq L''$s. Case (ii): L''s final term is less than t: then since a's final term is at least t, again a's log $\not\preceq L''$s. Case (iii): L''s final term t'' lies in (t, t'): the entry ending L''s log was created by the leader of t'', which by the induction hypothesis held e; by log matching, any log containing an entry created in t'' at an index $\ge i$ agrees with that leader's log through that index and so contains e — if L''s final index is $\ge i$, L' holds e after all; if it is $< i$... it cannot be, for the leader of t'' appends after its own copy of e, so every entry it creates sits at an index $> i$. All cases contradict; L' holds e. □

And the clause the tenant actually buys:

Corollary 4.8 (state-machine safety)**.** *If any server has applied the entry at index i to its state machine, no server ever applies a different entry at index i.*

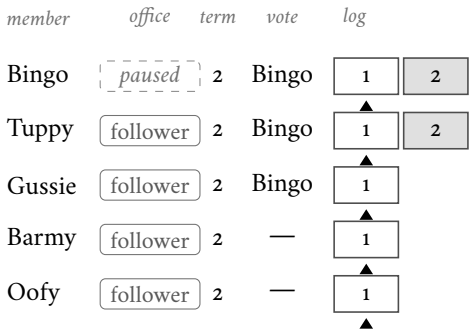
Proof of Corollary 4.8. A server applies index i only once i is committed within its leader's announcement chain; by Theorem 4.7 every later leader's log contains the committed entry at i, and by Lemma 4.6 any server's log that ever carries a committed prefix carries the identical entries there; two different applications at i would exhibit two committed entries at i, i.e. two leaders of some terms both completing announcements for conflicting entries, contradicting Theorem 4.7 applied to the earlier commitment. □

The figure eight. One trap remains, the paper's most famous figure; the ledger-by-ledger reconstruction is Problem 4.8, certified fatal in the solutions and drawn frame by frame in Exhibits 4.6 and 4.7. The rule it justifies — clause (b) of Box 9's commit rule: **a leader may count replicas toward commitment only for slots of his own term** — never for an inherited slot, however widely replicated. The staging, one term per beat:

- **Term two:** chairman Bingo writes a motion into his second slot and replicates it to Tuppy alone — Gussie's copy dawdles — before going quiet, a pause of the garbage-collecting kind.
- **Term three:** Barmy gathers votes from Gussie and Oofy, whose short ledgers see nothing wrong with his (Tuppy, holding the term-two slot, would refuse — but two votes and his own make the majority); Chairman Barmy writes his pet motion into *his* second slot, stamped term three, and crashes, having replicated it to no one.
- **Term four:** Bingo wakes, stands, and wins (Tuppy, Gussie, his own vote), then resumes his term-two business and replicates the old second slot to Gussie: three ledgers of five, durable on a majority — the naive rule pronounces it committed; Bertie is informed.
- **Term five:** Bingo pauses; Barmy recovers and stands; his ledger ends with a term-three slot, his voters' with term-two slots — later term wins the comparison, and Gussie and Oofy have no lawful grounds for refusal. Barmy replicates his term-three slot to the whole house; log matching does the rest; the motion a chairman counted to a majority and pronounced law is expunged from every ledger on the island.

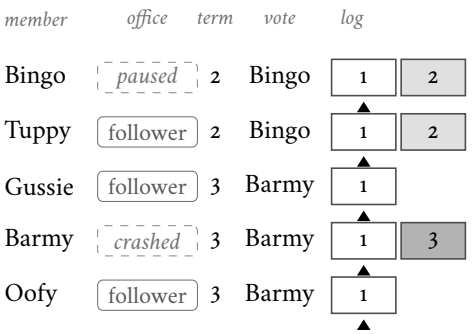
The counting was the crime: replication on a majority made the slot *durable*, but elections protect committed law by comparing *terms*, and an inherited slot's term is old news — durability is a fact about copies; commitment is a claim about elections yet to come. Under the lawful rule the trap fails to spring: Bingo in term four first commits a fresh term-four motion — a no-operation will do — log matching drags the inheritance into safety underneath, and Barmy's second candidacy dies at Gussie's veto, exactly where Theorem 4.7's witness stands. The deepest sentence in the paper: *replication makes a slot durable; only the sitting term's replication makes it committed.*

term two: the old motion strands



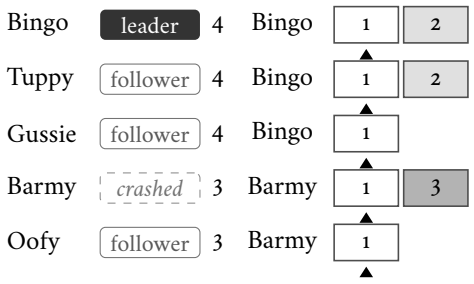
Bingo, leading term 2, appends (2 : 2) and replicates: Tuppy's copy lands; Gussie's dawdles in the Post; Barmy and Oofy see nothing. Then the chair goes quiet.

term three: elected, one scribble, crashed



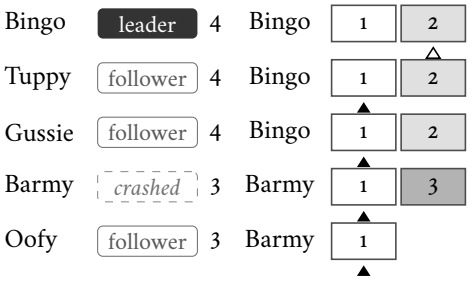
Barmy stands in term 3, soliciting Gussie and Oofy: their endings (1 : 1) match his own — granted. Tuppy, who would refuse, is not asked. Barmy appends (2 : 3) and crashes.

term four: Bingo restored



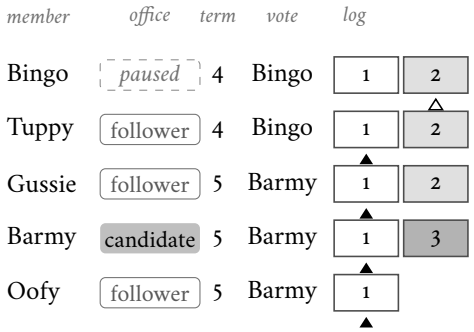
Bingo wakes, none the wiser, and stands in term 4. Tuppy: equal ending, equal length — granted. Gussie: (1 : 1) is behind — granted. Three of five; Bingo resumes his term-2 business.

term four: the fatal count



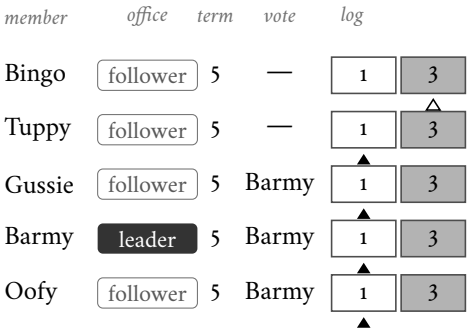
Bingo replicates (2 : 2) to Gussie: the term-2 entry now sits on three desks of five. The sabotaged rule counts across terms and pronounces it committed; Bertie is told.

term five: the compelled votes



Bingo pauses again; Barmy recovers and stands in term 5. His ending (2 : 3) beats (2 : 2) and (1 : 1) — later term wins, whatever the length — so Gussie and Oofy must grant. Two votes and his own.

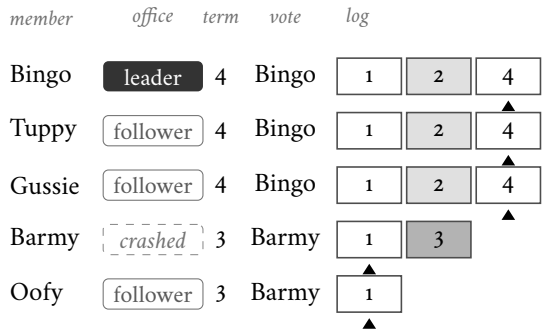
term five: the overwrite



Barmy replicates (2 : 3); the check passes at slot one; conflicting suffixes are deleted. The "committed" entry survives on no desk — and Bingo has applied a law his log no longer holds.

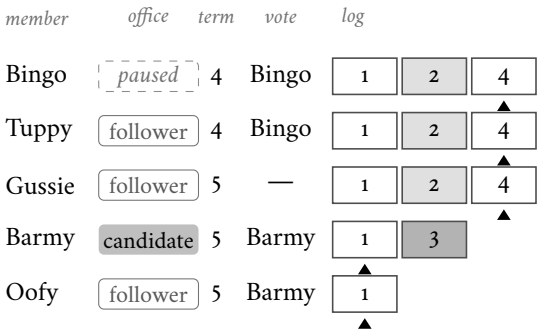
Exhibit 4.6 (the counting chairman's figure eight; the staging of Problem 4.8). Six panels, terms two through five, conventions as in Exhibit 4.5; each grid, as ever, shows the state *after* the moves narrated beneath it. The counting was the crime: replication on a quorum made the term-two slot *durable*, but elections protect committed law by comparing *terms*, and an inherited slot's term is old news. Bingo's hollow triangle in the final panel is the corpse: a state machine has applied a law its own log no longer contains.

term four, lawful: a fresh no-op first



Under the true rule, Bingo first replicates a fresh no-op (3 : 4) to Tuppy and Gussie — a quorum of his own term. Slot three commits by its own count; slot two beneath it, by log matching.

term five, lawful: the veto falls



Barmy solicits again: Gussie's ending (3 : 4) beats his (2 : 3) — refused; his vote in term 5 stays unspent. Only Oofy grants: two of five, no quorum. The by-law stands forever.

Exhibit 4.7 (the lawful replay; the trap fails to spring). The same schedule from Bingo's term-four election, under the true commit rule; the fresh entry's own-term count commits both slots, and Barmy's second candidacy dies at Gussie's veto — exactly where Theorem 4.7's witness stands (Lemma 4.6 dragging the inheritance into safety beneath the no-op).

4.6 Part Six — The Family Restored to Order

Two paragraphs of taxonomy; perform the reunion gently, for people grow attached to their vocabularies, having paid tuition in outages for each.

One algorithm, two management philosophies. Terms are ballots; votes are promises; the up-to-date veto is the confession-and-adoption rule turned inside out. Howard and Mortier, in a tidy comparison published in 2020, put it near enough thus: strip the presentation and the two are one algorithm wearing two management philosophies — Paxos trusts any proposer to learn the past on demand; Raft insists the past be pre-installed in the leader before the gavel (Problem 4.11 builds the family dictionary). Multi-Paxos in sensible tweeds.

Viewstamped Replication, restored to seniority. In 1988 — a decade before the island manuscript saw print — Brian Oki and Barbara Liskov published *Viewstamped Replication* at the ACM's symposium on the principles of distributed computing: a primary per numbered *view* (the term-analogue), view changes on suspicion of failure, a majority carrying the state forward across the change, the new primary obliged to adopt the survivors' latest state before proceeding. Views are terms are ballots; the view change is the election is phase one; the adoption obligation is the rule proved forced above — the same machine, discovered independently and published *first*, entered here into the record at its rightful seniority. Why the island conquered anyway: partly the drawer's own legend, mostly timing — the great industrial deployments were built as the island paper surfaced, and the war stories that followed (a celebrated Google account is titled, with feeling, "Paxos Made Live") cemented the vocabulary. Vocabulary is destiny in this trade. The census: one theorem-shaped idea, four decades of independent tailoring — Viewstamped Replication first in print; Paxos first in folklore; Zab, born industrial in ZooKeeper livery, first in the production fleet (Chapter 5); Raft first in the textbooks.

4.7 Part Seven — The Universal Machine

The last movement widens the lens on why the problem was worth two acts of derivation.

The consensus number. In 1991 Maurice Herlihy published "Wait-Free Synchronization," whose centerpiece ranks every shared object by a single number — the *consensus number*, the largest house the object can seat. Read/write registers — the substrate a lifetime of single-machine programming teaches one to trust — rank one: two gentlemen sharing any quantity of registers cannot solve consensus between just the two of them; the proof is bivalence in new rooms — writes at distinct registers as merely-elsewhere events that commute, the fence held in shared memory as it was held in the Post. Chapter 3's machinery was never about message passing; it was about asynchrony and one possible absentee, and it follows the pair wherever they go. Test-and-set, the humble queue, the stack: rank two — able to settle a pair, helpless before three. At infinity: compare-and-swap, wearing silicon in every processor, and consensus itself, which settles any house whatsoever. The theorem that earns the movement its title — *consensus is universal*: own the bit, and you own the world.

Remark 4.12 (Herlihy's hierarchy; universality). Assign each object type the largest n for which it solves n-process wait-free consensus: read/write registers rank 1 (the bivalence machinery of Chapter 3 transplants to shared memory: writes to distinct registers commute, a read learns only what the schedule permits, and the fence holds); test-and-set, queues, stacks rank 2; compare-and-swap and consensus itself rank ∞. The *universality theorem*: from consensus, a fault-tolerant linearizable implementation of any sequential object — agree, repeatedly, on the next operation; feed the agreed log through deterministic replicas. Statement with pointer only (Herlihy 1991); the construction in miniature is Chapter 5's opening subject, and "linearizable" is debt #13, payable in Chapter 7.

The architecture funnels here, and Chapter 5 is already written by it: one value must become a log; the log must become a service; and the services — Chubby, ZooKeeper, the captive parliaments of industry — must learn to live with tenants. Consensus in captivity; bring a lease.

4.8 The Whiteboard

For the design review you will sooner or later chair.

1. Never hand-roll consensus. The derivation above is the *short* version, and every corner it turned is a production incident in some company's private history. Rent the machinery: a maintained Raft library, a coordination service, a database that has done its Jepsen penance — Jepsen being the trade's independent audit harness (Chapter 7), which partitions and crashes databases on purpose and publishes what their brochures' promises were worth. The parliament is infrastructure, not a weekend project.

2. But recognize the skeleton anywhere: a total order of attempts (ballots, terms, views, epochs); two phases whose quorums overlap — a read of the past that freezes it, a write of the future bound by it; and adopt-the-highest in some costume. "Version numbers plus a majority write" with no read-and-adopt phase is the stale-ballot overwrite in embryo. Practise on machinery already in the fleet: ZooKeeper stamps every transaction with an identifier whose upper half is an epoch — a ballot number, so a deposed leader's letters die on arrival; etcd campaigns in terms; the fencing token of Chapter 5 is the skeleton's smallest bone.

3. The leader is a liveness optimization; safety lives in the quorums. Interrogate any leader-based design at exactly that joint: what breaks when two leaders exist at once? The correct answer is "throughput,

briefly"; any answer involving the word "corruption" means the ledgers are trusting the throne, and thrones wobble.

4. Persistence is part of the protocol, not an implementation detail. What is fsynced before what is acknowledged is a safety question, not a tuning question; treat a hand-wave there exactly as you would treat one about the quorum size.

The register. Paid: #7 — a total order may be rented from a leader, issued at Chapter 2's mutual-exclusion protocol and paid in full: the throne stands, rented from partial synchrony, defended by ballots, priced as an optimization rather than a foundation; and #9 — partial synchrony, cashed by the Synod the moment the distinguished proposer took the chair. Issued: #12 — one value must become a log: multi-decree composition, its lock services and leases (Chapter 5). #13 — "behaves as a single machine" is owed its proper name: linearizability, defined with teeth (Chapter 7). #14 — the parliament's own membership was quietly assumed fixed all along; reconfiguration deferred, with a respectful shudder, to Chapter 5.

4.9 Problems

Tier I — Calisthenics

Problem 4.1 (club arithmetic)**.** (a) For parliaments of three, five, and seven: the quorum size, the number of tolerated absentees, and the number of distinct quorums. (b) Show that a parliament of four tolerates no more absentees than a parliament of three; conclude the standing preference for odd houses. (c) Exhibit a pairwise-intersecting quorum family on nine acceptors in which every quorum has size four (arrange the house in a three-by- three grid), and verify Lemma 4.1's *conclusion* for it directly. (d) Which chapter proofs would survive replacing majorities by your grid family? (Answer: all of them — Rem. 4.3.)

Problem 4.2 (reading the reference trace)**.** For Exhibit 4.3, tabulate after every event: each acceptor's (*promised*, *accepted*), and what is chosen. (a) At which exact event does S become chosen? (b) Construct the latest-possible arrival of a stray $\langle \textsc{accept}, 2, S\rangle$ retransmission and show it changes nothing. (c) Suppose instead a stray $\langle \textsc{accept}, 4, F\rangle$ from an abandoned ballot four arrives at Tuppy *before* Bingo's prepare: recompute the run and say what Bingo must now propose.

Problem 4.3 (the refusal drill)**.** An acceptor's ledger reads *promised* = 6, *accepted* = (4, B). For each arrival, give the reply and the new ledger: (a) $\langle \textsc{prepare}, 5\rangle$; (b) $\langle \textsc{prepare}, 6\rangle$; (c) $\langle \textsc{prepare}, 9\rangle$; (d) $\langle \textsc{accept}, 6, \mathrm{C}\rangle$; (e) $\langle \textsc{accept}, 5, \mathrm{C}\rangle$; (f) a crash and recovery, then $\langle \textsc{accept}, 6, \mathrm{C}\rangle$ again. Justify (d) against Box 8's guard ($\geq$, not $>$) and say why the asymmetry with (a) is correct.

Problem 4.4 (the adoption drill)**.** A proposer of ballot nine holds promises from three of five acceptors with confessions $\{\bot,\ (3, \mathrm{A}),\ (7, \mathrm{B})\}$. (a) What must it propose? (b) Same, with confessions $\{(2, \mathrm{B}),\ (2, \mathrm{B}),\ (5, \mathrm{A})\}$ — popularity versus strata. (c) Same, with $\{\bot, \bot, \bot\}$. (d) In (b), could B have been chosen at ballot two consistently with the rest of the execution? Could A have been chosen at five? What does the proposer *know*, and what must it merely insure against?

Problem 4.5 (up-to-date drill)**.** Five Raft logs, given as sequences of entry terms: $L_1 = (1, 2, 2)$; $L_2 = (1, 2)$; $L_3 = (1, 2, 2, 2)$; $L_4 = (1, 3)$; $L_5 = (1)$. (a) Order them by $\preceq$ (Def. 4.11), noting incomparabilities if any. (b) Which servers would vote for a candidate with log L_4? With L_3? (c) A candidate with L_4 and a candidate with L_3 stand in the same term: who can win, and what does your answer illustrate about "latest term" versus "most entries"?

Tier II — The Real Thing

Problem 4.6 (the ordering subtlety, completed)**.** Lemma 4.3, Step 1, claims the witness accepted (b, v) before promising b'. (a) Rewrite that step as a standalone lemma about Box 8: *an acceptor's confession to a prepare it answers is never lower than any acceptance it made before answering*, and prove it from the monotonicity of *promised* and the acceptance guard. (b) Exhibit the counterexample execution when the promise is volatile (crash between reply and persist) — this is the bridge to Problem 4.7. (c) Point to the exact line of Box 8 that your proof in (a) leans on, and the exact clause of the model box. *(Full solution: completes the proof of Lemma 4.3.)*

Problem 4.7 (sabotage: the forgetful acceptor)**.** A well-meaning implementer keeps *accepted* on disk but *promised* in memory, "because it is rebuilt by traffic anyway." Three acceptors; two proposers with motions F and S. (a) Exhibit an execution in which both F and S are chosen. (b) State the invariant of Lemma 4.3 that dies, and mark the exact step of its proof that the amnesia invalidates. (c) Give the minimal repair and argue it restores Step 1 of Lemma 4.3. (d) Why does persisting *accepted* alone — the half the implementer kept — not help? *(Certified fatal per the standing rules: the double-choice execution is written out in the solutions and drawn as Exhibit 4.8.)*

Problem 4.8 (sabotage: the counting chairman)**.** Modify Box 9's commit rule to drop clause (b): a leader marks any index committed once a quorum stores it, whatever the entry's term. Five servers; the staging of Part Five. (a) Reconstruct the disaster ledger by ledger — every term, every vote (with the up-to-date check verified at each grant), every append, the fatal count, and the final overwrite of a "committed" entry. (b) Identify which of the five Raft properties survives and which dies, and reconcile with Lemma 4.5 and Theorem 4.7 (whose proof step fails without clause (b)?). (c) Show that under the true rule the same schedule is harmless: where exactly does Barmy's second candidacy now die? (d) Explain in two sentences why "wait for the entry to reach all five" is not an acceptable substitute for clause (b). *(Certified fatal: the reconstruction is written out in the solutions; Part Five's filmstrips, Exhibits 4.6 and 4.7, draw it frame by frame.)*

Problem 4.9 (the veto arithmetic, completed)**.** Theorem 4.7's proof compresses the case analysis of the up-to-date comparison. Prove the standalone claim it uses: *if voter a holds a committed entry e of term t at index i, and candidate c's log lacks e, and every leader of terms in (t, t_c) held e (where t_c is c's final log term), then $a \not\preceq c$* — splitting into $t_c < t$, $t_c = t$, and $t < t_c$ cases as in the text, and justifying the "every entry it creates sits at an index above i" claim from leader append-only. *(Solution sketch provided.)*

Tier III — For the Ambitious (starred)

Problem 4.10 (⋆ flexible quorums)**.** Let phase one use quorums from family $\mathcal{Q}_1$ and phase two from $\mathcal{Q}_2$. (a) Re-run the proof of Lemma 4.3 and mark every use of intersection; show the only requirement is $Q_1 \cap Q_2 \neq \emptyset$ for all $Q_1 \in \mathcal{Q}_1, Q_2 \in \mathcal{Q}_2$ — two phase-two quorums need never meet. (b) On six acceptors, design $\mathcal{Q}_2$ of size two and the matching minimal $\mathcal{Q}_1$; compute the fault tolerance of each phase and say what operational bet the asymmetry encodes (hint: how often does each phase run under a stable chair?). (c) Reconcile with Raft: which of its quorums could be shrunk by the same argument, and why did its authors decline? (Howard, Malkhi, Spiegelman, 2016.) *(Hints only.)*

Problem 4.11 (⋆ the family dictionary)**.** Construct the translation table Paxos ↔ Raft ↔ Viewstamped Replication: ballots, terms, views; promise, vote, START-VIEW-CHANGE; adopt-the-highest, the up-to-date veto, DO-VIEW-CHANGE state transfer; chosen, committed. (a) Where is the genuine structural difference between Paxos and Raft (which office does the learning: the proposer per decision, or the electorate per reign)? (b) Show each system's "deposed leader" letter dies by the same mechanism, and name that mechanism in

one word. (c) After Howard and Mortier (2020): state one concrete engineering consequence of Raft's choice (hint: log divergence bounds, or leader hand-off cost). *(Hints only.)*

4.10 Hints

Hint (Problem 4.1)**.** (c) A row alone is too small, and two rows never meet; the cure is a seat every quorum is obliged to include. (Rows as one family and columns as another — a row meets a column always — is a different, *cross-intersecting* economy: peek ahead to Problem 4.10.)

Hint (Problem 4.6)**.** (a) Two timestamps: the acceptance write and the promise write to *promised*; both raise the same persistent variable; run the guard at the earlier one. (b) The crash erases the ordering because one of the two writes never happened.

Hint (Problem 4.7)**.** Let the first proposer complete phase one against a quorum containing the future amnesiac; deliver its accept letters late. The amnesiac reboots between promise and accept.

Hint (Problem 4.8)**.** Follow Part Five's staging exactly: terms two through five, chairs Bingo, Barmy, Bingo, Barmy. The fatal count happens in term four over a term-two entry. For (c): Gussie's final log term after the term-four append of a fresh entry.

Hint (Problem 4.10)**.** Intersection is used exactly once, in Step 2 of Lemma 4.3: the choosing quorum (phase two) against the preparing quorum (phase one). Nowhere do two choosing quorums meet — two values chosen at two ballots are excluded by the invariant, not by intersection of their quorums.

Hint (Problem 4.11)**.** (b) One word: staleness — a number that only rises, checked at receipt. (a) Ask *who* runs adopt-the-highest, and how often.

4.11 Solutions

Tier I in full (tersely); Tier II in full for Problems 4.6, 4.7, and 4.8 (the first completes Lemma 4.3's proof; the other two are the sabotage certifications), sketch for Problem 4.9; hints only for Tier III, by standing policy.

Solution (to Problem 4.1)**.** (a) Three: quorums of two; one absentee; three quorums. Five: three; two; ten. Seven: four; three; thirty-five. (b) Four needs quorums of three and tolerates one — the fourth seat buys correlation risk and no tolerance; hence odd houses. (c) Fix the centre seat of the grid. The family: each row augmented by the centre seat (the middle row, which holds it already, takes any fourth member instead). Three quorums of size four, and any two share the centre seat — Lemma 4.1's conclusion verified by construction, no counting required. (Rows and columns *without* augmentation form two families of three that always meet each other though not themselves — the cross-phase economy of Problem 4.10.) (d) All: every proof invokes Lemma 4.1 only for the intersection *conclusion* (Rem. 4.3); a family that delivers the conclusion delivers the proofs.

Solution (to Problem 4.2)**.** Ledger table (P = promised, A = accepted): initially Gussie $(2, (2, \mathrm{S}))$, others $(\bot, \bot)$. After prepares of five: Tuppy $(5, \bot)$; Gussie $(5, (2, \mathrm{S}))$; Bingo (self) $(5, \bot)$. After accepts of five: each in turn $(5, (5, \mathrm{S}))$. (a) S is chosen at the moment the *second* acceptor persists $(5, \mathrm{S})$ — a quorum of the three — regardless of when replies arrive. (b) A stray $\langle \textsc{accept}, 2, \mathrm{S} \rangle$ is refused everywhere: $2 < \mathit{promised} = 5$; ledgers unchanged. (c) Tuppy's ledger becomes $(4, (4, \mathrm{F}))$ before the prepare; his promise to five confesses $(4, \mathrm{F})$; the highest confession across the quorum is now $(4, \mathrm{F})$ over Gussie's $(2, \mathrm{S})$, and Bingo must propose F. Nothing was chosen in either history; the rule simply insures against what it cannot see.

Solution (to Problem 4.3)**.** (a) Ignore (or nack six): $5 \not> 6$. (b) Ignore: $6 \not> 6$ — prepare needs strict excess. (c) Promise nine, confess $(4, \mathrm{B})$; ledger $(9, (4, \mathrm{B}))$. (d) Accept: $6 \geq 6$; ledger $(6, (6, \mathrm{C}))$; the $\geq$ lets a ballot use the very promise it obtained — without it no ballot could ever complete. (e) Ignore: $5 < 6$. (f) Recovery restores $(6, (4, \mathrm{B}))$ from disk; the accept at six is honoured as in (d). The asymmetry (strict for prepare, non-strict for accept) is correct because a second prepare at the same ballot could hand the ballot's owner two different confession sets — one ballot, one reading of the room.

Solution (to Problem 4.4)**.** (a) B — highest confession $(7, \mathrm{B})$. (b) A — the stratum at five outranks two copies of the stratum at two; popularity is not a credential. (c) Its own motion; the past is silent. (d) In (b): B at two *could* have been chosen (the two confessions might be two members of a choosing quorum) — and A at five could *not* have been chosen consistently if B was, by Lemma 4.3; but the proposer cannot distinguish "B chosen at two, A a later fossil that should be impossible" from "nothing chosen anywhere." Precisely because the first reading would make A's existence contradict the invariant, the consistent readings are: nothing chosen, or A's stratum descends from a lawful adoption chain — in either case the highest stratum is the only safe proposal. The proposer knows nothing; the rule insures against everything.

Solution (to Problem 4.5)**.** (a) $L_5 \preceq L_2 \preceq L_1 \preceq L_3 \preceq L_4$: the term-three ending of L_4 outranks every term-two ending regardless of length; among term-two endings, length orders. No incomparabilities — $\preceq$ is total on log endings. (b) For L_4: all five (every log $\preceq L_4$). For L_3: holders of L_5, L_2, L_1, L_3 grant; the holder of L_4 refuses. (c) Both can assemble a quorum of grants (L_3 from the four shorter-or- equal logs, L_4 from anyone), but whoever moves first with a quorum wins the term (Lemma 4.5); if both gather votes, the voters' one-vote rule splits the house and the term may elect nobody — randomized timeouts exist for exactly this. The illustration: L_4, *shorter* but ending later, outranks L_3 — "latest term" dominates "most entries," because terms, not lengths, encode electoral legitimacy.

Solution (to Problem 4.6)**.** (a) *Lemma.* If acceptor a accepts (b, v) at time s, and at time $s' > s$ answers $\langle \textsc{prepare}, b' \rangle$, then its confession at s' reports a ballot $\geq b$. *Proof.* At s, the guard of Box 8 (acceptor line 7) gave $b \geq \mathit{promised}(s)$, and line 8 set *accepted* to (b, v); *accepted*'s ballot coordinate never decreases (its only assignments are acceptances, each with ballot $\geq$ the standing promise $\geq$ every earlier acceptance's ballot — the variable *promised* is non-decreasing since both assignments, lines 3 and 8, only raise it, and recovery restores the persisted value). Hence at s' the stored acceptance has ballot $\geq b$, and the confession reports it. Conversely — the direction Step 1 uses — if a is known to have accepted (b, v) at *some* time and to have answered $b' > b$ at some time, the answer cannot precede the acceptance: after answering, *promised* $\geq b'$ forever (monotone, persistent), and the acceptance guard $b \geq \mathit{promised}$ would then fail for $b < b'$. (b) Volatile promise: a answers b' (promise in memory only), crashes, recovers with *promised* $= \bot$, then accepts the old (b, v) — acceptance after promise, confession to b' already sent as $\bot$. Step 1's "absurd" branch is now a real execution, and Problem 4.7 rides it to a double choice. (c) Box 8 acceptor lines 3–4 (persist, *then* reply); model box clause: recovery restores stable storage exactly.

Solution (to Problem 4.7)**.** (a) The double-choice execution — six moves, one reboot — drawn as Exhibit 4.8. Acceptors Bingo, Tuppy, Gussie; proposer one owns odd ballots (motion F), proposer two even (motion S). (1) $\langle \textsc{prepare}, 1 \rangle$ reaches Bingo and Tuppy: both promise (in memory), confessions $\bot$; proposer one lawfully issues $\langle \textsc{accept}, 1, \mathrm{F} \rangle$; the Post dawdles with both copies. (2) $\langle \textsc{prepare}, 2 \rangle$ reaches Tuppy and Gussie: Tuppy's guard $2 > 1$ passes — promise, confession $\bot$ (honest: he has accepted nothing); Gussie likewise; proposer two, the past silent, lawfully issues $\langle \textsc{accept}, 2, \mathrm{S} \rangle$. (3) The accept of ballot two lands at Gussie: guard $2 \geq 2$; persists $(2, \mathrm{S})$. The copy addressed to Tuppy dawdles. (4) **Tuppy reboots.** His *accepted* $= \bot$ survives on disk; both promises he ever made evaporate with his memory. (5) The dawdling accept of ballot one lands: at Tuppy, guard $1 \geq \bot$ passes — he persists $(1, \mathrm{F})$; at Bingo, guard $1 \geq 1$ passes — he persists

$(1, \mathrm{F})$. **F is chosen** at ballot one, quorum {Bingo, Tuppy}. (6) The dawdling accept of ballot two lands at Tuppy: guard $2 \geq 1$ passes — he persists $(2, \mathrm{S})$. **S is chosen** at ballot two, quorum {Tuppy, Gussie}. Two values, two lawful quorums, every guard honoured by the amnesiac protocol; without the reboot, step (5)'s guard at Tuppy reads $1 \geq 2$ and the run is harmless.

(b) Lemma 4.3 dies at Step 1 — the ordering subtlety. The invariant's proof shows a witness in both quorums must have accepted ballot one *before* promising ballot two, so that ballot two's confessions report the acceptance. Tuppy is that witness, and amnesia reverses him: his acceptance of ballot one *follows* his (evaporated) promise to ballot two, and ballot two's confession set lied by omission. The invariant's witness was disarmed, exactly as Problem 4.6(b) predicts. (c) Persist *promised* before every reply (Box 8, line 3 as written); recovery then restores the promise to two, step (5)'s guard reads $1 \geq 2$, the stale accept dies, F is never chosen, and Step 1 is a theorem again. (d) Persisting *accepted* alone keeps the confessions honest about the past but not the standing bar against the past's *return*: the fatal guard in step (5) consulted *promised*, not *accepted*. Both halves of the ledger are load-bearing.

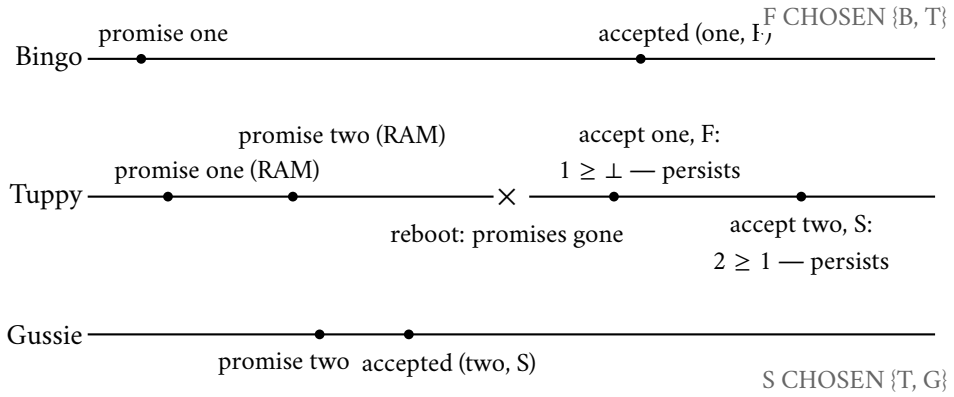


Exhibit 4.8 (the forgetful acceptor's double choice; Solution 4.7). Tuppy's promises live in memory and die in one reboot; his ledger of acceptances is honest throughout, and the parliament chooses two by-laws anyway. Both halves of the ledger are load-bearing. Fatality certified.

Solution (to Problem 4.8)**.** (a) The reconstruction, drawn frame by frame as Exhibit 4.6; servers Bingo, Tuppy, Gussie, Barmy, Oofy; log slots shown as (index: term). *Term two*: Bingo leads (votes: Bingo, Tuppy, Gussie, say — all logs empty, every $\preceq$ check passes). Appends (2: two) after an initial (1: one) common entry; replicates slot two to Tuppy only. *Term three*: heartbeats lapse; Barmy stands; voters Gussie and Oofy hold logs ending (1: one), Barmy likewise — $\preceq$ passes; Barmy leads, appends (2: three) locally, crashes before any replication (his debut, honoured). *Term four*: Bingo stands; voters Tuppy (ends (2: two) — equal to Bingo's ending, equal length: passes), Gussie (ends (1: one): passes); Bingo leads, resumes replicating (2: two) to Gussie — slot two now on {Bingo, Tuppy, Gussie}, a quorum. **The sabotaged rule marks index two committed.** Bingo pauses. *Term five*: Barmy recovers, stands; his log ends (2: three); voters Gussie and Oofy end (2: two) and (1: one): two $<$ three, both must grant. Barmy leads and replicates (2: three) everywhere; the consistency check at slot one passes, the conflicting (2: two) suffixes are deleted — the "committed" entry is expunged from every log. (b) Election safety survives (every term had one leader — Lemma 4.5 never depended on logs). Leader completeness dies: term five's leader lacks a "committed" entry — precisely because the committed-in-term-t premise of Theorem 4.7 was counterfeit: the count in term four was of a term-*two* entry, and the proof's witness argument needs the voter's veto to see the commitment's term, which it cannot when the count crosses terms. (c) Under the true rule, Bingo in term four first appends and replicates (3: four) to {Bingo, Tuppy, Gussie}; that count is of his own term: index

three commits, and index two commits beneath it by Lemma 4.6. Barmy's term-five candidacy then meets Gussie ending (3: four) > (2: three): refused; with only Oofy and himself, no quorum — the candidacy dies at Gussie's veto, exactly where Theorem 4.7's witness stands (Exhibit 4.7). (d) Waiting for all five confuses durability with commitment in the other direction and surrenders availability: one slow or dead server (Gussie on an ordinary day) blocks all progress — and even then the entry's term still loses the up-to-date comparison to any later-term log, so the window merely narrows. Terms, not counts, are what elections read.

Solution (to Problem 4.9)**.** (Sketch.) a's log ends at term $\geq t$ (it holds e; anything it appended later has term $\geq t$ by term monotonicity along one server's history). Case $t_c < t$: term comparison fails immediately. Case $t_c = t$: c's entries of term t were created by t's unique leader (Lemma 4.5); by Lemma 4.6, c's log is a prefix-consistent portion of that leader's log; lacking e at index i forces c's length $< i$; a's length $\geq i$; with equal final terms, length decides against c. Case $t < t_c$: t_c's leader held e (induction hypothesis of Theorem 4.7); it appends only above its own log (leader append-only), so every term-t_c entry sits at index $> i$; c holding a term-t_c entry at its end and passing the consistency chain beneath it (Lemma 4.6) therefore holds everything of that leader's log through some index $> i$ — including e: contradiction with "lacks e," so this case cannot occur at all (rather than failing the comparison, it fails the premise).

Solution (to Problems 4.10 and 4.11)**.** By standing policy: hints only (the Hints section above). For Problem 4.10(b), the arithmetic checkpoint: with $|Q_2| = 2$ on six acceptors, every Q_1 must meet every pair — Q_1 must have at least five members.

4.12 The Reading

Lamport, "Paxos Made Simple," ACM SIGACT News, 2001. This chapter's text. The abstract is one sentence; quote it with relish. Read §2.2 against Lemma 4.3 — his P2c is our invariant, derived there in the same spirit of forced moves — and §2.3 against Remark 4.10. The persistence requirement is stated in one quiet sentence; you now know its full price (Exhibit 4.8).

Ongaro and Ousterhout, "In Search of an Understandable Consensus Algorithm," USENIX ATC, 2014. Figure 2 is the source of Boxes 9 and 9b; §5.4.1 is the up-to-date veto; §5.4.2 is Problem 4.8 in the authors' own staging; the user study is §9's dare. The extended version and Ongaro's thesis (Stanford, 2014) carry the full safety argument our Theorem 4.7 follows.

Lamport, "The Part-Time Parliament," ACM Transactions on Computer Systems, 1998. Unassigned; unlocked. Read it after the others, for pleasure — the way one reads a founding document once the constitutional law is understood — and notice the jokes were, in fact, load-bearing.

Oki and Liskov, "Viewstamped Replication," PODC 1988. For the pleasure of watching the future arrive early: views, view changes, the adoption obligation — the same machine, first in print.

For the starred doors. Howard, Malkhi, and Spiegelman, "Flexible Paxos: Quorum Intersection Revisited," OPODIS 2016 (Problem 4.10). Howard and Mortier, "Paxos vs. Raft: Have We Reached Consensus on Distributed Consensus?," PaPoC 2020 (Problem 4.11). Herlihy, "Wait-Free Synchronization," ACM TOPLAS 1991 (Rem. 4.12). And Chandra, Griesemer, and Redstone, "Paxos Made Live" (PODC 2007), for what the island cost to industrialize — assigned properly in Chapter 5.

Standing companion: Kleppmann, chapter 9. His treatment of total order broadcast is Chapter 5's bridge; his pages on distributed transactions wait for Chapter 8. His consensus section pairs with the derivation above as a second telling in calmer prose.

Chapter 5

Consensus in Captivity

Companion to Episode Five. The episode tells the story — the Cloudflare afternoon, the corpse that walked back in, the generals and their forged letters; this chapter keeps the record: one value becomes a log and the log becomes the replicated state machine, the identical-replicas theorem proved in a page; the membership lemma; the two landlords and their furniture; the lease, proved safe from drift-bounded clocks alone, and the fencing token, by which the store rather than the lock has the last word; the three-node Byzantine impossibility, its worlds side by side; the $3f+1$ bound and the arithmetic of perjured witnesses; PBFT boxed; Nakamoto's lottery given its one respectful movement; and the discipline of when not to convene the parliament at all. Statements, proofs, protocol boxes, problems.

5.1 Part One — One Value Becomes a Log

The episode opens on the second of November, 2020: a Cloudflare postmortem titled "A Byzantine Failure in the Real World," in which a switch failing partially and asymmetrically left an etcd cluster — spoken *et-see-dee*; three members, running Raft — each holding a different, internally consistent account of who could hear whom. Safety survived without a scratch; everything around it thrashed — elections almost completing, leaders deposed by phantom silences, terms climbing — because the world had wandered outside the model the guarantees were purchased in. That gap, between the parliament in the proof and the parliament in the fleet, is the chapter's whole subject: Chapter 4 built consensus in the abstract; here it goes into captivity — put to work, rented out, operated, resurrected wrongly, and finally confronted with correspondents of genuinely low character. Two footnotes, load-bearing:

- **etcd:** the Chubby pattern rebuilt in the open, Raft where Chubby keeps Paxos, keeping every Kubernetes control plane honest: whoever has operated Kubernetes has operated a rented Raft parliament.
- **not, strictly, Byzantine:** no component lied with intent; a switch merely made honest machines *look* inconsistent to one another. The word proper enters through the front door in Part Five.

From one value to a log. The Synod of Chapter 4 decides one by-law in two round trips and stands complete; a shop needs a by-law every few milliseconds. So number the decisions: slot one, slot two, slot three — the *docket*, formally the log positions, slot k decided by its own instance k of the Synod, the sequence constituting the **log**: the ordered record of everything the parliament has ever resolved. Run naively this is extortionate — every slot fighting its own phase-one battle — and the remedy is Chapter 4's chairman: elect a distinguished proposer and let him run phase one *once*, one ballot number covering every slot (Box 10, line 1); thereafter every decision costs a single accept round. That arrangement — the Synod per slot, phase

one amortized under a stable chair — is **multi-Paxos**, the thing actually deployed wherever Paxos is spoken with a straight face; Raft is the same animal in different tailoring, the chairmanship made load-bearing and the log made primary.

The succession trace. The episode runs the machine at full correspondence; the skeleton for review, beside Exhibit 5.1:

- **fair weather:** Bertie submits a debit and a credit; Bingo, in the chair at ballot seven, assigns slots forty-one and forty-two and sends the accepts; each slot is chosen at a majority — clubs, not unanimity — and the state machines apply in docket order at every member as the news arrives. One round trip per decision.
- **the chair falls silent:** slot forty-one reached a majority; slot forty-two's accept reached only Gussie. Tuppy suspects and prepares ballot eight *for all slots*; the house confesses per slot.
- **per-slot succession:** forty-one confessed at ballot seven — adopt and re-ratify, Chapter 4's invariant working slotwise. Forty-two survives in Gussie's confession alone — adopt the highest; the credit outlives its author's silence.
- **the hole:** had forty-two's accept reached nobody, the confessions return empty and the docket has a hole; Box 10's apply rule dams the river at an unfilled slot, so the new chair fills it — with the lost motion if any fragment survives, otherwise with a formal **no-operation**, the identity command, undignified, perfectly lawful.

Protocol — Box 10. Multi-Paxos essentials (checked against "Paxos Made Simple" §3 and "Paxos Made Live")

1: **leadership:** the chair holds ballot b prepared *for all slots*: one $\langle \textsc{prepare}, b\rangle$; promises carry, per slot, any accepted fragments
2: **steady state:** client command c arrives; chair assigns the next free slot k; sends $\langle \textsc{accept}, b, k, c\rangle$; chosen at a quorum of $\langle \textsc{accepted}, b, k\rangle$
3: **apply rule:** each replica applies slot k only after slots $1..k-1$ — a hole dams the river
4: **succession:** new chair prepares $b' > b$ for all slots; per slot adopts the highest confessed fragment; unresolved slots below the frontier are proposed as NO-OP
5: **batching and pipelining** amortize the round trip; reads are served through the log, or under the leader lease of Box 11 (debt #16 fine print: Chapter 7)

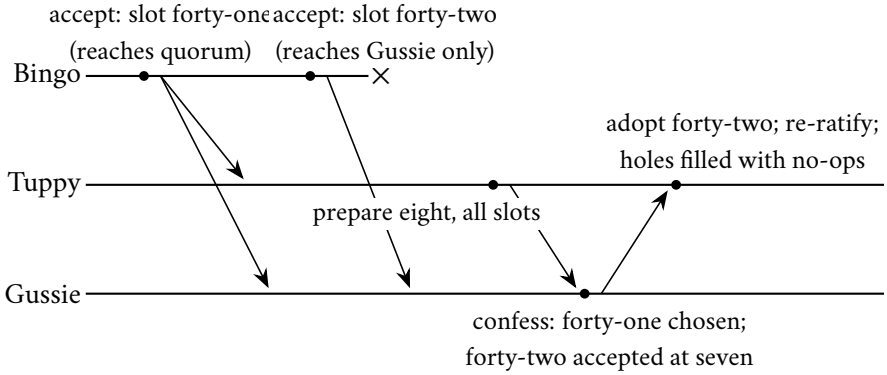


Exhibit 5.1 (multi-Paxos succession; the docket with a hole). Bingo's chairmanship ends mid-stream; Tuppy prepares a higher ballot for all slots at a stroke, adopts the surviving fragment of slot forty-two from Gussie's confession, and closes any void slots with no-operations so the state machines' river can flow. Raft's commit-own-term rite is this rite in other dress.

The chapter's models, fixed before the first formal claim; the second paragraph is Part Five's promissory note.

Model of this chapter

For the log, the landlords, and the exorcists (Defs. 5.1–5.4, Thms. 5.1–5.3, Lemma 5.2). As Chapter 4: asynchronous; crash-recovery with stable storage; quorums are majorities of the current configuration, membership changed only through the log. For the lease theorem *only*, a physical-clock purchase, small and declared: every clock's *rate* lies within a factor $[1-\rho,\ 1+\rho]$ of true time, $0 \le \rho < 1$; nothing whatever is assumed about offsets — no clock knows the hour.

For the Byzantine impossibility (Def. 5.5, Thms. 5.4–5.5). Deliberately generous timing, as in Chapter 1's impossibility: synchronous rounds, no message ever lost — the theorem is about *forgery alone*, and a fortiori kills the asynchronous case. Failures: up to f processes Byzantine (arbitrary state machines, full collusion, adversary-chosen). Links are *oral*: delivery is reliable and the immediate sender of each message is known to its receiver, but a relayed content is entirely the relayer's choice — hearsay cannot be audited. (The *signed-messages* variant — unforgeable, verifiable relays — changes the theorem: Problem 5.11.)

The christening owed since Chapter 2 — why the *log*, not the consensus, is the character the industry built its temples around — is the acorn grown to full oak: the **replicated state machine**. Run one copy of any deterministic machine on each gentleman; feed every copy the same operations in the same order — precisely what the log provides — and the copies cannot help but remain identical, having at every step the same state, the same input, and the same rulebook. Lamport tossed the observation off in a paragraph in 1978; Schneider's 1990 tutorial (ACM Computing Surveys) made it doctrine, and the doctrine is the industry's entire architecture: *fault tolerance is not a property added to a service; it is a property inherited by driving the service from a replicated log.* Agreement is needed once, at the narrow waist — slot by slot, on the order of operations — and everything above the waist gets consistency free, by determinism: Chapter 4's universality theorem wearing overalls.

Definition 5.1 (replicated state machine)**.** A *state machine* is a deterministic transition function $\delta(s, c) = (s', o)$: state and command in, state and output out. A *replicated state machine* over a log: each replica holds a copy of δ, an applied-prefix length, and applies committed log entries strictly in slot order, each exactly once. Commands must be pure in (s, c): any environmental input (time, randomness) is fixed at proposal and carried inside c.

The constitutional theorem is proved by the shortest useful induction in the volume: induction on the log prefix, determinism carrying equality one entry forward.

Theorem 5.1 (identical replicas)**.** *If two replicas of the same state machine have applied prefixes of the same committed log, then after applying prefixes of equal length their states and emitted outputs are identical; in particular, replicas that have applied unequal lengths differ only by not-yet-applied suffixes, never by divergence.*

Proof of Theorem 5.1. Induction on prefix length k. Base: equal initial states. Step: both replicas hold state s_{k-1} (induction hypothesis) and apply the same entry c_k — the same log, the same slot, and committed

entries are unique per slot (Chapter 4's safety theorems for whichever family runs the log). Determinism of δ and purity of c_k give the same (s_k, o_k) at both. Exactly-once, in-order application ensures k increments without gaps or repeats; divergence would require a first differing slot, contradicting the step. □

The determinism diet. The theorem's engine is determinism — a stricter diet than programmers assume, and production falls into the trap weekly: copies that agree on every slot drift apart anyway, the divergence surfacing weeks later as an inexplicable audit failure with the consensus machinery blameless. The hauntings — each a hidden second input, different at every replica:

- **clocks:** wall-clock timestamps consulted mid-apply;
- **randomness:** fresh draws made during application;
- **iteration order:** collections whose order the language declines to promise;
- **the local environment:** reads of the applying machine's own disk, load, or configuration.

The discipline is Def. 5.1's final clause: anything nondeterministic is decided *once*, by the leader, at proposal, and written into the log entry itself, so that what the replicas apply is pure clockwork (Problem 5.5 is the drill). And one clause banked: the ensemble now behaves, from outside, *as if it were a single machine* that survives the death of any minority of its parts — a phrase doing enormous, undefined work until Chapter 7 pays the debt.

5.2 Part Two — Amending the Parliament

Membership is not a line in a configuration file; it is *the definition of the quorum*, the denominator of the crowded club — change it carelessly and one does not misconfigure the system but dissolves the mathematics. The episode stages the naive disaster; the skeleton: a house of five is to become seven; the roster travels by ordinary letter; for one interval some gentlemen believe the house is five (majority three) and others seven (majority four), and the Post can arrange two *disjoint* clubs each lawfully considering itself a quorum — two chosen values, two leaders, the full split brain, without a single machine failing, purely by disagreement about what the parliament *is*. So membership is itself replicated state, changed with the same ceremony as any other — through the log — with the subtlety that the state in question defines the machinery doing the changing. Two disciplined solutions, both in production:

- **joint consensus** (Raft's original proposal; multi-Paxos folklore): transition through an interregnum in which every decision requires majorities of *both* houses, old and new; the two eras cannot contradict one another, and a second log entry retires the old house. Correct, general, and fiddly — two overlapping quorum disciplines live in the code at once.
- **one server at a time:** permit only single additions or removals and rest on Lemma 5.2: consecutive houses share a member, so no interregnum machinery is needed at all — provided each change commits through the log before the next begins.

Lemma 5.2 (consecutive houses police each other)**.** *Any majority of a set of n members and any majority of a set of $n+1$ members that extends it (or shrinks it by one) intersect.*

Proof of Lemma 5.2. Let $|Q| > n/2$ and $|Q'| > (n{+}1)/2$ with rosters differing by one member; the combined roster has at most $n+1$ members. Integrality sharpens the majorities: $|Q| \geq \lfloor n/2 \rfloor + 1$ and $|Q'| \geq \lfloor (n{+}1)/2 \rfloor + 1$, and since $\lfloor n/2 \rfloor + \lfloor (n{+}1)/2 \rfloor = n$ for either parity, $|Q| + |Q'| \geq n + 2 > n + 1 \geq |Q \cup Q'|$; the excess is the intersection. □

The one-at-a-time scheme is what most modern deployments run, with one honest asterisk: its original published form contained a subtle interaction between a half-completed membership change and a concurrent leadership change, found after publication and corrected by the scheme's own author — The Reading gives the trail. The moral is not unsoundness (patched, it is sound and standard) but that *this corner of the protocol is where the dragons winter*: membership interacts with leadership interacts with the log's tail; the new member must be caught up before he counts; the removed member must be told to stop campaigning, lest he haunt elections he no longer belongs to. When a design document proposes a bespoke reconfiguration scheme, that is the page to read twice.

5.3 Part Three — The Landlords

Almost nobody who needs consensus runs their own parliament — not laziness but Chapter 4's whiteboard correctly applied: consensus is rented, from a small number of intensively engineered coordination services. The pattern this course calls the **coordination kernel** — locks, leases, elections, membership, and small hot metadata, rented from one obsessively guarded parliament rather than rebuilt badly in every service that needs them — is the economics of consensus: the machinery is expensive to operate correctly, so operate it once, well, and sell coordination retail. Two monuments carry the record.

Chubby (Burrows, "The Chubby Lock Service for Loosely-Coupled Distributed Systems," OSDI 2006). On paper a lock service: a small filesystem-like namespace of tiny files, backed by five replicas running Paxos. The paper's confessions are the treasure — the tenants made it a name service, a rendezvous point, an election mechanism, a home for configuration that must not be lost; the outage table traces most failures not to Paxos but to everything around it: operator error, misconfiguration, client abuse. The mechanics that let ten thousand clients lean on five machines without flattening them:

- **the session:** not one lease per lock but a single master lease — the one client–landlord liveness contract under which every lock, cached file, and open handle shelters, kept alive by periodic exchanges;
- **tracked caches:** clients cache the tiny files aggressively and consistently, because the landlord tracks who caches what;
- **blocking invalidations:** before committing any change, the landlord posts invalidation notices and waits for the tenants' acknowledgments — the change blocks until the caches are provably clean;
- **the jeopardy callback:** when a session's lease enters jeopardy and recovers, or fails outright, the client library tells the application, in effect, *everything you cached is now suspect and every lock you held may be gone*. Applications that ignore the callback are Part Four's resurrection problem, volunteering.

ZooKeeper (Hunt, Konar, Junqueira, and Reed, USENIX 2010). The open-source world's coordination kernel of record: the metadata heart of Kafka, Hadoop, and a decade of operated systems. The tenancy pattern is Chubby's; the furniture *is* the product — a little tree of **znodes**, tiny files, kilobytes at most (a parliament, not a warehouse), and three modest mechanisms composing into everything else:

- **ephemeral** nodes live exactly as long as the session that created them — presence itself, as data;
- **sequence** nodes are created with a number appended, one that only rises, courtesy of the log underneath;
- **watches** are one-shot subscriptions: tell me when this node changes, once, cheaply.

Assembled, they make the smallest election protocol in production service: every candidate creates an ephemeral sequence node in the election directory; the lowest number reigns; and each candidate watches only the node immediately beneath his own — Gussie watches Tuppy, Tuppy watches Bingo — so when the chair's session dies and his node evaporates, the landlord notifies exactly one tenant, who finds nothing beneath him and reigns. No thundering herd at succession, no polling; presence, order, and notification three primitives deep, every one a rental of the log beneath. That is what a coordination kernel is: not a feature list but a small set of correct atoms from which tenants compose their own protocols, mostly without ruin.

Zab, and the family census. The protocol underneath ZooKeeper stamps every proposal with a **zxid** — the pair (epoch, counter), compared lexicographically, epoch first — and a follower discards anything from a stale epoch on sight. Ballot, term, view, zxid: the stamp in four dialects; the Lamport clock's salary is tenured. The distinction from Raft is what is promised about *order*: Zab is engineered as a **primary-order atomic broadcast** — everything a given primary proposed is delivered in exactly the order it was proposed, and a new epoch begins only after the new primary has provably absorbed every delivery the old epochs committed; recovery first, then broadcast, as strictly separated phases, because Zab sits under a primary-backup database whose semantics *are* the primary's proposal order. Raft reaches the matching end state with interleaved machinery — the up-to-date veto and log matching, policing continuously; Problem 5.9 sets the two recovery disciplines side by side. The family census completes: Viewstamped Replication first in print, Paxos first in folklore, Zab first in your production fleet, Raft first in the textbooks — one idea, four working dialects. And one sentence planted deliberately: everything the landlords rent — every lock, every election, every ephemeral node — is ultimately a *claim about time*: who holds what, until when, according to whom. Which delivers us directly into the crypt.

5.4 Part Four — The Resurrection Problem

The most instructive horror story in production distributed systems, a composite assembled from familiar parts per the postmortem clause. The episode stages it in full; the skeleton for review:

- **the grant:** to prevent double-processing, a worker must hold the lock — rented from the coordination service — before touching an account. Bingo acquires it for account seven and begins his business.
- **the pause:** Bingo does not crash; he *pauses* — a garbage collection of heroic length, a virtual-machine migration; the reasons are legion and all of them are Chapter 1's bath.
- **the expiry:** the service, hearing no heartbeat, does its designed duty: declares the session dead, releases the lock, and grants it to Tuppy, who begins processing account seven.
- **the resurrection:** Bingo wakes. The pause, from his side, was nothing at all — no exception, no flag; his memory says, with perfect internal consistency, *I hold the lock for account seven* — and he resumes mid-instruction. Two writers, one account, and every component behaved as designed.

No tuning fixes it: lengthen the session timeout and every legitimate failover slows; shorten it and resurrections multiply; no finite timeout eliminates them, because the pause is unbounded — that is what asynchrony means, and Chapter 3 proved the crashed-versus-slow confusion a conservation law. Nor does it present honestly: the double-write is rare, the corruption surfaces days later wearing the costume of an application bug, and the lock service was blameless. The tell, for the drawer every seasoned operator keeps:

a session-expiry event on one machine, bracketing a gap in its own logs, at the exact timestamp the service granted the lock elsewhere. The managed confusion's promised instalments fall due here, paid in two instruments.

Instrument one: the lease. A lock naively conceived is a permission without an expiry — granted over the Post to mortals who may pause indefinitely, the corpse's charter. The landlords instead grant **leases**: permission for a bounded time, renewable by heartbeat — and, the delicate part, the bound is enforceable without synchronized clocks, because the argument never compares two clocks. It compares two *durations*, assuming only the drift bound ρ of the model box — no clock runs at a wildly wrong *rate*; nothing whatever about offsets. The grantor counts a full period T from the grant by his own watch before re-issuing; the holder counts a discounted horizon — the *discount*, $T(1-\rho)/(1+\rho)$ — from the *request*, earlier than any grant could have occurred, the deliberately pessimistic anchor, and renounces when it passes. Problem 5.2 runs the arithmetic; Theorem 5.3 proves the general case: the holder's true-time window sits strictly inside the grantor's reissue guard, from rate bounds alone.

Definition 5.2 (lease)**.** A *lease* with period T is granted by a grantor (itself replicated; the grant is a log entry) to at most one holder at a time. Protocol as Box 11: the grantor measures T from the grant by its own clock and will not re-grant earlier; the holder measures a discounted horizon from its *request* by its own clock and renounces all authority when the horizon passes.

Theorem 5.3 (lease safety under bounded drift)**.** *Under the drift bound ρ, with holder horizon $H = T \cdot (1-\rho)/(1+\rho)$ measured from the request: at no instant of true time do a holder still trusting its lease and a grantor having re-granted (or a successor holder) coexist. Mutual exclusion in calendar time from rate bounds alone; no clock offsets are assumed or used.*

Proof of Theorem 5.3. Let the holder post its request at true time t_r, the grantor record the grant at true time $t_g \geq t_r$ (the request precedes the grant: the grant is the grantor's reaction to it), and let ρ bound both drift rates.

Holder's horizon, in true time. The holder counts H on its own clock starting at t_r. A clock running fast by the full factor $1+\rho$ accumulates H on its dial after only $H/(1+\rho)$ of true time — but that ends the tenancy early, which is safe. The dangerous direction is a slow clock: dial time H is reached after at most $H/(1-\rho)$ of true time. So the holder trusts its lease during at most

$$\big[\, t_r,\ \ t_r + H/(1-\rho) \,\big] \;=\; \big[\, t_r,\ \ t_r + T\,(1-\rho)/\big((1+\rho)(1-\rho)\big) \,\big] \;=\; \big[\, t_r,\ \ t_r + T/(1+\rho) \,\big].$$

Grantor's guard, in true time. The grantor counts a full T on its own clock from t_g before re-granting. A fast grantor clock is the dangerous direction: dial time T arrives after at least $T/(1+\rho)$ of true time. So no re-grant occurs before $t_g + T/(1+\rho)$.

No overlap. The holder's trust expires by $t_r + T/(1+\rho) \leq t_g + T/(1+\rho)$, the earliest possible re-grant instant — with strict slack whenever the request-to-grant delay $t_g - t_r$ is positive, which it is by any positive Post delay. A successor's authority begins no earlier than the re-grant. Hence no true-time instant hosts both an old trusting holder and a new grant. (The pessimistic anchor at t_r is what absorbs the unknowable request-to-grant delay; Problem 5.7 shows anchoring at grant-receipt is fatal.) □

It is the first real guarantee this course has bought from physical clocks, and the invoice reads: mutual exclusion in calendar time, purchased with a bound on *rate* error, never *offset* error — Chapter 8 shops the same market with a vastly larger budget. But the lease alone has not exorcised the corpse, and this is the step the industry forgets.

Remark 5.3 (what the lease does not fix). Theorem 5.3 guards the *schedule*; it cannot make a paused process consult its watch. The holder's renunciation is a check in the holder's code, and the holder may pause between check and effect. The residual window is closed only at the resource: Def. 5.4 and Problem 5.6. Enforcement belongs at the point of effect.

Instrument two: the fencing token. Argued with particular clarity in the distributed-locking debate of 2016 — Kleppmann's essay "How to Do Distributed Locking" against the Redlock recipe, whose safety rested on bounded pauses and well-behaved clocks: purchases, not defaults, and unpriced (both sides preserved in The Reading). The durable residue: the last line of defence cannot live in the lock holder, because the lock holder is the party whose sanity is in question; it must live in the *resource*. Every grant carries a **fencing token** — formally a monotone grant counter z, incremented at every grant; every write the holder issues is stamped with its token; the store keeps, per protected object, a durable **high-water mark** and refuses any stamp below it. Bingo's posthumous write arrives stamped thirty-three against a mark of thirty-four and dies at the door: the corpse may walk; the bank teller checks the date on his letters. Mark the architecture of the fix: the lock service *alone* cannot deliver the guarantee — safety migrated into the storage interface, one compare on one integer, and the system is safe against clients arbitrarily late, arbitrarily confused, and arbitrarily convinced of their own authority. The store, not the lock, has the last word; the enforcement point must be the point of effect — a general law the next two chapters keep meeting. Ballot number, term, epoch, fencing token: one skeleton, its smallest bone now in production clothing.

Definition 5.4 (fencing)**.** Each grant carries a *token* z, strictly increasing across grants (the grantor's log supplies monotonicity). Every operation a holder issues to a protected resource is stamped with its token. The resource keeps, durably, a high-water mark hw per protected object and executes an operation iff $z \geq hw$, setting $hw \leftarrow z$; lower stamps are refused.

Protocol — Box 11. Lease grant and hold, under drift ρ

Grantor state is replicated (grants are log entries); tokens rise monotonically (Def. 5.4).

1: **holder:** record local time $r \leftarrow$ now(); send ⟨ACQUIRE⟩
2: **grantor, on** ⟨ACQUIRE⟩ with no live lease: $z \leftarrow z + 1$; log the grant; record local time g; reply ⟨GRANTED, z, T⟩
3: **holder, on** ⟨GRANTED, z, T⟩: trust the lease while now() $- r \; < \; T \cdot (1-\rho)/(1+\rho)$; stamp every resource operation with z
4: **grantor:** treat the lease as live while its local clock shows less than T since g; renewals re-run the exchange (same token) before either horizon lapses
5: **resource, on** operation stamped z: execute iff $z \geq hw$; then $hw \leftarrow z$ (durable); else refuse

Lines 1–4 are Theorem 5.3; line 5 is Def. 5.4 and closes the residual window of Rem. 5.3. After Gray and Cheriton (1989) for the lease; the fencing discipline per the 2016 locking debate (The Reading).

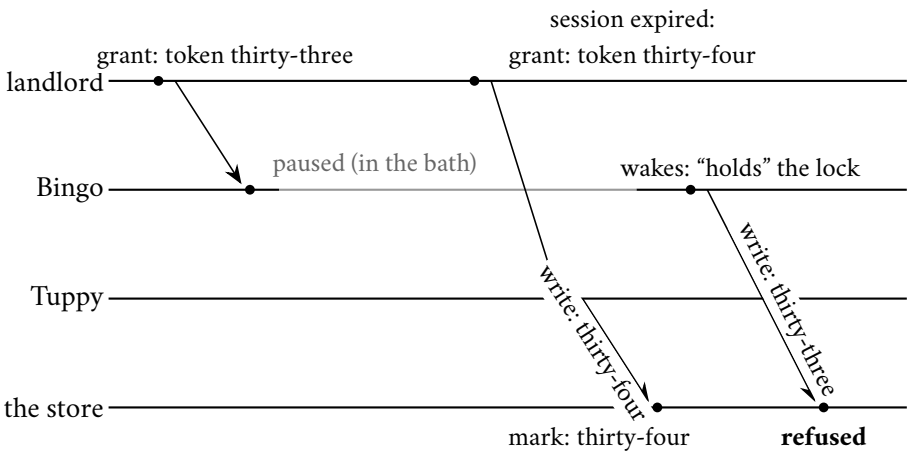


Exhibit 5.2 (the resurrection, fenced). Bingo's pause outlives his session; Tuppy is lawfully granted the successor lease with a higher token; Bingo's posthumous write dies at the store's high-water mark. The lock service never had the last word — the resource did.

5.5 Part Five — The Post Learns to Forge

The last model change, four chapters in the promising: clause five of the Post's charter — the single mercy, standing since Chapter 1 — is revoked. Every failure yet permitted has been a failure of *silence*; now letters may lie about their authorship, machines may say different things to different colleagues, and a member of the parliament may be suborned by Aunt Agatha: formally **Byzantine-faulty**, arbitrary and possibly colluding behaviour, adversary-directed; a member so corrupted in a staged run is *the nephew*. Malice is modelled not because every fault is malicious — the half-dead switch of the opening had no intentions whatever — but because malice is the *closure* of all misbehaviour: a guarantee that survives Aunt Agatha survives every bit-flip and every half-audible switch as a special case.

The literature. Pease, Shostak, and Lamport, "Reaching Agreement in the Presence of Faults," JACM 1980: the founding mathematics, containing the bound that closes this part. Lamport, Shostak, and Pease, "The Byzantine Generals Problem," TOPLAS 1982: the framing — loyal and traitorous generals agreeing on attack or retreat by messenger, chosen deliberately so the result would be remembered. It was; it is the field's most successful act of marketing.

The problem, arranged. In Chapters 3 and 4 every gentleman arrived with an opinion and the house settled on one; here one designated voice — the commander — holds the input, and the task is faithful broadcast through mud: every loyal participant must end holding the same account of what the commander said, even if the commander himself is the liar. Broadcast and consensus are cousins with an exchange rate: one faithful broadcast per member gives every loyal gentleman the same vector of everyone's opinions — the literature's **interactive consistency** — and a common rule applied to a common vector is agreement; impossibility for the broadcast therefore poisons consensus, and a bound proven for three generals is a bound on the whole Byzantine estate. The success clauses: IC1, all loyal lieutenants agree — the traitors' final opinions being nobody's concern; IC2, a loyal commander is obeyed — the usefulness clause, without which unconditional retreat would satisfy: universal cowardice in epaulettes. And the messages are *oral*: delivery unforgeable, relay forgeable — a report's content is its reporter's choice, and hearsay cannot be audited.

Definition 5.5 (interactive consistency; the generals)**.** A designated *commander* holds an order v — attack or abstain — and lieutenants exchange messages per the (oral) model box; every loyal participant must output an order. Required: **IC1** all loyal lieutenants output the same order; **IC2** if the commander is loyal, that common output is the commander's order. (With abstain read as "retreat," this is Lamport–Shostak–Pease 1982, oral-messages form.)

The celebrated first question: three participants, one of whom may be a traitor — majority intact, every intuition from the crash-stop world saying comfortably yes. The answer is no; the proof — the doppelganger returned, this time carrying forged letters — traps a loyal lieutenant between indistinguishable worlds, and needs only three of them, the third the second's mirror.

Theorem 5.4 (three cannot tolerate one; LSP 1982)**.** *With three participants of which one may be Byzantine, no protocol — of any length, in the oral model — satisfies IC1 and IC2.*

Proof of Theorem 5.4. Suppose a protocol P satisfies IC1 and IC2 with participants B (commander), T, G, tolerating one Byzantine member. The adversary constructs three executions; Exhibit 5.3 draws the two comparisons.

World 1 (B Byzantine; T, G loyal). B runs, toward T, the exact program a loyal commander with order attack would run; toward G, the exact program a loyal commander with order abstain would run. (A Byzantine member may run any program; in particular two loyal programs spliced.) T and G follow P honestly, including all lieutenant-to-lieutenant exchange.

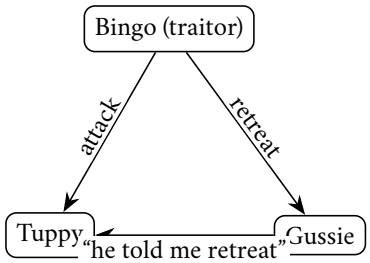
World 2 (G Byzantine; B, T loyal, order attack). B behaves loyally with attack toward both. G runs, toward T, the program it would run in World 1 — i.e. the honest lieutenant's program as executed when the commander's letters to G say abstain. (Feasible for a Byzantine G: the program is fixed, and the adversary feeds it the fabricated commander input.)

T's views coincide in Worlds 1 and 2. By induction on rounds: T's inputs are messages from B and from G. From B: in both worlds, the loyal-attack program's messages to T (that is World 2's definition; in World 1, B's splice sends exactly these). From G: in both worlds, the messages of the honest-lieutenant program running against a loyal-abstain commander (in World 1 because that is what G honestly experiences; in World 2 by the adversary's construction) — and inductively T's own messages to G are identical, so the simulated G's inputs, hence outputs, are identical. The views are equal round by round.

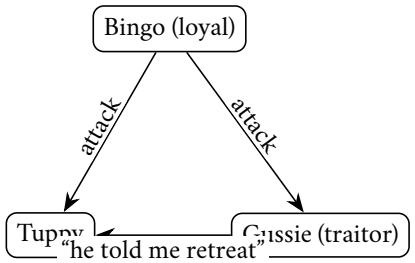
By IC2 in World 2 (B loyal, order attack), loyal T outputs attack. By indistinguishability, *T outputs attack in World 1.*

World 3 (T Byzantine; B, G loyal, order abstain). Mirror of World 2: B loyal with abstain toward both; T runs toward G the honest program as if fed attack from the commander. By the mirrored induction, G's views in Worlds 1 and 3 coincide (from B: loyal-abstain messages in both; from T: honest-program-fed-attack messages in both). By IC2 in World 3, loyal G outputs abstain; hence *G outputs abstain in World 1.*

In World 1 both lieutenants are loyal, so IC1 requires equal outputs; but T outputs attack and G outputs abstain. Contradiction. No such P exists — and note the proof nowhere bounded the number of rounds: longer transcripts extend the inductions verbatim, giving the forger more pages to copy. □



World one
(both lieutenants loyal)



World two
(IC2 compels Tuppy: attack)

Exhibit 5.3 (the forged-letter doppelgängers). Tuppy's complete evidence — the commander's letter and the colleague's report — is identical in the two worlds, so his verdict is too; IC2 in world two forces *attack*, exporting *attack* into world one. The mirrored pair (Worlds one and three, drawn by transposing the roles) forces Gussie to *retreat* in world one, and IC1 dies there. Full inductions: Theorem 5.4.

The engine, for the record: a lieutenant *cannot audit hearsay* — a report about a letter is worth only its reporter's character, and with every voice worth the same, one liar in a triangle can always place each honest member in a world where the liar is someone else. The pigeons of indistinguishability always come home. The general bound follows the same blood type: any house smaller than $3f+1$ can be partitioned into three blocs re-enacting the three-way impossibility — simulate and generalize, structured and starred as Problem 5.10.

Theorem 5.5 (the Byzantine bound; PSL 1980)**.** *Interactive consistency with f Byzantine participants is solvable in the oral model iff the number of participants is at least $3f + 1$. (Achievability: the oral-messages algorithm of LSP, $f+1$ rounds — pointer only. Necessity: reduction to Theorem 5.4; structured as Problem 5.10, starred.)*

What became of the old arithmetic is more instructive than the new number: the crash world's $2f+1$ rested on one property of the witness in the quorum overlap — that he *exists*, an honest-but-silent world having no false testimony. A Byzantine witness stands in the box and perjures himself — the **perjured witness**, formally a Byzantine member of a quorum intersection — and every line of Chapter 4's proof holds except the one that assumed testimony is true.

Remark 5.6 (why crash arithmetic dies: the perjured witness)*.* Chapter 4's safety proofs used quorum intersection through one property: a witness in the overlap *exists*, and his ledger tells the truth. A Byzantine witness perjures: in a three-member Synod with one nephew, the choosing and preparing quorums may intersect in the nephew alone, whose confession is Aunt Agatha's choice, and Lemma 4.3's Step 2 collapses. The repair prices the overlap: with $n = 3f+1$ and quorums of $2f+1$, two quorums share at least $f+1$ members — an honest *majority of the overlap* survives the deduction of f liars, and protocols demand $f+1$ matching voices before believing anything. Existence was enough against silence; against perjury one needs an honest outvoting. Same club, steeper cover charge.

Spoken as the design review will demand it: five seats survive two crashes but only one liar; seven seats, three crashes but two liars. When a vendor's five-node cluster claims to "tolerate two failures," the diligent question is Chapter 1's — failures of *which kind?* — with this arithmetic as the audit (Problem 5.3). The chapter's wonder budget is spent on the proof's economy: no probability, only two afternoons of correspondence a loyal gentleman cannot tell apart — the Two Generals' knife, honed now against forgery, the third great result of the course to fall to the identical thrust. Something close to aesthetic law: every fundamental limit this field possesses is a statement that someone, somewhere, cannot tell two worlds apart; and every protocol that works is a machine for manufacturing distinguishability — stamps, terms, tokens, certificates — faster than the adversary can erase it.

Sealed letters. The bound is beaten only by changing the model — Chapter 3's triage, applied on schedule. Give the generals unforgeable signatures — *sealed letters*: signed messages, relays verifiable, composition impossible — and hearsay becomes auditable: the lie in World 2 required Gussie to *compose* a letter in Bingo's name, and against seals composition fails, so the two worlds come apart in Tuppy's hands and the three-node wall dissolves. Signatures do not abolish Byzantine trouble — a sealed liar can still stay silent, or seal contradictory letters to different colleagues and be caught only when the seals are compared — but they re-price it; Problem 5.11 stars that door, with the honest label on the price tag: the seals themselves now carry the load, and cryptography enters the trusted base.

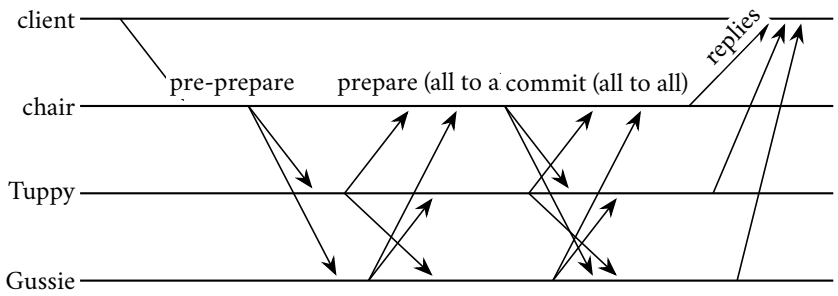
PBFT (Castro and Liskov, "Practical Byzantine Fault Tolerance," 1999). The practical monument, with or without seals: the Synod's spirit rebuilt for the three-f-plus-one world, its happy case played in the smallest interesting house — four members, f equal to one, working clubs of three. The chair assigns the request a slot and announces it (the *pre-prepare*); every recipient broadcasts having seen it (the *prepare*), collecting matching statements from three distinct members into a certificate — three of four leave room for one absentee or liar, so the nephew cannot tell Tuppy one story and Gussie another and have both notarized. The ceremony repeats one level up (the *commit*), making the decision durable across view changes; all members answer the client directly, believed at two matching replies, since two answers cannot both be nephews. On suspicion of the chair, a view change — an election, with epochs; the skeleton is eternal — installs a successor who must *prove*, with certificates, that he carries forward everything the old view committed. Three rounds in fair weather; correspondence quadratic in the house, the price of trusting no single voice — and the paper's very title chosen to answer a decade of scepticism about practicality.

Remark 5.7 (PBFT, and what the certificates buy). Box 12 gives the normal case of Castro–Liskov (1999) for $n = 3f+1$. A *prepare certificate* ($2f+1$ matching prepares) pins (view, slot, request): two conflicting certificates would share $f+1$ members, hence an honest member who prepared both — and honest members prepare one request per (view, slot). The *commit certificate* makes the binding survive view changes, whose successor must present certificates for everything committed. Replies are believed at $f+1$ matching answers. Full proof declined with a pointer (the paper, and the thesis behind it); the message pattern and the two-certificate architecture are the transferable content.

Protocol — Box 12. PBFT, normal case (checked against Castro–Liskov 1999)

$n = 3f+1$; view v has a fixed primary; certificates are $2f+1$ matching messages; all messages authenticated pairwise.

1: client sends request m to the primary of view v
2: **pre-prepare:** primary assigns slot k; sends $\langle \text{PRE-PREPARE}, v, k, m \rangle$ to all
3: **prepare:** each replica accepting the pre-prepare (fresh (v, k), valid m) broadcasts $\langle \text{PREPARE}, v, k, m \rangle$; a replica is *prepared* on collecting a certificate: the pre-prepare plus $2f$ matching prepares
4: **commit:** prepared replicas broadcast $\langle \text{COMMIT}, v, k, m \rangle$; on a commit certificate ($2f+1$ matching), execute m in slot order
5: **reply:** every replica answers the client directly; the client accepts a result vouched by $f+1$ matching replies
6: **view change** (outline): on primary suspicion, replicas broadcast their prepare certificates; the new primary of view $v+1$ must carry forward every certified slot — the adopt-the-highest rite, notarized



the nephew's line is whichever he pleases; certificates of three
out of four make his stories collide — the client believes two matching replies

Exhibit 5.4 (PBFT's normal case, four members, f equal to one). Pre-prepare fans out; prepare and commit are all-to-all; every certificate needs three of four, any two certificates share two members, at most one of whom lies. Box 12.

One deflationary note, in the opening's spirit. Between the crash world and Aunt Agatha lies most of production reality: switches that half-fail, disks that flip bits. The industry's honest position buys full Byzantine machinery where the threat model earns it — mutually distrustful institutions, blockchain frontiers, flight computers — and inside a single company's fleet buys checksums, sanity checks, and crash-tolerant consensus, accepting the Cloudflare afternoon as the residual risk, eyes open, with a signature on the risk register. What is negligence is the middle posture: believing one's crash-tolerant parliament protected against corruption because "consensus" is in the name. Raft agrees on what the leader says, faithfully, even when what the leader says is corrupted nonsense. Agreement is not truth; it is only, ever, agreement.

5.6 Part Six — Consensus by Lottery

One movement, by standing ruling (Decision 11), the eyebrow at the specified altitude. **Nakamoto consensus** — nine pseudonymous pages, 2008 — discards the membership roll: the parliament is unbounded and anonymous, anyone may join wearing any number of masks; with no roll there is no denominator, every quorum argument of this course evaporates, and Aunt Agatha, who can print nephews at will, would own any headcount. The substitution is the nine pages' genuine idea:

- **voting made expensive, not enumerated:** a lottery weighted by computational expenditure; each round's winner appends the next block to the log — it is always a log — and the honest strategy extends the longest chain seen; influence is proportional to electricity, not identities, which are free.
- **finality only probabilistic:** a block buried under six successors is expensive to unseat, under sixty prohibitive; the mathematics never says never — a sufficiently endowed adversary re-mines history.
- **safety as economics:** not quorum intersection but the requirement that the attack cost more than it yields; the ledger's security budget is, quite literally, a utility bill, paid in perpetuity merely to keep yesterday settled. Not hypothetical: on minor currencies the attack has been executed in the field — computation rented by the hour, history re-mined, settled payments unsettled — the theorem working precisely as stated.

A genuinely different theorem for a genuinely different model — open membership, monetary adversaries, synchrony assumptions about propagation doing quiet, load-bearing work — and elegant within its own terms; the eyebrow is raised at the terms, not the derivation. The course bows and moves on: this syllabus concerns parliaments with a membership roll, where the crowded club is purchasable at five machines rather

than a power grid — paying Nakamoto's premium inside a company's fleet is buying flood insurance for a submarine — and the engineering above Nakamoto (the finality gadgets, the proof-of-stake successions, the committee samplings that quietly reintroduce the quorums the scheme discarded) is a course of its own, taught in a different building, with its own aunts.

5.7 Part Seven — When Not to Convene the Parliament

The senior skill, last, because it is a discipline rather than a doctrine: consensus is the most expensive ordinary thing a fleet does. The bill, in round figures:

- **one decision:** a round trip to a quorum — a millisecond or so within a facility, a few across availability zones, tens to a hundred and more across regions — plus a synchronous disk write at every voting member before it answers;
- **the budget:** a coordination service in one region settles perhaps a few thousand to some tens of thousands of decisions per second — and every tenant in the estate shares that budget;
- **the gap:** a busy service admits hundreds of thousands of requests per second before breakfast — three orders of magnitude between what the parliament can decide and what the front door admits.

That gap is the entire architecture of coordination: it is why leases exist (one decision amortized across ten thousand requests), why leaders batch (one round trip across a hundred log entries), why the kernel stores only small hot metadata, and why every read that must be *current* is a question with a price tag.

Reads under leadership. The chapter's last debt, issued with ceremony. The instinct says the leader may answer a read from local state — he is the leader, after all; but the resurrection problem is now on the books: a leader can be deposed and not yet know it, and his confident local answer can be a posthumous letter — stale in the precise sense Chapter 7 will name. The disciplined options: route reads through the log — correct, and priced like a write — or serve them under a *lease*, the leader answering locally only while holding a majority-granted time bound, correct exactly insofar as the drift assumption holds; the fine print of that "exactly insofar" is deferred, with the register's blessing, to Chapter 7 (Box 10, line 5).

The catechism. Convene the parliament for what is genuinely constitutional: leadership, membership, locks, the small hot metadata whose loss or fork is catastrophic — and rent even that, from a landlord with a Jepsen dossier — a published audit from the independent harness of Chapter 4's Whiteboard, formally introduced in Chapter 7. Do not convene it for traffic: for caches, for feeds, for anything whose staleness is survivable — Chapters 6 and 7 build the honest vocabulary for *survivable* — and never on a per-request critical path if a lease can amortize it. The episode stages the catechism failing; the skeleton: a team, burned once by a stale cache, routed every read of the product catalogue through the kernel for the current version — a small read, they reasoned, and correctness is correctness; the kernel, shared by the entire estate, thereafter spent its decision budget confirming that toasters were still toasters, the locks guarding actual money queued behind the toasters, and the estate learned the constitutional distinction during an incident — the most expensive of the available classrooms. The parliaments of this course are constitutional monarchies at best, sir: essential, dignified, and kept well away from the day's actual commerce — which is the next chapter entire.

5.8 The Whiteboard

For the design review you will chair.

1. Rent coordination; do not build it. Keep the tenancy competence in-house — sessions, watches, and above all the session-invalidation callback, which applications ignore at the price of volunteering for resurrection.
2. Every lock in the estate is a lease, whether it admits it or not: find its expiry, then find its fence — one comparison on one rising integer at the resource, the cheapest insurance sold in this industry.
3. Count your f's aloud — $2f+1$ against crashes, $3f+1$ against lies — and say which failures the quorums actually insure. Five seats survive two crashes but only one liar; a Raft cluster is not Byzantine-tolerant because the word consensus appears in its documentation.
4. Know which failures live outside the model entirely, and buy detection there — checksums, sanity bounds, divergence alarms — because the theorem patrols only the terrain named in its premises, and the weather does not read the premises.

The register. PAID: #1, the Post learns to forge — the book's founding debt, receipted in full with theorem and aunt; #12, one value becomes a log; #14, reconfiguration. INSTALMENTS: #2 and #10, armed with leases and fences; final pricing at Chapter 11. ISSUED: #15 — the log stars in its own feature (Chapter 10); #16 — reads under leadership: what *current* means and what a lease buys (Chapter 7). WORD-COUNT DELTA RECORDED: the episode ran 9,858 words at delivery (Session 5), marginally under the 10,000 floor; the assembly session (Session 13) brought it to 10,036 with two staged additions — the rented-computation attack on minor chains, and the toaster catechism from the design review — under a LEDGER §5 drift note.

5.9 Problems

Tier I — Calisthenics

Problem 5.1 (docket bookkeeping)**.** For Exhibit 5.1: tabulate each member's per-slot ledger at every event; state exactly when slot forty-one is chosen, when slot forty-two is chosen, and what Tuppy's prepare must contain. (a) Which slots may the state machines have applied at each moment? (b) Suppose slot forty-two's accept had reached nobody: write Tuppy's succession actions. (c) Why may Tuppy *not* simply skip slot forty-two and renumber?

Problem 5.2 (lease arithmetic)**.** Period ten seconds, drift bound one part in a hundred. (a) Compute the holder's horizon per Box 11 and the worst-case true-time spans of both windows, and exhibit the gap. (b) Recompute with drift one part in twenty (a badly ventilated rack). (c) The holder renews at three-quarters of its horizon: what is the renewal cadence, and what happens to the guarantee if a renewal exchange straddles a pause? (d) At what drift bound does the scheme's usable tenancy fall to half the period?

Problem 5.3 (count your f's)**.** For houses of three, four, five, seven: (a) how many crash failures are survivable with majority quorums; (b) how many Byzantine members are survivable with $3f+1$ sizing and $2f+1$ quorums; (c) the overlap of two quorums in each regime, and the honest residue of the overlap at maximum f. (d) A vendor's five-node cluster claims to "tolerate two failures": write the two follow-up questions the claim requires.

Problem 5.4 (zxid order)**.** Zab stamps: (epoch three, counter nine), (epoch four, counter one), (epoch three, counter ten), (epoch four, counter nought). (a) Sort. (b) A follower in epoch four receives a proposal stamped

epoch three: state its action and the Chapter-2 name for the principle. (c) Compare with a Raft (term, index) pair: which comparisons are meaningful in each system?

Problem 5.5 (the determinism audit)**.** Four apply-functions for a replicated state machine; find each one's hidden second input and give the repair per Def. 5.1: (a) *set expiry to now plus one hour*; (b) *assign the request a fresh universally-unique identifier*; (c) *iterate over a hash table of accounts, applying interest in iteration order*; (d) *if the local replica's disk is over ninety percent full, reject the write*.

Tier II — The Real Thing

Problem 5.6 (fencing, designed and proved)**.** Using Box 11's resource rule: (a) prove that once any operation stamped z executes, no operation stamped $z' < z$ ever executes on that object — and conclude that a resurrected holder cannot write after its successor has. (b) State precisely what is *not* guaranteed about two operations of the same token (a holder racing itself), and what discipline restores it. (c) The high-water mark must be durable: exhibit the failure if the store keeps it in memory (Chapter 1's frugal clerk rides again). (d) Extend the design to reads: what must a fenced read check, and against which token? *(Full solution.)*

Problem 5.7 (the pessimistic anchor)**.** Modify Box 11 so the holder starts its horizon at the *receipt of the grant* rather than at the request. (a) Exhibit an execution violating mutual exclusion (let the grant letter dawdle). (b) Identify the exact inequality of Theorem 5.3's proof that fails. (c) Some real systems anchor at grant-receipt anyway and compensate: what quantity must then be bounded, and why is that a stronger model purchase than drift? *(Full solution.)*

Problem 5.8 (sabotage: fencing by wall clock)**.** A designer replaces the rising token with the holder's wall-clock timestamp at grant time: "fresher time means fresher authority, and the clocks are NTP-disciplined anyway." The store executes an operation iff its timestamp exceeds the stored high-water timestamp. (a) Exhibit an execution in which a paused-and-resurrected holder's write is *accepted* after its successor's write — give concrete clock readings; a modest skew suffices. (b) Exhibit the dual failure: a lawful successor whose writes are *all refused*. (c) State the invariant that rising tokens provide and wall clocks cannot, in one sentence, and name the Chapter-2 principle it instantiates. (d) Would tighter NTP fix it? Cite the relevant part of Chapter 2's Whiteboard. *(Certified fatal per the standing rules: the execution is written out in the solutions and drawn as Exhibit 5.5.)*

Problem 5.9 (Zab and Raft, side by side)**.** Both systems must ensure a new reign carries forward every committed decision. (a) Describe each system's recovery discipline in three sentences: Zab's explicit synchronization phase before broadcast; Raft's up-to-date veto plus commit-own-term rule. (b) For each, identify what a deposed primary's stale proposal encounters (name the stamp that kills it). (c) Exhibit a schedule that Zab's separated phases make easy to reason about and Raft handles with its interleaved rules — and state, with reasons, which discipline you would rather debug at three in the morning. *(Solution sketch.)*

Tier III — For the Ambitious (starred)

Problem 5.10 (⋆ simulate and generalize)**.** Prove the necessity half of Theorem 5.5: with $n \leq 3f$ participants and f Byzantine, interactive consistency is unsolvable. Route: partition the n participants into three non-empty blocs of size at most f each; from a hypothetical n-participant protocol, construct a three-participant protocol in which each general simulates one bloc; verify the simulated protocol would satisfy IC1 and IC2 with one Byzantine general (the bloc containing the traitors), contradicting Theorem 5.4. Mind the bookkeeping: where do the commander's bloc's internal messages live, and why does bloc size at most f let the adversary corrupt a whole bloc? *(Hints only.)*

Problem 5.11 (★ sealed letters)**.** Give each participant an unforgeable signature; relays carry the sealed originals. (a) Show Theorem 5.4's construction fails: locate the step where the adversary must compose a letter it cannot seal. (b) Sketch the Lamport–Shostak– Pease signed-messages algorithm for three participants and one traitor: commander signs and sends; lieutenants countersign and relay; decide from the set of validly-sealed values. Verify IC1 and IC2. (c) State what the seals cost the model: what must now be true of key distribution and of the adversary's computational power, and why the desk files this under "re-priced," not "solved." *(Hints only.)*

5.10 Hints

Hint (Problem 5.7)**.** The proof's inequality $t_r \le t_g$ is free; anchoring at receipt replaces t_r by a time *later* than t_g by an unknown postal delay, and the guard interval must absorb what nothing bounds.

Hint (Problem 5.8)**.** For (a): the resurrected holder's clock need only be *ahead* of the successor's; NTP disciplines toward agreement, not order of authority. Grant times: successor granted later by the landlord, stamped earlier by its own honest, slow clock.

Hint (Problem 5.10)**.** Each general runs its bloc's members faithfully, exchanging intra-bloc messages internally and inter-bloc messages over the three-general channels. IC for the simulation follows because every loyal n-protocol participant lives inside a loyal general.

Hint (Problem 5.11)**.** In World 2, Gussie must show Tuppy a *sealed* retreat order bearing Bingo's signature; Bingo never signed one. For (b), the decision rule "take the set of distinctly-sealed orders received; if singleton, obey it; else default" already works for $n = 3, f = 1$.

5.11 Solutions

Tier I in full (tersely); Tier II in full for Problems 5.6, 5.7, and 5.8 (the last is the sabotage certification), sketch for Problem 5.9; hints only for Tier III.

Solution (to Problem 5.1)**.** Slot forty-one is chosen when a second member persists the accept (quorum of three); slot forty-two is chosen only after Tuppy's re-ratification at ballot eight reaches a quorum — Gussie's lone acceptance at ballot seven chose nothing. Tuppy's prepare must cover all slots and returns confessions: forty-one (ballot seven, the debit) from at least one member; forty-two (ballot seven, the credit) from Gussie only. (a) Members may apply only through the longest chosen prefix they know: forty-one after its choice; never forty-two before forty-one. (b) Confessions for forty-two come back empty; Tuppy proposes NO-OP at forty-two (or fresh business), at ballot eight. (c) Renumbering rewrites history: clients acknowledged against slot numbers, and the apply rule's induction (Thm. 5.1) is over *this* sequence; holes are filled, never collapsed.

Solution (to Problem 5.2)**.** (a) $H = 10 \cdot 0.99/1.01 \approx 9.80$ seconds. Holder's window ends by $t_r + 10/1.01 \approx t_r + 9.90$ true seconds; grantor re-grants no earlier than $t_g + 9.90$; gap $= t_g - t_r > 0$. (b) $H = 10 \cdot 0.95/1.05 \approx 9.05$ s; both windows meet at ≈ 9.52 true seconds from their anchors. (c) Renew near 7.35 s of holder clock; a pause straddling renewal is safe — the old horizon still governs trust, and the renewal merely fails — provided the holder re-checks the horizon *after* any operation resumes; the fence (line 5) covers the residue regardless. (d) Usable tenancy $T(1-\rho)/(1+\rho) = T/2$ at $\rho = 1/3$ — far beyond any sane clock, which is the point: the scheme is robust precisely because ρ is tiny.

Solution (to Problem 5.3)**.** (a) Crashes: one, one, two, three. (b) Byzantine: three seats tolerate none; four, one; five, one; seven, two. (c) Crash regime: overlap at least one, all honest. Byzantine regime ($n = 3f+1$, quorums $2f+1$): overlap at least $f+1$; honest residue at least one — and the protocols demand $f+1$ matching voices, which the residue's honesty anchors. (d) "Failures of which kind?" and "at which quorum size — what does the system do between the second crash and repair?"

Solution (to Problem 5.4)**.** (a) (three, nine) < (three, ten) < (four, nought) < (four, one). (b) Discard: stale epoch; the principle is Chapter 2's "this instruction is stale — I have seen a higher number" — the Lamport clock's entire salary. (c) Zab zxids are totally ordered and comparison is always meaningful; Raft (term, index) pairs order by term for *authority* while index compares *position* — cross-comparing indices across terms is meaningful only under log matching's prefix guarantee.

Solution (to Problem 5.5)**.** (a) Clock input: chair evaluates *now* at proposal, writes the absolute expiry into the entry. (b) Randomness: draw the identifier at proposal, carry it in the entry. (c) Iteration order: sort by account key (or store ordered) — the hash walk differs per replica. (d) Local environment: the disk check is a per-replica fact; either make admission a chair-side check recorded in the entry, or make the rule a function of replicated state (quota accounting in the machine). The common repair: *decide once, upstream, into the log*.

Solution (to Problem 5.6)**.** (a) The mark is durable and monotone: executing z sets $hw \geq z$; any later arrival with $z' < z \leq hw$ fails the guard; induction over executions. The successor's first write carries $z_{\text{succ}} > z_{\text{old}}$ (tokens rise across grants), so after it executes, every posthumous stamp is below the mark. (b) Same-token operations are not ordered by the fence: a holder racing itself (two threads, one lease) may interleave freely; the repair is a per-token sequence number checked monotonically, or the discipline that a holder is single-threaded per resource. (c) In-memory mark: store crashes, mark reborn at zero, the resurrected holder's stale stamp passes — Chapter 1's frugal clerk with a new ledger; persist the mark with the object. (d) A fenced read checks $z \geq hw$ *without raising* the mark (or raises it, at the price of fencing concurrent same-token writers), and must be answered from state no older than the mark's last write — which is Chapter 7's subject in embryo.

Solution (to Problem 5.7)**.** (a) Grant at true t_g; the granted letter dawdles D true seconds; holder anchors at receipt $t_g + D$ and trusts until $\approx t_g + D + T/(1+\rho)$, while the grantor re-grants at $t_g + T/(1+\rho)$: overlap of nearly D, and D is the Post's to choose. (b) The containment $t_r \leq t_g$ becomes $t_g + D \geq t_g$ with the sign the wrong way — the holder's window is no longer inside the grantor's guard. (c) One must bound the grant letter's delivery delay — a synchrony purchase on the Post itself, strictly stronger than a drift bound on local crystals, and exactly the kind of assumption Chapter 3 taught us to see priced on the label.

Solution (to Problem 5.8)**.** (a) The execution, drawn as Exhibit 5.5. The landlord grants to Bingo; Bingo's clock reads fourteen seconds past the minute — *fast* by ten seconds against true time. Bingo stamps his lease-grant moment fourteen-past and pauses. Session expires; the landlord grants to Tuppy, whose honest clock reads *eight*-past (true time: six-past). Tuppy writes, stamped eight-past; the store's high-water timestamp becomes eight-past. Bingo wakes and writes, stamped fourteen-past: *accepted* — fourteen exceeds eight — and the successor's state is overwritten by a corpse with a fast watch. (b) Swap the skews: successor's clock slow by ten; every lawful write stamped below the corpse's mark is refused until the successor's clock crawls past it — a lawful reign refused service by arithmetic. (c) Rising tokens are issued by *one* authority in grant order, so stamp order equals authority order by construction; wall clocks are many authorities with no order contract — Chapter 2's principle: never order cross-node events by wall clock; a cross-machine timestamp comparison is an opinion wearing digits. (d) No: NTP shrinks typical skew, bounds nothing, and fails silently (Chapter 2, Part One); the Whiteboard's first item demanded the uncertainty bound in writing, and none exists here. Fatality certified.

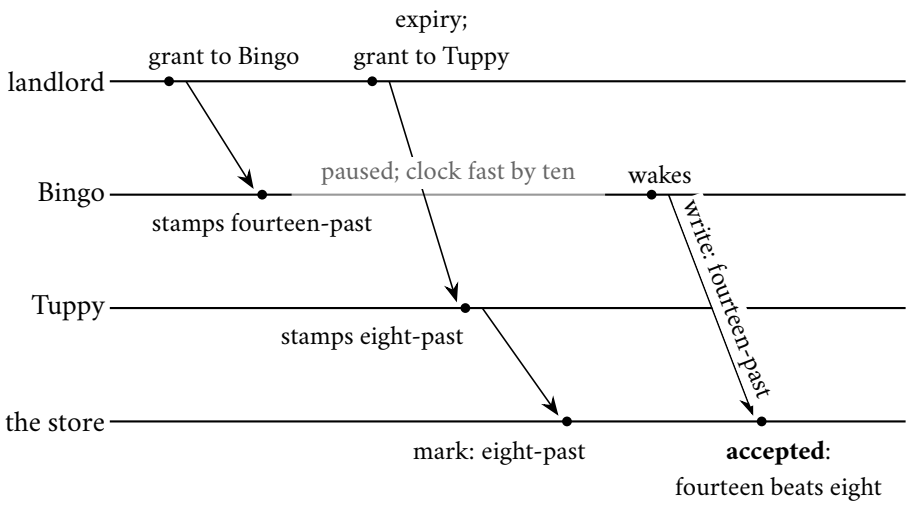


Exhibit 5.5 (fencing by wall clock; the counterfeit millisecond; Solution 5.8). The resurrected holder's fast watch outbids the lawful successor at the store. A timestamp is an opinion wearing digits; a token is an order of authority. Fatality certified.

Solution (to Problem 5.9)**.** (Sketch.) (a) Zab: on election, the new primary runs discovery (learn the highest zxids from a quorum), synchronization (install the chosen history on a quorum), and only then broadcast — three named phases, recovery strictly before new business. Raft: the up-to-date veto keeps ill-read candidates out; log matching repairs followers continuously; commit-own-term closes the inherited tail — recovery woven into normal operation. (b) The stale primary's proposal dies on its stamp: old epoch (zxid) / old term. (c) Schedules with deep follower divergence are naturally narrated in Zab's phases (the sync phase is the narrative); Raft handles them with backward-stepping consistency checks mid-traffic. Preference is honestly a matter of where one wants the complexity: in a phase diagram (audit it once) or in an invariant set (rely on it continuously). The desk declines to legislate taste, and observes only that both have passed their Jepsen penances.

Solution (to Problems 5.10 and 5.11)**.** By standing policy: hints only (the Hints section above). For Problem 5.10, the arithmetic checkpoint: three non-empty blocs of size at most f exist precisely when $n \leq 3f$ and $n \geq 3$.

5.12 The Reading

Burrows, "The Chubby Lock Service for Loosely-Coupled Distributed Systems," OSDI 2006. Assigned as literature. The candid sections — unexpected uses, abusive clients, the outage table — are the truest published portrait of a coordination kernel's tenancy. Read the caching and session mechanics against Part Three, and the lease discipline against Box 11.

Lamport, Shostak, and Pease, "The Byzantine Generals Problem" (1982); Pease, Shostak, and Lamport, "Reaching Agreement in the Presence of Faults" (1980). The framing paper and the founding mathematics. Read the 1982 three-general argument against Theorem 5.4 and Exhibit 5.3; the signed-messages section against Problem 5.11.

Castro and Liskov, "Practical Byzantine Fault Tolerance," OSDI 1999. The monument. Read §4 against Box 12 and Exhibit 5.4; the view-change section is the adopt-the-highest rite notarized.

Schneider, "Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial," ACM Computing Surveys 1990. The log's constitutional theory; Thm. 5.1 in its definitive setting.

The shelf. Chandra, Griesemer, and Redstone, "Paxos Made Live" (PODC 2007) — the industrialization bill, including the disk-corruption and reconfiguration clauses. Gray and Cheriton, "Leases" (SOSP 1989). Kleppmann, "How to Do Distributed Locking" (2016), with the reply from Redlock's author — both sides preserved; the fencing argument in its native habitat. The Cloudflare postmortem, "A Byzantine Failure in the Real World" (November 2020), against the chapter's opening. The Raft single-server membership-change correction (Ongaro, raft-dev, 2015) — the dragons' forwarding address.

Standing companion: Kleppmann, chapters 8–9. The lease-and-fencing treatment in chapter 8 pairs with Part Four; chapter 9's total order broadcast closes the loop on the log.

Chapter 6

Replication, or the Dynamo School

Companion to Episode Six. The episode tells the story — the Decembers of Amazon, the forty-three seconds at GitHub, the two kettles; this chapter keeps the record: the replication grid, the acknowledgment ledger, the school's paths and nets, the register taxonomy with the ABD theorem proved in full, the version-vector paperwork, and the menu folded into a decision procedure. Statements, proofs, price tags, problems.

6.1 Part One — The Classical Column: One Leader

The episode opens in Amazon's Decembers: relational databases doing the honourable thing under load — refusing the write they cannot certify — and the refusal priced as a store turning customers away at the door. Two house commandments come out of that arithmetic, and everything in this chapter is a priced answer to them:

- **Always writeable.** The cart accepts the write — during partitions, during failures, during December. Refusal is the one forbidden act.
- **Judged at the tail.** Service agreements written at the ninety-ninth-point-ninth percentile, on the finding that the unluckiest customers are disproportionately the heaviest ones, and that added latency is measurable lost revenue.

Dynamo (DeCandia et al., SOSP 2007) is the system those commandments built, candid about the price: several versions of a cart at once, reconciled later — consistency betrayed, availability kept, on purpose, with receipts. Set against Chapter 3's theorem, the betrayal is the rational one once refusal is denominated in December revenue. The chapter is the menu — every honest way to keep copies on machines that fail, price tags face up — and it runs under two model rooms, because Part Four's register theorem demands sterner hypotheses than the school's engineering posture.

Model of this chapter

For the menu and the school (Defs. 6.1–6.6, Lemma 6.1). Asynchronous; crash-recovery with stable storage; fair-loss links with retransmission beneath (Chapter 1); membership by gossip is advisory — quorum arithmetic is stated against the intended N; sloppy quorums explicitly weaken the fixed-roster hypothesis of Lemma 4.1, exchanging certainty of overlap for likelihood (stated, not proved — an engineering posture, not a theorem).

For ABD (Def. 6.3, Theorem 6.2). Fully asynchronous message passing; n servers of which any mi-

nority may crash-stop; reliable delivery to correct processes (retransmission); *one* designated writer client and any number of reader clients, all of which may also crash (a crashed reader mid-write-back harms nothing: Rem. 6.5). No clocks, no leader, no failure detector. The multi-writer extension is Problem 6.10 (starred).

Definition 6.1 (the replication grid)**.** A replication scheme is classified by two coordinates. *Acceptance*: which replicas may accept a client write — one (single-leader), one per site (multi-leader), any (leaderless). *Acknowledgment*: whose durable confirmation precedes the client's acknowledgment — the acceptor alone (asynchronous), a fixed set (synchronous; semi-synchronous for "any k of"), or a quorum (N, R, W). Chain replication occupies (single acceptor at the head; agreement = the entire chain, enforced by topology).

What travels down the links. Forward the client's *statement* and every follower re-executes it — haunted exactly as Chapter 5's determinism diet warned (clocks, random numbers, unordered iteration diverge silently while all machinery reports health). Forward the *effect* — rows as changed, bytes as written — and nothing is re-decided. Mature systems ship effects, or statements laundered through a determinism filter: decide once, upstream; replicate the decision, not the deciding.

The acknowledgment checkbox. Does the primary *wait*? The single most consequential setting in the column:

- **Synchronous:** acknowledge after followers confirm durability. Recovery point zero; the price is the round trip to the slowest waiter (PACELC's peacetime toll, Chapter 3) and availability coupled to the *least* available member — one follower's garbage-collection pause halts commerce.
- **Semi-synchronous:** wait for any k of the followers. The write provably exists beyond the primary, though nobody undertakes to say where. Production's usual compromise.
- **Asynchronous:** acknowledge at once, forward at leisure. At any instant the estate carries a window of confirmed-but-unreplicated writes; on unrecoverable primary death that window *is* the data lost. A chosen data-loss window — defensible daily, indefensible without the number attached.

The disaster-recovery ledger names both columns: **recovery point** (RPO — confirmed history that can vanish: the confirmed-but-unreplicated writes lost on primary death) and **recovery time** (RTO — how long the estate stands dark while failover deliberates). The asynchronous window prices on an index card — seconds of exposure, times writes per second, times the worth of a write — and Problem 6.1 fills one out.

Follower reads. The column's second product, shipped with fine print: every follower read is a read of the past, aged by the replication lag — milliseconds mostly; seconds or worse under load. Canonical symptom: a client writes, refreshes, and cannot read his own writing. The purchasable remedies (pin reads to the primary briefly after a write; route by session) generalize into the session guarantees, named and priced in Chapter 7. The honest summary: follower reads are cheap, plentiful, and stale by an *unstated* amount — and unstated is, as ever, the actionable defect.

Failover and the false death. The primary goes silent; dead-or-slow has no answer from across a network (Chapter 1); every failover system buys a verdict with a timeout. The wrong verdict's failure mode is the **false death**: a primary paused (not dead) is condemned; a successor is promoted; the original returns, unaware, and accepts writes from every stale routing table — two primaries, and every write the deposed

one accepts is a fork in the estate's history (the resurrected lock-holder of Chapter 5, at larger scale). The disciplined fencing is Chapters 4 and 5's equipment:

- promotion through consensus or a coordination kernel — at most one promotion can ever be chosen;
- an **epoch** stamped on all replication traffic — the deposed primary's letters bounce off followers who have seen a higher number (the fencing token's elder sibling).

Undisciplined arrangements meet Part Two's postmortem.

Chain replication (van Renesse and Schneider, 2004). Replicas in a chain, not a star: writes enter at the head and ride hop by hop, each node applying and forwarding; the *tail* acknowledges; reads consult only the tail, answered from local state. The topology's invariant: whatever the tail has, the whole chain has; whatever the tail lacks is by definition unacknowledged — so tail reads never expose an in-flight write and never miss a confirmed one. Strong reads, served locally, by geometry alone. Costs: a write pays the full chain length before confirmation, and a broken link demands a reconfiguration dance, spliced by a master typically rented from Chapter 5's landlords. The proof is Problem 6.6 (full solution given): order operations by their passage through the tail; geometry supplies the total order.

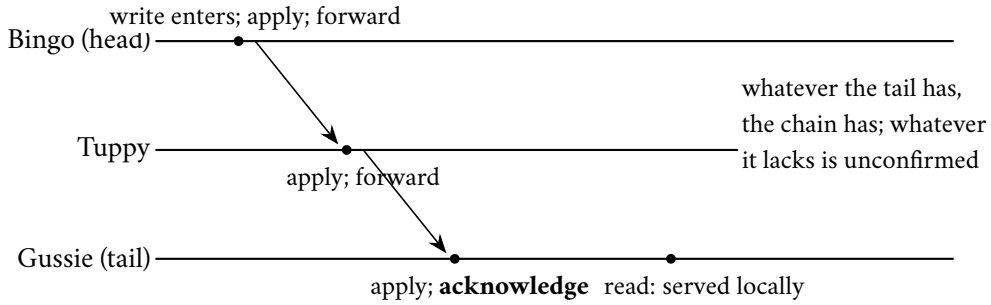


Exhibit 6.1 (chain replication). Writes ride the chain head to tail and are acknowledged at the end; reads consult only the tail. Strong reads by geometry: no quorum, no arithmetic. Problem 6.6 turns the picture into a proof.

6.2 Part Two — Several Leaders, and the Bill

Multi-leader replication: a leader per region, each accepting writes locally, each forwarding to the others — latency bought, and the bill arriving immediately, because two leaders accepting writes to the same object concurrently (*merely elsewhere*, in Chapter 2's earned vocabulary) will conflict, and something must adjudicate. The season's pattern, seen again in grander dress below: every time the trade widens a topology in the name of availability, the ordering bill arrives by return post.

The postal map among leaders. Circle and star topologies economize on connections and let one stopped node dam the relay for everyone downstream. All-to-all posting is robust but opens races: a correction can travel by a faster intermediary and arrive before the update it corrects — an effect presenting itself ahead of its cause — cured by Chapter 2's causal paperwork, the correction held until its ancestor arrives.

Last-writer-wins, arraigned. Attach a timestamp to every write; on conflict keep the larger; discard the smaller, silently. The episode's evidence in one line: concurrent updates to two fields of one record, adjudicated by cross-machine wall clocks Chapter 2 convicted of perjury; the losing row — carrying a customer's corrected address — ceases to exist, every log clean, nothing malfunctioned, a policy executed. Last-writer-wins is not conflict resolution; it is conflict *denial* — institutionalized data loss wearing a tie-breaking rule. It is honestly fine only where the schema itself makes any discarded write superseded by the survivor (telemetry samples, caches, cursor positions); it is never fine by accident. Problem 6.9 audits a schema column by column.

Conflict avoidance. The third posture, much of the industry's quiet default: multiple leaders for latency, but every write to a given key routed through one designated home — per key, single-leader; conflicts cannot form. It works precisely until the routing must change (the customer moves; the home region fails), whereupon the design discovers it was multi-leader all along, the adjudication debt merely deferred and now due at the worst hour.

The postmortem: GitHub, 21–22 October 2018. From the company's published account; the episode walks it in full. The skeleton for review:

- **The cut:** during routine maintenance, the East Coast datacentre lost its connection to the other facilities — for *forty-three seconds*.
- **The promotion:** the automated topology manager, observing East Coast primaries unreachable, did its commissioned duty — West Coast replicas promoted; applications rerouted; writes continued on both sides.
- **The fork:** the connection healed forty-three seconds old, and the estate held two recent histories — East Coast writes the West had never received, West Coast writes the East had never seen — neither discardable, both containing customers' truth, and timestamps unable to adjudicate because (Chapter 2) there was no fact of the matter to report.
- **The bill:** more than twenty-four hours of deliberately degraded service — failover, restoration from backups, replay of orphaned writes, reconciliation by hand and by script.

The exchange rate is the lesson: partitions are cheap to *enter* and ruinous to *leave*, and any design that can produce two primaries owes its reviewers, in writing, the reconciliation plan — before the maintenance window, not during.

The honest homes, and Bayou. Multi-leader cannot be abolished: an offline client writing locally (the laptop, the telephone) is structurally a leader of one, and collaborative editing is multi-leader by product definition. Both homes force the conclusion: conflicts must be *merged*, not adjudicated by clock. The honest ancestor is *Bayou* (Terry et al., Xerox PARC, 1995): writes accepted anywhere, conflict expected as the normal case, every write accompanied at write time by a **dependency check** (a predicate detecting that the world has changed under this write) and a **merge procedure** (what to do about it when it has) — both written by the application, which alone knows what the data *means*. Nothing about the mechanism scaled, and its total-ordering machinery had a primary hiding in it; but Dynamo inherited the moral whole: if concurrent writes are accepted, the conflict is application state, and the system's duty is to keep the evidence — both versions, causal papers in order — not to eat one quietly and call the ledger balanced.

6.3 Part Three — The Leaderless School

The Dynamo column proper, staged in the episode at the width the choreography needs (five gentlemen: Bingo, Tuppy, Gussie, Barmy, Oofy — the school requires understudies). No primary, no elections, no terms; any replica may accept any operation, and correctness is arranged, where it is arranged, by arithmetic. Each key has a **preference list** — the ordered set of replicas responsible for it, read off a hashed ring whose geometry is Chapter 9's subject; the first N are the home set, and any list member may coordinate.

Definition 6.2 (quorum configuration)**.** Each key has N replicas (its preference list's home set); a write is acknowledged after W durable confirmations; a read consults R replicas and adopts the causally newest version returned. The configuration is *overlapping* if $R + W > N$. *Sloppy* variants draw the W confirmations from the first healthy members of the extended preference list, tagging displaced copies with hints (Box 13).

With $R + W > N$, any read quorum intersects any write quorum in at least one member (Lemma 4.1): the newest acknowledged version is always somewhere in the room when you ask. The precise strength — and the precise weakness — of that sentence is the whole of Part Four.

Protocol — Box 13. The Dynamo-school read and write paths (checked against the 2007 paper, §4)

Per key: preference list $p_1, p_2, \ldots$; home set = first N; any list member may coordinate.

1: **write**$(k, v, context)$: coordinator assigns version = context advanced at its own entry (its dot); sends to the first N *healthy* list members
2: a member stores the version (as sibling if incomparable with holdings); if standing in for an unreachable home member, stores with a **hint** naming the home address
3: acknowledge to client after W durable confirmations (sloppy quorum: the confirmations may include understudies)
4: **hinted handoff:** each member periodically streams hinted holdings to their home addresses; on confirmation, the hinted copy is discharged
5: **read**(k): coordinator queries R members; collects all sibling versions and their vectors
6: answer client with the causally-maximal sibling set and merged context
7: **read repair:** send the returned set to any queried member holding strictly older state
8: **anti-entropy:** background Merkle reconciliation of replica pairs, per key range (Lemma 6.1)
9: **gossip:** membership and ring state exchanged pairwise at random; indirect probes before condemnation (SWIM)

Sloppy quorums and hinted handoff. Strict quorums are not *always writeable*: two home members unreachable, and a two-of-three write must refuse. So the school loosens its grip: the coordinator slides down the preference list, and the copy meant for an unreachable home member lands at an understudy, tagged with a **hint** naming the home address; the understudy holds it in a separate larder and acknowledges. On recovery, hinted holdings stream home and each hint is discharged on confirmation (**hinted handoff**). The durability arithmetic is honoured in *count* but not, temporarily, in *address* — and note what was traded, because the paper is honest about it and its cheaper imitators sometimes are not: Lemma 4.1 spoke of quorums drawn from a *fixed* roster; a sloppy write may land entirely on understudies while a later read consults the recovered home set. Overlap by arithmetic has become overlap by high likelihood (the model box's stated posture), with the nets below to sweep the corner.

Net one: read repair. A read at quorum R collects up to R versions; the newest is adopted for the answer, and in the same breath the coordinator posts it back to any queried member holding strictly older state. Every act of asking tidies the room asked: the hottest data is the best-repaired, by construction and for free. Attend to the mechanism — Part Four promotes it from janitor to load-bearing wall.

Net two: anti-entropy by Merkle tree. Read repair tidies only what is read; cold data drifts. The deep net has replica pairs reconcile their entire holdings — absurd said naively, civilized by the **Merkle tree**: over each key range, a tree of hashes, leaves fingerprinting small ranges, each parent hashing its children, one root summarizing the library. Roots equal: identical libraries, reconciled at the cost of one comparison (confidence bounded by the hash's collision resistance). Roots differing: descend, dismiss every matching branch unexamined, recurse into the disagreeing subtree until the differing leaf range stands isolated. Traffic scales with the disagreement, not the library — verification made exponentially cheaper than transfer, which is why the instrument recurs wherever honesty must be checked at scale: here, in content-addressed version control, and in the lottery-parliaments whose chain of blocks is this tree unrolled in time.

Lemma 6.1 (Merkle reconciliation cost)**.** *Let two replicas hold libraries over the same key space, indexed by a Merkle tree of branching factor b and depth d (leaves cover disjoint ranges; internal nodes hash their children). If the libraries differ in the contents of $\Delta \geq 1$ leaf ranges, the descent protocol (compare roots; recurse into differing children only) exchanges at most $b \cdot \Delta \cdot d + 1$ fingerprints, and transfers entry data only for differing ranges. If $\Delta = 0$, one exchange suffices, up to hash collision.*

Proof of Lemma 6.1. Consider the set D of tree nodes whose subtrees contain at least one differing leaf range; D is a union of Δ root-to-leaf paths, so $|D| \leq \Delta \cdot d$ (the root counted once via the +1 below). The protocol exchanges children fingerprints exactly for nodes in D found unequal — at most b fingerprints per such node — and never recurses into a node outside D, since equal fingerprints terminate the branch (up to hash collision, whose probability is bounded by the hash's collision resistance and union-bounded over comparisons). Total fingerprints: 1 (roots) + $b \cdot |D| \leq 1 + b\Delta d$. Data transfer occurs only at differing leaves by construction. If $\Delta = 0$ the roots match and the exchange stops at one. □

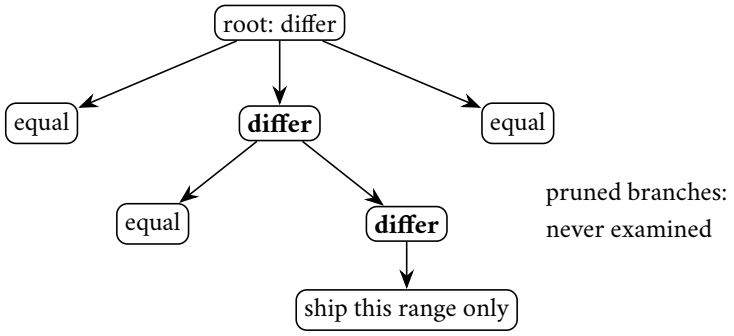


Exhibit 6.2 (the Merkle descent). Fingerprints are compared root-down; equal branches are dismissed with their entire contents unexamined; only the differing leaf range crosses the wire. Traffic scales with the disagreement, not the library (Lemma 6.1).

Net three: gossip membership. No coordination kernel holds the roster; each member periodically exchanges everything it believes about the ring with one colleague chosen at random, keeping the fresher account wherever they differ. Rumour doubles per round — $O(\log n)$ rounds to saturation, a fleet of a thousand fully informed in about ten — with no coordinator, no broadcast storm, and indifference to the

loss of any particular messenger. Lineage: Demers et al. (1987), epidemic algorithms taken literally from epidemiology; the modern refinement is **SWIM**, whose courtesy is the indirect probe — before condemning a silent colleague, ask third parties to probe him, publish suspicion as a state distinct from death, and give the accused a gossip round to protest — so one bad link between two healthy machines can slander neither. This is Chapter 3's failure detector rebuilt without a landlord, its verdicts exactly as purchasable, and as fallible, as every other timeout in this course.

Two mechanical notes. The coordinator is not appointed: any preference-list member may take the chair, and in several descendants the client library itself does — leaderless out to the caller, the saved hop being real money at the tail percentile. And siblings under heavy concurrent writing do not merely occur but *accumulate*, each unmerged rival breeding further rivals; the school's estates cap the family size and complain to their operators, because a key with fifty siblings is an application refusing its merge duty (Part Five), and the pager, for once, points at the right team.

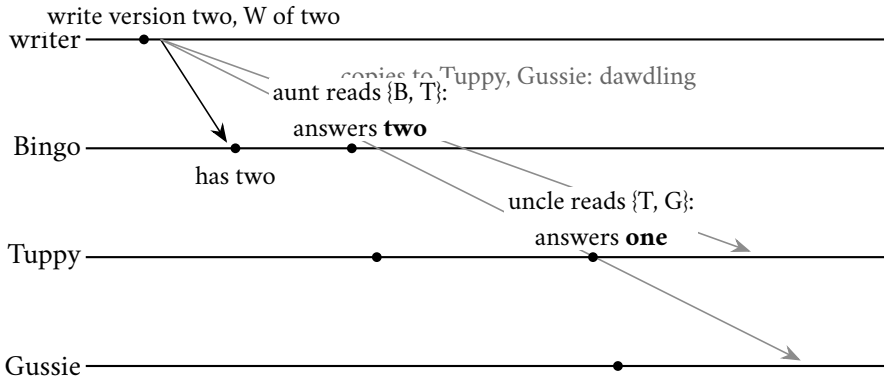
6.4 Part Four — The Fine Print: What the Overlap Buys

The industry belief, stated as held: *with $R + W > N$, reads always see the latest write — linearizability, near enough*. The episode's set piece dismantles the near-enough, then rebuilds it properly; the record here. (Honesty about the dials first: the belief is met at its strongest tuning — overlap real, every read quorum provably touching every acknowledged write's quorum — not at the W=1, R=1 tuning, which promises only eventual anything and whose buyers know it.)

The crime scene. Honest weather — no crashes, no partitions, merely the Post being the Post. N=3, W=R=2, strict. The run:

- Version one of a balance sits on Bingo, Tuppy, Gussie. The writer posts version two to all three: durable at Bingo at once; the copies to Tuppy and Gussie dawdle in the Post — the write is in flight.
- First read, quorum {Bingo, Tuppy}: versions two and one return; the causal paperwork ranks two newer; the reader is answered *two*. Lawful, fresh, no complaint.
- Second read, strictly after the first has completed, quorum {Tuppy, Gussie}: version one, twice — concurring, stale, served with confidence. The observed history ran backwards.

No quorum was violated: "the write set" was still a moving target, and the overlap delivered the new version to one reader and not the next. The overlap guarantees the newest acknowledged version is visible in the room; it does not guarantee that, having been seen once, it stays seen.



the aunt saw the new world; the uncle, later, the old:
regular, not atomic — the flicker

Exhibit 6.3 (the crime scene: new-then-old under $R + W > N$). The write of version two is in flight — durable at Bingo only. Both reads hold lawful quorums; the estate's observed history runs backwards. Visibility is not linearizability.

Definition 6.3 (the register taxonomy; Lamport)**.** A shared read/write register, accessed by operations with invocation and response times, is: *safe* if a read overlapping no write returns the latest completed write's value (reads during writes unconstrained); *regular* if additionally a read overlapping writes returns the value of some overlapping or latest-preceding write (old or new, never invented); *atomic* if the operations admit a total order, consistent with real time, under which every read returns its latest preceding write — atomicity is linearizability specialized to registers, and Chapter 7 adopts the general word. The flicker of Exhibit 6.3 is the canonical regular-not-atomic behaviour: two sequential reads straddling one in-flight write, returning new then old.

Remark 6.4 (what $R + W > N$ buys)*.* With strict quorums, completed-write visibility is a theorem: a read overlapping no write intersects the last completed write's quorum (Lemma 4.1) and the version paperwork ranks that version newest, so the read returns it — safety and, with the overlap applied to in-flight writes as well, regularity (each read returns a value some overlapping-or-preceding write wrote). Atomicity fails: nothing obliges the new version, once served to one reader, to be served to the next (Exhibit 6.3). The missing clause is exactly ABD's write-back. Sloppy quorums weaken even regularity to high likelihood; the nets (repair, anti-entropy) restore convergence, not ordering.

The repair: ABD. Hagit Attiya, Amotz Bar-Noy, and Danny Dolev — the field says *ABD* — solved exactly this (1990; final form 1995), years before Dynamo made the question commercial. One clause, met already in overalls: *the reader who has seen a version must lodge it before repeating it*. A read adopts the newest version and — before the client is answered — writes it back until a quorum holds it: read repair promoted from janitor to load-bearing wall, a blocking obligation rather than an optimization.

Protocol — Box 14. ABD single-writer atomic register (checked against Attiya–Bar-Noy–Dolev)

n replicas each store $(stamp, value)$, initially $(0, v_0)$; majorities = $\lfloor n/2 \rfloor + 1$.

1: **writer, write**(v): $s \leftarrow s + 1$ (own sequence, persistent); send $\langle \text{STORE}, s, v \rangle$ to all
2: complete on acknowledgments from a majority
3: **reader, read**(): send $\langle \text{QUERY} \rangle$ to all; await replies from a majority

4: adopt (s^*, v^*) = the maximal-stamp reply
5: **write-back:** send $\langle$STORE, $s^*, v^*\rangle$ to all; await a majority's acknowledgments (skippable iff the query replies were unanimous at s^*)
6: **then** return v^*
7: **replica, on** $\langle$STORE, $s, v\rangle$: if $s >$ stored stamp, store (s, v) (durably); acknowledge regardless
8: **replica, on** $\langle$QUERY$\rangle$: reply with stored (*stamp*, *value*)

Line 5 is the load-bearing clause (Theorem 6.2); line 7's monotone store is what makes lodging idempotent and helping safe.

The crime scene rerun under the clause: the first reader finds version two at Bingo; before she is answered, version two is lodged at a majority; the second reader's quorum of two must intersect it. The flicker is dead — once seen, forever seen.

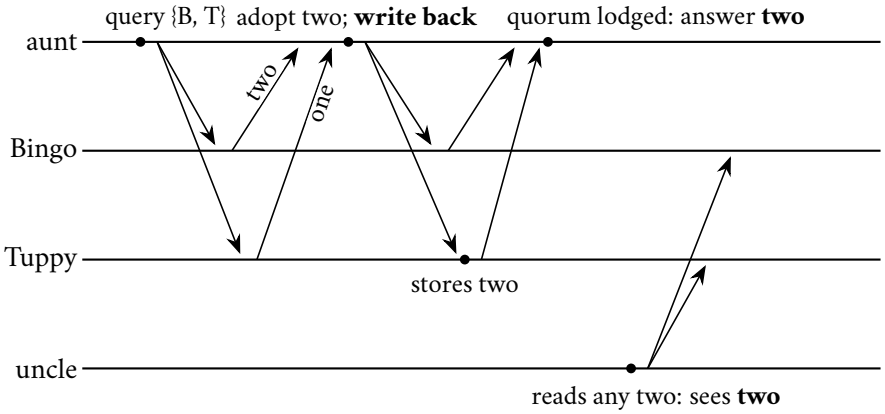


Exhibit 6.4 (ABD's write-back kills the flicker). The aunt lodges version two at a majority *before* being answered; the uncle's quorum must intersect it. Once seen, forever seen. Box 14, Theorem 6.2.

Theorem 6.2 (ABD; Attiya–Bar-Noy–Dolev)**.** *In the ABD model box, the protocol of Box 14 implements an atomic (linearizable) single-writer multi-reader register: every execution's completed operations admit a real-time-consistent total order in which each read returns the latest preceding write. Moreover every operation invoked by a correct client terminates, regardless of scheduling — no consensus, no leader, no synchrony.*

Proof of Theorem 6.2. Write operations carry stamps $1, 2, 3, \ldots$ from the single writer (its own sequence; persisted across its crashes by assumption of write-once-per-stamp). Notation: for a completed operation o, $\mathrm{inv}(o)$ and $\mathrm{resp}(o)$ are its real-time endpoints. For a completed write w_k (stamp k), its *lodging quorum* Q_k is the majority whose acknowledgments completed it. For a completed read r, its *adopted stamp* $st(r)$ is the maximal stamp among its query replies, and its *lodging quorum* is the majority acknowledging its write-back of $st(r)$.

Claim 1 (stamp monotonicity along real time). If completed operations o_1, o_2 each lodge stamps s_1, s_2 (a write lodges its own; a read its adopted) and $\mathrm{resp}(o_1) < \mathrm{inv}(o_2)$, then $s_2 \geq s_1$. *Proof:* o_1's lodging quorum holds stamp s_1 (replicas keep the maximum stamp seen: Box 14 line 8) before o_2 begins; o_2's query majority intersects it (Lemma 4.1); the intersection replica replies with stamp $\geq s_1$; a read adopts the maximum, so $s_2 \geq s_1$; a write's stamp exceeds every stamp the writer has used, and the writer's own previous operations include lodging s_1 or earlier — for the single-writer case, $s_2 = s_1 + j$, $j \geq 1$, when o_2 is a write following o_1 in the writer's sequence; when o_2 is a write not following o_1 (i.e., o_1 is a read that adopted a stamp the writer issued earlier), the writer's next stamp still exceeds all issued stamps, hence $\geq s_1$.

Claim 2 (reads return the stamped value). All operations with stamp k concern the single value v_k written at k: replicas store (stamp, value) pairs and Box 14 adopts them jointly; no creation (Chapter 1's FL3) bars other values.

The linearization. Order operations by primary key (lodged stamp), with ties broken: the write of stamp k first, then reads adopting k in the real-time order of their responses (concurrent reads in either order). This is a total order on completed operations. *Real-time consistency:* suppose $\text{resp}(o_1) < \text{inv}(o_2)$ but o_2 precedes o_1 in the constructed order. Then $s_2 < s_1$, or $s_2 = s_1$ with an ordering violation among the tie rules. The first contradicts Claim 1. For the second: equal stamps put the write first — but a write with stamp s cannot be invoked after the response of *any* operation lodging s, since lodging s requires the writer to have issued it (Claim 2 and no creation), i.e., $\text{inv}(w_s)$ precedes every lodging of s (though w_s's own response may follow). The delicate case is a read r with $\text{resp}(r) < \text{inv}(w_s)$ and $st(r) = s$: impossible, again by no creation — r adopted s from some replica, which received it from the writer's phase for w_s, which follows $\text{inv}(w_s)$. Ties among reads follow response order by construction. Hence the order extends real time. *Read correctness:* in the constructed order, the latest write preceding a read r is $w_{st(r)}$ (later writes have larger stamps and are ordered after), and r returns $v_{st(r)}$ by Claim 2.

Termination. Each phase awaits a majority of replies from n replicas of which a minority may crash: the correct majority eventually replies (reliable delivery, no waiting on any specific replica); no phase can block forever and no phase's completion depends on any race being decided. Both operations complete in two round trips (one, for the optimized read of Rem. 6.5(ii)). □

Remark 6.5 (three notes on the theorem). (i) *Helping*: a reader adopting a stamp lodged at fewer than a quorum completes the stranded write; doubt is resolved toward durability by whoever trips over it. (ii) *The toll*: reads pay a write's postage exactly when the quorum disagreed; the unanimous case may lawfully skip the write-back (Box 14, line 5). (iii) *The border*: the register cannot elect, lock, or compare-and-swap (Herlihy rank one); any operation whose outcome depends on winning a race needs Chapter 4's machinery. CAP is paid by the minority partition blocking.

Why FLP casts no shadow. FLP's fence stood on the adversary's power to keep two futures alive, and a register protocol has no fork to hold open — ABD never decides between anything; it lodges copies with majorities and adopts maxima, operations no schedule can leave on a fence. Patience assembles a majority; nothing further need be agreed. The CAP invoice is paid where Chapter 3 said it must be: the minority side of a partition waits.

Fairness to the school. Dynamo claims no linearizability — the paper claims eventual consistency with causal paperwork, and the descendants that bolted the write-back on offer it as a *setting*, at a documented latency premium, because the founding judgment was precisely that the cart does not need once-seen-forever-seen and should not pay for it. The set piece is a boundary marker, not a scandal, and it issues the one diagnostic question for any vendor page waving quorum arithmetic as a consistency claim: *where does the read-back live, and does it block?*

The Herlihy border. Linearizability is the badge Chapter 7 crowns strongest; yet registers rank one on Herlihy's ladder: reading and writing, however strongly consistent, cannot elect, cannot lock, cannot compare-and-swap — two processes with a linearizable register still cannot agree on one bit against a single crash. Linearizable *storage* is cheap — majorities and diligence; linearizable *decision* costs a parliament. The design-review test: does any operation's outcome depend on winning a race? If yes, parliament (Chapter 4); if no, majorities and diligence suffice, whatever the consistency badge required.

6.5 Part Five — The Two Kettles

Multi-writer reality, and the payment of a debt planted in Chapter 2's rival kettles: in the leaderless school, two writes to the same key from different clients, landing at different coordinators, are the normal case. If neither write's history includes the other they are *concurrent — merely elsewhere*, and no lawful system may pretend one superseded the other. The school keeps, per key, exactly Chapter 2's instrument, run as a loop: reads return causal context; writes declare their ancestry by carrying it back; coordinators compare paperwork.

Definition 6.6 (version vectors; siblings; dots)**.** Per key, each stored version carries a vector of (coordinator, counter) entries; reads return version(s) with a *context* (the merge of returned vectors); writes carry their read-context as declared ancestry. A stored version strictly below an incoming context is superseded; incomparable versions coexist as *siblings*, returned together until an application write merges them under a joint context. Deletion is a tombstone version. The *dotted* refinement identifies each version by its precise coordinator event (the dot) atop a summarized past, eliminating the false-sibling and false-order artifacts of bare per-coordinator counters (Preguiça et al., reading notes).

The kettle collision, distilled. The episode stages it in full; the skeleton: one client reads a cart (one umbrella, context supplied) and adds a kettle via Bingo; the same client, elsewhere and unsynchronized, reads the same cart and adds a different kettle via Gussie. The two contexts are incomparable — each descends from the umbrella-only cart, neither from the other — so two carts now exist, both real, provably concurrent, and the next read receives both siblings with their paperwork. Resolution belongs to the party that knows what a cart *means*: the application, whose merge for a cart is union — both kettles, the umbrella once — written back bearing both contexts, a descendant of each parent. (The merge runs in the reading client: the school's client libraries famously thick, its servers famously thin.)

The three merge laws. The union obeyed three laws the shop has been obeying unnamed — not etiquette but load-bearing algebra, precisely what lets replicas repair in any order, along any routes, meeting any duplicates, and still converge:

- **Order-indifference:** the result cannot depend on the order siblings are presented, or two readers repairing the same rivalry would manufacture new rivals.
- **Repetition-indifference:** the nets may deliver the same version twice; the merge must absorb duplicates, or the janitors breed the dust they sweep.
- **No invention:** every merged item traceable to some parent, or the reconciliation is a forgery.

The laws receive their government names, and the convergence theorem they underwrite, in Chapter 7.

Due diligence, three footnotes.

- **Which mode holds your keys.** Siblings are the honest mode; last-writer-wins remains on the school's menu as the dishonest default, and more than one descendant shipped timestamp adjudication where vector paperwork belonged, or made the quiet mode the default. Kingsbury's laboratory has published the autopsies: lost updates reproduced under partition on demand. The first audit question of a school system is not the quorum tuning but the mode.
- **Tombstones.** An erased entry looks, to every net of Part Three, like an entry the eraser never had — anti-entropy will helpfully restore it. Lawful deletion is a **tombstone**: a causally stamped write

that propagates, wins its comparisons, and suppresses resurrection. Tombstones must eventually be garbage-collected, which reopens Chapter 1's windowed-deduplication argument in miniature: discard one while any replica might still reintroduce its ancestor, and the ghost returns — hence the school's tombstone-lifetime settings, and the operators who met them the hard way.

- **Dots.** A bare per-coordinator counter says only how many updates a version has seen, never *which* event it is; the classical vector in coordinator hands can file concurrent versions (different clients through one coordinator) as ordered, or spuriously multiply siblings. The dotted refinement of Def. 6.6 restores exact causality at coordinator granularity. The pattern, seen thrice now: causal bookkeeping is an engineering budget, and every discount below full honesty is a specific, nameable lie one elects to tolerate.

What the kettles still lack is a *general* theory of merging — union worked because carts are sets that grow; counters, documents, and calendars want their own laws — and the discovery that a whole algebra of mergeable objects exists, convergence theorem attached, is Chapter 7's second half.

6.6 Part Six — The Menu as Decision Procedure

The menu folded into the form it takes at a whiteboard. The season's systems, read back through Def. 6.1's grid:

- **Primary-backup, synchronous:** one acceptor, fixed agreement — the classical bank.
- **Raft / multi-Paxos:** one acceptor per term, quorum agreement — the throne with arithmetic underneath.
- **Dynamo, classic tuning:** any acceptor, sloppy quorum — the December machine.
- **Chain replication:** one acceptor, agreement = the entire chain — geometry standing in for arithmetic.

Every system on the market is a cell in that grid, whatever its brochure says, and most production incidents in replicated estates are one of three category errors: a recovery window nobody priced, a second primary nobody planned the reconciliation for, or a quorum overlap mistaken for an ordering guarantee.

The grid, drawn. The filing cabinet itself — acceptance down the side, acknowledgment across the top, the season's tenants in their cells. The uninhabited cells are labelled with the reason nobody lives there, since *why a cell stands empty* is as much of the theory as who occupies the rest:

	asynchronous (acceptor alone)	**synchronous / semi-sync** (a fixed set; any k of them)	**quorum** (N, R, W)
single leader	classic primary-backup: cheap confirmation, and the RPO window priced on the index card (Prob. 6.1)	the classical bank; semi-synchronous the production compromise — the write provably exists beyond the primary, address unstated	Raft / multi-Paxos (Chapter 4): one acceptor per term, quorum agreement — the throne with arithmetic underneath
multi-leader	geo-replication as practised: conflicts by design, adjudication owed in writing (Part Two)	*uninhabited — the cross-site round trip is precisely what multi-leader exists to avoid*	*thinly settled — egalitarian parliaments exist in the literature; none sat this season*
leaderless	the loosest tuning ($R = W = 1$): last-writer-wins' one honest home, the telemetry stream (below)	*uninhabited — a fixed set of acknowledgers is a leader by another name*	Dynamo at the classic tuning (sloppy quorum, hinted handoff); the strict-quorum tunings of Part Four

Two lodgers file under standing exceptions: chain replication lives in the single-leader row with agreement = the *entire chain* — the acknowledgment axis's far extreme, geometry standing in for arithmetic — and Chapter 5's coordination kernel is the single-leader quorum cell rented rather than run.

Choosing by workload. In the order that actually decides: consistency need first (will the business absorb siblings, stale reads, lost updates? Chapter 7 arms the question properly), then the latency budget (synchronous hops and blocking write-backs are purchases against it), then topology. Four workloads, worked:

- **The shopping cart:** siblings absorbable, loss forbidden, latency sacred, topology global — leaderless, sloppy, vectors, union-merge. Dynamo is not *a* choice for the cart; the cart is why Dynamo exists.
- **The payments ledger:** loss forbidden *and* siblings forbidden — a double negative no quorum tuning satisfies; a throne with synchronous acknowledgment, or the parliament outright. December may queue.
- **The service-configuration store:** small, hot, catastrophic to fork — Chapter 5's rented coordination kernel, fine print included.
- **The telemetry stream:** last-writer-wins' one honest home — each sample supersedes its predecessors by the data's own meaning, so the loosest tuning serves and the tie-breaking rule discards only what the workload had already discarded. Mark the reasoning shape: supersession was in the *schema* before it was ever in the clock; the same test, run the other way, convicted the shipping address in Part Two.

Estates are plural, and the menu is consulted per dish; the school wars of the trade press are category errors.

The five questions. The procedure as exercised on a brochure — asked aloud at review, answers in writing:

1. Who may accept a write — one desk, one per region, or anyone?
2. Who must agree before the client hears yes — and if a quorum, is it strict, or does it slosh to understudies in weather?
3. If any acknowledgment is asynchronous, what is the recovery point — the number, on the index card: seconds of exposure, times writes per second, times the worth of a write?

4. Where does the read-back live, and does it block — is the consistency on the label bought with lodged evidence, or with arithmetic and optimism?
5. Which mode holds these particular keys — causal paperwork with an owned merge, or timestamps quietly eating the loser — and if tombstones are anywhere in the design, what is their lifetime, and who chose it?

A vendor — or a colleague, or a past version of oneself — who cannot answer all five has not designed a replication scheme; he has purchased a rumour of one.

A temperament note, folklore honestly labelled. Leader systems fail *legibly*: a throne to inspect, a failover to narrate, an incident with a protagonist, reconstructible from three logs. The school fails *diffusely*: latency creeps, sibling counts rise, hinted larders swell on machines nobody was watching; no single moment goes wrong because no single place can. The first demands an organization that executes a runbook crisply at night; the second, one that trusts dashboards and sweeps its corners on schedule. Neither is superior — but a team with the first temperament operating the second architecture will spend its incidents searching for a throne that does not exist.

6.7 The Whiteboard

For the design review.

1. A replication scheme answers two questions: who may accept a write; who must agree before acknowledgment. File every brochure adjective in the grid.
2. Asynchronous replication is a chosen data-loss window: seconds of lag × writes per second × worth of a write, plus the tail case — the index card, aloud.
3. Quorum overlap buys durability and freshness-on-offer, not linearizability; ask where the read-back lives and whether it blocks. Linearizable storage is majorities and diligence; linearizable *decision* is a parliament — the race test decides which you need.
4. If concurrent writes are possible: prevent them with a throne, or keep the evidence (causal paperwork, owned merges). Last-writer-wins is data loss with a tie-breaking rule; know where it holds your keys.

The register. Issued: #17 — the kettles must merge: the algebra of mergeable objects and its convergence theorem, payable Chapter 7; and the word *consistent* owes this course its definition — folded into the standing Chapter 7 ceremony with #11, #13, and #16. Word-count delta recorded: the episode ran 8,868 words against the 10,000 floor at delivery (Session 6); the assembly session (Session 13) expanded it to 10,174 with staged, within-spec material — the retail latency arithmetic; the multi-leader topology races; the merge function's three laws; the telemetry workload and the five-question review catechism; the two incident reports — under a LEDGER §5 drift note, per the standing rule that delivered scripts are amended on the record only.

6.8 Problems

Tier I — Calisthenics

Problem 6.1 (the index card)**.** A primary handles two thousand writes per second; asynchronous replication lags one second typically, twelve seconds at the recorded worst; a write is a customer order worth, on

average, thirty pounds of eventual revenue. (a) Compute typical and worst-case recovery-point exposure, in writes and in pounds. (b) The vendor proposes semi-synchronous acknowledgment at a cost of two milliseconds median latency: restate the trade in one sentence a merchant would sign. (c) Which number on the card does a partition change, and which does it not?

Problem 6.2 (tunings drill)**.** For $N = 3$: classify each tuning's guarantees (durability on acknowledgment; completed-write visibility; flicker possibility; write availability with one replica down): (a) W=1, R=1; (b) W=3, R=1; (c) W=2, R=2 strict; (d) W=2, R=2 sloppy; (e) W=2, R=2 strict with blocking write-back.

Problem 6.3 (parcels and hints)**.** Preference list Bingo, Tuppy, Gussie, Barmy, Oofy; N=3, W=2. Gussie is down for an hour; twelve writes to the key arrive. (a) Where do the copies land, and what hints exist? (b) Gussie recovers: describe the handoff traffic. (c) During Gussie's absence a strict-quorum read R=2 consults {Tuppy, Gussie}... it cannot; re-answer for the school's actual read path and say what may be missed. (d) State in one sentence what sloppy quorums trade against Lemma 4.1.

Problem 6.4 (sibling paperwork)**.** Stored versions of a key carry vectors $A = (\text{B:2}, \text{T:1})$ and $B = (\text{B:1}, \text{T:2})$. (a) Ordered or rivals? (b) A write arrives with context (B:2, T:2): what does it supersede, and what vector does its version carry (coordinator Tuppy)? (c) A write arrives with context (B:2) only: enumerate the resulting sibling set. (d) Which of these outcomes would a bare per-key timestamp have silently destroyed?

Problem 6.5 (fingerprint economics)**.** A million-entry range, branching factor thirty-two, leaves of a thousand entries. (a) Tree depth? (b) Fingerprints exchanged when the libraries are identical; when exactly one leaf range differs; when eight scattered ranges differ (bound via Lemma 6.1). (c) Compare with shipping the library, at one fingerprint's size per thousandth of an entry.

Tier II — The Real Thing

Problem 6.6 (chain replication is linearizable)**.** Model the chain of Exhibit 6.1: nodes apply writes in arrival order and forward; the tail acknowledges writes and serves reads from local state; assume reliable FIFO links (purchased per Chapter 1) and no failures (reconfiguration is excluded). (a) Show every node applies the same writes in the same order (induction down the chain). (b) Define the linearization: order each write at its tail-application instant and each read at its tail-service instant; show this order is total, real-time consistent, and read-correct. (c) Mark exactly where "acknowledged only at the tail" is spent — exhibit the violation if the head acknowledged. (d) Why does read-at-head break linearizability but read-at-tail not break write availability? *(Full solution.)*

Problem 6.7 (regularity, proved and bounded)**.** For strict $R + W > N$ with a single writer and versioned values: (a) prove completed-write visibility (Rem. 6.4); (b) prove reads never return invented or superseded-then-revived values (no-creation plus stamp monotonicity at replicas); (c) exhibit the flicker with N=3, R=W=2 and mark which clause of Def. 6.3's atomicity it violates; (d) state the smallest protocol change that restores atomicity, and cite the theorem. *(Full solution: the set piece, formalized.)*

Problem 6.8 (sabotage: acknowledged at the send)**.** A coordinator, "pipelining for the percentiles," counts a write toward W when the STORE letter is *sent*, not when the durable confirmation returns; everything else follows Box 13 with N=3, W=2, R=2 strict. (a) Exhibit an execution in which the client is acknowledged and *no surviving replica holds the write* (two crashes permitted, or one crash plus one lost letter — the Post's ordinary charter). (b) State which line of the recovery-point index card this design quietly rewrites, and by how much. (c) Exhibit the read-side symptom (an acknowledged write invisible to every subsequent R=2 read) and explain why read repair never heals it. (d) Give the repair and its true latency cost, and reconcile with the percentile commandment honestly. *(Certified fatal per the standing rules: the execution is written out in the solutions and drawn as Exhibit 6.5.)*

Problem 6.9 (the LWW audit)**.** A contact-record schema: address (corrected rarely, catastrophic to lose); phone (idem); marketing preferences (latest wins by intent); last-seen timestamp (monotone gauge); cart items (grow-and-merge). The store offers per-column LWW or sibling mode. (a) Assign each column a mode with one-sentence justifications. (b) For each LWW assignment, state the exact anomaly accepted and its worst business case. (c) Rewrite the last-seen column to be LWW-safe by construction. *(Solution sketch.)*

Tier III — For the Ambitious (starred)

Problem 6.10 (★ multi-writer ABD)**.** Extend Box 14 to many writers: a write first queries a majority for the highest stamp, then writes at (that stamp +1, breaking ties by writer identity). (a) Show stamps still totally order writes. (b) Re-prove Claim 1 of Theorem 6.2 — where does the write's query phase substitute for the single writer's private sequence? (c) Construct the linearization and verify real-time consistency. (d) Count round trips per operation and compare with the single-writer costs. *(Hints only.)*

Problem 6.11 (★ repair under fire)**.** Design a read-repair discipline that preserves atomicity when repairs race concurrent writes: specify how a repair's stamp interacts with in-flight writes (never overwrite a higher stamp; monotone stores), show a naive repair that re-lodges a *superseded* version cannot occur under your rule, and identify the exact property of Box 14 line 7 you are relying on. Then weaken the model: with sloppy quorums, exhibit the residual anomaly no repair discipline closes, and say which net (anti-entropy) bounds its lifetime. *(Hints only.)*

6.9 Hints

Hint (Problem 6.8)**.** Let both STORE letters dawdle; acknowledge; then crash the coordinator (its local copy dies with it) and lose one letter under fair loss. One survivor never heard; the other heard and crashed before persisting — or simpler: coordinator counts its own send too.

Hint (Problem 6.10)**.** (b) The query phase makes the writer's stamp exceed every stamp lodged before its invocation — exactly what the private sequence gave for free. Ties: lexicographic (stamp, writer).

Hint (Problem 6.11)**.** Monotone stores make every lodging idempotent and order-insensitive; a repair is just a write-back with an old stamp, and line 7 discards it wherever it has been superseded.

6.10 Solutions

Tier I in full (tersely); Tier II in full for Problems 6.6, 6.7, and 6.8 (the last is the sabotage certification), sketch for Problem 6.9; hints only for Tier III.

Solution (to Problem 6.1)**.** (a) Typical: two thousand writes, sixty thousand pounds at risk; worst: twenty-four thousand writes, seven hundred twenty thousand pounds. (b) "Two milliseconds on every order, to make losing any confirmed order impossible" — a sentence merchants sign quickly. (c) A partition stretches the lag (the exposure), but not the per-write worth; and it converts "typical" to "worst" precisely when failover is most likely — the card's two rows correlate at the wrong moment, which is the honest reading of the GitHub afternoon.

Solution (to Problem 6.2)**.** (a) Durability: one copy; visibility: none promised; flicker: trivially (staleness unbounded); availability: full. (b) Durable everywhere on ack, so any read sees it (visibility by containment); but W=3 blocks writes with one replica down — the unanimity trap of Chapter 4's Part One. (c) Visibility yes

(overlap); flicker possible (Exhibit 6.3); one-down: writes and reads both proceed. (d) As (c) in fair weather; under failure the overlap is probabilistic — a strict read may miss a sloppy write entirely until handoff completes. (e) Atomic register behaviour per Theorem 6.2; the toll is the write-back round on disagreeing reads.

Solution (to Problem 6.3)**.** (a) Copies at Bingo, Tuppy, and Barmy-with-hint (first healthy three); twelve hinted versions in Barmy's larder addressed to Gussie. (b) One stream Barmy → Gussie of the latest hinted versions (superseded intermediates may be coalesced), each discharged on confirmation. (c) The school's read consults the first *R healthy* members — {Bingo, Tuppy} or with Barmy substituted; a read landing on {Tuppy, Barmy} sees everything; on {Bingo, Tuppy} likewise (both are home members holding all twelve); the misfortune requires reads consulting a member that received neither the writes nor the hints — possible only under further failures; state it as: sloppiness moved the risk from write-time refusal to read-time likelihood. (d) Sloppy quorums trade the *certainty* of quorum intersection for availability, leaving intersection to probability plus the nets.

Solution (to Problem 6.4)**.** (a) Rivals: each ahead in one coordinate. (b) Context dominates both: supersedes both siblings; new version (B:2, T:3). (c) Context covers only *A*'s Bingo entries... compare: (B:2) versus *A*: not ≥ (*A* has T:1); versus *B*: not ≥. The write joins as a third sibling: $\{A, B, C\}$ with C = (B:3) (coordinator Bingo advancing its own entry). (d) A timestamp would have kept exactly one of the three — destroying two concurrent rivals rather than presenting them.

Solution (to Problem 6.5)**.** (a) A thousand leaves, branching thirty-two: depth two ($32^2 = 1024$). (b) Identical: one exchange. One differing leaf: roots, then one node's thirty-two children, then the leaf's thirty-two: at most sixty-five plus the range transfer. Eight ranges: bounded by $1 + 32 \cdot 8 \cdot 2 = 513$ fingerprints. (c) Shipping the library costs a thousand leaf-ranges' data; the descent costs hundreds of fingerprints — three orders and a bit, in the lemma's favour, growing with the library while the descent grows only with the disagreement.

Solution (to Problem 6.6)**.** (a) Induction down the chain: node i+1 receives exactly node i's applied sequence over a reliable FIFO link and applies in arrival order; base: the head's own sequence. Hence the tail's applied sequence is a prefix of every node's, and all agree on order. (b) The tail's applications and services are events of one process — totally ordered by its local sequence. Real time: a write's acknowledgment follows its tail-application; a read's response follows its tail-service; so if $\text{resp}(o_1) < \text{inv}(o_2)$, o_1's tail event precedes o_2's. Read correctness: the tail serves its local state, i.e., exactly the writes tail-applied so far, latest last. (c) If the head acknowledged: a write durable nowhere past the head is confirmed; head dies; the tail never applies it — a confirmed write invisible forever to reads (and lost): both atomicity and durability die. The violation trace is Exhibit 6.3's cousin with the chain's geometry. (d) Read-at-head returns state including unacknowledged writes — exposing values that may yet vanish (the converse sin); writes never wait on reads, so tail reads cost writes nothing save the tail's cycles.

Solution (to Problem 6.7)**.** (a) A completed write lodged at W replicas; a later non-overlapping read consults R; $R + W > N$ gives a common replica (Lemma 4.1); versions carry the writer's rising stamps, replicas store monotonically, so the common replica reports a stamp ≥ the completed write's, and the read adopts the max ≥ it; no higher stamp exists absent later writes. (b) By no-creation every returned pair was stored, every stored pair was sent by the writer; monotone stores bar revival of superseded stamps at any single replica; the adopted max is thus some genuinely-written, never-superseded-at-its-replica value. (c) The Exhibit 6.3 run: aunt's read adopts stamp two; uncle's later read adopts stamp one — in any total order extending real time the uncle's read follows the aunt's, whose latest preceding write must then be the stamp-two write; returning stamp one violates the read-correctness clause. (d) Blocking write-back before answering: Theorem 6.2.

Solution (to Problem 6.8)**.** (a) The execution, drawn as Exhibit 6.5. Coordinator Bingo receives the write; *sends* STORE to Tuppy and Gussie and — counting sends — acknowledges Bertie at once, while persisting locally in the background. Then: Bingo crashes before his local persist completes; the letter to Tuppy is lost (fair loss, clause two); the letter to Gussie dawdles and is delivered — to a Gussie who crashes after receipt, before the durable store, and recovers empty of it. Client acknowledged; no surviving durable copy anywhere. Even the gentler variant — only Bingo's crash — leaves one copy where two were promised: the W=2 durability contract is void either way. (b) The card's exposure line: acknowledged-but-durable-nowhere is recovery-point loss requiring no failover at all — loss without even a primary death; magnitude: every write inside one postal round trip, at all times. (c) All three replicas answer subsequent reads with the old version (nothing durable arrived); read repair propagates only what some replica *has* — it cannot heal what nowhere exists; the acknowledged write is simply gone, and no net notices, because every ledger is internally consistent: Chapter 1's silent divergence, upgraded to silent void. (d) Count confirmations of durable application, as Box 13 line 3 says; the honest cost is one round trip plus the replicas' fsync — and the percentile commandment is served lawfully by batching fsyncs and by sloppy quorums, which trade *address*, never *existence*. Fatality certified.

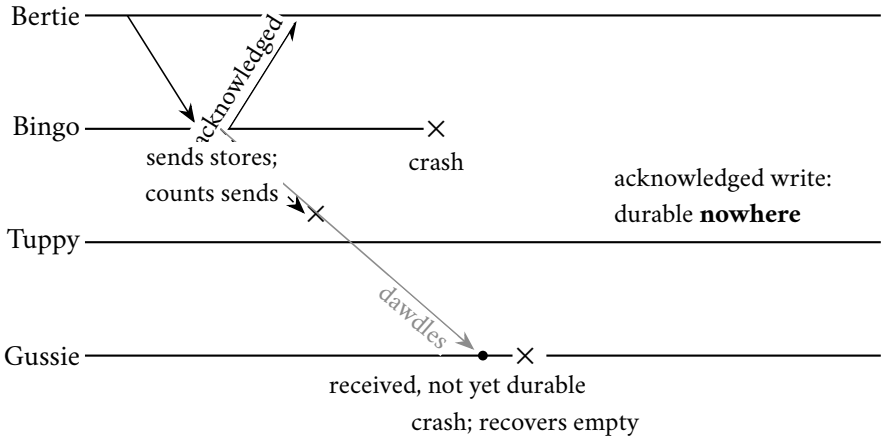


Exhibit 6.5 (acknowledged at the send; Solution 6.8). The coordinator counts letters posted, not durability confirmed; two ordinary misfortunes later, the confirmed write exists nowhere and no net can heal a void. Fatality certified.

Solution (to Problem 6.9)**.** (Sketch.) Address, phone, cart: sibling mode (loss catastrophic; merges: field-wise ask-the-user, ask-the-user, set union). Marketing preferences: LWW acceptable if truly "latest intent wins" — anomaly: a concurrent opt-out and opt-in resolve by clock, worst case a regulatory complaint; many shops therefore prefer siblings here too. Last-seen: LWW-unsafe as a raw timestamp write (clock skew regresses it); rewrite as max-merge — a monotone gauge is a semilattice, and Chapter 7 will name that observation into a theory.

Solution (to Problems 6.10 and 6.11)**.** By standing policy: hints only (the Hints section above). For Problem 6.10(d): two round trips for both operations (query then store), against ABD's one for writes and one-or-two for reads.

6.11 The Reading

DeCandia et al., "Dynamo: Amazon's Highly Available Key-Value Store," SOSP 2007. The chapter's text. Read §4 against Box 13 mechanism by mechanism; §4.4's version-vector discussion against Def. 6.6; the

evaluation's percentile tables against Part One's second commandment. Note on every page the watermark: each mechanism is a priced answer to *always writeable*.

Attiya, Bar-Noy, and Dolev, "Sharing Memory Robustly in Message-Passing Systems," JACM 1995. The register theorem in the founders' hand. Read after the proof above, for the pleasure of the original's economy; their multi-writer remarks seed Problem 6.10.

van Renesse and Schneider, "Chain Replication for Supporting High Throughput and Availability," OSDI 2004. Short and geometric; read §2–3 against Problem 6.6, and the reconfiguration section for what Exhibit 6.1 deliberately excluded.

The shelf. The GitHub incident analysis of October 2018 — read with a stopwatch: mark where forty-three seconds becomes twenty-four hours. Terry et al., "Managing Update Conflicts in Bayou" (SOSP 1995) — dependency checks and merge procedures in the ancestors' hand. Preguiça et al. on dotted version vectors — the footnote's machinery. Demers et al., "Epidemic Algorithms for Replicated Database Maintenance" (PODC 1987) — the gossip mathematics, lovely throughout. Bailis and Kingsbury, reread with landlord eyes.

Standing companion: Kleppmann, chapter 5. The replication chapter proper: his taxonomy of leader, multi-leader, and leaderless designs is this chapter's grid in prose, and his "problems with replication lag" section is Chapter 7's doorstep.

Chapter 7

Consistency: A Field Guide

Companion to Episode Seven, the season's hinge. The episode tells the story — the four professions' meeting, the telephone across the counter, the two doctors, the wedding of the kettles; this chapter keeps the record: the client's-eye doctrine and the hierarchy of models, linearizability with teeth and locality proved, the session guarantees and their receipts, eventual consistency unmasked, the transactional axis with write skew arraigned, the kettles wed under the strong-eventual-consistency theorem proved in full, and CALM at the horizon. Statements, proofs, price tags, problems.

7.1 Part One — The Client's-Eye Doctrine

The most overloaded word in computing has four professions behind it, each employing a different technical term under the one spelling; contracts and postmortems have been written across the ambiguity. The episode stages the four-way meeting — four nods, one word, zero shared propositions; this chapter pins the word to the bench: every promise a replicated system can make about what its clients observe, arranged in a hierarchy, priced, and given its government name.

The four senses of "consistent."

- **The theorist's (CAP):** the C of Chapter 3 means, precisely, linearizability — the replicated service behaving as one up-to-date copy.
- **The database hall's (ACID):** a transaction carries the database from one state satisfying the application's invariants to another — famously the one letter of the acronym naming an obligation of the application, not the engine.
- **The replication engineer's:** the degree to which the copies agree — a spectrum with "eventual" at one end.
- **The working programmer's:** nothing on the screen embarrasses anyone this afternoon.

Jepsen, the consumer-protection bureau. Since 2013, Kyle Kingsbury's **Jepsen** harness has taken the market's databases at their brochures' word — this chapter's doctrine in overalls:

- **raise** the system under test as a small estate of nodes;
- **load** it with clients performing randomly generated operations — reads, writes, increments, compare-and-swaps;

- **sabotage** it in the manner of Chapter 1's documentary villains: cut the network, pause processes mid-stride, skew clocks;
- **log** every invocation and response with its time;
- **check** the transcript afterwards for a lawful explanation under the model the vendor advertised.

No source code, no benefit of the doubt. Vendors claiming linearizability were shown histories no linearizable system could produce; a document store lost acknowledged writes under partition; marketing pages were rewritten across an industry. The vocabulary below is the language in which the market's claims are now audited; the chapter itself is the season's hinge — the customs house where the first half's guarantees are weighed, named, and priced.

The doctrine. One sentence governs everything, and adopting it is half the education: *a consistency model is a contract about observable histories, and it says nothing about implementation* — nothing about quorums, logs, leaders, vectors; only what clients can be shown. Clients interact through *operations*, each with an invocation $\mathrm{inv}(o)$ and a response $\mathrm{resp}(o)$, the open interval between them the operation *in flight*. Real-time precedence: $o \prec o'$ iff $\mathrm{resp}(o) < \mathrm{inv}(o')$; related neither way, the operations are *concurrent*. The *history* H is the whole transcript — every invocation and response, from all clients, with times — and it deliberately forgets everything inside the machine room: two systems that can produce the same histories are, to any contract, the same system, which is why the vocabulary transfers wholesale from the multiprocessors where it was born to planet-scale storage.

Models as paddocks. A model is a *set of permitted histories*; the system promises that every transcript it ever produces lies in the set. Before any definition:

- **strength is containment:** stronger models are smaller sets — fewer histories, more coordination — so comparisons become containment checks; hence the lattice;
- **the contract is a floor,** not a description: a healthy cluster produces summit-grade transcripts all week under a floodplain contract; audit what the contract permits on the worst night of the year;
- **testing is membership checking** — the Kingsbury manner, exactly;
- **the sequential specification** — what the object would do on one machine, one operation at a time — is the measuring rod: every model is a rule about how a messy concurrent history may be *explained* by a one-at-a-time story, differing only in what "explained" may forget.

The chapter's models, fixed before any formal claim:

Model of this chapter

For the hierarchy (Defs. 7.1–7.9, Thms. 7.3 and 7.5). Clients interact with named shared objects through operations; the system is a black box producing histories. Asynchrony, crashes, and partitions live *inside* the box and appear only through the transcripts the box can emit. Client clocks play no role; "real time" is the external observer's line on which invocations and responses are placed. Histories are *well-formed*: each client has at most one operation in flight (its own operations are sequential).

For transactions (Defs. 7.11–7.12, Prop. 7.6). Transactions are bracketed programs of reads and writes over named items, submitted by clients; the store is multiversion; commits are atomic events. Transaction programs may branch on what they read (the doctors' guard is a program, not a history).

For CRDTs (Defs. 7.13–7.14, Thm. 7.7). n replicas, each holding a state from a set S; updates execute at any single replica without coordination; replicas exchange *states* over an anti-entropy channel (Chapter 6's nets): eventual, unordered, possibly duplicated delivery to every correct replica; no clocks, no quorums, no leader. Crash-stop replicas simply cease gossiping.

The formal vocabulary follows the doctrine exactly — the history first, then the one-machine story it must be explained by.

Definition 7.1 (histories)**.** An *operation* o on object x by client c comprises an invocation at time $\mathrm{inv}(o)$ and, if completed, a response at time $\mathrm{resp}(o) > \mathrm{inv}(o)$, carrying argument and return values. A *history* H is a finite set of operations, possibly with some *pending* (invoked, unanswered), well-formed per the model box. Real-time precedence: $o \prec o'$ iff $\mathrm{resp}(o) < \mathrm{inv}(o')$; operations unrelated by $\prec$ are *concurrent*. $H|x$ and $H|c$ denote the restrictions to object x and client c. A *completion* of H removes some pending operations and supplies responses to the rest. A *consistency model* is a set of histories; H satisfies the model iff some completion of H lies in the set; model A is *at least as strong as* B iff $A \subseteq B$.

Definition 7.2 (sequential specification)**.** A *sequential history* is one whose operations are totally ordered by $\prec$ (no overlap). Each object type fixes a *sequential specification*: the set of single-object sequential histories it accepts (a register: every read returns the latest preceding write, or the initial value; a set: membership per the adds and removes so far; a queue: first in, first out). A multi-object sequential history is *legal* iff each per-object restriction lies in its object's specification.

One admission before the climb, made freely and with its citation.

Remark 7.3 (checking is hard)*.* Deciding whether a given complete history of reads and writes is linearizable is NP-complete in general (Gibbons–Korach 1997); the search over orderings of concurrent operations is real. The practical bureaus prune: short transcripts, simple objects, per-key locality (Thm. 7.3 audits shard by shard). The definitions are crisp; auditing against them is work.

7.2 Part Two — The Summit: Linearizability, With Teeth

The summit — the strongest of the landings — retires three debts at a stroke: the C of CAP, left as a promissory note in Chapter 3; the phrase *behaves as a single machine*, banked unexamined in Chapter 5; and the fine print on lease-served reads, deferred the same evening. All three were always the same debt; the payment is Herlihy and Wing's 1990 paper in TOPLAS.

Definition 7.4 (linearizability; Herlihy–Wing)**.** A complete history H is *linearizable* iff there is an assignment of a *linearization point* $p(o)$ to each operation, with $\mathrm{inv}(o) < p(o) < \mathrm{resp}(o)$ and all points distinct, such that the sequential history executing the operations instantaneously at their points is legal. Equivalently: there is a legal sequential history S on the same operations whose total order extends $\prec$. A history with pending operations is linearizable iff some completion is.

The definition, plainly. Every operation *appears* to take effect atomically at some moment — the *instant*, formally the linearization point — while it was genuinely in flight, and the resulting one-at-a-time story must be lawful for the object on a single machine. Two clauses carry all the weight: the story must be *sequentially lawful*, and the instants must respect *real time* — non-overlapping operations are never reordered in the explanation. Overlapping operations may be ordered either way: the definition's entire mercy, and exactly the freedom Chapter 2 taught us the world offers.

The founders' queue. Two ways to die, from Herlihy and Wing's own first exhibit:

- **lawful:** Bingo enqueues an apple; overlapping him, Tuppy enqueues a pear; Gussie then dequeues and receives the pear — the overlapping enqueues may be ordered pear-first.
- **against the specification:** Gussie dequeues *twice*, the pear both times — no ordering saves it; no sequential queue surrenders the same fruit twice.

Two deaths — against time, against the specification; the drills (Problems 7.1 and 7.2) hunt a witness to one or prove both hunts fail.

The teeth, tested on Chapter 6's crime scene. The aunt read the new balance; the uncle, invoked *after* her response, read the old. The conviction, stepped:

- the write overlaps both reads: its instant may be placed variously;
- the reads do not overlap: the aunt's instant precedes the uncle's;
- she read new: her instant follows the write's; he read old: his precedes it — so his precedes hers;
- contradiction; no assignment exists; not linearizable — quorums never mentioned; the verdict came entirely from the transcript, literally how Jepsen convicts a database.

The instants are only mystical until they are a line of code: in the chain, a write's point is its application at the tail; in Chapter 5's replicated state machine, application after commitment — why log-served reads preserve the summit and a lagging follower forfeits it; in the ABD register, within the lodging of the stamp at a majority, a read's dragged forward by its write-back. The definition asks only that the instants *exist*.

The three payments, formally.

- **The C in CAP:** Gilbert and Lynch construct exactly a history with no lawful assignment — a completed write on one shore of the partition, a later non-overlapping read of the old value on the other. Weaken the C to any junior tier and the theorem fails to bite: the whole coordination-free storey remains purchasable on both sides. The theorem forbids exactly the summit, during the severance, for the shore without a majority — no more.
- **Behaves as a single machine:** the phrase *is* Def. 7.4 — the single machine is the sequential story, "behaves as" the existence of the assignment.
- **The lease fine print:** a lease converts a timing assumption into a consistency guarantee and inherits its failure modes — let the drift bound fail (one paused leader, one fast clock) and the transcript can contain a stale read invoked after a new write's response; Jepsen-grade audits have caught it in the wild.

The pending-operations clause is where the harness spends much of its cleverness: a crashed client's unacknowledged write separates honest stores from lucky ones.

Remark 7.5 (the two clauses, and pending operations)*.* The equivalence in Def. 7.4 is Lemma 7.1 read in both directions: points give an extension of $\prec$ trivially; an extension of $\prec$ is realizable by points. Two independent ways to fail follow: the story may be illegal (a queue surrendering the same fruit twice), or every legal story may violate $\prec$ (the aunt and the uncle). The completion clause is where audits earn their keep: a crashed client's unacknowledged write may be declared either absorbed or void, but the checker must find *one* coherent verdict for the whole transcript — a write that visibly took effect cannot later be ruled void.

A small lemma carries the equivalence — any total order extending real time is realizable by instants — proved at once; Theorem 7.3 leans on it too.

Lemma 7.1 (interval realization)**.** *Let T be a finite set of operations and $<_T$ a total order on T extending $\prec$. Then there are distinct points $p(o) \in (\mathrm{inv}(o), \mathrm{resp}(o))$, one per operation, whose time order is exactly $<_T$.*

Proof of Lemma 7.1. Enumerate T in $<_T$-order as $o_1 <_T o_2 <_T \cdots <_T o_n$. Let ε be smaller than one n-th of the least positive gap between any two of the finitely many invocation and response times. Define $p(o_1) = \mathrm{inv}(o_1) + \varepsilon$ and, for $k \geq 2$,

$$p(o_k) \;=\; \max\big(p(o_{k-1}),\, \mathrm{inv}(o_k)\big) + \varepsilon.$$

The points are strictly increasing, hence distinct and ordered as $<_T$; each exceeds its own invocation by construction. It remains to show $p(o_k) < \mathrm{resp}(o_k)$. Unwinding the recursion, $p(o_k) = \mathrm{inv}(o_j) + m\varepsilon$ for some $j \leq k$ and $m \leq n$. Suppose $p(o_k) \geq \mathrm{resp}(o_k)$. By the choice of ε, the slack $m\varepsilon$ cannot bridge a genuine gap, so $\mathrm{inv}(o_j) \geq \mathrm{resp}(o_k)$, that is, $o_k \prec o_j$. Since $<_T$ extends $\prec$, this forces $o_k <_T o_j$, contradicting $j \leq k$ (if $j = k$ the interval would be empty, contradicting $\mathrm{inv}(o_k) < \mathrm{resp}(o_k)$). Hence every point lies strictly inside its interval. □

One clause from the founders corrects a natural slander: linearizability is a contract, not a synchronization scheme — it forbids no concurrency, and the property proved next separates it from the coarse machinery (the one big lock) that most cheaply satisfies it.

Proposition 7.2 (nonblocking)**.** *Let H be a linearizable history of total operations (the specification answers every invocation in every state) with a pending invocation i. Then some response r makes H extended by r linearizable. Linearizability never forces a pending operation to wait.*

Proof of Proposition 7.2. Let S be a linearization of H without the pending invocation i. The specification is total, so from the state reached by executing S there is a legal response r for i's operation. Append i's operation at the end of S with response r, and give it a linearization point later than every point of S and later than $\mathrm{inv}(i)$, with $\mathrm{resp}(i)$ later still. The extended story is legal (it extends a legal story by a legal step) and extends $\prec$ (nothing follows i's operation in it, and its point lies within its interval). Hence the extended history is linearizable. No completed or concurrent operation had to be awaited: the clause separates the contract from the coarse machinery that most cheaply implements it. □

Locality. The summit's superpower: **locality**. A system composed of linearizable parts is linearizable as a whole — each key, each queue, each register separately lawful, and any history over all of them lawful too. It sounds like a bookkeeping lemma; it is the licence for the modern storage industry — each shard linearizable, and the estate of ten thousand shards is linearizable, free. The proof's strategy is one sentence, and the sentence is the point: every object's instants are placed on the *shared* real-time line, and real time is the one resource every object already agrees about.

Theorem 7.3 (locality; Herlihy–Wing)**.** *A complete history H is linearizable if and only if $H|x$ is linearizable for every object x.*

Proof of Theorem 7.3. ($\Rightarrow$) A linearization of H restricts to a linearization of each $H|x$: the restricted order still extends $\prec$, and legality of the whole story is by definition per-object legality of its restrictions.

($\Leftarrow$) Suppose each $H|x$ is linearizable: a legal sequential order $<_x$ on $H|x$ extending $\prec$ restricted to x's operations. By Lemma 7.1 choose points $p_x(\cdot)$ for the operations of $H|x$, inside their intervals, ordered as $<_x$; perturbing by less than the least gap, make all points across all objects distinct. Pool every operation of

H with its point and let S be the sequential history executing operations at their points in time order. Three checks. *Same operations:* by construction. *Legality:* S's restriction to object x is ordered exactly as $<_x$ (the points for x realize $<_x$, and pooling with other objects does not reorder them), which is legal; a multi-object sequential history with legal restrictions is legal (Def. 7.2). *Real time:* if $o \prec o'$ then $p(o) < \text{resp}(o) \leq \text{inv}(o') < p(o')$, so S's order extends $\prec$ — and this is the entire trick: the points of different objects are placed on the *same* clock line, and real time is the one order every object already shares. By Def. 7.4, H is linearizable. □

The negative half is the standard interview trap: no weaker model has the property. Two queues, each perfectly sequentially consistent examined alone, jointly admit no single story — the founders' own history, Problem 7.6; work it once and you will never again believe per-object guarantees of the junior kind sum to an estate-wide guarantee. Linearizability composes because real time is global; sequential consistency does not, because private timelines negotiated per object owe each other nothing.

7.3 Part Three — Down the Hierarchy

Descend now, model by model, and at each landing: the promise, the price, and the folk name it has been wearing in your systems. The summit's immediate junior is Lamport's — 1979, a two-page note about multiprocessors: keep the sequential story, keep each client's *program order*, but surrender real time *across* clients.

Definition 7.6 (sequential consistency; Lamport 1979)**.** A complete history H is *sequentially consistent* iff there is a legal sequential history S on the same operations whose total order extends the *program order* of every client (the $\prec$-order within each $H|c$) — but not necessarily $\prec$ across clients.

The agreed story. Every client sees a world consistent with his own actions in order, and all clients see the *same* world — on a private timeline unmoored from the clock on the wall. The surrender drops the coordination that made once-seen-forever-seen hold across clients: Chapter 6's blocking write-back, the round trips. Modern hardware answers: not even this — every processor buffers writes privately, and the relaxed memory models and fence instructions of the hardware trades repeat the same economics from nanoseconds to continents: publishing every write to everyone before proceeding is the price of the agreed story, and both industries found it negotiable. Folk sighting, with an honest brochure: ZooKeeper serves reads from any replica by default — one agreed order of writes, program order per session, no freshness clause — and sells the missing real-time guarantee as a per-read sync operation. Sequential consistency under an accurate label; linearizability as an upgrade at the till.

The next landing surrenders even the single agreed story: clients may see *different* interleavings — but each must respect happens-before, the *roads* of Chapter 2: program order and reads-from, chained.

Definition 7.7 (causal consistency)**.** In a complete history H over registers, the *reads-from* relation links each read to the write whose value it returns. *Happens-before* is $\rightarrow = (\text{program order} \cup \text{reads-from})^+$. H is *causally consistent* iff for each client c there is a legal sequential history S_c over c's operations together with all writes in H, whose order extends $\rightarrow$ restricted to those operations. Different clients may use different S_c; concurrent writes may be ordered differently in different views.

The causal landing. If one operation could have influenced another, nobody, anywhere, may observe the effect before its cause; the merely-elsewhere may lawfully appear in different orders to different observers. Chapter 2's corridor purchased as a contract — the causal-delivery debt, paid — at exactly the machinery

Chapter 2 built: carry the causal paperwork (vector clocks or their rations) and hold mail until its ancestry has arrived. No consensus, no quorum round trips: the strongest model in this guide that remains available under partition and cheap under geography — in its real-time-respecting variant (Rem. 7.10), essentially the *strongest* contract an always-available, partition-tolerant store can honour; the ceiling of the coordination-free storey. Its weakness is equally exact: no agreed story means no arbitrating races — causality orders the roads; it cannot rank the merely-elsewhere. And the paperwork is not free: a write's ancestry fans out under heavy read traffic (read fifty items, depend on fifty histories), and the causal-store literature is largely a literature of pruning and bounding that fan-out. Cheap next to consensus; not cheap next to nothing.

The practitioner's landing: Terry and colleagues — the Bayou gentlemen, Chapter 6's receipts-keepers — published in 1994 four promises scoped not to the whole world but to one *session*: one client's continuing conversation.

Definition 7.8 (the session guarantees; Terry et al. 1994)**.** Scope: one session's operations, against the whole history. *Read your writes*: every read sees all session-preceding writes of its own session (returns their value or a later one in the object's version order). *Monotonic reads*: the set of writes visible to a session's reads grows monotonically along the session. *Monotonic writes*: the session's writes take effect everywhere in session order. *Writes follow reads*: any write of the session is ordered, everywhere, after the writes its preceding reads saw. All four together, imposed on every session and closed under transitivity, yield causal consistency; each alone is purchasable with session-local tokens (Box 15).

Four bruises, four names.

- **read your writes:** Bertie posts the notice, refreshes, and it is gone — routed to a replica his own write had not reached; Chapter 6's stale-follower farce, outlawed by name.
- **monotonic reads:** the aunt sees the new price, hands her device to the uncle, and it reverts before his eyes — the session hopped replicas backwards; the flicker, outlawed per session.
- **monotonic writes:** Bertie posts a notice, then a correction, and some replica applies the correction first — amendments to documents that do not yet exist.
- **writes follow reads:** Bertie reads Tuppy's question and posts the answer; at a distant replica the answer stands above a question not yet arrived — testimony preceding the evidence.

Bayou was built for machines that expected to be *disconnected* — the roaming laptops of the early nineties — hence promises scoped to the session, payable in tokens rather than rounds. The *session receipts* are those tokens: the session carries the stamp of its last write and of the freshest state it has read; a replica answering it first checks the receipt — am I at least this new? — and otherwise waits or redirects. Four guarantees, four disciplines of receipt-keeping: a token in a cookie, not a quorum in a datacentre; the highest tier purchasable for honest bookkeeping.

Protocol — Box 15. The session receipts (checked against Terry et al. 1994, §§4–5)

Each session keeps two receipts: a read-set R and a write-set W of write identifiers. Each server s knows the set $\mathrm{DB}(s)$ of writes it has applied.

1: **on read at server s:**
2: *read your writes:* require $W \subseteq \mathrm{DB}(s)$, else wait or redirect
3: *monotonic reads:* require $R \subseteq \mathrm{DB}(s)$, else wait or redirect
4: serve the read; add to R the identifiers of the writes the answer depends on
5: **on write at server s:**

6: *monotonic writes:* require $W \subseteq \mathrm{DB}(s)$
7: *writes follow reads:* require $R \subseteq \mathrm{DB}(s)$
8: apply; add the new write's identifier to W

Each guarantee is exactly one guard; enforce the guards you have sold. The receipts are session-local — a token in a cookie, not a quorum in a datacentre. Practical rations summarize R and W by version vectors rather than identifier sets (Terry et al., §6).

At the bottom lies the floodplain — eventual consistency, the beat owed the estate agents since the founding plan. The promise: if writing ceases, replicas eventually agree. Attend to the grammar.

Definition 7.9 (eventual consistency)**.** An infinite execution is *eventually consistent* iff every update is eventually applied at every correct replica and, if from some point onward no further updates are issued, all correct replicas' states (equivalently, all subsequent reads) eventually agree. This is a *liveness* property: a promise about the limit, not about any prefix.

Every other model in this chapter is a *safety* property — a constraint on every prefix: this you may never show a client. Eventual consistency constrains nothing at any finite moment; "eventually" is an estate agent's word, a promise about a future no one will be present to audit. Proved as a proposition: any finite history extends to a converging execution.

Proposition 7.4 (the costume)**.** *Every finite history over registers is a prefix of some eventually consistent execution. Eventual consistency, taken alone, therefore forbids no finite transcript whatsoever — it is a liveness property wearing a safety costume.*

Proof of Proposition 7.4. Given any finite history H over registers, extend it as follows: issue no further updates; deliver every write of H to every replica (the model's channels eventually deliver); let each replica adopt, once all deliveries have landed, the same deterministic choice among the delivered writes (say, the one latest in an agreed tie-breaking order on write identifiers — computable locally from the same delivered set). All subsequent reads at all replicas return that value; the execution converges. Nothing in this construction constrained H itself: every read in H may have returned anything at all. Hence the floodplain's paddock contains every finite transcript, which is the formal content of "liveness in a safety costume." □

The flood insurance. Not a dismissal: convergence is real, Chapter 6's nets deliver it, and for carts it is the honest choice deliberately made. The objection is to the costume, and the follow-up (drilled in Problem 7.5) is always: *eventual, plus what?* Plus session guarantees? Plus causality? Plus bounded staleness with a number? The trade has learned to measure the flood: Bailis and colleagues' probabilistic staleness bounds, computed from measured replication delays, answer *how eventual* — after so many milliseconds, with such-and-such confidence, a read returns the latest write; on a healthy cluster startlingly good, the contract's weakness invisible until the day it is not. And Kingsbury's consistency atlas — the models and their transactional cousins as a lattice, edges marking strict strength, annotations marking partition survival — is assigned in the reading as *equipment*: the standing one-page answer to "is this model stronger than that one."

The field protocol. Three questions, in order, for using the hierarchy in anger:

1. **Who compares notes?** Side channels — telephones, dashboards, a common aunt — make real time observable from outside: the summit or its strict transactional cousin goes on the requisition; solitary sessions may need only the session guarantees.

2. **What must survive partition?** If both sides of a severed cable must answer, nothing above the causal landing is available (Chapter 3); choose among causal, sessions, and the floodplain with insurance.
3. **What arbitrates races?** One agreed winner — locks, uniqueness, auctions — is beyond every landing here at any price: arbitration is agreement, and the requisition goes to the parliament of Chapters 4 and 5.

Most designs decompose room by room — a linearizable sliver where notes are compared, causal or session middleware for the conversation, floodplain for the analytics; the guide picks the cheapest landing for each room.

7.4 Part Four — The Star Exhibit

The rite of passage: the gap between the summit and its immediate junior is the subtlest in the guide, probed in interviews, blurred by vendors; the standing ruling is that the separating history be held whole. The episode stages it at full length; the skeleton, beside its exhibit:

- **the estate:** one register — the shop's advertised umbrella price — at Bingo and Gussie; sequential consistency via an agreed log, reads served locally; the old price, five shillings.
- **the write:** Bertie writes ten shillings; it lands in the log, applies at Bingo, and *responds* — confirmed, finished.
- **the side channel:** Bertie telephones his aunt — information flowing outside the transcript, honoured only by real-time-respecting models. "The price is ten shillings now."
- **the read:** the aunt rings off and reads — invoked after a call that began after the write's response, the real-time order total. Served at Gussie, where the log entry lawfully lags: **five shillings**.

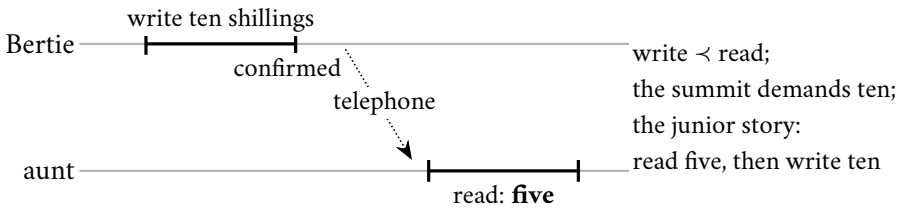


Exhibit 7.1 (the star: sequentially consistent, not linearizable). Bertie's write completes; the telephone (a channel the contract cannot see) precedes the aunt's read; the read returns the past. Not linearizable: write ≺ read forces the read to see ten. Sequentially consistent: the story "read five, then write ten" keeps both one-operation program orders and is legal. The gap between the tiers is exactly as wide as the side channels among your clients.

The two verdicts. Against the summit: the write responded before the read began — the operations do not overlap; any linearization orders the write first; the read must see ten; it saw five. *Not linearizable.* Against the junior model: the story *the aunt reads five, then Bertie writes ten* respects each client's one-operation program order and is sequentially lawful. *Sequentially consistent.* The system breached nothing it promised — yet the family has direct experience of the breach, because the telephone call *happened*. That is the whole gap: linearizability respects real time and therefore every side channel reality contains; sequential consistency respects only the orders each client can testify to from inside. Every support ticket of the form "I told her it was updated, and she looked, and it wasn't" was this exhibit, performed by amateurs.

Three variations wear the edge smooth.

- **Disconnect the telephone:** the transcript is identical, but nobody can testify to a breach — sequential consistency is the summit *for clients who never compare notes*; the gap is exactly as wide as the side channels among your clients.
- **Purchase the freshness check:** Gussie's replica confirms against the log — one round trip — before answering; the read returns ten; the summit is restored. The invoice is itemized: that round trip, on every read, is what linearizability *costs* and what sequential consistency *saves*.
- **Let the read overlap the write:** even the summit permits her five shillings, and no telephone can manufacture a breach — the call began before the write was confirmed and testifies to nothing. Overlap is the definition's mercy, seen across a counter.

The same history is set formally, both verdicts to be proved, in Problems 7.1(d) and 7.7(c).

With the exhibit in hand, the whole descent can be ruled formally: each containment is an implication between explanation obligations; each strictness is a two-client exhibit.

Theorem 7.5 (the hierarchy)**.** *As sets of complete histories,*

$$\text{Lin} \subsetneq \text{SC} \subsetneq \text{Causal},$$

and every model in the chain admits every history that eventual consistency (Prop. 7.4's floodplain) admits — the paddocks nest. Strictness witnesses: Exhibit 7.1 (in SC*, not* Lin*); Exhibit 7.2 (in* Causal*, not* SC*).*

Proof of Theorem 7.5. Lin $\subseteq$ SC. A linearization extends $\prec$; program order is $\prec$ within one client (well-formedness: a client's operations never overlap); so the same S witnesses sequential consistency. *Strict:* Exhibit 7.1 is sequentially consistent (the story "aunt reads five, Bertie writes ten" keeps both one-operation program orders and is legal) but not linearizable (write $\prec$ read forces the read to see ten; it saw five) — verdicts proved in the exhibit's caption and Problem 7.1(d).

SC $\subseteq$ Causal. Let S witness sequential consistency of H. In S, every read returns the latest preceding write, so each reads-from edge is ordered by S; program order is ordered by S by definition; hence $\rightarrow \subseteq <_S$ by transitivity. For each client c, let S_c be S restricted to c's operations and all writes: a restriction of a legal register story that keeps every read together with all writes remains legal (each read's latest preceding write is a write, hence retained), and its order still extends $\rightarrow$. The same S serves every client: sequential consistency is causal consistency with unanimity. *Strict:* Exhibit 7.2: the two writes are concurrent (no road connects them), so each reader's view may order them privately — both views respect $\rightarrow$ and are legal, so the history is causal; but any single story must order the two writes once, and then one reader's second read returns a value overwritten in the story before its first read's value — formally, the aunt's pair of reads forces "one then two," the uncle's forces "two then one," and no total order extends both requirements with register legality (worked in full in Problem 7.7(b)).

The floodplain containment is Prop. 7.4: no finite history is excluded, so every paddock lies inside it. □

The second strictness witness — causal but not sequential — follows, in the season's trace convention.

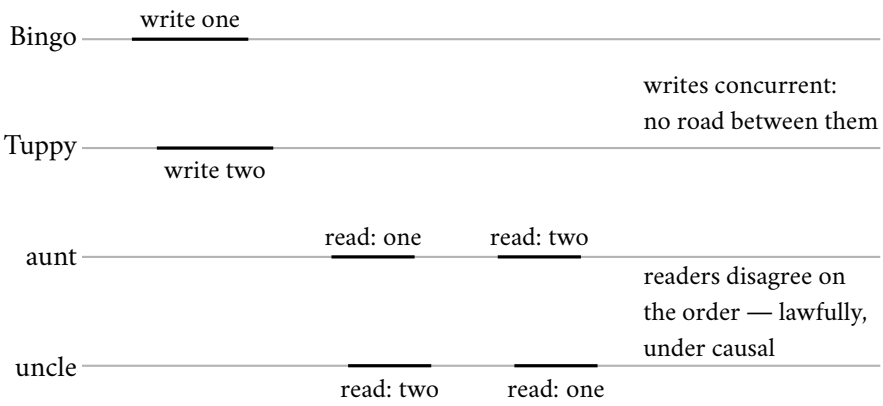


Exhibit 7.2 (causal, not sequential). Two concurrent writes to one register; the aunt reads one then two, the uncle two then one. Each view respects happens-before (the writes are unrelated by it), so the history is causally consistent. No single story serves both readers: the aunt's pair forces one-before-two, the uncle's forces the reverse (Problem 7.7). Causality orders the roads; it cannot rank the merely-elsewhere.

One correction of folklore before the hierarchy is filed, recorded formally because the corrected problem depends on it.

Remark 7.10 (real-time causal, and a folklore correction). Enrich happens-before with external order: $\rightarrow_{\mathrm{rt}} = (\text{program order} \cup \text{reads-from} \cup \prec)^{+}$; *real-time causal* consistency is Def. 7.7 with $\rightarrow_{\mathrm{rt}}$ in place of $\rightarrow$. Folklore asserts that causal and sequential consistency are incomparable; under the standard definitions this is half true — sequential *does* imply causal (Thm. 7.5) — and the genuine incomparability holds between sequential and *real-time* causal. Problem 7.7 settles all four directions.

7.5 Part Five — The Other Axis: Transactions and Isolation

Everything so far concerned single operations on single objects. Half the industry's vocabulary comes from the database hall, where the unit is the *transaction*: several reads and writes, bracketed, demanding to appear as a unit — and the two halls' vocabularies are confused daily, with a body count; presently, two doctors. The hall's summit is *serializability*: transactions appear to execute one at a time, in *some* order — any order; present, the one-at-a-time story; absent, any clause about real time. Serializability is the transactional cousin of sequential consistency, not of linearizability — a serializable database may lawfully run your transaction as though it ran yesterday — and neither implies the other (Problem 7.9 builds each without the other). Demand both and you have *strict serializability*, the strongest promise in the combined guide; in Chapter 8 a company purchases it across continents with atomic clocks, and the invoice makes sense only if the name is exact.

Definition 7.11 (serializability; strict serializability)**.** A history of committed transactions is *serializable* iff some total order of the transactions, executed one at a time from the initial state, gives every read the value it actually returned (and hence the same final state). It is *strictly serializable* iff the order can be chosen to extend transaction real-time precedence ($T \prec T'$ iff T's commit response precedes T''s first invocation). Strict serializability is exactly linearizability with whole transactions as the operations; serializability is its clock-free weakening, the transactional cousin of Def. 7.6.

The missing clock. A read-only transaction served from last night's snapshot is *perfectly serializable* — order it before every transaction that committed today; strictly serializable it is not, since today's deposits

responded before the read was invoked. Vendors have shipped exactly this, and the defence — backup-restore, made formal in Problem 7.9 — is technically sound, which is the lesson: the gold standard, as the industry recites it, contains no clock. When a brochure says serializable, ask the auditor's question: serializable *as of when?*

The negotiated tier. The hall's daily reality: every transaction reads from a consistent *photograph* of the database as of its start — the Chandy–Lamport spirit, industrialized into *multiversion concurrency control*, MVCC on every vendor's feature list — and writes succeed unless two transactions wrote the *same* item concurrently, in which case one aborts: first committer wins. Reads never block, writes conflict only on overlap, and the promise sounds like serializability if recited quickly.

Definition 7.12 (snapshot isolation)**.** The store is multiversion: each committed transaction T stamps its written versions with its commit point $c(T)$. A transaction reads, for each item, the newest version committed before its start point $s(T)$ (its *photograph*), plus its own writes. Commit rule (*first committer wins*): T may commit iff no transaction T' with $s(T) < c(T') < c(T)$ has write set intersecting T's. Reads are never blocked and never validated.

Notice the asymmetry the doctors are about to exploit: the *writes* are checked for collision; the *reads* — the evidence on which the transaction acted — are checked against nothing at all.

The two doctors. The hospital's rule — the application's invariant, the ACID-C sense of consistent: at least one doctor on call at all times. The episode stages the weary hour; the skeleton:

- **the photographs:** Tuppy and Gussie each open a transaction to book off; each photograph shows *two on call*; each guard passes; each writes himself off — his own row.
- **the commit rule:** write sets $\{t\}$ and $\{g\}$ — different items, no collision; first committer wins never invoked; *both commit*.
- **the corpse:** nobody on call. Each transaction, run alone, was impeccable; the invariant died *between* them — the rule lived across both rows while the conflict detector watched each row alone.

The anomaly is *write skew*; its formal residence is the 1995 critique that showed the industry's isolation levels were folk taxonomy.

Proposition 7.6 (write skew; Berenson et al. 1995)**.** *Snapshot isolation admits non-serializable executions. In particular, for the doctors' schema (two rows; guard: book off only if the photograph shows at least two on call), snapshot isolation admits a history in which both guarded programs commit and the roster empties, while* no *serial execution of the same two programs can empty it. Serializability, by contrast, preserves every invariant that each transaction program preserves in isolation.*

Proof of Proposition 7.6. The admitted history. Items: rows t (Tuppy) and g (Gussie), both initially `on`. Programs: P_T reads both rows and, iff at least two read `on`, writes $t :=$ `off`; P_G symmetrically writes $g :=$ `off`. History: $s(T_1) = s(T_2) = \sigma$, later $c(T_1) < c(T_2)$, no other transactions. Under Def. 7.12: both photographs at σ show (`on`, `on`); both guards pass; write sets are $\{t\}$ and $\{g\}$ — disjoint — so first committer wins is vacuous and both commit. Final state (`off`, `off`).

No serial equivalent. In either serial order of the two guarded programs, the second reads the first's committed write: its photograph shows exactly one `on`; its guard fails; it writes nothing. Both serial executions end with exactly one row `off`. The admitted history's reads (both seeing two `on`) and final state (none on call) match neither serial execution, so it is not serializable.

Serializability preserves guarded invariants. If each transaction program, run alone from a state satisfying invariant I, ends in a state satisfying I, then induction along any serial order preserves I; a serializable history's reads and final state agree with some serial execution's, hence its final state satisfies I. The doctors' guard preserves "at least one on call"; snapshot isolation nevertheless emptied the roster — which locates the anomaly precisely in the gap between Def. 7.12 and Def. 7.11: reads are never validated, and the invariant lived across both rows while the commit rule watched each row alone. □

The crime scene, drawn once for the whole guide.

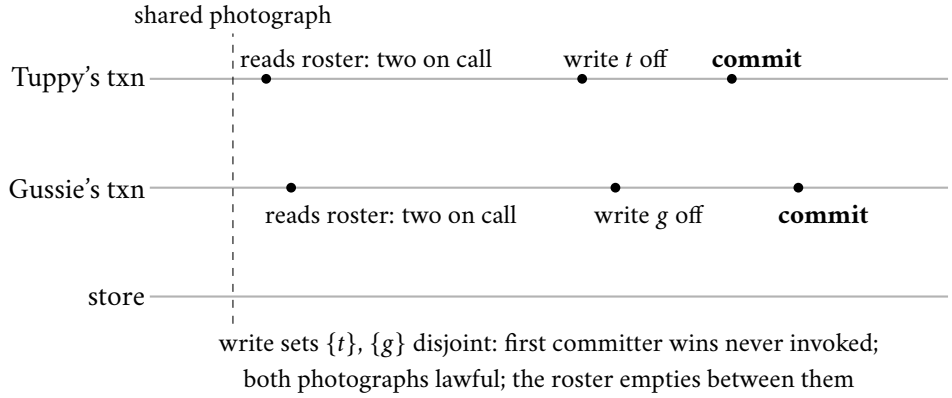


Exhibit 7.3 (write skew: the two doctors). Both transactions read the same photograph, satisfy the guard, and write different rows; snapshot isolation commits both (Prop. 7.6). No serial order of the same guarded programs empties the roster. The invariant lived across both rows; the commit rule watched each row alone.

The shape, and the remedies. Abstracted so it can be found without the doctors: *read a neighbourhood, decide on the evidence, write a corner the neighbourhood-rule watches but the detector does not.* Two registrars each check a username unclaimed and insert fresh rows; two withdrawals each verify a linked pair's combined balance and debit different accounts; two officers each confirm the vault keeps a second keyholder and sign themselves out. Any rule that lives across rows is a doctors' rule (Problem 7.4 is the hunt; Problem 7.8 works the vault in full). The remedies, priced:

- **serialize truly:** the serializable refinement of snapshot isolation keeps the photographs but watches the *pattern* of reads and writes among concurrent transactions (who read what another then wrote), aborting whenever the pattern could close into write skew's cycle. One doctor commits; the other aborts, retries, and stays. The premium is false alarms; the invariant survives with no application change.
- **materialize the conflict:** make the contending transactions touch a common row — an on-call token — so the detector can see the fight; a fine trick, provided every future transaction remembers the etiquette.
- **lock the neighbourhood** explicitly, and surrender the concurrency.

What is *not* a remedy is hoping. The axes stand assembled: the replication hall's models govern single operations against real time; the database hall's levels govern transactions against each other; strict serializability is the corner where both summits meet; and the ACID-C — the doctors' rule — is *your* obligation, which every level below the summit protects only partially, in ways you are now equipped to name.

7.6 Part Six — The Wedding of the Kettles

A change of key: from promises about *ordering* to a discipline that makes ordering *irrelevant*. Chapter 6's honest mode kept rivals as siblings and handed the merge to the application; Chapter 1 left a glinting observation in a drawer: the replicated *counter* forgave the Post's reordering entirely, *because addition commutes*. The drawer opens — the first instalment on the season's oldest debt. The question: for which objects can replicas accept updates independently — no coordination, no leader — and still be *guaranteed* to converge? The mature answer is 2011, Shapiro, Preguiça, Baquero, and Zawirski: *conflict-free replicated data types*, with a theorem attached. Make the states a join-semilattice — the *join* the least upper bound $\sqcup$ — make every update move a replica upward, and exchange states arbitrarily, merging on receipt: convergence is not hoped but *proven*. The guarantee is *strong eventual consistency*, and mark the upgrade from the floodplain: reconciliation is a pure function with algebraic manners — same news, same state, no siblings, no application on call at midnight.

Definition 7.13 (join-semilattice; state-based CRDT)**.** $(S, \sqcup, \bot)$ is a *join-semilattice* with bottom: $\sqcup$ is commutative, associative, idempotent, with $\bot \sqcup s = s$; the induced order is $s \leq t$ iff $s \sqcup t = t$, and $s \sqcup t$ is the least upper bound. A *state-based CRDT with join-decomposable updates* assigns to every update u a *contribution* $c(u) \in S$, fixed at the originating replica when u executes: the origin replaces its state s by $s \sqcup c(u)$; anti-entropy ships states and the receiver replaces s' by $s' \sqcup s$. (All of this chapter's objects are of this form; general inflation-based objects are treated in the reading.) A replica *has absorbed* u if u's contribution has reached it through any chain of updates and merges.

The order, and why *semi*. The order induced by $\sqcup$ is *partial*, not total: the grow-only counter's rows $(4, 0, 0)$ and $(0, 3, 0)$ satisfy neither $\leq$, incomparable states being the algebraic image of Chapter 2's concurrent events. That partiality is precisely what makes the least-upper-bound requirement substantive — in a total order $s \sqcup t$ is merely the larger of the two, whereas in a partial one an upper bound need not exist at all. A partially ordered set in which every pair has a least upper bound is a *join-semilattice*; one in which every pair also has a greatest lower bound — a *meet* $\sqcap$, the largest element below both — is a *lattice*. Replication buys the upward half only, whence the *semi*: a merge accumulates news, and no object in this chapter ever asks what two replicas hold *in common* in order to discard the rest. (The season's one genuine lattice is Chapter 2's: Problem 2.9 has the consistent cuts closed under both union and intersection, $\cup$ the join and $\cap$ the meet.) Note also the direction of this definition. The three laws are taken as axioms and the order induced from them; one may equally begin with the order, in which case commutativity, idempotence and associativity are *consequences* — "the least upper bound of a set" is a function of the set, and a set records neither the order in which its members were named, nor their multiplicity, nor any bracketing of them.

The theorem's skeleton: every replica's state is the join of the contributions it has absorbed; joins forget order and multiplicity — commutative, associative, idempotent; same news, same state.

Theorem 7.7 (strong eventual consistency; Shapiro–Preguiça–Baquero–Zawirski)**.** *For a state-based CRDT with join-decomposable updates: at every point of every execution, every replica's state equals the join of the contributions of exactly the updates it has absorbed,*

$$s_i \;=\; \bot \sqcup \bigsqcup \{\, c(u) : u \in D_i \,\},$$

where D_i is replica i's absorbed set. Consequently two replicas that have absorbed the same updates — in any order, along any routes, with any duplication — hold identical states; and under the model box's eventual delivery, all correct replicas converge once updates cease. Convergence is a pure function of the news, not of its itinerary.

Proof of Theorem 7.7. By induction over the events of an execution, we prove the displayed identity at every replica, with D_i the absorbed set defined inductively alongside: initially $s_i = \bot$, $D_i = \emptyset$, and the identity holds vacuously.

Local update u at replica i: the new state is $s_i \sqcup c(u)$; the new absorbed set is $D_i \cup \{u\}$. By the induction hypothesis and associativity–commutativity, the new state is the join of $\{c(v) : v \in D_i\} \cup \{c(u)\}$; if u was somehow already present (it is not, being fresh, but duplication costs nothing), idempotence absorbs it.

Merge of an incoming state s_j (sent when j's absorbed set was D_j) into replica i: the new state is $s_i \sqcup s_j$; the new absorbed set is $D_i \cup D_j$. By both induction hypotheses,

$$s_i \sqcup s_j = \Big(\bigsqcup_{v \in D_i} c(v)\Big) \sqcup \Big(\bigsqcup_{v \in D_j} c(v)\Big) = \bigsqcup_{v \in D_i \cup D_j} c(v),$$

the last step by commutativity, associativity, and — for the updates in $D_i \cap D_j$, delivered twice — idempotence. This one line is the theorem's engine: *the join of a set of contributions depends on the set, not on the order or multiplicity of joining.*

Equality of states for equal absorbed sets is now immediate, and it is a safety property holding at every instant. For convergence: after updates cease, the model box's anti-entropy delivers every update's contribution (transitively) to every correct replica, so all absorbed sets become equal to the full update set, and all states equal its join. Strong eventual consistency is the conjunction of the two. □

The inversion. Every other model fought the Post's disorder with coordination — holding mail, blocking reads, electing sequencers. The semilattice surrenders the battle: it re-designs the *object* until disorder is powerless, until merge forgets nothing and order was never the point. Chapter 1's counter knew it in embryo; the cart practised it commercially; the theorem names the species.

The grow-only counter. One tally per replica; merge takes the entrywise maximum; the value is the sum. Maximum, not addition, because merge must be idempotent — the Post duplicates, and a maximum absorbs repeats where a sum double-counts them: the vector clock's merge, repurposed from tracking causality to *being the data*. The tallies, run once — a machine, not a metaphor:

- **increments:** Bingo records four of his own — his row reads four, zero, zero; Tuppy records three; Gussie, offline in the storm, one.
- **merge:** Bingo merges Tuppy's row: four, three, zero — value seven.
- **duplicate:** the Post redelivers Tuppy's row — maxima of equals; nothing moves.
- **rejoin:** Gussie merges: four, three, one — eight; Bingo merges back: eight. Any order, any repeats, any routes: eight.

The discipline that makes it lawful: each replica increments *only its own* tally — the row is a ledger of testimonies, sound because each witness's own count only grows; let Bingo update Tuppy's column once and the algebra collapses into Chapter 6's lost-update crime scene. The grow-only counter cannot decrement; the *positive-negative counter* carries two of them and makes removal itself a growth — the down-tally only ever grows.

Protocol — Box 16. Grow-only and positive-negative counters (checked against the 2011 report)

1: **G-counter** at replica i of n: state P, a vector of n tallies, initially all zero
2: **increment:** $P[i] \leftarrow P[i] + 1$ — own entry only
3: **value:** sum of all entries of P
4: **merge**(P'): entrywise maximum of P and P'
5: **PN-counter:** two G-counters (P, N); increment raises $P[i]$; decrement raises $N[i]$
6: **value:** sum of P minus sum of N; **merge:** componentwise

Maximum, not addition: merge must be idempotent because the Post duplicates. Each replica writes only its own entry — the vector is a ledger of testimonies, and the maximum is lawful because each witness's own count only grows. Contribution form for Thm. 7.7: an increment's $c(u)$ is the vector that is zero save the incremented entry at its new value.

Neither counter yet holds *things*; the wedding gown is the *observed-remove set* — adds and removes of elements, adds winning against concurrent removes: the cart's own law. Every add attaches a fresh unique tag; a remove does not subtract the element — it records as removed *precisely the tags it has observed*. Tags and testimony, exactly: removal is testimony about what was seen, and one may only remove what one has observed.

Definition 7.14 (the observed-remove set)**.** State: a pair (A, R) with A a set of (element, tag) pairs and R a set of tags, both grow-only; $\bot = (\emptyset, \emptyset)$; join is componentwise union (a product of two grow-only semilattices). Operations at a replica with state (A, R): $\mathsf{add}(e)$ draws a globally fresh tag t and contributes $(\{(e,t)\}, \emptyset)$; $\mathsf{remove}(e)$ contributes $(\emptyset, \{\, t : (e,t) \in A \,\})$ — testimony against precisely the observed tags; $\mathsf{contains}(e)$ holds iff some $(e,t) \in A$ has $t \notin R$. Box 17 gives the protocol form.

Protocol — Box 17. The observed-remove set (checked against the 2011 report)

1: **state:** (A, R): added (element, tag) pairs; removed tags. Both grow-only
2: **add**(e): draw fresh tag t (replica id, counter); $A \leftarrow A \cup \{(e,t)\}$
3: **remove**(e): $R \leftarrow R \cup \{\, t : (e,t) \in A \,\}$ — testimony: observed tags only
4: **contains**(e): some $(e,t) \in A$ has $t \notin R$
5: **merge**(A', R'): $A \leftarrow A \cup A'$; $R \leftarrow R \cup R'$
6: **re-add after remove:** a fresh add draws a fresh tag, never covered by old testimony

Add-wins by jurisprudence (Prop. 7.8): a remove defeats exactly the adds it observed. The metadata bill is real: tags and testimony accrue; pruning them requires knowing what every replica has absorbed — a global fact, hence coordination (the season's irony; the settlement at this part's close).

The law in operation is Exhibit 7.4's trace: a remove testifying against tag one, the only kettle it has seen; a concurrent add wearing tag three; merged in any order, tag one covered, tag three alive. The add wins not by timestamp but because the remove could not testify against news it had never received — causality, baked into the algebra as jurisprudence.

Proposition 7.8 (add-wins jurisprudence)**.** *In the observed-remove set, if an* $\mathsf{add}(e)$ *is not absorbed by the replica executing a* $\mathsf{remove}(e)$ *at the time of the remove (the two are concurrent), then after any replica absorbs both, e is present. A remove defeats exactly the adds it observed: one may only remove what one has seen.*

Proof of Proposition 7.8. Let the remove execute at replica j with state (A_j, R_j), and let the concurrent $\mathsf{add}(e)$ have drawn tag t^*. Concurrency means j had not absorbed the add, so $(e, t^*) \notin A_j$, so $t^* \notin$ the remove's

testimony — and testimony sets only ever accumulate tags fixed at their origin. After any replica absorbs both updates, its state contains (e, t^*) in A and $t^* \notin R$ (no other operation testifies against t^*: tags are globally fresh, and removes testify only against observed tags). By Def. 7.14, $\texttt{contains}(e)$ holds. The freshly added kettle survives because the remove could not testify against news it had never received. □

The two kettles are wed: every replica, told the same history in any order, reaches the same cart.

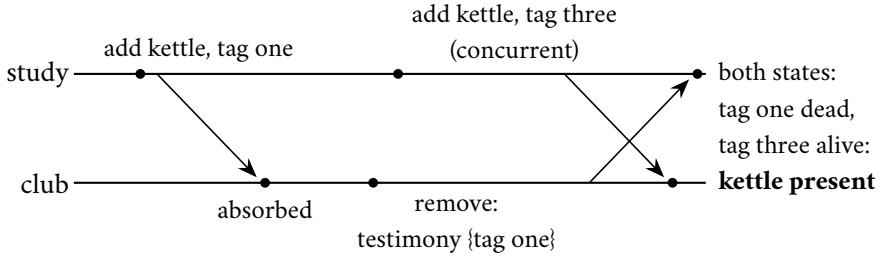


Exhibit 7.4 (the wedding). The club's remove testifies against tag one — the only kettle it has seen; the study's concurrent add wears tag three. Merged in either order, every replica reaches the same cart: tag one covered, tag three alive (Prop. 7.8). Not by timestamp, not by luck: by a law under which removal is testimony about what was observed.

One relation attends the wedding under a cloud, and honesty requires the introduction.

Remark 7.15 (the cautionary relation). The last-writer-wins register — state (value, timestamp), join keeps the larger stamp — satisfies every hypothesis of Thm. 7.7. The theorem guarantees convergence, never justice: the law it converges under is arbitration by the Post's perjured clock, and the discarded rival is discarded silently and forever. Choosing the object is choosing the verdicts; the observed-remove set is celebrated because its law encodes causality, not because it converges — the perjurer converges too.

Two invoices complete the settlement.

- **what travels:** the school above ships whole *states* — rows of tallies, bags of tags — and asks nothing of the Post beyond eventual gossip; Chapter 6's anti-entropy nets suffice. The dual school ships *operations* — "increment by one" — far lighter, but operations are not idempotent, so delivery must be exactly-once and for some objects causally ordered: Chapter 2's machinery re-hired as a delivery service. The standing compromise ships recently changed fragments of state — both advantages, mostly.
- **the metadata bill:** tags and testimony accrue; a busy year's set may weigh many times its visible contents. Pruning requires knowing what *every* replica has absorbed — a global fact, hence coordination. The season's irony at its purest: the coordination-free object may be *used* forever without agreement, but *slimmed* only with it.

And the costs of the law itself: the law must fit the business — a banking balance is not a semilattice, however devoutly one wishes it (Problem 7.12); and nothing here provides an agreed *order* — a CRDT cannot run an auction. A lawful, coordination-free province with a well-marked border, patrolled, as ever, by the race test.

7.7 Part Seven — The Horizon: CALM

The view from altitude. The season keeps meeting the same fork: some tasks yielded without coordination — the photograph, the register, the cart — while others — the bit, the lock, the auction — demanded a

parliament every time. Is there a *law* sorting the forks? There is a candidate, perhaps the deepest single sentence the field has produced this century. First the logician's sense of the load-bearing word.

Definition 7.16 (monotone program)**.** A program computing an output set from input sets is *monotone* iff larger inputs never shrink the output: $I \subseteq I'$ implies $P(I) \subseteq P(I')$ — conclusions, once emitted, are never retracted by further news. Union, selection, join, transitive closure: monotone. Negation, universal quantification over a live set, "the final state," "the winner": non-monotone.

Theorem 7.9 (CALM; Hellerstein–Alvaro; proved by Ameloot–Neven–Van den Bussche)**.** *A distributed program has a coordination-free, eventually consistent implementation if and only if it computes a monotone function of its inputs.* (Stated with respect and a pointer: the proof lives in the relational-transducer semantics of the reading, honestly beyond this course's syllabus.)

Find the negations. Non-monotonicity is where coordination becomes a mathematical obligation, not an engineering preference: anything asserting a completed totality over inputs still possibly in flight — "the auction is closed," "this is the *final* state" — must somewhere *seal the set*, and sealing a set across machines is agreement, Chapter 3's terrain with all its invoices. The practical edition changes design reviews: every "final," "exactly," "never," "the winner" in a specification is a place coordination will be spent; everything phrased as accumulation can flow free. Read your designs for their quantifiers, and the invoices become legible before the architecture is drawn.

The classification, on the season's case files.

- **a watchlist assembled by union** from many stations: monotone — more reports, more entries, nothing retracted; free to flow.
- **"this parcel's ancestry has arrived":** monotone — ancestry only accumulates; once yes, yes forever; exactly why causal consistency needed no parliament.
- **"this replica's copy is the latest":** non-monotone — a totality claim over writes still in flight; there stands the freshness-check round trip the star exhibit priced.
- **"the winner is Bingo, at seven shillings":** non-monotone by construction — one further bid retracts the conclusion; Chapter 3's bit in an auctioneer's coat.

The observed-remove set's jurisprudential trick: removal *sounds* like negation — and naive removal is — but accumulating testimony against observed tags re-drafts the negation as a growth; the theorem's licence extends exactly as far as such re-draftings can be found (Problem 7.11 sorts a page of fragments). Sometimes it cannot — finality is finality — and then the invoice is a theorem's invoice, paid with dignity.

7.8 The Map and the Specimens

Part Five closed with the axes assembled in prose, and the reading assigns Kingsbury's atlas as the standing wall chart. Between the two belongs a house edition: the guide's own map, restricted to exactly what the season taught — every box a definition of this chapter, every gap a witness this chapter proved — and, beneath it, the specimens filed as a naturalist files them. Neither adds a theorem; both are for the shelf above the desk.

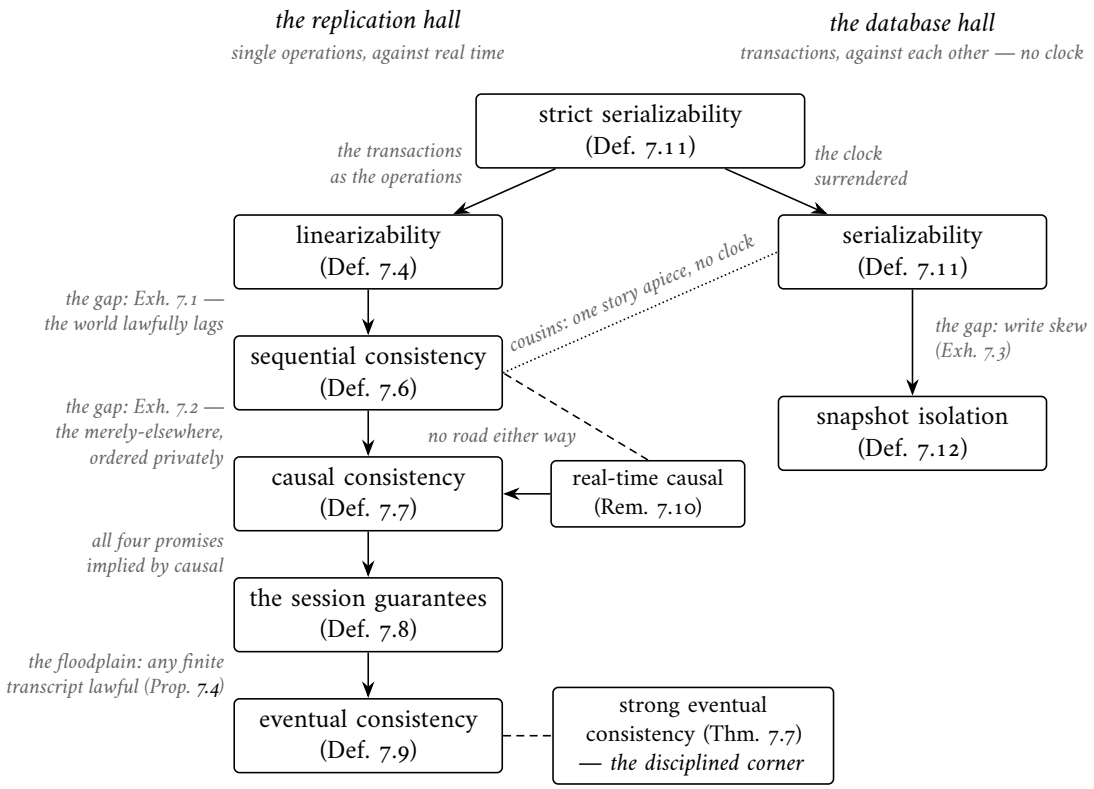


Exhibit 7.6 (the two axes, assembled). The whole guide on one plate. The left ladder is the replication hall's hierarchy (Thm. 7.5): each arrow points from the stronger paddock to the weaker, and the margin names the witness history living in each gap. The right ladder is the database hall's; below snapshot isolation lie the folk levels of the 1995 critique, unmapped here. Strict serializability is the corner where the summits meet (Def. 7.11); sequential consistency and serializability are the cousins — one story apiece, neither consulting a clock — and neither linearizability nor serializability implies the other (Problem 7.9). Real-time causal stands off the ladder: stronger than causal, incomparable with sequential (Rem. 7.10). Strong eventual consistency is the floodplain's disciplined corner (Thm. 7.7): convergence by algebra rather than by adjudication.

The specimens: the replication hall (single operations, against real time; the hierarchy of Thm. 7.5).

specimen	the promise	lawfully admits	the price
linearizability (Def. 7.4)	one story, real time respected: every operation takes effect at an instant between invocation and response	either order for genuinely overlapping operations — nothing else; the past, once seen, stays seen	coordination on the read path, evidence or a throne; the minority side of a partition declines service
sequential consistency (Def. 7.6)	one agreed story keeping every client's program order; no clock consulted	the whole world lawfully lagging — a fresh write invisible to a read invoked after it (Exh. 7.1)	one story still wants agreement machinery, and no freshness clause holds it to the present
causal consistency (Def. 7.7)	whatever could have influenced an operation appears before it: program order and reads-from, transitively	the merely-elsewhere ordered privately — concurrent writes seen in different orders by different readers (Exh. 7.2)	the causal paperwork, carried and pruned; in exchange, the strongest model still available under partition (the causal landing)
the session guarantees (Def. 7.8)	four promises to one session: read your writes, monotonic reads, monotonic writes, writes follow reads	anything whatsoever between different sessions	the session receipts — a token in a cookie, not a quorum in a datacentre (Box 15)
eventual consistency (Def. 7.9)	if writing ceases, replicas eventually agree — a promise about the limit	any finite transcript whatsoever (Prop. 7.4: liveness in a safety costume)	none up front; the bill arrives as the standing follow-up, *eventual, plus what?*
strong eventual consistency (Thm. 7.7)	replicas that have received the same updates are in the same state — convergence by algebra, not adjudication	divergent views while updates are in flight; no ordering promises at all	the data recast as a join-semilattice (Part Six's wedding); tombstones, and their lifetimes

The specimens: the database hall (transactions, against each other).

specimen	the promise	lawfully admits	the price
strict serializability (Def. 7.11)	transactions one at a time, in an order extending real time — linearizability with whole transactions as the operations	either order for genuinely overlapping transactions — nothing else	the strongest promise in the combined guide; Chapter 8 prices it across continents
serializability (Def. 7.11)	transactions one at a time, in *some* order — any order; no clause about real time	a serial order lawfully ignoring the clock — last night's snapshot, perfectly serializable; ask *serializable as of when?*	every invariant each transaction preserves alone survives (Prop. 7.6); the fee is conflict tracking and aborts
snapshot isolation (Def. 7.12)	every transaction reads a consistent photograph of its start; first committer wins among colliding writers	write skew — the two doctors (Exh. 7.3, Prop. 7.6): reads are never validated	reads never block; invariants that live across rows are yours to guard — serialize truly, or materialize the conflict

7.9 The Whiteboard

For the design review — four items, the course's densest.

1. Name the contract, not the vibe. "Consistent" is four professions; in a review, demand the government name — linearizable, sequential, causal, read-your-writes, eventual-plus-what — and for trans-

actions the isolation level by its admitted anomalies: does this level admit write skew? The Jepsen atlas on the wall settles strength disputes in ten seconds; and when the review stalls on the word itself, hand each profession its sentence from the opening.

2. Linearizability speaks of real time; serializability of transactions; strict serializability of both. Neither of the first two implies the other; the side-channel telephone is the test for the first; and the summit's fee is paid in coordination — buy it where clients compare notes, decline it where they cannot, and remember the field protocol's three questions: who compares notes; what must survive partition; what arbitrates races.

3. Snapshot isolation permits write skew, and your invariants that span rows are unguarded by it. Find the constraint that lives across items; then serialize truly, materialize the conflict, or own the anomaly in writing. The two doctors are on every schema; the drill is finding them before the roster does.

4. If your merge is a join, you may sleep: semilattice objects converge without coordination, by theorem — spend the design effort making the merge lawful rather than the delivery ordered, and audit the *law* as carefully as the lattice, for the perjurer's register is also a lattice. And where the specification negates — final, exactly, never, winner — budget a parliament, by theorem likewise. Between those two theorems runs the entire economics of this field.

The register. Debts paid: #8 (causal delivery, contracted and priced); #11 and #13 (CAP's C and the single-machine phrase, retired jointly by Def. 7.4); #16 (the lease-read fine print); #17 (the kettles wed, Thm. 7.7); and a first instalment on #4 — *commutativity forgives disorder*, first half paid by semilattice, the balance due in the finale, wearing gradients. Issued: #18 — serializability *across shards*: the two doctors were one database; make them two and isolation becomes distribution (Chapter 8). #19 — the Jepsen method industrialized: fault injection as a discipline (Chapter 11). The half-term audit, in summary: ten entries and two instalments paid across the first half; five standing open, all scheduled, crashed-versus-slow chief among them; the books balance, and the second half is financed. Drift note: the plan's Tier II problem "causal neither implies nor is implied by sequential" is false as stated under the standard definitions (sequential *does* imply causal); the desk sets the corrected problem and recovers the intended incomparability with the real-time-causal variant (Problem 7.7). Episode word count 10,428 — above the floor; the §5 corrective (per-part budgets) was applied for the first time this session.

7.10 Problems

Tier I — Calisthenics

Problem 7.1 (linearizability drills)**.** One register, initial value zero. For each history, give a linearization (as an order with points) or prove none exists. (a) Bingo writes one over $[1, 4]$; Tuppy reads over $[2, 3]$ returning zero; Gussie reads over $[5, 6]$ returning one. (b) Bingo writes one over $[1, 4]$; Tuppy reads over $[2, 6]$ returning one; Gussie reads over $[3, 5]$ returning zero. (c) Bingo writes one over $[1, 3]$; Tuppy writes two over $[2, 4]$; Gussie reads over $[5, 6]$ returning one; the aunt reads over $[7, 8]$ returning two. (d) Bertie writes ten over $[1, 2]$; the aunt reads over $[4, 5]$ returning five (the star, Exhibit 7.1).

Problem 7.2 (the queue drill)**.** A queue, initially empty. (a) Bingo enqueues an apple over $[1, 4]$; Tuppy enqueues a pear over $[2, 5]$; Gussie dequeues over $[6, 7]$ receiving the pear. Linearizable? (b) Same enqueues; Gussie dequeues twice, over $[6, 7]$ and $[8, 9]$, receiving the pear both times. (c) Same enqueues; Gussie dequeues over $[6, 7]$ receiving the pear, and the aunt dequeues over $[8, 9]$ receiving the apple. State, for each verdict, whether the fault (if any) lies against time or against the sequential specification.

Problem 7.3 (name the bruise)**.** Name the violated session guarantee, or declare all four intact. (a) Bertie posts a notice, refreshes, and the notice is absent. (b) The aunt sees the new price on her device, hands the same device (same session) to the uncle, and the price has reverted. (c) Bertie posts a notice and then a correction; a distant replica applies the correction first and the notice second, ending with the notice text. (d) Bertie reads Tuppy's question and posts an answer; at another replica the answer is visible while the question is not, to a fresh client with no history.

Problem 7.4 (the skew hunt)**.** For each schema, exhibit the write-skew shape (the cross-row invariant, the two transactions, the disjoint writes) and propose the smallest materialized conflict that closes it. (a) Usernames: a claim is lawful if no row holds the name; claims insert fresh rows. (b) A linked pair of accounts must keep a combined non-negative balance; withdrawals debit one account each. (c) The vault requires two keyholders signed in overnight; sign-outs update each officer's own row.

Problem 7.5 (name the contract)**.** Name the strongest model each description honestly supports. (a) All writes through one agreed log; reads served by any replica from its local copy, no freshness check. (b) As (a), plus: each client's reads are pinned to one replica, and a session token holds the log index of the client's last write, which the replica must have applied before answering. (c) As (a), plus: every read is relayed through the log itself. (d) A leaderless store, sloppy quorums, last-writer-wins; the brochure says "strongly eventually consistent."

Tier II — The Real Thing

Problem 7.6 (locality's negative half)**.** Two queues p and q, initially empty. Bingo's program: enqueue x on p; enqueue x on q; dequeue from q, receiving y. Tuppy's program: enqueue y on q; enqueue y on p; dequeue from p, receiving x. All six operations complete; overlaps are such that no real-time constraints bind across clients. (a) Show $H|p$ and $H|q$ are each sequentially consistent (exhibit the per-object stories). (b) Show H is not: no single story respects both program orders and both queues' specifications. (c) State in one sentence why Theorem 7.3's proof strategy has no purchase here.

Problem 7.7 (causal versus sequential, corrected)**.** (a) Prove: every sequentially consistent history is causally consistent (fill any gaps in Theorem 7.5's argument in your own hand). (b) Prove Exhibit 7.2 causally consistent and not sequentially consistent, in full detail: construct the two views; then show no single legal total order extends both readers' requirements. (c) Define real-time causal consistency (Rem. 7.10). Prove Exhibit 7.1 sequentially consistent but not real-time causal. (d) Prove Exhibit 7.2 real-time causal (arrange the intervals so the writes overlap) but not sequentially consistent. Conclude: the folklore incomparability is true of the real-time variant, and half true of the plain one.

Problem 7.8 (the vault keyholders)**.** Formalize Problem 7.4(c): two officers' rows, both `in`; guard: sign out only if the photograph shows two `in`. (a) Exhibit the snapshot-isolation history in which both sign out; verify each clause of Def. 7.12. (b) Prove no serial execution of the guarded programs reaches the empty vault. (c) Add the materialized conflict (a duty token row that every sign-out must update) and show first committer wins now aborts one officer. (d) Under the serializable refinement (abort when concurrent transactions form a read-write cycle), trace which of the two aborts and why the retry stays in.

Problem 7.9 (both axes at once)**.** (a) Exhibit a history of two transactions that is serializable but not strictly serializable (the backup-restore defence made formal): a committed deposit, then a later, non-overlapping read-only transaction returning the pre-deposit balance. Prove both halves of the claim. (b) Exhibit a history over two registers in which every single operation is linearizable per object, yet the transactional bracketing is not serializable (each of two transactions reads one register before the other writes it, crosswise). Conclude, in one sentence each way: neither axis implies the other.

Problem 7.10 (sabotage: the vanished kettle)**.** A junior engineer "optimizes" Box 17: removals record the *element value* instead of observed tags, and $\mathsf{contains}(e)$ holds iff some $(e, t) \in A$ and e is not in the removed-values set; merge unions both components. The states still form a semilattice, so the engineer cites Theorem 7.7 and ships it. Exhibit the fatal execution: a concurrent add whose kettle vanishes, permanently — including the exact states, tags, and merges, and the sentence of the specification it violates. Then state what the engineer's citation actually proves, and why that is not a defence.

Tier III — For the Ambitious (starred)

Problem 7.11 (★ the CALM classification)**.** Classify each as monotone (coordination-free by Thm. 7.9) or non-monotone (name the seal): (a) the union of watchlists from many stations; (b) "this parcel's ancestry has arrived" (causal delivery's guard); (c) "this replica holds the latest value"; (d) the auction's winning bid; (e) the observed-remove set's remove; (f) distributed garbage collection ("no live reference anywhere"); (g) a join of two grow-only tables; (h) "the report is final: exactly forty visitors attended." *(Hints only.)*

Problem 7.12 (★ the overdraft)**.** A bank demands a replicated counter that supports coordination-free increments and decrements at any replica *and* never exhibits a negative value at any replica. (a) Show the PN-counter fails the requirement. (b) Argue from Thm. 7.9 that no CRDT can meet it: locate the non-monotone clause. (c) Design the standing compromise — escrow: split the balance into per-replica allowances, spend locally against your allowance, coordinate only to rebalance — and state exactly which operations remain coordination-free. *(Hints only.)*

7.11 Hints

Hint (Problem 7.6)**.** (b) Whoever's dequeue is served first in the story must already have both enqueues of the received item's queue ordered suitably; chase the two program orders and derive that each dequeue must precede the other. (c) Locality assembles per-object stories using shared real time; sequential consistency has surrendered exactly that shared resource.

Hint (Problem 7.7)**.** (b) From the aunt: one-write before two-write (else her first read is illegal); from the uncle: the reverse. (c) In Exhibit 7.1 the write $\prec$ read edge enters $\rightarrow_{\mathrm{rt}}$, and the aunt's read returns a value whose write is $\rightarrow_{\mathrm{rt}}$-overwritten. (d) With overlapping writes, $\prec$ adds nothing; the plain-causal argument stands.

Hint (Problem 7.9)**.** (b) The crosswise reads force each transaction before the other in any serial order — a cycle; yet every individual read and write linearizes happily per register.

Hint (Problem 7.11)**.** Monotone: (a), (b), (g). The rest each assert a completed totality over inputs possibly in flight — find the phrase "no further" hiding in each. For (e): testimony accumulates (monotone as implemented), yet the *user-facing* membership predicate is non-monotone — the construction's whole cleverness is that re-drafting.

Hint (Problem 7.12)**.** (b) "Never negative" quantifies over all interleavings of decrements still in flight: admitting one more decrement can retract the lawfulness of another — the set of spendable funds must be sealed. (c) Increments and within-allowance decrements stay free; only rebalancing coordinates.

7.12 Solutions

Tier I in full (tersely); Tier II in full — Problem 7.10 is the sabotage certification; hints only for Tier III.

Solution (to Problem 7.1)**.** (a) Points: read at 2.5, write at 3, read at 5.5: zero then one; legal, extends $\prec$. Linearizable. (b) Points: write at 3.4, Tuppy's read at 3.6 (one), Gussie's read at 3.2 (zero): all inside their intervals, story zero–write–one lawful in the order read-zero, write, read-one. Linearizable (order the overlapping reads around the write's point). (c) The writes overlap: order two before one (points: two at 2.5, one at 2.8); then Gussie lawfully reads one at 5.5 — but the aunt, *after* Gussie ($\prec$), reads two: her read's point must follow Gussie's, and the latest write before both is one. Contradiction; and the symmetric ordering fails Gussie. Not linearizable (the flicker, in transaction-free clothing). (d) Write $\prec$ read; any points put the write first; a legal register story then answers ten, not five. Not linearizable — and no machinery was mentioned in the verdict.

Solution (to Problem 7.2)**.** (a) Yes: order pear-enqueue before apple-enqueue (they overlap); dequeue lawfully yields the pear. (b) No: both orderings of the enqueues are available, but no sequential queue yields the same item twice — the fault is against the specification, whatever the times. (c) Yes: pear first, apple second, dequeues in $\prec$ order return pear then apple — first in, first out honoured. Verdicts (a) and (c) exercise the definition's mercy on overlaps; (b) is the story-legality clause alone.

Solution (to Problem 7.3)**.** (a) Read your writes. (b) Monotonic reads (one session, view moved backwards). (c) Monotonic writes (the correction must be ordered after the notice everywhere). (d) All four intact: the fresh client has no session; the anomaly is a *causal* violation (writes-follow-reads binds Bertie's session, not third parties — the cross-client guarantee is Def. 7.7, and this is precisely the gap between the session tier and the causal landing).

Solution (to Problem 7.4)**.** (a) Invariant: at most one row per name — it spans the *absence* of rows; two claimants each read no-row (lawful photographs) and insert distinct fresh rows: disjoint writes, both commit, two rows. Materialize: a per-name reservation row that every claim must update. (b) Invariant spans two balances; each withdrawal reads both, debits its own. Materialize: a pair-total row updated by every withdrawal. (c) The vault: Problem 7.8. Materialize: the duty token. In each case the materialized row forces write-set intersection, so Def. 7.12's commit rule sees the fight.

Solution (to Problem 7.5)**.** (a) Sequential consistency (one story via the log; program order per client if clients are pinned or the log timestamps their submissions; real time surrendered at the local reads). (b) Sequential consistency plus read-your-writes and monotonic reads for the pinned session — the session tier atop (a). (c) Linearizability: every read's point is its passage through the log. (d) Eventual at best from the brochure — and last-writer-wins makes even convergence a convergence *onto data loss* (Rem. 7.15); "strongly" is doing no work in that sentence unless the merge is a lawful join, which a perjured-clock arbitration technically is — whereupon the honest label is: strong eventual convergence under an unjust law.

Solution (to Problem 7.6)**.** (a) $H|p$: story "Bingo enqueues x; Tuppy enqueues y; Tuppy dequeues x" — respects each client's order on p, and FIFO yields x (enqueued first). $H|q$: symmetrically "Tuppy enqueues y; Bingo enqueues x; Bingo dequeues y." Each object alone: sequentially consistent. (b) Suppose one story S for both. In S, Bingo's dequeue of y from q requires y's enqueue on q before it with x's enqueue on q not before y's (FIFO: the head must be y); since Bingo's program order puts his q-enqueue of x before his dequeue, S must order Tuppy's y-enqueue on q *before* Bingo's x-enqueue on q. By Tuppy's program order, his y-enqueue on q precedes his y-enqueue on p, which precedes his dequeue of x from p; that dequeue requires (FIFO on p) Bingo's x-enqueue on p before Tuppy's y-enqueue on p — and by Bingo's program order, Bingo's p-enqueue precedes his q-enqueue. Chain the requirements: Bingo's q-enqueue after Tuppy's q-enqueue, which is after … Bingo's p-enqueue, which is before Bingo's q-enqueue — consistent so far; the contradiction closes on the dequeues: FIFO on q forces y dequeued while x waits, so Bingo's dequeue precedes any dequeue of x; FIFO on p symmetrically forces Tuppy's dequeue of x to find x at the head,

requiring Bingo's dequeue of y *not yet* to have consumed … chase it fully and each dequeue must strictly precede the other in S. No such total order exists. (c) Locality's assembly pools per-object points on the shared real-time line; sequential consistency's per-object stories run on private timelines with no common line to pool them on.

Solution (to Problem 7.7)**.** (a) As Theorem 7.5: the single story orders program order by definition and reads-from by register legality; transitive closure follows; restriction to (client, all writes) preserves legality and order. (b) Views: aunt's S_a: write one; read one; write two; read two — legal, extends $\rightarrow$ (the writes are $\rightarrow$-unrelated, so either order is available to her). Uncle's S_u: write two; read two; write one; read one — likewise. Causally consistent. Not sequentially consistent: a single legal story must order write-one and write-two once. If one is first, the uncle's first read of two requires two after one *before* that read, and his second read of one requires one to be the *latest* write before it — but one precedes two in the story, so after two's appearance, one is never latest again; his second read is illegal. Symmetrically if two is first, the aunt's pair fails. No story serves both. (c) Real-time causal: replace $\rightarrow$ by $\rightarrow_{\mathrm{rt}}$ in Def. 7.7. In Exhibit 7.1, write-ten $\prec$ read, so write-ten $\rightarrow_{\mathrm{rt}}$ read; any legal view of the aunt must place her read after write-ten with no later write of five — five's write (the initial establishment) precedes write-ten in every view extending $\rightarrow_{\mathrm{rt}}$; the read returning five is illegal in all of them. Not real-time causal; yet sequentially consistent (Exhibit 7.1's caption story, which reorders across clients — permitted, since program order alone is preserved). (d) Stage Exhibit 7.2 with the two writes overlapping in real time: $\prec$ then relates neither write to the other nor (arranging the reads after both responses) constrains the reads' sources beyond plain causality; the views of (b) remain lawful, so the history is real-time causal; the proof in (b) that no single story exists is untouched. Hence: sequential and real-time causal are incomparable (7.1 separates one way, 7.2 the other); plain causal is strictly weaker than sequential (a and b). The folklore, corrected, is now equipment.

Solution (to Problem 7.8)**.** (a) Rows $a, b = \mathsf{in}$; $s(T_1) = s(T_2) = \sigma$; both photographs show (in, in); guards pass; T_1 writes $a := \mathsf{out}$, T_2 writes $b := \mathsf{out}$; $c(T_1) < c(T_2)$; write sets $\{a\}$, $\{b\}$ disjoint, so the commit rule passes both. Final: vault unattended. (b) In either serial order the second program's photograph shows one in; its guard fails; it writes nothing; every serial execution ends with at least one officer in — induction as in Prop. 7.6. (c) With the duty token in both write sets, $s(T_2) < c(T_1) < c(T_2)$ and the sets intersect on the token: T_2 aborts by first committer wins; the retry's photograph shows one in and the guard holds it inside. (d) The serializable refinement tracks the crosswise read-write dependencies (T_1 read b which T_2 wrote; T_2 read a which T_1 wrote): a cycle of two; the engine aborts the later committer, T_2; the retry reads $a = \mathsf{out}$, guard fails, stays — invariant preserved with no schema change, the premium paid in the abort.

Solution (to Problem 7.9)**.** (a) History: T_1 deposits (read balance one hundred, write one hundred fifty, commit); later, disjointly, T_2 reads and returns one hundred. Serializable: the order T_2 then T_1 explains every read from the initial state. Not strictly serializable: $T_1 \prec T_2$ in real time, and in the order T_1, T_2 the read must return one hundred fifty. Both halves proved; the defence "it is serializable" is sound and the complaint "it ignored the clock" is also sound — they are different contracts. (b) Registers x, y, both zero. T_1: read y (returns zero), then write $x := 1$. T_2: read x (returns zero), then write $y := 1$. Interleave the four operations without overlap in the order: T_1 reads y; T_2 reads x; T_1 writes x; T_2 writes y. Each individual operation linearizes trivially (single-register histories with no overlaps are their own linearizations — per-object legality throughout). Serializability: T_1 before T_2 requires T_2's read of x to return one; T_2 before T_1 symmetrically; neither serial order matches. Hence: per-object linearizability does not compose into transactional serializability (brackets are a different axis), and serializability does not imply real-time respect — neither axis implies the other.

Solution (to Problem 7.10 — the certification)**.** The fatal execution, tags and all. Two replicas, study and club; empty initial state.

(1) Study executes add(kettle): draws tag one; state $A = \{(\text{kettle}, 1)\}$, removed-values $V = \emptyset$.

(2) Study's state gossips to the club; club absorbs it.

(3) Club executes remove(kettle): under the "optimization" it records the *value*: $V = \{\text{kettle}\}$.

(4) Concurrently — before any further gossip — study executes add(kettle): fresh tag three; study's $A = \{(\text{kettle}, 1), (\text{kettle}, 3)\}$.

(5) The states merge (either direction; both, for completeness): $A = \{(\text{kettle}, 1), (\text{kettle}, 3)\}$, $V = \{\text{kettle}\}$. contains(kettle) tests membership of the *value* in V: **false**. The kettle of tag three — added concurrently with the remove, never observed by it — has vanished. Every replica agrees it has vanished (the semilattice converges faithfully), and no future gossip resurrects it; worse, *no future add ever succeeds*: any later add(kettle) draws a fresh tag, but the value "kettle" remains in the grow-only V forever — the customer can never buy a kettle again.

The violated sentence is Prop. 7.8 — the advertised add-wins specification: an add concurrent with a remove must survive their joint absorption. Step (5) exhibits the breach; the permanence claim follows from V's grow-only nature. What the engineer's citation proves: convergence — Theorem 7.7 holds, all replicas agree, forever, on the *wrong cart*. Convergence is a property of the algebra; correctness is a property of the law; the optimization changed the law (from testimony-against-observed-tags to condemnation-by-name) while keeping the algebra, and the theorem is loyal to the algebra. Certified fatal: acknowledged customer data (the tag-three kettle) is silently and permanently destroyed under an execution requiring no failures at all — one concurrent add and remove, the store's daily weather.

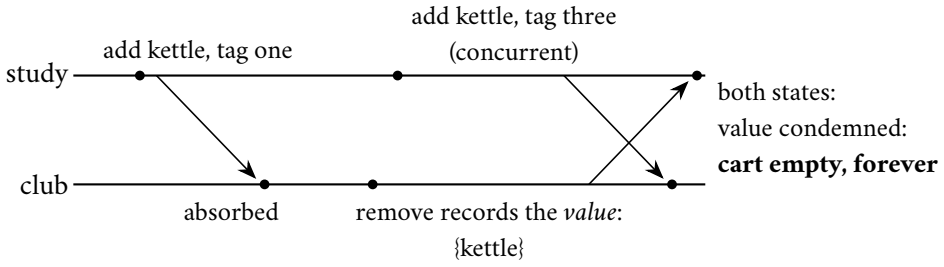


Exhibit 7.5 (the vanished kettle). The same staging as Exhibit 7.4 under the sabotaged rule: the remove condemns the *name*, and testimony it never gave defeats the concurrent add of tag three — permanently, since the condemned-values set only grows. Compare the two exhibits line by line: the jurisprudence is the entire difference.

Solution (to Problem 7.11)**.** Hints stand (Tier III). The classification to argue toward: monotone — (a), (b), (g); non-monotone — (c), (d), (f), (h), each concealing a "no further news" seal; (e) is the instructive straddle: the implementation (testimony accumulates) is monotone, the user-facing predicate (present: some tag not yet condemned) is not — CALM's licence extends exactly as far as such re-draftings.

Solution (to Problem 7.12)**.** Hints stand (Tier III). The shape to reach: (a) two replicas each decrement the last unit against stale photographs; merge; value negative. (b) "never negative" asserts a totality over in-flight decrements — one more input retracts another's lawfulness; the spendable set must be sealed, and sealing is agreement. (c) escrow: allowances are per-replica grow-only ledgers; local spends against local allowance are monotone; only rebalancing (moving allowance between replicas) coordinates — the invoice confined to the rare operation, the season's recurring settlement.

7.13 The Reading

Herlihy and Wing, "Linearizability: A Correctness Condition for Concurrent Objects," TOPLAS 1990. The summit in the founders' prose: the definition, locality, nonblocking, and the queue examples this chapter drills. Read §2–§3; the proof methodology of §4 rewards a second visit.

Lamport, "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs," IEEE Transactions on Computers, 1979. Two pages; sequential consistency named and priced. The hardware trades have spent four decades relaxing it.

Terry, Demers, Petersen, Spreitzer, Theimer, and Welch, "Session Guarantees for Weakly Consistent Replicated Data," PDIS 1994. Short, practical; the four definitions you will use in a meeting within the month, and Box 15's source.

Berenson, Bernstein, Gray, Melton, and the O'Neils, "A Critique of ANSI SQL Isolation Levels," SIGMOD 1995. Write skew's formal residence; the paper that showed the industry's isolation taxonomy was folklore.

Shapiro, Preguiça, Baquero, and Zawirski, "Conflict-Free Replicated Data Types" (the comprehensive 2011 technical report and the SSS conference paper). The catalogue reads like a seed catalogue; linger on the garbage-collection sections, where the honest costs live (Box 17's closing note).

Hellerstein and Alvaro, "Keeping CALM: When Distributed Consistency Is Easy," CACM 2020. The horizon in its authors' voice; the proof lives in Ameloot–Neven–Van den Bussche's transducer papers, cited therein.

Kingsbury: the Jepsen consistency map, with any one full analysis of your acquaintance. Assigned as standing equipment: the atlas beside the whiteboard for the rest of the season, and one autopsy read to watch this chapter's definitions arrest a production database in the act.

The shelf. Ahamad, Neiger, Burns, Kohli, and Hutto, "Causal Memory" (Distributed Computing, 1995) — the causal landing formalized, the hierarchy's middle floor in full. Bailis, Venkataraman, Franklin, Hellerstein, and Stoica, "Probabilistically Bounded Staleness for Practical Partial Quorums" (VLDB 2012) — the floodplain's actuarial tables: eventual, quantified. Gibbons and Korach, "Testing Shared Memories" (SIAM Journal on Computing, 1997) — Rem. 7.3's hardness, for the ambitious.

Chapter 8

Transactions at a Distance

Companion to Episode Eight. The episode tells the story — the fainting officiant, the couple at the altar, the pendulum of fashion swinging wall to wall; this chapter keeps the record: atomic commitment specified beside consensus with the veto doing the separating, Gray's two-phase ceremony with its full recovery matrix, the blocking theorem proved by two worlds, three-phase commit under museum glass, and the modern settlements one per verb — the fragile roles relocated onto parliaments, Spanner's commit-wait made a theorem one can carry, Percolator's coordinator dissolved into the data, Calvin's abolished conversation, the sagas' honest abandonment of isolation with its anomaly drawn as a trace. Statements, proofs, price tags, problems.

8.1 Part One — Atomic Commitment, Specified

One transaction has done its work across several *participants* — a debit staged at Tuppy's branch, a credit at Gussie's, locks held per Chapter 7's machinery — and somebody must decide: all of it permanent, or none. That is the atomic commitment problem. The episode opens it with a wedding, and the chapter keeps the image as equipment, for it is exact: the vows are prepare votes, irrevocable by protocol; the parish register is the coordinator's log; the officiant's faint is a crash; the couple's paralysis at the altar is the blocking theorem, proved below by the doppelgänger. Every clause of the canon is load-bearing; changing any one merely changes which party is ruined. The fashion has since swung wall to wall — empires built on the ceremony, a decade sworn off it, the planet-scale ledgers paying the price properly at last — and the one question for an architect at any station of the pendulum is: which theorem are you paying, and in what currency? The models are fixed first, per the season's standing habit.

Model of this chapter

For commitment (Defs. 8.1–8.5, Thms. 8.1–8.2). Asynchronous message passing; fair-loss links with retransmission beneath (Chapter 1); crash-recovery processes with stable storage: a process loses volatile state on crash, retains its log, and may recover after unbounded delay — or never. No failure detector; silence and death indistinguishable (Chapter 3). One designated coordinator, n participants; the transaction's provisional work and locks are in place before the protocol begins (isolation is Chapter 7's department).

For commit-wait (Def. 8.6, Thm. 8.4). An absolute time t_{abs} exists (the analysis is external; no process reads it). Every process may call $TT.\text{now}()$, receiving an interval $[e, l]$; the *TrueTime axiom*: at the instant of the call, $e \le t_{\text{abs}} \le l$. Intervals from different calls and processes need share nothing but the axiom.

For sagas (Def. 8.8, Prop. 8.5). Each step and each compensation is a local transaction, atomic and durable at its own service; no lock, snapshot, or isolation mechanism spans steps; concurrent transactions read committed states freely between steps.

Definition 8.1 (atomic commitment)**.** Each of n participants votes YES or NO; each participant and the coordinator may *decide* COMMIT or ABORT. Required: **AC1 (agreement)** no two processes decide differently; **AC2 (validity)** if any participant votes NO, no process decides COMMIT; if all vote YES and no failure occurs, all decide COMMIT; **AC3 (stability)** a decision is never changed; **AC4 (termination)** in every execution in which all failures are eventually repaired, every participant eventually decides (*weak*); a protocol is *non-blocking* if every correct participant decides even while other processes remain crashed (*strong*).

The veto. The resemblance to Chapter 3's consensus deceives everybody once. Consensus validity is generous — any proposed value will do; AC2's first half is unanimity in one direction, and that grants each participant a *veto*: the shyest teller in the branch can refuse, and the refusal is law. Consensus is a parliament — a silent minority is outvoted. Atomic commitment is a wedding: a silent party who has said *I do* cannot be left behind, because the vow may already be part of a completed marriage nobody present can see. The veto converts silence into paralysis; the rest of the chapter works that sentence out.

Remark 8.2 (the veto; commitment versus consensus)*.* Consensus validity permits any proposed value; AC2's first half grants every participant a veto, and its second half forbids gratuitous aborts in failure-free runs. Reductions run both ways in the crash-stop asynchronous model (votes, then consensus on the conjunction, gives commitment with consensus-grade liveness — the route of Boxes 18's parliaments; commitment instances decide consensus-like questions in return), so FLP shadows both. What the veto changes is failure economics: a consensus majority outvotes the silent; a commit ceremony cannot — the silent voter may hold a NO, or evidence of a sealed COMMIT, and Thm. 8.2 turns that asymmetry into a theorem.

The trade's proper nouns. Classically the participants are **resource managers** — databases, queues, ledgers, each sovereign over its own durable state — and the coordinator a **transaction manager**, standardized in the late eighties so one vendor's officiant could marry another vendor's parties; the interface survives in every application server, and so do its failure modes. A yes is not an opinion but a *certificate of staged durability*: I have written everything to stable storage such that, on command, I can commit through any crash. A participant answering yes from memory alone has counterfeited it; counterfeits are discovered at the worst possible audit.

Fine print on the specification. Four clauses worth carrying:

- **AC4's two grades:** weak termination (everyone decides if no failures occur) is cheap, and two-phase commit delivers it; the prize is the strong, non-blocking grade — every correct participant deciding while others lie dead.
- **Aborts are free only before the vote:** a coordinator may abort on any sneeze while collecting; after the yes votes are cast, generosity becomes forgery.
- **Isolation lives elsewhere:** commitment decides *whether* the work becomes permanent; Chapter 7's levels govern *who may peek* meanwhile. Separate pockets — Part Eight's sagas abandon one and not the other.

- **FLP shadows both:** commitment and consensus trade blows formally (Rem. 8.2 runs the reductions), so every protocol below leans on timeouts, majorities, or luck exactly where Chapter 4's did; the veto changes only the failure economics.

The founding text, and the three verbs. Gray's "Notes on Data Base Operating Systems" (1978) presents the two-phase ceremony fully formed, with the observation that would haunt it: a window in which a participant's fate lies in another machine's private log. Gray called the trapped state **in-doubt**; the industry, limbo; the couple at the altar, Tuesday. No cleverness removes a clause of the specification; what cleverness can do — the second half of the chapter is its catalogue — is one of three verbs: *relocate* the fragile roles onto sturdier machinery, *shrink* the windows, or *renegotiate* the specification itself. Every modern system below is one of the three.

8.2 Part Two — The Ceremony, Anatomized

The standing cast: Bertie moves ten pounds from Tuppy's branch to his aunt's account at Gussie's; the provisional work is staged, locks held; Bingo officiates as coordinator; the Post carries every word under the ordinary charter. The episode walks the ceremony line by line; the skeleton, beside Box 18:

- **Phase one, the vows:** Bingo sends PREPARE to each participant. An able participant performs the load-bearing act of the whole protocol: it *forces* its staged work and its vote to the **write-ahead log** — the record of an intention made durable before the intention is acted upon — and only then replies YES. From that instant the voter is bound — no unilateral abort, not on timeout, not on suspicion, because the yes may already be part of a commit sealed elsewhere. A NO is the one luxurious word in the protocol: a no-voter aborts at once and goes home.
- **Phase two, the pronouncement:** all votes yes — Bingo forces the commit decision to his own log, *the court of record*: the decision *is* the forced write, not the announcement — and only then tells the participants. Any no, or silence past his patience: abort. Participants apply, release locks, acknowledge.
- **The postage:** three messages per participant of consequence, plus acknowledgments, and — more dearly — two forced log writes on the critical path: each participant's vote, the coordinator's decision. Forced writes are the protocol's heartbeat: each marks a place where a crash is survivable; every economy on them, a place where it is not.

Two lawful economies. Both descend from Gray's world and respect the vow:

- **Presumed abort** — absence means annulment: the coordinator may abort freely before logging a decision, so let the absence of a record mean ABORT by standing convention. Forced writes are saved on every failure path, and the recovery question — no record; what happened? — acquires a standing answer. Its mirror, presumed commit, relocates a forced write rather than removing it (Problem 8.2).
- **The read-only vote:** a participant whose portion wrote nothing answers the prepare with a third word, READ-ONLY, releases its locks, and exits before the pronouncement — either outcome is the same to it. In estates that read widely and write narrowly, that word is worth more latency than any protocol cleverness of its decade.

The part Gray insisted upon and folklore forgets: the ceremony is defined as much by its recovery rules as by its happy path. The matrix goes by *the widow's rights* — the action on waking, per logged state — and Box 18 sets it out; the in-doubt line is the one the whole chapter orbits.

Protocol — Box 18. Two-phase commit with recovery matrix (checked against Gray 1978; presumed abort per the classical treatments)

1: **coordinator, phase one:** send PREPARE to all participants; start patience timer
2: **participant, on PREPARE:** if unable: reply NO, abort locally (never voted YES)
3: if able: *force* staged work + YES to log; reply YES — veto surrendered
4: if no writes performed: reply READ-ONLY; release locks; exit protocol
5: **coordinator, phase two:** all YES (or READ-ONLY):
6: *force* COMMIT to log *(the decision is this write)*
7: any NO, or patience expired: decide ABORT (no forced record under presumed abort)
8: send decision to all YES-voters; retransmit until each acknowledges
9: **participant, on decision:** apply; release locks; acknowledge (idempotent on repeats)
10: **recovery matrix — on waking, consult own log:**
11: participant, *working* (no vote): abort locally; a later PREPARE gets NO
12: participant, *prepared*: in doubt — query coordinator (and, cooperatively, fellows); hold locks; wait
13: coordinator, no decision record: decide ABORT; answer queries with ABORT (presumption)
14: coordinator, COMMIT record: re-announce to all; retransmit to the unacknowledged
15: either role, decision applied: answer queries from the log; nothing else to do

Two forced writes on the happy critical path (each vote; the decision). Cooperative termination: a queried fellow with a decision repeats it; a fellow still *working* may vote NO and release the querent. The in-doubt state (line 12) is Thm. 8.2's theorem made flesh: no rule for it decides.

The states, formally, with the couple's paralysis given its dry name.

Definition 8.3 (the two-phase protocol; in-doubt)**.** Box 18's protocol. Participant states, as logged: *working* (no vote recorded); *prepared* (staged work and YES vote forced to the log); *committed*; *aborted*. A participant is *in doubt* while its log says prepared and it holds no decision. The coordinator's states: *collecting*; *committed* (decision forced); *aborted* (forced, or presumed by absence under the presumed-abort convention).

The happy path, drawn once in the season's convention:

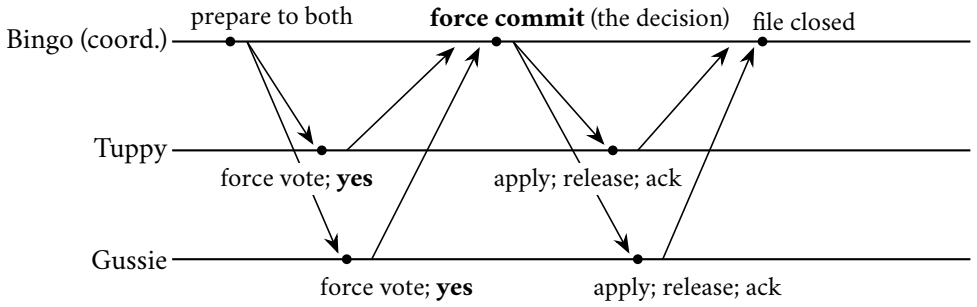


Exhibit 8.1 (the ceremony, happy path). Twelve utterances, two forced writes on the critical path (each vote; the decision). Cut the transcript at any line and consult Box 18's matrix for the widow's rights — Problem 8.1 does exactly that.

The ceremony's safety claim, its skeleton first: commit requires a durable unanimous record, and every recovery rule of Box 18 re-derives its action from forced state, so no crash-and-recovery sequence manufactures disagreement.

Theorem 8.1 (2PC safety)**.** *Under the model box, Box 18 satisfies AC1–AC3 in every execution, through any sequence of crashes and recoveries: no two processes ever decide differently, no* COMMIT *is decided against a* NO *vote or a missing vote, and decisions are stable. It satisfies weak AC4; it is not non-blocking (Thm. 8.2).*

Proof of Theorem 8.1. AC2, first half. A NO voter never enters prepared; the coordinator decides COMMIT only on *n* YES votes (Box 18, line 6), and a recovering coordinator decides only from its forced log, which can contain COMMIT only if the votes were unanimous when forced. Hence no execution with a NO vote contains a COMMIT record anywhere, and participants commit only on receipt of a coordinator COMMIT (or its repetition).

AC1. All participant decisions are copies of coordinator decisions, so it suffices that the coordinator never issues both. The decision is the first forced decision record (line 6/7); the log is sequential and stable; recovery re-reads and repeats the recorded decision (lines 12–14) and, under presumed abort, treats absence as ABORT — absence is possible only if no COMMIT was ever forced, since COMMIT is forced before any announcement (line 6 precedes line 8). A participant's own unilateral ABORT occurs only from *working* (never voted: lines 15–16), in which case no COMMIT record can ever come to exist (AC2 argument with the missing vote). So all decisions in one execution are equal.

AC3. Decisions are forced log records; every rule in Box 18 consults the log before acting and no rule overwrites a decision record.

Weak AC4. Suppose all failures are eventually repaired and retransmission is stubborn. A working participant eventually receives prepare or the coordinator's abort; a prepared participant eventually reaches a recovered coordinator whose log answers (decision, or absence = abort under presumption — lawful since no commit was forced); committed and aborted states are terminal. The coordinator eventually collects all votes or expires its patience and aborts. Every path terminates in a decision. □

Engineering footnotes, scars from real ledgers.

- **The pronouncement is the ink, not the breath:** a coordinator that tells participants commit before its own log is durable has built a world where its crash *un-decides* a pronounced marriage. The AC1 argument leans on exactly the line 6-before-line 8 ordering.
- **The in-doubt window holds locks:** a fainted officiant stalls the whole parish's business calendar (Chapter 7 taught what held locks do). The window's length — milliseconds happy, recovery-long crashed — is the number a practitioner watches above all others.
- **Two patience dials, pulling opposite ways:** the coordinator's patience with voters is cheap generosity — expiring it early aborts a transaction nobody had yet promised. The participant's patience with a silent coordinator, after a yes, buys nothing at any price — the theorem ahead — so the practical dial there is an escalation, not a timeout: page the operators, surface the transaction on the **in-doubt console**, and in the last resort adjudicate by hand with the auditors watching. Every mature transaction manager ships that console; its existence is the theorem's signature in the product.

8.3 Part Three — The Blocking Theorem, by Doppelgänger

The claim, stated with care: in any atomic commitment protocol in which participants surrender their veto — and with asynchronous messages, some vote-surrendering step is unavoidable for progress — there is a reachable situation in which a correct, live participant cannot decide anything and must wait, indefinitely, on the recovery of a crashed party. Blocking is not a defect of Gray's ceremony; it is a property of the specification meeting the world. The instrument is Chapter 3's doppelgänger, tuned one string tighter:

put Tuppy on the stand — voted yes, vow in his log, and now silence — between the *two worlds* of the proof below: one in which a commit exists, sealed by parties who then died, the letter to him still in flight; one in which nothing was ever decided anywhere. His evidence is identical in both; abort splits world one, commit forges in world two; only waiting survives — the paralysis is the couple's good character. The prosecution needs one crashed coordinator, one crashed participant, one slow letter: every ingredient ordinary by Chapter 1's documentary evidence.

Theorem 8.2 (blocking; Gray, Skeen)**.** *No atomic commitment protocol in the model box is non-blocking: every protocol satisfying AC1–AC3 has a reachable configuration in which some correct, live participant has voted* YES *and can neither decide* COMMIT *nor decide* ABORT *until some crashed process recovers. In particular there are executions in which locks are held for the full duration of a coordinator outage.*

Proof of Theorem 8.2. Fix any protocol satisfying AC1–AC3 in the model box, with coordinator B and participants T (Tuppy) and G (Gussie); the argument generalizes verbatim to n participants. Since the protocol satisfies AC2's second half, there is a failure-free execution ρ in which all vote YES and all decide COMMIT. Let ρ_0 be the prefix of ρ ending at the moment T completes its YES vote, and consider the following two extensions, in both of which T receives no further messages for as long as we please (the Post is asynchronous; delay is lawful).

World one. After ρ_0: messages flow between B and G as in ρ; the protocol being live in failure-free runs, B reaches its COMMIT decision and G decides COMMIT (AC2 second half applied to the run where T's silence is merely delay); then B crashes, and G crashes, with every message addressed to T still in flight. A COMMIT decision exists (at G, decided; stability AC3 makes it permanent).

World two. After ρ_0: every message between B and G after ρ_0 is delayed; B and G crash before deciding anything. No decision exists. (If the protocol would have B decide with no further input, run world one's argument from that decision instead; the construction adapts.)

T's local history — its own steps, its log, its received messages — is identical in both worlds: everything after ρ_0 happened elsewhere or not at all, and silence carries no signature. A protocol is a function from local history to action. If it directs T to decide ABORT, then in world one T's ABORT stands against G's COMMIT: AC1 violated. If it directs T to decide COMMIT, consider world two continued: B recovers and — being permitted by AC1–AC3, indeed possibly required, since a vote may never have reached it — resolves the transaction ABORT; alternatively note that in a variant of world two where G voted NO before crashing (indistinguishable to T, whose evidence never mentions G's vote), AC2 *forbids* COMMIT outright. Either way T's COMMIT violates a clause. Hence the protocol may direct neither decision: T must wait, and it waits precisely until some crashed process — a decision-holder — recovers. This is the reachable blocking configuration claimed. □

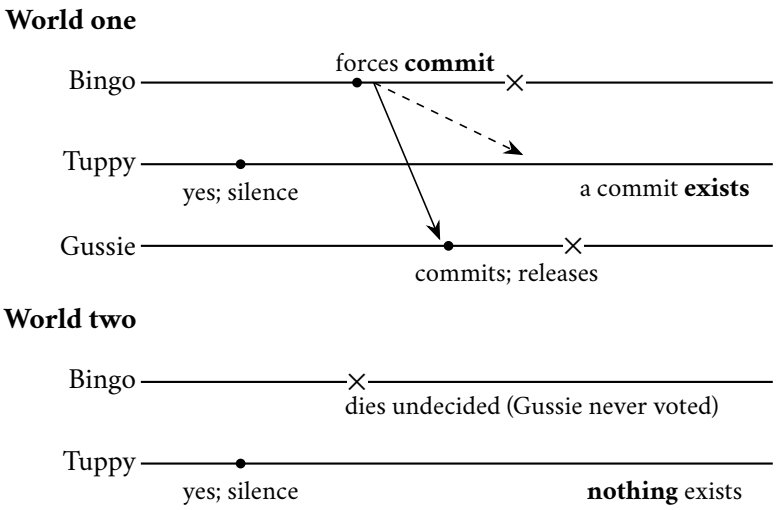


Exhibit 8.2 (the two worlds). Tuppy's evidence — his vote, his log, his silence — is identical in both. Abort splits world one; commit forges in world two; waiting alone is safe in both (Thm. 8.2). The crosses are crashes; the dashed letter to Tuppy is still in flight, indefinitely, lawfully.

Two mirages. First: let Tuppy ask Gussie — **cooperative termination**, decent practice, anticipated by the proof: a dead fellow testifies to nothing, and a reachable fellow who is himself yes-and-ignorant merely joins the vigil — limbo shortened when luck is moderate, removed never, and theorems are priced at poor luck. Second: let the coordinator distribute word of the impending decision before finalizing it — sir has just reinvented the third phase, and the museum in the next part is its mausoleum.

Remark 8.4 (cooperative termination, and its limit)*.* An in-doubt participant may query its fellows: any fellow holding a decision repeats it; any fellow that has *not* voted may vote NO, decide ABORT, and thereby release the querent (safe by AC2). The refinement fails exactly when every reachable fellow is prepared-and-ignorant and every decision-holder is dark — the total failure of decision holders — which is the configuration Thm. 8.2's proof constructs. Blocking is thereby confined, not removed.

The theorem's exact perimeter.

- **When blocking bites:** exactly when every party that might hold the decision is unreachable — a total failure of the decision-holders.
- **The companion results:** grant a synchronous network and forbid partitions, and non-blocking protocols exist (the museum's exhibit builds one); allow total failures or partitions, and every protocol has executions that block.
- **What it does not say:** not that the wait is forever — Bingo recovers, reads the record, repeats the pronouncement — nor that every failure blocks: a coordinator crash before any commit is sealed resolves to abort, and a participant crash strands nobody but itself if the coordinator lives.
- **What it says:** there exists a conjunction — yes votes surrendered, decision-holders dark, evidence symmetric — in which no rule can do better than wait. The practitioners' summary: two-phase commit blocks rarely and briefly when engineered well, always possibly; the distance between those adverbs is the craft of Box 18's matrix.

Heuristic abort, convicted in advance. The sabotage ships in real middleware: the participant configured to "time out and roll back" after voting yes — **heuristic abort**, with an adjective doing heroic work. The doppelgänger has shown which world it gambles against; Problem 8.6 — the chapter's sabotage, certified fatal — writes out the split-brain ledger it produces. The record is not hypothetical: every vendor's documentation instructs administrators in resolving in-doubt transactions by hand, and the interface standards define the *heuristic outcome* — "we guessed" — complete with error codes: heuristically committed, heuristically rolled back, and, the standards body's own name for the split ledger, heuristically mixed. When a protocol's official vocabulary includes a word for *we guessed*, the theorem has been acknowledged in the small print.

8.4 Part Four — The Museum of Three Phases

The first generation tried to beat the theorem; the handsomest attempt, three-phase commit (Skeen, 1981), earns its glass case. Blocking, Skeen observed, comes from a fatal adjacency: a state — voted yes, awaiting news — from which both commit and abort remain reachable. Very well: forbid the adjacency. Insert a buffer state between vowed and married — the *pre-commit* of the definition below — so the states march in single file: uncertain, prepared-to-commit, committed. A participant stranded in uncertainty knows nobody can yet have committed — abort is safe; one stranded in pre-commit knows nobody can still be uncertain — and there the cleverness lives, in the termination rule under a new officiant, elected by the survivors with Chapter 4's machinery, smuggled in at the door.

Definition 8.5 (three-phase commit)**.** Skeen's protocol: between prepared and committed insert *pre-commit*. Coordinator, on unanimous YES: disseminate pre-commit; on acknowledgment from all reachable participants, commit. Termination rule under a new coordinator: if any canvassed participant is committed, commit all; if any is still uncertain (prepared, no pre-commit), abort all; if all reachable are in pre-commit, complete the commit.

Proposition 8.3 (3PC, both verdicts)**.** *(i) In a synchronous crash-stop model with reliable bounded delivery and no partitions, 3PC with its termination rule is non-blocking. (ii) With partitions the protocol violates AC1: there is an execution in which one side of a partition commits while the other aborts (Problem 8.7 constructs it). The additional round and its forced writes are paid in every failure-free execution.*

The placard. Each termination rule is sound — against crashes, under assumptions seven chapters of the course have already buried:

- **The assumptions:** a synchronous network, bounded delays honoured (Chapter 1's fiction, Chapter 4's expensive rental), no partitions — and a partition is not exotic but a Tuesday.
- **The split verdict:** under a partition the buffer state inverts into a weapon — one shore's survivors, seeing pre-commit, complete the marriage; the other's, seeing uncertainty, lawfully annul it; the cable heals and the estate holds both verdicts (Problem 8.7 constructs it) — the precise catastrophe the extra phase was hired to prevent.
- **The bill in fair weather:** an additional round of letters and forced writes on every transaction, paid always, against a crash that comes rarely. The verdict of history: nobody runs it.
- **The scholarly line:** quorum-based variants repair the split verdict by requiring majorities before any shore may move — a parliament reinvented with extra steps; the literature's settlement kept the parliament and dropped the steps.

The placard, fairly written: *a correct solution to a problem the real network declines to pose, at a price the happy path declines to pay.* The true legacy is diagnostic: the enemy was never the number of phases but the assumption of a world in which silence can be told from death. What three-phase commit wanted, without knowing the words, was Chapter 5's failure detector — and the honest form of that bargain, agreement by majority rather than by clockwork, already sat in Chapter 4's parliament, waiting to be noticed.

8.5 Part Five — Buying Better Physics: Consensus Groups, Spanner, and the Seven-Millisecond Apology

The modern settlement begins with the first verb: *relocate*. The blocking theorem's pressure point was a singular mortal holding a singular log — the officiant and his register — and Chapter 5's remedy applies: make the throne an office, the log a parliament's decree. Run the coordinator as a replicated state machine — its decision a committed entry in a consensus group, the fainted officiant replaced by Chapter 4's election, the successor reading the *replicated* register and completing the ceremony; run each participant's vote the same way, and no single machine's death strands anyone.

What is and is not purchased.

- **The theorem stands:** consensus itself stalls while a partition denies majorities, the in-doubt locks held meanwhile. Limbo shortens from "until a particular machine is repaired" to "until a majority can be assembled" — shortened, not abolished.
- **The price:** every phase now costs a consensus round rather than a disk write — the correct trade, and the one the industry made.
- **The mechanics,** assumed hereafter: the transactional roles are played by group leaders (Chapter 5's thrones). A participant's yes is a decree — staged work and vote committed through its group's log, any successor inheriting the log and the promise with it; the decision is pronounced the instant the commit entry is chosen by majority, and any successor coordinator repeats it. The faint now triggers a by-election; the vigil lasts one election timeout, not one hardware repair.
- **The purists' edition:** consensus on transaction commit, written jointly by Gray and Lamport — the votes become Paxos instances outright, the coordinator dwindling to a convenience (Problem 8.11 reconstructs it). The industry builds the pragmatic edition; the founders proved the marriage lawful.

Spanner (Corbett and colleagues, Google, 2012) — the course's single most load-bearing industrial text; its architecture, in one breath, is everything the season has built: data partitioned into shards, each shard a Paxos group — Chapter 4's parliament, by the thousand; transactions within a shard decided by its parliament; transactions *across* shards — debt #18 presented for payment — by two-phase commit in which coordinator and participants are all parliaments; isolation by two-phase locking atop snapshot machinery of Chapter 7's kind. What made the paper famous is none of it, but a confession about *time* — and the audacity of answering a logical problem with a purchase order.

Why the clocks return. Google wanted **external consistency** — the paper's term; Chapter 7 licenses its government name, strict serializability: if a transaction commits, and afterwards — in the world's own order, telephone calls included — a second begins, the second must see the first. Within one parliament the log's order supplies this free; across parliaments, whose order? Wall clocks order nothing at fine grain (Chapter 2); vector clocks die at planetary scale and cannot see the telephone — happens-before tracks

messages, and reality carries channels the system never does. To respect *every* side channel, you must respect real time itself. Google's response: change what a clock *is*. TrueTime: GPS receivers and atomic clocks in every datacentre, and an API of magnificent honesty — ask the time and it returns not a number (Chapter 2's lie in a nicer suit) but the **confessed interval**: earliest and latest, the true instant staked to lie between, the width the confessed uncertainty — single-digit milliseconds in the paper's era. Time masters per datacentre — some slaved to the satellites, some to the caesium — are cross-examined so a liar is outvoted; between synchronizations each machine's interval *widens* at an assumed worst-case drift rate and snaps narrow at the next audit — the sawtooth of humility. Upon that one honest confession, commit-wait builds external consistency with arithmetic a child could check.

Definition 8.6 (TrueTime; commit-wait)**.** $TT.\text{now}()$ returns $[e, l]$ with $e \le t_{\text{abs}} \le l$ at the instant of the call. Commit-wait (Box 19): a committing transaction T calls $TT.\text{now}() = [e_1, l_1]$, takes commit timestamp $s(T) = l_1$, and delays announcing (and lock release) until a later call returns $[e_2, l_2]$ with $e_2 > s(T)$. Start rule: a transaction's start event is its first invocation reaching any server.

The set piece, on the episode's concrete numbers, beside Exhibit 8.3:

- **The call:** ready to commit, the coordinating parliament asks TrueTime: earliest ten o'clock and three milliseconds, latest ten o'clock and ten — a confessed uncertainty of seven.
- **Clause one, the stamp:** the *latest* edge, ten-and-ten — not the middle, not the earliest: the one instant no clock in the estate can yet have passed for certain.
- **Clause two, the vigil** — the seven-millisecond apology itself: hold the pronouncement until TrueTime's earliest edge climbs past ten-and-ten — one uncertainty's worth of silence — then announce and release the locks.
- **The payoff:** a second transaction begins genuinely after the first commits — Bertie sees the confirmation in London, telephones the aunt in Sydney, *she* issues the read. The first stamp was held until it lay in every clock's past; Sydney stamps hers at *its* latest edge, which cannot precede the true now: strictly above ten-and-ten. Afterwards in the world implies afterwards in the timestamps — no vector, no gossip, no message from London to Sydney: both parliaments merely deferred to the same purchased, confessed physics.

The argument, structured into a theorem one can carry:

Theorem 8.4 (commit-wait implies external consistency)**.** *Under Def. 8.6, if transaction T_1's commit is announced before transaction T_2 starts (in absolute time), then $s(T_1) < s(T_2)$. Consequently timestamp order extends real-time precedence of transactions, and snapshot execution at these timestamps yields strict serializability (Def. 7.11's clause satisfied with whole transactions as the operations).*

Proof of Theorem 8.4. Write $t_{\text{abs}}(x)$ for the absolute time of event x. *Lemma (the wait pays).* When T_1's announcement occurs, $s(T_1) < t_{\text{abs}}(\text{announce})$. Indeed the protocol announced only after some call returned $[e_2, l_2]$ with $e_2 > s(T_1)$, and the axiom gives $t_{\text{abs}}(\text{that call}) \ge e_2 > s(T_1)$; the announcement follows the call.

Main step. T_2 starts after the announcement: $t_{\text{abs}}(\text{start}_2) > t_{\text{abs}}(\text{announce}) > s(T_1)$ by the lemma. T_2's commit stamp is taken as the *latest* edge of a call made no earlier than its start: $s(T_2) = l' \ge t_{\text{abs}}(\text{call}) \ge t_{\text{abs}}(\text{start}_2) > s(T_1)$, the middle inequality by the axiom's upper half. Hence $s(T_1) < s(T_2)$.

Strictness of the serialization. Order transactions by commit stamp (ties broken arbitrarily but consistently). Multiversion execution at these stamps makes every transaction read the latest stamps below its own — a serial story in stamp order — and the main step shows stamp order extends real-time precedence.

By Chapter 7's Def. 7.11, the execution is strictly serializable. Note what was never used: no message between T_1's and T_2's coordinators, no shared machinery — only the axiom, twice, and one deliberate delay of approximately the confessed uncertainty per commit. □

Protocol — Box 19. Spanner commit-wait (checked against Corbett et al. 2012, §§3–4)

1: **roles:** every shard a Paxos group; participants and coordinator are group *leaders*; votes and decisions are group log entries, not private writes
2: **commit, coordinating leader:** collect prepares (each a committed log entry with a prepare stamp)
3: $[e, l] \leftarrow TT.\text{now}()$; then $s \leftarrow \max(l,$ prepare stamps, watermark)
4: commit the decision at stamp s through Paxos
5: **commit-wait:** block until $TT.\text{now}().e > s$; then announce, release locks, apply
6: **read-only transaction at stamp t:** at each shard, read versions below t; any replica whose safe time exceeds t may serve — no locks, no leader required

The wait is approximately one confessed uncertainty; it enforces Thm. 8.4's lemma. Keeping the confession narrow is an operations discipline: every microsecond of clock sloppiness is paid by every writer, in latency.

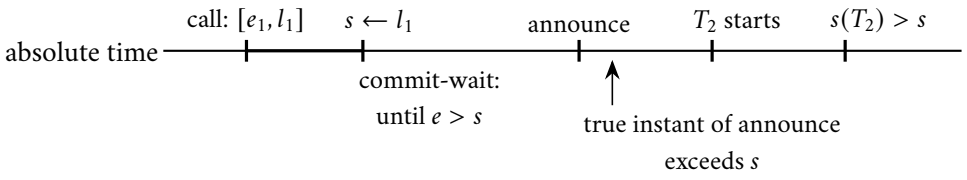


Exhibit 8.3 (commit-wait pays). The stamp is the latest edge; the vigil ends only when every clock's earliest edge has passed it; so the announcement — and everything after it, telephones included — lies strictly above the stamp (Thm. 8.4). One confessed uncertainty of silence per commit buys the ordering no message carried.

The photograph, purchased. Every commit bears a truthful timestamp, so the state of the entire estate as of half past ten is a well-defined object, assembled without locks from every shard's versioned history — Chapter 7's photograph, purchased rather than choreographed. A read-only transaction names a timestamp and reads that snapshot at every parliament it visits, blocking nobody and blocked by nobody; followers serve these reads too — each replica tracks its **safe time**, the watermark beneath which its log is known complete, and any read stamped below it needs no leader at all. The invoice is perpetual: every writer pays the little vigil, and every microsecond of clock sloppiness is paid in latency — keeping the confession narrow is henceforth an operations discipline. Debt #5: presented, and paid in caesium.

Hybrid logical clocks (Kulkarni and colleagues, 2014) — the civilian's TrueTime, and CockroachDB's arrangement of record, for estates without a hardware budget. Weld Chapter 2's two instruments into one stamp: a physical reading disciplined by ordinary NTP, plus a logical counter that ticks whenever causality demands order the physical part cannot certify — Lamport's clock in a wristwatch case. The stamp tracks wall time closely; the counter guarantees message-connected events never stand in the wrong order. What is *not* purchased, said aloud: without a bounded confession there is no commit-wait proof, and the telephone can outrun NTP's error bars; the civilian arrangements paper over the residue with retries and uncertainty

windows, paid in tail latency rather than in caesium. You may buy the bound, or live carefully without it; you may not have it for free.

Definition 8.7 (hybrid logical clock)**.** Each process keeps a stamp (pt, c): pt a physical part, c a logical counter, ordered lexicographically. On a local or send event with wall reading w: if $w > pt$ then $(pt, c) \leftarrow (w, 0)$ else $c \leftarrow c + 1$. On receiving stamp (pt', c') with wall reading w: set $pt'' = \max(pt, pt', w)$; set c'' to (the matching counter) +1 if pt'' equals pt or pt', else 0. The stamp satisfies the clock condition of Chapter 2, tracks wall time to within the clock error, and its counter is bounded (Problem 8.10, starred).

8.6 Part Six — Percolator: The Coordinator Dissolved

The second verb is *shrink*, and its most instructive specimen is Percolator (Google, 2010) — built to index the web incrementally atop Bigtable, a store offering single-row transactions and nothing more. No transaction manager anywhere in the building; the designers' move was cunning: if no machine may be the coordinator, let the *data* be the coordinator — dissolve the ceremony into the rows, and let the court of record be a column. The episode sets out the ledger-keeping in full; the skeleton, beside Box 20:

- **The furniture:** every row carries, beside its data, a lock column and a write column; writes are staged at a snapshot timestamp in Chapter 7's multiversion manner; timestamps come from a single *oracle* dispensing strictly increasing stamps in batches — agreement on the next number needs a till, not a parliament.
- **Phase one, prewrite:** for every written row, plant a lock — a single-row atomic act, Bigtable's one native sacrament. Among its rows the transaction anoints one *primary*: that row's lock cell *is* the transaction's court of record; every other lock is a signpost bearing the primary's address.
- **Phase two, the pronouncement:** at a fresh timestamp, a single-row atomic exchange upon the primary — erase the lock, inscribe the commit record. Before that cell-write nothing is committed; after it everything is; the remaining rows' inscriptions are paperwork, completed lazily. The officiant has become an entry in the parish register that writes itself.
- **Readers:** a reader at its snapshot stamp first checks the lock column — a lock from an earlier stamp means a ceremony is mid-air exactly where the photograph would be taken; wait briefly or, past patience, invoke the cleanup rite, never reading around a pronouncement in progress.

Protocol — Box 20. Percolator transactions (checked against the 2010 paper)

1: **columns per row:** data (versioned); lock; write (commit records). Single-row operations are atomic (Bigtable's sacrament)
2: **prewrite (phase one), for each written row:** abort if a newer write record or any lock exists; else stage data at start stamp and plant lock naming the *primary* row
3: **commit (phase two):** at a fresh commit stamp, on the primary, atomically: erase lock, inscribe write record — *this cell-write is the pronouncement*
4: then, lazily: inscribe write records and erase locks on the secondaries
5: **read at snapshot stamp t:** if a lock below t blocks the row: wait briefly, then invoke cleanup; else return newest data below t per the write column
6: **cleanup (any passerby), on a stale lock:** consult its primary: write record present $\Rightarrow$ roll *forward* (complete secondaries); primary lock still present $\Rightarrow$ atomically erase it (a binding abort), roll back debris

The primary's cell is a public court of record; limbo becomes a chore any passerby may finish. Timestamps from a single replicated oracle — agreement on "the next number" needs a till, not a parliament.

The passerby's rite, and the fate of the theorem. A Percolator client is mortal; it may faint at any point, leaving locks strewn like dropped vows. The difference from Part Three: *the evidence is not private*. The lock names the primary; the primary's cell holds the truth, readable by anyone; single-row atomicity makes consultation and cleanup safe. Commit record present at the primary: the marriage was pronounced — the finder rolls the dead transaction's writes forward and proceeds. Primary still locked: no pronouncement ever happened — the finder erases the primary lock atomically (the erasure *is* a binding abort, inscribed in the same court, so no late-waking client can pronounce afterward), rolls the debris back, and proceeds. This is the *passerby's rite* — **lazy resolution**: any reader completes or annuls a dead client's transaction. The in-doubt window has not vanished — fate still hangs in a single cell — but the *waiting* has become *work any passerby may finish*: limbo shrunk from a vigil to a chore.

The price, and the temperament. Every conflict is discovered by *encounter* rather than by announcement; cleanup rides on innocent bystanders' latency; and the edifice leans on single-row atomicity the way Part Two leaned on forced writes — the sacrament relocated, never removed. Percolator kept a web index fresh, where fleet throughput outranked any one transaction's latency and a stray hour-old lock costs nobody a customer; transplant it unmodified into a checkout page and the bet inverts. Transplant its *pattern* — the public court of record, the anointed primary, the passerby's right of completion — and it improves nearly any ledger it touches. Patterns travel; temperaments do not.

8.7 Part Seven — Calvin: Abolishing the Conversation

The third verb is *renegotiate*, and it has two very different practitioners. The first renegotiates the *machinery*: Calvin (Thomson and Abadi, 2012). Transactions quarrel — deadlock, block, abort, retry — because each node discovers a transaction's demands *during* execution. Calvin's answer: remove the discovery. *Agree on inputs, not outcomes.*

The mechanism.

- **Sequence first:** all transactions, before touching any datum, pass through a global sequencing layer — Chapter 5's replicated log — which batches them and stamps a single, total, replicated order. The layer is itself sharded and batched — parliaments agreeing on batch boundaries, not on each transaction singly — which is how one log avoids becoming the estate's narrowest door.
- **Execute deterministically:** every replica runs the same transactions in the same order; each scheduler grants locks strictly in the published order, so the interleaving is identical across replicas by construction — and deadlock dies of the same medicine, a fixed acquisition order admitting no cycles. Replication comes free and exact: ship the input log, not the effects — Chapter 5's determinism diet, served as the entire cuisine.
- **The consequence:** two-phase commit is not defeated but *mooted* — nothing to vote on; a distributed transaction is a scheduled performance of a score already published. The blocking theorem is not overturned but *starved*: no vetoes are surrendered mid-flight because no mid-flight decisions remain.

The price at the door. Determinism demands the read and write sets be knowable *before* sequencing — the score must list the instruments. A transaction that must read the database to learn its footprint runs a *reconnaissance pass* — once, without commitment — and is resubmitted, declared, with a re-check in case the world moved. The interactive transaction — begin; read; think in application code; write; commit — is surrendered entirely, because the *thinking* is not in the score. Calvin serves estates whose transactions arrive as complete stored procedures, and serves them magnificently: no distributed deadlock, no in-doubt windows, replication free out of the sequencer. The cleanest deal in the building, and the narrowest. One aside for the season's finale: *agree on inputs, then compute deterministically everywhere* is also, to the letter, how ten thousand accelerators share one optimizer — Calvin is a database wearing the synchronous training loop's constitution.

8.8 Part Eight — Sagas: Honest Abandonment

The second renegotiator renegotiates the *promise itself*: sagas (Garcia-Molina and Salem, 1987), written for long-lived transactions that would hold locks for hours, rediscovered wholesale by the microservices era, where no service will surrender its veto to another's coordinator. The bargain, without cosmetics: break the long transaction into the *chain and apologies* — a sequence of steps, each a local transaction atomic in its own house, and for each step, written in advance, a *compensation*: a second local transaction that undoes the step's business effect. Run the chain forward; on failure, run the completed steps' compensations in reverse. Atomicity is re-created at the business level — eventually, all of it or none of it, in effect — but isolation is abandoned, and abandoned *honestly*: between a step and any compensation that may later erase it, the intermediate state is visible to the world. No photograph, no snapshot, no lock spans the chain; the aunt may glimpse a booking that will never finally exist.

Definition 8.8 (saga; compensation; pivot)**.** A saga is a sequence $T_1, \ldots, T_n$ of local transactions with compensations $C_1, \ldots, C_{n-1}$, where C_i is a local transaction that reverses the *business effect* of T_i. Lawful executions: $T_1 \cdots T_n$ (success), or $T_1 \cdots T_k\, C_k \cdots C_1$ for some $k < n$ (failure and retreat). A *pivot* decomposition orders the chain so every step before the pivot is compensable, the pivot is the single uncompensable step, and every step after it is retriable to success (forward recovery only).

What survives and what does not, the skeleton first: compensations restore the business state along any failure prefix, but visibility of interim states is not restorable — the anomaly is exhibited, not hidden.

Proposition 8.5 (sagas: what survives, what does not)**.** *(i) Under Def. 8.8, every failure prefix is extended to a terminal state whose business effect equals either the full chain's or the empty chain's — atomicity in effect. (ii) Isolation does not survive: there are executions in which a concurrent transaction reads the state after T_i and acts on it, after which C_i runs; the observer's effects rest on retracted business, and no compensation reaches them (Exhibit 8.4). A compensation is a new forward transaction, not an undo of history.*

Proof of Proposition 8.5. (i) By induction on the failure point k: each C_i is assumed a total local transaction reversing T_i's business effect, and the reversed composition $C_k \circ \cdots \circ C_1$ applied to the state after $T_1 \cdots T_k$ yields the business effect of the empty chain; retriable steps after a pivot terminate by assumption, yielding the full chain's effect. (ii) Exhibit 8.4's interleaving: observer O runs entirely between T_i's commit and C_i's commit, reading T_i's output — lawful, since steps are ordinary committed local transactions and nothing spans them. O's own writes commit before C_i runs; C_i touches only the saga's state. The final state contains O's effects derived from business the chain has retracted; no lawful execution of $\{\text{saga absent}\} \cup \{O\}$ produces it. The anomaly is therefore real and unremovable without adding cross-step machinery — semantic locks, fences, or pivot placement — which is design, not protocol. □

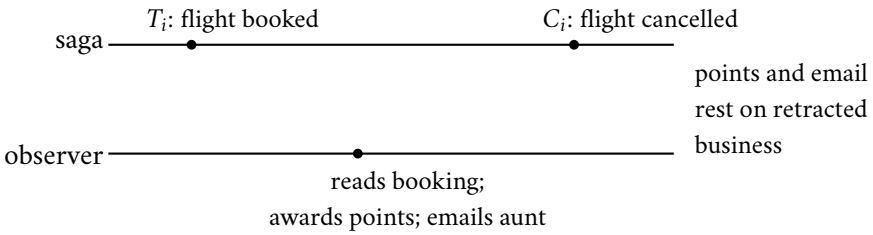


Exhibit 8.4 (the dirty read of half-done business). The observer runs entirely between step and compensation — lawfully, for nothing spans them (Prop. 8.5). A compensation is a new transaction, not an undo: it cannot reach the observer's effects. Fences (states marked provisional) and pivot placement are design remedies, not protocol ones.

The hazard, staged. The two doctors' shabbier cousin, walking through production daily; the episode stages it in full — the skeleton, beside Exhibit 8.4:

- **Step one commits:** Bertie's travel saga books the flight — committed, visible — and proceeds toward the hotel.
- **A third party acts:** another process *read the booked flight and acted* — awarded loyalty points, emailed the aunt the itinerary, sold cancellation insurance on the strength of confirmed travel.
- **The retreat:** the hotel refuses; the compensation cancels the flight — it cannot cancel the *reads*. The points ledger, the aunt's expectations, and the insurance book now rest on business that never happened: a dirty read of half-done business, made durable by third parties.

The discipline. What separates adult saga engineering from abdication with paperwork:

- **Compensations designed with their steps,** not discovered after the outage.
- **Interim states fenced:** marked provisional where outsiders might read them — a booking in state pending-saga is a fence, and cheap.
- **Uncompensable steps ordered last:** the email sent, the parcel shipped, the cash dispensed — pivots beyond which the saga only rolls forward.
- **The anomaly budget written down,** in prose, where the product manager signs: *these* interim states may be seen, by *these* observers, at *this* cost. A team that says "we use sagas" and cannot produce that page is practising not architecture but optimism with a vocabulary.

Two distinctions, rescued from the slide decks.

- **Backward versus forward recovery:** compensating backward is one lawful response to a failed step; the other is to press forward — retry until success, or substitute an equivalent — and a well-built saga declares, per step, which direction it takes, because some business processes must not run backward once begun.
- **The pivot** — the *point of no return*, formalized in Def. 8.8: compensable before, retriable after, the pivot itself the single moment of commitment. In Bertie's itinerary the pivot is the card charge: refundable bookings before it, the non-refundable ticketing and the aunt's email strictly after (Problem 8.8 designs the chain and convicts it under interleaving).

One operational schism: **orchestration** — a single conductor process driving the chain, the coordinator reborn at the business layer, with the same funeral to plan — versus **choreography**, each service reacting to the previous one's events, which distributes the conductor and thereby the confusion. Either can be built well; only one can be *read*, and outages are read at three in the morning.

The triage, one breath per verb. The cheapest distributed transaction is the one your schema made unnecessary — co-locate what commits together; where things live is Chapter 9's entire business, and this chapter issues its debt accordingly. Failing that: if your transactions can be declared, schedule them — Calvin's deal. If your invariants span shards and your purse is deep, buy the physics — Spanner's deal. If your steps are loosely coupled business anyway, compensate honestly — the saga's deal, page and all. And if you must hold the classical ceremony, hold it on parliaments, keep the windows short, and plan the officiant's funeral before the wedding.

8.9 The Whiteboard

For the design review.

1. Two-phase commit blocks by theorem, not by bug: plan the coordinator's funeral before the wedding — replicated coordinator state, cooperative termination, alarms on in-doubt age. Heuristic abort after a yes is a split-ledger generator with a reassuring name; the in-doubt console is the theorem's embassy in the product — staff it.
2. Strict serializability across shards is purchased (bounded uncertainty, confessed as an interval, one wait per commit), scheduled (determinism; the interactive transaction surrendered), or renounced (sagas; the apology plan in writing — compensations designed with their steps, pivots ordered last, interim visibility budgeted and signed).
3. The recovery matrix is the protocol; the happy path is the trivial part. Ask of any transactional system: where is the court of record, who may read it, and who may complete a stranger's ceremony?
4. Atomicity and isolation live in separate pockets: sagas abandon only the latter; snapshot stores relax isolation while committing atomically; "distributed transactions" in a brochure names neither. Ask for the anomaly, not the adjective.

The register. Paid, two entries: #5 — physical clocks, issued the moment Chapter 2 convicted wall clocks of ordering nothing: returned armed (TrueTime's confessed interval) and expensive (caesium in the data-centres; one uncertainty of latency per commit) — the invoice denominated exactly as promised. #18 — serializability across shards, issued only a chapter ago: two-phase commit over parliaments, with commit-wait standing guard over the clock. Issued: #20 — everything this chapter assumed the shards were simply *there*; where things live is a load-bearing decision this chapter never takes, and it is Chapter 9's entire business. Episode word count: 10,166 — above the floor; the amended corrective (§5 of the LEDGER: audit immediately after drafting) was applied and caught the shortfall while the material was warm.

8.10 Problems

Tier I — Calisthenics

Problem 8.1 (the widow's rights)**.** For each cut of Exhibit 8.1's transcript, state every party's recovery action per Box 18 on waking, and whether locks are held meanwhile: (a) after Bingo sends prepares, before any

vote; (b) after Tuppy forces YES, before Gussie receives prepare; (c) after both votes arrive, before the commit is forced; (d) after the commit is forced, before any announcement; (e) after Tuppy applies, before Gussie hears.

Problem 8.2 (presumed-abort bookkeeping)**.** Under presumed abort: (a) list the forced writes in a committed transaction and in an aborted one, for coordinator and for each participant; (b) explain why the coordinator may forget a transaction the moment all commit acknowledgments arrive, and what it answers, forever after, to a late query about it; (c) show the convention is unsound if a coordinator may answer queries *before* its abort decision is final — construct the two-answers execution.

Problem 8.3 (the third word)**.** A transaction reads at ten branches and writes at one. (a) Count forced writes and message rounds with and without the read-only vote. (b) The lone writer crashes after voting YES; the readers have all exited. Who is in doubt, and who is not, and why is the asymmetry harmless? (c) A reader, after replying READ-ONLY, is asked by an in-doubt fellow for the outcome. What can it truthfully say?

Problem 8.4 (commit-wait arithmetic)**.** TrueTime returns $[e, l]$ with the confessed uncertainty $\varepsilon = l - e$ at four milliseconds. (a) A transaction calls now at true time ten o'clock exactly and receives a centred interval; compute its stamp and the earliest true time its announcement may occur. (b) A second transaction starts one millisecond after the announcement; bound its stamp from below. (c) Uncertainty doubles to eight milliseconds during a time-master outage; restate the commit latency and name who pays it. (d) State in one sentence why the stamp is the *latest* edge and not the midpoint.

Problem 8.5 (name the verb)**.** Classify each design as relocate, shrink, or renegotiate, and name the residual limbo: (a) 2PC with coordinator state in a Raft group; (b) Percolator; (c) Calvin; (d) a saga with orchestrator; (e) Spanner's cross-shard commit.

Tier II — The Real Thing

Problem 8.6 (sabotage: the heuristic fleet)**.** A middleware vendor ships participants that *abort on timeout* after a YES vote ("to protect availability"), defaulting to thirty seconds. Exhibit the fatal execution: a complete message-and-crash schedule in which one participant heuristically aborts while another commits the same transaction, ending with the split ledger — Bertie debited, the aunt uncredited, both banks' books internally consistent and mutually irreconcilable. State the exact line of Box 18 the configuration violates, and which clause of Def. 8.1 dies. Then argue (three sentences) why no timeout value fixes it.

Problem 8.7 (3PC's split verdict)**.** Construct Prop. 8.3(ii)'s execution: five participants, coordinator crashed after disseminating pre-commit to two of them, network partitioned so those two are on one shore and the uncertain three on the other. Apply Def. 8.5's termination rule on each shore; exhibit the two verdicts; then state which assumption of Prop. 8.3(i) each shore violated, and why requiring a majority to proceed (the quorum repair) is tantamount to a parliament.

Problem 8.8 (the travel saga, designed and convicted)**.** Design the booking saga: reserve flight (compensable), reserve hotel (compensable), charge card (pivot), issue ticket (retriable), notify (retriable). (a) Write each compensation and mark forward- versus backward-recovering steps. (b) Exhibit the interleaving in which a concurrent budget report reads the card charge that a later retreat refunds; name the anomaly in Chapter 7's vocabulary. (c) Add the provisional-state fence and show which observers it protects and which it cannot. (d) Reorder the chain to put the ticket issue before the card charge and exhibit the strictly worse outcome, proving the pivot placement was load-bearing.

Problem 8.9 (the passerby's rite, under fire)**.** A Percolator client crashes (i) after prewriting one secondary, (ii) after prewriting all rows, (iii) after the primary commit cell-write, (iv) midway through secondary roll-forward. For each: what does a reader arriving at a locked secondary conclude from the primary, what does it do, and why can two concurrent passersby not resolve the transaction differently? Cite the single-row atomicity each step leans on.

Tier III — For the Ambitious (starred)

Problem 8.10 (★ the HLC bound)**.** For Def. 8.7: (a) prove the clock condition ($a \to b$ implies $\text{stamp}(a) < \text{stamp}(b)$ lexicographically); (b) prove pt never exceeds the largest wall reading yet issued anywhere, and $pt \geq$ the local wall reading, so $|pt - \text{wall}|$ is bounded by the clock error; (c) prove the counter bound: c at any event is at most the number of events inside one clock-error window of causally connected processes, and exhibit the adversarial schedule approaching it. *(Hints only.)*

Problem 8.11 (★ the founders' wedding)**.** Reconstruct Gray–Lamport Paxos Commit from its idea: one Paxos instance per participant's vote, commit iff every instance decides YES. (a) Specify the acceptors' role and how a new coordinator concludes an instance whose participant is dead. (b) Show AC1–AC3 hold and blocking is confined to minority-loss of acceptors. (c) Count messages and delays against Box 18 with a replicated coordinator, and state when the purists' edition is worth its postage. *(Hints only.)*

8.11 Hints

Hint (Problem 8.2)**.** (c) Let the coordinator answer "abort" from the presumption while its own patience timer still runs, then receive the last YES and force commit.

Hint (Problem 8.6)**.** Cut Exhibit 8.1 at (d): decision forced, announcements in flight. Deliver one, delay the other past thirty seconds. For the last part: the doppelgänger does not read clocks — any finite patience is silence in world one.

Hint (Problem 8.7)**.** The pre-commit shore commits by rule three; the uncertain shore aborts by rule two. Each rule's soundness argument quantifies over *all* participants; the partition breaks the quantifier, not the rule.

Hint (Problem 8.10)**.** (b) Induction on events: every assignment to pt is a max of wall readings and prior pts. (c) c resets whenever honest wall time overtakes; it can climb only while pt stands still, which the error bound confines to a window.

Hint (Problem 8.11)**.** (a) A new coordinator proposes NO in a dead participant's instance; Paxos's own safety arbitrates against a late YES. (b) AC1 reduces to each instance's consensus safety plus the deterministic conjunction.

8.12 Solutions

Tier I in full (tersely); Tier II in full — Problem 8.6 is the sabotage certification; hints only for Tier III.

Solution (to Problem 8.1)**.** (a) Nobody voted: participants abort from *working*; recovered coordinator finds no decision, presumes abort. No locks outlive the crash. (b) Tuppy in doubt (locks held, queries); Gussie aborts from working and answers any cooperative query NO-and-aborted, which releases Tuppy: cooperative termination at its best. (c) As (b) for both participants — votes received but no decision forced

means recovery presumes abort; both participants eventually hear it. (d) The decision exists: recovered coordinator re-announces commit; both participants are in doubt until then — this is the blocking window with the shortest fuse. (e) Tuppy is done; Gussie in doubt; coordinator log answers commit on query or re-announcement; cooperative query to Tuppy also answers it.

Solution (to Problem 8.2)**.** (a) Committed: each participant forces vote and commit record; coordinator forces commit (and an end record when acks complete). Aborted: participants that voted force the vote and an abort record; the coordinator forces *nothing*. (b) All acks in hand means no participant will ever ask again; the presumption answers any archaeologist: no record, hence aborted — which is false for this committed transaction, and harmless, because the end record is written before forgetting and no querent remains. (c) With answers permitted before finality: query arrives (answered "abort" by presumption), last YES arrives, commit is forced, second querent hears "commit" — AC1 dead. Presumption is sound only about the past of a *final* log.

Solution (to Problem 8.3)**.** (a) Without: eleven votes forced, eleven decision applications. With: one forced vote (the writer), ten early exits; message rounds unchanged in count but the decision fan-out shrinks to one. (b) Only the writer is in doubt; the readers hold no locks and made no promises — the asymmetry is harmless because AC1 quantifies over deciders, and the readers' outcome is identical under either verdict by construction. (c) Only "I exited read-only; either outcome is consistent with my state" — it holds no evidence, which is exactly why its early exit was lawful.

Solution (to Problem 8.4)**.** (a) Centred: the interval runs from two milliseconds before ten o'clock to two milliseconds after; stamp ten o'clock plus two milliseconds; announcement no earlier than the true instant at which some call's earliest edge exceeds the stamp — at worst-case drift, roughly ten o'clock plus four milliseconds: about one ε after the call. (b) The second transaction's stamp is a latest edge taken after its start: strictly above ten o'clock plus four milliseconds, hence above the first stamp. (c) Commit latency grows to roughly eight milliseconds; every writer pays; readers at old stamps pay nothing. (d) The midpoint may lie in some clock's future; the latest edge, by the axiom, lies in nobody's past at call time and can be waited into everybody's past.

Solution (to Problem 8.5)**.** (a) Relocate; limbo shrinks to a Raft election plus outstanding in-doubt queries. (b) Shrink; limbo becomes the passerby's chore, bounded by encounter time. (c) Renegotiate (the machinery): no mid-flight votes exist; residual stall is the sequencer's own consensus. (d) Renegotiate (the promise): limbo is gone but isolation went with it; the orchestrator's death revives the funeral problem one layer up. (e) Relocate plus purchase: by-election limbo plus one uncertainty of commit-wait per transaction.

Solution (to Problem 8.6 — the certification)**.** The schedule, against Box 18. Participants Tuppy and Gussie, coordinator Bingo; the vendor's thirty-second fuse armed in both participants.

(1) Prepares delivered; both force YES; both fuses start.

(2) Bingo receives both votes, *forces commit* (Box 18 line 6): the decision exists in the court of record.

(3) Bingo emits the two commit letters and crashes. The letter to Gussie arrives at once: Gussie commits, releases locks, credits the aunt — permanently (AC3).

(4) The letter to Tuppy is delayed thirty-one seconds — lawful weather under the Chapter 1 charter. At thirty, Tuppy's fuse fires: he "heuristically aborts," rolls back the staged debit, releases locks, reports failure to Bertie.

(5) Bingo recovers, reads commit, re-announces. Tuppy has already acted; his abort is applied and, per AC3 on his own ledger, he will not now commit. Final state: credit committed at Gussie, debit aborted at Tuppy — ten pounds minted; the banks reconcile never.

The violated text: Box 18 line 12 — a prepared participant's only lawful acts are query and wait; the configuration substituted a decision. The dead clause: AC1 (two processes decided differently); AC2 is also complicit, since Tuppy's ABORT contradicts the unanimous-yes commit in the record. Why no timeout value repairs it: the schedule delays the letter to *fuse plus one second*, whatever the fuse; Thm. 8.2's worlds are indistinguishable at every finite patience, because silence carries no signature — a longer fuse changes the incident's date, not its existence. Certified fatal: money created, AC1 dead, under one crash and one slow letter, both documentary-ordinary.

Solution (to Problem 8.7)**.** The coordinator disseminates pre-commit to P_1, P_2, then crashes; a partition isolates $\{P_1, P_2\}$ from $\{P_3, P_4, P_5\}$ (prepared, no pre-commit). Left shore termination: all reachable in pre-commit, none committed ⇒ complete the commit (rule three). Right shore: some reachable uncertain ⇒ abort (rule two, whose soundness argued "no absent party can have committed" — true against crashes, false against absence-by-partition, since the absent left shore held pre-commits and is now committing). Heal the cable: AC1 dead. Each shore's rule quantified over "all participants" and silently substituted "all reachable." The quorum repair — neither shore moves without a majority — restores AC1 by making one shore wait, i.e., block: non-blocking was the entire purchase, so the repaired protocol is a parliament with an extra phase, and Part Five's settlement (drop the phase, keep the parliament) follows.

Solution (to Problem 8.8)**.** (a) Cancel-flight, cancel-hotel (backward, before the pivot); charge is the pivot (uncompensable by policy: refunds exist but the business declares the charge the commitment point); issue ticket and notify retriable (forward). (b) Budget report reads the charge after the pivot— no: place the report between charge and a failed ticket issue that forces manual retreat (the pivot's "retriable" promise broken by an operator override): the report now counts revenue the refund retracts — Chapter 7's vocabulary: a dirty read made durable by a third party; strictly, a read of state later compensated, which no isolation level that spans only single transactions even names. (c) The fence (charge marked provisional-until-ticketed) protects observers that honour the flag — the budget report can exclude provisional rows; it cannot protect the aunt's email or any external side effect already emitted. (d) Ticket before charge: a crash between them yields a ticketed, unpaid journey, and the "compensation" is now revoking a delivered external good — strictly worse than refunding money, which is why the uncompensable step is *defined* to be the pivot and everything after it must be retriable, never revocable.

Solution (to Problem 8.9)**.** (i) Primary lock present, no write record: erase primary lock atomically (binding abort), roll back the secondary; a racing passerby attempting the same finds the erase idempotent, and a late-waking client's commit attempt fails at the missing primary lock — the single-row exchange arbitrates. (ii) Same as (i): prewrites complete but no pronouncement; abort is lawful and, once the primary lock is erased, final. (iii) Write record present at primary: roll *forward* — inscribe the secondary's write record; concurrent passersby perform idempotent inscriptions; nobody may abort, since the primary lock no longer exists to erase. (iv) As (iii); the remaining secondaries are completed by whoever arrives. In every case the primary cell's single-row atomicity makes the pronouncement (or its annulment) a unique, linearizable event — Chapter 7's language earning its keep in a column.

8.13 The Reading

Corbett et al., "Spanner: Google's Globally-Distributed Database," OSDI 2012. The course's single most load-bearing industrial text. Read the TrueTime section twice — once for the API's honesty, once for commit-wait's arithmetic against Thm. 8.4 — and notice the season's earlier chapters on every page: Paxos groups, leases, snapshots, locks.

Garcia-Molina and Salem, "Sagas," SIGMOD 1987. Short, clear, decades ahead of the architecture that would need it. Read the compensation discussion with your own outage postmortems open beside it, and note how much of Prop. 8.5(ii) the authors already knew.

Gray, "Notes on Data Base Operating Systems," 1978. The ceremony and the in-doubt state in the founder's plain hand; the recovery discussion is Box 18's ancestor and still its best commentary.

Gray and Lamport, "Consensus on Transaction Commit," TODS 2006. The ledger and the parliament formally wed by their founders; Paxos Commit (Problem 8.11) with the message-count comparison done honestly.

The shelf. Skeen's three-phase paper (1981), as the museum label. Peng and Dabek, "Large-scale Incremental Processing Using Distributed Transactions and Notifications" (OSDI 2010) — Percolator; Box 20 against §2. Thomson, Diamond, Weng, Ren, Shao, Abadi, "Calvin" (SIGMOD 2012) — the determinist bargain. Kulkarni, Demirbas, Madeppa, Avva, Leone, "Logical Physical Clocks" (OPODIS 2014) — HLC and Problem 8.10's bounds. Bernstein, Hadzilacos, and Goodman, *Concurrency Control and Recovery in Database Systems* (1987), chapter seven — the recovery matrix and presumed abort/commit in full dress, free online, and the place to check Box 18's every line.

Chapter 9

Partitioning, or Where Things Live

Companion to Episode Nine. The episode tells the story — the cache fleet dead of arithmetic at four in the afternoon, the clock face built by hand, the aunt's notice taken up by the national press; this chapter keeps the record: the resignation letter that hash modulo N writes to itself; hash against range and the compound-key treaty; the ring, constructed and proved in full — balance, movement, the one-dart lottery, the virtual-node repair; the rendezvous auction; Chord, and the road the industry actually took; the two churches of the shard map; rebalancing without storms; routing epochs, the fencing token's civilian cousin; and the survival kit for celebrity. Friendly territory, as the constitution instructs me to flag it: the mathematics is coin-flipping throughout, and the sabotage is, as ever, certified fatal in the solutions.

9.1 Part One — Why Shard At All

The episode opens on a cache fleet dying of arithmetic: ten servers, keys placed by hash-the-key-take-the-remainder, an eleventh machine added at four in the afternoon — and the divisor's change reshuffles roughly ten addresses in eleven at a stroke, sending the database beneath the entire internet at once. No machine failed; the arithmetic did. Everything in this chapter is the repair and its consequences.

The resignation letter. *Hash modulo N is a resignation letter one writes to oneself in advance, postdated to the day the fleet changes size*; Problem 9.3 audits the exact cost, a moved fraction of $N/(N{+}1)$ against the ring's $1/(N{+}1)$. The repair had been published in advance: *consistent hashing* (Karger and colleagues, MIT, 1997), with two modest-sounding promises — each server holds about its fair share of the keys, and a membership change moves about one key in n. The authors founded Akamai on it; the technique escaped into storage, databases, and Chapter 6's Dynamo school; and the fix is always the same fix: never again shall the address of a key depend on a number that changes.

The three ceilings. Before any ring is drawn, the disciplined question: must we? The alternative is the bigger machine — a genuinely excellent answer for longer than fashion admits, and every theorem of this course is trivially satisfied by an estate of one. But the ceiling is real, and it is triple:

- **physics and the catalogue:** past some size the machine is not sold, and the one below it is priced as jewellery — the estates that define this field were built from warehouses of commodity boxes;
- **blast radius:** one machine is one failure domain, and Chapter 1's documentary evidence applies in full — availability was never about the mean but about the worst Tuesday;

- **geography:** a single machine is somewhere, and the speed of light bills every client who is elsewhere — Chapter 2's constant, unrepealed.

Outgrow any of the three and the data must be cut into *partitions* — shards, tablets, ranges, vnodes; the trade has a dozen words and one idea — each piece small enough to live on a machine, move between machines, and fail alone. The shard then becomes the **unit of everything**: of placement (the map speaks in shards, not keys), of movement (rebalancing ships shards whole), of recovery (a dead machine's obligations re-homed shard by shard, in parallel), of throttling, quota, billing, and blame. Too large, and every operation is a lumbering indivisible; too small, and the map is the estate's biggest table; the trade's instinct is shards of modest, movable size, thousands per fleet, many per machine — every operational verb conjugates more gracefully in the plural.

Partitioning and replication. Different axes of one placement decision: replication (Chapter 6's business) makes *copies* of the same shard for durability and availability; partitioning cuts *different* pieces for capacity. Every serious estate does both at once, and the result is the *seating chart*: shards down one side, replicas across the top, each shard's replica group commonly a little parliament with its own log (Chapters 4 and 5), each machine hosting replicas of many shards, leading some, following others — under the quiet constraint that no shard's members share a rack, a switch, or a storm: replicas sharing a failure domain are Chapter 1's lesson purchased twice. Replication answers *what survives*; partitioning answers *what fits and where*; and no amount of replication rescues a scheme that put all the popular keys in one place. Chapter 8's parting debt names this chapter's obligation: co-locate two accounts and the transfer is one shard's local transaction; separate them and the entire fainting-clergy apparatus deploys. The cheapest distributed transaction is the one the schema made unnecessary. This chapter, somebody decides.

The economic frame. A partitioning scheme is the choice of which fan-out you pay. Cut the data one way and reads scatter across every shard while writes stay local; cut it another and writes fan out while reads stay put; rebalance eagerly and the movement itself becomes load. The trilemma appears in four costumes — hash against range, local against global indexes, eager against lazy balancing, pull against push routing — and in every costume the resolution is identical: no scheme makes reads, writes, and rebalances all free; there is only the scheme whose expensive operation is the one the business performs least.

9.2 Part Two — Hash Versus Range

Two great schemes divide the world, each best introduced by the pathology of the other; Part One's fan-out sentence takes its first concrete costume here.

Partition by range. Order the keys and cut the ordered line into contiguous stretches — the encyclopaedia, spines labelled with their boundaries. The virtue is *locality*: neighbours in the order are neighbours in the estate, so a scan — every account opened in March — touches one shard or a few adjacent ones, not the fleet. The boundaries need not be uniform, and must not be: the volumes are cut where the content is dense, and the boundaries are therefore *state* — somebody keeps the list of spines, a fact that matters in Part Five. The characteristic pathology follows from the virtue; call it the **monotonic-key dogpile**: let the key be a timestamp or an ever-increasing order number, and every write lands in the newest range, on one shard, always — and on the busiest day of the year, since monotonic keys and seasonal peaks arrive together; the Christmas post, addressed by construction to one door. A hundred machines, one at work; when the range fills and splits, the load moves wholesale to the new last shard, tomorrow. The remedies — salting the key

with a prefix, splitting the tail eagerly, writing to the day's shard round-robin — are all confessions that the pure order was unaffordable.

Partition by hash. Feed the key through a good hash and place by the value: identity scrambled into uniform anonymity. The virtue is *uniformity*: a decent hash spreads any workload's keys evenly, monotonic or not, and the dogpile dissolves. The price is the virtue inverted: locality is destroyed, and the scan that touched one shard now touches every shard — scatter the query, gather the results, the fan-out paid on every read that asks for a stretch. Hash is the scheme of point lookups — carts, sessions, profiles, caches — and Chapter 6's Dynamo school is hash-partitioned to a member: a cart is the point lookup made flesh. And no uniformity cures celebrity: the hash spreads *keys*, not *popularity* — one key requested a million times an hour hashes to one place. Part Seven's business.

Three footnotes of craft, each a scar.

- **Pin the placement hash.** The office asks for uniformity and stability, not secrecy — the standard tenant is a fast non-cryptographic hash — and the one non-negotiable property is that the function *never change*: a fleet whose upgraded hash library disagrees with the old about where everything lives has re-sent itself the resignation letter through a dependency file. The placement hash has tenure.
- **The compound key,** the standing armistice: hash the outer component, range the inner — hash the customer, order by time within the customer. Point lookups and per-customer scans land on one shard; the dogpile dissolves across a thousand customer arcs; only the estate-wide scan pays scatter-gather. Most industrial schemas are precisely this treaty, signed per table.
- **Skew.** Uniformity claims are claims about *keys in expectation*, and businesses are not expectations — one tenant a hundred times the median, one county holding half the population. The schemes bound what arithmetic can bound; the residue is Part Seven's celebrity.

The catechism per table is mercifully short: asked for in stretches — range, and manage the tail; fetched only by exact name — hash, and never think about it again; honestly both — the compound key, or two tables and a pipeline. The remaining machinery — rings, directories, rebalancers — operates beneath either scheme: given the cut, which machine holds which piece, and how does that answer survive the fleet changing size?

9.3 Part Three — The Clock Face

The set piece, proved in full. Bend the hash space into a circle — a clock face marked not with hours but with hash values, wrapping at the maximum, normalized at the desk to the unit circle $[0, 1)$ — and place the *servers* on it by hashing their names: Bingo at one o'clock, Tuppy at five, Gussie at nine. The placement rule is one sentence: **hash the key onto the same ring, then walk clockwise; the first server you meet is home.** Formally the walk is the *successor* function — the first server position at or after the key's point, modulo one — and each server owns the *arc* its position closes: its key tenancy. Every key has exactly one home, computable by any party who knows the roster — no table, no negotiation, no message. The model beneath every probabilistic claim, fixed first:

Model of this chapter

For the probabilistic results (Thms. 9.2–9.5, Prop. 9.4). Hash values are modelled as independent uniform random points on the unit circle $[0, 1)$ — the standard idealization of a good hash; keys

and server names hash independently; the workload's keys are not chosen adversarially against the hash. Load is counted in keys (or, equivalently, in key-value mass under the same independence). All results are exact distributional statements under this model; the engineering caveats (skewed *popularity*, adversarial keys) are handled in the text where they arise — the model covers where keys *live*, not how often they are visited.

For routing and rebalancing (Prop. 9.6, Problems 9.7, 9.8). Chapter 1's asynchronous network; crash-stop servers; a shard map with a monotonically increasing version (epoch), administered either by a directory (Chapter 5 kernel, leases and watches) or by gossip. Clients may cache maps of any staleness.

The definition gathers the construction, the aliases of the variance repair below, and the replication walk in one place.

Definition 9.1 (the ring; consistent hashing)**.** Fix independent uniform positions on the unit circle for m *tokens*; each of n servers owns k tokens (its *virtual nodes*, or *aliases*; $m = nk$), the single-alias scheme being $k = 1$. A key hashes to a uniform point and is placed at its *successor*: the first token clockwise from the key's point, wrapping at one. A server's *share* is the total length of the arcs its tokens close; by uniformity of keys, share equals expected fraction of keys held. Replication walks onward: a key's R replicas are the first R tokens belonging to distinct servers (industrially: distinct failure domains) met clockwise.

The episode walks three tenants to their lodgings; the skeleton, against Exhibit 9.1:

- **the umbrella ledger,** hashing to half past two (the key's *name*, mind, nothing about its size or worth): clockwise, first doorstep is Tuppy's at five;
- **the aunt's flower-committee notice,** hashing to twenty past nine, just beyond Gussie: past ten, past eleven, wrap the dial — Bingo at one takes her in;
- **Bertie's cart,** hashing to four: Tuppy again — the hash neither knows nor cares that Tuppy already has the ledger; uniformity is a statistical promise, not a courtesy call.

Three keys, three walks, no conversation with anybody — the same walk at three keys or three billion.

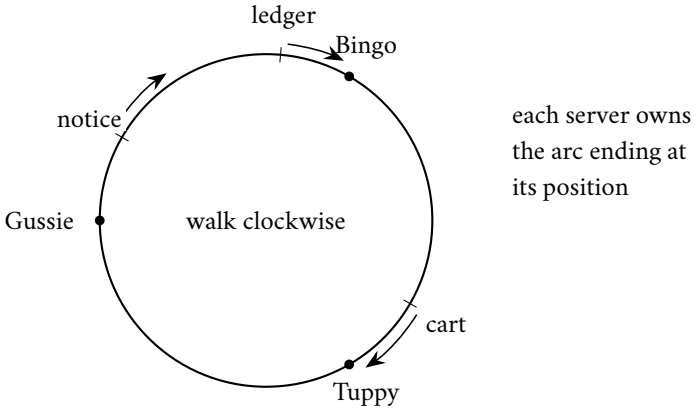


Exhibit 9.1 (the clock face). Servers and keys hash to the circle; a key walks clockwise (arrows) to its successor. Clockwise here runs toward decreasing angle; the ledger and the cart reach Bingo and Tuppy, the notice walks to Gussie's successor. No table, no negotiation: the roster is the map (Def. 9.1).

The wonder. Barmy joins, landing at three o'clock in the middle of Tuppy's arc: the keys between Bingo at one and Barmy at three — formerly walking on to Tuppy — now meet Barmy first; they, and *only* they, change address, every other tenancy on the dial untouched, unaware. **Add a server, move an expected** $1/(n{+}1)$**; remove a server, and only its tenants move:** $1/n$ — the celebrated "one over n," Theorem 9.3.

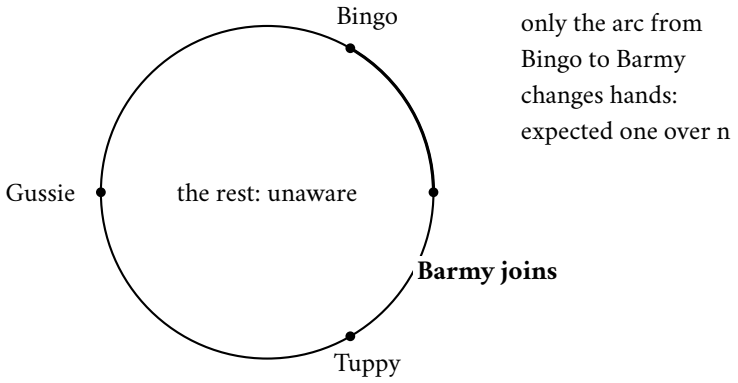


Exhibit 9.2 (one over n). Barmy lands at three o'clock; the keys between Bingo's position and his — the thickened arc, formerly walking on to Tuppy — now stop at Barmy. Nothing else moves (Thm. 9.3); compare modulo N, which reshuffles $n{-}1$ keys in n on the same event.

Departure, planned and unplanned. The decommissioning: the departed's tenants walk clockwise past the vacated position to the next doorstep — one inheritor, one data stream, throttled at leisure. The storm: the ring's marriage to replication pays off — the arc was never held alone but by the walk's next distinct machines as replicas, so a death moves no data at all in the first instant; the ring simply stops routing to a dead position, and replacement copies are rebuilt in the background at the roadworks budget (Part Six). Placement schemes are judged at four in the afternoon, but they are *designed* for the storm at nine.

The proofs now, friendly as advertised: uniform points, symmetry, one union bound, and the Gamma trick, nothing sharper. The skeleton in words: arc lengths of uniform points are exchangeable spacings, so a server's share is a Beta variable with mean one over n; a key changes home only if the newcomer lands between the key and its old successor, so exactly the newcomer's arcs move.

Lemma 9.1 (spacings)**.** *Let m independent uniform points partition the circle into m spacings $S_1, \dots, S_m$ (the spacing S_i closed by token i). The vector $(S_1, \dots, S_m)$ is exchangeable, and the sum of any k designated spacings has the Beta distribution with parameters $(k, m-k)$; in particular each S_i has mean $1/m$, and the designated sum has mean k/m and variance*

$$\frac{k\,(m-k)}{m^2\,(m+1)}.$$

Proof of Lemma 9.1. Rotate the circle so token one sits at the origin; rotation leaves the joint law of the other $m-1$ uniform points unchanged, so the spacings are the gaps of $m-1$ iid uniforms on $[0,1)$ plus the wrap gap. For exchangeability and the Beta law, use the standard Gamma representation: let $G_1, \dots, G_m$ be iid exponential variables and $T = G_1 + \cdots + G_m$; the vector $(G_1/T, \dots, G_m/T)$ has exactly the law of the spacings (both are uniform on the simplex — the flat Dirichlet — a classical fact provable by computing the joint density of normalized exponentials). Exchangeability is now manifest. For a designated index set K of size k,

$$\sum_{i\in K} S_i \;\overset{d}{=}\; \frac{\sum_{i\in K} G_i}{\sum_{i\in K} G_i + \sum_{i\notin K} G_i} = \frac{A}{A+B},$$

with A a Gamma variable with shape k, B an independent Gamma with shape $m-k$; and $A/(A+B)$ is, by definition of the Beta family, Beta$(k, m-k)$. The mean k/m and the displayed variance are the standard Beta moments. □

Theorem 9.2 (balance)**.** *Under Def. 9.1, each server's share has mean $1/n$ exactly. With k aliases the share is* Beta$(k,\ (n-1)k)$*-distributed, with standard deviation*

$$\frac{1}{n}\sqrt{\frac{n-1}{nk+1}}\ \approx\ \frac{1}{n\sqrt{k}},$$

so the relative deviation from the fair share falls as $1/\sqrt{k}$: a hundred aliases confine typical imbalance to about ten percent, ten thousand to one percent. Weighted servers take aliases in proportion to capacity and hold, in expectation, exactly their weighted share.

Proof of Theorem 9.2. A server's share is the sum of the k spacings closed by its tokens: by Lemma 9.1 with $m = nk$, it is Beta$(k, (n-1)k)$ with mean $k/m = 1/n$ and variance $k(n-1)k/(m^2(m+1)) = (n-1)/(n^2(nk+1))$; the standard deviation displayed in the statement is its square root, and for large n it is $\approx 1/(n\sqrt{k})$, giving relative deviation $\approx 1/\sqrt{k}$. A server with weight w (taking wk aliases among total m') has share Beta$(wk, m'-wk)$, mean wk/m': exactly its weighted proportion. The numerical clauses: $k = 100$ gives relative deviation about one tenth; $k = 10^4$ about one hundredth. □

Theorem 9.3 (movement; monotonicity)**.** *Under Def. 9.1: (i)* Monotonicity. *When a server joins, a key changes home only if its new successor is a token of the newcomer; no key moves between two incumbent servers. (ii)* Join. *The expected fraction of keys that move when a k-alias newcomer joins n incumbent servers is exactly $1/(n+1)$. (iii)* Leave. *When a server departs, exactly its keys move, each to the next surviving token clockwise; expected fraction $1/n$. With aliases, the departing tenancy is inherited by up to k distinct successors (Problem 9.8 quantifies the single-alias lump).*

Proof of Theorem 9.3. (i) A key's home is its clockwise successor. Adding tokens can only change a key's successor *to one of the new tokens*: if the old successor were replaced by another incumbent token, that token would have been the closer successor already — incumbent positions did not move. Hence all movement is into the newcomer, and no incumbent-to- incumbent movement occurs. (ii) The moved keys are exactly those whose successor is now a newcomer token, i.e. the keys in the newcomer's arcs; the expected total length of those arcs is the newcomer's expected share among $n+1$ servers, which is $1/(n+1)$ by Theorem 9.2. (iii) Removing a server's tokens changes successors only for keys in its arcs (mean share $1/n$); each such key's new successor is the next surviving token clockwise, by definition. With $k = 1$ that entire arc lands on one survivor; with aliases, each sliver lands on its own clockwise neighbour, generically k distinct servers (Problem 9.8). □

The one-dart lottery. *Expected* share, and the adjective bears leaning on: three darts at a circle rarely cut fair arcs — a sliver here, a third of the dial there. The fair share holds *in expectation*; the guarantee that holds *with high probability* carries a logarithmic factor of slack — the maximum single-alias arc runs to $\Theta(\log n/n)$ — and the difference between those two phrases is precisely the machine that pages you.

Proposition 9.4 (the one-dart lottery)**.** *With m tokens in all, the largest single arc exceeds $c \ln m/m$ with probability at most m^{1-c} (any $c > 1$); and the expected largest arc is of order $\ln m/m$ (both bounds proved below, the lower direction sketched). With one alias per server this is the whole story of ring imbalance: the unluckiest server expects roughly a $\ln n$ multiple of the fair share — a lottery with logarithmic, not catastrophic, winners — and Thm. 9.2's aliases are the repair.*

Proof of Proposition 9.4. Upper bound. Fix $t \in (0,1)$. Token i's spacing exceeds t exactly when none of the other $m-1$ points lands in the arc of length t clockwise-preceding token i's position, an event of probability $(1-t)^{m-1}$. By the union bound,

$$\Pr\Big[\max_i S_i > t\Big] \le m(1-t)^{m-1} \le m\,e^{-t(m-1)}.$$

With $t = c\ln m/m$ this is at most $m \cdot m^{-c(m-1)/m} \le m^{1-c}\,e^{c\ln m/m}$, which is m^{1-c} up to a factor tending to one — negligible for any $c > 1$ and large m. *Lower direction (sketch, via the Gamma picture).* The spacings are normalized iid exponentials; the maximum of m iid exponentials has mean $H_m = 1 + \frac{1}{2} + \cdots + \frac{1}{m} \sim \ln m$, and the normalizer concentrates at m; hence the expected maximum spacing is $\sim \ln m/m$, and the upper bound is tight to its constant. (The desk records the sketch honestly: making the normalization rigorous is a page of routine concentration, assigned to nobody.) □

The alias repair. Worse than the steady-state lottery: when a single-alias server leaves, its *entire* arc lands on one successor — a lump of exactly the kind a fleet under stress cannot digest (Problem 9.8 quantifies it). The standing repair is the *aliases* — virtual nodes, k positions per server (Def. 9.1): k slivers scattered around the dial, lucky averaged against unlucky by the law of large numbers, the relative deviation falling as $1/\sqrt{k}$ (Theorem 9.2: a hundred aliases confine typical imbalance to about ten percent). The lump dissolves — a departing machine's hundred slivers fall to a hundred different clockwise successors, each survivor picking up a crumb — and heterogeneity comes free: twice the disk, twice the aliases, twice the estate in expectation. Chapter 6's Dynamo ran precisely this arrangement, its preference lists read off the walk — the home trio for a key is the first three distinct machines met clockwise: the ring placing, the walk replicating. Not free: the ring representation is k times longer, the successor bookkeeping k times busier, and a departing tenancy arrives as k small transfers to coordinate; the alias count is a dial, turned to the low hundreds.

Protocol — Box 21. The ring, industrial dress (checked against Karger et al. 1997 and Dynamo 2007 §4)

1: **state:** sorted list of (position, server) for all nk tokens; the placement hash is pinned forever
2: **place**(x): binary-search the first token at or after hash(x), wrapping; that token's server is home
3: **replicas**(x): continue clockwise; take the first R tokens of *distinct servers, distinct domains*
4: **join**(s): insert s's k tokens; stream to s exactly the keys in its new arcs (from each arc's old owner); expected $1/(n{+}1)$ of the estate
5: **leave**(s): delete s's tokens; each sliver's keys stream to its clockwise successor (planned) or are rebuilt from replicas (unplanned)
6: **weighting:** alias count proportional to capacity

Movement is confined by Thm. 9.3; balance by Thm. 9.2; the sorted list is the entire map — computable by any party from the roster, which is the scheme's lightness and, per the two churches, its price.

One refinement of the modern era deserves its sentence, because the pure ring caps nobody — a machine simply receives what its arcs receive.

Remark 9.2 (bounded loads). The pure ring caps nobody. The refinement of Mirrokni, Thorup, and Zadimoghaddam (2017) fixes a capacity factor $c > 1$, caps every server at c times the mean load, and lets a key whose successor is full continue clockwise to the first uncapped server; the paper proves both balance (no server exceeds the cap, by construction) and a movement bound that survives the forwarding. Adopted by the load-balancing trade within the year; the citation is in the reading.

The rendezvous auction. The elegant sibling: the same two promises, no ring at all. Hash the *pair* — key with each server's name — obtaining one score per server; the key lives with the highest bidder. That is the whole algorithm: rendezvous (highest-random-weight) hashing, Thaler and Ravishankar, mid-nineties, a whisker before the ring and long overshadowed by it.

Definition 9.3 (rendezvous hashing; Thaler–Ravishankar)**.** For key x and servers $s_1, \ldots, s_n$, compute scores $h(x, s_1), \ldots, h(x, s_n)$ — modelled as independent uniforms, fresh per (key, server) pair — and place x at the highest-scoring server. Weighted variant: transform each score so that server s wins with probability proportional to its weight (the standard transform is recorded in the reading); no ring, no state beyond the roster.

Theorem 9.5 (rendezvous properties)**.** *Under Def. 9.3: (i)* Exact balance: *each server is the winner for each key with probability exactly* $1/n$*, independently across keys. (ii)* Join: *when a server joins, key x moves iff the newcomer's score beats the incumbents' — probability exactly* $1/(n+1)$ *— and every moved key moves to the newcomer. (iii)* Leave: *exactly the departed server's keys move, each to its runner-up. Balance here is per-key exact with no aliases; the price is n score evaluations per placement.*

Proof of Theorem 9.5. (i) For fixed key x the n scores are iid, hence exchangeable: each is the maximum with probability exactly $1/n$; independence across keys is independence of the score families. (ii) After the join there are $n+1$ iid scores; the newcomer holds the maximum with probability $1/(n+1)$, and precisely in that event does the key move — the incumbents' relative order is untouched, so a key not won by the newcomer keeps its old winner. (iii) Deleting a server deletes one score; keys for which it was not the maximum keep their winner; keys it won pass to the second-highest score, their runner-up. Every clause is per-key and exact; no expectation over arc geometry is involved. □

Protocol — Box 22. Rendezvous placement (checked against Thaler–Ravishankar)

1: **place**(x): for each server s in the roster compute $h(x, s)$; home is the argmax
2: **replicas**(x): the top R scorers (in distinct failure domains)
3: **join/leave:** no state to update — the roster *is* the state; movement per Thm. 9.5
4: **weighted:** transform scores so win probability is proportional to capacity (reading carries the formula)

Exact per-key balance, no aliases, no ring; n evaluations per placement. The scheme of choice where n is modest — balancers, cache clients, sidecar routers.

Ring against auction. The interview question is inevitable; side by side:

- **the ring** pays memory and setup — aliases, sorted positions, a binary search per lookup — and answers each placement in one hash; its balance needs the alias repair;
- **the auction** pays arithmetic — a hash per server per placement — and holds no state at all beyond the roster; its balance is exact from birth;
- **both** move $1/(n+1)$ on a join — the ring in expectation over arc geometry, the auction per key, by symmetry of the lottery itself — and both are effectively instantaneous at deployed sizes: the choice is temperament and fleet size — the ring won the very largest fleets; rendezvous the balancers and cache clients choosing among dozens.

Two constructions, one moral: **the resignation letter was never about hashing — it was about coupling every key's address to a number that changes. Decouple the addresses from the fleet size, by geometry or by auction, and growth becomes a local event.**

9.4 Part Four — The Apotheosis and the Road Not Taken

The ring conquered the academy's imagination as well as the datacentre's basements. **Chord** (Stoica, Morris, Karger — the same Karger — Kaashoek, and Balakrishnan, 2001) asked: suppose the fleet is not a fleet but a rabble — thousands of machines across the open internet, joining and leaving at will, no operator, no roster. Can a key still find its home? Yes, and in logarithmic time, by way of *fingers* — a routing table of successors at halving distances.

Remark 9.4 (Chord, for the ambitious). Chord (Stoica–Morris–Karger–Kaashoek–Balakrishnan 2001) keeps per node a successor pointer (correctness) and $O(\log m)$ *fingers* — the first node at distance at least $1/2$, $1/4$, $1/8, \ldots$ around the ring — (acceleration); greedy routing by largest non-overshooting finger reaches any key's home in $O(\log m)$ hops with high probability (Problem 9.10, starred). Inside the datacentre the premises invert — membership is a file, the map is megabytes, accountability is wanted — and the industry took the directory road.

Surviving the rabble. Correctness rests on the successor pointers alone; the fingers are mere acceleration; and each node runs a standing *stabilization* conversation — verify the successor, repair the fingers, hand off keys — with the paper proving the ring heals faster than plausible churn tears it. It launched a thousand papers and every distributed hash table of the peer-to-peer era, and through them file-sharing networks carrying, at peak, a respectable fraction of the internet's entire traffic.

The premise audit. The honest observation, delivered with the respect it is owed: **inside the datacentre, the industry mostly took the other road** — not because Chord is wrong, but because Chord answers a question the datacentre does not ask. The exercise — weighing an imported design's premises one by one — generalizes; here it runs:

- **membership:** Chord assumes it unknowable; the datacentre's membership is a file;
- **special nodes:** Chord assumes none may exist; the datacentre appoints them for breakfast;
- **hops against state:** Chord assumes hops cheap relative to global state; the datacentre's global state is megabytes and a hop is a microsecond spent twenty times over;
- **accountability** (Chapter 5's word): when a rebalance goes wrong at three in the morning the operator asks *who holds the truth about where things live*, and a protocol whose truth is smeared across every node's private fingers, converging eventually, answers poorly at exactly the wrong moment; the peer-to-peer world accepted that price deliberately, having no operator and wanting none.

The plain arithmetic closes the case: the data may be petabytes, but the map — shard boundaries to machine names — is megabytes, the one component in the estate that never needed to be distributed for capacity. The two roads compress to a sentence apiece: *Chord distributes the map because it must; the datacentre centralizes the map because it may.* Distribute the data; keep the map close — half the over-engineering this course has witnessed consists of distributing things whose entire contents fit in a pamphlet.

9.5 Part Five — Two Churches: The Directory and the Gossiped Ring

Where does the shard map live? The industry split into two churches on the question, both congregations thriving — the schism is about temperament as much as truth.

The directory church. Cathedral: Bigtable (Google, 2006). Doctrine: placement is *explicit state*, held in a *directory*, administered by a master, guarded by the consensus kernel. Tables are cut into contiguous ranges called tablets — range partitioning, for the scan-loving workloads it serves — and the map from ranges to servers is itself *data*: a metadata table, itself split into tablets, rooted in a single root tablet whose address is held in Chubby — Chapter 5's lock service in its most famous cameo. The walk — "root, metadata, landlord" — against Exhibit 9.3:

- **kernel:** the application asks Chubby where the root tablet lives;
- **root tablet:** names the metadata tablet covering the key's stretch of the alphabet;
- **metadata tablet:** names the data tablet's landlord, who serves the row;
- **steady state:** three lookups on the cold path, all ferociously cacheable — one direct hop, the hierarchy consulted only on a miss.

The tenancy is fenced with the full Chapter 5 ritual, two landlords for one tablet being its nightmare: a tablet server holds its tenancy under a *lease* in the kernel, and the lease machinery guarantees the old landlord's authority expires before the new one's begins; map changes travel by *watches*, pushed to whoever cached the old address. What the arrangement buys: placement is *policy*, not arithmetic — move a tablet for a filling disk, split a range that runs hot, park a tenant away from a noisy neighbour, decisions no hash can express; and the map is a court of record — at three in the morning, *who holds the truth?* is answered by a service with a name, a log, and a page in the runbook. The price is the church's standing anxiety: a directory on the critical path of every cold lookup, a master that must itself be elected and fenced, a hierarchy that can be misconfigured at scale. The church pays gladly — the caching, the parliament beneath, and two decades of the largest estates on earth.

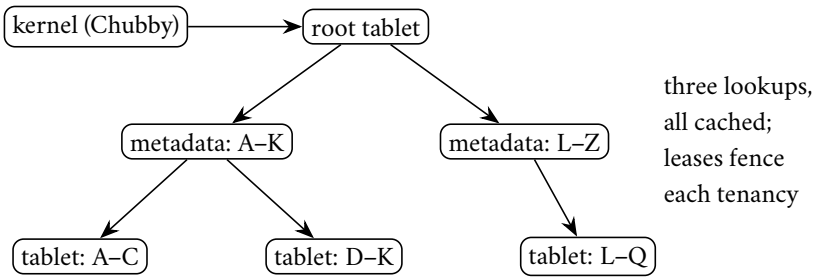


Exhibit 9.3 (the directory church). Bigtable's placement walk: kernel to root to metadata to landlord, every hop cacheable, every tenancy leased through the kernel. The map is data; its changes are pushed by watches; the court of record has a name and a log.

The gossiped-ring church. Cathedral: Cassandra, heir to Dynamo's furniture. The clock face itself is the map — placement *is* the arithmetic — and what little membership state exists spreads by Chapter 6's gossip: no master, no directory to fail, any node able to answer any client, every node doing the placement sum itself. The church has kept up with Part Three's craft: the early cathedrals ran one token per machine and suffered the arc lottery (Prop. 9.4); the modern service ships virtual nodes as standard issue. The virtues are Dynamo's, inherited intact: no single component whose death stops placement; a symmetry that makes the fleet easy to reason about; writes accepted through weather that would have the directory church paging its bishops. The anxieties are the mirror image: placement-by-arithmetic cannot express policy — the arc sits where the hash says, not where the operator wishes; hot arcs are repaired by moving tokens, gossip-coordinated surgery; and the truth about membership is *eventually consistent* — during the

disagreement window two nodes can both believe they own an arc, writes accepted under both beliefs, reconciled afterward by the Chapter 6 janitorial staff (read repair, anti-entropy, hinted handoff). The church's honest answer: it *planned* for this — one broom for replica and placement disagreement both, an economy of mechanism the directory church cannot claim, at the price Chapter 6 itemized.

The schism, adjudicated. Explicit map with an accountable owner, versus arithmetic map with no owner to lose. The industry's newest cathedrals — the Spanners, the sharded parliament estates of Chapter 8 — have overwhelmingly built directory-style, on the grounds that the kernel was already paid for and that at three in the morning, accountability compounds better than symmetry; the ring church's insight remains banked wherever the fleet is caches or balancers and a stale map costs a miss rather than a lost write. Match the church to the price of being briefly wrong, sir. The three review questions, applicable to any storefront:

- **where is the map, and who may write it?** A service with a log, or every node's gossip state.
- **what happens when the map is wrong?** A redirect with a correction, a miss, or a lost write — descending order of civilization.
- **who notices first, the system or the customer?** The directory church can alarm on its own court of record; the arithmetic church discovers disagreements when the nets reconcile them.

No church answers all three perfectly; a storefront that cannot answer them at all is not a church, merely a hope with a logo. And in fairness to the smallest estates, a third pew, unfashionable and frequently correct: the map as a *configuration file* — twelve shards, twelve addresses, changed by pull request: a directory whose parliament is the code review, scaling to precisely the point where placement changes outpace the patience of humans, and possessed until then of the chapter's finest observability property — one can *read* it. Architecture reviews routinely propose a gossip mesh for what is, on inspection, nine machines and a spreadsheet.

9.6 Part Six — Rebalancing, the Operational Crucible

The map, however held, must sometimes change — a shard outgrows its machine, a machine joins, a hot range needs splitting — and *rebalancing* is where partitioning schemes go to be judged, because moving data is itself load, served by the same disks and cables as the customers. Three commandments, each purchased with an outage in the trade's memory.

First: move on purpose, not by arithmetic. The resignation letter was the extreme case — growth *implying* mass movement — but milder versions lurk in every fixed size-threshold split rule and every eager balancer chasing perfect uniformity. The mature posture treats every movement as a decision with a budget: so many megabytes per second, throttled beneath customer traffic, preemptible the moment latency climbs. The drain arithmetic, done aloud at review: a machine holding a few terabytes, drained politely, takes *hours* — so the fleet's true capacity question is never "can we add a machine" but "how many machines can we be mid-draining at once while surviving the next failure." Shard size falls out of the same sum: tens of gigabytes move in minutes and spread a dead machine's inheritance across the fleet; terabytes move in shifts and land like pianos. A rebalance is roadworks: necessary, scheduled, signposted, budgeted in advance, and never conducted across every lane at rush hour.

Second: fear the storm. The episode stages the spiral in full; the skeleton, for review:

- **the trigger:** Gussie's failing disk retries its reads — not dead, merely laboured; crashed-versus-slow, the doppelgänger's oldest trick, in overalls;
- **the response:** the balancer sees three missed health checks, declares him unhealthy, and conscientiously begins moving his forty shards to the survivors;
- **the propagation:** moving is copying, copying is disk and network; Tuppy, receiving eleven shards while serving his own customers, slows — and misses three health checks; the balancer begins moving *Tuppy's* shards; Barmy receives the double burden and follows;
- **the signature:** within the half hour the fleet's bandwidth is furniture removals, customer traffic starving — and, mark it, replace Gussie's disk and the storm *continues*: self-sustaining now, each move causing the slowness causing the next move.

That is a **rebalancing storm** — formally, a rebalancing feedback loop, self-sustaining after its cause is repaired; the medicine became the disease and then the epidemic. The remedies are dampers on the loop: *hysteresis* — a node unhealthy for minutes, not moments, before its tenancy moves; the balancer, of all components, must not believe the doppelgänger's first performance; *rate limits* capping total concurrent movement whatever the apparent need; a *global view* — many nodes degrading at once means "storm, stand down," not "move faster"; and a *panic brake* — the one-switch freeze every mature system ships and every postmortem in this genre wishes had been pulled earlier. The full theory of such self-sustaining overloads — metastability, the outage that persists after its cause is gone — is Chapter 11's centrepiece; here, note only that the balancer is a control loop, and control loops without damping oscillate.

Third: split before you must. Splitting a shard *under* load is surgery on a struggling patient: the split costs reads, writes, and a map update, all charged to the busiest machine in the fleet at the hour it can least afford them. The craft pre-splits: a table expected to be large is born as many shards, boundaries guessed from the key distribution and corrected at leisure; a range observed to be warming is divided while still comfortable, the boundary chosen where the access histogram actually peaks rather than at the arithmetic midpoint — split at the celebrity's doorstep, not halfway down the street. The balance of evidence is the directory church's home fixture: a master with a map and a load report can *decide*, where a ring must wait for its arithmetic to be overruled by hand.

Secondary indexes. One estate-wide fan-out question belongs in this part, because rebalancing exposes it: the data is partitioned by primary key, and the business insists on querying by something else — colour, county, price. Two designs, the fan-out on opposite sides, and no third — only the choice of victim:

- **the local index:** each shard indexes its own tenants — a write updates data and index together, one machine, one local transaction, cheaply; the browse for red umbrellas must ask *every* shard, red items living wherever the hash scattered them: scatter-gather on every query;
- **the global index:** the index is itself a partitioned table, partitioned by the indexed value — the browse reads one shard, cheaply; every write fans out to the data shard *and* the index shard, coupled either distributed-transactionally (Chapter 8 entire, invoiced per write) or asynchronously, in which case the index lags the truth and the shopper occasionally clicks an umbrella that has just sold out: Chapter 7's staleness wearing shop clothes.

Local indexes tax reads; global indexes tax writes — Part One's sentence in an index's clothes — and the mature stores ship both and make one declare, per index, which side of the ledger the business can afford.

9.7 Part Seven — Routing, and the Survival Kit for Celebrity

Two practical matters close the tour, and the second is the one every operator marks with dread.

Routing: who consults the map. Three arrangements, in ascending order of client sophistication; the tell for each is who gets paged when it misbehaves:

- **the dumb client** sends any request to any node — or to a load balancer that does — and the receiving node forwards to the right tenant and relays the reply: one extra hop, zero client knowledge; Cassandra's coordinator-any-node is the type specimen, and the database operators are paged, which is where the expertise lives;
- **the proxy tier** centralizes the map in a routing layer between clients and fleet: clients stay dumb, proxies stay current, one throat to throttle and one place to observe every query — at the price of another tier to run, scale, and page, plus a hop nothing removes;
- **the smart client** holds the map itself — fetched at startup, refreshed by subscription — and goes straight to the tenant: zero extra hops, and a failure mode of the first importance distributed into every application binary in the company: the client's map can be *stale*.

The mature protocols therefore version the map — an epoch, a generation number, stamped on every request — so a tenant asked for an arc it no longer owns answers not with silence or, catastrophically, obsolete service, but with a correction: wrong address, new epoch, ask again. The shape is Chapter 5's — the routing epoch is the fencing token's civilian cousin — and the general law recurs in every system with a map: **a stale map is harmless only if the estate refuses to honour it**, and the refusal must live at the *server*, by epoch check, because the client, by definition of staleness, does not know to refuse. Any design in which an old map is silently honoured has decided that placement is advisory; and placement, this chapter has argued, is law.

Definition 9.5 (the routing epoch)**.** The shard map carries a version e, increased on every ownership change. A client stamps each request with the epoch of the map it used; a server knows, for each arc it owns, the epoch at which its tenancy began, and the current epoch it has heard. Handover discipline (directory church: leases, Chapter 5): the old owner's tenancy is terminated — lease expired or revoked, no further writes accepted — strictly before the new owner's tenancy begins.

The safety claim, in words: server-side version checks plus fenced handover make every applied write land at the current owner; the sabotage deletes the check and loses a write.

Proposition 9.6 (epoch safety)**.** *Under Def. 9.5, if every server rejects any request for a key it does not currently own (answering with a correction carrying the current epoch), then every applied write is applied by the key's owner at application time, and a client with a stale map suffers at worst an extra round trip. If instead some server honours requests for arcs it has ceded (no check, or tenancy not fenced), there is an execution in which an acknowledged write is never visible to any subsequent read — the lost write of Problem 9.7, exhibited in the solutions.*

Proof of Proposition 9.6. Safety with checks. Consider any applied write. The applying server owned the key at application time (the check is part of application, and tenancy, under the handover discipline, is exclusive: the old owner's authority ends before the new one's begins — Chapter 5's lease argument verbatim). A stale-mapped client's request reaches a non-owner, is rejected with the current epoch, and the client retries against the corrected map: one extra round trip, no lost writes. *The lost write without checks.* Let the arc move from server A (epoch three) to server B (epoch four); client C holds the epoch-three map. C writes key

x to A; A, unfenced and unchecking, applies and acknowledges; all subsequent readers consult the current map and read x at B, which never saw the write. The acknowledged update is visible to no future read — the execution of Problem 9.7, drawn as Exhibit 9.4 in the solutions. □

Protocol — Box 23. The routing epoch (distilled from Bigtable §5, Dynamo §4.8, and Chapter 5's fencing)

1: **map:** versioned by epoch e, bumped on every ownership change; held by directory (with watches) or gossip
2: **client:** caches (map, e); stamps every request with e; on correction, refreshes and retries
3: **server, on request for key x:** if it does not currently own x's arc: **reject** with (current e, new owner hint) — never serve, never apply
4: **handover:** old owner's tenancy terminates (lease expiry/revocation) strictly before new owner begins; writes in flight at the boundary are re-driven by clients against the correction

A stale map is harmless only if the estate refuses to honour it; the refusal must live at the server, for the client, by definition of staleness, does not know to refuse (Prop. 9.6).

This chapter's sabotage — smart clients that cache the ring "for performance" and skip the version check — ends with a write applied at an address already condemned: Problem 9.7, certified fatal in the solutions, the trace drawn as Exhibit 9.4.

The celebrity. The row every scheme dreads, because it defeats the premise beneath this chapter's mathematics: that load, like keys, is divisible. One key — the viral post, the flash-sale item, the national election result — draws a fleet's worth of traffic to a single address, and no scheme divides the indivisible: one arc on the ring, one range in the directory, one machine the estate's brightest ember. The episode stages it with the aunt's flower-committee notice, taken up by the national press at teatime; by six o'clock her single row outdraws the rest of the estate combined. The hash placed her fairly; fairness was never the issue — the world's affection is not uniformly distributed. The survival kit, in escalating order of surrender:

- **cache it:** celebrity is read-heavy in the overwhelming case, and a small cache in front — even a few seconds of staleness — absorbs orders of magnitude. The famous failure is the *stampede*: the cached copy's lease expires and ten thousand admirers miss *together* — the thundering herd, an outage manufactured by an expiry timestamp. The discipline is cheap — one request per key refreshes while the rest wait a beat or take the slightly stale copy: request coalescing, stale-while-revalidate; several names, one idea, that a cache miss should be an event, not a demographic. Full pricing lodged with debt `#22`;
- **split the reads:** replicate the celebrity's shard wider than the standing factor and spread reads across the copies — replication, for once, rescuing partitioning, since reads divide even when keys do not. The staleness fine print is Chapter 6's, which for a flower count is no fine print at all; the widening can be automatic — extra followers for the afternoon, shed at dusk, fame handled as weather;
- **salt the writes:** when the celebrity is write-hot — the live counter, the auction of the century — split the key itself: a hundred sub-counters under a hundred salted names, each hashing to its own shard, every write choosing a salt at random, every read summing the hundred rows — a scatter-gather paid only by readers of that one key. A hundred thousand increments a second against one row is one shard's funeral; against a hundred salted rows, a thousand apiece — a shrug. The construction is Chapter 7's grow-only counter wearing a wig: dividing the indivisible by making the merge lawful;

- **special-case it:** past some fame, generality is the wrong tool and the honest architecture says so aloud: a dedicated fleet for the day's viral keys, a broadcast tier that pushes the election result outward rather than letting the world pull it. The detection is a small standing census of the hottest keys, cheap to run and worth its weight in postmortems unwritten.

What does not work is denial: the load report always finds the celebrity eventually; the only question is whether the architect finds it first — and the estates that answer "we have no hot keys" divide, on inspection, into those with the census and those with the surprise.

9.8 The Whiteboard

For the design review — economics throughout.

1. Partitioning is the choice of which fan-out you pay — reads (hash, local indexes, scatter-gather), writes (range under monotony, global indexes), or rebalances (modulo N, eager balancers). No scheme escapes all three; match the expensive one to the operation the business performs least, and write the choice down where the next architect can find it — an undocumented fan-out decision is a trap set for one's successor.

2. Hash modulo N is a resignation letter, postdated to the day the fleet changes size. Decouple addresses from fleet size — the ring with virtual nodes, or the rendezvous auction — and membership change becomes a local event: one over n moves, the rest of the estate unaware. And, as item two and a half for the schema page specifically: co-location is transaction avoidance — keys that commit together should live together, the compound key is the standing treaty between hash and range, and the placement hash has tenure.

3. Every shard map needs an owner, and every rebalance a budget. Directory or ring, someone answers "where does this key live, and who says so" at three in the morning — prefer the answer with a name and a log. Movement is roadworks: throttled, scheduled, preemptible, with hysteresis on the triggers and a panic brake the operators have rehearsed. The balancer is a control loop; undamped control loops are Chapter 11's opening exhibit.

4. The map is state: version every routing table, stamp every request, and let wrong-address answers carry the correction — the routing epoch is the fencing token's cousin, and a client that skips the version check is this chapter's sabotage. For the celebrity: keep the census of hot keys running, cache with stampede discipline, widen the replicas, salt the writes, and past some fame, special-case with dignity — the load report will find the celebrity regardless; arrange to have found it first.

The register. Debt #20 — where things live is a decision, issued only last chapter — is PAID: placement decided, by arithmetic on a dial or a directory under a parliament, the trade-offs priced and the churches named. ISSUED: #21 — everything here partitioned data *at rest*, and the same question stalks computation: where does the *work* live (payable Chapter 10, where the log takes its starring role); #22 — stampedes, storms, and the other self-sustaining weather glimpsed in the balancer's spiral and the cache's herd (payable Chapter 11). For the first time this season the paid column outnumbers the open by better than two to one. Episode word count: 10,023 — above the floor; the corrective's warm-expansion drill required two passes this session (drafts continue to arrive short; see LEDGER §5).

9.9 Problems

Tier I — Calisthenics

Problem 9.1 (placement drills)**.** On Exhibit 9.1's ring (Bingo at one o'clock, Tuppy at five, Gussie at nine, positions read as clock hours): (a) place keys hashing to two, four, six, eight, ten, and twelve o'clock. (b) Barmy joins at three (Exhibit 9.2): which of your six keys move, and to whom? (c) Gussie retires: which keys move, and to whom? (d) With replication factor two (next distinct server clockwise), list each key's replica pair before and after Barmy's arrival, and confirm no pair changed except where Barmy's arc is involved.

Problem 9.2 (the lottery, quantified)**.** Ten servers. Using Theorem 9.2: (a) compute the relative standard deviation of a server's share for one alias, one hundred aliases, and ten thousand aliases. (b) Using Prop. 9.4, bound the probability that some single-alias arc exceeds five times the fair share. (c) A vendor proposes one alias per server "for simplicity" on a twelve-server fleet; write the two-sentence objection, with numbers.

Problem 9.3 (the resignation letter, audited)**.** Under hash modulo N: (a) show the expected fraction of keys whose home changes when N grows to $N{+}1$ is exactly $N/(N{+}1)$ (hint: a key stays iff its hash's residues agree, which for a uniform hash has probability about $1/(N{+}1)$ — make the "about" precise). (b) Compare with Thm. 9.3's $1/(N{+}1)$ and state the ratio. (c) Name one deployment where modulo is nevertheless defensible, and the exact property that excuses it.

Problem 9.4 (name the fan-out)**.** For each design, name which fan-out pays (reads, writes, or rebalances) and the chapter's remedy: (a) local secondary indexes; (b) global secondary indexes, asynchronous; (c) range partitioning keyed by timestamp; (d) hash partitioning queried by ranges; (e) an eager balancer chasing perfect uniformity.

Problem 9.5 (the auction, run by hand)**.** Three servers; scores for four keys given as a table of your own construction with distinct values. (a) Place the keys. (b) Add a fourth server with fresh scores; verify only keys it wins move. (c) Remove the original winner of one key; verify the runner-up inherits. (d) State the exact join-movement probability and check your table's empirical fraction against it in one sentence.

Tier II — The Real Thing

Problem 9.6 (the split policy)**.** Design a split policy for a range-partitioned parcels ledger keyed by dispatch time. Requirements: bound any shard's write rate to a target; survive the Christmas peak; keep historical shards cold and mergeable. Specify: when to split (trigger and measurement window), where (boundary choice against the access histogram), what pre-splitting the calendar justifies, and the salting fallback if a single hour still exceeds one shard's capacity. Defend each choice in a sentence, and state which church (directory or arithmetic) your policy presumes and why.

Problem 9.7 (sabotage: the stale ring)**.** A smart-client fleet caches the ring at startup "for performance" and omits the epoch stamp; servers, trusting the clients, apply any request addressed to them. Exhibit the lost-write execution of Prop. 9.6: the membership change, the stale client, the acknowledged write, and the proof that no future read at any replica returns it. Then state the two independent repairs (server-side epoch check; fenced handover) and show each alone closes your execution.

Problem 9.8 (the inheritance lump)**.** Single-alias ring, n servers. Server s departs unplanned. (a) Show the successor's expected share doubles: compute the distribution of the merged arc (use Lemma 9.1 with $k{=}2$). (b) With k aliases, show the inheritance splits across up to k successors and compute one inheritor's expected increment. (c) Combine with Prop. 9.4 to bound the probability the merged single-alias arc exceeds four

times fair share. (d) One sentence: why this, and not steady-state balance, is the strongest argument for aliases.

Problem 9.9 (epoch liveness)**.** Under Box 23: (a) prove that a client whose every request is rejected-with-correction terminates (reaches the true owner) provided ownership changes finitely often during its attempt, and bound its retries by the number of changes. (b) Exhibit the starvation execution when ownership oscillates forever (the storm's routing shadow), and propose the damper.

Tier III — For the Ambitious (starred)

Problem 9.10 (★ Chord's bound)**.** With m nodes at uniform positions and correct finger tables (Rem. 9.4): (a) show each greedy hop at least halves the clockwise distance to the target; (b) conclude $O(\log m)$ hops to reach the predecessor of any key with certainty, and $O(\log m)$ with high probability to finish; (c) bound the routing state per node and the expected stabilization work per membership change. *(Hints only.)*

Problem 9.11 (★ the lottery's tail, both ways)**.** Prove the lower-bound direction of Prop. 9.4 rigorously: with m uniform tokens, the expected maximum spacing is at least $c' \ln m/m$ for some constant $c' > 0$; conclude the single-alias worst server holds $\Theta(\log m)$ times fair share in expectation. *(Hints only.)*

9.10 Hints

Hint (Problem 9.3)**.** (a) Count residue collisions of a uniform integer hash over a large range; the discrepancy from exact uniformity is the "about." (c) A fleet whose size never changes — and say why that property is rarer than its owners believe.

Hint (Problem 9.8)**.** (a) The merged arc is the sum of two designated spacings. (c) Union-bound the two spacings separately, or apply the Beta tail directly.

Hint (Problem 9.10)**.** (a) If the target is more than halfway, some finger jumps at least half; if less, induction on the remaining interval. (b) Distances shrink geometrically; after $\log_2 m$ halvings the interval holds, in expectation, one node.

Hint (Problem 9.11)**.** Exponential spacings: the maximum of m iid exponentials exceeds $\ln m - \ln\ln m$ with probability tending to one; carry the normalizer's concentration (law of large numbers) through the ratio.

9.11 Solutions

Tier I in full (tersely); Tier II in full — Problem 9.7 is the sabotage certification; hints only for Tier III.

Solution (to Problem 9.1)**.** (a) Reading the walk toward the next server position: two o'clock → Tuppy (five); four → Tuppy; six → Gussie (nine); eight → Gussie; ten → Bingo (one, wrapping); twelve → Bingo. (b) Barmy at three captures the arc (one, three]: the key at two moves to Barmy; the key at twelve stays with Bingo (twelve walks past the wrap to Bingo at one — unmoved); all others unmoved. (c) Gussie's tenants (six, eight) walk on to the next survivor clockwise — with Barmy present, that is Bingo at one (wrapping past nine): both move to Bingo; nothing else stirs. (d) Before: two/four → (Tuppy, Gussie); six/ eight → (Gussie, Bingo); ten/twelve → (Bingo, Tuppy). After Barmy: two → (Barmy, Tuppy); four → (Tuppy, Gussie) unchanged; ten/twelve → (Bingo, Barmy) — the second replica changed because Barmy is now Bingo's clockwise neighbour; six/eight unchanged. Only pairs meeting Barmy's position changed, as monotonicity demands.

Solution (to Problem 9.2)**.** (a) Relative deviation $\sqrt{(n-1)/(nk+1)}$: k=1: $\sqrt{9/11} \approx 0.9$ — share swings of order its own size; k=100: $\sqrt{9/1001} \approx 0.095$ — about ten percent; k=10^4: about one percent. (b) Fair share is one tenth; five times is one half: $\Pr[\max > 1/2] \le 10 \cdot (1/2)^9 \approx 0.02$. (c) "With one alias each, the typical spread between your luckiest and unluckiest machine is a factor of several, and the expected worst arc is roughly two and a half times fair share ($\ln 12 \approx 2.5$); a hundred aliases cost kilobytes and confine imbalance to ten percent."

Solution (to Problem 9.3)**.** (a) For a hash uniform on a range vastly exceeding $N(N{+}1)$, residues mod N and mod $N{+}1$ are jointly uniform on the $N(N{+}1)$ pairs up to negligible edge error; the key stays iff the pair maps both residues to the same server, which happens for exactly one new-residue value per old, probability $1/(N{+}1)$ up to that error. Moved fraction: $N/(N{+}1)$. (b) The ratio to the ring's $1/(N{+}1)$ is N: the letter costs N times the neighbourhood. (c) A statically sized fleet — compile-time sharding of a fixed scoreboard, say — where the divisor is a constant of the architecture; the excusing property is not smallness but *provable constancy*, and fleets believed constant have a documented habit of growing on the eve of a peak.

Solution (to Problem 9.4)**.** (a) Reads pay (scatter-gather per query); remedy: accept, or promote hot query axes to global indexes. (b) Writes pay (fan-out, plus staleness of the async lag); remedy: budget the lag, monitor it, and mark provisional reads. (c) Rebalances and the tail shard pay (the dogpile); remedy: compound key or salting, pre-split the tail. (d) Reads pay (every range scan is estate-wide); remedy: compound keys, or a range-partitioned projection maintained as a pipeline. (e) Rebalances pay (movement as standing load, storm risk); remedy: budgets, hysteresis, and a definition of "balanced enough."

Solution (to Problem 9.5)**.** Any table with distinct scores works; the checks are mechanical. (d) Exactly $1/(n{+}1) = 1/4$ per key; with four keys the empirical fraction is a multiple of one quarter, and agreement with expectation is approximate by design — four coin flips do not an expectation make, which is rather the point of stating exact per-key probabilities.

Solution (to Problem 9.6)**.** Sketch of the defensible design. Trigger: sustained write rate over a window of minutes (hysteresis against bursts) crossing a fraction — half, say — of a shard's measured capacity; splitting at half leaves headroom to perform the split off the peak. Boundary: at the access histogram's current watermark — for a monotonic key, near the present moment, so the cold past stays whole and the hot tail is freshly capacious; never the arithmetic midpoint, which for monotonic keys splits history, not load. Pre-splitting: the calendar knows the peak; create the season's expected tail shards in advance, boundaries at forecast hourly volumes, so Christmas arrives to find the doors already open. Fallback: if one hour still exceeds one shard, the key ceases to be purely temporal — salt the tail (hour, salt) across a small fan of shards and merge on read; this is the counter split, Chapter 7's algebra, applied to placement. Church: directory — every clause above is a *decision* against load reports, which is the directory's home fixture; a pure arithmetic ring can express none of it without a human moving tokens.

Solution (to Problem 9.7 — the certification)**.** The execution, against Box 23 with lines three and four deleted. Epoch three: arc α (containing key x) owned by server A; client C caches the epoch-three ring.

(1) Rebalance: α moves $A \to B$; epoch four published. A, un-fenced, retains its copy and its willingness; C, un-notified (smart client, no watch, no stamp), retains epoch three.

(2) C writes x=v addressed to A. A applies and acknowledges. (A owns nothing; nobody checks.)

(3) All current maps say B. Every subsequent reader — including C itself after any refresh — consults epoch four and reads x at B: the value is the old one; v is on a machine no route reaches.

(4) Optional aggravation: anti-entropy between A's stale copy and B runs *backward* (Chapter 6's nets reconcile replicas of one shard, and A is no longer enrolled as one), so no janitor ever carries v home. Acknowledged, then invisible forever: the lost write.

Repairs, each singly sufficient against this execution. *Server-side check:* at step (2), A consults its tenancy, finds α ceded at epoch four, rejects with correction; C refreshes, writes at B; no loss. *Fenced handover:* the transfer at step (1) revokes A's tenancy (lease expiry) before B opens; a revoked A cannot apply at step (2) — the write fails visibly, the client retries, no silent loss. Certified fatal as shipped: one routine rebalance, one cache, zero crashes — acknowledged customer data unreachable by every future read.

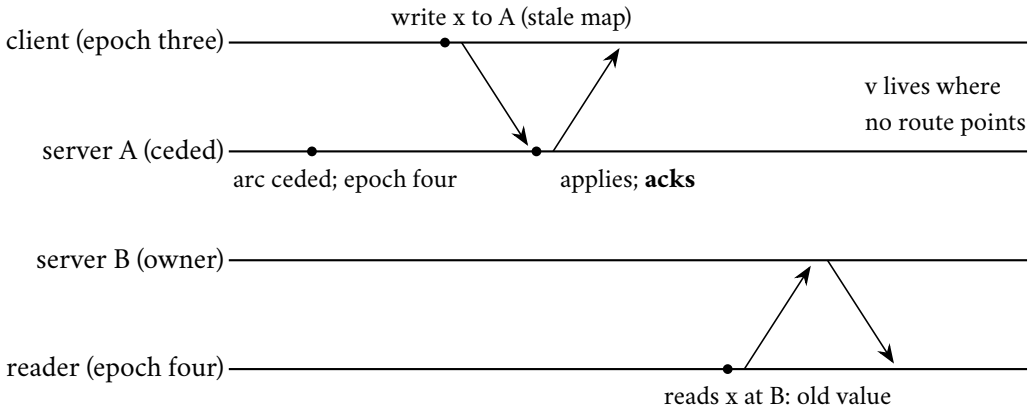


Exhibit 9.4 (the lost write). The stale client writes to the ceded server, which applies unchecked; every current map points elsewhere; the acknowledged value is stranded. Either repair — the server-side epoch check or the fenced handover — breaks step two (Prop. 9.6).

Solution (to Problem 9.8)**.** (a) The merged arc is two designated spacings of $m = n$ tokens: $\mathrm{Beta}(2, n-2)$, mean $2/n$ — the successor's tenancy doubles in expectation, on the worst possible day. (b) Each inheriting successor gains one sliver of the nk-token ring: mean $1/(nk)$, so an inheritor's share rises from $1/n$ by about a $1/k$ fraction of itself — a crumb. (c) $\Pr[\mathrm{Beta}(2, n-2) > 4/n]$: bound each constituent spacing by Prop. 9.4's event or integrate the Beta tail; either route gives a small constant for moderate n — the desk accepts $\Pr \le 2(1-2/n)^{n-1} \approx 2e^{-2}$ by bounding each spacing at $2/n$, roughly a quarter: not rare. (d) "Aliases exist not to smooth the average Tuesday but to ensure that a machine's death is inherited by the fleet rather than by its neighbour: steady-state variance is a nuisance; a doubling under failure is an outage."

Solution (to Problem 9.9)**.** (a) Each rejection carries an epoch strictly larger than the client's; epochs are integers increasing only on ownership change; with c changes during the attempt the client can be corrected at most c times before its map's epoch is current, whereupon its request reaches the owner and is served: retries bounded by c plus one. (b) Oscillation: the arc ping-pongs between servers each round faster than the client's refresh-and-retry; every retry arrives at last round's owner; starvation. Damper: the same as the balancer's — hysteresis on ownership (a minimum tenancy period bounds changes per unit time, restoring (a)'s bound), or have the rejecting server *proxy* the request onward once instead of bouncing the client, converting the last hop into Part Seven's dumb-client mode exactly when the map is too lively to cache.

9.12 The Reading

Karger, Lehman, Leighton, Panigrahy, Levine, and Lewin, "Consistent Hashing and Random Trees," STOC 1997. Short, elementary in the honourable sense, and the rare theory paper whose corporate consequences you have used today. Read the balance and monotonicity claims against Thms. 9.2 and 9.3.

Chang et al., "Bigtable: A Distributed Storage System for Structured Data," OSDI 2006. Read it for the placement machinery — tablets, the metadata hierarchy, the master's duties, the Chubby tenancy — against Exhibit 9.3; the data model was yesterday's news a decade ago, but §5 is the directory church's liturgy.

Thaler and Ravishankar, "Using Name-Based Mappings to Increase Hit Rates," IEEE/ACM Transactions on Networking, 1998. Rendezvous hashing from its authors: the auction, the exact bounds, and the weighted transform Box 22 references.

Stoica, Morris, Karger, Kaashoek, and Balakrishnan, "Chord: A Scalable Peer-to-Peer Lookup Service," SIGCOMM 2001. The fingers, the logarithm, the stabilization protocol — and the premises to audit, one by one, before importing any of it indoors (Rem. 9.4).

Mirrokni, Thorup, and Zadimoghaddam, "Consistent Hashing with Bounded Loads," SODA 2018 (announced 2017). The cap on the unlucky server; balance and movement bounds that survive the forwarding walk (Rem. 9.2).

The shelf. DeCandia et al., Dynamo (SOSP 2007), §4 reread with this chapter's arithmetic — tokens, preference lists, the walk as replicator. Burrows, "The Chubby Lock Service" (OSDI 2006), reread from Chapter 5 now that the tenant list is visible. And Kleppmann, chapter six — partitioning in prose, hot spots and rebalancing included, the standing companion's steadiest chapter.

Chapter 10

The Log and the Dataflow

Companion to Episode Ten. The episode tells the story — the blog post that outread the journals, the Friday poison, the train through the tunnel, the double payment, the forty-year reveal; this chapter keeps the record: the log defined with its cursor, the lineage's recovery doctrine from the Google File System through MapReduce to Spark, the watermark proposition, the exactly-once lemma proved in full, the barrier-snapshot theorem with the Chandy–Lamport correspondence tabled, the outbox proposition, and the sabotage certified fatal with the wire transfer exhibited. Statements, proofs, protocol boxes, problems.

10.1 Part One — The Lineage: Failure Made Boring, Computation Made Pure

The chapter's thesis arrived in 2013 as a company blog post — Jay Kreps, "The Log: What every software engineer should know about real-time data's unifying abstraction" — and proceeded to outread the journals. The claim: the humble log — an append-only sequence of records, numbered in order — is the unifying abstraction of distributed systems, the seam along which every estate can be taken apart and reassembled. Its power is its poverty: append at the tail, read forward from an offset, nothing else; the past immutable, the order total within the log, every replaying reader arriving at the identical conclusion. A log is *agreement, serialized* — Chapter 5 built parliaments precisely to decree one — and its three properties — appended only, totally ordered, durably replayable — are why writers never conflict, why readers never disagree, and why the past can always be revisited.

Definition 10.1 (log; cursor)**.** A *log* is a sequence of records at consecutive integer offsets, supporting append-at-tail and read-from-offset; records are immutable and retained per policy (time, size, or compaction: newest record per key retained, null-body tombstones marking deletions, cleared only after every enrolled reader has passed). A *consumer group* holds one cursor per partition: the offset up to which processing is acknowledged. Rewinding a cursor is the licensed operation that makes input *replayable*.

The hub and the hairball. The post's operating misery: a dozen producers and a dozen consumers integrated point to point is on the order of a dozen dozens of bespoke pipelines, each with its own format, its own failure modes, its own departed owner. One central, durable, ordered, replayable structure collapses the dozen dozens to a dozen plus a dozen — producers write once, consumers read at leisure. The hub was the business case; the log as *theory* — commit logs, replication, the database's cellar machinery promoted to civic architecture — is what made the post outlive its trigger. Nor is the structure a stranger here: Chapter 5

decreed one into existence entry by entry, Chapter 8's court of record was one, Chapter 9's directory church kept its map as one.

The Google File System (Ghemawat, Gobioff, and Leung, 2003). The lineage's foundation, three decisions. Files are huge and append-mostly, so cut them into large chunks, replicated threefold across the fleet. Failure is a climate, not an event — at fleet size, something is always dead — so the posture is to *metabolize* it: a chunk's death detected, its replicas re-spread, automatically, as background digestion; *failure made boring* is the paper's real invention, and the temperament survives in every storage system since. And the map of chunks to machines lives with a single master — Chapter 9's directory church founded in one stroke: megabytes of map for petabytes of data, kept current by chunkserver heartbeats, off the data path (clients fetch addresses, then speak to chunkservers directly), itself recovered from a checkpoint plus an operations log. The honest postscript from a decade's operation: the single master aged into the design's one regret — metadata outgrowing one machine's comfort, failover manual in the early years — and the successors sharded the map and put parliaments under it. Even directories eventually need partitioning; the lineage's own history obliged by demonstrating it.

MapReduce (Dean and Ghemawat, 2004). The deep move: restrict the programmer to two *pure functions* — a map, applied record by record, and a reduce, applied per key to grouped intermediates — with a *shuffle* between them, regrouping every intermediate record by key; nothing else on offer. The canonical job — count every word's occurrences across the library — the episode walks with the full cast; the skeleton for review:

- **map:** each machine passes over its own chunk of the library, emitting the pair (word, one) per occurrence — purely local, no conversation;
- **shuffle:** every pair travels to the machine responsible for its word, the destination chosen by hashing the word across the reducers — Chapter 9's placement arithmetic partitioning not stored data but work in flight; debt #21's payment begins in this clause;
- **reduce:** each reducer, holding every pair for its words, sums the ones and emits the totals.

No step of the description mentions failure, which is the point: pure functions commute with re-execution — Chapter 7's idempotence arriving by another staircase — so a failed task is simply run again, a lost machine's tasks re-dealt like cards; fault tolerance stops being a protocol and becomes a scheduler's habit. Purity also pays debt #21's first instalment: *move the computation to the data* — dispatch the map task to the machine already holding its chunk, a kilobyte of code shipped to a gigabyte of data, never the reverse. The household accounts: maps and reduces are cheap and local; the shuffle is the pattern's entire freight bill — every intermediate record crossing the network once, landing on disk at both ends — and two decades of engineering in this lineage are a sustained campaign against its invoice: combine locally before shipping, compress in flight, keep the regrouped data in memory rather than paying the disk twice.

Stragglers and backup tasks. Jobs are delayed less by crashes than by *stragglers* — machines not dead, merely slowed: a failing disk retrying, a noisy neighbour — crashed-versus-slow in overalls yet again. Chapter 6's tail arithmetic makes them compulsory at scale: a task slow one time in a hundred puts, in a job of ten thousand tasks, a hundred slow ones in expectation, and the job waits for the slowest of them. The remedy, *backup tasks*: when a task lingers near a job's end, launch a duplicate on another machine and take whichever finishes first — purity makes the race safe, and the paper reports the price and the prize with engineering candour: a few percent of duplicated work, nearly a third of the job's latency returned. Filed

open as debt #24: in the finale, ten thousand accelerators face the same unluckiest-machine arithmetic every few seconds.

Spark and the recipe (Zaharia and colleagues, 2012). The bureaucratic descendants made the pattern's cost visible: every stage's output written to replicated disk before the next stage read it back, the storage tax paid per stage — and per lap, for the iterative computations revisiting the same data. Spark's inversion: a *resilient distributed dataset* is a collection, partitioned across the fleet, defined not by its bytes but by its *recipe* — the lineage of pure transformations that produced it from stable inputs. Datasets live in memory, uncommitted, unreplicated; when a machine dies, the system *recomputes* the lost partitions — reads the recipe backward to the last stable ancestor and replays, for those partitions alone. *Recompute, don't replicate*: fault tolerance as bookkeeping about provenance rather than redundant bytes; the recipe is kilobytes where the data is terabytes, and purity guarantees the second cooking tastes exactly as the first. Two honest footnotes, both in the paper:

- **narrow versus wide:** a transformation whose every output partition depends on one input partition — a map, a filter — recomputes a lost partition from one ancestor, cheaply; a transformation that shuffles — a grouping, a join — makes every output partition depend on *all* input partitions, dragging the whole ancestry back into the kitchen; whence the licence to *checkpoint* — materialize a dataset durably, truncating the lineage — precisely at the expensive waists;
- **memory as a lease:** datasets are cached as advice, evicted under pressure, recomputed on demand — the recipe always the fallback, which is what lets the promise be cheap.

The course files the idea beside Chapter 5's determinism diet and Chapter 8's Calvin: agreement on inputs and recipe, with outputs derived rather than negotiated, keeps proving to be the cheapest agreement on the menu.

10.2 Part Two — The Log Ascendant

Kreps's post distilled what LinkedIn had built in *Kafka*: Chapter 5's log — replicated, append-only, numbered — stripped of the parliament's other furniture and offered alone, as a public utility; debt #15, the log's stardom, is paid here. A *topic* is a set of *partitions*; each partition is a log (Def. 10.1), replicated across brokers by a leader-and-followers arrangement — a leader taking appends, followers fetching in order, a committed offset advancing as the in-sync replicas confirm — with Chapter 6's fine print still printed on the tin. Beneath the product, a physical insight, delightfully unfashionable: the log is the *disk's* favourite data structure — random access is where disks suffer; sequential appends and reads are where they excel, by orders of magnitude — so retention costs shelf space rather than performance, and the kept week of the estate's events buys the *replayability* that is the recovery story of the entire modern stack. The design is the discipline of what the product refuses to do.

First refusal: global order. Order is guaranteed within one partition and nowhere else. A partition is a shard of the stream, placed and keyed by Chapter 9's arithmetic: choose the key so that records whose order matters share a partition — one customer's events together, ordered; different customers interleaved arbitrarily, and lawfully. Order is a *per-partition commodity*: within the shard free, across shards the price of a parliament. The review's interrogation — *ordered for whom?* — dissolves most global-order demands: Bertie's own events out of order would corrupt his story; whether his Tuesday purchase files before or after the aunt's is a fact no customer, no invariant, and no auditor consults. The residue that survives — a true global sequence, an auction's book — is Chapter 4's business and priced accordingly.

Second refusal: delivery bookkeeping. The queues of the older tradition tracked, per message, per consumer, delivery, acknowledgment, redelivery — the broker as anxious postmaster, its state growing with its conscience. The log stores nothing of the kind: records sit at their offsets, immutably, for the retention period; each consumer group holds merely a *cursor*, the committed read position — and a cursor is not a receipt: consuming is reading, not taking, and ten groups read the same log at ten offsets without disturbing one another or the records. The bookmark is the recovery story entire, and the rewind the product's finest feature — the trade's Friday poison, staged in the episode; the skeleton:

- **the injury:** a four-o'clock Friday deploy ships a consumer that processes every record and quietly writes rubbish; the rubbish flows all weekend; Monday's dashboards confess;
- **the postmaster tradition:** the letters were consumed, acknowledged, destroyed; the weekend's truth is gone; the recovery plan is called the backup, may it exist;
- **the log tradition:** fix the bug, reset the group's cursors to Friday afternoon, replay the weekend through the corrected code into a fresh view, cut over. The estate's memory was never in the consumer; the consumer was always disposable.

Groups, rebalancing, and the ceiling. A consumer group is one logical reader, its members dividing the topic's partitions among themselves — a placement problem in miniature, *rebalanced* when a member joins or fails, storms included: a flapping member can stall a group's reading entirely, and the remedies are Chapter 9's dampers, verbatim. The parallelism ceiling is set at the keyboard: the partition count bounds the group's useful width — twelve partitions feed at most twelve readers — so Chapter 9's partitioning decision is also every downstream consumer's concurrency budget, decided earlier than anyone realized it mattered. And the refusal's operational purchase: the broker's work per record is constant — append, replicate, serve reads by offset — regardless of how many consumers exist or how far they lag; a group down for maintenance costs shelf space; a new analytics team subscribing to a year-old topic is a bookmark at offset zero. The log scales its readership the way a library does and a courier service cannot.

Compaction, and the database turned inside out. For topics whose records are keyed updates, a *compacted* topic retains, per key, at least the latest record, discarding superseded ancestors at leisure; deletion is a record with a key and a null body — a *tombstone*, retained until every reader has witnessed the death before the stone itself is cleared (Def. 10.1) — Chapter 6's prop reprised without alteration, because resurrection-by-replay is the same disease as resurrection-by-anti-entropy. The compacted log is a full snapshot of current state *and* a subscription to its changes: replay it from the top and you have rebuilt the table; keep reading and you stay current. Whence Kreps's organizing phrase: *the database turned inside out* — publish the log as the primary artifact, and let every table, cache, index, and search engine in the estate be a *materialized view*: a consumer group with a cursor, rebuilding itself by replay whenever its logic changes. The dividend, at Bertie's shop: the search team ships a better analyzer by starting a fresh index from offset zero and cutting over when its cursor reaches the head, rollback the same trick backward; the cache suspected of corruption is discarded and replayed, views being disposable by construction. Rebuild anything by rereading; trust nothing but the sequence — version control's idea, applied to data.

KRaft. Kafka's brokers leaned for years on ZooKeeper, the consensus kernel in its standard cameo, for their metadata — which brokers live, who leads each partition. In maturity the project folded the machinery inside: *KRaft*, the brokers' metadata itself a Raft-replicated log managed by the brokers themselves — the log-as-a-service come of age by becoming its own parliament, the metadata log and the data logs one species

at last. The estate's map, Chapter 9 insisted, wants a court of record; the finest court of record Kafka knew was the product it already sold.

10.3 Part Three — The River and the Table, and Time Redux

The old org chart put *batch* — nightly jobs over finished files — and *streams* — messages, reactions, dashboards — in different departments, with different tools and different bugs. The log dissolves the wall in one sentence: *a stream is an unbounded table being read in motion; a table is a stream compacted to now* — the river and the table. The nightly batch job is a consumer whose cursor jumps a day at a time; the dashboard is a consumer riding the head; the distinction reduces to a bookmark's velocity — debt #21's second instalment, computation partitioned across time as the shuffle partitioned it across space. But the dissolution revives the course's oldest ghosts, arriving, as Chapter 2 promised, wearing lag. The models under which every claim below is made:

Model of this chapter

For the dataflow results (Defs. 10.3–10.4, Thm. 10.3, Cor. 10.4). A job is a finite directed *acyclic* graph of operators; channels are FIFO and reliable between correct processes (retransmission beneath, Chapter 1); sources read a durable log supporting rewind to any offset; operators hold state and process one record at a time, deterministically (randomness seeded by record identity; time taken from records and watermarks); processes crash-stop and are replaced. Checkpoint storage is stable.

For delivery and deduplication (Lemma 10.2, Prop. 10.5). At-least-once channels: every sent record is delivered one or more times, finitely often, with a bounded retry horizon T; records carry unique identifiers. Exactly-once *delivery* is unavailable, per Chapter 1's Two Generals theorem — all constructions below are effect-level.

For watermarks (Def. 10.2, Prop. 10.1). Records carry event times; each source emits nondecreasing watermarks and *promises*: a perfect source emits w only after every record with $t_{\text{ev}} < w$ has been emitted; a heuristic source may violate the promise at a measured rate, producing late records.

Event time, processing time, windows. A batch job over yesterday's file knows the day is over; a stream processor computing orders-per-hour must decide when an hour is *finished*. Records carry two times, and confusing them is the beginner's injury in this trade: *event time* — when the thing happened, $t_{\text{ev}}(r)$, stamped at the source — and *processing time* — when the record reaches the processor, the clock on the wall. The windows the business asks for — *tumbling*, back to back like paving stones; *sliding*, the last hour as of every minute; *session*, bounded by gaps in a key's activity — are all groupings by event time, which is precisely the grouping the network declines to deliver in order. The classic offender is the tunnel: Bertie rides the four o'clock train, his telephone sensibly buffering what it cannot send; at six the train emerges and disgorges two hours of commerce truthfully stamped four-oh-five, four-twelve, four-forty. Sort the ledger by arrival and the afternoon's history is fiction; hold every window open forever against possible tunnels and no total is ever final.

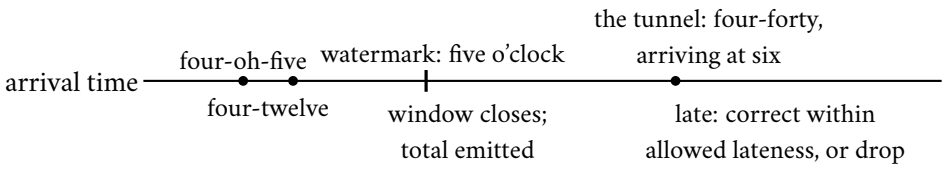


Exhibit 10.1 (the tunnel). Event times on the labels, arrivals on the line. The watermark's passage closes the four o'clock window on stated belief; Bertie's buffered four-forty emerges at six, behind it — late by declaration, handled by policy, counted either way (Def. 10.2).

The trade's instrument, from the MillWheel and Dataflow lineage at Google (Akidau and colleagues), is the one this course respects most — the honest confession, flowing through the dataflow as a marker in the stream:

Definition 10.2 (watermarks; lateness)**.** A watermark w at a point in the dataflow asserts: no further record with $t_{\text{ev}} < w$ will arrive at this point. Sources emit watermarks per the model box; an operator with several inputs holds its watermark at the *minimum* of its inputs' and emits accordingly. An event-time window $[a, b)$ may be *closed* when the local watermark passes b. A record arriving behind the watermark is *late*; policy admits it within an allowed lateness (reopening the window and emitting a keyed correction) or drops it with a metric.

An operator closing a window acts not on certainty, which Chapter 3 forbids, but on a stated, tunable, monitorable assumption — Chapter 4's partial synchrony, wearing a timestamp. The soundness claim, its skeleton first: induction on DAG depth — min-propagation preserves the sources' completeness promise, so window closure never precedes its records except where a heuristic source confessed it might.

Proposition 10.1 (watermark soundness)**.** *If every source's promise holds (perfect watermarks) and operators propagate by minimum, then whenever any operator closes window $[a, b)$, it has already received every record of that window destined for it. With heuristic sources, every violation at closure corresponds to a late record at some source, so the failure rate downstream is bounded by the sources' confessed rates. (The watermark is thus a failure detector for time: complete — windows eventually close, if watermarks advance — and accurate exactly as often as the confession is honest; a stalled input freezes closure estate-wide, the price of the minimum.)*

Proof of Proposition 10.1. Induction on topological depth. Depth zero: a perfect source emits w only after all records with $t_{\text{ev}} < w$; the promise is the base case. Depth d: operator P's watermark is min over inlets of watermarks emitted by depth-$< d$ nodes. When P closes $[a, b)$, its local watermark passed b, so every inlet's upstream node had emitted watermark $\geq b$, so (induction) every record with $t_{\text{ev}} < b$ destined for P had been emitted upstream — and FIFO channels deliver emissions before the subsequent watermark's arrival at P. Hence all of $[a, b)$'s records precede closure. With heuristic sources, any record violating closure was late at its source (it arrived behind an emitted watermark there — the violation does not arise in transit, by the same induction), so downstream violations inject no new failures beyond the sources' confessed rate. The failure-detector reading: completeness is watermark progress (add idle-source timeouts, or a stalled inlet freezes the minimum estate-wide); accuracy is the promise's honesty — and as with Chapter 5's detectors, one buys accuracy with waiting, here explicitly, by widening the source's lateness allowance. □

The stall, and the paperwork downstream. The proposition's parenthesis names the instrument's known failure mode: one idle source partition, one stuck consumer, and every downstream window freezes — totals never finalizing, state never released, the pipeline healthy by every throughput metric and dead by the only one that matters; whence idle-source timeouts, watermark alarms (watermark lag, per input, is the

first graph a streaming operator watches), and the reviewer's question that goes with them: what advances your watermark when nothing arrives, and who is paged when nothing does? (Problem 10.4 drills both.) Late data have downstream manners too: a refined total supersedes its predecessor, so the sink must treat window results as keyed updates — the *output* wants a compacted log as well; honesty about time, once admitted anywhere, propagates its paperwork to the end. The ghosts of Chapter 2 are not exorcised; they are *employed*: event time for truth, processing time for freshness, the watermark brokering between them — a portfolio of nows, each labelled, each monitored, none pretending to be the one true clock the world declined to provide.

10.4 Part Four — Exactly-Once, Prosecuted

The injury before the law. Bertie's pipeline reads a payment instruction from the log, executes the wire transfer, and crashes before the cursor is committed; the restart resumes from the last committed cursor, reads the same record, and the machinery, blameless and obedient, executes the transfer again. That is *the double payment* — not lost data but repeated effect, a non-idempotent side effect re-executed on replay (Problem 10.7) — and no phrase in the industry's brochures has caused more confusion per syllable than the one that claims to prevent it. The charge sheet distinguishes two defendants that marketing insists on conflating.

Exactly-once delivery, dismissed. The claim that a message arrives at its recipient once, as a property of the channel. The prosecution's oldest witness: the Two Generals, subpoenaed from Chapter 1. The final acknowledgment of any conversation may be lost; the sender who has not heard it cannot distinguish a delivered message from a drowned one, and must choose, forever, between resending, which risks twice, and desisting, which risks never. Exactly-once delivery over a fair-loss network, in the presence of crashes, is impossible in exactly the sense Chapters 1 and 3 made precise; every vendor claiming it as a transport property is selling a phrase the mathematics does not underwrite. Case dismissed.

Exactly-once effect, acquitted. However many times the message arrives, the state of the world advances as if it had been processed once — achievable, routinely achieved, and built from exactly three materials, all already on the course's shelf:

- **idempotence:** make the operation absorb repetition — set the balance rather than add to it; upsert the row keyed by the record's identity; add to a set that ignores duplicates — Chapter 7's algebra hired as plumbing. The craft is the rewrite of verb into fact: "increment the count" resists idempotence; "record that event seventeen occurred, and let the count be the recorded set's size" embraces it — CALM's re-drafting-toward-accumulation, practised as a plumbing discipline;
- **deduplication:** when idempotence is unavailable, make the processing remember — unique identifiers and a *dedup ledger* of those already applied, consulted before applying. Three clauses of fine print, each a production scar: the ledger persists *atomically with the state it protects* (persisted elsewhere, it merely relocates the double-payment window); the check-then-apply is atomic against concurrent duplicates (a uniqueness constraint or a single-writer partition under the ledger); and the ledger's window outlasts the retry horizon (remember an hour against a day of retries, and the day's tail replays as fresh business). Sizing is rate times identifier size times horizon — usually gigabytes, cheaper than the apology (Problem 10.1);
- **transactional sinks:** commit the effect and the bookmark together — the output rows and the new cursor in one transaction, Chapter 8's machinery at last earning its keep in this hall. Crash before the

commit: neither happened, and replay performs both once; crash after: both happened, and replay resumes beyond the record. The window has no interior, because the two effects were never two.

The lemma beneath all three:

Lemma 10.2 (idempotence and deduplication)**.** *Let records with unique identifiers be applied to a state under at-least-once delivery with retry horizon T. The final state equals the exactly-once state if either: (i) the application function is* idempotent *— applying a record twice, in any position among other applications, equals applying it once (sufficient condition: the state is a function of the* set *of applied records, not their multiset or arrival order; or the record's application overwrites a register keyed by its identity); or (ii) a* dedup ledger *of applied identifiers is consulted before and updated atomically with each application, persisted with the state, and retains each identifier for at least T after first application. Both clauses fail if the ledger is separately persisted (a crash between state and ledger re-opens the window) or expires before T (the tail replays as fresh).*

Proof of Lemma 10.2. (i) Set-function form: the final state depends only on the set of applied records; at-least-once delivery applies a superset of multiplicities over the same set; equality follows. Keyed-overwrite form: the last application of record r overwrites the register at key $\mathrm{id}(r)$ with the same value every time (determinism), so the final register bank matches the once-each execution regardless of repetition or interleaving. (ii) Induction on delivery events. Invariant: after any prefix, (state, ledger) equals the exactly-once pair for the set of identifiers in the ledger. A fresh identifier passes the check; the atomic (apply + enrol) step preserves the invariant. A repeated identifier within T of its first application is in the ledger (retention clause) and is discarded, preserving the invariant. No repeat arrives after T (horizon clause). Atomicity is load-bearing twice: a crash between apply and enrol would readmit the identifier (invariant broken — the separately-persisted-ledger failure), and two concurrent copies must serialize on the enrol (the single-writer clause). Both failure modes are exhibited in Problem 10.1(d). □

The materials, boxed for the design review, with the fencing the producers need and the border rule:

Protocol — Box 24. The exactly-once sink materials (distilled from the trade's standard practice and the Kafka design)

1: **idempotent sink:** effects keyed by record identity; apply is overwrite; replays converge (Lemma 10.2(i))
2: **dedup sink:** check ledger; apply and enrol *in one atomic step*, persisted with the state; retain past the retry horizon (Lemma 10.2(ii))
3: **transactional sink:** effects and cursor advance in one transaction; commit on checkpoint completion (two-phase between engine and sink where the sink is external — Chapter 8, redeployed)
4: **producer fencing:** persistent producer identity with epoch; brokers reject stale epochs — the zombie officiant refused (Chapter 5's token, Chapter 8's uniform)
5: **border rule:** any effect outside the wrapped jurisdiction (mail, payments) needs its own material at the end — the end-to-end argument, arriving formally in Chapter 11

Kafka's transactions. All three materials and one old friend assembled. A producer enrolls with a persistent identity and receives an *epoch* — Chapter 5's fencing token in a new uniform: a resurrected or duplicated producer instance carries a stale epoch and is refused by the brokers, the zombie fenced at the door. Idempotent production — sequence numbers per partition, brokers discarding repeats — makes append-at-least-once into append-once. And the transaction wraps a batch of appends across partitions *together with the consumer's cursor advance* into one atomic unit: a transaction coordinator, prepare-and-commit markers written into the partitions — Chapter 8's two-phase ceremony conducted entirely inside

the log, the markers playing the court of record, the epochs fencing the officiants. The result is exactly-once *effect* within the log's jurisdiction, at a price in the usual currencies: a coordinator conversation per transaction, and a visibility lag for read-committed consumers, who must wait at the oldest transaction still open rather than drink from the head — one dawdling transaction holds back every record appended behind it, however committed; nothing in Chapter 8 got cheaper by moving indoors. The honest default: for the great majority of pipelines — metrics, views, caches, anything idempotent by nature or by the rewrite — plain at-least-once delivery with idempotent application is simpler, faster, and exactly as correct; the transactional apparatus is for the money paths, and knowing which of your paths are money paths is the actual engineering. The jurisdiction's border is where this chapter's sabotage lives: the moment an effect steps outside the log — an email sent, a payment initiated — the guarantee stops at the threshold (Problem 10.7 exhibits it wire by wire). Under the border glints the *end-to-end argument*, named formally in Chapter 11: a guarantee manufactured in the middle of a system extends only as far as the middle reaches, and correctness at the ends — the sink that pays, the mailer that sends — must be arranged at the ends, whatever the middle promises.

10.5 Part Five — The Photograph of the River

The set piece, and the payment of the register's oldest debt. A streaming job of the modern kind — Flink is the reference house; the mechanism is Carbone and colleagues, 2015 — runs as a dataflow: Bingo, the *source*, reads the partitioned log; Tuppy, the counting *operator*, keeps running totals per key — *state*, the accumulated memory of everything processed; Gussie, the *sink*, writes results onward. State is what separates this hall from Part One's: a batch task was pure and stateless, but Tuppy's totals accumulate from a job begun weeks ago — recompute-from-genesis is a recovery plan measured in days of replay, past the log's retention — which is why streaming state must be *photographed*. The question the machinery must answer: if Tuppy's machine dies at half past four, how does the job resume without either losing the afternoon or counting it twice? The stage, formally:

Definition 10.3 (dataflow job; epochs)**.** Per the model box, a job has sources S, operators, sinks; each source's input is a partitioned log. Checkpoint k is initiated by injecting *barrier* b_k into every source stream; barriers travel in-band. A record belongs to *epoch* k if it was emitted after b_{k-1} and before b_k at its source; FIFO channels preserve the record/barrier order per channel.

The naive answers, failed instructively. Pause the world and copy the state: the estate has no pause — Chapter 2 settled that. Snapshot each operator on its own schedule, then, and the failure deserves its numbers: Bingo photographs his bookmark at offset one thousand; Tuppy photographs five minutes later, his totals already containing offsets to one thousand two hundred. Restore both, rewind Bingo, replay: two hundred records counted twice, and no amount of care at either machine alone fixes it — the defect is *between* them, in the photographs' disagreement about which instant they depict.

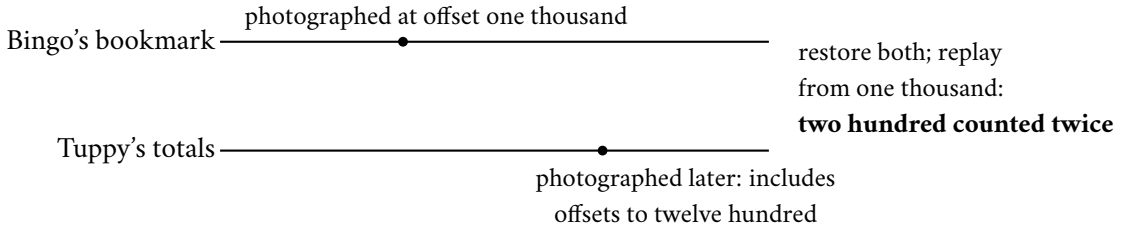


Exhibit 10.2 (the inconsistent snapshot). Photographs on private schedules disagree about which instant they depict; the recovered world double-counts the difference. The defect is *between* the machines — exactly what Def. 10.4 forbids and alignment repairs.

What is wanted is a *consistent* photograph — one logical instant across machines that share no clock; formally, checkpoint k as source offsets plus operator states at the epoch boundary:

Definition 10.4 (consistent snapshot)**.** A *snapshot* of a job is: per source, an offset vector; per operator, a state. It is *consistent at* k if each source offset is exactly the position of b_k, and each operator's state equals the state reached by processing precisely the epoch-$\leq k$ records on every input, in per-channel order — no epoch-$> k$ record absorbed, no epoch-$\leq k$ record missing. (This is a consistent cut in Chapter 2's sense: the frontier of events at the barrier.)

The mechanism: barriers and alignment. The episode stages it at full ceremony; the moves for review:

- **inject:** the job manager, on a timer, drops a *barrier* — a marker record stamped with the checkpoint number; picture a coloured card in the stream — into every source; the source records its offsets and forwards the card downstream, in-band: everything ahead of the card is the old epoch, everything behind it the new — a moving frontier in the river;
- **align:** an operator with several inlets, on the card's arrival at the first, does not yet snapshot: it holds that inlet — buffering its new-epoch records — and drains the other inlets' old-epoch records into its state until the card arrives there too;
- **photograph:** cards on all inlets — the old epoch entirely absorbed, the new untouched — the operator photographs its state asynchronously, forwards the card, releases the buffers, resumes; the sink does likewise and acknowledges;
- **complete:** every photograph landed in stable storage — one coherent picture of sources' bookmarks, operators' totals, and sinks' positions, a single logical instant no wall clock anywhere witnessed.

Alignment's price is a deliberate, bounded backpressure — the held inlet's upstream fills its buffers; Chapter 11 treats the healthy refusal of work as a first-class discipline — and the modern engines offer an *unaligned* variant that photographs the in-flight records themselves rather than waiting: a fatter photograph for a shorter pause, and, as the reveal will note, a restoration of the original algorithm's channel recording with almost comic fidelity.

Protocol — Box 25. Asynchronous barrier snapshotting (checked against Carbone et al. 2015)

1: **initiate** k**:** job manager injects barrier b_k into every source; each source records its offsets, forwards b_k downstream
2: **operator, on** b_k **from inlet** c**:** mark c aligned; buffer (do not process) subsequent records from c
3: continue processing records from unaligned inlets
4: **when all inlets aligned:** photograph state asynchronously to stable storage; forward b_k on all outlets; release buffered records; resume
5: **sink, on all inlets aligned:** photograph (positions, pending effects per Box 24); acknowledge k
6: **checkpoint** k **complete** when all acknowledgments arrive; retain last complete; discard failed/-partial

7: **recover:** restore all states from last complete k; rewind sources to recorded offsets; replay
8: **unaligned variant:** photograph immediately on first barrier, *including* in-flight records of unaligned inlets — channel recording restored, pause traded for photograph size

Alignment applies bounded backpressure (Chapter 11's discipline, previewed); the interval dial reads: how much river you will re-drink, against how often you photograph.

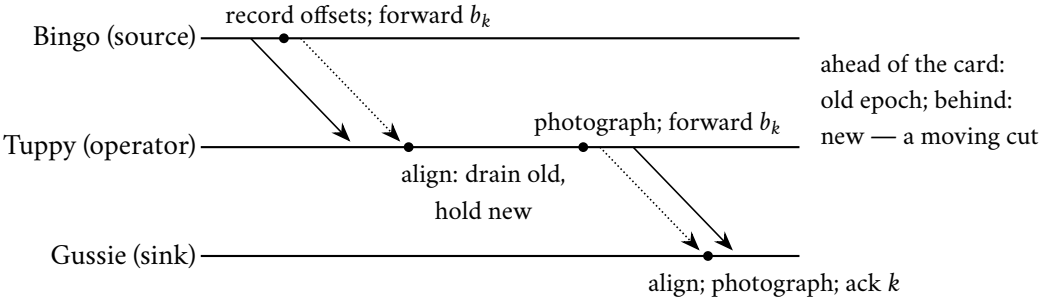


Exhibit 10.3 (the barrier walk). The card flows in-band (dotted); records (solid) ahead of it are old epoch, behind it new; each operator photographs precisely at the frontier (Box 25, Thm. 10.3). Compare Chapter 2's marker figures line for line: the correspondence table is drawn from this picture.

Correctness, the skeleton before the proof: barriers flow with the records, so FIFO channels deliver to every operator a stream cleanly cut into epochs; alignment makes each operator absorb all of epoch k and none of epoch k+1 before photographing; the union of the photographs is a consistent cut — the marker rule, verbatim.

Theorem 10.3 (barrier snapshot correctness; after Chandy–Lamport)**.** *Under the model box, Box 25's aligned protocol terminates for every checkpoint k, and the recorded snapshot is consistent at k (Def. 10.4).*

Proof of Theorem 10.3. Termination. The graph is acyclic and finite; barriers are broadcast on every outgoing channel upon completion of alignment; induction on topological depth: sources emit b_k immediately on injection; an operator at depth d receives b_k on each inlet from depth $< d$ nodes (which emit it, by induction, after finitely many records — FIFO channels deliver it after the finite epoch-k traffic), completes alignment after draining finitely many records, photographs (asynchronously; completion is bounded by stable-storage availability), and forwards. Sinks acknowledge; the checkpoint completes.

Consistency. Fix operator P and checkpoint k. On each inlet c, FIFO order means every record before b_k on c is epoch $\leq k$ and every record after is epoch $> k$ (induction along the path from the source: sources emit epoch-$\leq k$ records precisely before b_k; an upstream operator, by its own alignment discipline, emits its epoch-$\leq k$ *output* before forwarding b_k — outputs caused by epoch-$\leq k$ inputs are produced during alignment or earlier, and the operator forwards b_k only after those emissions). During P's alignment, inlets on which b_k has arrived are held (their post-barrier records buffered, unprocessed); inlets awaiting b_k are drained — so at the photograph, P has processed exactly the pre-barrier prefix of every inlet: all of epoch $\leq k$, nothing of epoch $> k$. The recorded source offsets are the positions of b_k by construction. Hence every component of the snapshot agrees with Def. 10.4: the checkpoint is the consistent cut at the barrier frontier — Chapter 2's theorem, in the promised overalls. □

The reveal. Chapter 2 walked an algorithm for photographing a system that refuses to hold still: markers injected at a node, propagated along every channel; each process recording its state on first marker; channel

contents captured between marker and marker; the whole a consistent cut. Chandy and Lamport published it in 1985; the course has kept it on the shelf, under a promissory note eight chapters old, ever since. *The asynchronous barrier snapshot is the Chandy–Lamport algorithm in industrial dress.* The tailoring amounts to two alterations, both set as the correspondence's exercise (Problem 10.10): the dataflow is acyclic, so markers flow one way and terminate at the sinks; and where the original recorded the channels, the industrial edition may instead rewind the source — the log's replayability standing in for channel capture (and the unaligned variant, forced to photograph in-flight records after all, restores the channel-recording clause verbatim, closing the circle). Alterations of dress; the theorem inside is the 1985 theorem, unchanged. The historical note sweetens the payment: Chandy and Lamport wrote it for the analysts of the eighties — detecting termination, catching deadlocks, debugging — and the notion that it would one day be the beating recovery heart of planetary accounting pipelines appears nowhere in the paper and everywhere in the industry. Forty years from theorem to overalls, and the overalls fit without alteration. Debt #6 — the register's oldest — presented, and paid in full.

The correspondence, structured. The reveal, clause by clause; Chapter 2's algorithm on the left, the industrial edition on the right.

Chandy–Lamport (1985)	**Barrier snapshots (2015)**
marker, injected by initiator	barrier b_k, injected at every source by the job manager
marker propagates on every outgoing channel	barrier broadcast downstream, in-band, per channel
process records state on first marker	operator photographs after *alignment* (all-inlet barriers)
channel contents recorded between markers	aligned variant: drained into state before the photograph; unaligned variant: recorded verbatim — the original clause restored
consistent cut: no effect without its cause	epoch coherence: all of epoch $\leq k$, none of $> k$ (Def. 10.4)
arbitrary strongly-connected topology	acyclic dataflow; markers terminate at sinks
channel logging for recovery	rewindable log sources replace channel replay

Recovery is then almost anticlimactic: restore the cut, rewind the sources to its offsets, replay — determinism makes the recomputed suffix equal the lost one, and the sink materials convert re-emission into exactly-once effect.

Corollary 10.4 (recovery exactness)**.** *On failure, restore all operators from the last complete snapshot k, rewind every source to its recorded offsets, and replay. The resulting execution's states coincide, record for record, with a failure-free execution's; consequently (i) no input record's effect on state is lost or duplicated, and (ii) with sinks made idempotent, deduplicating, or transactional per Lemma 10.2 and Box 24, emitted effects are exactly-once. Without such sinks, effects between snapshot k and the failure are re-emitted — the double payment of Problem 10.7.*

Proof of Corollary 10.4. Restore and rewind place the job in exactly the Def. 10.4 configuration: states as of the cut, sources at the cut's offsets. The replayed suffix of each source log is byte-identical to the original suffix (immutability, Def. 10.1); channels are FIFO; operators are deterministic per the model box; so by induction on processing steps (per operator, per channel prefix), every operator's state trajectory after restoration coincides with the failure-free trajectory after the cut. (i) follows: each input record's effect is applied exactly once relative to state, since the cut contained precisely the pre-cut effects and the replay applies precisely the post-cut records. (ii) Re-emissions to sinks during replay carry the same identifiers as

the lost originals; Lemma 10.2's materials convert them to exactly-once effect. Without those materials, any sink effect performed between the cut and the crash is performed again — Exhibit 10.4's wire transfer. □

With numbers, and the trinity. Checkpoints every minute; a crash at four-forty restores the four-thirty-nine photograph, rewinds the sources one minute, and re-drinks sixty seconds of river at replay speed — the log serving sequential reads at the disk's happy pace — neither losing the minute nor counting it twice; Box 25's closing line states the interval's dial. But the photograph alone recovers nothing; the recovery doctrine is a trinity:

- **replayable input:** the log, retaining and rewindable, so the past can be re-read;
- **deterministic-enough compute:** operators whose replay converges to the same totals — time taken from the record and the watermark, never the wall; randomness seeded by record identity; external calls fenced behind Part Four's materials;
- **snapshotted state:** the consistent photograph, so replay begins from coherence rather than from the beginning of time.

Name all three in a design review or own none of them: snapshots without replayable input restore a memory of a world they cannot revisit; replayable input with nondeterministic compute replays into a different world; both without the photograph replay from genesis. Each leg is a chapter of this course wearing overalls: the log, the determinism diet, the photograph. And the lineage's recovery models, laid side by side, turn out to be one doctrine:

Remark 10.5 (lineage recovery, compared)*.* MapReduce persists every stage's output (replicated) and re-runs individual tasks: recovery cost is re-execution of the lost task alone; the price is paid up front, every stage, disk and replication. Spark persists nothing by default and records recipes: recovery recomputes the lost partitions' lineage back to stable input — cheap for narrow dependencies (one ancestor partition per lost partition), expensive past a shuffle (all ancestor partitions), whence checkpointing to truncate lineage at wide waists. The streaming engine of Thm. 10.3 is the third point of the triangle: state that lineage cannot cheaply regenerate (weeks of accumulation) is photographed instead. One doctrine spans all three — replayable input plus determinism makes recomputation a substitute for redundancy — with the systems differing only in what they materialize.

10.6 Part Six — Architectures, Seams, and the Limits of the Church

The chapter's machinery, deployed at architectural scale — and then the borders of the parish, stated plainly lest the church of the log acquire converts beyond its writ.

Lambda and Kappa. When batch was trusted and streams were new, the *Lambda architecture* ran both: a batch layer recomputing truth nightly from the master dataset, a speed layer approximating the last few hours, a serving layer stitching the two — every metric computed twice, in two codebases, by two teams, disagreeing in exactly the ways two implementations of anything disagree; month-end arrives, the batch number and the stream number differ, and the answer to which is wrong is, reliably, both. The fashion had a reason: the early stream processors genuinely could not be trusted with correctness — no consistent photographs, loose delivery discipline — and the batch layer was the apology, recomputed nightly because the daytime arithmetic was known to leak. The *Kappa architecture* — Kreps again, coining wryly — observes that once the stream processor acquired this chapter's machinery, the photograph and the trinity, the apology became redundant: keep one pipeline, and when logic changes, *replay the log* through the new

code into a new view — the Friday-poison manoeuvre of Part Two, promoted from incident response to standing architecture. The audit of any proposed architecture is now one question: is this second codebase an apology for a limitation that still exists?

Change-data-capture, and the outbox. Every serious database already keeps a log in the cellar — the write-ahead log that Chapter 6's replicas drink from — and *change-data-capture* taps it: the committed changes read in commit order and published outward, an initial snapshot stitched to the tail of changes so a new consumer can build the whole table and then stay current — the database's most private structure promoted to its most public interface. The motivating bug is the *dual-write anomaly*, Chapter 8's lesson ignored at small scale: a service commits a row to its database *and* announces the event to the estate — two effects, two systems, no transaction spanning them. Commit-then-announce, crash between: the order exists, the announcement never fired, and the warehouse never hears of an order the customer can see — an orphan. Announce-then-commit: an event for an order that does not exist — a ghost, and a warehouse packing a phantom. No retry discipline fixes either. The *outbox pattern* repairs it with the chapter's whole philosophy in miniature: write the announcement *into the database itself* — an outbox row committed atomically with the business row, one transaction, one system, atomicity where it is cheap — and let a relay tail the change-data-capture stream, carrying outbox rows into the public log at-least-once, deduplicated downstream by Part Four's materials. The skeleton: one local transaction plus an at-least-once tail with downstream dedup equals a ghost-free, orphan-free announcement stream:

Proposition 10.5 (outbox correctness)**.** *Let a service commit business rows and announcement rows in one local transaction (the outbox), and let a relay read the database's commit log in order, publishing each outbox row at-least-once to a log, with consumers deduplicating by announcement identifier (Lemma 10.2). Then the published announcement stream, after dedup, contains exactly one announcement per committed business event: no ghosts (announcements for uncommitted business), no orphans (committed business without announcement), in commit order. Dual writes — announcing outside the transaction — admit both ghosts and orphans; the crash windows are exhibited in Problem 10.5.*

Proof of Proposition 10.5. No ghosts: an announcement row exists only if its transaction committed (atomicity of the local transaction), and the relay reads only committed log entries; dedup collapses the relay's at-least-once repetitions to one. No orphans: a committed transaction's outbox row is in the database's commit log; the relay reads the log in order and publishes every entry, at-least-once, so the announcement eventually appears (relay crashes resume from its own cursor in the commit log — replayable input, once more). Order: the commit log's order is the publication order. Dual writes fail both ways: commit- then-announce loses the announcement to a crash between (orphan); announce-then-commit publishes a ghost when the commit fails — the two crash windows of Problem 10.5(a). □

The borders of the parish. The log's three refusals of ministry:

- **no queries:** finding, filtering, joining the current state is the business of tables, indexes, and Chapter 7's stores — the log feeds them, never replaces them; the read patterns (sequential and total against selective and immediate) do not convert, and the inside-out doctrine says *derive* the filing cabinet from the spinal cord, never abolish it;
- **no arbitration:** appending is not deciding; two producers racing to append conflicting claims have recorded their conflict in order, not resolved it — uniqueness, locks, the race test's whole domain remain Chapter 4's business;

- **no multi-key invariants:** an invariant across two streams needs Chapter 8's machinery, or a redesign per Chapter 7, exactly as before — the doctors and the officiant are not dismissed by publishing their rows as events.

The log is the estate's circulatory system — magnificent at moving truth around the body, no substitute for the organs.

10.7 The Whiteboard

For the design review.

1. Order is a per-partition commodity: key partitions to co-locate must-be-ordered records; the partition count is the ordering granularity and the consumer ceiling in one integer — the most consequential integer in the deployment file, set, as usual, on day one. Most global-order demands dissolve under *ordered for whom*.
2. Exactly-once is idempotence engineering, not delivery magic: delivery is at-least-once, by theorem — the Two Generals hold the copyright; effect is exactly-once by materials — idempotent operations, persistent dedup ledgers with sized windows, transactional sinks, fenced producers. Ask which material a brochure means, and where the jurisdiction's border runs: the money paths cross borders by profession, and the double payment lives precisely at the crossing.
3. Recovery is a trinity — replayable input, deterministic-enough compute, snapshotted state; name all three or own none, and audit retention against the outage: a log kept for three days and a bug discovered on the fourth is a trinity with a missing leg, discovered at the worst possible hour.

The register. A ceremony of settlements. PAID: #6 — the photograph in its industrial costume (issued in Chapter 2, the register's oldest resident, retired with the reveal at forty years' maturity). #15 — the log's stardom (issued Chapter 5, paid with a whole chapter's billing). #21 — partitioned computation (issued Chapter 9, paid twice over: the shuffle's move-code-to-data, and batch-and-stream as two speeds of one flow). ISSUED: #23 — the retry's dark side, armed for Chapter 11: every recovery in this chapter leaned on retries and replays, and Chapter 9's stampedes already hinted the retry is also the estate's favourite way of attacking itself. #24 — backup tasks: racing duplicates against the unluckiest machine, filed open for the finale, where the tail arithmetic returns at the scale of ten thousand accelerators and a gradient every few seconds. With the ceremony the season's oldest promissory note has cleared: nothing now stands open that was issued before Chapter 5. EPISODE WORD COUNT: 10,017 — above the floor; the half-draft-by-design drill (LEDGER §5) held.

10.8 Problems

Tier I — Calisthenics

Problem 10.1 (the dedup ledger, sized)**.** A pipeline processes five thousand records per second; identifiers are sixteen bytes; upstream retries persist for at most six hours. (a) Size the ledger's window and memory honestly (with an index overhead factor of your choosing, stated). (b) The operators propose one hour of retention "to save memory"; write the two-sentence objection naming the exact failure. (c) State where the ledger must live relative to the state it protects, and cite the lemma clause. (d) Exhibit the two-step crash that defeats a ledger persisted in a separate transaction.

Problem 10.2 (cursor drills)**.** A topic retains seven days; a consumer group's cursor stands at Friday noon. For each event, state what is possible, impossible, and advisable: (a) a bug shipped Friday noon is discovered Monday; (b) the group is down for maintenance for two days; (c) an analyst wants totals from last month; (d) a second group starts fresh at offset zero on a compacted topic — what exactly does it see, tombstones included?

Problem 10.3 (idempotent or not)**.** Classify under replay, and re-draft the resisters where possible (verb into fact): (a) set customer status to `gold`; (b) increment login count; (c) append line to invoice; (d) send welcome email; (e) upsert row keyed by event identifier; (f) debit account by ten pounds.

Problem 10.4 (watermark propagation)**.** An operator joins three inputs whose watermarks read seven fifty-eight, eight-oh-two, and seven-forty. (a) The operator's watermark, and which window closures are licensed. (b) The third input's partition has been idle an hour; what happens to every downstream window, and name the two standard remedies. (c) One input is heuristic with a confessed one-percent late rate; bound what closure of the eight o'clock window may miss, and say who chose that number.

Tier II — The Real Thing

Problem 10.5 (the outbox, argued end to end)**.** (a) Exhibit both dual-write crash windows (orphan and ghost) with explicit crash points. (b) Prove Prop. 10.5's no-ghost and no-orphan clauses in your own hand, marking where atomicity, log order, and dedup are each load-bearing. (c) The relay crashes mid-publish; trace recovery and name which member of the trinity does the work. (d) A reviewer proposes deleting outbox rows after publishing "to keep the table small"; state what replaces the deletion safely and why deletion-before-confirmed-publish re-opens a window.

Problem 10.6 (alignment's necessity)**.** Modify Box 25: the operator photographs on the *first* barrier and keeps processing all inlets. Exhibit the resulting inconsistency with a two-inlet counting operator and four records (state the totals photographed versus owed); identify which clause of Def. 10.4 dies; then explain in two sentences why the *unaligned* variant (which also snapshots on first barrier) nevertheless recovers correctly, and what it records that the broken variant does not.

Problem 10.7 (sabotage: the double payment)**.** A pipeline reads payment instructions, calls the bank's wire API (non-idempotent, no request identifier), then advances its cursor; checkpoints every minute; the vendor's brochure says "exactly-once semantics end to end." Exhibit the fatal execution: the checkpoint, the transfer, the crash, the replay, and the second wire — timestamps and offsets supplied. State which side of the jurisdiction border the wire lives on, which member of Box 24 was absent, and give the two repairs (an idempotency key at the bank; a transactional pending-transfers outbox with the bank driven from it) with one sentence each on their residual assumptions.

Problem 10.8 (recovery models compared)**.** A ten-stage pipeline loses one machine at stage seven. Compare, with the arithmetic sketched: (a) MapReduce (all intermediates materialized, replicated); (b) Spark, no checkpoints, stages three and eight are shuffles; (c) Spark checkpointed after the stage-three shuffle; (d) a streaming job with minute checkpoints. For each: what is re-read, what is recomputed, what was paid in advance, and which trinity leg is doing the work.

Tier III — For the Ambitious (starred)

Problem 10.9 (⋆ watermark design under a distribution)**.** Lateness is observed distributed as: ninety percent of records within one second, ninety-nine within ten, ninety-nine point nine within two minutes, a

heavy tail to hours (the tunnels). Design the watermark and lateness policy for (a) a billing aggregate that must be complete to five significant figures, and (b) a live dashboard refreshed every five seconds. For each: the watermark delay, the allowed lateness, the correction policy, and bounds on error and latency, derived from the distribution. *(Hints only.)*

Problem 10.10 (★ unaligned equals channel recording)**.** Formalize the unaligned variant: photograph on first barrier, record in-flight post-barrier records per inlet as part of the checkpoint. Prove its checkpoint is consistent in a suitably extended sense (state + recorded channels reconstruct Def. 10.4's cut), and exhibit the exact correspondence to Chandy–Lamport's channel-recording clause. *(Hints only.)*

10.9 Hints

Hint (Problem 10.6)**.** Let inlet one's barrier arrive, then a post-barrier record from inlet one is processed before inlet two's barrier: the photograph contains an epoch-k+1 effect. On replay that record is replayed too.

Hint (Problem 10.7)**.** Checkpoint at four-thirty-nine, offsets to nine hundred; wire fired for offset nine-fifty at four-forty; crash before the next checkpoint. Replay re-reads nine-fifty.

Hint (Problem 10.9)**.** (a) The five-figure clause fixes the tolerable miss rate; read the delay off the distribution's quantile, and let allowed lateness plus corrections absorb the tail rather than the watermark. (b) Invert the priorities: a short delay, wide lateness irrelevant — the dashboard's next refresh is its own correction.

Hint (Problem 10.10)**.** Define the cut at each operator as (photographed state, recorded in-flight records); replay = restore state, replay recorded records first, then the rewound sources. The recorded records *are* Chandy–Lamport's channel contents.

10.10 Solutions

Tier I in full (tersely); Tier II in full — Problem 10.7 is the sabotage certification; hints only for Tier III.

Solution (to Problem 10.1)**.** (a) Six hours at five thousand per second is one hundred eight million identifiers; sixteen bytes each is roughly one point seven gigabytes raw; with a factor-of-two index overhead, three and a half gigabytes — disk-resident, cheap. (b) "Retries persist for six hours; a ledger remembering one means a duplicate arriving in hours two through six passes the check and is applied as fresh business — the window must outlast the horizon, or it is theatre." (c) Atomically with the protected state, in the same transaction domain (Lemma 10.2(ii)); a ledger elsewhere merely relocates the crash window. (d) Apply effect; crash before enrolling; restart; the identifier is unknown; the effect applies again — the double payment in miniature.

Solution (to Problem 10.2)**.** (a) Possible and advisable: rewind to Friday noon, replay through fixed code (the Friday-poison manoeuvre); the four days of retention headroom made it possible. (b) Nothing is lost; the group resumes at its cursor; the only casualty is lag, monitored. (c) Impossible from this topic — retention is seven days; last month lives in the archived tier or a derived table; the trinity's first leg has a hem. (d) Current state per key in log order plus tombstones for keys deleted-but-not-yet-cleared: replaying yields exactly the live table, deletions honoured; it must treat a tombstone as a delete, not a record.

Solution (to Problem 10.3)**.** (a) Idempotent (overwrite). (b) Resists; re-draft: record login event by identifier, count is the set's size. (c) Resists; re-draft: upsert line keyed by (invoice, event identifier). (d) Resists and cannot be re-drafted inside the pipeline — an external effect; needs Box 24's border rule (idempotency key at the mailer, or an outbox the mailer drains). (e) Idempotent by construction. (f) Resists; re-draft as (e): a posting row keyed by transfer identifier, balance as the sum — the ledger discipline that predates computing, sir, for this exact reason.

Solution (to Problem 10.4)**.** (a) Seven-forty, the minimum; only windows ending at or before seven-forty may close. (b) Every downstream window freezes — no totals finalize, state accumulates; remedies: idle-source timeout (advance the idle partition's watermark by declaration) and watermark-lag alarms on the inlet. (c) At most the confessed one percent of the window's records, in expectation, arriving late and handled by the lateness policy; the number was chosen by whoever accepted the heuristic source's configuration — which is the honest answer's point: the error budget is an *input*, someone's signature is on it.

Solution (to Problem 10.5)**.** (a) Commit-then-announce: crash after commit, before announce — orphan (business without event). Announce-then-commit: crash after announce, before commit — ghost (event without business). (b) No-ghost: the outbox row shares the business transaction; the relay reads only the committed log; dedup absorbs relay repeats — atomicity kills the ghost, the log's committed-only property keeps it dead, dedup tidies the copies. No-orphan: commit implies the row is in the log; the relay's cursor advances only past published entries, so a crash resumes and republishes; at-least-once plus dedup lands exactly one. (c) Relay restarts at its cursor in the commit log and republishes the uncertain suffix; replayable input (the database's own log) does the work; consumers' dedup absorbs the seam. (d) Mark-published (or advance a cursor) instead of delete; deletion before confirmed publication makes the crash window an orphan factory — the row must outlive the uncertainty about its publication, which is the retry horizon again, wearing a table.

Solution (to Problem 10.6)**.** Counting operator, inlets A and B, all records worth one. Barrier b_k arrives on A after two records; the broken operator photographs "two" and keeps processing: takes one more record from A (epoch k+1) and one from B (epoch k), then B's barrier arrives. Owed at the cut: epoch-k records = two on A, one on B = three; photographed: two — and worse, on replay the sources resend A's epoch-k+1 record *and* B's epoch-k record, the former now double-processed relative to any consistent story, the latter missing from the photograph: the state is neither the cut before nor the cut after — no epoch boundary explains it; Def. 10.4's "all of $\leq k$, none of $> k$" dies in both clauses. The lawful unaligned variant also photographs early — but it *records the in-flight post-barrier records as data*, so recovery replays exactly the missing difference: what it records, and the broken variant does not, is the channel contents — Chandy–Lamport's original clause, which alignment merely replaced with draining.

Solution (to Problem 10.7 — the certification)**.** The execution. Checkpoints each minute; the last complete one at four-thirty-nine records cursor offset nine hundred.

(1) Four-forty: the pipeline reads offset nine-fifty — "pay the plumber two hundred pounds" — calls the wire API; the bank executes; money moves. No identifier accompanies the call; the bank knows only that a request arrived.

(2) Four-forty and ten seconds: the worker crashes — before the four-forty checkpoint completes. The cursor of record remains nine hundred.

(3) Recovery restores the four-thirty-nine snapshot and rewinds to offset nine hundred; replay reaches nine-fifty; the pipeline, deterministic and obedient, calls the wire API; the bank, seeing a fresh request, executes. The plumber is paid four hundred pounds; the pipeline reports a clean recovery; every internal guarantee held.

The wire lives *outside* the jurisdiction: the engine's exactly-once wraps state and cursors, and can wrap sinks that accept its materials; the bank's API accepted none. Absent from Box 24: everything — no idempotency key (line 1's material), no dedup at the boundary, no transactional coupling. Repairs: (i) idempotency key — send the payment instruction's identifier; the bank deduplicates; residual assumption: the bank's ledger retention exceeds the replay horizon (Lemma 10.2's window, now on the bank's side of the border). (ii) pending-transfers outbox — the pipeline's transactional sink writes a pending row (idempotent, keyed); a separate driver executes pending transfers and marks them done, itself idempotent against the bank via (i) — which reveals the honest truth that (ii) reduces to (i) at the last hop: at the border, someone must deduplicate, and the end-to-end argument (Chapter 11) says it must be the far end. Certified fatal as shipped: double payment under one crash and one replay, both routine; the brochure's "end to end" extended exactly as far as the vendor's own walls.

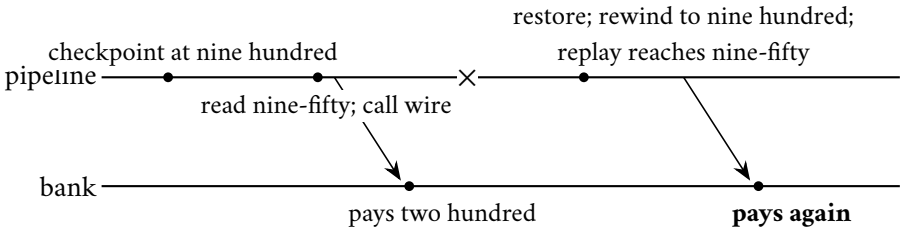


Exhibit 10.4 (the double payment). Every internal guarantee holds; the wire lives past the border, and the border does not dedup; the cross is the crash. The repair is an identifier the far end honours — correctness at the ends, arranged at the ends (Problem 10.7).

Solution (to Problem 10.8)**.** (a) Re-run the lost stage-seven tasks only; inputs read from stage six's replicated output; paid in advance: every stage's materialization and replication, every job, always. (b) The lost partitions at stage seven depend narrowly back to stage four; stage four's inputs cross the stage-three shuffle, a wide dependency, so every stage-three partition — and with it the whole prefix, stages one through three — contributes: recompute back to the last wide boundary and beyond, for want of a checkpoint. (c) The checkpoint truncates lineage at three: recompute stages four through seven for the lost partitions only; paid in advance: one materialization, chosen at the waist. (d) Restore last minute's photograph, rewind, re-drink sixty seconds; paid in advance: a photograph per minute, asynchronous. In every cell the trinity's legs shift weight: (a) leans on materialized input, (b) on determinism alone, (c) and (d) on the chosen mix — which is Rem. 10.5 as a bill of works.

10.11 The Reading

Kreps, "The Log: What every software engineer should know about real-time data's unifying abstraction," 2013. The chapter's text, assigned whole. The clearest statement of the thesis and a model of technical prose without apparatus; read the "logs and consensus" section against Chapter 5 and smile.

Dean and Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," OSDI 2004. Section three point six — backup tasks — above all: one page the finale will spend a chapter enlarging. The rest is the purity dividend, itemized.

Carbone, Fora, Ewen, Haridi, and Tzoumas, "Lightweight Asynchronous Snapshots for Distributed Dataflows," 2015. Read against your Chapter 2 notes with the correspondence table at your elbow; the

pleasure is watching the 1985 constructions acquire buffer pools. Chandy and Lamport (1985) itself, re-read after: the overalls come off and the theorem is still warm.

Zaharia et al., "Resilient Distributed Datasets," NSDI 2012. Lineage as recovery; the narrow/wide distinction and checkpoint truncation of Rem. 10.5 in the authors' hand.

The shelf. Ghemawat, Gobioff, and Leung, "The Google File System" (SOSP 2003) — failure made boring, and the single master's whole story. Akidau et al., "The Dataflow Model" (VLDB 2015) — watermarks, windows, and triggers in full generality. The Kafka design documentation — better literature than most proceedings; the transactions section against Box 24. Kleppmann, chapter eleven, and his "Turning the Database Inside Out" talk — the standing companion on this chapter's whole territory.

Chapter 11

The Engineering of Failure

Companion to Episode Eleven. The episode tells the story — the September 2015 DynamoDB outage that outlived its cause, the doppelgänger's last bow, the regiment on the bridge; this chapter keeps the record: the phi-accrual dial done honestly; the retry-amplification model with its exponent proved; the defused retry, the circuit breaker, and the request's lifecycle as protocol boxes; the deadline-propagation invariant; the metastable tipping point with the loop-gain criterion derived; and, per the season's Decision 8, the one starred appendix — an annotated TLA plus specification, with a checker to run and a protocol to break — contained here and nowhere else. Statements, proofs, price tags, problems.

11.1 Part One — Suspicion With a Dial

The paying chapter: five debts due, none issued, and the season's grammar inverted — ten chapters ran guarantee, price, theorem; this one runs death, mechanism, discipline. The thesis: *mature systems are rarely killed by their failures; they are killed by their reactions to their failures.* Timeout, retry, queue, health check — each installed in the name of resilience, and each, in this chapter, caught moonlighting as the murder weapon.

The overture: DynamoDB, 20 September 2015. From the published postmortem (the course walks only outages their owners have written up). The episode stages it in full; Part Four re-walks it with instruments; the skeleton:

- **The house:** DynamoDB — Chapter 6's shopping cart at planetary scale — keeps its partition assignments in an internal metadata service (Chapter 9's directory); the storage fleet renews its map of who holds what on schedule (Chapter 5's leases); a recently popular feature had made the answers substantially heavier.
- **The trigger:** a brief network disruption; a portion of the fleet misses its renewals — Chapter 1's weather, routine.
- **The reaction:** the servers ask again, in a synchronized clump; the heavier renewals outrun their timeouts and are asked again; servers that cannot renew prudently decline to serve on a stale map and take themselves out of service — asking more urgently still. Within minutes the metadata service is underwater, not with customer traffic but with the fleet's own requests for permission to work.
- **The sentence for which the chapter exists:** the network disruption ended almost immediately, and the outage continued for hours. The cause was cured; the illness sustained itself.

- **The recovery:** against every instinct of the trade, the operators *turned the fleet away*: throttled the renewals, added capacity behind the breathing room, readmitted gradually.

The timeout as wager. Begin where Chapter 1 left the wound open and Chapter 3 salted it: **crashed versus slow**, the confusion no protocol can abolish. Every "is it dead?" in an asynchronous world is answered by a timeout, and a timeout is not a measurement but a *wager* on the tail of the silence distribution: too short, and the doppelgänger collects — a slow-but-living machine condemned (Chapter 5's fencing makes the loss survivable); too long, and the estate idles behind a corpse. The numbers in the wild are archaeological; the discipline is to *derive rather than inherit* — a high percentile of observed latency plus margin, revisited when the distribution moves — and to nest within the caller's patience: a tier that waits eight seconds for a caller who abandons at two is conducting a seance. The timeout, the retry allowance, and Part Three's travelling deadline are one budget viewed from three chairs; budget it once, at the top, and subdivide downward.

The three species. Set by different logics; an audit that finds all three at "thirty seconds" has found not a policy but a superstition:

- **connection timeout:** prices a cheap, local question — an unreachable machine is best discovered in milliseconds; short.
- **request timeout:** prices the work itself; derives from the latency distribution; the one that must nest within the caller's budget.
- **lease:** Chapter 5's tenancy — a timeout with a title deed, whose expiry transfers authority rather than merely abandoning a request; longest of all, the fencing drill rehearsed.

The chapter's three toys, declared together before the first formal claim:

Model of this chapter

For phi-accrual (Def. 11.1). Heartbeats from a peer arrive with inter-arrival times drawn, in the healthy regime, from a stationary distribution F estimated over a sliding window; the detector reads the current silence against F's tail. Stationarity is the stated assumption; regime changes (the network's moods) re-estimate F with the window.

For the retry results (Props. 11.2–11.4). A chain of k tiers; a request at tier i issues one call to tier $i{+}1$, retrying on failure or timeout up to r total attempts; during the incident window the bottom dependency fails or times out every attempt (the worst case, stated as such: an honest toy, not a queueing model — its role is to bound the blast, not to predict the weather).

For the metastable toy (Def. 11.3, Prop. 11.6). One service of capacity C requests per second; offered customer load $L < C$; work rejected or expired is retried by clients after a timeout, adding reaction load R; degradation is the unserved fraction. Deliberately crude — one server, memoryless retries — and flagged so; the starred problem refines it.

The phi-accrual dial. The pricing insight, from Hayashibara and colleagues (2004): **stop returning a verdict; return a suspicion level**. The classical detector emits a boolean manufactured from a threshold it will not show you; the phi-accrual detector keeps, per peer, a running record of heartbeat inter-arrival times — that peer's observed rhythm in this weather — and answers with a number: how implausible is the current silence? The scale is logarithmic — phi of one, silences this long happen one time in ten; two, one in a hundred; three, one in a thousand — suspicion with a dial, the odds printed on the dial face.

Definition 11.1 (phi-accrual suspicion; Hayashibara et al.)**.** Let t be the time since the last heartbeat and F the estimated inter-arrival distribution. The suspicion level is

$$\varphi(t) \;=\; -\log_{10}\bigl(1 - F(t)\bigr),$$

the log-scaled improbability of a silence at least this long under the healthy regime. A consumer acts at its own threshold Φ; under stationarity, acting at Φ misfires (declares a live peer) on roughly the fraction $10^{-\Phi}$ of silences.

The drill, on the cast. Tuppy's heartbeats to Bingo arrive about every half second, with one-second stragglers and, once in the watch, a two-second garbage-collection pause. Tuppy falls silent:

- **one second:** phi reads low — silences of this length are Tuesday; nobody moves.
- **three seconds:** phi crosses the routing threshold; Bingo's router quietly stops sending Tuppy new work — a hedge costing nothing if wrong.
- **eight seconds:** phi crosses the failover threshold — silence this long has never been observed — and the heavy machinery engages: election, fencing, the doppelgänger drill standing by in case this was the pause of Tuppy's career.

Three thresholds, one evidence stream, each consumer paying for exactly the confidence its own blunder would cost. And the detector has quietly *learned the network*: congestion widens the rhythms and discounts the silences a quiet Sunday would flag. Problem 11.2 drills the calibration; the honest limits are the formal record:

Proposition 11.1 (dial properties)**.** *φ is nondecreasing in the silence t; thresholds order consistently (a higher-Φ consumer never acts before a lower one); and the false-alarm rate at fixed Φ is invariant under re-estimation of F — the dial re-prices itself with the weather, which a fixed timeout does not. What no dial changes: a pause and a death produce the same evidence at every Φ (Chapter 3), so downstream machinery remains fencing-obligated (Chapter 5); and per-peer dials cannot see correlated silence — fleet-level judgment sits above them.*

The price list. Chapter 5's suspicion services — sessions, leases, ephemerals, the gossip fleet's accusations — are subscription products on this wager; the bill has three lines: *heartbeat traffic*, a standing chorus under everything else; *lag*, for no detector may be both instant and honest — Chapter 3's theorem in an actuary's suit; and, if careless, *correlated panic* — the theatre below.

Health-check theatre, and gray failure. **Health-check theatre**: the check that measures the wrong thing, splendidly; the sabotage of the week (Problem 11.6) lives here. Two productions: the probe answered from a thread that does no real work — disks seizing, queue a mile long, verdict instantly "healthy"; and the deadlier inverse — the probe exercises the real path, and under load, precisely when every machine is slow-but-correct, the probes time out fleet-wide and the orchestrator, obeying its constitution, drains the healthy-but-busy onto the healthy-but-busier. Between the theatres lies **gray failure**, christened by Huang and colleagues: the machine healthy by every measure the *system* takes and broken by every measure the *customer* takes — the disk serving probes from cache while failing real reads. The definitional insight is **differential observability**: the checker and the customer are watching different things; the failure lives in the difference. The remedy is humility about probes: measure the paths customers travel, weight verdicts by customer-visible error, and read divergence between dashboard and complaint queue as the finding, not the noise.

The mercy bias. The design sentence: *a health check taken under load must be biased toward mercy, because the alternative converts load into membership churn — and membership churn into more load* (Chapter 9's rebalancing storm; Part Four's subject). The concrete implementation is the **disruption budget**: at most so many members lost per minute to health verdicts, whatever the probes claim — a probe panic drains the estate at a governed drip, not a gulp. Above the per-peer dials sits the fleet-level question: a death, or weather? The doppelgänger takes his final bow: crashed-versus-slow was never solved; it was *priced*, hedged, and dialled — which is what engineering does with the impossible.

11.2 Part Two — The Retry, Weaponized and Defused

The sweetest poison in the pharmacy. The retry is the honest child of Chapter 1 — the Post loses letters; asking again is the only remedy the charter permits. The crime is never the retry; the crime is the retry *unaccompanied* — without a schedule, without a conscience, without knowledge of its ten thousand siblings.

The amplification, in small numbers. The episode stages the three-storey establishment in full; the skeleton, beside Exhibit 11.1:

- **the stack:** Bingo's web tier calls Tuppy's service tier, which calls Gussie's storage; each tier, prudently, up to three attempts — each cap chosen at a different desk, in a different year.
- **the blip:** Gussie runs briefly slow — every request delayed, none failed.
- **the multiplication:** Tuppy's first calls time out; he retries: Gussie sees three requests where one was meant. Bingo's calls to the busy Tuppy time out; he retries in turn: **three at the middle floor, nine at the bottom** — a nine-fold fury for the component least able to bear it.
- **the fourth storey:** an edge proxy with its own three attempts: three, nine, twenty-seven at the bottom floor.

Independent per-tier attempts multiply, exponentially in stack depth, across teams who never met — no local review can catch it; the reform must be constitutional rather than corrective.

Proposition 11.2 (retry amplification)**.** *Under the model box, a single top-level request generates at most r^i attempts at tier i, and r^k at the bottom; with $r = 3$: one at the top, then three, nine, twenty-seven down the four-storey stack. If instead each attempt independently fails with probability p (partial brownout), the expected attempts at tier i are $\big((1-p^r)/(1-p)\big)^i \le r^i$ — already exponential in depth for any fixed p. Limits of the toy, stated: no queueing, no timeout interaction, no jitter; its office is the exponent, which survives every refinement.*

Proof of Proposition 11.2. Induction on depth. One top-level request is one attempt at tier zero. If tier i sees at most A attempts, each attempt issues at most r attempts to tier $i{+}1$ (the per-call retry cap), so tier $i{+}1$ sees at most rA. Hence r^i at tier i. For the expected form: an attempt sequence at one call site is a truncated geometric — it continues while attempts fail (probability p) up to r — with expected length $(1-p^r)/(1-p)$; independence across sites (the toy's assumption) multiplies expectations down the chain. Both bounds are tight in the stated regimes; neither models queueing — the exponent, not the constant, is the deliverable. □

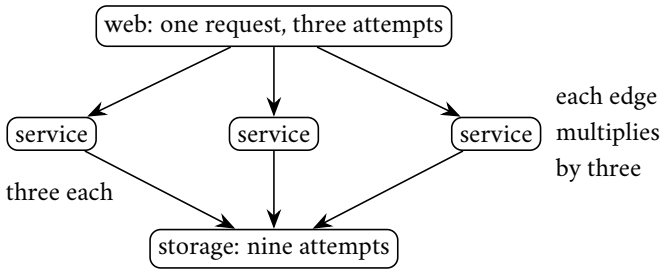


Exhibit 11.1 (the amplification tree). One request, three attempts per tier, two tiers of fan-down: nine arrivals at the floor least able to bear them (Prop. 11.2). Every tier reasonable, the building not; the budget (Prop. 11.4) is the constitutional repair.

The reforms, in the order the trade learned them, each one a scar.

Backoff, and the regiment. **Backoff**: do not ask again at once; wait, and wait longer each time — exponentially, so the fleet's ardour cools geometrically. Necessary, and famously insufficient: synchronized failure produces synchronized retries — ten thousand clients struck in the same instant back off by the same schedule and return in the same instant, a regiment marching in step across the bridge. Whence **full jitter** — the Brooker doctrine: randomize each delay across the full range from zero to the backoff ceiling; the regiment breaks step — area preserved, peak divided by the window, and it is the peak, never the area, that breaks the bridge. If one implementation detail survives the chapter: *exponential backoff without jitter is a metronome for stampedes.*

Proposition 11.3 (full jitter)**.** *Let n clients fail simultaneously and retry after backoff W. Without jitter all n arrive in one instant: peak n. With full jitter (delay uniform on $[0, W]$), arrivals form a uniform scatter: expected rate n/W per unit time throughout the window, and the probability that any interval of length δ receives more than $2n\delta/W$ arrivals decays exponentially (a standard Chernoff bound; the proof below records it). Area preserved, peak divided by the window: the regiment breaks step.*

Proof of Proposition 11.3. Without jitter, all n delays equal W: the arrival process is $n\delta$-functions at one instant. With full jitter the arrival times are n independent uniforms on $[0, W]$: the count in any interval of length δ is binomial with mean $n\delta/W$; Chernoff gives $\Pr[\text{count} > 2n\delta/W] \le e^{-n\delta/(3W)}$. The server's experience changes from one burst of n to a drizzle at rate n/W — with n ten thousand and W two seconds, from a ten-thousand-request instant to five thousand per second, within capacity for any service sized to its steady load and a margin. □

The retry budget. Backoff-with-jitter still decides locally. The **retry budget** globalizes the conscience: a client — or a whole tier — may spend on retries at most a fraction, say one tenth, of its first-attempt volume; within the allowance, retry freely; beyond it, fail fast and report honestly. Invisible in fair weather; in foul, it caps the amplification *by construction*, the building's appetite rising gently where it used to explode. The retry converts from an open credit line into a metered indulgence.

Proposition 11.4 (retry budgets)**.** *Let every tier cap retries at fraction b of its first-attempt volume over any window. Then tier i's total volume is at most $(1+b)^i$ times the top-level volume; for $b = 0.1$ and $k = 3$ — the four-storey estate — at most a third again, versus twenty-seven-fold uncapped. The cap binds* by construction*, independent of failure pattern, timeout alignment, or the number of teams who never met — which is why the reform is constitutional where backoff alone is behavioural.*

Proof of Proposition 11.4. Induction on tiers. Tier zero's volume is V. If tier i's total volume (first attempts plus retries) is at most $(1+b)^i V$, its first attempts downstream number at most that, and its retries downstream at most b times its first attempts by the cap, so tier i+1's volume is at most $(1+b)^{i+1} V$. The bound uses only the cap's definition — no assumption about why retries occur or how failures correlate — which is the constitutional character claimed. □

Classify before retrying. Half the fury in any storm should never have been kindled: the budget governs how often one may ask again; the taxonomy governs whether asking again is even a coherent act.

- **transient** (a timeout, a connection refused, an explicit try-again-later): retriable, within budget.
- **permanent** (not permitted, not found, malformed): never retry — one honest refusal becomes a drumbeat of identical refusals, load without hope, for the answer will not change because the question has not.
- **overload,** the subtlest class: retrying against a shedding service is pouring petrol on the fire door; the rejection carries its own advice — back off this long, or do not return — the server setting the terms of re-engagement.

The hedge. The **hedge** — for the latency tail rather than for errors: send a second copy of a *slow* request after the ninety-ninth-percentile delay, to a different replica, and take the first answer — Chapter 10's backup tasks, miniaturized to a single call. Its etiquette: it fires on one call in a hundred by construction, so its budget is built in; reads and idempotent writes only; the polished version cancels the loser.

Idempotence, and the owning layer. The condition on all the rest: *no retry, no hedge, without idempotence* — Chapter 10's lemma cashed at retail: idempotency keys on every mutating request, dedup at the far end, or the retry storm arrives bearing double payments as well as load. And one structural doctrine above the four adjectives: *retry at one layer, not at every layer*. The multiplication came from stacked independent policies; designate, per call path, the tier that owns the retrying — usually nearest the user, who alone knows what the request was worth — the tiers beneath failing fast upward. A stack that retries everywhere has no retry policy; it has a retry *ecology*, and ecologies breed storms. The retry, defused, reads so:

Protocol — Box 26. The defused retry (distilled from the Brooker doctrine and the Builders' Library)

1: **classify:** timeout / connection-refused / try-again ⇒ retriable; not-permitted / not-found / malformed ⇒ permanent (never retry); overloaded ⇒ obey the server's advice
2: **budget:** retries ≤ b (a tenth) of first attempts per window; over budget ⇒ fail fast, report honestly
3: **schedule:** delay for attempt a drawn uniform from zero to $\min(\text{cap}, \text{base} \cdot 2^a)$ — full jitter
4: **identify:** every mutating request carries an idempotency key; the far end deduplicates (Ch. 10, Lemma 10.3)
5: **own:** one designated layer retries per path; the tiers beneath fail fast upward
6: **hedge (tail latency only):** after the ninety-ninth-percentile delay, duplicate to another replica; first answer wins, loser cancelled; idempotent calls only

Budgeted, jittered, backed off, idempotent, owned by one layer — each clause an outage's tombstone.

11.3 Part Three — Queues, Deadlines, and Organized Cowardice

The retry was the demand side of self-inflicted overload; now the supply side — what a service does when more work arrives than it can perform. The framework default is the queue: buffer the excess, work it off presently — a fine instrument, with one catastrophic misreading.

The queue as debt instrument. Work enqueued is a promise to perform work later; a *bounded* queue is a modest overdraft smoothing an afternoon's bumps; an *unbounded queue is an unbounded promise* — a bank issuing unlimited paper against capacity it does not have. The governing arithmetic is one sentence old: waiting time is queue length divided by service rate — the queue length *is* the latency, wearing units. The default death, the commonest in the genre:

- load exceeds capacity by a mere tenth; the queue grows — not to some equilibrium but *without limit*, arrivals outpacing service perpetually;
- latency climbs into the minutes; upstream callers, long past their patience, time out and retry — depositing *fresh copies* of requests whose originals are still queued;
- presently the whole capacity performs work whose requesters hung up minutes ago — answers no one is listening for, in strict arrival order, forever.

The service did not refuse a single request, and therefore served none.

Load shedding. The reform the trade resisted for a generation because it sounds like defeat — until one sees the alternative was never "serve everyone" but "serve no one, slowly, in order of arrival." Engraved: *healthy systems refuse work.* The bound derives rather than guesses: a queue absorbs *bursts*, not deficits — size it to the largest burst the business genuinely produces, never more than the service rate multiplied by the deadline, since any deeper position is a request already sentenced to expire in line; past that product, a morgue with optimistic signage. When the bound is met, shed:

- **shed early:** at admission, before the request has consumed parsing, authentication, a connection slot;
- **shed cheap:** the rejection path must be the fastest code in the building, exercised precisely when nothing else is fast; an instant "no" lets the caller act, a slow "yes" past the deadline helps no one;
- **shed by priority:** the checkout survives, the analytics wait — priority travels with the deadline, declared at the edge where the business context lives; an estate that cannot tell them apart sheds by lottery — revenue in proportion to traffic.

Two counter-instinctive refinements from the deep end. Under genuine overload, serve the queue last-in, first-out: the newest caller is still waiting, the oldest has likely timed out, and first-come-first-served under overload is a machine for prioritizing the dead — adaptive establishments switch discipline when the queue ages past the deadline horizon. And degrade before refusing where the product allows — the recommendations vanish, the thumbnails coarsen; the trade calls it **brownout**, and customers forgive a dim room far more readily than a locked door.

The deadline, travelling. Load shedding's portable companion: the **deadline**, the request's parking meter. Every request enters with a patience — the human two seconds, the calling service five hundred milliseconds — and that patience must *travel with the request*, decremented at each hop; a tier that fails to propagate it counterfeits currency, spending downstream effort against a budget already exhausted. The drill, with numbers:

- the shopper's page budget is two seconds; the web tier spends two hundred milliseconds, stamps one and eight tenths on the call downstream;
- the service tier spends three hundred, stamps one and a half to storage;
- the request stands in storage's queue for one and six tenths; the stamp, consulted before working, is a tenth of a second in arrears — expired while it stood in line;
- storage declines the corpse, answers "expired" in a microsecond, and is free for a request that can still be saved.

Without the stamp, storage labours earnestly and ships its answer into a hung-up telephone — under overload, for *every* request: the default death wearing a tiered costume. The invariant, whiteboard-simple: no work without a live deadline attached; expired work dropped where it stands, with a metric to mark the grave.

Definition 11.2 (deadline propagation)**.** Each request carries a remaining budget d. A tier receiving a request with budget d: if $d \leq 0$, reject instantly; otherwise it may work, and may call downstream with the reduced budget $d' = d - \text{elapsed} - \text{margin}$, rejecting the moment its own elapsed time reaches d.

Proposition 11.5 (the parking-meter invariant)**.** *Under Def. 11.2, no tier performs work after the originator's patience has expired, up to accumulated clock-rate error over the request's lifetime (relative drift times duration — microseconds in practice, per Chapter 2's bounds; budgets are durations, not wall times, precisely so that absolute clock skew cancels). Proof by induction on hops: each hop's local spend is bounded by the budget it received, which is bounded by the originator's remaining patience at issue time, monotonically decremented.*

The circuit breaker. The pattern that assembles these reflexes into a household appliance: the **circuit breaker** — organized cowardice, endorsed. Three states. *Closed*: commerce flows; failures are scored on a sliding window. *Open* — the threshold crossed, say half the calls in ten seconds: for a cooling period every call fails instantly, locally, without touching the wire; the dependency receives the gift of silence. *Half-open*: the cooling elapses; one probe call is admitted — success closes the breaker, failure snaps it open again, the probe's polite knock replacing a thousand hammerings. Underneath: a failure detector (Part One's wager) driving an admission decision (this part's shedding) with hysteresis against flapping (Chapter 9's damper) — the season's parts snapping together. Its virtue is reflex speed and locality: when the storage tier drowns, ten thousand clients each independently stop bailing water within seconds — cowardice, decentralized, is the fastest form of mercy the estate possesses. The dishonour is never in refusing work; it is in accepting work one cannot perform.

Protocol — Box 27. The circuit breaker, with fine print

1: **closed:** calls flow; failures scored on a sliding window
2: **open** (threshold crossed): calls fail instantly, locally, for a cooling period — the dependency receives silence
3: **half-open** (cooling elapsed): admit one probe; success ⇒ closed; failure ⇒ open again
4: **scope** per dependency *and* endpoint — a shared breaker converts one sick route into a general strike
5: **jitter** the cooling periods across the client fleet, or ten thousand breakers probe as one
6: **announce** every transition loudly: a silently open breaker is a silently amputated service

A failure detector driving an admission decision with hysteresis: Parts One, Three, and Chapter 9's damper, snapped together — cowardice, decentralized, as the fastest mercy.

The part's discipline, admission to grave, in one lifecycle:

Protocol — Box 28. The request's lifecycle under pressure (admission to grave)

1: **at the edge:** stamp deadline and priority; reject before parsing if the front queue is full (shed early, shed cheap)
2: **per queue:** bound ≤ service rate × deadline horizon; under overload consider newest-first and age-out expiry
3: **per hop:** check remaining budget; decrement; pass it on; drop expired work where it stands, with a metric
4: **under sustained overload:** degrade (brownout) before refusing; refuse by priority before refusing by lottery
5: **on recovery:** readmit gradually, below the ignition threshold (Def. 11.3); the fold is waiting for the simultaneous return

11.4 Part Four — Metastability: The Christening

The set piece's theory, and the payment of Chapter 9's debt. The trade had known the shape for decades — congestion collapse brought the early internet to its knees in 1986 — but the species was christened in 2021, when Bronson and colleagues named it: **metastable failure**. The shape: a trigger — a blip, a deploy, a surge — pushes a healthy system into degradation; an ordinary failure recovers once the trigger passes; a metastable one does not, a feedback loop — retries, re-elections, cache misses, health-check churn — sustaining the degradation on its own. The degraded state is, in the physicist's borrowed vocabulary, stable: a second equilibrium on the wrong side of a hill, where the system burns its whole capacity on its own feedback until pushed back over the crest by force.

Definition 11.3 (metastable failure; after Bronson et al.)**.** A system state is *vulnerable* when healthy at current load but within reach of a trigger; a failure is *metastable* when, after a transient trigger moves the system into a degraded state, a feedback loop — reaction load generated by the degradation itself — sustains the degraded state with the trigger absent. Diagnostic signature: goodput does not recover when offered load returns to a level previously served with margin (the fold, Exhibit 11.2); recovery requires exogenous loop-breaking (throttles, restarts with admission control) and gradual readmission below the ignition threshold.

The fold. The diagnostic picture every operator should carry (Exhibit 11.2): offered load across the bottom, useful work — goodput — up the side. The curve climbs, flattens at capacity, then bends *downward*: past a certain point, additional load yields less useful work, a growing share of capacity consumed by the feedback. And the curve is not retraced on the way back — the fold, the hysteresis: reduce the load to a level handled easily this morning and goodput does not recover, the loop's own traffic now occupying the margin; the only road home passes below the loop's ignition point.

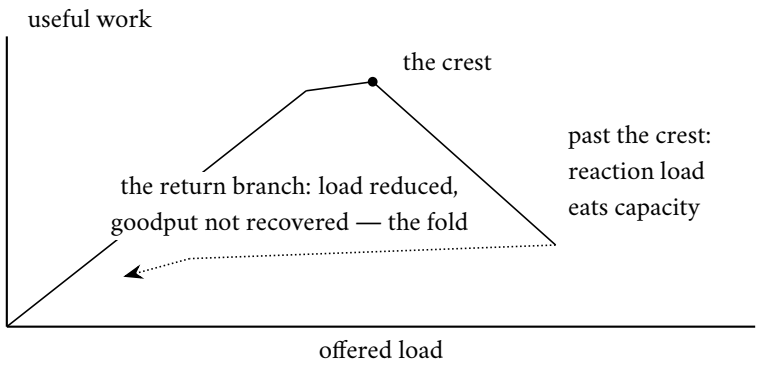


Exhibit 11.2 (the goodput fold). Rising load climbs the healthy branch to the crest, then bends down as feedback consumes capacity; retreating load (dotted) does not retrace the climb — the system is on the degraded branch, and the road home passes below the loop's ignition point (Def. 11.3, Prop. 11.6).

Triggers, and the alibi dissolved. The paper's trigger taxonomy: load surges, yes — but also deploys and rollbacks, cache restarts, routine maintenance, a dependency's wobble, even a *drop* in capacity at steady load: anything that momentarily narrows the gap between demand and supply. The alibi it dissolves is "we could not have foreseen the trigger" — quite so, and irrelevant: triggers are legion, cheap, and perpetual; the vulnerability was never the trigger but the loop and the margin, both in the blueprints.

The family of loops. Collected across three chapters, one skeleton in every costume — *the system's response to degradation is itself load of the very kind that degrades it*:

- **the retry storm:** overload slows answers; slowness breeds timeouts; timeouts breed retries; retries are load — Part Two's arithmetic, closed into a circle;
- **the cache-miss avalanche:** the cache fails or cools; misses fall through to an origin sized for a tenth of the traffic; entries expire faster than the struggling origin can refill them — the shield, once dropped, cannot be lifted under fire;
- **the connection stampede:** timeouts kill connections; re-establishing them — handshakes, credentials, warm-up — costs more than the steady state, and the re-connection load keeps the peer too slow to accept connections;
- **the membership churn** of Chapter 9's storm and Part One's theatre: slowness reads as death; declared deaths trigger data movement and elections; movement and elections are load; load is slowness.

Everything turns on the loop's gain: reaction load below the margin damps out unnoticed, a hundred times a week in every healthy estate; above it, each turn of the loop finances a larger next — the difference was a coefficient nobody had measured. The honest toy makes the criterion exact:

Proposition 11.6 (tipping in the honest toy)**.** *In the model box's toy, let unserved work be retried so that total demand is* $T = L + \gamma \cdot \max(0,\, T - C)$*, where* $\gamma \geq 0$ *is the reaction gain (retries per unserved request, net of abandonment). If* $\gamma \leq 1$*, the only equilibrium with* $L < C$ *is healthy (*$T = L$*). If* $\gamma > 1$*, then once any transient pushes demand past capacity, a fixed point* $T^* = (L - \gamma C)/(1 - \gamma) > C$ *exists: demand settles above capacity and stays there even for values of L the system formerly served with margin. Capacity increases move the threshold; only reducing* γ *below one (budgets, jitter, abandonment, admission control) removes the degraded equilibrium. The algebra is Problem 11.9 (starred); the moral is the whiteboard's item four.*

Proof of Proposition 11.6. Demand satisfies the fixed-point equation $T = L + \gamma \max(0, T - C)$. Below capacity the max vanishes: $T = L$, available iff $L \leq C$. Above capacity, $T = L + \gamma(T - C)$ gives $T^* = (L - \gamma C)/(1 - \gamma)$, which for $\gamma > 1$ is positive and exceeds C whenever L is near C — and, for large γ, even for L well below it. Stability: perturb T above T^*; the update map has slope $\gamma > 1$ — and here the honest toy shows its seams, for a slope above one is unstable in the naive iteration — the physical stabilizer is capacity saturation: served work is $\min(T, C)$, so reaction load saturates at $\gamma(T - C)$ only until abandonment and timeouts cap it; the refined model of Problem 11.9 adds the cap and recovers a genuinely stable degraded equilibrium. What the crude algebra already delivers, correctly, is the threshold: *no* degraded fixed point exists at any load when $\gamma \leq 1$ — the loop-gain criterion — and that is the design-review deliverable: measure and reduce γ, do not merely raise C. □

The anatomy, re-walked. The overture in the chapter's instruments (Exhibit 11.3):

- **the vulnerable state:** the renewals grown heavy; the margin between the metadata service's capacity and the fleet's routine demand quietly narrowed — healthy, and nearer the crest than anyone had measured;
- **the trigger:** the network disruption — brief, ordinary, cured within minutes — synchronizing a cohort of storage servers into simultaneous renewal;
- **the loop closes:** the clumped renewals exceed capacity; requests run past their timeouts; the requesters retry — unjittered, by history's misfortune, in step; servers whose renewals fail take themselves out of service — Part One's prudence, correct in isolation, catastrophic in chorus — and ask *more* urgently; the trigger's weather passes, no longer needed;
- **the escape:** *break the loop by force*: throttle the renewals themselves; add capacity behind the shelter of the throttle; readmit *gradually*, below the ignition threshold — gradually being the load-bearing adverb, for simultaneous readmission is the trigger fired again, by one's own hand, at maximum amplitude.

Hours, for an outage whose cause lasted minutes — and the postmortem's remedies are this chapter's syllabus in order: heavier capacity margins (move the crest); tighter retry discipline and slower self-removal (lower the loop gain); admission control on the metadata service (the permanent, honest throttle).

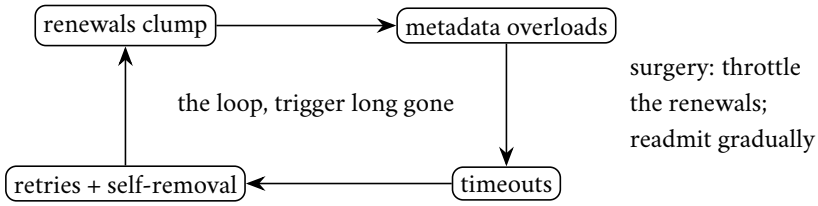


Exhibit 11.3 (the DynamoDB loop). The September 2015 anatomy from the published postmortem: each box feeds the next; the network disruption that started the wheel is nowhere on it. Recovery broke the cycle by force at the renewals edge and readmitted below ignition (the set piece, Part Four).

Loop surgery. Capacity alone prevents nothing — it moves the crest; the loop remains, waiting for a larger trigger. The operating table pairs each loop with its governor: the retry storm takes the budget and the jitter (Part Two); the health-check churn, hysteresis and load-biased mercy (Part One); the cache avalanche, coalescing, jittered expiry, and serve-stale (Part Five); the connection stampede, capped pools and reconnection backoff; the election flurry, Chapter 9's minimum-tenancy damper; and beneath them all,

admission control on the shared dependency, so no chorus of governed loops can, in sum, drown the well. One governor per loop, one fire door on the building. The review question that finds the loops: *when this system is at one hundred and ten percent of capacity, list every message it sends that it would not send at ninety* — every entry a candidate conspirator, and the empty list always, always wrong. And the organizational clause: the postmortem that stops at the trigger has learned nothing — the blip caused the *transition*; the outage was caused by the loop, which was designed, reviewed, shipped, and waiting. The trigger is weather; the loop is architecture; file these incidents under weather and the same loop returns, ignited by a different Tuesday.

11.5 Part Five — The Herd, Coalesced

The cache stampede takes its own short part, as Chapter 9 promised: the metastable seed most estates plant on purpose, in their fastest code, with pride. The scene, from that chapter's storm: the aunt's celebrated flower-committee notice, cached everywhere and expiring everywhere at the same instant — the expiry set by one line of configuration, and the herd's watches, unlike its members, agree perfectly. Ten thousand requests miss together and strike the origin as one; the origin, sized for the cache's leavings, staggers; and while it staggers, *more* entries expire.

The survival kit. Five instruments, in order of cost — together, an afternoon of engineering:

- **jittered expiries:** never let the herd's watches agree — randomize each entry's lifetime a little, so expirations smear across minutes rather than detonating on the second.
- **request coalescing** — the trade also says *single-flight*, which says it exactly: the first miss for a key travels to the origin; every concurrent miss for that key waits for the same answer — ten thousand misses become one query; a miss becomes an event rather than a demographic.
- **serve-stale-while-revalidating:** on expiry, keep answering with the slightly stale copy while a single background refresh fetches the new — the origin hears a whisper, and the staleness is bounded and declared: after Chapter 7, a contract, not a confession.
- **negative caching:** when the herd asks for something that does not exist — the deleted notice, the mistyped identifier — cache the absence too, briefly, or the misses-by-definition bypass every layer of protection forever.
- **prewarming:** a cache restarted cold under full traffic is Part Four's avalanche scheduled by the deploy calendar — warm it from a snapshot, or admit traffic to it gradually; a cold cache behind a hot service is a lit fuse.

The trade's failure to deploy the kit is never expense; it is the belief, refuted at teatime weekly, that the herd will not visit *this* pasture. Problem 11.5 designs it with its bounds stated as contracts.

11.6 Part Six — The End-to-End Argument, In Its Own Words

The oldest and most quietly load-bearing idea in the chapter, glimpsed at Chapter 10's border and owed its full ceremony: Saltzer, Reed, and Clark, 1984, "End-to-End Arguments in System Design" — a paper about where, in a layered system, a function should live.

The worked example. Move a file between machines, correctly. The network offers help — checksums on every hop, acknowledgments link by link — and even with every hop engineered to perfection the transfer is not correct, for the paper's immortal catalogue: the file may be corrupted *before* the first hop (the sending host's own disk or memory), *between* hops (a router's buffer), or *after* the last (the destination disk). Correctness is a property of the *ends* — did the bytes that left the source application arrive at the destination application intact? — and only the ends can check it: a checksum computed by the sender's application, verified by the receiver's, retry from the top on mismatch. The cutting edge, licensing simplicity: once the ends must check anyway — and they must — the middle's promises are properly performance optimizations, catching common errors early so end-to-end retries stay rare; never truth. Build the network fast and simple; put correctness where only it can live.

Remark 11.4 (the end-to-end argument; Saltzer–Reed–Clark). A design principle, not a theorem, stated in the founders' frame: a function whose correct implementation requires the knowledge and cooperation of the application endpoints cannot be completely provided by the communication system alone; the endpoints must implement it regardless, whereupon the middle's version is justified only as a performance optimization. The gallery: file-transfer integrity (checksum at the ends; hop checks catch errors early, cheaply, incompletely); delivery acknowledgment (host receipt is not application action); encryption (hop-by-hop leaves plaintext in every middle); duplicate suppression (the transport cannot see the double click). Jurisdictional reading, from Chapter 10's border: every manufactured guarantee has a wrapper boundary, and the double payment lives at the crossing — dedup at the far end (Lemma 10.3) is the argument in overalls.

The gallery, generalized. The shape of the error is always the same: mistaking a property of the pipe for a property of the conversation. The network's acknowledgment attests that the destination *host* received the bytes, when the sender needs the destination *application* to have acted — which only the application's own reply attests. Data protected hop by hop lies naked in every intermediate machine's memory; only end-to-end encryption, keys held at the ends, delivers the property the user named. The transport suppresses its own duplicates, while the double click — the duplicate above the transport — sails through; only an end-to-end identifier catches it. Chapter 10 met the argument twice without its name: the exactly-once verdict — the double payment at the border, the middle's transactions performance narrowing the window, the end's dedup (the idempotency key the *bank* honours) correctness closing it — and every fencing token, routing epoch, and producer epoch: the receiving end declining to trust the middle's claim that the sender is current.

Two qualifications, and the question. Both from the argument's authors. Hop-by-hop still earns its keep where the arithmetic favours it: twenty hops, one dropping a hundredth of its packets — end-to-end retries alone re-run the whole transfer, discarding nineteen hops of honest work per failure; local retransmission on the lossy hop makes the end-to-end check a formality that almost never fires — throughput multiplied, performance purchased locally, exactly as licensed. And some functions — routing, congestion control, this chapter's backpressure — genuinely live in the middle, being properties *of* the middle. The argument is not "the middle must be dumb"; it is "the middle cannot be trusted with the ends' correctness, so do not pay it to pretend." As a review instrument it is one question, retiring half the proposals it meets: *which end checks this, and what happens when the middle lies?* It scales into organization: the platform team's queue, mesh, and retry layer are the middle — genuinely valuable, honestly performance; the product team's invariants — the money adds up, the email sends once — are the ends, and no platform feature discharges them; the recurring postmortem is each side believing the other held the checksum.

11.7 Part Seven — Testing the Untestable

The auditors: a course that has spent ten chapters proving what can go wrong owes a part to how the trade now finds it going wrong before the customers do. Three disciplines, in ascending order of ambition, and a fourth beneath them all.

Fault injection with verification. The Jepsen method, industrialized. Chapter 7 introduced Kingsbury as consumer protection; the discipline now has industrial parts: operations generated randomly against a model of what the store claims to be; a *nemesis* process scheduling the faults — partitions from Chapter 1's documentary catalogue, process pauses long enough to expire leases, clocks jerked forward and back; and the checker hunting the transcript for a lawful explanation. The same harness logic runs in the vendors' own pipelines: the audited have internalized the auditor.

Chaos engineering. The civilian descendant extends the method to whole estates: inject the Post's malice *on schedule* — kill instances, sever links, inflate latencies — in production or its faithful replica, under a declared hypothesis ("the checkout survives the loss of any one zone") and, non-negotiably, under **blast-radius discipline**:

- begin in the replica, graduate to a sliver of production, grow the scope only as the hypothesis survives;
- watch the *customer* metrics, not the system's self-regard, with automatic abort on breach;
- run it when the responders are awake and the business is calm;
- treat a surprising result as a finding, not a failure of the exercise — the point was to be surprised at a moment of one's own choosing.

The lineage: the *chaos monkey*, loosed on a streaming company's own production fleet on the reasoning that a failure mode rehearsed daily cannot ambush anyone; the modern form is less monkey than method: hypothesis, injection, measurement, abort handle. The failures *will* occur; the only choice is whether their first rehearsal is attended by engineers with clipboards or by customers with lawyers.

Formal specification, in anger. The trade long filed TLA plus — Lamport's specification language — under academic luxuries; Amazon's engineers, in Newcombe and colleagues' Communications of the ACM report (2015), ended that. A specification: the system as states and transitions, with the invariants that must hold in every reachable state and the progress properties that must eventually hold — not code, but the *design*, made precise enough to be checked by machine. The model checker then does what no reviewer can: visits every reachable state, under every interleaving and every failure the model permits, without the human reviewer's fatal gift for assuming the plausible. On Amazon's production designs — DynamoDB's replication among them — it found genuine bugs requiring **thirty-five steps** to manifest: sequences no review would imagine and no test would stumble upon. The report's sentence for any whiteboard: formal methods *find bugs that cannot be found any other way*, at design time, when they cost a meeting rather than a postmortem. The honest boundaries: the checker verifies the design, not the code — hence the harnesses on either side of this part; the state space stays modest (three servers, not three thousand: the bugs live in the interleavings, not the multitudes); and the investment is real, repaid on exactly the systems whose failures make newspapers. Per the season's Decision 8, the starred appendix is the on-ramp: an annotated specification, a checker to run, a protocol to break.

Deterministic simulation testing. The FoundationDB school: the answer to the tester's oldest grief — the bug that appears once, at three in the morning, under an interleaving no one can summon again; engineering without reproduction is archaeology. The designers removed the impossibility at the root: the entire database runs, in test, inside a simulator in a single thread — network, disks, and clocks simulated, every timeout, delivery, and crash driven by one pseudo-random number generator with one seed. Determinism — Chapter 5's diet, Chapter 8's Calvin, Chapter 10's replay — applied to reality itself: same seed, same universe, same bug, every time. The harness lives a million catastrophic lifetimes a night, and the code is seeded with cooperating treachery: *buggify* markers throughout the source, activated at random under simulation, making every buffer smaller, every rare branch likelier, every error path a boulevard. Any lifetime that ends in a violated invariant is preserved: the seed *is* the incident report, replayable forever. The candour clause: the simulator must own every source of nondeterminism, which constrains the languages, the libraries, the style of the code — bought at the foundation or not at all, which is why the exemplars are storage engines and kernels rather than retrofits. The field's final answer to the Post's malice was to hire the Post, on a fixed salary, with a logbook.

Remark 11.5 (the testing portfolio)*.* Four instruments, four jurisdictions: the model checker audits the *design* (exhaustive interleavings on small instances; the thirty-five-step bug); the fault-injection harness audits the *implementation* (generated operations, a nemesis schedule, history checking against Chapter 7's definitions); chaos audits the *estate* (hypothesis, bounded blast radius, customer metrics, abort handle, business hours); deterministic simulation audits all three at once for systems built inside it from the foundation (every nondeterminism owned; seeds as permanent incident reports; rare paths promoted by buggify). None substitutes for another; the failure path that has never run does not exist.

Observability, the postscript. Beneath the portfolio: one cannot repair what one cannot narrate. *Distributed tracing* — a request identifier propagated through every hop, each hop contributing a span (who I am, whom I called, when I started and finished), stitched afterward into the request's family tree — is the three-in-the-morning story, mechanized; sampled in practice, the craft being to keep the interesting biographies — the slow, the failed, the shed — at full fidelity. Its ancestry: a trace is *happens-before made visible* — Chapter 2's causal roads, instrumented and drawn. The division of labour:

- **metrics** say *that* something is wrong — cheaply, continuously, across everything;
- **traces** say *where* — the one afflicted request followed through the maze;
- **logs** say *what,* in the afflicted component's own words;
- and this chapter's machinery — deadline stamps, retry counts, breaker states — faithfully recorded into all three, says *why* — the only question the postmortem is actually obliged to answer.

And every loop of Part Four is visible in traces before it is fatal in dashboards — retry counts climbing, deadline expiries clustering, breaker states flickering — the storm announcing itself whole minutes before the goodput folds, to any estate arranged to be listening.

11.8 The Whiteboard

For the design review.

1. Every retry carries a budget and jitter, backs off exponentially, and rides an idempotency key — four adjectives or it is a weapon. The amplification arithmetic is exponential in stack depth, and the stack was assembled by teams who never met; the budget is the only clause that binds them all.

2. Every queue carries a bound, and every bound a shedding path — designed, tested, and fast, before the happy path is tuned. A queue is a debt instrument; an unbounded queue is an unbounded promise; and a slow yes is worth less than an instant no.

3. Every request carries a deadline, and the deadline travels — decremented per hop, honoured at every tier, with expired work dropped where it stands. Effort spent past the deadline is counterfeit currency, and the deepest tier pays it.

4. Hunt the loops: list every message the system sends at a hundred and ten percent of capacity that it would not send at ninety — retries, health checks, elections, refills, reconnections — and put a governor on each. Capacity moves the crest; only loop surgery disarms the second equilibrium. Bias health checks toward mercy under load, or membership churn will finish what the load began — and when the fire comes anyway, recovery re-admits gradually, below the ignition point, or dawn's first act relights the night's work.

5. Inject the Post's malice on schedule, before it volunteers: fault injection with history checking for the protocols, chaos with blast-radius discipline for the estate, a model checker for the designs that must not be wrong, and — where the investment is warranted — a seeded universe in which every failure is a specimen. The four are a portfolio, not a menu: the checker audits the design, the harness the implementation, the chaos the estate, and the simulation all three at once. The failure path that has never run does not exist; it merely has not yet cost anything.

The register. All payments, nothing issued — the first such chapter of the season, itself the signal that the season is landing. Paid: #2, crashed-versus-slow — issued in Chapter 1, priced and dialled here, the doppelgänger retired with full honours; #10, the suspicion services — priced in heartbeats, lag, and correlated panic; #19, the Jepsen method — industrialized into the trade's own pipelines and its chaos-engineering descendants; #22, Chapter 9's storms and stampedes — christened metastable, anatomized in the set piece, disarmed by loop surgery; #23, the retry's dark side — convicted by arithmetic, reformed by four adjectives. The register closes at the finale, where #24 (stragglers and backup tasks, racing the unluckiest machine) and #4's remainder (commutativity's second half, the gradients) fall due. Episode word count: 10,002 — at floor; the half-draft drill required its usual passes.

11.9 Problems

Tier I — Calisthenics

Problem 11.1 (the budget, sized)**.** A tier makes ten thousand first attempts per second; the dependency's steady error rate is half a percent, with incidents to twenty percent. (a) Size the retry budget so steady-state retries are never throttled but incident amplification stays below five percent of first-attempt volume... state the tension and resolve it with a number and a sentence. (b) Compute bottom-tier volume in a four-tier stack at your budget versus uncapped three-attempt retries during a total brownout. (c) State what the budget does to customer experience during the incident, honestly, and why that is the correct trade.

Problem 11.2 (reading the dial)**.** Heartbeats every half second with occasional one-second stragglers and a once-per-hour two-second pause. (a) Choose phi thresholds for: routing away (cheap, reversible), failover (expensive, fenced); justify each with the false-alarm rate. (b) The network enters a congested afternoon and inter-arrivals double. What happens to the silences' phi under a re-estimated F, and what would a fixed eight-hundred-millisecond timeout have done? (c) Fifty peers go silent together. What should fleet-level logic conclude, and why can no per-peer dial conclude it?

Problem 11.3 (the meter runs)**.** A page has a two-second budget; tiers spend two hundred, three hundred, and four hundred milliseconds; queues add what they add. (a) Compute each hop's passed budget with a fifty-millisecond margin per hop. (b) Storage dequeues a request with ten milliseconds remaining and a two-hundred-millisecond median service time: what should it do, and what metric records it? (c) A reviewer proposes padding every timeout "to be safe"; explain, via the seance clause, what padding the deepest tier's timeout beyond the top's budget purchases.

Problem 11.4 (classify before retrying)**.** For each response, retry or not, and under what schedule: (a) connection refused; (b) malformed request; (c) overloaded, retry after two seconds; (d) deadline exceeded upstream; (e) a timeout on a non-idempotent payment call with no idempotency key (careful, sir).

Tier II — The Real Thing

Problem 11.5 (serve-stale, with bounds)**.** Design the aunt's-notice cache: target hit rate, jittered expiry, single-flight refresh, serve-stale window. Given a one-second origin refresh and a ten-thousand-per-second read rate: (a) bound the origin load in steady state and during a refresh; (b) bound worst-case staleness seen by any reader, and state it as a Chapter 7 contract (eventual-plus-what); (c) add negative caching for a deleted notice and show the resurrection window it must respect (Ch. 6's tombstone clause); (d) state what fails if the refresh lock has no timeout, and the repair.

Problem 11.6 (sabotage: the merciless probe)**.** A fleet's health check calls the real read path with a three-hundred-millisecond timeout, three strikes and out; the orchestrator drains and replaces strikers. A surge pushes median latency to four hundred milliseconds fleet-wide — slow, correct, and profitable. Exhibit the collapse loop turn by turn: strikes, drains, load concentration, more strikes — to the empty fleet; identify the loop gain and the exact clause that makes it exceed one; then repair three ways (mercy bias under load; disruption budget; probe measuring customer error rather than latency) and state which combination survives the surge *and* still catches a genuinely dead machine.

Problem 11.7 (the breaker, tuned)**.** A dependency's contract: five hundred milliseconds at the ninety-ninth percentile, half a percent errors. Tune Box 27: window, threshold, cooling, probe policy, scope — each with a sentence of justification against two adversaries: a real outage (the breaker should open within seconds) and a half-percent bad Tuesday (it should not open at all). Then exhibit what a fleet of un-jittered breakers does to the dependency at cooling expiry.

Problem 11.8 (the amplification audit)**.** A described stack: edge (three attempts), API (three), service (two), storage client (three), each with its own timeouts, no budgets, no jitter, and an edge timeout shorter than the sum beneath it. (a) Compute worst-case storage amplification. (b) Identify the seance: which tiers labour for departed callers, and why the edge's short timeout makes it worse. (c) Prescribe the constitutional reform in four lines (owner layer, budgets, jitter, deadline propagation) and recompute the worst case.

Tier III — For the Ambitious (starred)

Problem 11.9 (⋆ the tipping model, refined)**.** Extend Prop. 11.6's toy: served rate $\min(T, C)$; unserved work retried once after timeout with probability q (abandonment $1-q$); latency rising with utilization so that above capacity, timeouts fire for served work too at a rate you model. (a) Derive the demand fixed points and their stability. (b) Exhibit the hysteresis: the range of L for which both healthy and degraded equilibria exist. (c) Show how a retry budget (q capped) and admission control (effective L capped) each eliminate the degraded branch, and which does so at lower cost to good traffic. *(Hints only.)*

Problem 11.10 (★ the appendix's exercises)**.** Using the appendix: (a) run the checker on TCommit and record the state count; (b) add the invariant that no resource manager is committed while another is aborted, and confirm it holds; (c) break the protocol — allow commit without unanimous prepare — and read the checker's counterexample trace; (d) sketch how TwoPhase (the coordinator edition) exhibits Chapter 8's blocking as a liveness failure under a crashed-coordinator model. *(Hints only.)*

11.10 Hints

Hint (Problem 11.1)**.** Steady retries are half a percent of volume — any budget above that is invisible in fair weather; the incident then sets the cap's value, not the weather.

Hint (Problem 11.6)**.** The gain: each drained node moves its share of a fleet-wide surge onto survivors already past the probe's timeout; strikes per drain exceed one when latency is load-coupled — write the inequality.

Hint (Problem 11.9)**.** (b) Sweep L upward on the healthy branch and downward on the degraded; the fold is where the branches overlap. (c) Budgets lower γ; admission lowers effective L; compare shed volume at equal stability margin.

Hint (Problem 11.10)**.** (c) The checker's trace is a numbered sequence of states; read it bottom-up and name the step where unanimity was needed.

11.11 Solutions

Tier I in full (tersely); Tier II in full — Problem 11.6 is the sabotage certification; hints only for Tier III.

Solution (to Problem 11.1)**.** (a) Ten percent: twenty times the steady half-percent (never throttled in fair weather), yet capping incident retries at a thousand per second against ten thousand first attempts — five percent... the stated target requires half that; either tighten to five percent (still ten times steady) or accept ten — the sentence: "the budget is set between the weather and the storm, closer to the weather." (b) At $b = 0.1$ over four tiers: $(1.1)^4 \approx 1.46$ times top volume; uncapped $3^4 = 81$. (c) During the incident, most failed requests are not retried: customers see errors sooner — honestly the point, since the alternative was errors later plus a downed dependency for everyone; the budget converts a building fire into a room fire.

Solution (to Problem 11.2)**.** (a) Routing at phi around one (one false route in ten — harmless); failover at three or higher (one false election per thousand silences, each survivable by fencing; with the two-second pause on record, place the threshold's implied silence beyond it). (b) Re-estimated F widens; the same absolute silence maps to lower phi; the detector stays quiet through the congestion — the fixed timeout would have declared half the fleet dead at the worst hour. (c) Correlated silence is a network or switch event: freeze membership changes (the storm brake), do not fifty-fold the failover machinery; a per-peer dial models one rhythm and cannot represent common cause.

Solution (to Problem 11.3)**.** (a) Two thousand minus two hundred minus fifty: one thousand seven hundred fifty to the service tier; minus three hundred and fifty: one thousand four hundred to storage; the queues consume from whatever remains at dequeue. (b) Reject instantly: expected completion exceeds remaining budget twenty-fold; the metric is deadline-expired-at-dequeue, the queue-sizing signal of Box 28. (c) Padding the deepest tier past the top's budget buys work performed for the departed: the seance, institutionalized — deeper timeouts must *shrink*, never grow.

Solution (to Problem 11.4)**.** (a) Retriable, jittered backoff within budget — likely a restart or a bad host; hedge to another replica sooner. (b) Permanent: the answer will not change; retrying is a drumbeat. (c) Obey: wait the stated two seconds, jittered, budgeted; the server knows its condition. (d) Do not retry: the caller is gone; count the grave. (e) Neither retry nor not-retry is safe: without a key, a retry risks double payment, no retry risks none — the correct act is to *query* (was instruction seventeen executed?) or fail to a reconciliation path; the deeper repair is the key, after which (e) becomes (a).

Solution (to Problem 11.5)**.** (a) Steady state: one refresh per expiry per key — with single-flight, at most one origin call per second for the hot key, against ten thousand reads; during refresh, concurrent misses wait or take stale: origin load unchanged. (b) Worst staleness: expiry jitter ceiling plus serve-stale window plus one refresh time — state it as "reads may lag writes by at most *s* seconds, always": bounded staleness, Chapter 7's honest tier. (c) The negative entry must outlive anti-entropy's resurrection window for the deletion (the tombstone clause), else the cache re-learns a deleted notice from a stale replica. (d) A crashed refresher holding the single-flight lock starves the key into permanent staleness; the lock carries a lease (Chapter 5, as ever) and a second flight departs on expiry.

Solution (to Problem 11.6 — the certification)**.** The loop, turn by turn, fleet of ten at a surge that puts median latency at four hundred milliseconds.

(1) Probes (three-hundred-millisecond fuse) begin failing fleet-wide — every node slow-but-correct, none dead.

(2) First striker drained: its tenth of the surge lands on nine survivors; their latency rises further above the fuse.

(3) Strikes accrue faster on the more-loaded survivors: each drain raises per-survivor load by a growing increment — the gain exceeds one as soon as one drain produces, on average, more than one new striker, which load-coupled latency guarantees here from the first drain.

(4) The orchestrator, constitution in hand, drains the fleet to zero. Customer availability: zero. Machines dead at any point: zero. The health system executed the healthy.

Loop and gain named: drains concentrate load; concentrated load fails probes; failed probes drain. Repairs: mercy bias (raise the fuse or require sustained breach when fleet-wide latency is elevated) breaks step three's coupling; the disruption budget (at most one drain per interval) caps the gain below one regardless of coupling; the customer-error probe removes the false signal entirely but, alone, would also excuse a machine serving errors slowly — the surviving combination is the budget plus error-weighted probes, which still catches the genuinely dead (they fail *error* checks too, and the budget merely paces their removal). Certified fatal as shipped: total self-inflicted outage from a profitable surge, no failed hardware anywhere in the building.

Solution (to Problem 11.7)**.** Window ten seconds sliding; threshold: error rate above ten percent *and* a minimum call count (twenty), so a quiet period cannot open on three unlucky calls; cooling five seconds with full jitter; probe: single call, real endpoint, budget-stamped; scope per endpoint. Against the outage: ten percent of a busy window crosses within a second or two — open fast. Against the bad Tuesday: half a percent never approaches ten — closed all day. Unjittered fleet at cooling expiry: ten thousand probes in one instant — the herd, Part Five, self-assembled; with full jitter, two thousand per second across five seconds, a drizzle.

Solution (to Problem 11.8)**.** (a) Three times three times two times three: fifty-four attempts at storage per edge request, worst case. (b) The edge abandons before the tiers beneath finish even one honest attempt

each; everything below the first tier labours for the departed on every edge timeout — and the retries below continue after abandonment, so the seance outlives the customer by the sum of the lower timeouts. (c) Edge owns retries (three, budgeted at a tenth, full jitter); all lower tiers: one attempt, fail fast, deadline-checked. Worst case at storage: three — one per owned attempt — and only while the deadline lives.

11.12 Appendix (starred): TLA+ for the Curious

Per the season's Decision 8, the volume's one formal-methods appendix lives here and nowhere else. The specimen is Lamport's own teaching specification of transaction commit — the *abstract* problem of Chapter 8, before any coordinator exists — annotated for a reader who has never opened the toolbox.

The specification, annotated. A specification is a state machine: variables, initial states, permitted steps. TCommit has one variable, the array of resource-manager states.

Protocol — The TCommit specification (after Lamport's *Specifying Systems* lineage)

1: **constant** RM — the set of resource managers, for checking a small concrete set, three names
2: **variable** $rmState$ — a function from RM to {"working", "prepared", "committed", "aborted"}
3: **Init:** $rmState$ = every manager at "working"
4: **Prepare(r):** enabled when $rmState[r]$ = "working"; effect: r moves to "prepared"
5: **canCommit:** true when every manager is "prepared" or "committed"
6: **DecideCommit(r):** enabled when $rmState[r]$ = "prepared" and canCommit; effect: r to "committed"
7: **notCommitted:** true when no manager is "committed"
8: **DecideAbort(r):** enabled when $rmState[r]$ is "working" or "prepared", and notCommitted; effect: r to "aborted"
9: **Next:** some r takes one of the three steps
10: **Invariant (TCConsistent):** no pair (r, s) with $rmState[r]$ = "committed" and $rmState[s]$ = "aborted"

Read line 6 twice: a manager may commit only when *all* are prepared-or-committed — unanimity as an enabling condition, the veto of Def. 8.1 rendered as a guard; and line 8's notCommitted guard is stability: no abort after any commit. The specification says *what* transitions are lawful and nothing about messages, coordinators, or crashes: it is the problem, not a protocol.

Running the checker. Install the TLA+ toolbox or the command-line tools; transcribe the specification (the original text of TCommit ships with the tools' examples); give the checker a model: set RM to three concrete names, name TCConsistent as the invariant, and run. The checker enumerates every reachable state — a few dozen here — and reports the invariant satisfied. Then perform Problem 11.10(c): delete canCommit from DecideCommit's guard and run again; the checker returns a numbered trace — Init, a Prepare, a premature commit, an abort elsewhere — the shortest lawless history, printed like a police report. That trace, sir, is the entire pedagogy: the tool does not argue, it *exhibits*. The two-phase protocol proper (TwoPhase, also shipped with the examples) adds a coordinator and messages and can be checked to *implement* TCommit; add a crashed-coordinator model and the blocking of Thm. 8.2 appears as a liveness failure — the checker, unlike the folklore, will show you the exact stuck state. The thirty-five-step bugs of the Amazon report were found with precisely this workflow, scaled up and made routine.

11.13 The Reading

Bronson, Aghayev, Charapko, and Zhu, "Metastable Failures in Distributed Systems," HotOS 2021. Short, incident-fed, and the rare paper that names a thing sir has already survived; read with your own postmortems open, and Def. 11.3 beside it.

Newcombe, Rath, Zhang, Munteanu, Brooker, and Deardeuff, "How Amazon Web Services Uses Formal Methods," CACM 2015. The thirty-five-step bug, the cultural argument, and the honest costs; the appendix here is its on-ramp.

The Amazon Builders' Library: "Timeouts, Retries, and Backoff with Jitter" (Brooker). The doctrine in the house style, with the graphs Prop. 11.3 only sketches; read alongside the entry on avoiding fallback and the one on load shedding.

Saltzer, Reed, and Clark, "End-to-End Arguments in System Design," TOCS 1984. Eight pages that outrank most shelves; the gallery of Rem. 11.4 in the original prose.

The shelf. Hayashibara et al., "The φ Accrual Failure Detector" (SRDS 2004). The DynamoDB service disruption summary (September 2015) — the set piece's primary source, a model of the postmortem genre. Huang et al., "Gray Failure: The Achilles' Heel of Cloud-Scale Systems" (HotOS 2017) — differential observability named. Dean and Barroso, "The Tail at Scale" (CACM 2013) — hedges and the tail arithmetic. The FoundationDB testing talks and paper — the universe built twice. And Kingsbury's Jepsen analyses, rereadable now as the industrialized method of Rem. 11.5.

Chapter 12

Capstone: The Distributed Systems You Already Run

Companion to Episode Twelve, and the season's last. The episode tells the story — the costume coming off, the training estate read, entry by entry, as the distributed system the reader has operated all along; this chapter keeps the record: the dictionary with every alias resolved to a named chapter, the ring all-reduce proven bandwidth-optimal against the two-ledger floor, the season's oldest debt paid in gradients with the rhyme honestly labelled, the three geometries, the Young–Daly interval, the mercurial cores, skew with its profit line, and the grand audit — the register ruled, closed, and tabled as permanent equipment. Statements, proofs, price tags, problems. Two theorems, and no new impossibilities: the finale's work is recognition.

12.1 Part One — The Dictionary

The claim on which the finale balances: the largest coordinated computations in history are not the banks or the search engines but training runs — tens of thousands of accelerators, gradient traffic in terabits per second, millions of consecutive lockstep rounds, hardware failures on a daily schedule (Part Five's arithmetic), and one logical object — the model — that every machine must agree about continuously or the investment diverges into noise. The disguise held because the machine-learning trade hired its infrastructure vocabulary from high-performance computing — collectives, ranks, barriers — a lineage raised on reliable, private, exquisite networks that never needed this course's paranoia until commodity scale made it compulsory. Two dialects, one mathematics; the dictionary performs the introduction, and the full mapping is tabled in §12.8, permanent equipment.

- **A training step is a round:** a barrier-synchronized iteration, the fleet advancing in lockstep — Chapter 4's rented synchrony assumed wholesale and paid in the renter's eternal coin: the round lasts as long as its slowest participant, which makes the straggler this estate's national enemy (Part Five).
- **Gradient accumulation is batching:** several rounds of local work folded into one round of agreement, amortizing the barrier and the parcels — group commit in Chapter 5's log, the batched sequencers of Chapter 8; latency for throughput, the oldest trade in the ledger.
- **An all-reduce is agreement on a sum:** strictly weaker than consensus — the dictionary's first fine distinction, given its formal card below (Def. 12.1).
- **A checkpoint is a crash-recovery snapshot:** the Chandy–Lamport spirit's third appearance (Chap-

ters 2 and 10), with alignment pre-solved — the barrier between steps is a global quiescent instant, this river agreeing to hold still sixty times a minute; the live questions move to *how often* and *how cheaply* (Part Five).

- **A straggler is Chapter 10's unluckiest machine,** reborn on silicon: one accelerator a few percent slow — thermals, a failing memory module, a noisy neighbour — sets the pace for ten thousand peers; the backup-task remedy planted there comes due here, with interest.
- **Parameter staleness is replication lag:** a worker computing gradients against weights three versions old is Chapter 6's stale follower; every bounded-staleness convergence argument is a session-guarantee negotiation — Chapter 7's eventual-plus-what, the *what* denominated in steps of drift.
- **Elastic training is a view change:** members join and fail, the job continues — Chapter 4's reconfiguration and Chapter 5's dragons at rack scale, the membership epoch stamped on every collective. The fencing token in its final uniform: authority must expire visibly before it may be reassigned safely.
- **The data pipeline is the log** — Chapter 10 entire: a shuffled, sharded, replayable stream read by cursors and rewound for reproducibility; the input leg of the recovery trinity.
- **The job scheduler is Chapter 9's placement church:** shards seated against failure domains and interconnect locality, with the standing commandment about not seating a parliament's replicas on one switch.
- **The feature store is a materialized view** (Chapter 10): one definition, one log, every consumer a cursor.
- **The training metrics are the watermark:** the loss curve is the computation's own confession of progress and health — the invariant that reports, before any machine confesses, that something in the estate has begun to lie (Chapters 10 and 11).

A dozen entries, and the dictionary could run to fifty; somewhere around the fifth it becomes clear that there is nothing else in the building — the field is made, joist by joist, of this course, the translation identity rather than metaphor. The practical privilege follows at once: the dictionary makes forty years of other people's postmortems the reader's own, every lesson the databases and queues purchased with outages reading directly onto the cluster, name by translated name.

The first fine distinction. Chapter 3 forbade deterministic asynchronous consensus because consensus must *arbitrate* — pick one value, exclude the rest. A sum excludes nobody: every contribution folded in, order irrelevant, nothing to arbitrate — no race, in Chapter 6's language, and the race test was always the border. Weaker demand, cheaper machinery: the all-reduce runs at wire speed precisely because it is not a parliament — and the estate keeps an actual parliament on retainer, the job controller, for the questions that *are* decisions: who is a member, which checkpoint is blessed, who leads. Two agreement products, two prices, one building.

Definition 12.1 (all-reduce)**.** Given vectors $g_1, \ldots, g_n$ at nodes $1, \ldots, n$ and an associative, commutative reduction (element-wise sum throughout), the *all-reduce* terminates with every node holding $g_1 + \cdots + g_n$. It is an agreement on a *value determined by all inputs symmetrically*: no arbitration, no exclusion, no winner — by the race test (Chapter 6), a problem strictly below consensus, and priced accordingly.

12.2 Part Two — Data Parallelism, and the Parcels Passed Round

The workhorse: *data parallelism*, synchronous stochastic gradient descent across the fleet. Every worker holds a complete copy of the model — replication, Chapter 6, at full factor; each computes a gradient on its own shard of data — partitioning, Chapter 9, of the work rather than the parameters; and the fleet's one point of true coupling is the agreement, each round, on the *average* gradient — the mean of the shard-gradients, a sum arrived at by pure accumulation, no arbitration (Def. 12.1). Once agreed, every replica applies the identical update to identical weights, and the copies stay identical *by determinism rather than by reconciliation* — Chapter 5's state machine replication, verbatim, recorded with its fine print.

Remark 12.2 (synchronous data parallelism as SMR)*.* With deterministic updates and an agreed summed gradient per round, all replicas of the model evolve identically from an identical start: state machine replication (Chapter 5), with the all-reduce supplying the agreed input and determinism supplying the coherence — replicas equal by construction, not by reconciliation; no anti-entropy, ever, for months. Floating-point non-associativity is the fine print: the reduction *order* must itself be fixed (Box 29 fixes it) or the replicas drift in the last bits and reproducibility dies by topology (Chapter 5's diet, enforced down to the rounding).

The parameter server. The naive agreement: ship every gradient to a central estate, sum there, ship the result back — Chapter 9's directory church, made an architecture by Li and colleagues' 2014 paper, the server tier itself sharded, each server owning a range of the parameters. Price it with the course's arithmetic: a thousand workers means the server tier moves two thousand ledgers per round while each worker moves two — the fan-in is the ceiling, the ceiling is bought in server bandwidth, and it does not rise as workers join: a tax collected per round, forever. The trade wanted the sum computed *by the fleet itself*, and the answer came from an unglamorous corner: high-performance computing had been passing parcels around rings since before the web. Terms fixed in the box.

Model of this chapter

For the ring (Thms. 12.1–12.2). n nodes in a directed ring; each holds a vector of size G (the gradient); links carry β bytes per second each way, full duplex, all links simultaneously usable; per-message latency α acknowledged in remarks, with the bandwidth regime (G large) primary. The goal: every node holds the element-wise sum of all n vectors.

For checkpointing (Thm. 12.3). Failures strike the job as a memoryless process with mean time M between failures (fleet-wide); a checkpoint costs δ of wall time; on failure the job rewinds to the last completed checkpoint, losing on average half an interval; $\delta \ll M$ (the regime of interest; the exact treatment is cited in the reading).

For staleness (Rem. 12.4). Statements only, with pointers, per the season's honesty clause: the convergence analyses live in the optimization literature and are not re-proved at this desk.

The ring, in two acts. Each node holds its own gradient — a ledger of four hundred pages, in the episode's staging — cut into n parcels, the ring's standing convention (Box 29). *Act one, reduce-scatter*, $n-1$ passes: each node sends its designated running-sum parcel to its clockwise successor and *adds* the parcel arriving from its predecessor, so that after $n-1$ passes each node holds one parcel *complete* — the full n-way sum of those pages, scattered across the ring. *Act two, all-gather*, $n-1$ passes more: the finished parcels circulate — copied, not added — until every node holds the entire summed gradient, and the barrier releases. Two tricks are easy to admire past. First: no machine ever held, or needed to hold, anybody's full ledger but

its own — the convergence itself was partitioned, Chapter 9's doctrine applied not to data at rest but to the agreement in flight. Second: every link worked every pass, and that full employment is where the economy comes from.

Protocol — Box 29. Ring all-reduce (the classical collective, checked against the standard formulations)

1: **setup:** nodes $0 \ldots n-1$ in a ring; each vector cut into n parcels; fixed parcel-accumulation order (reproducibility)
2: **reduce-scatter,** $n-1$ **passes:** in each pass, send the designated running-sum parcel to the successor; add the parcel arriving from the predecessor
3: **all-gather,** $n-1$ **passes:** circulate the finished parcels; copy, do not add
4: **result:** every node holds the full sum; traffic $2(n-1)/n$ ledgers per node; every link busy every pass
5: **regimes:** large G: rings; small G: trees (latency); mixed fleets: hierarchical rings, heavy traffic on short wires

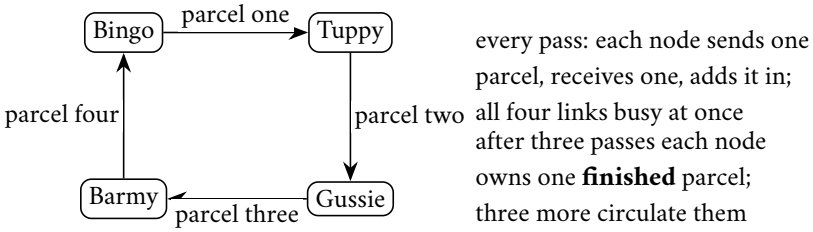


Exhibit 12.1 (the parcels, mid-pass). One reduce-scatter pass of Box 29 on four nodes: running sums travel clockwise, every wire working. Six passes in all; traffic per node six of the four-hundred-page ledger's hundred-page parcels — one and a half ledgers, approaching two as the ring grows (Thm. 12.1).

The arithmetic's punchline: *two ledgers, regardless*. Total traffic per machine approaches twice the gradient's size no matter how many machines join the ring — the sum of ten thousand contributions computed for the postage of two.

Theorem 12.1 (ring all-reduce cost)**.** *Cut each vector into n parcels of size G/n. The ring algorithm of Box 29 completes in $2(n-1)$ passes, each node sending and receiving one parcel per pass; total traffic per node $2(n-1)G/n < 2G$, and wall time in the bandwidth regime*

$$T = \frac{2(n-1)}{n} \cdot \frac{G}{\beta},$$

approaching $2G/\beta$ from below as n grows: the fleet size cancels. Every link carries traffic in every pass — no idle wire, no bottleneck node.

Proof of Theorem 12.1. Index parcels and nodes modulo n. *Reduce-scatter:* in pass p (for $p = 1, \ldots, n-1$), node i sends to node $i+1$ the running sum of parcel $i-p+1$ accumulated so far, and adds the parcel arriving from node $i-1$ to its local copy. Induction: after pass p, node i holds, in parcel slot $i-p$, the sum of that parcel's copies from nodes $i-p, \ldots, i$ — $p+1$ contributions. After pass $n-1$, node i holds the complete n-way sum of parcel $i+1$ (indices mod n): each node owns one finished parcel. *All-gather:* $n-1$ further passes circulate the finished parcels unchanged; after pass $n-1$ every node has all n. Each node sent exactly one parcel of size G/n in each of $2(n-1)$ passes: traffic $2(n-1)G/n$; passes are simultaneous on all links (each

node sends to its successor in every pass), so wall time is traffic over β, as displayed. The reduction order within each parcel slot is fixed by the ring's geometry — node i+1's finished parcel always accumulated in the order i+1, i+2, … — giving Rem. 12.2's reproducibility clause for free. □

The floor. The punchline is not the ring's cleverness but the problem's own floor: every machine must *receive* the parts of the sum it did not compute — very nearly one full ledger — and its contribution must *leave* it, another ledger outbound, to reach the totals everyone must hold. One ledger in, one ledger out, for any algorithm whatsoever that ends with everyone holding everything; the ring's achievement is merely — merely! — to reach the floor while keeping every link busy every pass.

Theorem 12.2 (the two-ledger floor)**.** *Any all-reduce algorithm in the model box has some node whose total traffic is at least* $2(n-1)G/n$*: at least* $(n-1)G/n$ *received — the summands' information it does not hold locally, without which the sum at generic inputs is undetermined — and at least* $(n-1)G/n$ *sent — its own contribution's share of what all other nodes' outputs require. The ring (Thm. 12.1) meets the floor exactly: bandwidth-optimal.*

Proof of Theorem 12.2. Fix any correct algorithm and consider generic inputs (no cancellations; formally, inputs algebraically independent over the reduction). *Receiving floor:* node i's output is the full sum, which at generic inputs is not a function of g_i alone; every other node's contribution must be represented in what i receives. The summands arriving at i may arrive pre-combined — that is the algorithm's freedom — but combination does not compress below the size of one vector's worth of information about the other $n-1$ contributions to each element: whatever i receives must determine, with g_i, all G elements of the sum, and at generic inputs the received data must carry at least the G-element combination of the others' values — of which i can locally reconstruct nothing. A counting argument on element shares (each element's foreign part is $(n-1)$ of n equal contributions) gives at least $(n-1)G/n$ received by *some* node under any balanced scheme, and at least that at the maximum node in general (averaging: total foreign information across nodes is $n\cdot(n-1)G/n$, so the mean per node is $(n-1)G/n$; the maximum is at least the mean). *Sending floor:* symmetrically, g_i appears in every other node's output; the information leaves i only through its sends, pre-combined or not, and the same averaging yields at least $(n-1)G/n$ sent at the maximum node. Summing the two floors at the averaging node gives the bound; the ring achieves it at *every* node, hence optimal. (The desk notes the argument's honest character: an information-counting bound over generic inputs in the standard style of the collectives literature, not a bit-level coding theorem; the reading carries the formal treatments.) □

The courier. NCCL — spoken *nickel*, as the industry insists and the lexicon has recorded since Chapter 1 — is this Part conducted at nanosecond stakes: discovering the fleet's topology, choosing among rings and trees, pipelining the parcels so the wires never breathe, and doing it all again when the membership epoch turns. Scale the arithmetic once, for awe's sake: a frontier gradient weighs hundreds of gigabytes, the two-ledger bound prices each round at roughly twice that per machine, and the loop demands a round every few seconds for a season — which is why the interconnect is the estate's costliest citizen after the accelerators, and why an algorithm at the bandwidth floor is not an optimization but the difference between the computation existing and not.

Remark 12.3 (latency, trees, and hierarchy)*.* With per-message latency α, the ring pays $2(n-1)\alpha$ in serialized hops — negligible for large G, ruinous for small messages, whence tree-shaped all-reduces ($O(\log n)$ depth, a constant-factor bandwidth premium) for the latency regime, and hierarchical composites — rings within pods, exchanges between — placing the heavy traffic on the short wires (Chapter 9's oldest commandment). Problem 12.6 budgets one.

Ring versus server. The ring is a barrier in miniature — every pass waits for the slowest link and the slowest machine, so its magnificent bandwidth is bought at maximal exposure to Part Five's straggler. The server pattern buys *asynchrony* — workers push and pull at their own pace against the central copy — and pays in fan-in bandwidth and in the staleness Part Three prices; and it keeps one virtue the ring cannot counterfeit: *elasticity*. A ring is a topology — a member's death breaks it, a member's arrival re-forms it, each transition a view change with a pause; the server is a shopfront — workers come and go as customers, unannounced — which is why the elastic and the preemptible provinces still worship there. Topology is destiny: the collective for the tight synchronous core, the server pattern where elasticity and asynchrony matter more than the last drop of wire; modern estates deploy both, per layer.

12.3 Part Three — Asynchrony's Bargain, and the Season's Oldest Debt

The tempting escape from the barrier's tax: stop waiting — let each worker read the current weights, compute, and apply its gradient *whenever it arrives*, against whatever weights now stand; no barrier, no round, the lost-update crime scene of Chapter 6 re-enacted at every step. The astonishment — published as *Hogwild* by Recht and colleagues in 2011, the name a confession — is that it works: no locks whatsoever, and under honest assumptions about sparsity and step size, convergence survives at nearly full speed. The reason is the season's oldest promissory note, presented at last: *addition commutes*. A gradient update is an increment, not an overwrite; the interleaving that destroys a read-modify-write ledger merely *reorders contributions* to an accumulation, and an accumulation does not care. Chapter 1's replicated counter forgave the Post's disorder for this reason; Chapter 7's kettles wed under a join that forgot order by design; and the largest optimization processes ever run survive raw lockless concurrency exactly as the shopping cart did. Debt #4, issued in Chapter 1: half paid at the kettle wedding with a semilattice; the remainder paid here, wearing gradients.

The honesty clause. This is *rhyme, not theorem*. Gradients are not a semilattice: an update is not idempotent (apply mine twice and the model has moved twice); there is no join, no once-absorbed-always-absorbed, and Chapter 7's convergence theorem does not apply. What holds is an *analytic* tolerance: the optimization, itself a noisy, error-correcting process, absorbs bounded disorder as one more source of noise. The negotiated middle is *stale-synchronous*: a staleness window — workers may run ahead of the slowest by at most s steps, enforced at the barrier — which the reader can now read at sight as bounded staleness, Chapter 7's honest tier, a session guarantee for gradients, the drift's cost measured in convergence rather than in customer complaints.

Remark 12.4 (staleness: statements with pointers). Per the honesty clause, stated and not proved here. *Hogwild* (Recht et al. 2011): for sparse problems with suitable step sizes, lock-free concurrent updates achieve convergence rates comparable to serial execution; the mechanism is that updates are *increments* (commutative) and collisions are rare (sparsity). *Stale-synchronous* (bounded staleness s): convergence guarantees degrade gracefully in s, recovering the synchronous rates as $s \to 0$. The rhyme with Chapter 7 is deliberate and bounded: gradients commute but are not idempotent and form no semilattice — the tolerance is analytic (noise absorption), not algebraic (join), and the desk labels it so. Pointers in the reading.

Floating point, and the fashion. One caveat the practitioner meets before the theorist: *floating-point addition does not associate*. Sum the same gradients in a different order and the last bits differ; the reduction order, the worker count, the very topology change the arithmetic, and two runs from the same seed on differently shaped fleets diverge — bit by bit, then visibly. Whence the discipline: bitwise-reproducible training fixes the reduction order (Box 29 does; Rem. 12.2) — Chapter 5's determinism diet enforced down to the

rounding. The fashion's arc is familiar: the asynchronous school flourished when fleets were small, heterogeneous, and rented; the frontier estates, on purpose-built pods of engineered uniformity, marched back to full synchrony — the barrier's tax fell as the hardware homogenized, and coherence bought cleaner convergence, comparable experiments, and determinism for debugging runs that cost more than office buildings (Chapter 8's Calvin aside, cashed). Asynchrony keeps its honest provinces: fleets too heterogeneous to march — spot instances, mixed generations — and the federated frontier, learning across telephones that are Chapter 1's documentary weather personified, where synchrony is not expensive but *meaningless* and the parameter-server pattern, staleness budgets and all, remains the only sane church. Per layer, as ever.

12.4 Part Four — The Geometry: Model, Pipeline, and the Optimizer's Estate

When the model outgrows one machine's memory, Chapter 9 takes over entirely. The frontier estates run all three geometries at once — tensor shards within a pod, pipeline stages across pods, data-parallel replicas across the hall — every accelerator addressed by its coordinates in a three-axis seating chart, each axis's chatter arranged onto the wires that can afford it. The composition is the design problem; the axes themselves are already owned.

Tensor parallelism. Shard the individual weight matrices across accelerators — range partitioning of the parameter space itself — so that each layer's arithmetic becomes a small collective among the shardholders, fired within every forward and backward pass: agreement not per round but per layer, thousands of little all-reduces a second. It therefore lives strictly inside a pod, on interconnect measured in terabits, and dies of latency anywhere else — Chapter 9's co-location doctrine with the speed of light billing per centimetre.

Pipeline parallelism. Shard the model by layers and stream micro-batches through the stages like parcels on a conveyor — Chapter 10's dataflow discipline with the operators pinned to silicon and the buffers audited to the megabyte. The tax is *the bubble*: stages idle at the head and tail of every round while the conveyor fills and drains — with eight stages and eight micro-batches in flight, roughly half the conveyor's area; with sixty-four in flight, toward an eighth. The classic remedy — many parcels airborne — is bought with activation memory: Chapter 10's buffer-versus-latency ledger, backpressure enforced by the conveyor itself.

The optimizer's estate. The revelation industrialized as ZeRO — spoken *zero*, per the lexicon — and its fully-sharded descendants: the heaviest tenants were never the weights but the optimizer's bookkeeping — momenta, variances, master copies, several times the model's own weight — replicated in the naive arrangement on every one of ten thousand workers, though Chapter 6's rationale for replication, durability, is conspicuously absent. The repair proceeds in the staged discipline of a good rebalance: shard the optimizer state first — each worker keeps only its slice, which updates from the summed gradient everyone already holds; then the gradients; then the weights themselves, gathered layer by layer just in time for each layer's arithmetic and released behind it — a working set streamed through memory the way Chapter 10 streamed rivers. The memory bill falls by the fleet size; the payment is more traffic on wires the ring taught you to budget. Partition the state, not merely the data: where things live, one final time, deciding what fits at all.

12.5 Part Five — Failure on a Schedule

The arithmetic that turns the course's climate into a calendar entry. The components are individually excellent; the indictment is of multiplication. Grant a part a mean time between failures of five years — generous — and put ten thousand in one job: the fleet's mean time between failures is M_{part}/n — north of five failures a day, one every four or five hours, on schedule, forever. Generous twice over: parts age, new parts arrive with fabrication's infant troubles, and the failures that matter include the optics, the switches, and the attendants' software (Chapter 1's catalogue), so the observed cadence at frontier scale runs faster than the arithmetic, not slower. A month-long run on that fleet is killed every working afternoon of its life — Chapter 1's climate, quantified; a job that loses everything at each interruption never finishes at all.

The sharded photograph. Petabytes of optimizer state cannot march through one door, so the checkpoint is *sharded*: each worker writes its own slice, in parallel, to nearby storage — the photograph partitioned exactly as the state it depicts — with a *manifest*, a small court of record in Chapter 8's habit, declaring which slices constitute a blessed, complete cut. Restore is the gather; a torn checkpoint, half-written at the moment of death, is excluded by the manifest the way a coordinator's unforced decision never happened.

How often is arithmetic, not folklore. Two costs pull against each other: the standing tax δ/τ, paid always, falling as the interval lengthens; the expected rework $\tau/(2M)$, about half an interval per failure, rising with it — and the sum bottoms, with the inevitability of all good square roots, at the Young–Daly interval.

Theorem 12.3 (Young–Daly interval)**.** *In the checkpoint model, the overhead fraction of wall time for checkpoint interval τ is, to first order,*

$$f(\tau) \;=\; \frac{\delta}{\tau} \;+\; \frac{\tau}{2M},$$

(standing tax plus expected rework rate), minimized at

$$\tau^{*} \;=\; \sqrt{2\,\delta M}, \qquad f(\tau^{*}) \;=\; \sqrt{\tfrac{2\delta}{M}}.$$

Derived as the guided Problem 12.5; with δ five minutes and M four hours, τ^{*} is about fifty minutes and the overhead about twenty percent — the finale's worked numbers. The sensitivity is the practitioner's clause: halving δ halves the overhead but tightens τ^{*} only by root two — invest in cheap photographs, not long gambles.

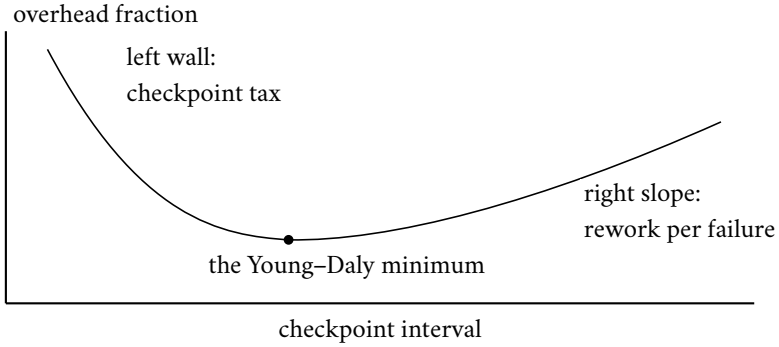


Exhibit 12.2 (the cadence curve). Standing tax δ/τ falls, expected rework $\tau/2M$ rises; their sum bottoms at $\sqrt{2\delta M}$ (Thm. 12.3, Problem 12.5). Argue with the formula, not with opinions — and note how flat the basin is: the win is in cheapening δ, which lowers the whole curve.

With the finale's numbers — a five-minute checkpoint, a four-hour cadence — the formula prescribes roughly every fifty minutes, a standing tax of about a tenth. The practical gift is the sensitivity: halve δ — asynchronous checkpointing, sharded writes, the photograph taken without stopping the river, Chapter 10's whole craft — and the tax halves outright while the interval tightens only by root two; the engineering effort belongs on cheapening the photograph, not on gambling with the interval.

The straggler swap. The barrier makes every straggler everyone's straggler, and a gradient worker is stateful where Chapter 10's map tasks were pure — so the estate deploys the whole of Chapter 11: phi-grade detection on step-completion times (the round barrier is a fleet-wide heartbeat of perfect regularity, the failure detector's dream client); hedged scheduling of the input pipeline, where the work *is* stateless and the duplicates race free; and hot spares standing by the ring. The swap, in one motion: the detector's dial crosses the acting threshold; at the next barrier — a natural quiescent point, no Chandy–Lamport gymnastics required — the membership epoch increments and the ring re-forms with the spare spliced in; the spare loads the current weights from a peer or the last photograph and resumes the departed machine's data shard from the committed cursor (Chapter 10's bookmark); and the late parcels, stamped with the dead epoch, are refused at every doorstep rather than folded into sums they no longer belong to. Seconds of pause, not a restart — the doppelgänger's oldest trick answered by the season's oldest token. The epochs are decreed by a small controller — replicated, elected, fenced: Chapter 5's consensus kernel in a lab coat, holding the tiny truths (the member list, the golden checkpoint's manifest, the current epoch) while the herd holds the tonnage. Debt #24, presented at batch scale in Chapter 10, paid at silicon scale here. The assembly, boxed:

Protocol — Box 30. The elastic training step (epochs on every collective)

1: **controller** (a small replicated parliament, Ch. 5): holds member list, current epoch e, blessed checkpoint manifest
2: **each round:** workers compute; all-reduce stamped with e; any parcel bearing $e' \neq e$ is refused (fencing, Ch. 5)
3: **straggler detected** (phi on step times, Ch. 11): at the next barrier, controller increments e, splices the hot spare into the ring, publishes the new view
4: spare loads weights from a peer or the manifest's checkpoint; input cursor resumes from the committed bookmark (Ch. 10)
5: **on failure mid-round:** abort the round; view change as above; recompute the round — rounds are idempotent by determinism from an agreed state
6: **checkpoint every τ^*:** each worker writes its shard; manifest committed by the controller when all shards land; recovery = restore manifest's cut, rewind cursors, replay

Chapters four, five, ten, and eleven, snapped together; the biggest computation governed by its smallest component.

The mercurial cores. The deferred horror, kept in a locked drawer since Chapter 5's Byzantine hour: the fleet studies of Meta and Google — published, sober, quietly hair-raising — document *silent data corruption*: not disks lying about writes but cores lying about *arithmetic*. The *mercurial cores* (the fleet study's own coinage) — one in some thousands — under particular operands, temperatures, and voltages return the wrong answer to a correct computation and raise no flag. At estate scale that is a standing population of liars — Aunt Agatha, tenured, in the arithmetic unit, her final posting — and their products are not crashes but *poisoned numbers*: a gradient with one wrong element, a checkpoint with a corrupted page,

propagating through a computation whose entire posture assumes the arithmetic is honest. The sobering clauses: rates far above the certification models; errors data-dependent and intermittent, so the offending core passes every standard test; damage characteristically quiet, the investigation consuming weeks while every component reports itself healthy — gray failure (Chapter 11), driven down into the transistors.

Remark 12.5 (silent data corruption, economically). The fleet studies ("Cores that don't count," Google; Meta's companion) document cores that miscompute data-dependently, intermittently, and silently, at rates far above certification models — the Byzantine ghost below the protocols. Detection is an economics ladder: full duplication (gold, double cost, reserved for the un-wrongable); random spot-duplication (statistical coverage at a few percent); targeted screening in idle cycles (catches the reproducible); checksums on everything that moves (in-flight and at-rest, blind to birth defects); checkpoint-time validation (poisoned runs caught within one τ^*); and invariant tripwires on the loss (the last, cheapest, slowest alarm). The remedy's shape is the end-to-end argument's final appearance: verify where the meaning lives, by comparison at the ends — here, the ends of the arithmetic itself.

The arithmetic unit was the last *middle* anyone thought to distrust, and the remedy — verify at the level that owns the meaning, by ends-side comparison — is the end-to-end argument's final word, delivered one layer beneath where anyone expected to need it.

12.6 Part Six — Serving, Briefly: The Consistency Bug With a Profit Line

The estate's other half, briefly — and the trade's own postmortems suggest more machine-learning value dies in the gap this part names than in all the training outages combined; the training failures are loud, and this one whispers.

Training-serving skew. A consistency violation between two replicas of the same computation. One staging suffices: the training pipeline computes "customer's average purchase this month" in batch, from the log, the month complete and the late records settled; the serving path computes "the same feature," reimplemented by a different team in a different language, online, from a cache, mid-month — and the two disagree in a hundred small ways nobody itemized: null handling here, timezone there, late data in one and dropped by the other. The model degrades for a reason no model metric will name, because the bug is not in the model — it is *between two copies of a computation that no one declared to be replicas*, and therefore no one monitored for divergence (Chapter 7's vocabulary, exactly). The remedy is Chapter 10 verbatim: one feature definition, materialized to both consumers from the same log — the inside-out doctrine ending the schism by abolishing the second codebase, as Kappa ended Lambda's — with the residual monitored like any replication pair: sampled online features diffed against their batch recomputation, divergence alarmed before it is a revenue line.

The serving fleet, read at sight. Model replicas behind a balancer: Chapter 6 at its most benign — inference is stateless and reads divide. Rollouts are view changes with canaries — a new model version admitted to a sliver of traffic, watched, then promoted: Chapter 11's blast-radius discipline applied to weights instead of code. Rollback is the rewind, kept cheap by keeping the old version warm. And *feature freshness is replication lag with a profit-and-loss line*: the model reading a user's behaviour thirty seconds stale is Chapter 6's follower read, except that here the staleness has a measurable revenue gradient — bounded staleness with a number, Chapter 7's honest tier, priced in currency per second of lag. The Chapter 7 catechism transplants whole: eventual, plus what? Plus which session guarantees for the user's own actions, plus what staleness bound at what revenue cost, signed by whom.

12.7 Part Seven — The Grand Audit, and the Arc Retraced

A season closes with a curtain call, the books, and the arc; the episode stages all three in full, and the audit is itemized in §12.8, permanent equipment. The record here, in the desk's compact hand.

The company.

- **The two generals,** who opened the season on their hill and stand at its close — the field's first and final image.
- **The Post,** the magnificent villain — loser of letters, breeder of twins, perjurer of timestamps — hired, in the finale's finest irony, by the simulation harness of the very trade it tormented.
- **The doppelgänger,** the master key — Two Generals, FLP, the Byzantine bound, the false death, the blocking proof: what cannot be distinguished cannot be acted upon differently — retired a chapter ago, pricing published.
- **The crowded club,** whose Sunday-shift lemma held up four chapters of quorums; **the log,** the understudy who became the lead; **the photograph,** forty years from theorem to overalls.
- **The standing cast:** Bingo, coordinator, head of chains, officiant; Tuppy, literal-minded waiter, in-doubt widow, on-call doctor; Gussie — crashed, lagged, decommissioned, dead in a storm, degraded once more in this finale — the season's most put-upon machine; Barmy, rehabilitated from crash-prone understudy to hot spare, the arc of the season in one career; Oofy, benched with distinction; Bertie, whose carts, transfers, tunnels, and telephones supplied every crime scene; the aunt, formal test and flower-committee celebrity, retired undefeated; and Aunt Agatha, the Byzantine terror, tenured, at the last, in the arithmetic unit.

The books. Twenty-four notes were issued across twelve evenings, and twenty-four were paid — itemized, issue to payment, in the register table of §12.8. The shape of the accounts: the founding evening wrote four, one of them an unnumbered drawer note about commutativity forgiving disorder — caught by the register, which counts what is filed rather than what is announced — and that note outlived them all, half paid in Chapter 7 under the wedding canopy, the remainder in Part Three above, wearing gradients. The most patient debt was the most profitable: crashed versus slow — managed in Chapter 3, where it powered an impossibility; hedged in Chapter 5, where fencing made its losses survivable; priced and dialled in Chapter 11, where phi printed the odds on the face. The longest-held note, the photograph in industrial costume, waited eight evenings and was paid in Chapter 10, its spirit taking a third bow here at petabyte weight, a manifest for a blessing. And the last note, the racing duplicates from the MapReduce paper's one immortal page, was paid in this chapter, with the spares standing by the ring. The early evenings issued, the middle evenings traded, the last evenings only paid — because a course that ends owing nothing was the constitution's quiet definition of *finished*.

The arc, priced. From one drowning messenger the course derived a world, and not one impossibility of Chapter 3 has been repealed; not one system in twelve chapters violated the Two Generals proof, and their descendants agree five million times a day anyway — not by refuting theorems but by wanting what the theorems do not forbid: convergence rather than certainty, idempotent action under imperfect knowledge, a priced approximation the business can bank. Nothing was ever solved. It was *priced*, and the price list is the season's true syllabus:

- **The Two Generals** were never satisfied — bought off with idempotence and retries, certainty exchanged for convergence.
- **FLP** was never overturned — rented around with partial synchrony, randomness, and failure detectors, liveness purchased by the timeout and repossessed in every storm.
- **CAP** was never escaped — *chosen*, per room, per table, per feature flag, Chapter 7's field guide being the choice made legible.
- **The shared clock** was never restored — replaced by causal paperwork where messages suffice, purchased outright, in caesium, where they do not.
- **Crashed versus slow** was never told apart — the confusion dialled, fenced, and budgeted until it cost less than the certainty would have.

Engineering, the course has argued from its first hour, is the discipline of paying for what mathematics refuses to give away free, and the register is twelve evenings of exactly those receipts. The three unpleasant facts of the founding arc — machines fail independently, messages are delayed or lost, there is no shared clock — read, at the season's end, as design instincts: assume the failure and make it boring; carry the causality or buy the clock; never trust the middle with the ends' correctness. And the closing wink, laid in the plan from the first day: stochastic gradient descent survives asynchrony for the same reason the shopping cart does — the merge is a join, near enough, in the mean, with the caveats duly sworn; addition forgets nothing but order, and order, for a sum as for a cart, was never the point. The humblest object in this course and the grandest computation on earth, kneeling at the same commutative altar: Chapter 1 placed that counter in a drawer with a glint and a promissory note; twelve chapters later the note retires the season.

12.8 The Grand Tables

The course dictionary (the finale's Part One, tabled).

The estate says	The course says (chapter)
training step	barrier-synchronized round (4)
all-reduce	agreement on a sum; below consensus by the race test (6, 12)
checkpoint	consistent snapshot; the photograph (2, 10, 12)
straggler	the unluckiest machine; backup tasks and hedges (10, 11)
parameter staleness	replication lag; eventual-plus-what (6, 7)
elastic training	view change with epoch fencing (4, 5)
data pipeline	the log: replayable input, cursors (10)
job scheduler	placement against failure domains (9)
feature store	materialized view of one log (10)
loss-curve monitoring	the watermark; invariant tripwires (10, 11)
gradient accumulation	batching: latency for throughput, the oldest trade (5, 8)
training-serving skew	an undeclared replica pair (6, 7)

The debt register, final (issue → payment; the grand audit of the finale's Part Seven, ruled).

Debt	Issued → paid
1. The Post learns to forge	E1 → E5 (theorem and aunt)
2. Crashed versus slow	E1 → E3, E5, E11 (priced, dialled)
3. No shared clock	E1 → E2 (happens-before)
4. Commutativity forgives disorder	E1 → E7 (semilattice), E12 (gradients)
5. Physical clocks return, armed	E2 → E8 (TrueTime, commit-wait)
6. The photograph, industrialized	E2 → E10 (barrier snapshots)
7. Order rented from a leader	E2 → E4
8. Causal delivery as a contract	E2 → E7
9. Partial synchrony cashed	E3 → E4
10. Suspicion services priced	E3 → E5, E11 (phi-accrual)
11. CAP's C defined	E3 → E7 (linearizability)
12. One value becomes a log	E4 → E5
13. "Single machine" named	E4 → E7
14. Reconfiguration's dragons	E4 → E5
15. The log stars	E5 → E10
16. Lease-read fine print	E5 → E7
17. The kettles must merge	E6 → E7 (SEC theorem)
18. Serializability across shards	E7 → E8
19. Jepsen industrialized	E7 → E11
20. Where things live	E8 → E9
21. Partitioned computation	E9 → E10
22. Stampedes and storms	E9 → E11 (metastability)
23. The retry's dark side	E10 → E11
24. Backup tasks return	E10 → E12 (spares by the ring)

12.9 The Whiteboard

For the design review; the season's last.

1. Choose synchrony per layer: synchronous gradients where the mathematics wants coherence, asynchronous data pipelines where the log absorbs the slack, checkpointed everything, always. The barrier is a purchase, the ring is its efficient form, and the straggler is its standing invoice — detect, hedge, and swap by view change, with the epoch on every collective; a fleet without hot spares has decided, implicitly, that every straggler is worth ten thousand machines' idleness, and should at least decide it explicitly.

2. An all-reduce is not consensus. Agreement on a sum arbitrates nothing, excludes nobody, and therefore costs wire speed instead of a parliament; the moment the estate needs a *decision* — membership, leadership, which checkpoint is golden — the race test fires and Chapter 4 presents its bill. Know, at every layer, which of the two you are paying for, and be suspicious in both directions: a parliament asked to move gradients is a bottleneck by design, and a collective asked to arbitrate membership is a split brain on order.

3. Checkpoint cadence is arithmetic, not folklore: $\sqrt{2\delta M}$, then argue with the formula rather than with opinions. The failure rate at fleet scale is a schedule; treat rework as a budget line and the interval falls out — and spend the engineering on cheapening the photograph, where the returns are linear,

rather than on stretching the interval, where the square root blunts them. A blessed checkpoint needs a manifest; a torn one needs to be impossible to mistake for blessed.

4. Skew is a consistency bug wearing a machine-learning costume, and freshness is replication lag with a revenue line. One definition, one log, both consumers — and monitor the divergence between training and serving the way Chapter 11 taught you to monitor anything: before it is a metric the business notices. Two computations nobody declared to be replicas will diverge exactly as thoroughly as two that were declared and neglected; the declaration is the whole of the fix.

The register. Paid: #4's remainder (commutativity forgives disorder, wearing gradients — the rhyme honestly labelled) and #24 (the backup tasks, at silicon scale, with the spares by the ring). Issued: nothing. The register is closed: twenty-four issued, twenty-four paid, nothing outstanding, nothing silently forgotten — the grand audit itemized in §12.8. Episode word count: 10,024 — above the floor. Season status: twelve episodes, twelve chapters; the assembly session (continuity audit, consolidated registers, errata from sir's listening) remains per the constitution's schedule.

12.10 Problems

Tier I — Calisthenics

Problem 12.1 (parcel arithmetic)**.** A gradient of four hundred gigabytes, links of fifty gigabytes per second each way. (a) Compute per-node traffic and bandwidth-regime wall time for rings of four, one hundred, and ten thousand nodes. (b) At what message size does the latency term $(2(n-1)\alpha$, with α ten microseconds and n one thousand) equal the bandwidth term — and which collective shape does the answer recommend for gradient chunks versus for the tiny control messages? (c) State, in one sentence, why the answer to (a) barely changed across three orders of magnitude of fleet.

Problem 12.2 (the schedule)**.** Ten thousand parts at five-year part MTBF. (a) Fleet failure cadence, per day. (b) A month-long run: expected interruptions. (c) The vendor doubles part reliability; how does the answer change, and what does that say about relying on parts over process? (d) With δ four minutes and the cadence from (a), compute τ^* and the overhead fraction.

Problem 12.3 (which agreement)**.** Classify each as sum-grade (collective) or decision-grade (parliament), with the race-test sentence: (a) the averaged gradient; (b) which of two divergent checkpoint manifests is golden; (c) the global batch count; (d) whether node forty is a member this round; (e) the maximum loss across workers.

Problem 12.4 (staleness taxonomy)**.** Translate into Chapter 7's vocabulary and name the price: (a) fully synchronous training; (b) Hogwild; (c) stale- synchronous with window three; (d) a parameter server with unbounded worker lag; (e) federated rounds with stragglers dropped.

Tier II — The Real Thing

Problem 12.5 (Young–Daly, guided)**.** Derive Thm. 12.3. (a) Justify the overhead model $f(\tau) = \delta/\tau + \tau/(2M)$: where does the half come from, and what first-order assumptions are made? (b) Minimize f and obtain τ^* and $f(\tau^*)$. (c) Recompute the finale's numbers (δ five minutes, M four hours) and then the sensitivity: δ halved, M halved, both. (d) State two effects the model ignores (failures during checkpoint and recovery time itself) and argue the direction each pushes τ^*.

Problem 12.6 (the hierarchical budget)**.** Two pods of eight nodes; intra-pod links of two hundred gigabytes per second, inter-pod fibre of twenty. Design the hierarchical all-reduce (reduce-scatter within pods; exchange between; gather within), compute each phase's time for a one-hundred-sixty-gigabyte gradient, identify the bottleneck, and compare against one flat sixteen-node ring forced through the fibre. State the general rule you have rediscovered.

Problem 12.7 (elastic membership, designed)**.** Design Box 30 in full from Chapters 4, 5, 10, and 11: the controller's replication and election; the epoch's issuance and its fencing check in the collective; the spare's state transfer (peer copy versus manifest restore — when each); the input cursor handoff; and the abort-and-recompute rule for mid-round failures, with the determinism assumption that licenses it. Then break your own design twice: remove the fencing and exhibit the corrupted sum; remove the controller's replication and exhibit the stalled fleet.

Problem 12.8 (sabotage: the uncorrected latecomer)**.** An asynchronous scheme applies each arriving gradient to the *current* weights with full step size, however stale the weights it was computed against, no staleness cap, no correction. Characterize the drift: (a) write the update mismatch for a gradient computed at version t and applied at version $t{+}s$; (b) on a one-dimensional quadratic with two workers, one fast and one slow, trace six updates and exhibit the systematic overshoot the slow worker's stale gradients inject near the optimum; (c) state the two standard repairs (staleness cap; staleness-scaled step size) and which Chapter 7 posture each corresponds to. *(Solution in full: the certification of drift, per the standing rules, with the trace written out.)*

Tier III — For the Ambitious (starred)

Problem 12.9 (★ the spare pool)**.** Given the fleet cadence of Problem 12.2, a swap time of ninety seconds, a repair turnaround of six hours, and the cost ratio of an idle spare to a stalled fleet, model the spare-pool size that minimizes expected cost; exhibit the regime where zero spares is defensible and the regime where the answer is "a full rack." *(Hints only.)*

Problem 12.10 (★ the two-ledger bound, tightened)**.** Formalize Thm. 12.2's counting argument: define the algebraic model (linear combinations over generic reals), prove the per-node receive and send floors exactly, and determine whether the $2(n{-}1)/n$ factor is achievable simultaneously at *every* node by any algorithm other than ring-equivalent schedules. *(Hints only.)*

12.11 Hints

Hint (Problem 12.3)**.** (e) is the instructive one: a maximum is associative, commutative, idempotent — sum-grade machinery suffices, and it is even a semilattice, unlike the sum.

Hint (Problem 12.5)**.** (a) Failures land uniformly within an interval in the memoryless model; the mean loss is half. (d) Both push τ^* up slightly; write the second-order term.

Hint (Problem 12.8)**.** (b) Take the quadratic with minimum at zero, learning rate below the stability bound for serial descent; let the slow worker's gradient arrive three steps late, computed at a point the fast worker has since crossed. The stale gradient points *away* from the optimum on arrival.

Hint (Problem 12.9)**.** Poisson arrivals against a repair pipeline: the spare pool is a buffer sized against the arrival rate times the turnaround, plus safety stock at the service level the fleet-stall cost implies.

12.12 Solutions

Tier I in full (tersely); Tier II in full — Problem 12.8 is the sabotage certification; hints only for Tier III.

Solution (to Problem 12.1)**.** (a) Traffic $2(n-1)G/n$: at n=4, six hundred gigabytes; at n=100, seven hundred ninety-two; at n=10^4, essentially eight hundred. Wall time at fifty gigabytes per second: twelve, just under sixteen, and sixteen seconds respectively. (b) Latency term: two thousand hops at ten microseconds is twenty milliseconds; the bandwidth term matches it at $G \approx$ one gigabyte for this n — so: rings for the gradient's giant chunks, trees for the small control traffic, exactly the courier's standing practice. (c) The fleet size cancels in the traffic formula: the ring's whole point, and the reason the training estate can grow without renegotiating its wires per node.

Solution (to Problem 12.2)**.** (a) Part MTBF sixty months; fleet cadence sixty months over ten thousand: about four and a half hours; north of five per day. (b) Roughly one hundred sixty. (c) Doubling part reliability halves the cadence to eleven-ish a week — still a schedule, not an event: process (checkpoints, spares, swaps) dominates parts at fleet scale, which is the paragraph's whole point. (d) $\tau^* = \sqrt{2 \cdot 4 \cdot 270}$ minutes $\approx$ forty-six minutes; overhead at the optimum $f(\tau^*) = \sqrt{2\delta/M} = \sqrt{8/270} \approx$ seventeen percent — painful, and exactly why the engineering (Thm. 12.3's sensitivity clause) attacks δ rather than the interval: at δ one minute the optimum overhead falls below nine percent.

Solution (to Problem 12.3)**.** (a) Sum-grade. (b) Decision-grade: two candidates, one winner — a race; the controller's parliament rules. (c) Sum-grade (a count is a sum). (d) Decision-grade: membership is the archetypal arbitration (Chapters 4, 5). (e) Sum-grade and better: max is idempotent — a true semilattice, the only entry in the list that would have satisfied Chapter 7's wedding requirements outright.

Solution (to Problem 12.4)**.** (a) Strict coherence: every read of the weights sees the latest agreed version — the linearizable tier, bought with barriers. (b) Eventual-plus-sparsity: unbounded staleness in principle, tolerated analytically — the floodplain with an actuary's note. (c) Bounded staleness, window three — Chapter 7's honest middle tier, priced in convergence rate. (d) Eventual with no bound: the costume without insurance; the desk declines to endorse. (e) Synchronous per round with membership churn — a view-change regime where the dropped stragglers are the price of the barrier.

Solution (to Problem 12.5)**.** (a) Per interval of length τ: one checkpoint, cost δ (tax rate δ/τ); failures arrive at rate $1/M$ and destroy, in expectation, half the interval's work (uniform landing under memorylessness): rework rate $\tau/(2M)$. First order: $\delta \ll \tau \ll M$, failures during checkpointing and recovery ignored. (b) Set $f'(\tau) = -\delta/\tau^2 + 1/(2M) = 0$: $\tau^* = \sqrt{2\delta M}$; substitute for $f(\tau^*) = \sqrt{2\delta/M}$. (c) Five minutes and two hundred forty: $\tau^* = \sqrt{2400} \approx$ forty-nine minutes, overhead $\approx \sqrt{1/24} \approx$ twenty percent; halving δ: thirty-five minutes and fourteen percent; halving M: thirty-five minutes and twenty-nine percent; both: twenty-four minutes and twenty percent — the overhead scales as $\sqrt{\delta/M}$, each lever a square root. (d) Failures mid-checkpoint waste the attempt: push τ^* up (checkpoint less often than naive); recovery time adds a constant per failure, indifferent to τ but raising total overhead — both second-order in the stated regime, both worth modelling when δ/M is not small.

Solution (to Problem 12.6)**.** Within pods: reduce-scatter eight ways at two hundred gigabytes per second: $\frac{7}{8} \cdot 160/200 = 0.7$ seconds. Between pods: each of eight partner pairs exchanges its parcel share — twenty gigabytes each way over the twenty-gigabyte fibre split across pairs: with one fibre, $2 \cdot 160/8$ (the pod-reduced shares) over twenty = two seconds — the bottleneck; with a fibre per pair, a quarter second. Gather within: another zero point seven. Flat sixteen-ring through the fibre: nearly all thirty of the $2 \cdot \frac{15}{16} \cdot 160$ gigabytes cross the fibre at twenty: fifteen seconds. The rule: hierarchy confines the unavoidable inter-pod

traffic to the pod-reduced residue — Chapter 9's commandment, heavy traffic on short wires, with the arithmetic showing a factor of five to sixty depending on the fibre count.

Solution (to Problem 12.7)**.** Controller: three or five nodes, Raft-grade (Ch. 4), leader issues epochs; epoch in every collective header, checked by every participant before folding a parcel (Ch. 5 fencing). Spare transfer: peer copy when a healthy replica holds current weights (fast, no storage round trip); manifest restore when the round aborted or peers are suspect. Cursor handoff: the committed offset vector lives in the manifest; the spare resumes from it (Ch. 10). Mid-round failure: abort, view-change, recompute — licensed because a round is a deterministic function of (agreed weights, agreed batch), so recomputation is idempotent in effect (Ch. 10's trinity). Break one: without fencing, the dead view's straggler folds a stale parcel into the new view's sum — a corrupted average applied everywhere; determinism then faithfully replicates the corruption, the diet's dark side. Break two: an unreplicated controller is a single point whose crash stalls every view change — the fleet runs until the first straggler, then waits forever: Chapter 8's blocking, self-inflicted.

Solution (to Problem 12.8 — the certification)**.** (a) The scheme applies $-\eta \nabla f(w_t)$ at w_{t+s}; the serial process would apply $-\eta \nabla f(w_{t+s})$; the mismatch per update is $\eta(\nabla f(w_{t+s}) - \nabla f(w_t))$, which grows with both the staleness s and the curvature crossed in the interim — worst precisely near sharp optima, where the landscape changes fastest. (b) The trace, written out. Quadratic $f(w) = w^2/2$ (gradient $= w$), $\eta = 0.5$, start $w = 4$. Fast worker updates every step; slow worker's gradients arrive three steps late. Steps: (one) fast, grad at 4: $w = 4 - 2 = 2$. (two) fast, grad at 2: $w = 1$. (three) fast, grad at 1: $w = 0.5$ — honest descent, nearly home. (four) *slow worker's parcel arrives*, computed back at $w = 4$: applied now, full step: $w = 0.5 - 0.5 \cdot 4 = -1.5$ — flung past the optimum to the far side, three times farther out than it stood. (five) fast, grad at -1.5: $w = -0.75$. (six) the slow worker's next stale parcel (computed at 2): $w = -0.75 - 1 = -1.75$ — driven *outward* again. The pattern is systematic, not noise: every stale parcel points where the optimum *was*, and near convergence the stale corrections dominate the honest ones — the iterate orbits or diverges instead of settling, and the run's loss curve shows the signature sawtooth every practitioner has met and few have named. Certified as drift, per the standing rules: a deterministic two-worker schedule, six steps, systematic displacement away from the optimum, no randomness required. (c) Repairs: a staleness cap — refuse parcels older than $s_{\max}$ (bounded staleness: Chapter 7's middle tier); or staleness-scaled steps — damp η by the parcel's age (graceful degradation, the actuary's posture). Uncapped, uncorrected lateness is the floodplain costume with weights on: eventual, plus nothing, minus convergence.

12.13 The Reading, Consolidated

The season's shelf, organized for the reader going onward; each chapter's own list stands, and this essay points across them.

The spine, in six papers. Lamport's time-clocks paper (1978; Ch. 2) — reread last of all: the whole course is in it, folded. FLP (1985; Ch. 3), for the shape of impossibility. "Paxos Made Simple" (2001; Ch. 4), for agreement rented honestly. Herlihy–Wing (1990; Ch. 7), for the summit named. The Dynamo paper (2007; Ch. 6), for the other church, priced. Kreps's "The Log" (2013; Ch. 10), for the seam that joins them all.

The industrial texts. Spanner (Ch. 8) — the course's single most load-bearing industrial paper; GFS, MapReduce, and Spark (Ch. 10) — the lineage arc; Bigtable and Chubby (Chs. 5, 9) — the directory church's liturgy; Kafka's design documentation (Ch. 10); the parameter server (Ch. 12).

The corrective literature. Berenson et al. on isolation folklore (Ch. 7); the metastable-failures paper and the DynamoDB postmortem (Ch. 11); gray failure (Ch. 11); "Cores that don't count" with Meta's companion (Ch. 12); Kingsbury's analyses throughout — the consumer-protection bureau whose existence improved every brochure it audited.

The methods. Chandy–Lamport (Ch. 2) beside Carbone et al. (Ch. 10), read as one paper forty years apart; the Amazon formal-methods report with Ch. 11's appendix as on-ramp; the FoundationDB testing lineage; the Brooker Builders' Library essays, for the reflexes.

The standing companion. Kleppmann, whole, at last: the course's chapters map onto his as siblings rather than as summaries, and the reader finishing this volume will find him now readable in an afternoon per chapter, which was, in part, the design.

Onward. For depth in theory: Lynch's *Distributed Algorithms* and Cachin–Guerraoui–Rodrigues for the proofs this desk structured but did not exhaust. For depth in practice: the SRE and Builders' Library corpora, read with Ch. 11's instruments. For the frontier this finale opened: the systems-for-machine-learning literature moves too fast for a reading list to hold; the durable preparation is the season's method itself — find the round, the log, the photograph, and the race test in whatever arrives next, and read its brochure with the register open. *The three unpleasant facts will be waiting there too, sir. They are the terms of trade.*

Chapter 13

The Full-Time Parliaments

Companion to Episode Thirteen — the supplementary sitting. The episode reads the tenancy agreement: etcd anatomized — the revision store, the watch, the lease, the lock, the read-index ceremony — and Kubernetes revealed as one watch loop over a rented log; then Consul's federal constitution — the parliament above, the gossiping parish below — with SWIM walked to acquittal and conviction, Lifeguard's amendments, the registry's offices, and the Roblox affair read by the season's instruments. This chapter keeps the record: the store and watch contracts formalized; the read-index theorem proved from Chapter 4's lemmas; the parish register as a lattice, its convergence bought from Chapter 7 and its completeness proved on a schedule; the menus of both houses mapped onto the hierarchy; two exhibits; and the problems, including a sabotage certified fatal. Statements, proofs, price tags, problems.

13.1 Part One — Two Successors and Two Constitutions

The episode opens on a point of order: seven evenings of the season gesture at etcd — the coordination service beneath every Kubernetes control plane — and none dissects it; Consul, the registry office of a large fraction of the industry's datacenters, appears in the course not at all. This chapter is the concordance completed. Both services rent out Chapter 5's coordination kernel — membership, configuration, locks, elections, discovery — and the chapter's organizing contrast is constitutional:

- **The unitary constitution (etcd, 2013, CoreOS lineage):** one Raft parliament (Chapter 4), one disk-backed ledger with history, everything and everyone appearing before it; failure detection is centralized bookkeeping (leases filed with the house). Tenant of record: Kubernetes, which keeps its entire book of record — every object — in the store.
- **The federal constitution (Consul, 2014, HashiCorp):** a Raft parliament of three or five servers for the record, and beneath it a gossiping parish — an agent on every node of the estate, membership and failure detection by the SWIM protocol (Part Four) — with services registered and health-checked at the edge, where they live.

Model of this chapter

For the parliaments (Parts Two, Three, Five). Asynchronous network, partial synchrony assumed for liveness only (Chapter 3); servers crash-recovery with stable storage (write-ahead log, snapshots); fair-loss links with retransmission beneath (Chapter 1); crash quorums $f < n/2$ (Chapter 4); no Byzantine faults — both houses agree on what the leader says even when it is corrupted

nonsense (Chapter 5's caution stands). Leases and leader-lease reads assume bounded clock *drift*, never synchronized clocks (Theorem 5.3).

For the parish (Part Four). A group of n members with unique identifiers; each runs a local metronome cutting time into protocol periods of length T (no agreement on phase or rate required beyond bounded drift); the Post may lose, delay, duplicate, and reorder gossip, but not forge (crash faults only); a member's roster holds the $n - 1$ others. Claims about detection are stated in protocol periods.

13.2 Part Two — etcd: The Revision Store

The write path is Chapter 5's replicated state machine (Theorem 5.1) driven by Raft: append to the leader's log, persist to the write-ahead file, commit by majority, apply in order at every member, acknowledge. What earns etcd its own section is the machine being driven: the store keeps history.

Definition 13.1 (the revision store)**.** The store maintains a single counter *rev*, the **revision**, incremented once per committed write transaction. State is a map from keys to sequences of *generations*; each generation is a sequence of versions, each version stamped with the revision that wrote it. A key's record carries: *create* (the revision opening its current generation), *mod* (the revision of its newest version), and *version* (its ordinal within the generation). A delete closes the generation at its revision; a later write opens a fresh generation. A read may be issued *as of* any revision r: it returns the newest version of each requested key with stamp $\leq r$, in the generation open at r — a consistent snapshot of the keyspace as it stood at revision r.

The episode's staged trace, for the drills (the store's revision stands at seven before the first write):

1. Bertie writes the teapot at two pounds: revision 8; record ($create = 8,\ mod = 8,\ version = 1$).
2. Bertie re-prices it: revision 9; $(8, 9, 2)$.
3. Gussie deletes it: revision 10; the generation closes.
4. Bertie writes it anew: revision 11; $(11, 11, 1)$ — a fresh generation, ancestry severed.

A read as of 9 sees the second version; a watch from 8 replays the biography (modified at 9, deleted at 10, created at 11); after compaction at 10, the as-of-9 read is refused.

Definition 13.2 (compaction; the watch contract)**.** A *compaction at* c discards all versions with stamp $< c$ except each key's newest at c; thereafter reads as of $r < c$ fail with a *compacted* refusal. A *watch* on keys K from revision r delivers exactly the committed events (puts and deletes) on K with stamp $> r$, in stamp order, first replaying the retained past and then following the present; if r has been compacted, the watch fails with the same refusal, and the client's recovery is the informer ritual: fetch current state whole, note its revision, watch from there.

Remark 13.3. The store is Chapter 5's log wearing a filing cabinet: the revision is the log's index surfaced as data, an as-of read is Chapter 2's consistent photograph purchased retroactively, and the watch is Chapter 10's consumer cursor — resumable by number, honest about gaps only at the compaction boundary, where it refuses rather than guesses. Jepsen's 2020 audit of version 3.4.3 confirmed the contract as sold: key-value operations (transactions included) strictly serializable; watches delivering every change, in order.

Leases and locks. A lease is granted with a time-to-live, kept alive by heartbeat, and holds attached keys, which are deleted — through the log, at one revision — on expiry; expiry is judged on the leader's clock, conservatively, per Chapter 5's drift discipline (Theorem 5.3). The lock and election recipes are ZooKeeper's re-cut for a flat keyspace: contenders write keys under a prefix, lowest *create* revision reigns, each waiter watches the key immediately beneath its own. The fencing token is not a separate product:

Proposition 13.1 (the revision is a fencing token)**.** *Let $g_1, g_2, \dots$ be the successive holders of a lock, in the order the grants commit, and let c_k be the create revision of g_k's lock key. Then $c_1 < c_2 < \cdots$, and a resource that records the highest token presented and refuses any write stamped lower admits no write by g_j after it has admitted one by g_k with $j < k$. The create revision therefore satisfies Chapter 5's fencing contract: a rising number, totally ordered across handovers, enforceable at the point of effect.*

Proof. Grants are writes; writes are totally ordered by the log; the revision counter increments per committed write (Definition 13.1), so later grants bear strictly larger create revisions. The refusal rule then admits stamps in nondecreasing order of grant, and a stale holder's stamp $c_j < c_k$ arriving after any write stamped c_k is refused. □

Jepsen's audit says the same in the auditor's register: the lock interface alone is *fundamentally unsafe* in an asynchronous world — no lock service can promise mutual exclusion to clients that may pause (Chapter 5) — and the recommended repair is precisely Proposition 13.1: treat the lock as advisory and fence at the resource with the revision.

Reconfiguration. etcd amends its parliament one member at a time (Lemma 5.2 polices the eras), and ships Chapter 5's caution as a feature: a new member may join as a **learner** — receiving and replicating the log, voting not at all — promoted to the franchise by a second decree once caught up, so the quorum arithmetic never counts a gentleman who knows nothing.

The read, and its ceremony. Reads are linearizable by default; the naive implementation (drive the read through the log) prices a read as a write. The production ceremony:

Protocol — Box 31. The etcd linearizable read (ReadIndex; checked against Ongaro §6.4 and the etcd documentation)

1: **on** read request R at leader ℓ, term t:
2: $r \leftarrow \ell$'s commit index — the *read index* for R
3: **require** ℓ has committed an entry of term t (the term-opening entry)
4: send heartbeats; await acks from a majority Q, each at term t
5: wait until ℓ's applied index $\geq r$
6: answer R from local applied state
7: **serializable option:** any member answers from local applied state, no round, possibly stale

Theorem 13.2 (read-index reads are linearizable)**.** *In the model above, a read served by Box 31 reflects every write acknowledged before the read was invoked, and respects real-time order among reads so served: the reads are linearizable (Chapter 7's contract) against the log's write order.*

Proof. Let R be invoked at real time τ, served by ℓ in term t, with read index r and answering majority Q. Consider any write W acknowledged before τ.

If W was acknowledged by ℓ itself: ℓ acknowledges only after advancing its commit index past W's entry, so W's index $\leq r$, and the apply-wait (line 5) puts W in the served state.

If W was acknowledged by a different leader ℓ' of term $t' \neq t$: we show $t' < t$, whence W's entry is committed in a term before t and, by leader completeness (Theorem 4.7), sits in ℓ''s log; the required term-t entry (line 3) commits at an index above it, so W's index $\leq r$ and the apply-wait covers it as before. Suppose instead $t' > t$. Election at t' requires a majority Q'; by the crowded club (Lemma 4.1), some member $m \in Q \cap Q'$. m acknowledged ℓ's heartbeat *at term* t (line 4); a member's term never decreases, so m's vote at $t' > t$ happened after its ack. The election, hence every commit and acknowledgment of ℓ''s term — W's among them — completed after the round's ack, hence after the round began; but W was acknowledged before $\tau \leq$ the round's start. Contradiction; so $t' < t$.

Real-time order among Box 31 reads: each read's answer reflects a committed prefix of the log at least r long, and a later read's round completes after an earlier read's, so its read index is at least as large; prefixes grow monotonically, so a later read never shows an earlier state. Reads write nothing, so no other clause of the contract is engaged. □

Remark 13.4 (the menu, house of etcd). The serializable option (Box 31, line 7) returns a committed prefix from one member: no real-time clause, staleness bounded only by that member's lag — Chapter 7's vocabulary, printed on the option's name. Note the default: the strict item, discount by request. The federal house inverts this (Part Five).

Exhibit 13.1 stages the posthumous read the ceremony refuses.

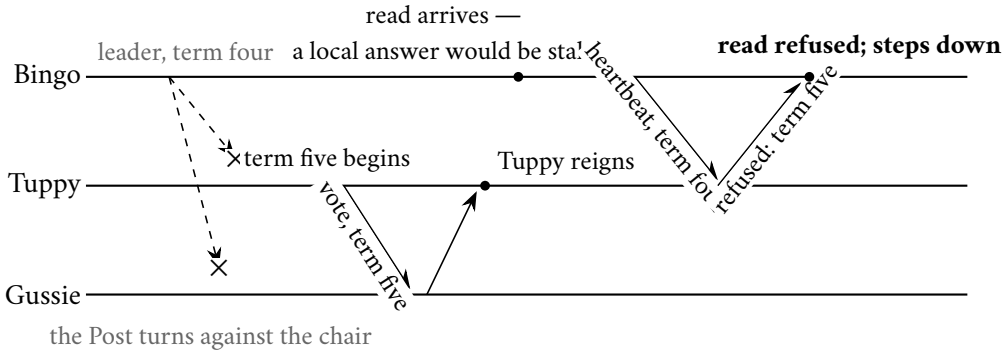


Exhibit 13.1 — The read-index round refusing a posthumous answer. Bingo, deposed but uninformed, would serve a stale read from local state; Box 31's heartbeat round collides with the crowded club — Tuppy, in both majorities, answers with the higher term — and the read is refused rather than served. Compare Chapter 5's reads-under-leadership fine print, here performed.

Quotas. The store is capped small — a couple of gigabytes by default, eight at the documented ceiling; a request cap near a megabyte and a half — Chapter 5's small-hot-metadata doctrine, enforced at the door.

13.3 Part Three — The Greatest Tenant

Kubernetes keeps its entire book of record in etcd — every object a key under a registry prefix — behind a stateless clerk, the API server. The episode's anatomy, in review form:

- **The resource version is the revision.** Every object carries an opaque version; beneath the clerk it is the store's *mod* revision. Updates are submitted citing the version read and refused if it has moved: optimistic concurrency (Chapter 8) enforced by Proposition 13.1's comparison at the point of effect.

- **The watch cache.** The clerk holds one watch per resource type against the store and fans events to thousands of client watchers from an in-memory recent history — Chubby's cache-scaling lesson (Chapter 5) rebuilt as fan-out.
- **The informer ritual.** List, note the version, watch from it; on the *too old* refusal (the compaction boundary of Definition 13.2, surfaced two rentals up), relist and resume. Controllers are *level-triggered*: events are hints to re-run a comparison of desired against observed state; a missed event is repaired by the next listing, so no message is trusted further than one relist.
- **Elections and heartbeats are leases in the book.** Controller replicas campaign by compare-and-swap on a lease object (the refused writer re-reads and waits); each node's kubelet renews a per-node lease on a cadence of seconds, and the node controller condemns the stale — failure detection as centralized bookkeeping, the unitary signature.

Remark 13.5 (the tenancy, in one sentence)*.* A strictly serializable history-keeping store at the centre; a stateless clerk fanning its ledger outward; a thousand level-triggered reconciliation loops writing back under a fencing number: the orchestrated industry is one watch loop over a rented log, and every part of it is on this volume's shelf.

13.4 Part Four — SWIM and the Parish Register

Consul's servers are a Raft parliament; its estate-wide membership is the parish register, kept by gossip. The protocol beneath (Serf, on the memberlist library) is SWIM — Das, Gupta, and Motivala, DSN 2002 — with HashiCorp's Lifeguard amendments (2017). The founding budget line: all-to-all heartbeats cost each member a chorus that grows with the group; SWIM makes detection a constant-size errand and lets dissemination ride the errand's own envelopes, so per-member load is constant in the group size.

Definition 13.6 (the parish register)**.** Each member keeps a register: a map from member identifiers to records $(inc, status)$, where *inc* is the subject's **incarnation number** — raised only by the subject itself, and only to refute suspicion — and *status* is one of $\{\mathsf{alive}, \mathsf{suspect}, \mathsf{faulty}\}$. Records about a fixed member are ordered totally:

$$\mathsf{alive}(0) < \mathsf{suspect}(0) < \mathsf{alive}(1) < \mathsf{suspect}(1) < \cdots < \mathsf{faulty},$$

with faulty absorbing (a convicted member re-enters only as a fresh identity). A member incorporates a received record by keeping the larger of the received and held records — pointwise maximum over the register.

Theorem 13.3 (the register converges)**.** *Under Definition 13.6, each per-member record space is a total order, hence a join-semilattice with join = max; the register, ordered pointwise, is a join-semilattice with join = pointwise max; local updates (publishing a suspicion, raising one's own incarnation, confirming a fault) each replace a record by a strictly larger one, hence are monotone; and incorporation is the join. By the strong eventual consistency theorem (Theorem 7.7), any two members that have received the same set of records hold identical registers, in whatever order and multiplicity the Post delivered them.*

Proof. Each hypothesis of Theorem 7.7 is verified in the statement: totally ordered record spaces are semilattices; pointwise max is the product join; the three update forms move up the chain of Definition 13.6 ($\mathsf{alive}(i) \to \mathsf{suspect}(i)$ by an accuser; $\mathsf{suspect}(i) \to \mathsf{alive}(i{+}1)$ by the subject; anything $\to \mathsf{faulty}$ by conviction); merge forgets nothing because max discards only dominated records. □

Remark 13.7. This is why refutation by incarnation defeats a suspicion *wherever the two ever meet*: rank is carried in the record, not the timing. The sabotage (Problem 13.6) abolishes the incarnation order and watches convergence die.

Theorem 13.4 (time-bounded completeness)**.** *Let each live member select probe targets by traversing a uniformly shuffled roster, reshuffling after each full traversal. If member m crashes, then every live member directly probes m within at most* $2(n-1)-1$ *protocol periods of the crash, and hence (after the probe, indirect-probe, and suspicion timeouts, a constant number of periods) every live member's register records m as* faulty*: detection is certain, on a schedule, not merely likely.*

Proof. Fix a live member p with roster of size $n-1$. In the traversal current at the crash, m occupies some position; the worst case places m just probed (position 1) at the crash and last (position $n-1$) in the next shuffle: at most $(n-1)+(n-1)-1 = 2(n-1)-1$ periods separate successive probes of m by p, and the first probe after the crash falls within that bound. A crashed member answers no ping and no proxy's ping, so p raises a suspicion after its timeouts; no refutation can arrive — only m raises m's incarnation (Definition 13.6), and m is crashed — so the suspicion ripens to faulty at the suspicion timeout. Dissemination places the conviction in every live register (gossip completeness at the standard epidemic rate; Chapter 6). □

Proposition 13.5 (expected time to first detection)**.** *In the random-selection variant (each of the* $n-1$ *live members probes a uniform random target each period, independently), the number of periods until* some *live member first probes the crashed m is geometric with success probability* $1-(1-\frac{1}{n-1})^{n-1}$*; its expectation is*

$$\left(1-\left(1-\tfrac{1}{n-1}\right)^{n-1}\right)^{-1} \longrightarrow \frac{e}{e-1} \approx 1.58 \quad (n\to\infty)\,:$$

about a period and a half, independent of the group's size — the paper's celebrated constant, by Chapter 9's friendly probability.

Proof. Each live member misses m with probability $1-\frac{1}{n-1}$, independently across members and periods; a period detects m unless all $n-1$ miss. The limit is the standard $(1-1/k)^k \to e^{-1}$ with $k=n-1$. □

Protocol — Box 32. The SWIM probe round with suspicion (Lifeguard-adjusted; checked against Das–Gupta–Motivala and Dadgar et al.)

1: **each period** $T \cdot LHM$: pick next target m from shuffled roster
2: send ping to m; await ack within timeout scaled by LHM
3: **on timeout:** send ping-req(m) to k random members; each pings m and relays any ack
4: **on no ack at all:** publish suspect(m, inc_m) — piggybacked on subsequent pings and acks, the suspect told first (the buddy rule)
5: **on hearing** suspect($self, i$): $inc \leftarrow i+1$; publish alive($self, inc$)
6: **suspicion timeout:** shrinks as independent confirmations arrive; on expiry publish confirm(m, inc_m) — faulty
7: **local health** LHM: a saturating counter — up on own missed acks, proxy refusals, and forced self-refutations; down on clean rounds — so an unwell member accuses slowly

Exhibit 13.2 walks the two verdicts.

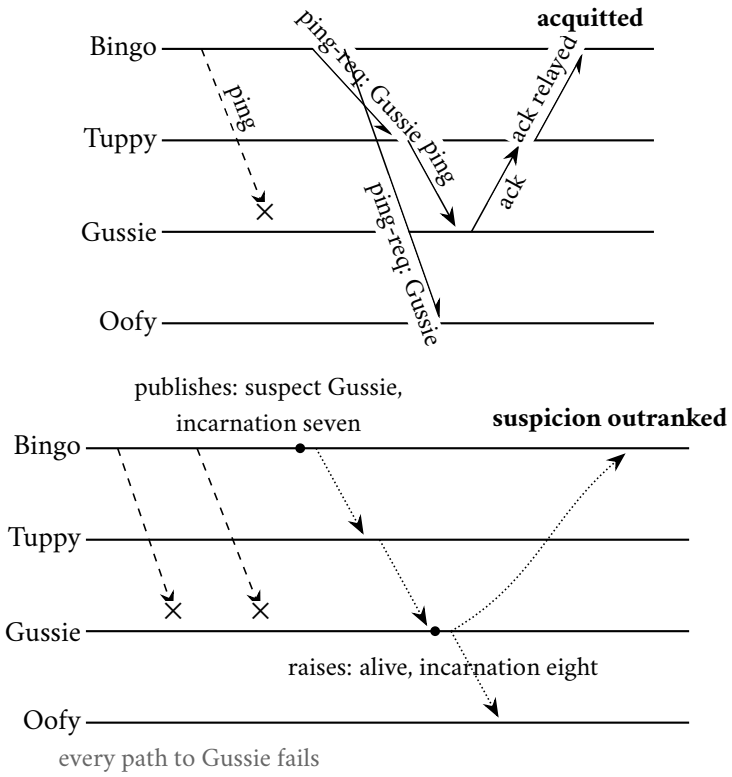


Exhibit 13.2 — The parish's two verdicts. Above: a lost link slanders nobody — the indirect probe through Tuppy acquits Gussie by a second path. Below: all paths fail; suspicion is published citing incarnation seven, travels by gossip (dotted); Gussie, hearing of his own death, raises his incarnation and the refutation outranks the suspicion wherever the two meet (Theorem 13.3). A genuinely crashed member holds no pen, and the suspicion ripens to conviction on Theorem 13.4's schedule.

13.5 Part Five — The Offices of the Registry

Services register with the agent on their own node and are health-checked there — the probe beside the thing probed; the agent syncs its local truth to the servers' catalog by anti-entropy (Chapter 6's word, the manuals' own), the agent authoritative for its node. Node liveness is the parish's verdict (the serf-health check); service fitness is the local checks' — two-tier detection, at the edge, against the unitary house's central leases.

The menu of reads. Every read names its consistency mode; Chapter 7's field guide prices the menu:

Mode (house)	Mechanism, and the contract
linearizable (etcd, default)	Box 31 at the leader: linearizable (Theorem 13.2).
serializable (etcd, option)	any member, local state: a committed prefix; no real-time clause; staleness = that member's lag.
consistent (Consul, option)	leader confirms its reign with a quorum round, then answers: linearizable — Box 31 by another name.
default (Consul)	leader answers from local state under a leader lease: linearizable while the drift assumption holds; within a leadership-change window a deposed leader may answer stale — the manual says so plainly.
stale (Consul, option)	any server: a committed prefix, staleness unbounded but measurable; reads survive a leaderless interregnum.

Note the defaults: the book of record defaults strict; the directory defaults fast-under-a-lease. Temperament, priced.

Definition 13.8 (the blocking query; the level and the reel)**.** A blocking query presents an index i (a position in the servers' log) and returns, when anything relevant changes or a timeout lapses, the *current* state σ' and its index $i' > i$ (or i at timeout). Successive states between i and i' are unobservable: the reply is the level, coalesced, not the event reel. Contrast Definition 13.2: etcd replays retained history (edge-triggered, resumable, paying compaction rent); Consul returns current state (level-triggered, nothing owed about the past, and every consumer is thereby a reconciler). The storage choice forces the watch philosophy: history on disk permits the reel; a current-edition memory store permits only the level.

Sessions and the lock-delay. A session binds a time-to-live, the node's parish verdict, and nominated local checks; invalidation of any strand releases the session's locks. On system invalidation the house declines to re-let the lock for the *lock-delay* (default fifteen seconds, configurable to zero) — the documentation names its own pedigree, a safeguard after Chubby — so a possibly-living former holder may notice and stand down. Chapter 5's ranking stands: postponement narrows the resurrection window; only the fence closes it. The fence is on sale here too: every key carries a rising modify index, and writes may be submitted check-and-set against it — Problem 13.5 has you assemble and prove it.

Federation. Each datacenter runs its own servers, its own Raft, its own catalog; the servers of all datacenters share a wide-area gossip pool, and queries for a remote datacenter are forwarded to its sovereign parliament. Federation routes; it does not replicate: partition the sea and each shore governs itself — Chapter 3's triage taken with both eyes open. The parish moonlights as a surveyor: gossip round trips nudge per-agent coordinates (the Vivaldi scheme) until coordinate distance predicts latency, and nearest-service answers come from arithmetic on the invented map.

13.6 Part Six — The Kernel's Blast Radius

The published Roblox account (January 2022) is the coordination kernel's liability clause, itemized. The timeline, compressed:

- **The estate:** one Consul cluster — 64-core servers — carrying discovery, health, KV, locks for the whole platform, *and* the platform's telemetry; a new streaming feature (a cheaper fan-out for blocking queries) lately enabled.

- **October 28, 2021:** under unusually high read and write load, cluster performance collapses; the estate follows — discovery is the switchboard. Total outage: 73 hours.
- **Cause one — contention:** the streaming path under that load contends on its concurrency plumbing; throughput falls from fighting, not from want of resources. Hardware is doubled to 128 cores; nothing improves — Chapter 11's metastable signature: a degraded state sustained from within, indifferent to capacity.
- **Cause two — the freelist:** the Raft log lives in BoltDB, which wrote its entire free-page list to disk on every commit; after the growth spike the list held on the order of a million entries, so each small append (the postmortem's appendix: sixteen kilobytes or less) paid a fixed bookkeeping write near eight megabytes. The ledger's own filing system became the load; elections resolved nothing, each new leader inheriting the same bindery.
- **The fog:** telemetry depended on the ailing cluster — the estate could not see the failure through the failure; diagnosis consumed days.
- **Recovery and remedies:** streaming disabled; slow leaders circumvented; careful restart. Afterwards: the kernel's tenancy split by concern; observability housed with a different landlord.

Remark 13.9 (the coda)*.* The freelist pathology was already repaired in bbolt — the fork of the same library in which etcd binds its ledger. The two houses keep their books in sibling binderies; the cure existed a fork upstream of the outage. Rent the kernel (Chapter 5), read the liability clause (this part), and keep the estate's eyes outside the blast circle.

13.7 Part Seven — The Comparative Anatomy

Six axes, read as a design review would:

- **Where state lives.** etcd: on disk, with history (Definition 13.1). Consul: in memory, current edition only; the log alone durable.
- **How membership is known.** etcd: its own five seats only; tenants build presence above (Kubernetes: leases). Consul: the parish register, constant load per member, Lifeguard humility included.
- **How one watches.** The reel (Definition 13.2) against the level (Definition 13.8).
- **What a read costs by default.** Strict (Theorem 13.2) against fast-under-a-lease; both houses stock all three items.
- **What guards a lock.** Both: tenancy by lease/session; the true fence from the store's rising number (Proposition 13.1; modify index); Consul adds Chubby's courtesy delay.
- **How far it federates.** One parliament, stretched at consensus prices — against many, gossip-joined, queries routed, sovereignty local.

Choose the unitary temperament for a book of record — configuration that must not fork, elections, load-bearing facts, small and hot and worth strictness by default. Choose the federal for a directory — large heterogeneous fleets, health checked where things live, datacenters plural, seconds of staleness doing no harm and priced honestly at the counter. No house sells exemption from the season: same crowded club, same fine print on reads, same corpse at the same door with the same two exorcists.

13.8 The Whiteboard

For the design review.

1. Know which constitution you rent — unitary book of record or federal registry — because the failure geometry follows: the unitary house fails as one fact; the federal degrades by province, the parish outliving the parliament. Shaped, either way; put the shape in the design.
2. Every coordination-service read is a purchase off Chapter 7's menu, and the flagship houses stock the same three items under opposite defaults. Name the mode aloud; look up the house default; price it like the contract term it is.
3. The delay is a courtesy; the fence is a guarantee; and the fencing token is the store's own rising number — revision or modify index — checked in one comparison at the point of effect. Theory and the Jepsen dossier sign the same sentence.
4. The kernel's blast radius is a design input: centralized coordination is a single point of *correlated* failure on purpose. Draw the circle of what stops when it stops; split tenancy where the circle alarms; never let the telemetry tenant with the patient.

The register. Issued: nothing — the season's register closed at twenty-four paid (Chapter 12's grand tables) and this supplementary chapter opens no account. Receipted: the standing discourtesy — seven episodes' gestures at etcd, and Consul's absence entire — paid in full by this sitting. Provenance: episode and chapter added by the supplementary session (Session 29, August 2026) at sir's motion, under a dated LEDGER drift note; episode delivered at 10,040 words, within the season's floor.

13.9 Problems

Tier I — Calisthenics

Problem 13.1 (revision arithmetic)**.** The store's revision stands at 20. Apply, in order: put *a*; put *b*; put *a*; delete *b*; put *c*; put *b*. (a) Give each key's record (*create*, *mod*, *version*) after the sequence. (b) List the events delivered to a watch on *b* from revision 21, and to a watch on *b* from revision 24. (c) A compaction runs at 24: which of the reads *as of* 21, 24, 26 survive, and what does each show for *b*? (d) Which watch resumptions now draw the *compacted* refusal?

Problem 13.2 (the menu, classified)**.** For each read below, name the strongest Chapter 7 contract it is sold under, and the clause that caps it: (a) etcd default read at the leader; (b) etcd serializable read at a lagging follower; (c) Consul consistent read; (d) Consul default read during a stable reign; (e) Consul default read in the window of a leadership change; (f) Consul stale read while no leader exists.

Problem 13.3 (the parish's arithmetic)**.** A parish of $n = 4$ runs the random-selection variant. (a) Compute the probability that a crashed member goes unprobed for a whole period, and the expected number of periods to first probe (compare Proposition 13.5's limit). (b) Under round-robin with reshuffle, give the worst-case gap in periods between successive probes of a fixed member by one prober, and check Theorem 13.4's bound.

Tier II — The Real Thing

Problem 13.4 (the register is a CRDT)**.** Write out in full the verification that Definition 13.6's register satisfies the three hypotheses of Theorem 7.7 (join-semilattice, monotone updates, merge = join), including

the product construction over member identifiers, and conclude Theorem 13.3. State precisely where the argument uses the rule that only the subject raises its own incarnation.

Problem 13.5 (a fence from the modify index)**.** Using only Consul's primitives — sessions, key acquisition, per-key modify indices, and check-and-set writes — design a lock protocol in which a resource (a store you also control) rejects every write of a deposed holder, lock-delay set to zero. State the invariant your fence maintains and prove it, in the style of Proposition 13.1. Where does your design need the resource's cooperation, and why can no arrangement avoid it (Chapter 5)?

Problem 13.6 (sabotage: the parish without incarnations)**.** A cost-conscious engineer simplifies SWIM: incarnation numbers are abolished; a register holds plain statuses $\{\mathsf{alive}, \mathsf{suspect}, \mathsf{faulty}\}$; on receipt of a status for a member, an agent *adopts it*, newest-arrival-wins, reasoning that fresh gossip is fresher truth. The Post retains its ordinary licence to delay and duplicate. Exhibit an execution in which two correct members' registers disagree about a third correct member forever — convergence dead, no fault required beyond the Post's charter — and a second in which a crashed member is resurrected on paper by a stale rumour. The fatal executions are written out in the solutions, per the standing rule.

Tier III — For the Ambitious (starred)

Problem 13.7 (Lifeguard as a governor)**.** Model a member's local health multiplier as a saturating counter driven by a Bernoulli process of its own message losses, and the probe timeout as $T \cdot LHM$. Show that under local degradation (loss probability q at the member, network healthy) the member's false-accusation rate falls at least geometrically in the saturation ceiling, while detection latency for true crashes grows only linearly in it. *(Hints only.)*

Problem 13.8 (the informer invariant)**.** Specify a watch-cache-and-informer system over Definition 13.2: one store watch, a bounded in-memory history, k clients running list-then-watch with relist on the compacted refusal. State and prove the invariant that each client's cache always equals the store's state at some revision no older than its last relist, and that under a quiet store every cache converges to the store's current state. *(Hints only.)*

13.10 Hints

Hint (Problem 13.4)**.** The product of join-semilattices, ordered pointwise, is a join-semilattice with pointwise join. Monotonicity of the refutation step is exactly the only-the-subject rule: an accuser who could raise a stranger's incarnation could publish a record *incomparable* with a concurrent honest one.

Hint (Problem 13.5)**.** Acquire the lock key under a session; read the key's modify index x at acquisition; stamp every resource write with x; the resource keeps a high-water mark and refuses lower stamps. Prove: handovers strictly raise the lock key's modify index (every acquisition is a write). The resource's cooperation is Chapter 5's law — the enforcement point must be the point of effect — and the lock holder's own diligence cannot substitute (it is the party whose sanity is in question).

Hint (Problem 13.6)**.** Duplicate one alive envelope before a crash and deliver the copies late, alternately, to two different members; with newest-arrival-wins each delivery re-adopts alive. For the second execution, the same envelope after the conviction.

Hint (Problem 13.7)**.** A false accusation requires the scaled timeout to lapse despite a healthy target: bound the probability the member's loss process outlasts $T \cdot LHM$ while the counter, fed by the same losses, has climbed. Detection latency is additive in the ceiling; false accusation is multiplicative.

Hint (Problem 13.8)**.** The cache is only ever assigned (i) a full listing at revision r, or (ii) the previous cache plus the next event, at revision r+1-and-onward in stamp order (Definition 13.2 guarantees no gaps above the listing revision). Induct on assignments; the compacted refusal resets the induction at a newer listing.

13.11 Solutions

Solution (to Problem 13.1)**.** Writes stamp revisions 21–26 in order. (a) a: $(21, 23, 2)$ — created at 21, rewritten at 23; b: $(26, 26, 1)$ — the first generation $(22, 22, 1)$ closed by the delete at 24, a fresh generation at 26; c: $(25, 25, 1)$. (b) From 21: put b at 22, delete b at 24, put b at 26. From 24: delete b at 24 is *not* delivered (stamps > 24 only): put b at 26. (c) As-of 21 is refused (compacted); as-of 24 shows no b (generation closed at 24; the delete's tombstone is the newest fact at 24); as-of 26 shows b's new generation. (d) Any watch resuming from revision < 24 draws the refusal and must relist.

Solution (to Problem 13.2)**.** (a) Linearizable (Theorem 13.2); capped by nothing on the menu — the price is the heartbeat round. (b) A committed prefix; no real-time clause; the cap is the follower's replication lag. (c) Linearizable; the cap is the quorum round's latency. (d) Linearizable *conditional on the drift bound* (Theorem 5.3's hypothesis); cap: the lease assumption itself. (e) A committed prefix only — the deposed leader's local state; the manual's admitted window. (f) A committed prefix, staleness unbounded; the cap is availability's price, and the reason the mode exists.

Solution (to Problem 13.3)**.** (a) Each of 3 live members misses with probability $2/3$; all miss with $(2/3)^3 = 8/27 \approx 0.30$; expectation $(1 - 8/27)^{-1} = 27/19 \approx 1.42$ periods — already near the limit $e/(e-1) \approx 1.58$. (b) Roster size 3; worst gap $2 \cdot 3 - 1 = 5$ periods (probed first in one traversal, last in the next), matching the theorem's $2(n-1) - 1$.

Solution (to Problem 13.4)**.** Fix a member m. The record space — the alive and suspect records at every incarnation, with faulty atop — under Definition 13.6's chain is totally ordered, and any totally ordered set is a join-semilattice with $x \sqcup y = \max(x, y)$: idempotent, commutative, associative by inspection of max. The register is the product $\prod_m S_m$ ordered pointwise; joins are pointwise maxima, and the product of join-semilattices is one. Updates: an accuser holding $\mathsf{alive}(i)$ publishes $\mathsf{suspect}(i)$, its immediate successor; a subject holding (or hearing) $\mathsf{suspect}(i)$ publishes $\mathsf{alive}(i{+}1)$, again above; conviction publishes the top. Each update maps the local register to a strictly larger one, so updates are monotone, and incorporation-by-max is the join, which discards only dominated records — merge forgets nothing. Theorem 7.7 then yields strong eventual consistency: identical delivered *sets* of records give identical registers, orders and duplicates notwithstanding. The only-the-subject rule is used exactly where updates must be *comparable*: if an accuser could mint $\mathsf{suspect}(j)$ for arbitrary j, two honest parties could publish records neither of which dominates the other's intent (a forged high suspicion against a genuine refutation at the same incarnation), and while max still converges as algebra, it converges to slander: the *semantic* guarantee — a live subject can always outrank any suspicion about it — rests on the subject's monopoly over its own counter.

Solution (to Problem 13.5)**.** Protocol: (1) create a session s; (2) acquire the lock key L under s; the acquisition is a write, so read back L's modify index x in the same exchange; (3) stamp every write to the resource with x; (4) the resource keeps, per protected object, the highest stamp admitted, and refuses lower stamps; (5) on any session doubt, stop writing (optional — safety does not depend on it). Invariant: successive lock holders carry strictly increasing stamps. Proof: each acquisition writes L, and every write to a key strictly raises its modify index (the servers' log totally orders writes and the index rises per write — the federal Proposition 13.1); a deposed holder's stamp was minted at its own acquisition, which preceded its successor's, so its stamp is strictly smaller; once the successor's first write is admitted, the high-water mark

exceeds the deposed stamp and every late write of the corpse is refused — with lock-delay zero, note, the safety never rested on the delay. The resource's cooperation is unavoidable by Chapter 5's argument: the last line of defence cannot live in the lock holder, because the holder is the party whose pauses are in question; only the point of effect observes every write, so only it can refuse the stale ones.

Solution (to Problem 13.6)**.** *Execution one — permanent disagreement between correct members about a correct member.* Parish: Bingo, Tuppy, Gussie, Oofy; the subject is Gussie, alive and correct throughout. (1) Gussie's routine alive announcement is carried in several envelopes; the Post duplicates one, holding two copies in flight, addressed to Tuppy and to Oofy. (2) A transient link failure makes Bingo's probes of Gussie fail; Bingo publishes suspect(Gussie); the suspicion reaches Tuppy and Oofy, who adopt it (newest arrival). (3) The Post now delivers the stale alive copy to Tuppy — who adopts alive — but delays the copy addressed to Oofy. Registers now read: Tuppy: alive; Oofy: suspect. (4) The Post, within its charter, repeats the performance: it holds further duplicates of old suspect and alive envelopes (both exist; both were legitimately sent) and delivers them alternately — suspect to Tuppy just after each alive, alive to Oofy just after each suspect. Under newest-arrival-wins every delivery is adopted; at every moment after step (3) some pair of correct members disagrees, and nothing in the protocol ever distinguishes a stale envelope from a fresh one — the registers flap forever and eventual agreement is dead. With incarnations, every one of those envelopes carries a rank; after Gussie's single refutation at incarnation i+1, all circulating records at $\leq i$ are dominated and the flapping is algebraically impossible (Theorem 13.3). *Execution two — resurrection on paper.* Barmy crashes; suspicion ripens; faulty circulates and the parish removes him. One stale alive(Barmy) envelope, mailed before the crash and duplicated by the Post, arrives at Tuppy after the conviction. Newest-arrival-wins re-adopts alive: the corpse walks on paper, is re-announced by Tuppy's gossip, and the conviction must be re-litigated from scratch — indefinitely often, as the Post may duplicate at pleasure. With incarnations, faulty is absorbing and the stale envelope is dominated on arrival. Both executions use only delay and duplication — the Post's ordinary licence; no forgery, no second fault. The simplification is fatal.

Solution (to Problems 13.7 and 13.8)**.** By standing policy: hints only (the Hints section above), the doors being shown rather than the corridors walked.

13.12 The Reading

Das, Gupta, and Motivala, "SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol," DSN 2002. The parish's charter. Read §3 against Box 32 mechanism by mechanism, and §4.2's suspicion rules against Definition 13.6 — the override table is the semilattice, drawn before the field had the word. The round-robin refinement behind Theorem 13.4 is §4.3.

Dadgar, Phillips, and Currey, "Lifeguard: Local Health Awareness for More Accurate Failure Detection," 2017. Chapter 11's missing appendix: the suspicion service auditing itself, with measurements — read the false-positive tables and note which of the three amendments buys each decade.

The shelf. The Roblox return-to-service account (January 2022), read against Chapter 11's metastability grammar and Part Six's timeline; the Jepsen etcd 3.4.3 analysis (2020), read against Chapter 5's fencing sermon and Proposition 13.1 — the independent audit arriving at the course's own whiteboard; and Burrows on Chubby (Chapter 5's assignment), reread last with both anatomies in mind — every organ of both houses is somewhere in it.

Chapter 14

The Apparatus

This closing chapter is the only one in the volume with no episode behind it. It is the apparatus the masthead promised: a map of the consolidated registers, an index of the protocol boxes, and the pronunciation register in print. Thirteen chapters formalize what the episodes narrate — twelve of the season and one supplementary sitting; this one merely makes the volume navigable from the back, which is where a desk volume is most often opened.

14.1 The Consolidated Registers

The season's ledgers were consolidated where the season ended, and this section is chiefly a signpost, so that no reader hunts through twelve chapters for equipment that lives in one. *The course dictionary* — the twelve load-bearing terms, each with the chapter that owns it — and *the final debt register* — all twenty-four notes, issue to payment — stand as the grand tables of Section 12.8, in Chapter 12, where the finale read them aloud. *The reading, consolidated* — the spine in six papers, the industrial texts, the corrective literature, and the methods — closes Chapter 12 as an essay, superseding no chapter shelf but gathering them. *The notation ledgers* remain deliberately chapter-local: each chapter's spoken-phrase-to-symbol table binds the vocabulary of one episode, and a merged table would only launder context out of notation. What the volume lacked until now was an index of its thirty-three protocol boxes, which the practising reader consults more often than any theorem; it follows, in two sittings.

Protocol box	Chapter
Box 1. The naive replicated counter (the inaugural bug)	1
Box 2. Stubborn point-to-point link, over fair-loss	1
Box 3. Perfect link over fair-loss	1
Box 4. The Lamport clock	2
Box 5. The vector clock	2
Box 6. The Chandy–Lamport snapshot	2
Box 7. Ben-Or's randomized consensus, crash-fault form	3
Box 8. The Synod: single-decree Paxos	4
Box 9. Raft essentials	4
Box 9b. Raft, stepped: the handlers in full	4
Box 10. Multi-Paxos essentials	5
Box 11. Lease grant and hold, under drift	5
Box 12. PBFT, normal case	5
Box 13. The Dynamo-school read and write paths	6
Box 14. ABD single-writer atomic register	6
Box 15. The session receipts	7

Protocol box	Chapter
Box 16. Grow-only and positive-negative counters	7
Box 17. The observed-remove set	7
Box 18. Two-phase commit with recovery matrix	8
Box 19. Spanner commit-wait	8
Box 20. Percolator transactions	8
Box 21. The ring, industrial dress	9
Box 22. Rendezvous placement	9
Box 23. The routing epoch	9
Box 24. The exactly-once sink materials	10
Box 25. Asynchronous barrier snapshotting	10
Box 26. The defused retry	11
Box 27. The circuit breaker, with fine print	11
Box 28. The request's lifecycle under pressure	11
Box 29. Ring all-reduce	12
Box 30. The elastic training step	12
Box 31. The etcd linearizable read (ReadIndex)	13
Box 32. The SWIM probe round with suspicion (Lifeguard-adjusted)	13

One specification stands outside the numbering by design: the TCommit specification, in Chapter 11's starred appendix, written in the TLA+ idiom after Lamport — filed as an exhibit of the method rather than as equipment of the course, per the season's numbered decision on formal specification.

14.2 The Pronunciation Register

The audio edition of this course ships with a pronunciation dictionary — two files in the ElevenReader dialect, one carrying alias respellings, one carrying phonemes — generated from a single source of truth

in the repository's lexicon directory. This section prints the register's one hundred and two entries in the alias form, for the reader who wants to know how the narrator intends a name before the narrator says it, and for the listener auditing what was said. The phonetic-alphabet forms live in the phoneme file and are omitted here; they say nothing the respelling does not, in a typeface fewer readers enjoy. Entries *flagged for the ear* carry a verification note in the source: the respelling follows the best written evidence, but the course awaits its listener's verdict, and corrections are folded back into the single source, never patched downstream. Possessive forms of every entry are generated automatically and are not listed.

Grapheme	**Spoken as (alias respelling)**
Lamport	LAM-port
Fischer	FISH-er
Paterson	PAT-ur-sun
Deutsch	DOYTCH
Gosling	GOZ-ling
Dwork	DWORK
Stockmeyer	STOCK-my-er
Toueg	too-EG *(flagged for the ear)*
Ghemawat	GEM-uh-waht
Ousterhout	OH-ster-howt
Junqueira	zhoon-KAY-ruh *(flagged for the ear)*
Preguica	preh-GEE-suh — *script spells Preguica plain (flagged for the ear)*
Baquero	buh-KAIR-oh
Zawirski	zuh-VEER-skee *(flagged for the ear)*
Vogels	VOH-gulz
Stoica	STOY-kuh
Renesse	ruh-NESS-uh — *as in van Renesse*
Attiya	ah-TEE-yuh
Dijkstra	DYKE-struh
Dolev	DOH-lev
Herlihy	HER-li-hee
Chandy	CHAHN-dee
Mattern	MAH-tern
Fidge	FIJ

Grapheme	**Spoken as (alias respelling)**
Zaharia	zuh-HAHR-ee-uh
Akidau	AK-ih-dow *(flagged for the ear)*
Kleppmann	KLEP-mahn
Kingsbury	KINGZ-bair-ee
Bailis	BAY-liss
Skeen	SKEEN
Shostak	SHOH-stak
Pease	PEEZ
Liskov	LIS-kov
Oki	OH-kee
Ongaro	on-GAR-oh
Akkoyunlu	ah-koh-YOON-loo *(flagged for the ear)*
Ekanadham	ek-uh-NAHD-um *(flagged for the ear)*
Halpern	HAL-purn
Guerraoui	gair-ah-WEE *(flagged for the ear)*
Cachin	KAH-shin *(flagged for the ear)*
Rodrigues	rod-REE-gish — *Portuguese (flagged for the ear)*
Tanenbaum	TAN-en-bowm
Demers	duh-MEERZ *(flagged for the ear)*
Saltzer	SALT-zer
Karger	KAR-gur
Hellerstein	HEL-er-styne
Alvaro	AL-vuh-roh *(flagged for the ear)*
Recht	REKT

Grapheme	Spoken as (alias respelling)
Kulkarni	kool-KAR-nee
Hayashibara	hah-yah-shee-BAH-rah
Burrows	BURR-ohz
Corbett	KOR-bit
Thaler	THAY-lur *(flagged for the ear)*
Abadi	uh-BAH-dee
Ben-Or	ben-OR
Hadzilacos	hah-dzee-LAH-kohs *(flagged for the ear)*
Mortier	MOR-tee-ay *(flagged for the ear)*
DeCandia	deh-KAN-dee-uh *(flagged for the ear)*
Bar-Noy	bar-NOY
Garcia-Molina	gar-SEE-ah moh-LEE-nah
Salem	SAY-lem
Kaashoek	KAHS-hook *(flagged for the ear)*
Balakrishnan	bah-luh-KRISH-nun
Ravishankar	rah-vee-SHAHN-ker *(flagged for the ear)*
Akamai	AH-kuh-my — *company's own pronunciation*
Kreps	KREPS
Carbone	kar-BOH-neh *(flagged for the ear)*
Gobioff	GOH-bee-off *(flagged for the ear)*
Leung	lee-UNG *(flagged for the ear)*
Brooker	BRUK-er
Newcombe	NEW-kum
Bronson	BRON-sun

Grapheme	Spoken as (alias respelling)
Daly	DAY-lee
Li	LEE — *Mu Li, parameter server*
Hashimoto	hah-shee-MOH-toh
Dadgar	DAD-gar *(flagged for the ear)*
Das	DAHSS — *Abhinandan Das, SWIM*
Gupta	GOOP-tuh
Motivala	moh-tih-VAH-luh *(flagged for the ear)*
Paxos	PAK-soss
Zab	ZAB
etcd	et-see-DEE — *never 'etsed'*
NCCL	NIK-ul — *industry says 'nickel'*
Riak	REE-ak
Ceph	SEF
Jepsen	JEP-sen
ZeRO	ZEE-roh — *the optimizer, Episode Twelve*
CRDT	C R D T — *letterism*
PBFT	P B F T — *letterism*
HLC	H L C — *letterism*
TLA+	T L A plus — *grapheme includes plus sign*
TLA	T L A
Παρλιαμεντ	par-lee-ah-MENT — *Episode Four's Greek aside; reads as faux-Greek 'parliament'*
fsync	EFF-sink
fsynced	EFF-sinkt
bbolt	BEE-bolt — *etcd's fork of BoltDB*

Grapheme	Spoken as (alias respelling)
BoltDB	bolt-DEE-BEE
Roblox	ROH-bloks — *company's own pronunciation*
Oofy	OO-fee
Gussie	GUSS-ee
Tuppy	TUP-ee
Barmy	BAR-mee

That closes the apparatus, and with it the volume: thirteen chapters of argument and one of furniture, which is, I believe, still the correct ratio.